Sistem Rekomendasi: Dari Nol hingga Skala Shopee

Panduan praktik dengan marketplace yang berjalan

Dibuat 02-11-2026

Daftar isi

Kata Pengantar

- Buku ini untuk siapa
- Bentuk buku ini
- Isi repositori
- Persiapan
- Contoh berjalan: marketplace simulasi
- Bagaimana setiap bab disusun
- Catatan tentang skala

Bab 1 — Apa yang Dilakukan Sistem Rekomendasi

- Masalah Senin pagi
- Intuisi: pramuniaga yang mengingat Anda
- Algoritma: membingkai masalahnya
- Penelusuran kode
- Jalankan
- Hasil: membaca bentuk data
- Jebakan dan praktik di produksi
- Ringkasan
- Kuis

Bab 2 — Mengukur keberhasilan

- Apa yang akan Anda pelajari
- Intuisi: menilai ujian yang jawabannya tidak Anda tulis
- Evaluasi offline versus online
- Pembagian berbasis waktu, dan mengapa pembagian acak membocorkan data
- Apa yang kita ukur: label dan daftar berperingkat
- Algoritma: metrik pemeringkatan dari dasar
- Penelusuran kode: `recsys/evaluate.py`
- Jalankan
- Hasil: berapa skor sistem rekomendasi acak?
- Jebakan, dan bagaimana dilakukan di produksi
- Ringkasan
- Kuis

Bab 3 — Baseline popularitas

- Intuisi: rak di depan toko
- Algoritma: lima cara menghitung
- Penelusuran kode
- Jalankan
- Hasil
- Jebakan dan praktik di produksi
- Ringkasan
- Kuis

Bab 4 — Content-based filtering

- Intuisi: pustakawan yang tak pernah melihat para pembaca
- Algoritma
- Penelusuran kode
- Jalankan
- Hasil
- Jebakan dan praktik di produksi
- Ringkasan
- Kuis

Bab 5 — Collaborative filtering

- Tidak ada yang menulis kata "kopi" di kartu itu
- Intuisi: bertanya pada orang yang berselera sama
- Algoritmenya
- Penelusuran kode
- Menjalankannya
- Hasil
- Jebakan dan praktik di produksi
- Ringkasan
- Kuis

Bab 6 — Faktorisasi matriks (*matrix factorisation*)

- Pembeli yang tidak mirip siapa pun
- Intuisi: dimensi selera yang tidak pernah ditulis siapa pun
- Algoritma
- Penjelasan kode
- Jalankan
- Hasil
- Jebakan dan cara produksi melakukannya
- Ringkasan
- Kuis

Bab 7 — Embedding dan retrieval two-tower

- Intuisi: dari nama menjadi koordinat
- Arsitektur two-tower
- Algoritma: pelatihan dengan softmax in-batch
- Penelusuran kode
- Pencarian tetangga terdekat hampiran
- Jalankan
- Hasil
- Jebakan dan praktik di produksi
- Ringkasan
- Kuis

Bab 8 — Pembangkitan kandidat multi-sumber

Intuisi: pustakawan dan daftar pendek
Mengapa dua tahap: corong
Empat retriever
Menggabungkan sumber: `generate`
Mengukur retriever: recall, pair recall, dan unique recall
Jalankan
Hasil
Filter keras: yang tidak boleh sampai ke ranker
Penelusuran kode: bagaimana skrip evaluasi dibangun
Jebakan dan cara produksi melakukannya
Ringkasan
Kuis

Bab 9 — Rekayasa fitur untuk pemeringkatan

Intuisi: fitur adalah catatan briefing
Algoritma: dari peristiwa mentah ke baris fitur
Penelusuran kode
Jalankan
Hasil: membaca satu baris fitur
Jebakan dan cara produksi menanganinya
Ringkasan
Kuis

Bab 10 — Learning to rank

Intuisi: mengurutkan berdasarkan skor yang dipelajari seseorang
Algoritma
Penjelasan kode
Jalankan
Hasil
Jebakan dan praktik produksi
Ringkasan
Kuis

Bab 11 — Re-ranking, diversitas, dan aturan bisnis

Halaman yang isinya susu formula semua
Intuisi: menata etalase toko, bukan memilih produk terbaik
Algoritmanya
Penelusuran kode
Jalankan
Hasil: berapa harga diversitas?
Melampaui diversitas: aturan bisnis di re-ranker
Jebakan dan praktik produksi
Ringkasan
Kuis

Bab 12 — Cold start, konteks, dan bias

Intuisi: kolega baru, menu restoran, dan eskalator
Algoritma: tangga, propensity, dan loop
Penelusuran kode
Jalankan
Hasil
Jebakan dan praktik di produksi
Ringkasan
Kuis

Bab 13 — Membangun sistemnya

Intuisi: restoran dengan buku besar
Algoritma: perjalanan satu klik
Penelusuran kode, bagian 1: menangkap perilaku di browser
Penelusuran kode, bagian 2: server mencatatnya
Penelusuran kode, bagian 3: recommender membacanya kembali
Penelusuran kode, bagian 4: sisi Rust dari jembatannya
Kontrak antarbahasa: satu `.proto`, dua generator kode
Jalankan
Hasil
Jebakan dan praktik produksi
Ringkasan
Kuis

Bab 14 — Dari prototipe ke skala Shopee

Intuisi: pasar malam dan jaringan supermarket
Algoritma: pipeline yang sama, diperluas skalanya
Panduan kode: pseudo-code untuk bagian yang belum ada
Jalankan: papan peringkat buku
Hasil: membaca papan peringkat
Jebakan dan cara produksi menanganinya
Ringkasan
Kuis

Lampiran A — Kunci jawaban kuis

Bab 1 — Apa yang Dilakukan Sistem Rekomendasi
Bab 2 — Mengukur keberhasilan
Bab 3 — Baseline popularitas
Bab 4 — Content-based filtering
Bab 5 — Collaborative filtering
Bab 6 — Faktorisasi matriks (*matrix factorisation*)
Bab 7 — Embedding dan retrieval two-tower
Bab 8 — Pembangkitan kandidat multi-sumber
Bab 9 — Rekayasa fitur untuk pemeringkatan
Bab 10 — Learning to rank
Bab 11 — Re-ranking, diversitas, dan aturan bisnis
Bab 12 — Cold start, konteks, dan bias
Bab 13 — Membangun sistemnya
Bab 14 — Dari prototipe ke skala Shopee

Lampiran B — Glosarium

Lampiran C — Mereproduksi setiap hasil

1. Perangkat
2. Skrip pendamping offline (bab 1–12)
3. Sistem langsung (bab 13)
4. Grafik
5. Tangkapan layar
6. PDF

Daftar Pustaka

Kata Pengantar

Buka aplikasi Shopee pada Selasa malam. Sebelum Anda mengetik satu huruf pun, layar beranda sudah menampilkan empat puluh produk: casing untuk ponsel yang Anda beli bulan lalu, makanan kucing karena minggu lalu Anda menjelajahi produk hewan peliharaan, dan sebuah ketel listrik *flash sale*. Tidak ada orang yang menyusun slot-slot itu dengan tangan. Sebuah *pipeline* menelusuri jutaan produk, membuang hampir semuanya dalam hitungan milidetik, menilai yang tersisa, lalu mengacak urutan para pemenangnya agar halaman tidak tampak seperti tembok berisi casing ponsel yang seragam.

Pipeline itu adalah **sistem rekomendasi** (*recommender system*). Buku ini mengajarkan cara kerjanya dengan membangun satu: mula-mula daftar item paling populer, lalu, bab demi bab, setiap gagasan yang mengubah daftar itu menjadi sistem multi-tahap modern.

Buku ini untuk siapa

Buku ini ditulis untuk **software engineer tingkat menengah**: Anda nyaman membaca Python, mengenal REST dan SQL, dan pernah merilis kode ke produksi. Latar belakang *machine learning* tidak diperlukan; setiap bagian matematika dijelaskan saat pertama kali muncul dan dipakai pada contoh kecil yang dihitung dengan tangan. Semuanya berjalan di laptop dalam hitungan menit, tanpa GPU.

Dua janji membentuk seluruh isi buku ini:

1. **Setiap algoritma dijelaskan selangkah demi selangkah.** Anda akan melihat intuisi, rumus, contoh hitungan, lalu kode sungguhan yang mengimplementasikannya.
2. **Setiap angka dalam teks berasal dari program yang bisa Anda jalankan.** Tidak ada hasil dalam buku ini yang diketik tangan. Setiap bab punya skrip pendamping; keluarannya ditempel ke buku langsung dari berkas yang ditangkap saat skrip dijalankan. Jika Anda menjalankan ulang skripnya, Anda akan mendapat angka yang sama, karena semua *random seed* dikunci.

Bentuk buku ini

Buku ini mengikuti jalur historis dan praktis bidangnya. Setiap bab menambahkan perangkat baru hanya setelah pendekatan sebelumnya menunjukkan batasnya pada data yang sama.

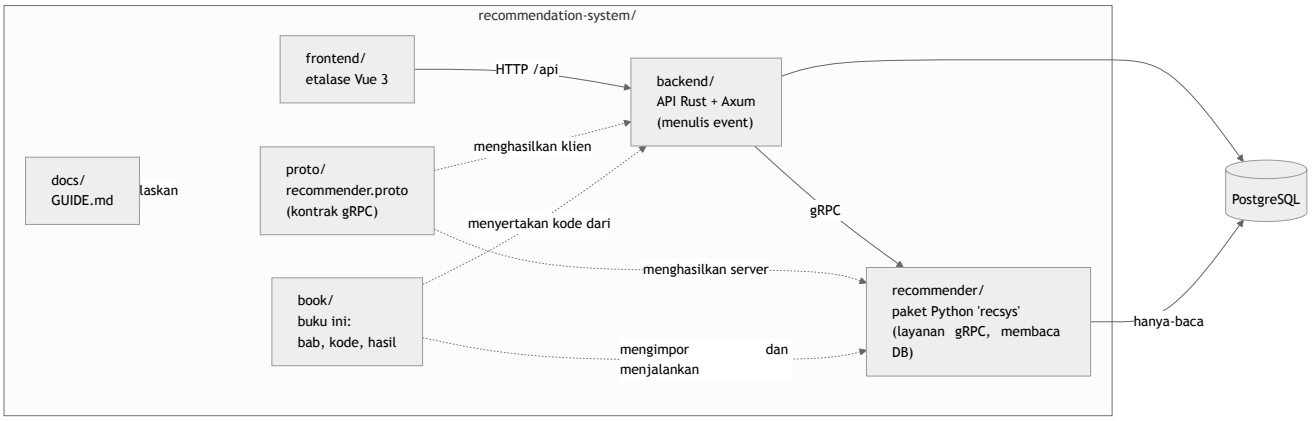
- **Fondasi (Bab 1–2).** Masalahnya, datanya, dan penggarisnya: precision, recall, NDCG, dan evaluasi berbasis waktu.
- **Recommender model tunggal (Bab 3–7).** Popularitas, *content-based filtering*, *collaborative filtering*, faktorisasi matriks, serta *embedding* dengan *retrieval* dua menara (*two-tower*).
- **Sistem multi-tahap (Bab 8–12).** Marketplace sungguhan menjalankan sebuah corong (*funnel*): tahap murah mengambil beberapa ratus **kandidat** (*candidate*), tahap presisi **memeringkat** (*rank*) kandidat itu, dan tahap akhir **memeringkat ulang** (*re-rank*) demi keberagaman dan aturan bisnis. Bab-bab ini membahas pembangkitan kandidat, rekayasa fitur, *learning to rank*, *re-ranking*, serta *cold start*, konteks, dan bias.

- **Produksi (Bab 13–14).** Sebuah aplikasi Vue, Rust, Postgres, dan Python gRPC yang berjalan, tempat Anda mengikuti satu klik dari ujung ke ujung, lalu apa yang berubah pada puluhan juta pengguna.

Lampiran berisi jawaban kuis, glosarium, perintah reproduksi, dan daftar pustaka.

Isi repositori

Buku ini berada di dalam repositori sebuah proyek lengkap yang berjalan. Repositori itu berisi buku, kode yang diajarkan buku, dan aplikasi yang memakainya.



Gambar P.1: *Gambaran repositori sekilas. Panah penuh adalah pemanggilan saat runtime; panah putus-putus adalah hubungan saat build atau dokumentasi.*

Setiap direktori punya satu tugas:

Direktori	Isinya	Kemunculannya dalam buku
`recommender/`	Paket Python `recsys`: simulator marketplace, *retriever* kandidat, pembangun fitur, ranker LightGBM, re-ranker, dan layanan gRPC	Bab 1–3 dan 8–13
`book/code/`	Satu skrip pendamping per bab algoritma, ditambah `common.py` yang dipakai bersama	Setiap bab
`book/results/`	Keluaran (`.txt`) dan metrik (`.json`) hasil tangkapan skrip-skrip itu	Setiap blok `output`
`backend/`	API Rust dan Axum: autentikasi, katalog, keranjang, *checkout*, pencatatan event, klien gRPC	Bab 13
`frontend/`	Etalase Vue 3 dengan penjelasan "Why this?" dan log event langsung	Bab 13
`proto/`	`recommender.proto`, satu berkas yang mendefinisikan kontrak antara Rust dan Python	Bab 13
`docs/`	`GUIDE.md`, panduan berbahasa sederhana tentang aplikasi yang berjalan	Bab 13
`book/`	Bab, grafik, tangkapan layar, dan perangkat build untuk PDF yang sedang Anda baca	–

Dua keputusan desain muncul berulang kali. **Rust menulis semua perilaku pengguna ke Postgres; Python hanya membacanya**, lewat koneksi hanya-baca. Dan **rekomendasi dihitung saat diminta**, sehingga klik lima detik lalu sudah bisa mengubah beranda, sementara model dilatih ulang secara berkala.

Persiapan

Anda bisa membaca buku ini tanpa menjalankan apa pun, tetapi Anda akan belajar lebih banyak jika mengubah satu angka dan melihat apa yang terjadi.

Yang Anda butuhkan

Perangkat	Kegunaan
`uv`	Lingkungan Python untuk `recommender/` dan `book/`
Node.js dan `npm`	Frontend Vue, serta pustaka `mermaid` dan `katex` yang dipakai merender diagram dan matematika di PDF
PostgreSQL	Hanya untuk aplikasi langsung di Bab 13. Berjalan di `localhost:5432`, user `postgres`, password `postgres`
Rust toolchain (`cargo`)	Hanya untuk backend
`protoc`	Hanya untuk mengompilasi kontrak gRPC bagi Rust

Bab algoritma (1–12) **hanya membutuhkan `uv`**. Postgres, Rust, dan Node baru diperlukan saat Anda ingin menjalankan aplikasi web lengkap.

Skrip pendamping dan perangkat build buku

Clone repositori, lalu dari root repositori jalankan:

```
cd book && uv sync && uv run playwright install chromium && npm i
```

`uv sync` memasang dependensi Python, termasuk paket `recsys` dari `../recommender` dalam mode *editable*, sehingga skrip mengimpor kode yang sama dengan yang ditampilkan bab. Langkah `playwright install` mengunduh Chromium *headless*, yang dipakai untuk merender PDF sekaligus mengambil tangkapan layar. `npm i` mengunduh `mermaid` dan `katex`.

Setiap skrip pendamping dijalankan dari direktori `book/`:

```
uv run python code/ch01_explore.py
```

Untuk membuat ulang semua berkas hasil secara berurutan, gunakan `uv run python tools/run_all.py`. Lampiran C mencantumkan semua perintah, termasuk yang membangun ulang grafik dan PDF.

Aplikasi web

Untuk Bab 13 Anda memerlukan ketiga layanan berjalan. Petunjuk langkah demi langkah yang otoritatif ada di `docs/GUIDE.md`, bagian "Run it". Garis besarnya:

1. Buat basis data dan terapkan skemanya: `createdb marketplace`, salin `backend/.env.example` ke `backend/.env`, lalu `cargo run -- migrate` di dalam `backend/`.
2. Isi basis data dengan marketplace simulasi: `uv run recsys seed-db --yes` di dalam `recommender/`.
3. Buka tiga terminal: `uv run recsys serve` (Python, port 50051), `cargo run` (Rust, port 8080), dan `npm run dev` (Vue, port 5173).

Lalu buka `http://localhost:5173`. Data seed menyertakan akun demo seperti `user43@demo.test` dengan password `password`; halaman login mencantumkan akun lainnya.

Contoh berjalan: marketplace simulasi

Log marketplace asli bersifat privat, dan dataset publik tidak memuat posisi, durasi tinggal (*dwell time*), dan stok, sehingga buku ini memakai **marketplace simulasi** yang disertakan dalam repositori. Karena kita sendiri yang menulis simulatornya, kita juga tahu kebenaran tersembunyi di balik perilaku itu, yang dipakai Bab 1 untuk menunjukkan bagaimana data log memelintir apa yang sebenarnya diinginkan pengguna.

Simulasi bawaan, yang dikonfigurasi di `recommender/recsys/simulate.py`, memiliki 1,500 pengguna, 2,500 item, 150 toko, dan 60 hari aktivitas, dengan *seed* tetap. Skrip Bab 1 mengonfirmasi ukuran tabelnya:

KELUARAN TERMINAL

```
=== Table sizes ===
categories          25
shops              150
items            2,500
users            1,500
sessions        19,455
searches         8,680
events         276,378
```

Selain ukurannya, simulator mereproduksi pola-pola yang membuat rekomendasi menarik:

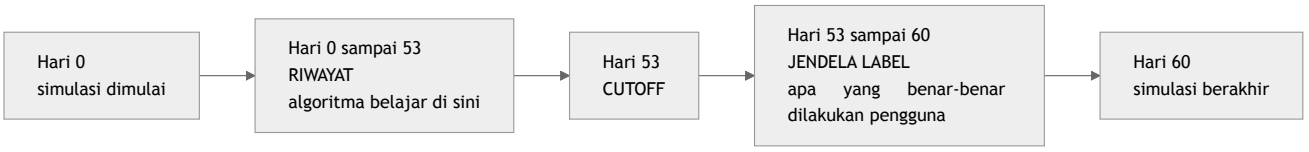
- **Popularitas timpang.** Sebagian kecil item menarik sebagian besar interaksi (*long tail*), persis seperti di toko sungguhan.
- **Selera bersifat personal.** Setiap pengguna punya preferensi tersembunyi terhadap kategori, rentang harga, dan merek.
- **Waktu berpengaruh.** Lalu lintas lebih tinggi pada akhir pekan dan sekitar tanggal gajian (tanggal 25 hingga tanggal 1 setiap bulan), dan jendela simulasi memuat **mega-sale 9.9**, hari saat lalu lintas dan pesanan melonjak. Bab 1 mengukur dan membuat grafik efek-efek ini.
- **Perilaku berbentuk corong.** Pengguna melihat *impression*, sebagian menjadi *click*, lebih sedikit menjadi *add-to-cart*, lebih sedikit lagi menjadi *order*. Beberapa item di-*hide* dan beberapa pembelian di-*return*.

- **Stok habis dan item baru muncul.** Penjual mendaftarkan produk selama jendela simulasi dan sebagian terjual habis, yang menciptakan masalah *cold start* dan ketersediaan di Bab 12.

Satu pembagian waktu untuk seluruh buku

Konvensi terpenting dalam buku ini adalah satu titik waktu tetap bernama **cutoff**. Marketplace simulasi mencakup 60 hari. Kita memilih **hari ke-53 sebagai cutoff** dan menerapkan satu aturan di mana-mana:

> Setiap algoritma boleh belajar dari, dan menghitung fitur dari, event yang terjadi **sebelum** cutoff. Algoritma itu lalu dinilai berdasarkan apa yang benar-benar dilakukan pengguna dalam **7 hari setelahnya**.



Gambar P.2: *Pembagian waktu yang dipakai setiap skrip pendamping. Tidak ada data dari jendela label yang pernah terlihat saat pelatihan atau pembangunan fitur.*

Berkas bersama `book/code/common.py` menegakkan pembagian ini dan menyediakan satu fungsi `evaluate`, sehingga angka-angka dapat dibandingkan antarbab.

Bagaimana setiap bab disusun

Bab 1 sampai 14 mengikuti templat yang sama, sehingga Anda selalu tahu posisi Anda:

1. **Cerita pembuka** dengan skenario ala Shopee, dan daftar singkat hal yang akan Anda pelajari.
2. **Intuisi**, memakai analogi sehari-hari sebelum rumus apa pun.
3. **Algoritma**: rumus, langkah bernomor, dan contoh hitungan yang cukup kecil untuk dikerjakan dengan pensil.
4. **Penelusuran kode**: kode sungguhan dari repositori, dijelaskan bagian demi bagian.
5. **Jalankan**: perintah untuk dieksekusi beserta keluaran hasil tangkapannya.
6. **Hasil**: grafik dan interpretasi yang terkait dengan angka sebenarnya.
7. **Jebakan dan praktik produksi**: apa yang bisa salah, dan bagaimana sistem besar menanganinya.
8. **Ringkasan** dan **Kuis**.

Konvensi visual dan tipografi

Kode diambil langsung dari berkas repositori, dan blok keluaran disalin dari berkas hasil yang ditangkap. Setiap angka dalam paragraf juga muncul di blok keluaran atau berkas JSON di bawah `book/results/`. Matematika sebaris tampak seperti $a \cdot b$; Bab 2 membangun NDCG dari nol. Angka ditulis dengan titik desimal, sama seperti pada keluaran program.

Empat jenis kotak callout menyela teks. Tampilannya di PDF seperti berikut:

Kotak "Tahukah kamu?" (biru) memuat fakta yang dapat diverifikasi dari literatur riset atau sejarah bidang ini, dengan sumbernya disebut di bagian bawah kotak dan tercantum dalam daftar pustaka. Setiap bab memiliki sedikitnya dua.

> **Tips**
>
> **Tip** adalah pintasan praktis atau kebiasaan yang akan menghemat waktu Anda.

> **Peringatan**
>
> **Peringatan** menandai kesalahan yang mudah dibuat dan mahal untuk ditemukan, seperti mengevaluasi pada data yang sudah pernah dilihat model.

> **Coba sendiri**
>
> **Kotak "Coba sendiri"** menyarankan eksperimen kecil: ubah satu parameter di sebuah skrip, jalankan ulang, dan bandingkan. Inilah cara terbaik mengubah bacaan menjadi pemahaman.

Terakhir, setiap bab ditutup dengan **kuis** berisi delapan sampai sepuluh soal: pilihan ganda, jawaban singkat, satu soal membaca kode, satu soal hitungan, dan satu latihan praktik yang memodifikasi skrip pendamping. Jawaban beserta penjelasan singkat ada di Lampiran A. Cobalah menjawab sebelum melihatnya; usaha mengingat kembali sebuah jawaban inilah yang membuatnya melekat.

Catatan tentang skala

Saat kami menjelaskan cara kerja platform besar, kami mengandalkan makalah yang telah dipublikasikan dan tidak mengarang angka internal. Ketika metode sederhana mengalahkan yang lebih canggih pada dataset ini, buku ini mengatakannya dan menjelaskan alasannya.

Dengan peta di tangan, bukalah halaman berikutnya. Bab 1 dimulai dari tempat setiap proyek rekomendasi dimulai: dari data, dan dari tinjauan jujur tentang apa yang bisa dan tidak bisa dikatakannya kepada kita.

Bab 1 — Apa yang Dilakukan Sistem Rekomendasi

Masalah Senin pagi

Pukul 08.55 hari Senin, Rina, seorang *product manager* di sebuah marketplace yang tidak jauh berbeda dari Shopee, menatap dasbornya. Platformnya memajang jutaan produk. Para pembelinya membuka aplikasi, menggulir layar selama sembilan puluh detik, lalu pergi. Sebagian pergi dengan puas, membawa casing ponsel di keranjang. Sebagian besar pergi dengan tangan kosong, karena barang yang sebenarnya akan mereka sukai berada di urutan ke-48,213 dalam daftar yang tidak pernah digulir siapa pun sampai sejauh itu.

Kepala engineering-nya mengucapkan kalimat yang membuka setiap proyek sistem rekomendasi: "Kita perlu menampilkan hal yang tepat untuk setiap orang." Rina mengajukan pertanyaan yang benar-benar penting: *apa yang dimaksud "hal-hal" itu, siapa "orang" itu, dan bagaimana kita tahu bahwa kita sudah benar?* Bab ini menjawab pertanyaan-pertanyaan tersebut sebelum ada kode pemodelan apa pun, dan memperkenalkan marketplace simulasi yang dipakai di setiap eksperimen dalam buku ini.

Yang akan Anda pelajari

- Cara membingkai "rekomendasi" sebagai dua pekerjaan terpisah, *pengambilan kandidat* (*retrieval*) dan *pemeringkatan* (*ranking*), serta mengapa personalisasi adalah inti dari keduanya.
- Perbedaan antara umpan balik eksplisit (*explicit feedback*) dan implisit (*implicit feedback*), dan mengapa e-commerce hampir selalu hidup dari jenis yang implisit.
- Apa itu *kebenaran tersembunyi (hidden truth) milik simulator*, mengapa data yang Anda catat hanyalah bayangan darinya, dan mengapa platform sungguhan tidak pernah melihat kebenaran itu.
- Tujuh tabel mentah (*raw table*) marketplace dalam buku ini dan enam tipe event yang mengisi tabel `events`.
- Cara mengukur *sparsity* (kejarangan data), *long tail* (ekor panjang), aktivitas per pengguna, dan efek waktu (tanggal gajian, mega-sale 8.8 dan 9.9) dengan angka nyata dari skrip pendamping `ch01_explore.py`.

Intuisi: pramuniaga yang mengingat Anda

Bayangkan dua pramuniaga di sebuah pasar. Yang pertama berdiri di pintu masuk dan berteriak, "Barang terlaris hari ini ada di sini!" setiap kali ada orang masuk. Yang kedua memperhatikan siapa yang datang. Saat Anda tiba, ia ingat bahwa minggu lalu Anda membeli makanan kucing, bahwa Anda lama memandangi sebuah mug keramik, dan bahwa Anda dua kali melewati meja sepatu begitu saja. Ia berkata: "Merek makanan kucing yang baru sudah datang, dan mug sedang diskon." Tak satu pun dari keduanya berbohong. Tetapi yang kedua sedang melakukan *personalisasi*: keluarannya bergantung pada siapa Anda dan apa yang sudah Anda lakukan.

Sistem rekomendasi adalah pramuniaga kedua itu, diotomatisasi dan dikalikan beberapa juta pelanggan. Ia punya tiga bahan dasar, dan ada baiknya kita menamainya sekarang karena seluruh buku ini adalah variasi dari ketiganya:

1. **Pengguna (*user*)**: orang (atau akun) yang bertindak.
2. **Item**: hal-hal yang bisa ditampilkan, diklik, dibeli.
3. **Interaksi (*interaction*)**: catatan tentang siapa melakukan apa terhadap item mana, dan kapan.

Otak si pramuniaga harus memecahkan masalah yang terdengar mudah tetapi brutal dalam praktik: diberikan seorang pengguna dan himpunan item yang sangat besar, hasilkan daftar pendek berurutan yang kemungkinan besar bernilai bagi pengguna itu. "Pendek" penting karena layar itu kecil. "Berurutan" penting karena orang tidak menggulir jauh-jauh. "Kemungkinan bernilai" penting karena kita harus mengubah gagasan manusiawi yang kabur menjadi sesuatu yang bisa dioptimalkan komputer.

Ia juga tidak membandingkan Anda dengan *setiap produk di pasar*. Ia lebih dulu menyempitkannya menjadi tiga puluh hal yang masuk akal, baru kemudian menentukan urutannya: struktur dua tahap yang dipakai setiap sistem rekomendasi berskala besar.

Algoritma: membingkai masalahnya

Pengambilan kandidat dan pemeringkatan

Secara formal, misalkan U adalah himpunan pengguna dan I himpunan item. Kita menginginkan sebuah fungsi skor

$$s(u, i) \in \mathbb{R}$$

sedemikian rupa sehingga, untuk setiap pengguna u, mengurutkan item berdasarkan $s(u, \cdot)$ menempatkan item yang paling bernilai bagi u di urutan pertama. Jika $|I|$ adalah 2,500, seperti pada marketplace dalam buku ini, kita bisa saja menghitung skor semua item untuk setiap permintaan. Jika $|I|$ adalah 100 juta, seperti di marketplace besar, menskor semuanya dengan model mahal dalam anggaran latensi (*latency budget*) beberapa ratus milidetik adalah hal yang mustahil.

Maka sistem produksi membagi pekerjaannya:

- **Pengambilan kandidat** (*retrieval*, disebut juga *candidate generation*) adalah tahap murah dengan *recall* tinggi. Tahap ini mereduksi jutaan item menjadi beberapa ratus *kandidat* (*candidates*). Urutannya boleh salah dan bahkan boleh menyertakan sampah, tetapi tahap ini tidak boleh *melewatkan* item yang akan disukai pengguna, karena tidak ada tahap sesudahnya yang bisa mengembalikannya.
- **Pemeringkatan** (*ranking*) adalah tahap mahal dengan presisi (*precision*) tinggi. Tahap ini mengambil beberapa ratus kandidat tersebut dan menskor masing-masing dengan model kaya yang memakai banyak fitur tentang pengguna, item, dan hubungan keduanya. Tahap inilah yang menentukan urutan akhir.
- **Pemeringkatan ulang** (*re-ranking*, bab 11) menerapkan aturan bisnis dan keberagaman di atasnya: jangan ada lima casing ponsel identik berderet.



Gambar 1.1: *corong (funnel) yang dipakai bersama oleh setiap sistem rekomendasi besar. Bab 3 sampai 7 membangun retriever, bab 9 dan 10 membangun ranker, bab 11 membangun re-ranker.*

Corong ini adalah kerangka buku ini. Bab 3 sampai 7 membahas cara menghasilkan *sebuah* skor dengan metode yang makin cerdas, dan kita akan mengevaluasi masing-masing seolah-olah ia sendirian harus mengerjakan seluruh tugas. Mulai bab 8 kita menggabungkannya seperti yang dilakukan sistem sungguhan.

Personalisasi

Rekomender **non-personal** mengembalikan daftar yang sama untuk semua orang ("terpopuler minggu ini"); rekomender **personal** mengondisikan hasilnya pada pengguna. *Baseline* popularitas lebih kompetitif daripada yang diduga pemula (bab 3), sehingga personalisasi harus menang di tempat yang penting: di tempat pilihan populer itu keliru. "Tepat" bisa berarti jawaban pengguna (item yang akan saya nikmati), jawaban bisnis (pesanan, pendapatan), atau jawaban ekosistem (sebaran penjual yang sehat); di sini perilaku adalah sinyalnya, yang menggabungkan dua yang pertama.

Umpan balik eksplisit versus implisit

Bagaimana sistem belajar apa yang disukai pengguna? Ada dua keluarga sinyal.

Umpan balik eksplisit adalah pernyataan yang disengaja: rating bintang 1 sampai 5, jempol ke atas, ketukan "tidak tertarik". Sinyal ini bersih, karena pengguna memang bermaksud demikian. Namun ia juga langka. Kebanyakan orang tidak pernah memberi rating apa pun, dan mereka yang melakukannya tidak mewakili mereka yang tidak.

Umpan balik implisit adalah hasil sampingan dari perilaku biasa: sebuah item ditampilkan, diklik, ditatap selama 40 detik, dimasukkan ke keranjang, dibeli. Sinyal ini melimpah, karena setiap sesi menghasilkannya. Ia juga bising (*noisy*) dan ambigu. Sebuah klik bisa berarti rasa ingin tahu atau salah pencet. Ketiadaan klik tidak berarti tidak suka; pengguna mungkin tidak pernah melihat item itu. Pembelian adalah "ya" yang kuat, tetapi juga condong ke item yang murah, praktis, dan gencar dipromosikan.

Di marketplace, umpan balik implisit mendominasi, dan simulator mencatat persis jenis event yang dicatat aplikasi sungguhan. Kita memberi masing-masing kekuatan bertingkat, yang didefinisikan di `schema.py`:

$$w(\text{click}) = 1, \qquad w(\text{add_to_cart}) = 3, \qquad w(\text{order}) = 5.$$

Pengguna yang mengklik sebuah item dua kali, memasukkannya ke keranjang satu kali, dan kemudian membelinya memiliki kekuatan interaksi $2 \cdot 1 + 1 \cdot 3 + 1 \cdot 5 = 10$ untuk item itu. Contoh sederhana, tetapi inilah persis cara matriks interaksi pada bab 5 dan 6 dibangun dari event mentah.

Dua tipe event bersifat *negatif*: `hide` ("tidak tertarik") dan `return`. Keduanya jarang, tetapi satu `hide` bercerita lebih banyak tentang ketidaksukaan daripada seribu impresi yang diam.

Kebenaran tersembunyi versus data yang tercatat

Berikut poin halus yang membedakan seorang *data scientist* dari orang yang sekadar menjalankan scikit-learn. Di balik setiap log sungguhan ada **kebenaran** yang tidak pernah Anda amati: apa yang *sebenarnya* disukai pengguna, seberapa bagus setiap item *sebenarnya*, seberapa sensitif setiap pembeli *sebenarnya* terhadap harga. Log adalah bayangan yang kehilangan banyak informasi dari kebenaran itu, tersaring melalui apa yang Anda pilih untuk ditampilkan.

Karena marketplace kita adalah simulasi, kita bisa melakukan sesuatu yang tidak bisa dilakukan platform sungguhan. Simulator menyimpan kebenaran itu secara eksplisit: setiap pengguna memiliki vektor selera laten (*latent taste vector*), preferensi kategori yang jarang (*sparse*), dan sensitivitas harga; setiap item memiliki kualitas laten dan vektor selera. Kemudian simulator membangkitkan sesi, menampilkan dua belas item per sesi, dan membiarkan sebuah *utilitas* (*utility*) menentukan apakah tiap item yang ditampilkan diklik, dimasukkan ke keranjang, atau dibeli. Dalam kata-kata, dengan σ sebagai sigmoid logistik dan pos sebagai slot tempat item ditampilkan (0 adalah paling atas), probabilitas klik adalah

$$P(\text{click}) = \sigma\big(\text{utility}(u, i) - 4.2 - 0.12 \cdot \mathrm{pos}\big).$$

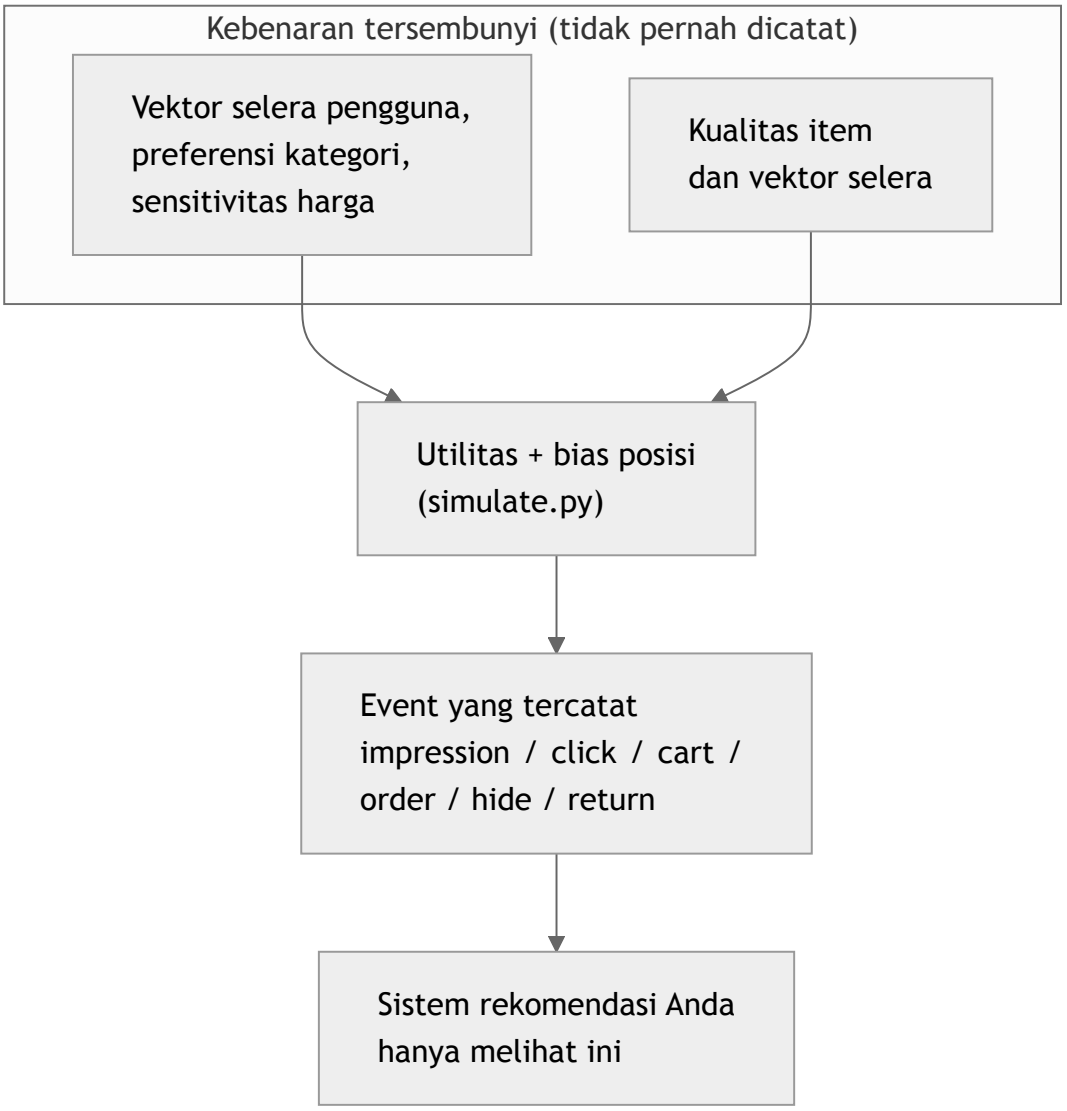
Utilitas adalah jumlah dari suku-suku yang mudah dibaca: kecocokan selera, preferensi kategori, bonus "niat" (*intent*) ketika item berada di kategori yang dituju pengguna, kualitas item, penalti kecocokan harga yang diskalakan oleh sensitivitas harga pengguna, diskon efektif, gratis ongkir, pengiriman satu kota, bonus untuk membeli ulang barang habis pakai, dan penalti berat untuk barang tahan lama yang sudah dimiliki pengguna.

Contoh hitung. Misalkan utilitasnya 3.0. Maka:

Slot (`pos`)	Argumen σ	$P(\text{click})$
0 (paling atas)	$3.0 - 4.2 - 0 = -1.2$	$\sigma(-1.2) \approx 0.231$
5	$3.0 - 4.2 - 0.6 = -1.8$	$\sigma(-1.8) \approx 0.142$

Item yang sama, pengguna yang sama, tetapi memindahkannya dari slot teratas ke slot keenam membuatnya kehilangan sekitar 39% probabilitas klik. Inilah **bias posisi** (*position bias*), dan karena simulator mencatat `position` pada setiap event, kita akan bisa mempelajari dan mengoreksinya di bab 12. Utilitas 5.0 di slot teratas memberikan $\sigma(0.8) \approx 0.690$; utilitas 1.0 memberikan $\sigma(-3.2) \approx 0.039$. Utilitas sebagian besar menentukan *apakah* Anda mengklik, sedangkan posisi menaikkan atau menurunkan probabilitasnya sedikit.

Aturan desain yang krusial tertulis di *docstring* `schema.py`: *variabel laten tidak pernah ditulis ke disk*. Hanya apa yang akan dicatat oleh sebuah aplikasi yang keluar dari simulator. Setiap algoritma dalam buku ini harus menemukan kembali kebenaran dari bayangan itu, persis seperti di produksi. Dan karena kita tahu kebenarannya, kita juga tahu bahwa data ini *dapat dipelajari*: ketika sebuah metode gagal, kita boleh curiga pada metodenya, bukan pada dataset yang murni derau.



Gambar 1.2: *simulator merahasiakan kebenaran. Sistem rekomendasi hanya melihat event yang tercatat dan tabel katalog.*

Tahukah kamu?

Collaborative filtering bermula dari rating *eksplisit* dengan sedikit sentuhan implisit. Pada sistem GroupLens untuk berita Usenet (1994), rating berupa angka 1 sampai 5, yang opsional disertai jumlah detik yang dihabiskan pembaca pada artikel, sinyal waktu yang dicatat para penulis tetapi belum dipakai oleh server rating mereka. Tiga puluh tahun kemudian, waktu di halaman (`dwell_sec` kita) menjadi salah satu sinyal implisit standar.

Sumber: Resnick et al., ACM CSCW, 1994

Waktu: kalender adalah fitur dunia

Lalu lintas marketplace tidak stasioner. Orang berbelanja lebih banyak pada akhir pekan, lebih banyak tepat setelah gajian, dan jauh lebih banyak lagi pada hari mega-sale. Marketplace di Asia Tenggara menjalankan kampanye "tanggal kembar" (9.9, 10.10, 11.11, dan seterusnya), dan sistem rekomendasi sebuah platform harus menghadapi kenyataan bahwa pada hari-hari itu perilaku pengguna berubah: lebih banyak diskon, lebih banyak klik, konversi lebih tinggi.

Simulator mereproduksi hal ini dengan fungsi kecil yang mudah dibaca, `_day_weights`. Setiap hari dimulai dengan bobot 1. Kemudian bobot dikalikan 1.2 pada hari Sabtu dan Minggu, 1.4 pada hari gajian (tanggal 25 hingga tanggal 1 bulan berikutnya), dan 3.0 pada hari sale "tanggal kembar", yang dideteksi sebagai *bulan sama dengan tanggal*. Akhirnya bobot dinormalisasi sehingga berjumlah satu agar menjadi distribusi probabilitas atas hari. Pada jendela 60 hari yang dimulai 1 Agustus 2026 ini, artinya sale 8.8 (8 Agustus) dan sale 9.9 (9 September) sama-sama terjadi. *Docstring* modul hanya menyebut 9.9, tetapi aturan `d.month == d.day` berlaku untuk setiap tanggal kembar, itulah sebabnya 8.8 juga ikut terdongkrak.

Contoh hitung. Hari sale yang jatuh pada hari Sabtu (seperti 8 Agustus) memiliki bobot mentah $1.2 \times 3.0 = 3.6$, dibandingkan 1.0 untuk hari Rabu biasa, sebelum normalisasi.

Tabel-tabel mentah

Simulator membangkitkan tujuh tabel mentah yang didefinisikan di `recommender/recsys/schema.py`. Berkas ini sengaja dijadikan "satu-satunya sumber kebenaran": simulator menulisnya, *loader* Postgres membacanya, dan pembangun fitur memeriksa terhadapnya. Tabel-tabel itu terbagi dalam tiga kelompok:

Kelompok	Tabel	Satu baris adalah...
Katalog	`categories`, `shops`, `items`	satu kategori daun, satu penjual, satu listing produk
Orang	`users`	satu akun pembeli
Perilaku	`sessions`, `searches`, `events`	satu kunjungan aplikasi, satu kueri yang dikirim, satu aksi pengguna pada satu item

`events` adalah log interaksi utama, dan tabel inilah yang menjadi sandaran sisa buku ini. Kolomnya adalah `event_id`, `ts`, `user_id`, `session_id`, `item_id`, `event_type`, `position`, `dwell_sec`, dan `price_paid`. Empat di antaranya perlu dikomentari.

- `event_type` bernilai salah satu dari enam: `impression`, `click`, `add_to_cart`, `order`, `hide`, `return`.
- `position` mencatat slot, sehingga kita bisa mempelajari bias posisi.
- `dwell_sec` hanya diisi pada klik: detik yang dihabiskan di halaman produk.
- `price_paid` hanya diisi pada order, setelah diskon.

Impresi (*impression*) adalah event yang paling tidak glamor dan paling penting. Ia menyatakan "item ini ditampilkan kepada pengguna ini di slot ini". Tanpa impresi Anda tidak bisa menghitung *click-through rate*, karena Anda tidak tahu penyebutnya, dan Anda tidak bisa membedakan "diabaikan" dari "tidak pernah terlihat".

Sparsity dan long tail

Dua fakta statistik membentuk setiap metode di bab-bab berikutnya.

Sparsity (kejarangan data). Susun pengguna sebagai baris dan item sebagai kolom. Setiap sel menyimpan kekuatan interaksi antara satu pengguna dan satu item. Pada katalog nyata hampir setiap sel kosong. Kita mengukurnya sebagai *densitas* (*density*), yaitu pecahan sel yang tidak kosong:

$$\text{density} = \frac{\text{non-zero cells}}{|U_{\text{active}}| \cdot |I_{\text{active}}|}, \qquad \text{sparsity} = 1 - \text{density}.$$

Ilustrasi sederhana: toko dengan 1,000 pengguna dan 10,000 item, yang di antara para penggunanya menghasilkan 50,000 interaksi pengguna-item yang berbeda, memiliki densitas $50{,}000 / 10{,}000{,}000 = 0.5\%$. Sebagian besar algoritma di bab-bab berikutnya harus menyimpulkan selera pengguna dari segelintir sel terisi dalam baris yang hampir kosong. Itulah sebabnya **collaborative filtering** (bab 5) membutuhkan tetangga (*neighbours*) dan **matrix factorisation** (bab 6) memampatkan matriks menjadi vektor berdimensi rendah.

Long tail (ekor panjang). Interaksi tidak tersebar merata di seluruh item. Segelintir item sangat populer; sebagian besar jarang disentuh. Urutkan item berdasarkan jumlah interaksi dan kurva yang dihasilkan turun tajam lalu menyusuri dasar untuk waktu yang lama: sebuah "kepala" (*head*) dan sebuah "ekor" (*tail*). Ringkasan yang berguna adalah pangsa dari semua interaksi yang diterima oleh $p\%$ item teratas. Semakin terkonsentrasi pangsanya, semakin mudah *baseline* popularitas menang secara default, dan semakin sulit membuat sistem rekomendasi yang tampak jelas mengunggulinya.

Ekor adalah tempat personalisasi membuahkan hasil: di sanalah produk yang cocok dengan selera tak biasa berada, juga penjual yang tak pernah mendapat kesempatan.

> **Tahukah kamu?**
>
> Gagasan bahwa "ekor" dari distribusi popularitas bermakna secara ekonomi dipopulerkan oleh artikel Chris Anderson, "The Long Tail", di *Wired* pada Oktober 2004. Ia berargumen bahwa ritel daring, yang bebas dari batas ruang rak, bisa meraup untung dengan melayani produk niche yang secara agregat menyaingi produk hit. Apakah ekor benar-benar tempat uang berada masih diperdebatkan, tetapi kosakata kepala dan ekor kini menjadi standar dalam dunia rekomender.
>
> Sumber: Anderson, Wired, 2004

Penelusuran kode

Kita akan membaca tiga potong kode nyata: pembantu skema yang mencetak kontrak data, konfigurasi simulator, dan fungsi bobot hari yang mengimplementasikan kalender.

Skema sebagai kontrak data

`schema.py` memodelkan tabel dengan dua *dataclass* beku (*frozen*) yang sangat kecil. Sebuah `Column` adalah nama, label tipe, dan deskripsi yang mudah dibaca manusia. Sebuah `Table` adalah nama, *grain*, dan tuple kolom berurutan.

recommender/recsys/schema.py · Table

```python
@dataclass(frozen=True)
class Table:
    name: str
    grain: str  # what one row represents
    columns: tuple[Column, ...]

    @property
    def column_names(self) -> list[str]:
        return [c.name for c in self.columns]
```

Kata **grain** (granularitas baris) itu penting: untuk `events`, satu baris adalah satu aksi pengguna pada satu item, dan setiap agregat berikutnya adalah `GROUP BY` di atasnya; grain yang salah akan diam-diam menghitung ganda.

Fungsi `describe` mengubah kontrak menjadi laporan yang bisa dicetak, yaitu yang ditampilkan oleh `uv run recsys schema`:

recommender/recsys/schema.py · describe

```python
def describe() -> str:
    """Human-readable dump of the schema (used by `recsys schema`)."""
    out: list[str] = ["RAW TABLES", "==========="]
    for t in RAW_TABLES:
        out.append(f"\n{t.name}  (one row = {t.grain})")
        for c in t.columns:
            out.append(f"  {c.name:<18} {c.dtype:<14} {c.description}")
    out += ["", "FEATURES (one row = one (user, item) candidate at cutoff time)", "=" * 62]
    group = None
    for f in FEATURES:
        if f.group != group:
            group = f.group
            out.append(f"\n[{group}]")
        out.append(f"  {f.name:<26} {f.description}")
    return "\n".join(out)
```

`describe` mencetak setiap tabel mentah beserta grain-nya, lalu fitur-fitur yang dikelompokkan menurut grup fitur, sehingga dokumentasi berubah setiap kali skema berubah.

Mengonfigurasi dunia

Segala hal tentang dunia simulasi dikendalikan oleh satu *dataclass* beku:

recommender/recsys/simulate.py · SimConfig

```python
@dataclass(frozen=True)
class SimConfig:
    n_users: int = 1500
    n_items: int = 2500
    n_shops: int = 150
    n_days: int = 60
    start_date: str = "2026-08-01"  # 60 days -> includes 9.9 sale and two paydays
    seed: int = 7
```

Enam angka mendefinisikan eksperimen: 1,500 pengguna, 2,500 item, 150 toko, 60 hari mulai 1 Agustus 2026, dan *seed* 7. *Seed* adalah yang terpenting: dengan *seed* yang sama, simulator membangkitkan dunia yang sama, byte demi byte. Begitulah setiap hasil dalam buku ini bersifat *dapat direproduksi*: jika Anda menjalankan ulang sebuah skrip, Anda mendapatkan angka yang sama dengan yang saya kutip.

Kalender

Fungsi di bawah ini adalah seluruh model waktu:

recommender/recsys/simulate.py · _day_weights

```python
def _day_weights(start: pd.Timestamp, n_days: int) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Traffic multiplier per day + flags for payday and mega-sale."""
    days = pd.date_range(start, periods=n_days, freq="D")
    payday = np.array([d.day >= 25 or d.day == 1 for d in days])
    sale = np.array([d.month == d.day for d in days])  # 9.9, 10.10, ... style sales
    w = np.ones(n_days)
    w[days.day_of_week >= 5] *= 1.2
    w[payday] *= 1.4
    w[sale] *= 3.0
    return w / w.sum(), payday, sale
```

`payday` bernilai benar untuk tanggal dalam bulan minimal 25 atau tepat 1; `sale` bernilai benar ketika `d.month == d.day`. Bobot dikalikan melalui *mask* boolean, lalu dinormalisasi. *Flag* dikembalikan karena *loop* sesi memakainya lagi: pada hari sale, diskon efektif naik 0.2 dan probabilitas order mendapat bonus, jadi waktu mengubah perilaku, bukan hanya lalu lintas.

Kita tidak menempelkan badan panjang `simulate()`: fungsi itu membangun katalog dan pengguna, lalu untuk setiap sesi memilih hari, *surface*, dan kategori niat, menampilkan 12 item, menghitung utilitas, dan mengambil sampel event. Cari `utility =` untuk membacanya.

Jalankan

Skrip pendamping `book/code/ch01_explore.py` memuat tabel simulasi melalui pembantu bersama `common.load_tables()` (seed 7, di-*cache*) dan mencetak enam ringkasan. Jalankan dari direktori `book/`:

```
cd book
uv run python code/ch01_explore.py
```

Keluaran yang tertangkap ada di bawah. Setiap angka yang saya kutip dalam bab ini berasal dari sini atau dari `ch01_explore.json` yang bersesuaian.

KELUARAN TERMINAL

```
=== Table sizes ===
categories          25
shops              150
items            2,500
users            1,500
sessions        19,455
searches         8,680
events         276,378

=== Event funnel (all 60 days) ===
event_type         count    % of impressions    % of previous step
impression       233,460             100.00%
click             24,726              10.59%                10.59%
add_to_cart       10,619               4.55%                42.95%
order              5,858               2.51%                55.17%
hide               1,594               0.68%
return               121               0.05%

=== Events per day ===
median clicks/day on ordinary days : 347
mean clicks/day on payday days     : 488
sale day 2026-08-08 (month == day): 1,386 clicks, 405 orders
sale day 2026-09-09 (month == day): 1,262 clicks, 410 orders
weekend vs weekday mean clicks/day  : 496 vs 376

=== Popularity concentration (clicks + carts + orders per item) ===
top  1% of items (  25) get  12.5% of interactions
top 10% of items ( 250) get  50.3% of interactions
top 20% of items ( 500) get  68.6% of interactions
items with zero interactions: 317 of 2500

=== Sparsity of the user x item matrix (before cutoff) ===
users with interactions : 1,408
items with interactions : 2,120
non-zero cells          : 18,256
density                 : 0.612%  (sparsity 99.388%)

=== Clicks per user (before cutoff) ===
p10 :      1
p25 :      5
p50 :     11
p75 :     20
p90 :     31
p99 :     56
mean: 14.2   max: 106   users with 0 clicks: 92
```

Hasil: membaca bentuk data

Ukuran dan corong

Dunia ini memiliki 25 kategori, 150 toko, 2,500 item, 1,500 pengguna, 19,455 sesi, 8,680 pencarian, dan 276,378 event: kecil menurut standar industri, tetapi cukup besar untuk efek yang ingin kita pelajari.

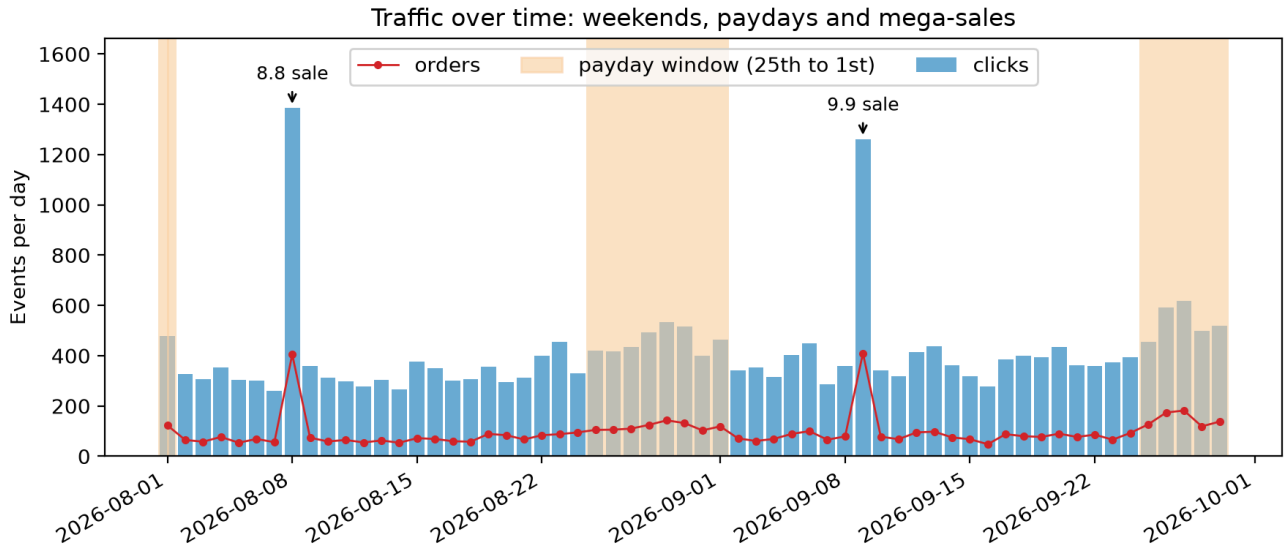
Berikutnya corong event. Dari 276,378 event, 233,460 adalah impresi; 24,726 adalah klik, yaitu 10.59% dari impresi: **click-through rate** (CTR). Dari klik-klik itu, 42.95% diikuti add-to-cart (10,619 event). Dari penambahan keranjang itu, 55.17% diikuti order (5,858 event). Mengalikan ketiga rasio ini menghasilkan 2.51% impresi yang berakhir menjadi order. Sementara itu, 1,594 event `hide` (0.68% dari impresi) dan 121 event `return` (0.05%) adalah negatif yang jarang.

Corong ini adalah alasan pertama mengapa "memprediksi klik" bukanlah pekerjaan yang sama dengan "memprediksi pembelian". Jika kita hanya mengoptimalkan klik, kita akan menguntungkan item murah yang menarik perhatian; karena itu ranker di bab 10 dilatih dengan label **bertingkat** (*graded*) (click 1, cart 2, order 3), sehingga sebuah order bernilai lebih daripada sebuah klik.

Perhatikan juga betapa sedikitnya event positif untuk sistem yang menampilkan lebih dari dua ratus ribu impresi. Melatih ranker adalah masalah kejadian langka (*rare-event*). Kita memperhatikannya di sepanjang buku: ketika kita menyebut "positif" di bab-bab berikutnya, yang dimaksud adalah puluhan ribu baris yang sedikit ini.

Waktu

Blok "Events per day" menunjukkan kalender bekerja. Pada hari biasa, mediannya 347 klik. Pada hari gajian, rata-ratanya naik menjadi 488 klik. Pada akhir pekan, rata-ratanya 496 klik per hari dibandingkan 376 pada hari kerja. Dua sale tanggal kembar tampak mencolok: 1,386 klik dan 405 order pada 2026-08-08, serta 1,262 klik dan 410 order pada 2026-09-09.

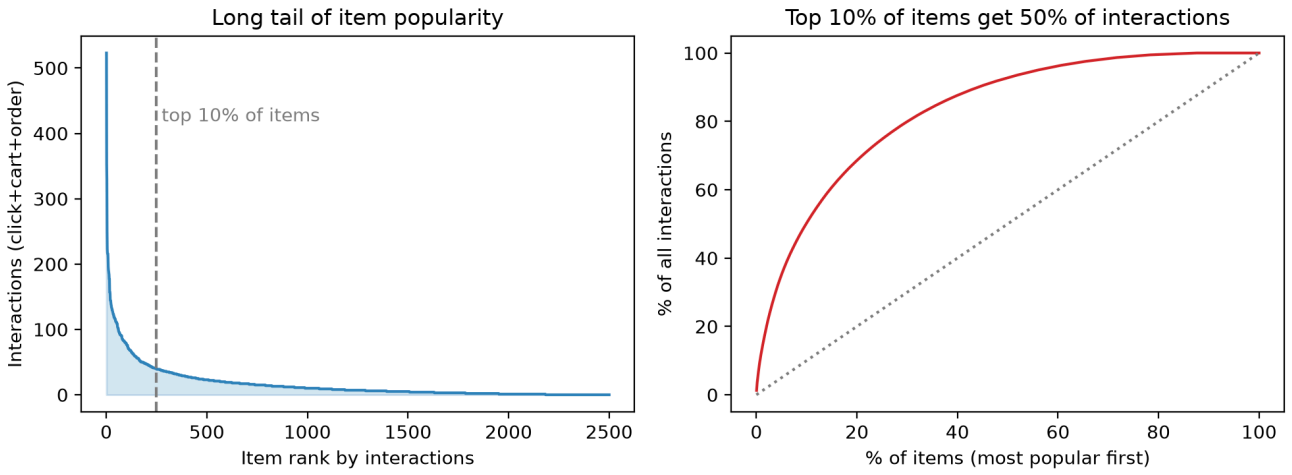


Gambar 1.3: *Klik harian (batang) dan order (garis) selama 60 hari; jendela gajian diberi bayangan dan sale 8.8 serta 9.9 diberi anotasi.*

Jendela gajian mengangkat batang-batang secara moderat; hari sale adalah lonjakan hampir empat kali lipat tingkat biasa untuk klik (1,386 dibandingkan 347).

Konsentrasi: long tail dalam angka

Blok popularitas menghitung setiap klik, add-to-cart, dan order per item. 1% item teratas (25 item) menerima 12.5% dari seluruh interaksi. 10% teratas (250 item) menerima 50.3%, dan 20% teratas (500 item) menerima 68.6%. Di sisi sebaliknya, 317 dari 2,500 item memiliki interaksi *nol*, lebih dari satu dari delapan item di katalog.



Gambar 1.4: *Kiri: interaksi item diurutkan dari terpopuler hingga paling sepi, memperlihatkan kepala yang curam dan ekor panjang yang datar. Kanan: pangsa kumulatif interaksi yang ditangkap oleh fraksi item teratas; diagonal bertitik adalah katalog yang sepenuhnya merata.*

Panel kiri adalah kurva kepala-dan-ekor yang klasik. Panel kanan, pangsa kumulatif, menceritakan hal yang sama: semakin jauh kurva merah melengkung menjauhi diagonal bertitik, semakin terkonsentrasi interaksinya. Setengah volume interaksi berasal dari hanya sepersepuluh katalog. Ini realistis: marketplace biasanya memiliki sedikit item yang dilihat semua orang, dan sangat banyak item yang hampir tidak dilihat siapa pun. Ini juga *ramah bagi baseline popularitas*, yang akan kita manfaatkan lalu coba kalahkan.

317 item tanpa interaksi itu adalah item **cold-start** pada bab 12: hanya fitur konten, *boost*, dan eksplorasi yang bisa memunculkannya.

Sparsity

Sebelum *cutoff* waktu, 1,408 pengguna dan 2,120 item memiliki setidaknya satu interaksi. Mereka terhubung oleh 18,256 sel bukan nol dalam matriks pengguna-kali-item, sehingga densitasnya 0.612% dan sparsity-nya 99.388%. Dengan kata lain, pengguna aktif yang tipikal hanya menyentuh sebagian kecil sekali dari item yang bisa ia sentuh, dan sisa barisnya tidak diketahui. Matriks ini cukup kecil untuk disimpan di memori, tetapi bentuk yang sama pada skala nyata (ratusan juta pengguna, ratusan juta item) adalah alasan mengapa setiap metode di bab-bab berikutnya harus dirancang untuk matriks yang hampir kosong.

Pengguna pun tidak setara

Blok terakhir menunjukkan berapa banyak klik yang dibuat setiap pengguna sebelum *cutoff*. Pengguna median mengklik 11 item; kuartilnya 5 dan 20; persentil ke-90 adalah 31 dan persentil ke-99 adalah 56. Rata-ratanya 14.2 klik, maksimumnya 106, dan 92 pengguna tidak memiliki klik sama sekali. Ke-92 pengguna itu adalah **pengguna cold-start**. Untuk mereka, yang terbaik yang bisa kita lakukan adalah menampilkan item populer atau sedang tren, atau memakai konteks seperti kota atau tingkat perangkat. Kita akan melihat bagaimana aplikasi langsung melakukan ini untuk pengunjung anonim di halaman beranda pada bab 13.

Tahukah kamu?

Rekomendasi bukan fitur sampingan di layanan streaming besar. Para insinyur Netflix melaporkan bahwa sistem rekomendasi mereka memengaruhi pilihan untuk sekitar 80% jam yang ditonton di layanan tersebut, sedangkan 20% sisanya berasal dari pencarian. Angka itu dipublikasikan dalam artikel tentang algoritma dan nilai bisnis dari sistem rekomendasi Netflix.

Sumber: Gomez-Uribe et al., ACM Transactions on Management Information Systems, 2015

Jebakan dan praktik di produksi

Menyangka log adalah kebenaran. Log adalah sampel dari apa yang *dipilih untuk ditampilkan* oleh sistem. Item yang tidak pernah ditampilkan tidak punya klik, yang tidak menyatakan apa-apa tentang apakah orang akan mengkliknya (bias eksposur, *exposure bias*). Tim produksi mencatat impresi beserta posisi dan *surface* agar model dapat memperhitungkannya.

Memperlakukan semua event setara. Satu impresi bukan satu order. Bobot 1 / 3 / 5 yang dipakai dalam buku ini bersifat sembarang tetapi masuk akal; di produksi, bobot tersebut, atau pilihan *tujuan utama* seperti order atau pendapatan, adalah keputusan produk yang perlu ditinjau ulang ketika bisnis berubah.

Perbedaan di produksi. Marketplace besar mencatat tipe event yang sama melalui *stream* (misalnya Kafka), dengan validasi, deduplikasi, *warehouse* untuk pelatihan, dan *feature store* untuk penyajian (*serving*). Impresi mahal pada skala besar, sehingga banyak platform mencatatnya di sisi klien secara *batch*, seperti yang dilakukan frontend Vue kita (bab 13).

Tips

Sebelum membangun model apa pun, luangkan satu jam dengan *notebook* dan tabel mentah. Cetak ukuran tabel, corong event, distribusi event per pengguna dan per item, serta deret waktu event per hari. Kelima pemeriksaan ini menangkap sebagian besar bug data, dan membentuk ekspektasi yang jujur tentang apa yang bisa dicapai model.

Peringatan

Jangan pernah menghitung click-through rate sebagai klik dibagi *pengguna* atau *sesi*. CTR membutuhkan jumlah **impresi** sebagai penyebutnya. Jika sistem Anda tidak mencatat impresi, Anda tidak bisa membedakan item populer dari item yang sekadar sangat sering ditampilkan.

Coba sendiri

Buka `book/code/ch01_explore.py` dan temukan *loop* `for pct in (1, 10, 20)`. Ubah tuple-nya menjadi `(1, 5, 10, 50)`, jalankan ulang skripnya, dan bandingkan pangsanya. Lalu lihat `common.history()`, yang membangun tampilan data *point-in-time* yang akan dipakai di sisa buku ini.

Ringkasan

- Sistem rekomendasi mengubah himpunan item yang sangat besar menjadi daftar pendek, berurutan, dan personal; agar dapat ditangani, pekerjaannya dibagi menjadi *retrieval* yang murah dengan recall tinggi dan *ranking* yang mahal dengan presisi tinggi, dengan *re-ranking* di atasnya.
- Umpan balik eksplisit (rating) bersih tetapi langka; umpan balik implisit (impresi, klik, keranjang, order) melimpah tetapi bising, dan ketiadaan klik bukan berarti tidak suka.
- Simulator menyimpan kebenaran tersembunyi (selera, kualitas, sensitivitas harga) dan hanya mencatat apa yang akan dilihat sebuah aplikasi. Metode Anda harus menemukan kembali kebenaran dari log.
- Marketplace ini memiliki tujuh tabel mentah; `events` adalah log utama, dan grain-nya adalah satu aksi pada satu item. Impresi sangat penting sebagai penyebut CTR.
- Corongnya berjalan dari 233,460 impresi ke 24,726 klik (10.59%), 10,619 keranjang, dan 5,858 order; matriksnya 99.388% jarang (*sparse*).
- Popularitas sangat terkonsentrasi: 10% item teratas mengumpulkan 50.3% interaksi, dan 317 item tidak memiliki interaksi sama sekali.
- Waktu penting: akhir pekan, tanggal gajian, serta sale 8.8 dan 9.9 mengubah volume lalu lintas dan konversi, itulah sebabnya kita membagi data berdasarkan waktu dan memakai fitur berjendela (*window*).
- 92 pengguna tidak memiliki klik sebelum *cutoff*; setiap sistem nyata membutuhkan *fallback* untuk cold-start.

Kuis

Q1.1 Pernyataan manakah yang paling tepat menggambarkan tahap *retrieval* pada sistem rekomendasi dua tahap?

A. Mengurutkan 20 item dengan model besar yang memakai ratusan fitur.
B. Menyempitkan jutaan item secara murah menjadi beberapa ratus kandidat, dengan mengutamakan recall.
C. Menegakkan keberagaman dan aturan bisnis pada daftar akhir.
D. Menghitung hasil A/B test.

Q1.2 Manakah yang merupakan contoh umpan balik *implisit*?

A. Pengguna memberi rating 4 dari 5 bintang pada sebuah produk.
B. Pengguna menulis ulasan teks.
C. Pengguna menghabiskan 40 detik di halaman produk dan memasukkannya ke keranjang.
D. Pengguna mengetuk jempol ke atas pada sebuah rekomendasi.

Q1.3 Pada simulator buku ini, mengapa penting bahwa selera laten dan kualitas item *tidak* ditulis ke tabel keluaran?

A. Menghemat ruang disk.
B. Memaksa sistem rekomendasi belajar dari jenis data yang sama dengan yang dicatat aplikasi

sungguhan, sementara kebenarannya tetap tersedia untuk bernalar tentang apa yang dapat dipelajari.
C. Variabel laten tidak bisa disimpan dalam DataFrame pandas.
D. Postgres menolak kolom *floating-point*.

Q1.4 Dalam satu atau dua kalimat, jelaskan mengapa event `impression` diperlukan untuk menghitung click-through rate, dan apa yang salah jika sistem hanya mencatat klik.

Q1.5 Jelaskan apa arti *sparsity* pada matriks interaksi pengguna-kali-item dan sebutkan satu konsekuensinya bagi desain algoritma rekomendasi.

Q1.6 Baca fungsi `_day_weights` di atas. Berapa bobot mentah (sebelum normalisasi) untuk hari Sabtu yang bertepatan dengan sale tanggal kembar, seperti 8 Agustus 2026, dan untuk hari Minggu yang merupakan hari gajian tetapi bukan hari sale? Mengapa fungsi ini mengembalikan *flag* `payday` dan `sale` selain probabilitasnya?

Q1.7 Dengan memakai rasio corong dari bab ini (click-through rate 10.59%, click-to-cart 42.95%, cart-to-order 55.17%), berapa order yang Anda harapkan dari 10,000 impresi? Bulatkan ke bilangan bulat terdekat.

Q1.8 Sebuah marketplace memiliki 1,000 pengguna aktif dan 10,000 item aktif, dan 50,000 pasangan pengguna-item yang berbeda memiliki setidaknya satu interaksi. Berapa densitas dan sparsity matriksnya, dalam persen?

Q1.9 *Praktik langsung.* Pada `book/code/ch01_explore.py`, ubah *loop* `for pct in (1, 10, 20)` menjadi `for pct in (1, 5, 10, 50)` dan jalankan ulang skripnya. Berapa pangsa seluruh interaksi yang diterima oleh 50% item teratas, dan apa yang dikatakannya tentang separuh bawah katalog? Sebutkan juga apa yang Anda harapkan untuk pangsa 5% teratas relatif terhadap 1% teratas dan 10% teratas.

Q1.10 Seorang rekan mengusulkan memperlakukan setiap impresi yang tidak diklik sebagai label "tidak suka" untuk pelatihan. Berikan dua alasan mengapa asumsi ini berisiko pada aplikasi marketplace.

Bab 2 — Mengukur keberhasilan

Senin pagi di sebuah perusahaan marketplace, dua engineer berdebat di depan dashboard. Dina baru saja menyelesaikan model pemeringkatan (*ranking*) baru untuk home feed. Di laptopnya model itu "terlihat bagus": item teratas untuk akun ujinya sendiri semuanya barang yang akan ia klik. Raka, yang memegang funnel checkout, belum yakin. Kuartal lalu, model lain yang "jelas lebih baik" dirilis, konversi turun selama seminggu, dan tak ada yang bisa memastikan apakah penyebabnya model itu, promo akhir pekan, atau sekadar *noise*.

Keduanya benar untuk khawatir. Sebuah sistem rekomendasi tidak punya satu keluaran yang pasti benar, dan tidak ada unit test yang berbunyi "kesepuluh item ini tepat untuk user 43". Yang kita punya adalah **perilaku**: apa yang dilakukan orang setelah kita menampilkan sesuatu. Mengukur keberhasilan berarti mengubah perilaku itu menjadi angka yang cukup jujur untuk dipakai mengambil keputusan, cukup murah untuk dihitung seratus kali sehari, dan cukup berperilaku baik sehingga angka yang lebih tinggi benar-benar berarti produk yang lebih baik. Setiap bab berikutnya ditutup dengan tabel hasil yang dihasilkan oleh kode yang sama, jadi penting untuk memahami persis apa yang dilakukan kode itu, dan di mana ia bisa menyesatkan Anda.

Apa yang akan Anda pelajari

- Perbedaan antara evaluasi **offline** dan **online**, dan mengapa Anda membutuhkan keduanya.
- Mengapa **pembagian berbasis waktu** (*time-based split*) adalah satu-satunya cara yang jujur untuk mengevaluasi sistem rekomendasi, dan bagaimana pembagian acak diam-diam membocorkan masa depan.
- Cara menghitung **precision@K**, **recall@K**, dan **average precision** secara manual, serta apa yang dihargai oleh masing-masing.
- Bagaimana **DCG** dan **NDCG** diturunkan langkah demi langkah, dan mengapa repo ini memakai gain 1, 3, dan 7 untuk klik, tambah ke keranjang, dan pesanan.
- Apa yang diukur oleh **coverage** dan **candidate recall**, dan mengapa sistem rekomendasi bisa tampak "akurat" padahal tidak berguna.
- Bagaimana `recommender/recsys/evaluate.py` mengimplementasikan semua ini dalam sekitar empat puluh baris pandas, baris demi baris.

Intuisi: menilai ujian yang jawabannya tidak Anda tulis

Bayangkan Anda guru yang ingin tahu apakah seorang murid sudah menguasai aljabar. Anda tidak akan membagikan ujian, membiarkan murid melihat kunci jawaban, lalu menilai ujian yang sama. Anda akan menyisihkan soal yang belum pernah dilihat murid. Dan Anda juga tidak akan memakai ujian *peramal* yang soalnya diambil dari silabus tahun depan; ujian harus sesuai dengan pengetahuan yang memang bisa Anda miliki pada saat itu.

Mengevaluasi sistem rekomendasi mengikuti aturan yang sama, dengan satu keunikan: "kunci jawabannya" adalah masa depan. Kita membekukan dunia pada satu titik waktu (**cutoff**), membuat

rekomendasi hanya dari apa yang diketahui sebelum titik itu, lalu melihat apa yang benar-benar dilakukan pengguna *setelahnya*. Jika daftar top-20 kita memuat sekantong makanan kucing yang kemudian dipesan pengguna, kita mendapat poin; jika isinya empat puluh casing ponsel yang tak pernah ia sentuh, kita tidak mendapat apa-apa.

Dua hal mengikuti dari sini. Pertama, **murid tidak boleh melihat kunci jawaban**: apa pun yang dihitung dari data setelah cutoff terlarang saat kita menghasilkan rekomendasi. Inilah *kebocoran data* (*leakage*), penyebab nomor satu hasil yang tampak indah secara offline lalu ambruk di produksi. Kedua, **satu ujian tidak pernah cukup**: sistem rekomendasi bisa mendapat skor akurasi bagus sambil tetap menampilkan dua puluh barang terlaris yang sama kepada semua pengguna, sehingga kita memerlukan metrik untuk akurasi, kualitas pemeringkatan, keluasan, dan batas atas yang ditetapkan oleh tahap-tahap pipeline sebelumnya. Jawaban yang berbeda juga layak diberi bobot berbeda: pesanan lebih bernilai daripada klik, dan slot 1 lebih bernilai daripada slot 18.

Evaluasi offline versus online

Ada dua tempat untuk mengukur sistem rekomendasi, dan keduanya menjawab pertanyaan yang berbeda.

Evaluasi online berarti menempatkan model di hadapan pengguna sungguhan dan mengamati apa yang terjadi. Alat standarnya adalah **A/B test**: membagi pengguna secara acak menjadi kelompok kontrol (model lama) dan kelompok perlakuan (model baru), lalu membandingkan metrik bisnis, seperti *click-through rate* $\mathrm{CTR} = \frac{\text{clicks}}{\text{impressions}}$, tingkat konversi, jumlah pesanan per sesi, atau *gross merchandise value* (GMV) per pengguna. Hasil online adalah kebenaran dasar (*ground truth*). Ia menangkap segalanya: efek kebaruan, tata letak halaman, latensi, dan fakta bahwa pengguna bereaksi terhadap apa yang ditampilkan kepada mereka. Namun ia lambat (berhari-hari atau berminggu-minggu untuk mencapai signifikansi statistik), mahal (model yang buruk merugikan pendapatan sungguhan selama Anda mencari tahu bahwa ia buruk), dan Anda hanya bisa menjalankan beberapa eksperimen sekaligus.

Evaluasi offline memutar ulang sejarah. Anda mengambil perilaku yang tercatat, berpura-pura sekarang adalah hari sebelum sesuatu terjadi, menghasilkan rekomendasi, lalu menilainya terhadap apa yang terjadi berikutnya. Ia cepat (hitungan menit), gratis, dapat diulang, dan Anda bisa menguji seratus varian dalam satu sore. Kelemahannya, sejarah hanya berisi apa yang dilakukan pengguna sebagai respons terhadap pilihan sistem *lama*. Item yang tak pernah ditampilkan sistem lama tidak punya klik; model yang akan memunculkannya tidak mendapat kredit. Angka offline hanyalah **proksi**.

Dalam praktik, evaluasi offline adalah filter murah di depan beberapa eksperimen online yang mahal: ide yang kalah secara offline dibuang, dan hanya yang lolos yang mendapat A/B test. Metrik offline berguna untuk **menolak ide buruk dengan cepat** dan untuk memilih di antara varian yang mirip. Hasil offline yang bagus itu perlu, tetapi jarang cukup.

> **Tahukah kamu?**
>
> Ketika Netflix menggelar kompetisi berhadiah yang terkenal untuk prediksi rating film, targetnya adalah sebuah proksi: menurunkan *root mean squared error* (RMSE) prediksi rating bintang sebesar 10% dimaksudkan sebagai wakil dari "rekomendasi yang lebih baik". Gabungan ratusan model dari tim pemenang memang melewati garis itu, namun Netflix melaporkan bahwa tambahan akurasi yang terukur secara offline tampaknya tidak sepadan dengan upaya rekayasa untuk membawa ensemble itu ke produksi. Skor offline yang hebat tidak sama dengan produk yang hebat.
>
> Sumber: Amatriain & Basilico, Netflix Technology Blog, 2012

Pembagian berbasis waktu, dan mengapa pembagian acak membocorkan data

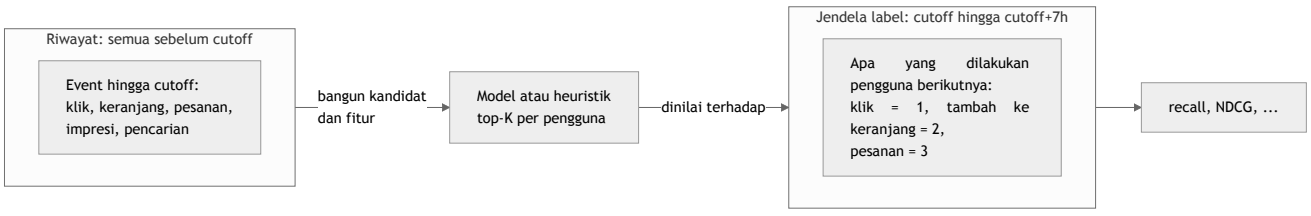
Tutorial machine learning biasanya berkata "acak datanya dan sisihkan 20% untuk pengujian". Untuk sistem rekomendasi, saran ini berbahaya. Misalkan kita punya 60 hari event lalu mengacaknya menjadi 80% train dan 20% test. Sebuah klik pada hari ke-52 bisa masuk ke set training sementara klik pengguna yang sama pada hari ke-14 masuk ke set test. Model lalu "memprediksi" hari ke-14 dengan memanfaatkan pengetahuan tentang hari ke-52. Secara konkret ia bisa mempelajari:

- **Popularitas dari masa depan.** Item yang meledak saat sale 9.9 menjadi laris *karena* klik yang terjadi setelah event uji.
- **Selera pengguna dari masa depan.** Jika pengguna beralih dari ponsel ke makanan kucing pada minggu ke-7, model yang dilatih dengan minggu ke-7 "memprediksi" klik minggu ke-3 dengan menghafal di mana ia akhirnya berlabuh.
- **Kebocoran sesi.** Klik dalam satu sesi sangat berkorelasi (orang menelusuri item serupa berturut-turut). Membelah satu sesi ke train dan test membuat set test hampir menjadi salinan set training.

Di produksi Anda tidak pernah bisa melihat masa depan, sehingga model yang bergantung padanya gagal pada hari pertama ia berjalan. Solusinya adalah menghormati arah waktu.

Pembagian yang dipakai di buku ini

Repo ini memakai **pembagian waktu rolling-origin**. Pilih sebuah cutoff t_c. Semua yang terjadi sebelum t_c adalah *riwayat* (*history*); $H = 7$ hari berikutnya adalah *jendela label*; dan model dinilai dari seberapa baik ia memprediksi jendela itu.



Gambar 2.1: *Fitur dan kandidat hanya berasal dari riwayat; label hanya berasal dari jendela label setelah cutoff.*

Untuk angka evaluasi di buku ini, skrip pendamping menetapkan akhir dataset pada hari simulasi terakhir, menaruh cutoff 7 hari sebelumnya, dan memakai minggu terakhir sebagai jendela label.

Untuk *melatih* ranker (bab 10) kita memerlukan pembagian kedua yang lebih awal, agar label training seluruhnya berada sebelum cutoff evaluasi:

Tujuan	Riwayat berakhir pada	Jendela label
Melatih ranker	end − 14 hari	end − 14 hari hingga end − 7 hari
Evaluasi	end − 7 hari	end − 7 hari hingga end
Serving (produksi)	sekarang	masa depan, belum teramati

Ini persis diagram pada docstring `recommender/recsys/pipeline.py`. Model tidak pernah dilatih pada minggu yang kelak dipakai untuk menilainya.

Contoh kecil kebocoran

Misalkan Rina mengklik mainan C pada hari ke-13 dan memesannya pada hari ke-14. Pembagian **acak** bisa menaruh hari ke-14 di training dan hari ke-13 di test, sehingga pertanyaan "akankah Rina mengklik mainan C?" diajukan kepada model yang sudah melihat pesanannya atas C. Pembagian **waktu** pada hari ke-12 memaksa model menebak hanya dari klik-klik sebelumnya, seperti yang harus dilakukan sistem langsung.

Aturan yang sama berlaku untuk fitur

Membagi event dengan benar baru separuh pekerjaan: fitur seperti "jumlah pesanan item ini dalam 7 hari terakhir" harus dihitung dari event sebelum cutoff, bukan "hingga hari ini". Bab 9 menyebutnya **ketepatan point-in-time** (*point-in-time correctness*). Aturannya: *pada saat prediksi t, Anda hanya boleh memakai data dengan timestamp $< t$.*

Apa yang kita ukur: label dan daftar berperingkat

Sebelum masuk ke rumus, mari tetapkan kosakata. Untuk pengguna u dan cutoff t_c tertentu, sistem rekomendasi menghasilkan **daftar berperingkat** item $r_1, r_2, \ldots, r_K$, terbaik lebih dulu. Kita membandingkannya dengan apa yang benar-benar dilakukan pengguna selama jendela label. Repo ini menurunkan **label** bergradasi untuk setiap pasangan (pengguna, item) yang disentuh pengguna:

Perilaku dalam jendela label	Label
Mengklik item	1
Menambahkan item ke keranjang	2
Memesan item	3
Tidak ada	(tidak ada dalam tabel label)

Jika pengguna melakukan beberapa hal pada item yang sama, kita ambil yang terkuat (maksimum); ini dilakukan oleh `data.labels` di `recommender/recsys/data.py`. Fungsi itu mengembalikan *hanya* pasangan dengan label positif, sehingga pengguna yang tidak melakukan apa pun di jendela label tidak pernah muncul: mereka tidak dapat dinilai dan dikecualikan dari setiap rata-rata di bawah. Angka-angkanya adalah rata-rata atas *pengguna yang kembali dan melakukan sesuatu*.

Sebut item dengan label positif sebagai **positif masa depan** (*future positives*) milik pengguna, dan misalkan R adalah jumlahnya. Semua yang berikut menyangkut seberapa baik daftar berperingkat menangkap mereka.

Algoritma: metrik pemeringkatan dari dasar

Precision@K dan recall@K

Dua pertanyaan paling dasar tentang daftar top-K adalah:

- **Precision@K**: dari K item yang kita tampilkan, berapa bagian yang direspons pengguna?
- **Recall@K**: dari hal-hal yang direspons pengguna, berapa bagian yang kita tampilkan?

$$\text{precision@}K = \frac{|\text{hits in top } K|}{K}, \qquad \text{recall@}K = \frac{|\text{hits in top } K|}{R}$$

"Hit" adalah item dalam daftar yang merupakan positif masa depan. Untuk satu pengguna, keduanya hanyalah sebuah hitungan yang dibagi dengan bilangan berbeda. Precision peduli pada *pemborosan* (setiap slot adalah ruang layar yang mahal), recall peduli pada *cakupan terhadap apa yang diinginkan pengguna*. Di marketplace dengan ribuan item per kategori, dan pengguna yang hanya merespons segelintir dalam seminggu, nilai precision wajar saja sangat kecil; recall lebih mudah dibaca dan paling sering dilaporkan buku ini. Ketika K tetap dan besar dibandingkan R, precision dan recall membawa informasi yang sama hingga faktor skala, itulah sebabnya kita lebih sering menampilkan recall.

Keduanya mengabaikan **urutan** di dalam top K: menemukan hit di slot 1 atau slot 20 memberi skor sama. Di halaman sungguhan itu keliru, karena slot 1 dilihat hampir semua orang sedangkan slot 20 menuntut scroll. Kita butuh metrik yang peka terhadap posisi.

Average precision (AP)

Average precision menghargai penempatan hit di awal. Telusuri daftar; setiap kali Anda menemukan hit pada posisi k, catat precision pada titik potong itu, $P@k$. Lalu rata-ratakan nilai-nilai itu atas *seluruh* positif masa depan pengguna:

$$\text{AP@}K = \frac{1}{R} \sum_{k=1}^{K} P@k \cdot 1[\text{item at } k \text{ is a hit}]$$

Contoh berukuran tangan. Daftar tiga item memiliki hit pada posisi 2 dan 3, dan pengguna memiliki $R = 2$ positif masa depan. Maka $P@2 = 1/2$ dan $P@3 = 2/3$, sehingga

$$\text{AP} = \frac{1}{2}\left(\frac{1}{2} + \frac{2}{3}\right) = 0.583.$$

Dengan hit pada posisi 1 dan 2, kita akan mendapat $\frac{1}{2}(1 + 1) = 1$. Membagi dengan R (bukan dengan jumlah hit) menghukum positif yang tidak pernah muncul. Namun AP bersifat biner: pesanan dan klik sama-sama dihitung sebagai "hit", jadi untuk relevansi bergradasi kita memakai DCG.

Cumulative gain, dan diskon

Misalkan rel_i adalah label (0, 1, 2, atau 3) item pada peringkat i, dengan peringkat dihitung dari 1. Skor bergradasi yang paling naif hanya menjumlahkannya:

$$\mathrm{CG@}K = \sum_{i=1}^{K} \mathrm{rel}_i$$

Ada dua masalah. Pertama, label 3 bukan sekadar "tiga kali" sebuah klik: pesanan adalah sinyal yang jauh lebih kuat dan bernilai. Kita memperbaikinya dengan mengubah label menjadi **gain** yang tumbuh secara eksponensial:

$$g(\mathrm{rel}) = 2^{\mathrm{rel}} - 1 \quad \Rightarrow \quad g(0)=0,\ g(1)=1,\ g(2)=3,\ g(3)=7.$$

Angka-angka ini persis yang ada pada dictionary `GAIN` di `evaluate.py`, dan sama dengan `label_gain` yang dipakai ranker LightGBM di bab 10, sehingga training dan evaluasi sepakat tentang apa arti "bagus".

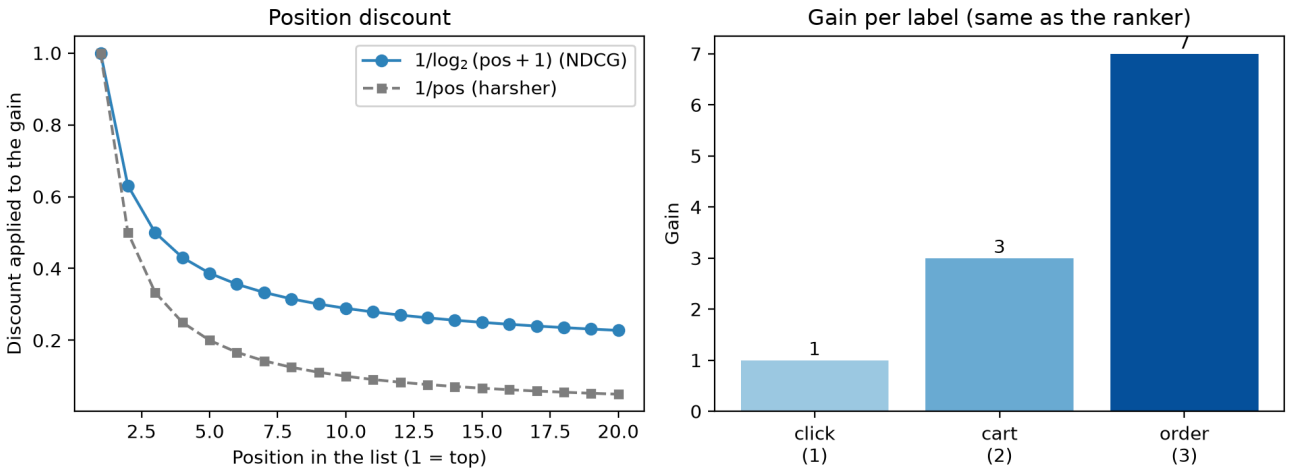
Kedua, CG tidak peduli urutan. Kita memperbaikinya dengan **diskon posisi**: item pada peringkat i bernilai lebih kecil semakin ke bawah posisinya. Pilihan klasiknya adalah

$$d(i) = \frac{1}{\log_2(i+1)}$$

sehingga $d(1)=1,\ d(2)=0.63,\ d(3)=0.5$, dan diskon terus menyusut, secara perlahan. Menggabungkan kedua perbaikan menghasilkan **discounted cumulative gain**:

$$\mathrm{DCG@}K = \sum_{i=1}^{K} \frac{2^{\mathrm{rel}_i} - 1}{\log_2(i+1)}$$

Mengapa logaritma? Penurunan antara slot 1 dan 2 besar, antara slot 19 dan 20 dapat diabaikan, karena sebagian besar pengguna tidak pernah sampai ke sana; diskon $1/i$ akan jauh lebih kejam. Logaritma adalah konvensi, bukan hukum alam, tetapi itulah yang disepakati bidang ini. Gambar di bawah memperlihatkan kedua kurva, serta gain yang dipakai repo ini.



Gambar 2.2: *Kiri: diskon posisi* $1/\log_2(\text{pos}+1)$ *dibandingkan* $1/\text{pos}$ *yang lebih keras. Kanan: gain yang diberikan untuk klik, tambah ke keranjang, dan pesanan (1, 3, dan 7).*

Normalisasi: NDCG

DCG tidak dapat dibandingkan antarpengguna. Pengguna dengan satu klik masa depan memiliki DCG maksimum 1; pengguna dengan lima pesanan masa depan memiliki maksimum yang jauh lebih tinggi. Untuk merata-ratakan antarpengguna kita memerlukan skala bersama. Solusinya adalah membagi dengan **DCG ideal** (IDCG): DCG dari urutan terbaik yang mungkin atas positif masa depan pengguna itu, yaitu diurutkan menurut gain, tertinggi lebih dulu.

$$\text{NDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K} \in [0, 1]$$

NDCG 1 berarti "Anda mendaftar setiap positif masa depan, terbaik lebih dulu". Skor 0 berarti Anda tidak menemukan satu pun. Angka akhir yang dilaporkan adalah rata-rata NDCG atas semua pengguna yang dievaluasi.

Perhatikan kehalusan pada daftar ideal: ia memuat *setiap* positif masa depan milik pengguna, termasuk item yang tidak mungkin ditemukan sistem rekomendasi mana pun (misalnya item yang kehabisan stok pada cutoff, atau yang ditemukan pengguna lewat tautan dari teman). Itulah yang membuat NDCG@K jujur: ia diukur terhadap perilaku nyata pengguna, bukan terhadap apa yang dapat direkomendasikan.

> **Tahukah kamu?**
>
> Keluarga ukuran DCG diperkenalkan oleh Järvelin dan Kekäläinen dalam makalah tahun 2002 tentang evaluasi sistem *information retrieval* dengan relevansi bergradasi. Mereka mengusulkan agar gain yang diterima pengguna sepanjang daftar berperingkat dikumulasikan dan item didiskon menurut peringkatnya, justru agar menemukan dokumen yang sangat relevan lebih awal dihargai lebih besar daripada menemukannya belakangan. Dua dekade kemudian, hampir setiap leaderboard pencarian dan sistem rekomendasi masih melaporkan NDCG.
>
> Sumber: Järvelin & Kekäläinen, ACM Transactions on Information Systems, 2002

Coverage

Metrik akurasi adalah rata-rata atas pengguna, sehingga sistem rekomendasi bisa mendapat skor bagus dengan menampilkan item populer yang nyaris sama kepada semua orang. **Catalog coverage** mengukur keluasan:

$$\text{coverage@}K = \frac{|\bigcup_u \text{top-}K(u)|}{|\text{recommendable items}|}$$

Dari semua item yang boleh kita rekomendasikan, berapa bagian yang pernah muncul di top K milik setidaknya satu pengguna? Untuk marketplace ini bukan sekadar estetika: penjual yang itemnya tidak pernah ditampilkan tidak akan pernah menjual. Coverage *bukan* skor kualitas yang berdiri sendiri (sistem rekomendasi acak memiliki coverage sempurna dan akurasi terburuk), jadi pakailah ia sebagai **guardrail** di samping metrik akurasi.

Candidate recall: batas atas

Sistem rekomendasi produksi, termasuk milik kita, bersifat **multi-tahap**: tahap pertama yang murah memilih beberapa ratus *kandidat* (*candidate*) dari ribuan (atau jutaan) item, lalu tahap kedua yang lebih cerdas mengurutkannya. Tahap kedua hanya bisa mengurutkan ulang apa yang disediakan tahap pertama, sehingga

$$\text{candidate recall} = \frac{|\text{future positives that appear in the candidate set}|}{R}$$

(dirata-ratakan atas pengguna) adalah **batas atas** recall pemeringkatan akhir: jika *candidate generation* tidak pernah mengambil item yang dibeli pengguna, tidak ada ranker yang bisa menaruhnya di slot 1. Ketika ranker baru mengecewakan, periksa angka ini lebih dulu. Bab 8 membahas cara menaikkannya.

Penelusuran kode: `recsys/evaluate.py`

Seluruh modul evaluasi cukup pendek untuk dibaca dalam satu kali duduk. Mulailah dengan `metrics`:

recommender/recsys/evaluate.py · metrics

```python
def metrics(recs: pd.DataFrame, labels: pd.DataFrame, k: int) -> dict[str, float]:
    """`recs`: user_id, item_id in display order (top K per user). `labels`: user_id, item_id,
label."""
    recs = recs.groupby("user_id").head(k)[["user_id", "item_id"]].copy()
    recs["pos"] = recs.groupby("user_id").cumcount()
    hits = labels.merge(recs, on=["user_id", "item_id"], how="left")

    per_user = hits.groupby("user_id").agg(n_pos=("label", "size"), n_hit=("pos", "count"))
    recall = (per_user["n_hit"] / per_user["n_pos"]).mean()

    orders = hits[hits["label"] == 3]
    ou = orders.groupby("user_id").agg(n=("label", "size"), h=("pos", "count"))
    order_recall = (ou["h"] / ou["n"]).mean()

    found = hits.dropna(subset=["pos"])
    dcg = (found["label"].map(GAIN) / np.log2(found["pos"] + 2)).groupby(found["user_id"]).sum()
    ideal_gains = labels.assign(g=labels["label"].map(GAIN)).sort_values(["user_id", "g"],
ascending=[True, False])
    ideal_gains = ideal_gains.groupby("user_id").head(k)
    ideal_gains["pos"] = ideal_gains.groupby("user_id").cumcount()
    idcg = (ideal_gains["g"] / np.log2(ideal_gains["pos"] +
2)).groupby(ideal_gains["user_id"]).sum()
    ndcg = (dcg.reindex(idcg.index, fill_value=0.0) / idcg).mean()

    return {f"recall@{k}": recall, f"order_recall@{k}": order_recall, f"ndcg@{k}": ndcg}
```

Fungsi ini menerima dua frame. `recs` berisi baris `user_id, item_id` dalam urutan tampil; `labels` adalah tabel dari `data.labels`. Kita menelusurinya dalam empat langkah.

Langkah 1: beri nomor pada rekomendasi. `recs.groupby("user_id").head(k)` mempertahankan k baris pertama tiap pengguna, sehingga posisi *diturunkan dari urutan baris*: frame yang tidak terurut diam-diam menilai daftar acak. `cumcount()` memberi nomor `0, 1, 2, …` ke `pos`. Posisi berbasis nol, karena itu diskon memakai `np.log2(pos + 2)` untuk peringkat berbasis satu i pada rumus, $\log_2(i+1)$.

Langkah 2: kaitkan rekomendasi ke label (merge). `labels.merge(recs, ..., how="left")` dimulai dari *label*, sehingga setiap positif masa depan bertahan dalam merge, dengan `pos` terisi jika kita merekomendasikannya dan `NaN` jika tidak. Itu membuat penyebutnya benar: jumlah baris per pengguna adalah R, dan `count()` dari `pos` (yang mengabaikan `NaN`) adalah jumlah hit. Pengguna yang tidak kita rekomendasikan apa pun mendapat recall 0 alih-alih dibuang, sehingga sebuah metode tidak bisa menaikkan skornya dengan melewati pengguna yang sulit. `order_recall` melakukan hal yang sama hanya untuk baris berlabel 3, metrik yang paling dekat dengan pendapatan; rata-ratanya hanya mencakup pengguna yang memesan.

Langkah 3: DCG. `found` hanya menyimpan hit, lalu `GAIN[label] / log2(pos + 2)` dijumlahkan per pengguna. `GAIN = {1: 1.0, 2: 3.0, 3: 7.0}` adalah tabel $2^{\text{rel}} - 1$ dari atas.

Langkah 4: IDCG dan rasio. Daftar ideal dibangun dari label yang sama: urutkan label tiap pengguna menurut gain menurun, ambil k pertama, dan diskon dengan cara yang sama. DCG (pengguna tanpa hit mendapat 0) dibagi IDCG, dirata-ratakan atas pengguna, adalah NDCG. Karena daftar ideal juga dipotong pada k, skor sempurna tetap dapat dicapai untuk pengguna yang memiliki lebih dari k positif.

Sekarang pemeriksaan batas atasnya:

recommender/recsys/evaluate.py · candidate_recall

```python
def candidate_recall(cands: pd.DataFrame, labels: pd.DataFrame) -> float:
    """Upper bound for any ranker: share of future positives present in the candidate set."""
    m = labels.merge(cands[["user_id", "item_id"]].assign(found=1), on=["user_id", "item_id"],
how="left")
    return float(m.groupby("user_id")["found"].apply(lambda s: s.notna().mean()).mean())
```

Ini adalah ide left-merge yang sama: setelah merge, `found` bernilai `NaN` untuk positif yang tidak ada dalam set kandidat, sehingga rata-rata per pengguna dari `found.notna()` adalah porsi positif pengguna itu yang menjadi kandidat. Tidak ada `k`: kandidat adalah himpunan, bukan daftar berperingkat.

Bab-bab berikutnya memanggil helper bersama, `evaluate` di `book/code/common.py`, yang membungkus `metrics` dan melaporkan recall pada 5, 10, 20, dan 50, order recall, NDCG pada 20, serta coverage pada 20 (item berbeda yang direkomendasikan dibagi item yang tersedia pada cutoff). Setiap metode dinilai oleh kode yang identik, sehingga angka di bab 6 dapat dibandingkan dengan angka di bab 3 tanpa catatan kaki.

Jalankan

Skrip pendamping `book/code/ch02_metrics.py` membangun daftar sepuluh item yang hasilnya kita kendalikan, dan mencetak setiap nilai antara agar Anda bisa mengikuti aritmetikanya dengan pensil. Jalankan dari direktori `book/`:

```
cd book
uv run python code/ch02_metrics.py
```

Blok pertama menampilkan daftar beserta label tiap item (apa yang kemudian dilakukan pengguna terhadapnya), gain-nya, diskonnya, dan gain terdiskonnya:

KELUARAN TERMINAL

```
=== The ranked list (what the user later did with each item) ===
pos item  label  gain  1/log2(pos+2)  discounted gain
  1 A         0   0.0         1.0000           0.0000
  2 B         3   7.0         0.6309           4.4165
  3 C         0   0.0         0.5000           0.0000
  4 D         1   1.0         0.4307           0.4307
  5 E         0   0.0         0.3869           0.0000
  6 F         0   0.0         0.3562           0.0000
  7 G         2   3.0         0.3333           1.0000
  8 H         0   0.0         0.3155           0.0000
  9 I         0   0.0         0.3010           0.0000
 10 J         1   1.0         0.2891           0.2891
```

Item B dipesan (label 3, gain 7), G ditambahkan ke keranjang (label 2, gain 3), dan D serta J diklik (label 1, gain masing-masing 1). Perhatikan bahwa header kolom `1/log2(pos+2)` memakai posisi berbasis nol dari kode; sebagai peringkat berbasis satu, itu adalah $1/\log_2(i+1)$ pada rumus.

Precision dan recall pada setiap titik potong

Pengguna memiliki total lima positif masa depan, termasuk item kelima, `Z`, yang tidak pernah ada di daftar kita:

KELUARAN TERMINAL

```
=== Precision and recall at each cut-off ===
future positives in total: 5 (one, 'Z', is never recommended)
  k  hits  precision@k  recall@k
  1     0        0.000     0.000
  2     1        0.500     0.200
  3     1        0.333     0.200
  4     2        0.500     0.400
  5     2        0.400     0.400
  6     2        0.333     0.400
  7     3        0.429     0.600
  8     3        0.375     0.600
  9     3        0.333     0.600
 10     4        0.400     0.800
```

Ikuti satu baris: pada $k = 4$ kita telah menampilkan item A sampai D, dengan B dan D sebagai hit, sehingga precision adalah $2/4 = 0.500$ dan recall adalah $2/5 = 0.400$. Dua pola terlihat. Recall hanya naik (lebih banyak slot hanya dapat menambah hit), mencapai maksimum 0.800 karena `Z` tidak pernah direkomendasikan. Precision bergerigi: ia melonjak setiap kali hit mendarat (posisi 2, 4, 7, 10) dan meluruh di antaranya. Gerigi itulah yang dirata-ratakan oleh average precision.

Average precision

KELUARAN TERMINAL

```
=== Average precision ===
hit at pos 2: precision@2 = 0.5000
hit at pos 4: precision@4 = 0.5000
hit at pos 7: precision@7 = 0.4286
hit at pos 10: precision@10 = 0.4000
AP = sum / 5 future positives = 1.8286 / 5 = 0.3657
```

Keempat hit berada pada peringkat 2, 4, 7, dan 10, dengan precision 0.5000, 0.5000, 0.4286, dan 0.4000, berjumlah 1.8286. Bagi dengan *lima* positif masa depan (bukan empat hit), dan AP adalah 0.3657. Positif kelima, `Z`, menarik skor turun karena menyumbang nol.

DCG, IDCG, dan NDCG

KELUARAN TERMINAL

```
=== DCG, IDCG and NDCG ===
ideal order of the 5 future positives (labels): [3, 2, 2, 1, 1]
  ideal pos 1: label 3, gain 7.0 / log2(2) = 7.0000
  ideal pos 2: label 2, gain 3.0 / log2(3) = 1.8928
  ideal pos 3: label 2, gain 3.0 / log2(4) = 1.5000
  ideal pos 4: label 1, gain 1.0 / log2(5) = 0.4307
  ideal pos 5: label 1, gain 1.0 / log2(6) = 0.3869
DCG  = 6.1362
IDCG = 11.2103
NDCG = DCG / IDCG = 0.5474
```

Urutan ideal kelima positif pengguna adalah menurut gain: pesanan (label 3), lalu dua tambah-ke-keranjang (label 2: item G dari daftar kita, ditambah `Z`, yang tidak pernah kita rekomendasikan), lalu dua klik (label 1). Suku-sukunya adalah 7.0000, 1.8928, 1.5000, 0.4307, dan 0.3869, yang berjumlah IDCG 11.2103. Daftar kita mencapai DCG 6.1362, sehingga

$$\text{NDCG} = \frac{6.1362}{11.2103} = 0.5474.$$

Kita menemukan sebagian besar nilainya (pesanan di peringkat 2 dan keranjang di peringkat 7) tetapi kehilangan sebagian karena `Z` tidak ada dan karena B tidak berada di urutan pertama.

Urutan berpengaruh: hit yang sama, daftar yang lebih buruk

Sekarang eksperimen yang membenarkan NDCG. Pertahankan empat hit yang persis sama, tetapi pindahkan item B yang dipesan dari peringkat 2 ke peringkat 10, dengan hit lainnya di peringkat 2, 5, dan 8:

KELUARAN TERMINAL

```
=== Same positives, but the order item B pushed to the last slot ===
DCG = 4.1304   NDCG = 0.3684   (recall@10 is unchanged: 0.800)
```

Himpunan item yang direkomendasikan sama, sehingga recall@10 tidak berubah (0.800). Tetapi NDCG turun dari 0.5474 menjadi 0.3684. Dashboard yang hanya memantau recall akan menyebut kedua daftar ini identik; pembeli yang harus scroll sampai dasar untuk menemukan barang yang ia inginkan tidak akan setuju.

Hasil: berapa skor sistem rekomendasi acak?

Terakhir, kita alirkan metode yang sengaja tidak berguna melalui helper `evaluate` bersama: untuk setiap pengguna dalam set label, tarik 50 item secara seragam acak dari katalog yang tersedia stoknya.

KELUARAN TERMINAL

```
=== Random recommender through common.evaluate (same code every chapter uses) ===
        recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20  coverage@20
method
random    0.0005     0.0025     0.0069     0.0212           0.0066   0.0026          1.0

evaluated users: 947, available items: 2311
```

Inilah **lantai** (*floor*) bagi setiap tabel di buku ini. Ada beberapa hal yang perlu dicermati.

Pertama, populasi evaluasi: 947 pengguna memiliki setidaknya satu positif pada minggu terakhir, dan 2311 item tersedia pada cutoff. Jika Anda memilih 20 item dari 2311 untuk satu pengguna dengan satu positif, peluang mengenainya adalah $20/2311 \approx 0.0087$. Itulah nilai recall@20 yang kita harapkan dari metode tanpa informasi (hampir semua positif, sekitar 99.85%, adalah item yang tersedia stoknya pada cutoff, sehingga nilai harapannya hanya sedikit lebih rendah). Angka terukur 0.0069 hanyalah satu kali pengundian acak: kebanyakan pengguna hanya memiliki beberapa positif, sehingga recall tiap pengguna adalah 0 atau pecahan kecil, dan rata-rata atas 947 pengguna bersifat *noisy*. Angkanya akan sedikit bergeser antar *seed* acak, tetapi akan selalu berkisar di sekitar 0.0087. Sistem rekomendasi acak ini juga menemukan 0.0212 positif pada recall@50, naik seiring K sebagaimana diharapkan, sementara NDCG@20 hanya 0.0026: hit acak mendarat di kedalaman acak, dan hit di peringkat 15 hanya memberi sedikit nilai.

Kedua, coverage@20 adalah 1.0: pengambilan sampel acak menyentuh setiap item, sehingga ia memiliki keluasan *sempurna* dan akurasi terburuk yang mungkin, itulah sebabnya coverage adalah

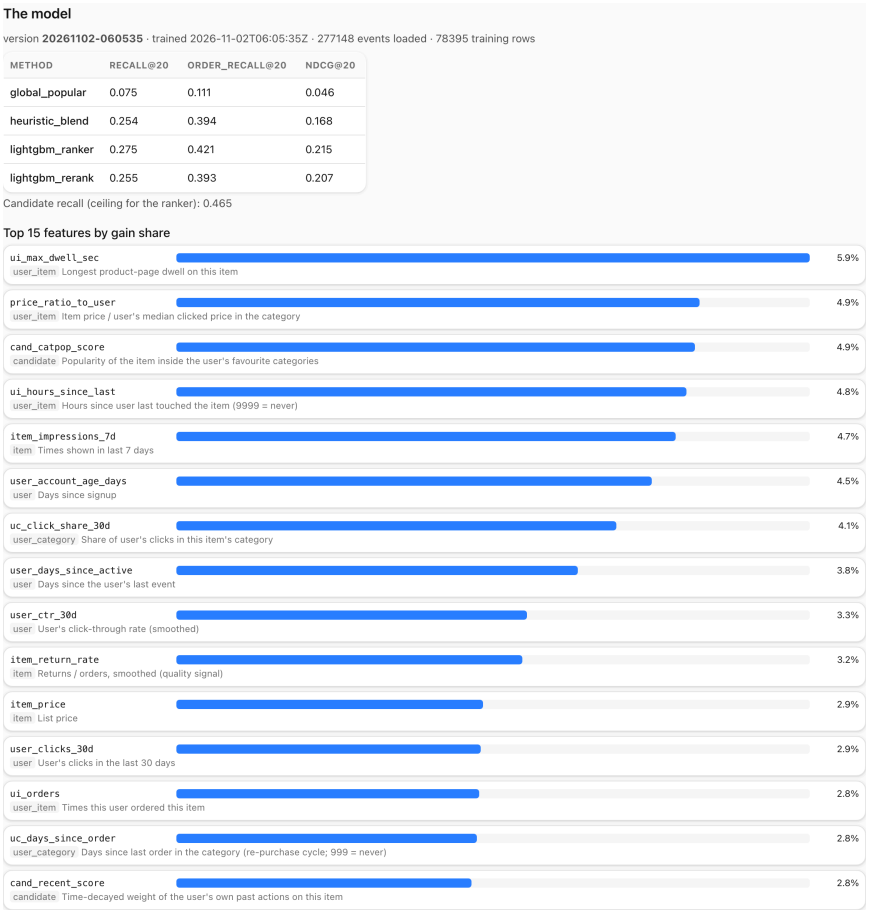
guardrail dan tidak pernah menjadi target.

Ketiga, setiap metode berikutnya harus mengalahkan angka-angka ini dengan selisih lebar, dan juga mengalahkan baseline popularitas satu baris, yang menjadi topik bab berikutnya.

> **Tahukah kamu?**
>
> Para evaluator offline sering mempercepat metrik mereka dengan memeringkat setiap positif uji terhadap sampel acak kecil item (misalnya 100) alih-alih seluruh katalog. Krichene dan Rendle menunjukkan bahwa **sampled metrics** semacam itu tidak konsisten dengan metrik eksak: mereka bahkan tidak mempertahankan urutan sistem rekomendasi secara ekspektasi, sehingga "model A mengalahkan model B" di bawah sampling bisa keliru pada katalog penuh. Saran mereka adalah menghindari sampling bila Anda sanggup melakukan perhitungan penuh, yang persis dilakukan `evaluate.py`.
>
> Sumber: Krichene & Rendle, KDD, 2020

Metrik yang sama di dalam aplikasi

Layanan rekomendasi mengevaluasi setiap model yang dilatih dengan fungsi `metrics` yang sama dan menyimpan hasilnya; halaman "How it works" menampilkannya.



Gambar 2.3: *Bagian "The model" pada halaman How it works: tabel recall@20, order_recall@20, dan ndcg@20 untuk empat metode, batas atas candidate recall, dan fitur teratas menurut gain share.*

Nilainya berbeda dari buku ini karena aplikasi melatih pada database langsung ketika Anda mengklik "Retrain", sedangkan skrip buku memakai simulasi, cutoff, dan seed yang tetap. Perhatikan *bentuknya*: beberapa metode diurutkan menurut NDCG, dengan batas atas di atasnya.

Jebakan, dan bagaimana dilakukan di produksi

Kebocoran lewat fitur, bukan hanya event. Pembagian di atas menjaga label di luar riwayat, tetapi fitur yang ceroboh (misalnya jumlah pesanan item sepanjang masa, dihitung dari database hari ini) diam-diam memasukkan kembali masa depan. Bab 9 memperagakan model yang NDCG offline-nya lebih dari dua kali lipat karena satu kolom bocor, lalu runtuh dalam evaluasi yang jujur. Jika angka offline tampak terlalu bagus, anggap ada kebocoran sampai terbukti sebaliknya.

Noise dan survivorship. Jendela label kita satu minggu dan 947 pengguna, sehingga selisih pada desimal ketiga antara dua metode kemungkinan besar hanya noise; tim produksi melaporkan *confidence interval* bootstrap dan merata-ratakan beberapa cutoff atau seed (bab 10 melakukannya untuk ablasi). Metrik juga hanya mencakup pengguna dengan setidaknya satu positif masa depan, jadi iris menurut kohort (baru versus kembali, kota, tingkat aktivitas; bab 12).

Bias pencatatan (*logging bias*). Klik hanya terjadi pada apa yang dipilih sistem lama untuk ditampilkan, dan slot yang lebih tinggi mendapat lebih banyak klik terlepas dari kualitas, sehingga label offline menghargai model yang meniru sistem lama. *Inverse-propensity weighting* dan traffic eksplorasi ada karena alasan ini (bab 11 dan 12); A/B test tetap menjadi penentu akhir.

Mengoptimalkan satu metrik saja. NDCG tidak mengatakan apa pun tentang keragaman, kesegaran, keadilan bagi penjual kecil, atau harga barang yang Anda dorong. Tim produksi menjaga dashboard **metrik guardrail** di samping metrik utama: coverage, porsi impresi untuk item baru, keragaman kategori (bab 11), latensi, dan tingkat error. Model yang menaikkan NDCG 2% sambil meruntuhkan coverage katalog bukanlah perbaikan.

Praktik produksi singkatnya. Pertahankan satu harness evaluasi dan satu definisi tiap metrik; jalankan pada setiap job training (layanan di repo ini menyimpan hasilnya di sebelah file model); beri versi pada dataset dan cutoff; dan bandingkan setiap metode baru dengan *model produksi sebelumnya* dan baseline sepele.

Ringkasan

- Evaluasi offline memutar ulang sejarah untuk menolak ide buruk dengan murah; A/B test online adalah kebenaran dasar. Gunakan offline sebagai gerbang, bukan sebagai janji.
- Selalu bagi menurut **waktu**: bangun kandidat dan fitur dari event sebelum cutoff, ambil label dari 7 hari berikutnya. Pembagian acak membocorkan masa depan.
- Label bergradasi: klik 1, tambah ke keranjang 2, pesanan 3, dengan perilaku terkuat per (pengguna, item) dipertahankan; hanya pengguna dengan setidaknya satu positif yang dinilai.
- Precision@K mengukur pemborosan, recall@K mengukur seberapa banyak yang diinginkan pengguna berhasil Anda temukan, dan AP menambahkan kepekaan terhadap letak hit.
- DCG mengubah label menjadi gain $2^{rel} - 1$ (1, 3, 7), mendiskon dengan $1/\log_2(i+1)$, dan NDCG membagi dengan DCG ideal sehingga skor antarpengguna dapat dibandingkan. Memindahkan satu hit dari peringkat 2 ke peringkat 10 menurunkan NDCG dari 0.5474 menjadi 0.3684 sementara recall tetap.
- Coverage adalah guardrail, bukan tujuan: sistem rekomendasi acak memiliki coverage sempurna dan akurasi terburuk. Candidate recall adalah batas atas bagi ranker mana pun yang dibangun

di atas set kandidat.
- `evaluate.py` melakukan merge label **left** ke rekomendasi, sehingga positif yang terlewat tetap dihitung dan daftar kosong mendapat skor 0.

Kuis

Q2.1 Sebuah tim mengevaluasi sistem rekomendasi dengan mengacak semua event dan menyisihkan 20% untuk pengujian. Pernyataan manakah yang paling tepat menggambarkan kelemahan utama pengaturan ini?

A. Set training menjadi terlalu kecil untuk mempelajari apa pun.
B. Event test bisa lebih awal daripada event training, sehingga model dapat belajar dari masa depan.
C. Ia menghasilkan nilai precision yang selalu lebih tinggi daripada recall.
D. Ia mencegah pemakaian label bergradasi seperti klik, keranjang, dan pesanan.

Q2.2 Dua sistem rekomendasi mengembalikan sepuluh item yang persis sama untuk seorang pengguna, tetapi sistem X menaruh item yang dipesan di urutan pertama dan sistem Y menaruhnya di urutan terakhir. Metrik manakah yang akan membedakan keduanya?

A. recall@10
B. precision@10
C. NDCG@10
D. coverage katalog

Q2.3 Dalam pengaturan buku ini, pernyataan manakah tentang *candidate recall* yang benar?

A. Ia selalu lebih rendah daripada recall pemeringkatan akhir.
B. Ia adalah batas atas recall yang dapat dicapai ranker mana pun yang dibangun di atas kandidat tersebut.
C. Ia mengukur berapa banyak item berbeda yang direkomendasikan sistem di seluruh pengguna.
D. Ia hanya dapat dinaikkan dengan melatih ranker yang lebih baik.

Q2.4 Pada rumus average precision, mengapa kita membagi dengan jumlah total positif masa depan R dan bukan dengan jumlah hit?

Q2.5 `evaluate.metrics` menggabungkan `labels` dengan `recs` memakai `how="left"` mulai dari label. Jelaskan apa yang akan berubah jika merge dimulai dari `recs` (inner join antara kedua frame).

Q2.6 Perhatikan dua pernyataan dalam fungsi `metrics` yang disertakan, yang membangun `found` dan `dcg` (yang memakai `dropna(subset=["pos"])` dan `np.log2(... + 2)`). Apa yang dibuang oleh `dropna(subset=["pos"])`, dan mengapa konstanta `+ 2` dipakai dan bukan `+ 1`?

Q2.7 Daftar top-3 seorang pengguna memiliki label relevansi `[0, 1, 3]` pada peringkat 1, 2, 3 (gain 0, 1, 7). Satu-satunya positif masa depan pengguna adalah kedua item itu. Hitung DCG@3, IDCG@3, dan NDCG@3, hingga tiga desimal.

Q2.8 Pengguna yang sama memiliki $R = 4$ positif masa depan, dan daftar top-5 mengenai dua di antaranya, pada peringkat 1 dan 3. Hitung precision@5 dan recall@5.

Q2.9 Mengapa perlu melihat coverage dan candidate recall selain NDCG ketika membandingkan dua model pemeringkatan? Berikan satu kegagalan yang akan diungkap masing-masing yang tidak akan diungkap NDCG.

Q2.10 *Praktik langsung.* Pada `book/code/ch02_metrics.py`, ubah sistem rekomendasi acak di bagian akhir sehingga setiap pengguna menerima 50 item tetap yang sama (misalnya 50 id pertama dari `common.available_items()`). Menjadi berapakah seharusnya coverage@20, dan apa artinya bagi hubungan antara coverage dan akurasi?

Bab 3 — Baseline popularitas

Hari Senin setelah tanggal gajian. Dewi, seorang product manager di marketplace sekelas Shopee, punya masalah. Home feed baru akan rilis dua minggu lagi, tim *machine learning* masih berdebat soal *embedding*, dan CEO baru saja mengajukan pertanyaan sederhana: "Kalau kita rilis *tanpa yang canggih-canggih*, home page menampilkan apa?" Jawaban Dewi adalah jawaban yang menjadi titik awal setiap proyek sistem rekomendasi: tampilkan apa yang sedang dibeli orang lain. Satu query SQL, `GROUP BY item_id ORDER BY count(*) DESC LIMIT 20`, dan halaman itu tidak lagi kosong.

Query itu adalah sistem rekomendasi tertua di dunia. Ia juga yang tetap dipertahankan oleh setiap sistem serius: sebagai *fallback* untuk *cold start*, sebagai sumber kandidat, sebagai fitur, dan terutama sebagai **tolok ukur** (*yardstick*). Kalau model canggih tidak bisa mengalahkan "tampilkan produk terlaris", ia bukan model, melainkan cara mahal untuk salah. Di bab ini kita membangun satu keluarga *baseline* popularitas, dari acak murni sampai "populer di kategori yang *Anda* sukai", mengukur semuanya pada data yang sama dengan metrik dari Bab 2, dan mempelajari sedikit matematika (peluruhan eksponensial, *Bayesian shrinkage*) yang akan muncul lagi di sepanjang buku ini.

Yang akan Anda pelajari

- Mengapa *baseline* didahulukan, dan seperti apa *baseline* yang baik.
- Popularitas global, berjendela (*windowed*), dan berpeluruhan waktu (*time-decayed*), termasuk matematika *half-life* di balik peluruhan.
- Rata-rata Bayesian (dan generalisasinya, `_smoothed`, di repositori ini) agar kita tidak terlalu percaya pada sampel kecil.
- Bagaimana sedikit personalisasi, "populer di kategori yang Anda sukai", lebih dari menggandakan ndcg@20 dibanding daftar global terbaik.
- Apa itu bias popularitas (*popularity bias*), mengapa cakupan (*coverage*) runtuh, dan apa yang dilakukan sistem produksi untuk mengatasinya.

Intuisi: rak di depan toko

Bayangkan toko buku di lingkungan rumah Anda. Saat Anda pertama kali masuk, pemiliknya tidak tahu apa-apa tentang Anda, lalu apa yang ia lakukan? Ia menaruh buku terlaris di meja dekat pintu. Ia tidak perlu tahu selera Anda: buku yang dibeli ratusan orang lain adalah taruhan yang cukup baik untuk orang asing. Meja itu adalah **sistem rekomendasi popularitas global** (*global popularity recommender*): satu peringkat, sama untuk semua orang.

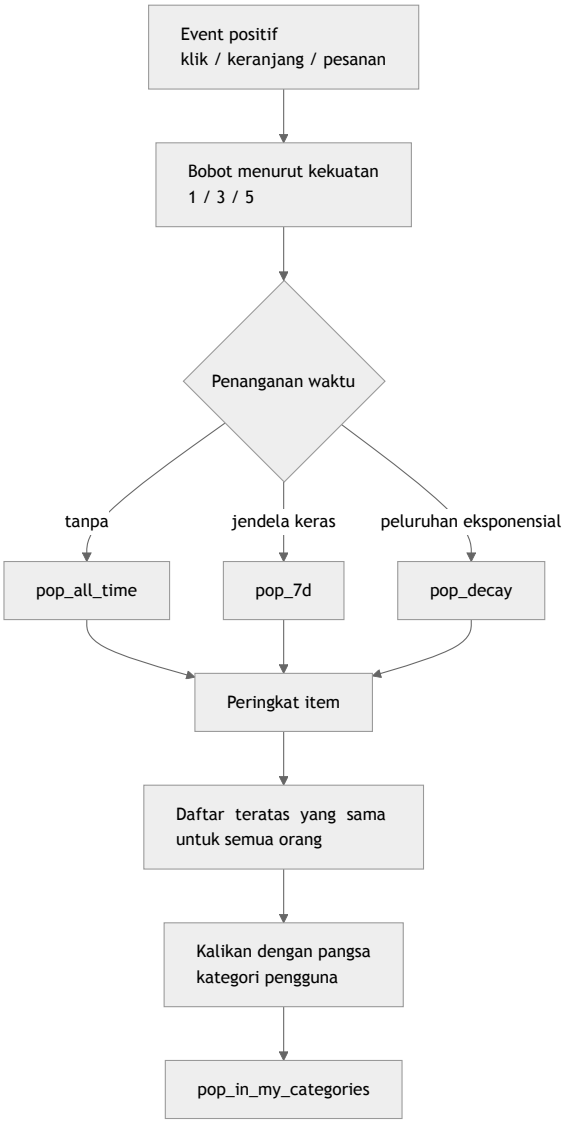
Sekarang perhatikan apa yang ditambahkan seorang pemilik toko yang cerdik seiring waktu.

1. **Ia melupakan secara perlahan.** Buku yang terjual 500 eksemplar tahun lalu tetapi hanya dua minggu lalu tidak lagi ada di meja. Itulah *peluruhan waktu* (*time decay*): penjualan terbaru dihitung lebih besar daripada yang lama. Toko yang tidak pernah lupa akhirnya menghias meja depannya dengan hit musim lalu.
2. **Ia tidak mempercayai sampel kecil.** Buku dengan skor bintang lima sempurna dari satu pengulas, sepupu sang penulis, tidak boleh mengalahkan buku dengan 4.7 bintang dari dua ribu

pembaca. Itulah *shrinkage*, atau rata-rata Bayesian: ketika bukti tipis, bersandarlah pada apa yang kita ketahui tentang buku secara umum.

3. **Ia memperhatikan siapa Anda.** Begitu Anda bilang suka memasak, ia mengantar Anda ke rak buku resep, dan *di sana* ia kembali menunjuk buku terlaris. Ia tidak memakai model canggih tentang selera Anda; ia hanya mengiriskan "apa yang Anda suka" dengan "apa yang populer". Itulah **popularitas yang dipersonalisasi** (*personalised popularity*), dan itu jembatan dari bab ini ke sisa buku.

Setiap langkah itu punya bentuk numerik yang presisi. Tidak satu pun membutuhkan lebih dari `groupby` pandas. Namun, seperti yang akan kita lihat pada hasilnya, langkah ketiga saja menghasilkan skor 2.5 kali lebih tinggi daripada tabel terlaris biasa.



Gambar 3.1: *Setiap baseline popularitas bermula dari event positif berbobot yang sama dan hanya berbeda dalam cara memperlakukan waktu serta apakah ia memperhatikan pengguna.*

Algoritma: lima cara menghitung

Kelima metode di bawah ini pada intinya satu gagasan: *jumlahkan bukti, urutkan, ambil yang teratas.* Bedanya ada pada apa yang dianggap sebagai bukti. Kita menjaga evaluasi tetap identik untuk

semuanya: mereka menilai setiap item yang tersedia untuk setiap pengguna evaluasi, mengambil 50 terbaik, dan dinilai pada minggu setelah titik potong (*cutoff*) (Bab 2 menjelaskan mengapa pemisahan ini jujur).

Langkah 0: Mengubah event menjadi bobot

Marketplace mencatat banyak jenis event, tetapi hanya tiga yang merupakan sinyal *positif*: klik, *add-to-cart*, dan pesanan. Repositori memberi mereka kekuatan di `schema.EVENT_WEIGHT`: klik = 1, add-to-cart = 3, pesanan = 5. Sebuah pembelian lima kali lebih bermakna daripada satu klik. `History.interactions()` adalah tempat hal itu terjadi: ia menyaring log event menjadi ketiga jenis tersebut, menempelkan bobot `w`, dan membawa serta `age_days`, jumlah hari antara event dan titik potong.

Semua yang ada di bab ini membaca dari satu tabel itu. Misalkan w_e adalah bobot event e, dan a_e usianya dalam hari.

Metode 1: Popularitas global

Aturan penilaian paling sederhana adalah menjumlahkan bobot untuk setiap item di seluruh riwayat:

$$\text{pop}(i) = \sum_{e\,:\,\text{item}(e)=i} w_e.$$

Urutkan menurun dan Anda punya daftar terlaris. Kelebihannya: sepele, kokoh, dan tidak butuh informasi pengguna, sehingga bekerja untuk pengunjung yang belum pernah login. Kekurangannya: ia buta terhadap waktu. Item yang enam minggu lalu menjadi hit besar dan hari ini sudah mati tetap bertengger di dekat puncak.

Metode 2: Jendela waktu keras

Perbaikan standarnya adalah menghitung hanya event terbaru. Untuk jendela selebar W hari:

$$\text{pop}_W(i) = \sum_{e\,:\,\text{item}(e)=i,\ a_e<W} w_e.$$

Dengan $W = 7$ inilah daftar "tren minggu ini", dan persis itulah yang dihitung `candidates.item_popularity(hist, 7)`. Jendela adalah alat yang kasar: event berusia 6.9 hari dihitung penuh, event berusia 7.1 hari tidak dihitung sama sekali. Ada pula harga statistiknya. Event yang lebih sedikit berarti hitungan yang lebih berisik: jika sebuah item terjual sepuluh unit seminggu, hitungan mingguannya naik-turun cukup besar hanya karena kebetulan.

Metode 3: Peluruhan waktu eksponensial

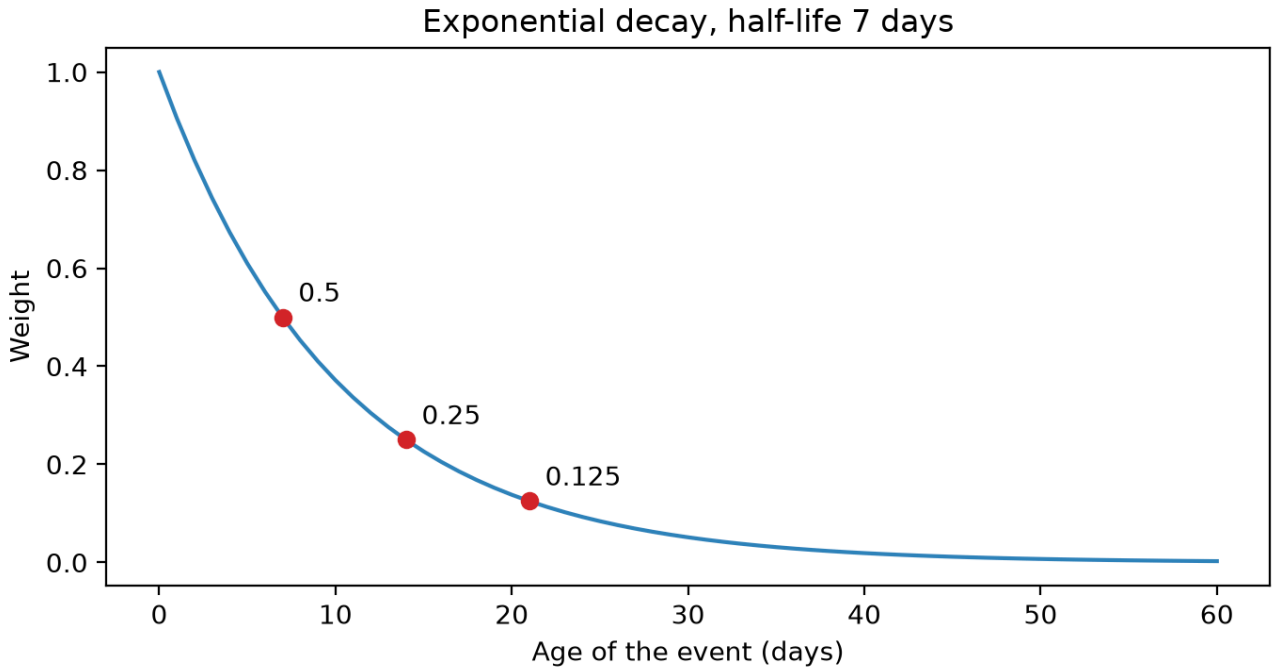
Alih-alih tebing, kita bisa memakai lereng yang mulus. Setiap event kehilangan nilai seiring bertambahnya usia, dan pilihan yang lazim adalah peluruhan eksponensial yang dikendalikan oleh **half-life** (waktu paruh) h: usia ketika sebuah event bernilai separuh dari event yang masih baru.

$$d(a) = 2^{-a/h} = e^{-\lambda a}, \qquad \lambda = \frac{\ln 2}{h}.$$

Kedua bentuk itu adalah fungsi yang sama. Yang pertama lebih mudah dinalar (setiap h hari bobotnya separuh); yang kedua adalah bentuk yang disukai kalkulus. Dengan $h = 7$ hari, yaitu `HALF_LIFE_DAYS` di repositori, kita mendapat $\lambda = 0.693/7 \approx 0.099$ per hari. Popularitas terluruh (*decayed*) adalah

$$\text{pop}_{\text{decay}}(i) = \sum_{e \,:\, \text{item}(e) = i} w_e \cdot 2^{-a_e / h}.$$

Mari kita periksa bahwa bentuknya sesuai klaim kita. Pada usia 0 bobotnya 1. Pada usia 7 bobotnya $2^{-1} = 0.5$. Pada 14 menjadi $2^{-2} = 0.25$, pada 21 menjadi 0.125. Setelah sebulan sekitar 5%, dan event dari awal riwayat nyaris tidak bernilai. Skrip mencetak angka-angka ini, dan grafik memplotnya.



Gambar 3.2: *Peluruhan eksponensial dengan half-life 7 hari: bobot separuh setiap tujuh hari, dari 1 pada usia nol menjadi 0.5, 0.25, dan 0.125 pada usia 7, 14, dan 21.*

Peluruhan eksponensial menjadi bawaan produksi karena tiga alasan. Ia **tak bermemori** (*memoryless*): nilai relatif hanya bergantung pada selang antar event, sehingga penilaian ulang esok hari tidak mengubah peringkat. Ia **bisa diperbarui secara inkremental**: simpan satu angka S per item dan atur $S \leftarrow S \cdot 2^{-(t - t_{\text{last}})/h} + w$ pada setiap event, satu perkalian dan satu penjumlahan. Dan ia punya **satu kenop yang intuitif**: agar sebuah event bernilai 10% setelah 30 hari, selesaikan $2^{-30/h} = 0.1$, sehingga $h = 30 \ln 2 / \ln 10 \approx 9.0$ hari.

Contoh hitungan kecil

Ambil dua item dan riwayat mainan. Item A mendapat tiga klik baru-baru ini: dua kemarin dan satu tiga hari lalu. Item B adalah hit tiga minggu lalu, dengan lima klik, semuanya 20 hari lalu. Semua klik berbobot 1. Angka-angka mainan di bawah ini dibuat-buat untuk ilustrasi; angka sesungguhnya muncul belakangan.

Metode	Item A	Item B	Pemenang
Sepanjang waktu: jumlah bobot	3	5	B
Jendela 7 hari	3	0	A
Peluruhan, $h = 7$	$2 \times 0.906 + 0.743 = 2.55$	$5 \times 2^{-20/7} = 5 \times 0.138 = 0.69$	A

Metode sepanjang waktu memilih B karena ia tidak pernah lupa. Jendela 7 hari memilih A, tetapi hanya karena B jatuh dari tebing; seandainya klik B berusia 6 hari, klik itu akan dihitung penuh. Peluruhan memberi jawaban yang lembut: A menang karena buktinya lebih segar, namun B tidak nol, jadi bila ia terus mendapat klik ia akan naik kembali. Kemulusan inilah alasan peluruhan muncul di separuh pembangkit kandidat (*candidate generator*) di repositori.

Metode 4: Rata-rata Bayesian (tidak mempercayai sampel kecil)

Sejauh ini kita meranking berdasarkan *permintaan*. Sinyal populer lainnya adalah *kualitas*: rata-rata rating bintang. Tetapi rata-rata bisa berbohong saat sampelnya kecil. Produk dengan satu rating bintang 5 punya rata-rata 5.0, lebih tinggi daripada produk terlaris dengan 4.8 dari ribuan pembeli. Kita menginginkan skor yang dimulai dari rata-rata *global* ketika tidak ada bukti, lalu perlahan bergeser ke rata-rata item itu sendiri seiring bukti menumpuk.

Misalkan R adalah rating rata-rata sebuah item, v jumlah ratingnya, C rating rata-rata semua item, dan m **kekuatan prior** (*prior strength*): berapa banyak rating "bukti khayalan" pada rata-rata global yang kita tambahkan. Rata-rata Bayesian adalah

$$\mathrm{WR} = \frac{v}{v+m}\,R + \frac{m}{v+m}\,C.$$

Bacalah sebagai campuran berbobot. Bobot untuk rata-rata item itu sendiri, $v/(v+m)$, bernilai 0 saat $v = 0$ dan menuju 1 seiring v membesar. Bobot untuk rata-rata global adalah sisanya. Saat $v = m$ campurannya 50/50, jadi m adalah "jumlah rating pada titik di mana kita mempercayai item sebesar kita mempercayai prior".

Sudut pandang yang setara membuat gagasan ini bisa dipakai ulang. Kalikan pembilangnya: $R \cdot v$ adalah total bintang, sehingga

$$\mathrm{WR} = \frac{(\text{total bintang}) + m \cdot C}{v+m}.$$

Dengan kata lain: *anggaplah setiap item menerima m rating tambahan, semuanya sama dengan rata-rata global.* Bentuk umum "(pembilang + prior × kekuatan) ÷ (penyebut + kekuatan)" berlaku untuk rasio apa pun, bukan hanya rating bintang. *Click-through rate* adalah klik ÷ impresi, konversi adalah pesanan ÷ klik, tingkat retur adalah retur ÷ pesanan. Repositori membungkusnya dalam satu fungsi kecil, `features._smoothed`, dan memakainya berkali-kali di Bab 9.

Berikut pemeriksaan dengan angka kecil. Sebuah item punya $v = 20$ rating dengan rata-rata $R = 4.8$. Rata-rata global $C = 4.0$ dan kekuatan prior $m = 100$. Maka

$$\text{WR} = \tfrac{20}{120} \cdot 4.8 + \tfrac{100}{120} \cdot 4.0 = 0.8 + 3.33 = 4.13.$$

Item itu tampak bagus (4.8), tetapi dua puluh rating belum banyak, sehingga estimasinya tetap dekat dengan prior. Aritmetika yang sama berlaku pada tingkat konversi: 3 klik dari 10 impresi (30% mentah) dengan CTR global 5% dan kekuatan 50 di-*smooth* menjadi $(3 + 2.5)/60 = 9.2\%$, sedangkan 50 klik dari 1,000 impresi tetap di 5.0%. Sampel kecil dikerutkan keras; sampel besar nyaris tak bergeser.

> **Tahukah kamu?**
>
> **"Top 250" adalah rata-rata Bayesian.** IMDb secara terbuka mendokumentasikan rumus yang dipakainya untuk meranking Top 250 film: weighted rating = (v ÷ (v+m)) × R + (m ÷ (v+m)) × C, dengan R adalah rating rata-rata judul tersebut, v jumlah ratingnya, m jumlah rating minimum agar dapat masuk daftar (saat ini 25,000), dan C rating rata-rata di seluruh judul. IMDb sendiri menyebutnya "a true Bayesian estimate". Itu persis rumus di bagian ini.
>
> Sumber: IMDb, IMDb Help Center, 2026

Metode 5: Populer di kategori yang *Anda* sukai

Empat metode pertama mengembalikan *daftar yang sama untuk setiap pengguna*. Yang terakhir menambahkan dosis personalisasi sekecil yang bisa dibayangkan. Ia memakai dua bahan yang sudah dihitung repositori.

1. **Afinitas kategori** (*category affinity*) $s_{u,c}$: pangsa aktivitas (terluruh) pengguna u, yaitu klik, keranjang, pesanan *dan* pencarian, yang jatuh pada kategori c. Jumlahnya 1 atas semua kategori untuk setiap pengguna, dan dihasilkan oleh `candidates.category_affinity`.
2. **Popularitas ternormalisasi dalam kategori**: untuk item i di kategori $c(i)$,

$$\tilde{p}(i) = \frac{\text{pop}_7(i)}{\max_{j \in c(i)} \text{pop}_7(j)} \in [0, 1].$$

Item terbaik di setiap kategori mendapat 1, sisanya adalah pecahan darinya. Skor akhir mengalikan keduanya:

$$\text{score}(u, i) = s_{u,\,c(i)} \cdot \tilde{p}(i).$$

Mengapa dinormalisasi dalam kategori? Tanpa itu, kategori besar seperti Camilan akan menenggelamkan yang lain. Dengan normalisasi, setiap kategori punya "langit-langit" yang sama, dan afinitas pengguna menentukan *bintang dari kategori mana* yang muncul. Suku tambahan kecil (sepersejuta dari popularitas global) memecah seri dan memberi pengguna *tanpa* riwayat, yang afinitasnya nol semua, daftar `pop_7d` biasa sebagai *fallback*.

Contoh hitungan: Sari menghabiskan 60% aktivitasnya pada Makanan Kucing, 30% pada Mainan Anjing, dan 10% pada Perawatan Kulit. Item makanan kucing terbaik bernilai $0.6 \times 1 = 0.6$, runner-up dengan 80% popularitasnya bernilai 0.48, mainan anjing terbaik 0.3, item perawatan kulit terbaik hanya 0.1. Feed-nya sebagian besar berisi item hewan peliharaan, diurutkan menurut popularitas dalam setiap rak.

> **Tahukah kamu?**
>
> **Bias popularitas bukan bug pada implementasi Anda; ia adalah sifat sistem rekomendasi yang terdokumentasi.** Abdollahpouri dan rekan-rekannya menunjukkan pada dataset film bahwa banyak algoritma rekomendasi memusatkan keluarannya pada item populer, bahkan untuk pengguna yang seleranya kebanyakan niche, dan mereka mengukurnya sebagai "bias disparity" antara apa yang pengguna inginkan dan apa yang mereka dapatkan.
>
> Sumber: Abdollahpouri et al., RMSE Workshop at RecSys, 2019

Penelusuran kode

Repositori sudah berisi komponen-komponen dasarnya. Kita membacanya lebih dulu, lalu skrip yang membungkusnya dalam sebuah evaluasi.

Tabel interaksi

Semuanya bermula dari `History.interactions()`. Ingat bahwa objek `History` adalah log event yang *dipotong pada suatu titik waktu*, sehingga tidak ada yang setelah titik potong dapat bocor masuk (Bab 9 membahas ini lebih dalam).

`recommender/recsys/data.py · History.interactions`

```python
def interactions(self) -> pd.DataFrame:
    """Positive actions (click/cart/order) with their strength weight."""
    pos = self.events[self.events["event_type"].isin(schema.EVENT_WEIGHT)].copy()
    pos["w"] = pos["event_type"].map(schema.EVENT_WEIGHT)
    return pos
```

Ia hanya menyimpan jenis event yang ada di `EVENT_WEIGHT` dan menambahkan kolom bobot `w`. Kolom `age_days` ditempelkan saat `History` dibangun. Semua yang ada di bawah ini adalah reduksi yang berbeda atas baris-baris ini.

Peluruhan waktu dalam satu baris

`recommender/recsys/candidates.py · _decay`

```python
def _decay(age_days: pd.Series) -> pd.Series:
    return np.power(0.5, age_days / HALF_LIFE_DAYS)
```

Itulah seluruh rumus *half-life*: `0.5 ** (age_days / HALF_LIFE_DAYS)`. Karena diterapkan pada seluruh Series pandas, ia bekerja pada jutaan event sekaligus tanpa *loop* Python. Konstanta `HALF_LIFE_DAYS = 7.0` dideklarasikan di bagian atas `candidates.py`. Perhatikan bahwa fungsi ini privat (diawali garis bawah) namun dipakai beberapa *retriever*: `recent` mengalikan bobot setiap interaksi dengannya, dan `category_affinity` memakainya untuk klik maupun pencarian. Kurva yang sama dengan demikian mengungkapkan "berapa lama minat bertahan" di seluruh tahap pertama.

Popularitas berjendela keras

recommender/recsys/candidates.py · item_popularity

```python
def item_popularity(hist: History, window_days: float) -> pd.Series:
    pos = hist.interactions()
    return pos[pos["age_days"] < window_days].groupby("item_id")["w"].sum()
```

Tiga operasi: ambil interaksi positif, simpan baris dengan `age_days < window_days`, jumlahkan bobot per item. Perhatikan `<` yang ketat: event berusia tepat `window_days` *dikecualikan*. Fungsi ini mengembalikan `pd.Series` berindeks `item_id`, dan item tanpa event di dalam jendela sama sekali tidak ada, sehingga pemanggil harus melakukan `fillna(0)`. *Retriever* "trending" (jendela 3 hari), *retriever* populer-per-kategori (7 hari), dan `similar_items` semuanya memanggilnya, sehingga seluruh sistem berbagi *satu* definisi "populer" dan hanya jendelanya yang berubah.

Smoothing

recommender/recsys/features.py · _smoothed

```python
def _smoothed(num: pd.Series, den: pd.Series, prior: float, strength: float) -> pd.Series:
    return (num + prior * strength) / (den + strength)
```

Lagi-lagi satu baris, dan lagi-lagi persis rumusnya: `(num + prior * strength) / (den + strength)`. Petakan rata-rata Bayesian ke dalamnya dengan menetapkan `num` sebagai total bintang, `den` sebagai jumlah rating, `prior` sebagai C, dan `strength` sebagai m. Di tempat lain dalam `features.py` fungsi ini dipanggil dengan kekuatan yang berbeda: 20 pseudo-impresi untuk CTR pengguna, 50 untuk CTR item, 20 untuk konversi, dan 10 untuk tingkat retur. Nilai-nilai ini ditentukan dengan perasaan, bukan dipelajari; semakin kecil penyebut tipikal sebuah statistik, semakin kecil kekuatan yang bisa Anda tanggung.

Skrip eksperimen

Skrip pendamping `book/code/ch03_popularity.py` membangun keenam *scorer* di atas item dan pengguna yang sama lalu mengevaluasinya dengan `common.evaluate` bersama. Dua fungsi pembantu mengurus pembukuan: `per_item` menyelaraskan Series per-item ke larik tetap item yang tersedia (nol untuk item tanpa event), dan `recs_from_scores` mengubah matriks skor (atau satu vektor, disiarkan ke setiap pengguna) menjadi 50 baris teratas per pengguna.

book/code/ch03_popularity.py · per_item

```python
def per_item(values: pd.Series) -> np.ndarray:
    return values.reindex(item_ids).fillna(0.0).to_numpy()
```

book/code/ch03_popularity.py · recs_from_scores

```python
def recs_from_scores(S: np.ndarray, name: str) -> pd.DataFrame:
    """S: (n_users, n_items) or (n_items,) scores -> top-50 rows per user. Ties broken by item
order (stable)."""
    if S.ndim == 1:
        S = np.broadcast_to(S, (len(users), len(item_ids)))
    top = np.argsort(-S, axis=1, kind="stable")[:, :TOP_N]
    return pd.DataFrame({"user_id": np.repeat(users, TOP_N), "item_id": item_ids[top].ravel()})
```

Urutan stabil (*stable sort*) itu penting: banyak item seri di nol, dan pengurutan tak stabil akan membuat hasil antar-run tidak dapat direproduksi.

Sekarang *scorer*-nya sendiri. Tiga yang pertama hampir satu baris (berada di tingkat modul dalam skrip):

book/code/ch03_popularity.py · ch03_scores

```
pop_all = per_item(pos.groupby("item_id")["w"].sum())
scores["pop_all_time"] = pop_all

pop7 = per_item(candidates.item_popularity(hist, 7))
scores["pop_7d"] = pop7

w_decay = pos["w"] * np.power(0.5, pos["age_days"] / HALF_LIFE)
pop_decay = per_item(w_decay.groupby(pos["item_id"]).sum())
scores["pop_decay"] = pop_decay
```

Varian terluruh mengulang rumus `_decay` secara inline alih-alih mengimpor fungsi bantu privat, agar skrip bab ini mandiri. Rata-rata Bayesian bekerja pada kolom `rating_avg` dan `rating_count` katalog:

book/code/ch03_popularity.py · ch03_bayes

```
v, R = items["rating_count"].to_numpy(float), items["rating_avg"].to_numpy(float)
m = v.mean()
C = float((R * v).sum() / v.sum())
bayes = v / (v + m) * R + m / (v + m) * C
```

Dua pilihan layak didiskusikan. Kekuatan prior m ditetapkan sebagai *rata-rata jumlah rating per item*, heuristik yang lazim: pada ukuran sampel rata-rata, bukti dan prior dihitung sama. Rata-rata global C diberi bobot v, sehingga ia adalah rata-rata atas semua *rating*, bukan rata-rata dari rata-rata item. Keduanya murah dihitung dan tidak butuh *tuning*.

Terakhir, versi yang dipersonalisasi. Ia membangun matriks pengguna × kategori `A` dari `category_affinity`, matriks per-item `P` berisi popularitas yang dinormalisasi dalam kategori, lalu mengalikannya:

book/code/ch03_popularity.py · ch03_personal

```
scores["pop_in_my_categories"] = A @ P + 1e-6 * pop7 / max(pop7.max(), 1e-9)
```

`A` punya satu baris per pengguna evaluasi dan satu kolom per kategori; `P` punya satu baris per kategori dan satu kolom per item, dan bernilai bukan nol hanya di tempat item itu termasuk kategori pada baris tersebut. Hasil kali matriks dengan demikian menghasilkan, untuk setiap pengguna dan item, persis $s_{u,c(i)}\,\tilde{p}(i)$. Suku di ujung adalah pemecah seri yang dijelaskan sebelumnya.

Jalankan

Dari direktori `book/`, dengan dependensi terpasang (lihat Kata Pengantar):

```
cd book
uv run python code/ch03_popularity.py
```

Skrip ini juga mencetak lima item teratas dari setiap metode global; kita tampilkan tiga kelompok di sini. Pertama, penyiapan rata-rata Bayesian dan empat baris contoh yang menunjukkan *shrinkage* bekerja:

KELUARAN TERMINAL

```
=== Building the baselines ===
m (mean rating_count) = 105.2, C (global mean rating) = 4.324

Smoothing examples (few ratings are pulled toward C, many ratings are trusted):
      rating_avg  rating_count  bayes
0            4.5           0.0  4.324
892          4.4           0.0  4.324
1840         4.4          34.0  4.343
610          5.0        2718.0  4.975

eval users with no category signal (fall back to pop_7d): 21 of 947
```

Lalu tabel utama: satu baris per metode, dievaluasi pada pengguna yang sama, minggu label yang sama, dan kode metrik yang sama:

KELUARAN TERMINAL

```
=== Results (top 50 per user, same labels and metrics for everyone) ===
                     recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20
coverage@20
method
random                 0.0026     0.0055     0.0090     0.0210           0.0081   0.0035
1.0000
pop_all_time           0.0323     0.0515     0.0914     0.1575           0.1202   0.0527
0.0087
pop_7d                 0.0310     0.0493     0.0786     0.1335           0.1066   0.0475
0.0087
pop_decay              0.0318     0.0519     0.0813     0.1557           0.1091   0.0491
0.0087
bayesian_rating        0.0216     0.0329     0.0648     0.1178           0.0716   0.0325
0.0087
pop_in_my_categories   0.0926     0.1425     0.2173     0.3529           0.3050   0.1302
0.2164
```

Terakhir, bobot peluruhan pada beberapa usia:

KELUARAN TERMINAL

```
=== Decay weights (half-life 7 days) ===
age  0 days -> weight 1.0000
age  1 days -> weight 0.9057
age  3 days -> weight 0.7430
age  7 days -> weight 0.5000
age 14 days -> weight 0.2500
age 21 days -> weight 0.1250
age 30 days -> weight 0.0513
age 53 days -> weight 0.0053
```

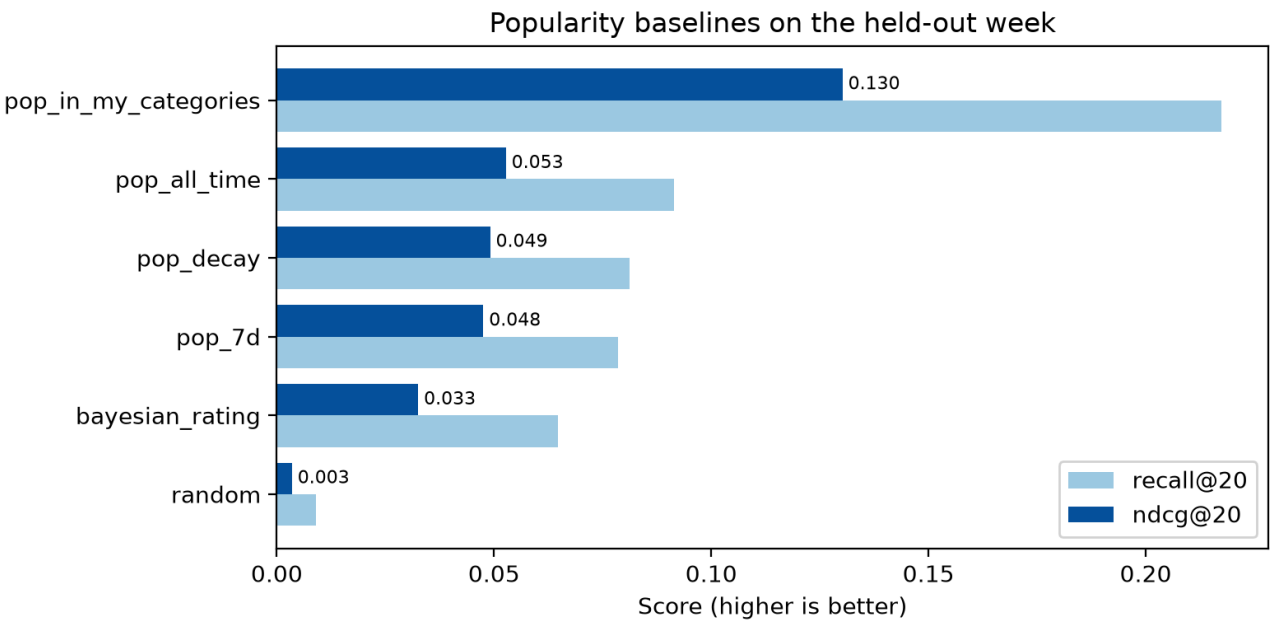
Skrip juga menulis `results/ch03_popularity.json`, yang dibaca skrip figur untuk menggambar dua grafik di bawah.

Hasil

Membaca kurva peluruhan

Kurva peluruhan pada Gambar 3.2 mengonfirmasi aritmetikanya. Bobot yang tercetak sesuai dengannya: 0.9057 setelah satu hari, 0.7430 setelah tiga, tepat 0.5000 setelah tujuh, 0.2500 setelah empat belas, dan 0.1250 setelah dua puluh satu. Pada titik 30 hari bobotnya sudah turun menjadi 0.0513, dan event berusia 53 hari, yang tertua dalam riwayat ini, hanya bernilai 0.0053 dari event baru. Artinya, untuk keperluan peringkat, peluruhan dengan half-life satu minggu berperilaku seperti jendela lunak sekitar dua hingga tiga minggu.

Papan peringkat



Gambar 3.3: *Batang horizontal recall@20 (terang) dan ndcg@20 (gelap) untuk enam baseline popularitas; pop_in_my_categories memimpin dengan selisih lebar dan random berada di dasar.*

Bacalah tabel utama baris demi baris. Kita berfokus pada ndcg@20 (satu angka yang memberi ganjaran bagi penempatan hit di dekat puncak) dan recall@20 (berapa banyak interaksi *held-out* pengguna yang kita temukan dalam 20 slot).

Acak adalah lantainya. `random` mencapai ndcg@20 sebesar 0.0035 dan recall@20 sebesar 0.0090. Tidak ada yang di bab ini boleh berskor lebih rendah, dan apa pun yang begitu pasti rusak. Cakupannya 1.0000: setiap item direkomendasikan kepada seseorang. Kita akan kembali pada keanehan bahwa metode *terburuk* punya cakupan *terbaik*.

Popularitas global sudah sekitar empat belas hingga lima belas kali lebih baik daripada acak dalam ndcg@20. `pop_all_time` berskor ndcg@20 0.0527 dan recall@20 0.0914, `pop_7d` berskor 0.0475 dan 0.0786, dan `pop_decay` berada di antaranya dengan 0.0491 dan 0.0813. Selisih antara acak dan metode masuk akal yang paling murah jauh lebih besar daripada selisih antar metode masuk akal itu sendiri. Sebagian besar nilai "mudah" yang dapat dicapai terletak pada sekadar mengetahui item mana yang diinginkan orang.

Sepanjang waktu sedikit mengalahkan jendela 7 hari, kebalikan dari yang disarankan intuisi. Banyak praktisi mengira "trending" akan menang. Di sini tidak, dan alasannya adalah sifat *data ini*: dalam simulator, setiap item punya kualitas tersembunyi yang tetap, sehingga kualitas tersembunyi sebuah item tidak berubah dan item yang populer sebulan lalu masih populer. Di bawah popularitas yang stabil, riwayat lebih panjang berarti lebih sedikit derau, dan hitungan 53 hari adalah estimasi yang lebih baik atas angka dasar yang sama daripada hitungan satu minggu. Varian peluruhan berada di antara keduanya karena alasan yang sama: jendela efektifnya lebih panjang dari tujuh hari namun tetap memberi bobot lebih kecil pada yang lama.

Pada *cut-off* yang lebih dalam, daftar terluruh jauh lebih dekat dengan daftar sepanjang waktu (recall@50 adalah 0.1557 berbanding 0.1575, sementara jendela keras turun ke 0.1335): lereng yang mulus mempertahankan ekor panjang bukti yang dibuang oleh tebing.

> **Peringatan**
>
> Jangan menganggap "sepanjang waktu mengalahkan 7 hari" sebagai hukum alam. Itu hanya mengatakan bahwa katalog *simulasi* memiliki popularitas yang stasioner. Di marketplace sungguhan, *flash sale*, produk viral, dan kategori musiman (Ramadan, 9.9, 11.11) membuat popularitas bergeser, dan daftar berjendela atau terluruh biasanya menang. Pelajarannya: **ukur pada data Anda sendiri**, dan perlakukan jendela atau half-life sebagai hiperparameter yang disetel pada minggu validasi.

Kualitas bukan permintaan. `bayesian_rating` punya ndcg@20 0.0325 dan recall@20 0.0648: sekitar sembilan kali lebih baik dari acak, tetapi lebih buruk daripada setiap metode berbasis permintaan. Pada baris-baris contoh, item 892 (tertulis 4.4, nol rating) bernilai 4.324, rata-rata global C; item 1840 (4.4, 34 rating) bergeser sedikit ke 4.343; item 610 (5.0, 2718 rating) berakhir di 4.975 karena bukti menenggelamkan prior ($m = 105.2$). *Shrinkage* bekerja sesuai rancangan, tetapi rating bagus bukan alasan untuk mengklik: produk niche berating tinggi hanya penting bagi segelintir orang yang membutuhkannya. Pakai rating Bayesian sebagai fitur atau rak "top rated", bukan sebagai *ranker* home feed.

Patut pula diperhatikan *placeholder* yang bersembunyi di data mentah. Baris pertama yang tercetak punya `rating_avg` 4.5 dengan nol rating: nilai bawaan yang ditulis ke setiap item baru. Tanpa *smoothing*, setiap item tanpa rating akan membawa 4.5 fiktif, yang tampak *lebih baik* daripada kebanyakan produk nyata. Rata-rata Bayesian memperbaikinya secara diam-diam dengan memperlakukan nol rating sebagai "tidak ada informasi".

Sedikit personalisasi berdampak besar. `pop_in_my_categories` mencapai ndcg@20 sebesar 0.1302 dan recall@20 sebesar 0.2173. Dibanding daftar global terbaik, itu faktor sekitar 2.5 pada ndcg, dan recall@50 naik ke 0.3529 berbanding 0.1575. Order recall@20, yang hanya menghitung *pesanan* held-out (sinyal terkuat), adalah 0.3050 berbanding 0.1202. Kita tidak melatih apa pun. Satu-satunya informasi baru adalah "kategori apa yang disentuh pengguna ini", dan itu memindahkan metrik dari "biasa saja" ke "lumayan". Inilah angka terpenting bab ini: *mengetahui bahkan preferensi pengguna yang kasar lebih berarti daripada perbaikan sebanyak apa pun pada daftar global.*

Perhatikan 21 pengguna evaluasi, dari 947, yang sama sekali tidak punya sinyal kategori dan jatuh kembali ke `pop_7d` biasa. Itu sekitar 2% pengguna, dan *fallback* itu berarti mereka tetap mendapat halaman yang masuk akal alih-alih kosong: degradasi yang anggun (*graceful degradation*) yang sama yang diandalkan sistem produksi ketika pengunjung baru tiba.

Jebakan dan praktik di produksi

Popularitas memperkuat dirinya sendiri

Masalah terpenting bukanlah akurasi, melainkan **lingkaran umpan balik** (*feedback loop*). Jika Anda menampilkan dua puluh item yang sama kepada semua orang, orang mengklik kedua puluh item itu (karena hanya itu yang bisa mereka lihat), hitungan klik item-item tersebut bertambah, dan minggu depan mereka makin "populer". Item di luar daftar tidak dapat memperoleh bukti, sebagus apa pun mereka. Para ekonom menyebutnya efek Matthew; orang rekomendasi menyebutnya bias popularitas.

Lingkarannya sederhana: eksposur menghasilkan klik, klik menghasilkan skor, dan skor menentukan eksposur, sehingga item di luar daftar tidak bisa masuk.

Tabel memperlihatkannya. Ketiga metode global semuanya punya coverage@20 sebesar 0.0087: di antara 947 pengguna, mereka merekomendasikan *dua puluh item yang sama*. Metode yang dipersonalisasi, pada 0.2164, menyentuh sekitar dua puluh lima kali lebih banyak item berbeda, dan acak, pada 1.0000, menyentuh semuanya.

Sistem produksi menanganinya berlapis-lapis; tidak ada yang menjadi tugas *baseline*, tetapi semuanya dibangun di atasnya:

- **Personalisasi** (sisa buku ini) mengurangi dominasi satu daftar global.
- **Eksplorasi**: sisihkan beberapa slot untuk item dengan data sedikit, dan tampilkan cukup sering agar bisa diukur. Bab 11 memperkenalkan ε-greedy dan Thompson sampling.
- **Koreksi bias posisi** agar "diklik" tidak keliru dianggap "relevan" padahal item itu hanya ditaruh di posisi 1 (Bab 12).
- **Re-ranking yang terkalibrasi atau didiversifikasi** seperti MMR, agar 20 teratas bukan dua puluh item yang nyaris kembar (Bab 11).
- **Normalisasi terhadap eksposur**: pakai *click-through rate* dengan *smoothing* alih-alih hitungan klik mentah, sehingga item diberi ganjaran atas *konversi per tampilan*, bukan atas seberapa sering ia ditempatkan di halaman. Di sinilah `_smoothed` muncul kembali.

Memilih jendela dan half-life

Baik W maupun h adalah hiperparameter. Terlalu pendek mengejar derau (satu *flash sale* sejam menentukan daftar); terlalu panjang menyajikan berita kemarin. Fesyen, elektronik, dan makanan bergerak dengan kecepatan berbeda, sehingga sebagian marketplace memakai satu half-life per kategori tingkat atas. Cobalah tiga nilai (misalnya 3, 7, 14 hari) pada minggu *held-out*, seperti yang diminta latihan praktik.

Jangan campur skala, hitung pengguna

Skala `pop_7d`, `pop_decay`, dan rating $[1, 5]$ tidak sebanding: normalisasikan ke $[0, 1]$ sebelum dicampur, atau umpankan masing-masing ke *ranker* sebagai fitur terpisah. Dan hitunglah pengguna yang berbeda (atau batasi kontribusi tiap pengguna), karena satu pembeli yang mengklik 40 kali menggelembungkan skor sebesar 40.

Popularitas sebagai sumber kandidat dan sebagai fitur

Di repositori ini *baseline* bertahan dalam tiga peran: **fallback** (*retriever* trending untuk pengguna tanpa riwayat, dan query pesanan-terbaru-teratas milik API Rust ketika Python mati), **sumber kandidat** (`catpop` di Bab 8 hampir sama dengan metode 5), dan **fitur** (kolom popularitas ter-*smooth*, CTR, dan konversi yang dibobot sendiri oleh *ranker*).

Ringkasan

- Sistem rekomendasi popularitas menjumlahkan event positif berbobot per item lalu mengurutkan. Murah, kokoh, tidak butuh data pengguna, dan menetapkan batas yang harus dilampaui setiap metode lain.
- Jendela keras hanya menghitung event terbaru; peluruhan eksponensial dengan half-life h memberi setiap event bobot $2^{-a/h}$, sehingga bukti memudar secara mulus dan skor dapat diperbarui secara inkremental dengan satu perkalian dan satu penjumlahan.
- Rata-rata Bayesian $\frac{v}{v+m}R + \frac{m}{v+m}C$ mengerutkan sampel kecil ke rata-rata global. Repositori menggeneralisasinya sebagai `_smoothed(num, den, prior, strength)` untuk CTR, konversi, dan tingkat retur.
- Pada data ini popularitas sepanjang waktu sedikit mengalahkan jendela 7 hari, karena popularitas simulasi stabil. Jendela mana yang menang bergantung pada seberapa cepat popularitas nyata bergeser, jadi setel pada data *held-out*.
- Meranking berdasarkan rating saja kalah dari meranking berdasarkan permintaan: kualitas tidak sama dengan relevansi.
- Mengalikan afinitas kategori dengan popularitas dalam kategori menaikkan ndcg@20 dari sekitar 0.05 menjadi 0.13 tanpa pelatihan model, argumen paling jelas untuk personalisasi.
- Popularitas global memberi setiap pengguna beberapa item yang sama (cakupan 0.0087) dan menyuapi lingkaran yang memperkuat dirinya sendiri; bab-bab berikutnya menambahkan personalisasi, eksplorasi, dan diversitas untuk melawannya.

Kuis

Q3.1 Mengapa `pop_all_time` sedikit mengalahkan `pop_7d` pada data simulasi buku ini?

A. Jendela 7 hari tidak berisi klik sama sekali.
B. Popularitas item simulasi stabil, sehingga riwayat yang lebih panjang memberi estimasi yang kurang berisik atas besaran yang sama.
C. `pop_7d` memakai minggu evaluasi yang berbeda.
D. Peluruhan tidak diterapkan pada `pop_all_time`, dan peluruhan selalu merugikan.

Q3.2 Sebuah item punya rating katalog 4.4 tetapi `rating_count = 0`. Berapa skor rata-rata Bayesian-nya?

A. 4.4, karena rating item itu sendiri selalu dipakai.
B. 0, karena tidak ada bukti.
C. Rata-rata rating global C, karena dengan $v = 0$ seluruh bobot jatuh pada prior.
D. Rata-rata dari m dan C.

Q3.3 Ketiga metode popularitas global semuanya punya `coverage@20 = 0.0087`. Apa artinya?

A. Mereka gagal untuk 99% pengguna.
B. Setiap pengguna menerima 20 item yang sama, sehingga metode-metode itu hanya merekomendasikan sebagian sangat kecil dari katalog.
C. Mereka merekomendasikan sebuah item sekali setiap 115 permintaan.
D. Metriknya rusak, karena cakupan harus melebihi recall.

Q3.4 Dengan kata-kata Anda sendiri, tunjukkan bagaimana `features._smoothed(num, den, prior, strength)` mereproduksi rata-rata Bayesian dari rating bintang sebuah item. Sebutkan apa yang Anda berikan sebagai masing-masing dari keempat argumennya.

Q3.5 Jelaskan lingkaran umpan balik popularitas dalam dua atau tiga kalimat dan sebutkan satu teknik dari bab ini atau bab berikutnya yang melemahkannya.

Q3.6 Bacalah kode di bawah. Untuk sebuah interaksi dengan `age_days` tepat `7.0`, apakah ia berkontribusi pada hasil? Berapa bobot interaksi yang lolos, dan apa yang akan Anda ubah agar event dari 7 hari terakhir *dan* hari ke-8 ikut dihitung?

`recommender/recsys/candidates.py · item_popularity`

```python
def item_popularity(hist: History, window_days: float) -> pd.Series:
    pos = hist.interactions()
    return pos[pos["age_days"] < window_days].groupby("item_id")["w"].sum()
```

Q3.7 Numerik. Dengan half-life 7 hari, hitung skor popularitas terluruh sebuah item yang memiliki event berikut (bobot dari repositori: klik = 1, add-to-cart = 3, pesanan = 5): satu pesanan 7 hari lalu, satu klik 14 hari lalu, dan satu add-to-cart 3 hari lalu. Gunakan $2^{-3/7} \approx 0.743$.

Q3.8 Numerik. Sebuah item punya $v = 20$ rating dengan rata-rata $R = 4.8$. Rata-rata rating global adalah $C = 4.0$ dan kekuatan prior $m = 100$. Berapa rata-rata Bayesian-nya? Berapa jadinya jika $v = 400$ rating dengan rata-rata yang sama?

Q3.9 Jawaban singkat. Anda ingin sebuah event bernilai tepat 25% dari event baru setelah 10 hari. Half-life berapa yang Anda butuhkan? Lalu jelaskan dalam satu kalimat mengapa half-life yang *lebih kecil* membuat sistem rekomendasi bereaksi lebih cepat tetapi juga lebih berisik.

Q3.10 Latihan praktik. Buka `book/code/ch03_popularity.py` dan ubah `HALF_LIFE = 7.0` menjadi 2.0, lalu menjadi 30.0, dengan menjalankan `uv run python code/ch03_popularity.py` setiap kali (setelahnya pulihkan berkas dan hasilnya dengan `git checkout code/ch03_popularity.py results/`). Bandingkan ndcg@20 pada baris `pop_decay` dengan baris `pop_7d` dan `pop_all_time` setiap kali. Apa yang Anda amati saat half-life menjadi sangat pendek dan sangat panjang, dan apakah itu cocok dengan penjelasan di bagian Hasil?

Bab 4 — Content-based filtering

Seminggu sebelum sale 9.9, Sari, seorang product manager di tim *discovery* marketplace, menatap dashboard dengan satu angka yang tidak ia sukai. Setiap hari beberapa ratus produk baru didaftarkan penjual: dripper kopi keramik dari toko di Bandung, sepasang sneakers dari merek yang belum pernah dicari siapa pun, paket makanan kucing yang baru tayang sepuluh menit lalu. Tak satu pun punya klik. *Baseline* popularitas dari bab sebelumnya tidak bisa melihat produk-produk ini, karena popularitas adalah *riwayat*, dan produk-produk ini belum punya riwayat. Padahal pembeli yang minggu lalu menjelajahi peralatan kopi *pour-over* hampir pasti akan menyukai dripper itu.

Yang dibutuhkan Sari adalah sistem rekomendasi yang melihat apa *wujud* sebuah item, bukan hanya siapa yang pernah menyentuhnya. Jika judul dripper memuat kata "coffee" dan pembeli sering mengeklik barang-barang kopi, keduanya bisa dicocokkan tanpa satu pun interaksi pada produk baru itu. Gagasan inilah yang disebut **content-based filtering** (penyaringan berbasis konten).

Yang akan Anda pelajari

- Cara mengubah teks dan metadata sebuah item menjadi vektor numerik, dan mengapa kata diberi bobot dengan TF-IDF alih-alih sekadar dihitung.
- Apa yang diukur oleh *cosine similarity* (kemiripan kosinus), dan mengapa ia menjadi ukuran jarak bawaan untuk vektor teks yang jarang (*sparse*).
- Cara membangun profil pengguna (*user profile*) sebagai rata-rata berbobot peluruhan waktu dari vektor item yang disentuh pengguna, lalu menilai setiap item terhadap profil itu.
- Bagaimana skrip pendamping `ch04_content.py` bekerja pada marketplace simulasi, dan mengapa ia mengalahkan popularitas global tetapi tidak mengalahkan popularitas yang sadar kategori.
- Di mana metode berbasis konten unggul (*cold start*) dan di mana ia gagal (spesialisasi berlebihan, metadata yang dangkal).

Intuisi: pustakawan yang tak pernah melihat para pembaca

Bayangkan seorang pustakawan yang tidak boleh melihat catatan peminjaman. Ia tidak bisa berkata "orang yang meminjam buku ini juga meminjam buku itu". Yang boleh ia lakukan hanyalah membaca sampul: judul, rak subjek, pengarang. Ketika Anda menyerahkan tiga novel yang Anda sukai, ia berjalan ke rak, mencari buku yang sampulnya *terdengar mirip* dengan ketiga buku tadi, lalu menyerahkannya kepada Anda.

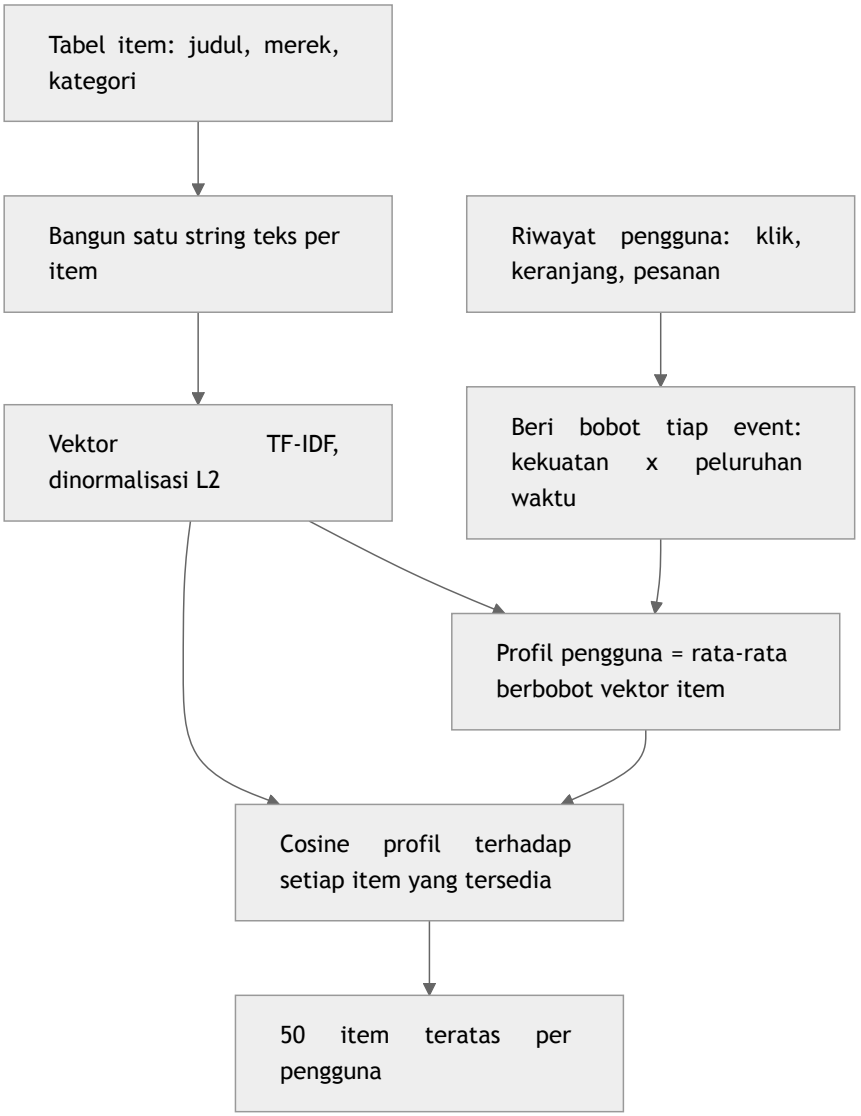
Pustakawan itu adalah *content-based recommender*. Ia punya dua keahlian:

1. Ia bisa menggambarkan setiap buku sebagai daftar singkat kata deskriptif (**representasi item**, *item representation*).
2. Ia bisa merangkum selera Anda sebagai daftar kata yang sejenis (**profil pengguna**, *user profile*), lalu membandingkan kedua daftar itu.

Perhatikan apa yang tidak ia butuhkan: pembaca lain. Itulah kekuatan sekaligus kelemahannya: buku yang baru tiba pagi ini langsung bisa direkomendasikan, tetapi ia akan terus menggiring Anda ke rak kriminal selamanya.

Di marketplace kita, "sampul" itu adalah kolom `title`, `brand`, dan kategori (beserta kategori induknya) pada tabel `items`.

Pipeline bab ini tampak seperti berikut:



Gambar 4.1: *Pipeline content-based. Vektor item dihitung sekali; profil pengguna dihitung ulang dari riwayat terbaru.*

Semua langkah di sini murah, deterministik, dan dapat dijelaskan. Itulah sebabnya penilaian berbasis konten tetap menjadi andalan untuk widget "item serupa" dan *fallback* saat *cold start*.

Algoritma

Langkah 1: representasi item sebagai teks

Cara paling sederhana, dan ternyata cukup efektif, untuk merepresentasikan item adalah merekatkan field deskriptifnya menjadi satu string dan memperlakukan string itu sebagai dokumen pendek. Di

skrip pendamping ini hanya satu baris:

book/code/ch04_content.py · item_text

```
items["text"] = items["title"] + " " + items["category_name"] + " " + items["brand"] + " " +
items["parent_name"]
```

Item berjudul "Volt Skincare Lokal" pada kategori "Skincare" dengan merek "Volt" dan kategori induk "Beauty" menjadi dokumen `Volt Skincare Lokal Skincare Volt Beauty`. Perhatikan bahwa kata kategori dan merek sengaja diulang: judul di simulator ini pendek, dan mengulang field terstruktur membuatnya dihitung lebih dari sekali.

Selanjutnya kita ubah tiap dokumen menjadi vektor. Cara naifnya adalah **bag of words** (kantong kata): hitung berapa kali setiap kata dalam kosakata muncul.

Langkah 2: pembobotan TF-IDF

Bag-of-words punya cacat yang jelas. Kata "home" muncul di setiap item kategori "Home & Living" (dan juga di teks kategori induk setiap item itu), sehingga hampir tidak memberi tahu *item rumah tangga yang mana* yang diinginkan pembeli, sedangkan kata "sneakers" spesifik dan jauh lebih informatif. Kita ingin kata langka dihitung lebih berat daripada kata umum.

TF-IDF (*term frequency–inverse document frequency*) melakukan persis itu. Untuk suatu term t pada dokumen d dari koleksi N dokumen:

$$\mathrm{tfidf}(t, d) = \mathrm{tf}(t, d) \times \mathrm{idf}(t)$$

dengan $\mathrm{tf}(t, d)$ adalah berapa kali t muncul di d, dan *inverse document frequency* menurunkan bobot term yang muncul di banyak dokumen. Versi yang dipakai `TfidfVectorizer` milik scikit-learn, yang digunakan skrip kita, adalah versi *smoothed*:

$$\mathrm{idf}(t) = \ln \frac{1 + N}{1 + \mathrm{df}(t)} + 1$$

dengan $\mathrm{df}(t)$ adalah jumlah dokumen yang memuat t. Suku "+1" menjaga logaritma tetap terhingga dan menjamin bahwa term yang ada di setiap dokumen pun tetap memiliki bobot positif kecil.

Terakhir, setiap vektor dokumen $\mathbf{x}_d$ di-**normalisasi L2** hingga panjangnya satu:

$$\hat{\mathbf{x}}_d = \frac{\mathbf{x}_d}{\|\mathbf{x}_d\|_2}$$

sehingga judul yang panjang tidak otomatis tampak lebih penting daripada judul yang pendek.

Langkah 3: cosine similarity

Untuk membandingkan dua vektor, kita mengukur sudut di antara keduanya, bukan jarak antar ujungnya:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$

Karena vektor TF-IDF tidak punya elemen negatif, nilai kosinus berada antara 0 (tidak ada term yang sama sama sekali) dan 1 (arah identik). Ketika kedua vektor sudah berpanjang satu, seperti vektor kita setelah langkah 2, penyebutnya bernilai 1 dan **cosine similarity tak lain adalah *dot product***. Itulah sebabnya skrip dapat menghitung kemiripan semua pasangan item dengan satu perkalian matriks *sparse*, `X @ X.T`.

Mengapa sudut, bukan jarak Euclidean? Teks yang diulang dua kali menghasilkan vektor dua kali lebih panjang tetapi menunjuk ke arah yang sama; kosinus dengan tepat menyebutnya identik.

Langkah 4: profil pengguna

Item kini hidup dalam ruang vektor. Pengguna digambarkan oleh apa yang ia lakukan terhadap item, jadi profil yang paling wajar adalah **rata-rata berbobot dari vektor item yang ia interaksikan**. Dua jenis bobot berlaku pada setiap event:

- **Kekuatan.** Di repo ini, klik bernilai 1, *add-to-cart* 3, dan pesanan 5 (`EVENT_WEIGHT` di `recommender/recsys/schema.py`). Membeli sesuatu adalah bukti selera yang lebih kuat daripada sekadar melihatnya sekilas.
- **Kebaruan (*recency*).** Selera bergeser; klik minggu lalu lebih penting daripada klik bulan lalu. Kita menerapkan peluruhan eksponensial dengan waktu paruh (*half-life*) h sebesar 7 hari: event berumur a hari dikalikan $2^{-a/h}$.

Dengan w_e sebagai kekuatan event e, a_e sebagai umurnya, dan $\hat{\mathbf{x}}_{i(e)}$ sebagai vektor item yang disentuhnya, profilnya adalah

$$\mathbf{p}_u = \frac{\sum_{e \in E_u} w_e \, 2^{-a_e/h} \, \hat{\mathbf{x}}_{i(e)}}{\sum_{e \in E_u} w_e \, 2^{-a_e/h}}, \qquad \hat{\mathbf{p}}_u = \frac{\mathbf{p}_u}{\|\mathbf{p}_u\|_2}.$$

Penyebut menjadikannya rata-rata yang sebenarnya, sehingga pengguna dengan 100 event tidak mendapat vektor lebih besar daripada pengguna dengan 5 event. Fungsi peluruhannya sama dengan yang sudah dipakai repo untuk *retriever* "baru dilihat":

`recommender/recsys/candidates.py · _decay`

```python
def _decay(age_days: pd.Series) -> pd.Series:
    return np.power(0.5, age_days / HALF_LIFE_DAYS)
```

Langkah 5: penilaian dan pemeringkatan

Untuk setiap item j yang tersedia, skor bagi pengguna u adalah

$$\text{score}(u, j) = \hat{\mathbf{p}}_u \cdot \hat{\mathbf{x}}_j.$$

Urutkan menurun dan ambil 50 teratas per pengguna. Pengguna tanpa riwayat memiliki profil berisi nol semua; skrip memutus kesamaan skor mereka dengan suku popularitas kecil sehingga mereka mendapat daftar `pop_7d`.

Contoh hitungan manual

Mari jalankan seluruh prosesnya pada katalog mini berisi empat item, cukup kecil untuk dicek dengan kalkulator saku. Kosakatanya empat kata dan tiap item hanya memiliki beberapa di antaranya:

Item	Teks	aurora	coffee	sneakers	volt
A	`aurora coffee`	1	1	0	0
B	`volt coffee`	0	1	0	1
C	`aurora sneakers`	1	0	1	0
D	`volt sneakers sneakers`	0	0	2	1

Setiap kata muncul di tepat dua dari empat item, sehingga $\text{idf} = \ln(5/3) + 1 \approx 1.511$ untuk semuanya dan saling meniadakan setelah normalisasi. Setelah normalisasi L2:

- A $= (0.707,\ 0.707,\ 0,\ 0)$ dan B $= (0,\ 0.707,\ 0,\ 0.707)$ dan C $= (0.707,\ 0,\ 0.707,\ 0)$ (koordinat berurutan: aurora, coffee, sneakers, volt).
- D memiliki TF-IDF mentah $(0,\ 0,\ 2 \cdot 1.511,\ 1.511)$, panjang $1.511\sqrt{5}$, sehingga $\hat{\mathbf{x}}_D = (0,\ 0,\ 0.894,\ 0.447)$. Kata yang diulang dua kali mendominasi vektor.

Kemiripan kosinus langsung menyusul. A dan B hanya berbagi "coffee": $0.707 \times 0.707 = 0.5$. A dan C hanya berbagi "aurora": juga 0.5. A dan D tidak berbagi apa pun: 0. C dan D berbagi "sneakers": $0.707 \times 0.894 = 0.632$.

Sekarang seorang pengguna mengeklik item A hari ini (kekuatan 1, umur 0) dan memesan item C tujuh hari lalu (kekuatan 5, peluruhan $2^{-7/7} = 0.5$). Bobot efektifnya $1 \times 1 = 1$ dan $5 \times 0.5 = 2.5$, jumlah 3.5. Profilnya adalah

$$\mathbf{p} = \frac{1 \cdot \mathbf{x}_A + 2.5 \cdot \mathbf{x}_C}{3.5} = (0.707,\ 0.202,\ 0.505,\ 0),$$

dan setelah normalisasi $\hat{\mathbf{p}} = (0.793,\ 0.226,\ 0.566,\ 0)$. Skornya: A 0.721, B 0.160, C 0.961, dan D 0.506. C menang karena memuat kedua kata terkuat milik pengguna; D mengungguli B karena sinyal sneakers kuat sedangkan B hanya berbagi sinyal "coffee" yang lemah. (A dan C sudah pernah dilihat; sistem produksi menyaringnya, sedangkan *baseline* sederhana ini tidak.)

Penelusuran kode

Skrip lengkapnya adalah `book/code/ch04_content.py`; potongan di bawah diambil langsung darinya dan dijalankan dari atas ke bawah.

Menyiapkan teks item

Skrip dimulai dengan memuat data simulasi bersama lewat `common.py`, sehingga setiap algoritma di buku ini melihat riwayat, *cutoff*, dan pengguna evaluasi yang persis sama. Lalu ia menggabungkan tiap item dengan nama kategori dan kategori induknya, dan membangun teksnya:

book/code/ch04_content.py · data_prep

```
hist = common.history()
avail = common.available_items()
users = common.eval_users()
cats = hist.categories.set_index("category_id")

items = hist.items.copy()
items["category_name"] = items["category_id"].map(cats["name"])
items["parent_name"] = items["category_id"].map(cats["parent_name"])
```

Kemudian kolom teks dibangun:

book/code/ch04_content.py · item_text

```
items["text"] = items["title"] + " " + items["category_name"] + " " + items["brand"] + " " +
items["parent_name"]
```

Perhatikan bahwa `hist` adalah `History.at(tables, CUTOFF)`: hanya data yang terlihat pada *cutoff*. Item yang didaftarkan setelah *cutoff* tidak ada bagi model ini, sehingga evaluasi tetap jujur.

Melatih TF-IDF

book/code/ch04_content.py · tfidf

```
vec = TfidfVectorizer(lowercase=True)
X = vec.fit_transform(items["text"])  # rows are L2-normalised already
print(f"\nitems: {X.shape[0]}, vocabulary: {X.shape[1]} terms, non-zeros per item: {X.nnz /
X.shape[0]:.1f}")
idf = pd.Series(vec.idf_, index=vec.get_feature_names_out()).sort_values()
print("\nLowest IDF (common, uninformative):")
print(idf.head(5).round(3).to_string())
print("Highest IDF (rare, discriminative):")
print(idf.tail(5).round(3).to_string())
item_row = {it: i for i, it in enumerate(items["item_id"])}
```

`fit_transform` mengerjakan tiga hal sekaligus: mempelajari kosakata dan *document frequency*, menghitung IDF tiap kata, dan mengembalikan matriks *sparse* `X` dengan satu baris ternormalisasi L2 per item. `item_row` memetakan id item ke nomor barisnya agar kita bisa mencari item nanti. Menjaga matriks tetap *sparse* itu penting: dengan kosakata nyata puluhan ribu kata, matriks padat tak akan terkelola, padahal tiap item hanya menyentuh segelintir kolom.

Membangun profil dengan aljabar sparse

Inilah inti skrip, dan ia menghindari *loop* Python atas pengguna:

book/code/ch04_content.py · profile_build

```
pos = hist.interactions()
pos = pos[pos["item_id"].isin(item_row)]
pos = pos.assign(s=pos["w"] * np.power(0.5, pos["age_days"] / HALF_LIFE))
uidx = {u: i for i, u in enumerate(users)}
p = pos[pos["user_id"].isin(uidx)]
p = p.groupby(["user_id", "item_id"], as_index=False)["s"].sum()
W = sp.csr_matrix((p["s"], (p["user_id"].map(uidx), p["item_id"].map(item_row))), shape=
(len(users), X.shape[0]))
wsum = np.asarray(W.sum(axis=1)).ravel()
```

Baris terakhir dalam pembangunan profil adalah langkah normalisasi:

book/code/ch04_content.py · profile_norm

```
profile = normalize(sp.diags(1 / np.where(wsum > 0, wsum, 1)) @ W @ X)
```

`hist.interactions()` mengembalikan event klik, *add-to-cart*, dan pesanan dengan kolom `w` yang memuat bobot kekuatan:

recommender/recsys/data.py · History.interactions

```
def interactions(self) -> pd.DataFrame:
    """Positive actions (click/cart/order) with their strength weight."""
    pos = self.events[self.events["event_type"].isin(schema.EVENT_WEIGHT)].copy()
    pos["w"] = pos["event_type"].map(schema.EVENT_WEIGHT)
    return pos
```

Skrip mengalikan `w` dengan faktor peluruhan untuk mendapatkan bobot efektif `s`, lalu menjumlahkan bobot efektif per pasangan (pengguna, item). Jumlah-jumlah ini mengisi **matriks bobot pengguna × item** `W` yang *sparse*. Setelah itu rumus profil menjadi aljabar matriks murni:

- `W @ X` menghitung, untuk setiap pengguna, $\sum_e s_e \, \hat{\mathbf{x}}_{i(e)}$, yaitu jumlah berbobot vektor item.
- `sp.diags(1 / wsum)` membagi baris tiap pengguna dengan total bobotnya, menghasilkan rata-rata berbobot.
- `normalize(...)` menskalakan tiap profil ke panjang satu, sehingga *dot product* akhir benar-benar kosinus.
- `np.where(wsum > 0, wsum, 1)` adalah pengaman: pengguna tanpa event positif memiliki `wsum == 0`, dan pembagian dengan nol akan menghasilkan `inf`. Mengganti nol dengan satu membiarkan barisnya yang berisi nol tetap nol.

Penilaian, pemutus kesamaan, dan evaluasi

book/code/ch04_content.py · scoring

```
item_ids = avail["item_id"].to_numpy()
Xa = X[[item_row[i] for i in item_ids]]
S = np.asarray((profile @ Xa.T).todense())
pop7 = candidates.item_popularity(hist, 7).reindex(item_ids).fillna(0).to_numpy()
S = S + 1e-6 * pop7 / max(pop7.max(), 1e-9)  # cold users: tie-break by popularity
top = np.argsort(-S, axis=1, kind="stable")[:, :TOP_N]
```

`profile @ Xa.T` menghasilkan seluruh matriks skor pengguna × item-tersedia dalam satu perkalian. Hanya item yang ada stoknya pada *cutoff* (`avail`) yang dinilai. Pemutus kesamaan berbasis popularitas diskalakan ke 10^{-6} sehingga hanya memisahkan skor yang memang sama: ia menentukan urutan di antara pengguna *cold* (nol semua) dan di antara item yang nyaris kembar, tetapi tidak pernah mengalahkan perbedaan konten yang nyata. `argsort` terakhir memilih 50 teratas per pengguna, dan `common.evaluate` menghitung metrik yang persis sama dengan yang dipakai semua bab lain.

Jalankan

Dari direktori `book/`:

```
uv run python code/ch04_content.py
```

Skrip pertama-tama mencetak statistik kosakata dan nilai IDF ekstrem:

KELUARAN TERMINAL

```
items: 2495, vocabulary: 64 terms, non-zeros per item: 4.8

Lowest IDF (common, uninformative):
living         2.861
home           2.861
baby           2.896
mother         2.896
electronics    3.008
Highest IDF (rare, discriminative):
men            4.518
smartwatches   4.532
skincare       4.546
shampoo        4.648
sneakers       4.695
```

Dokumennya pendek (4.8 term tak-nol per item) dan kosakatanya hanya 64 term, karena simulator membangun judul dari templat; marketplace sungguhan memiliki jutaan kata, nomor model, dan salah ketik. Peringkat IDF berperilaku seperti dijanjikan teori: "living" dan "home" memiliki bobot terendah, sekitar 2.86, sedangkan "sneakers", "shampoo", dan "skincare" tertinggi, sekitar 4.5 hingga 4.7.

Hasil

Seperti apa profilnya

KELUARAN TERMINAL

```
Strongest profile terms for user 894:
diapers    0.809
baby       0.301
mother     0.301
bamboo     0.148
aurora     0.145
garuda     0.117
```

Dari 947 pengguna evaluasi, 919 memiliki profil yang dapat dipakai; 28 tidak punya riwayat positif dan jatuh ke pemutus kesamaan berbasis popularitas. Untuk pengguna 894, term terkuatnya adalah "diapers" (0.809), diikuti "baby" dan "mother" (keduanya 0.301): orang tua yang membeli perlengkapan bayi, dan profilnya langsung bisa dibaca manusia.

Seberapa baik peringkatnya?

KELUARAN TERMINAL

```
=== Results (cosine(profile, item), top 50 per user) ===
               recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20  coverage@20
method
content_tfidf    0.0589     0.0837     0.1378     0.2416           0.1834   0.0909       0.9892
```

Bab 3 melaporkan `pop_7d` sebesar 0.0475 dan `pop_in_my_categories` sebesar 0.1302 pada label yang sama.

Tiga kesimpulan:

1. **Content-based mengalahkan popularitas global.** NDCG@20 `content_tfidf` adalah 0.0909 berbanding 0.0475 untuk `pop_7d`, hampir dua kali lipat. Selera pribadi, bahkan hanya dari kata kategori dan merek, membawa informasi nyata.
2. **Ia kalah dari popularitas yang sadar kategori.** `pop_in_my_categories` mencetak 0.1302, jelas di atas 0.0909 milik kita. Mengapa? Kedua metode memanfaatkan fakta yang sama: pengguna berbelanja di dalam kategori. Tetapi `pop_in_my_categories` juga memeringkat *di dalam* kategori berdasarkan popularitas, yang memisahkan item bagus dari yang kurang. Vektor TF-IDF kita hanya memuat kata kategori dan merek, sehingga setiap item dalam satu kategori tampak nyaris sama, dan pilihan di antaranya seperti lempar koin (hanya ditentukan oleh pemutus kesamaan popularitas 10^{-6}).
3. **Cakupan (*coverage*) sangat baik.** `coverage@20` sebesar 0.9892 berarti hampir seluruh katalog yang ada stoknya muncul di 20 teratas seseorang, dibandingkan 0.0087 untuk popularitas, yang merekomendasikan beberapa lusin item yang sama kepada semua orang. Metode berbasis konten adalah penawar alami efek "yang kaya makin kaya" yang kita temui di bab 3. Angka cakupan ini sedikit terdongkrak oleh pilihan yang nyaris acak di antara skor yang hampir imbang, tetapi arahnya nyata.

Tetangga terdekat dan pelajaran "cosine = 1.0"

KELUARAN TERMINAL

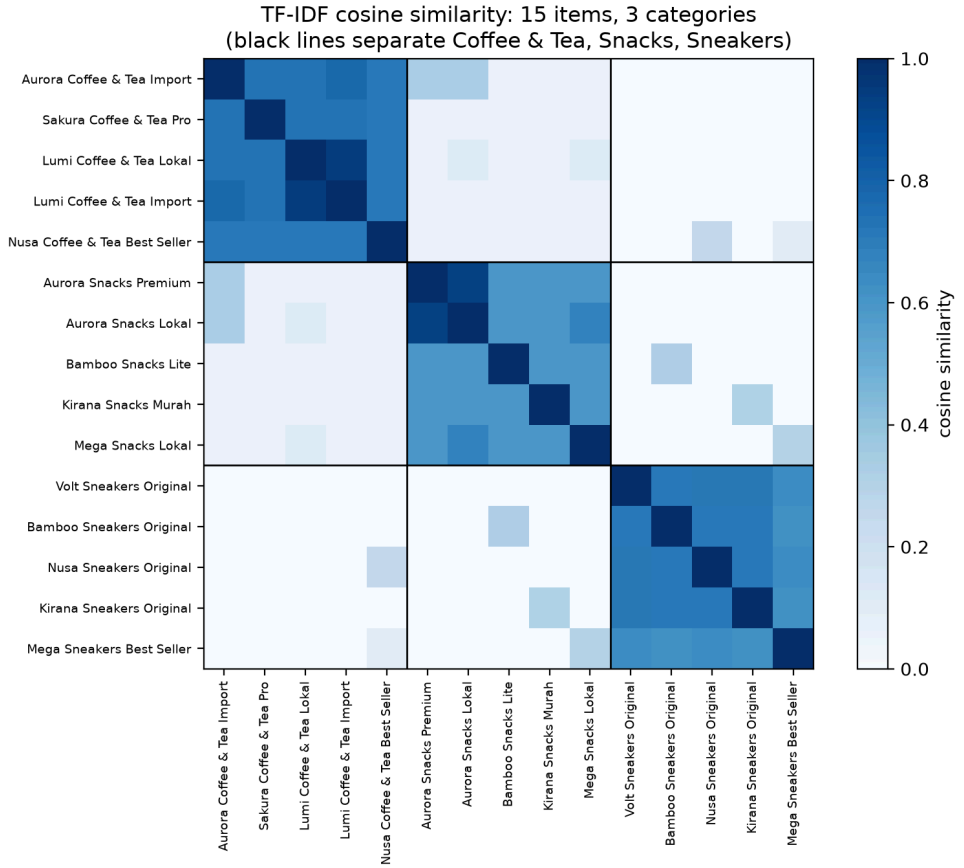
```
=== Five nearest items for three example items ===

Query: item 12 'Aurora Coffee & Tea Import' (Coffee & Tea)
   1.000  item 2399  Aurora Coffee & Tea Import  [Coffee & Tea]
   0.953  item 449   Aurora Coffee & Tea Lokal  [Coffee & Tea]
   0.953  item 1016  Aurora Coffee & Tea Lokal  [Coffee & Tea]
   0.952  item 1936  Aurora Coffee & Tea Lite  [Coffee & Tea]
   0.952  item 1476  Aurora Coffee & Tea Premium  [Coffee & Tea]
```

Lihat kueri pertama. "Aurora Coffee & Tea Import" memiliki tetangga terdekat dengan kosinus **1.000**: item lain (id 2399) dengan judul dan kategori yang persis sama. Yang berikutnya, 0.953, hanya berbeda pada akhiran "Lokal", "Lite", atau "Premium", yang nyaris tidak menyumbang apa-apa.

Ini poin pembelajaran, bukan bug. Teks identik menghasilkan vektor identik, sehingga tak ada metode berbasis konten yang dapat membedakan item-item itu; *recommender* yang hanya berbasis konten akan dengan senang hati mengisi 5 slot teratas dengan salinan produk yang sama. Membedakannya memerlukan sinyal *perilaku* seperti harga, rating, dan popularitas.

Heat map berikut menunjukkan kosinus antara 15 item, lima dari masing-masing tiga kategori:



Gambar 4.2: *Kemiripan kosinus antara 15 item dari tiga kategori (Coffee & Tea, Snacks, Sneakers). Garis hitam memisahkan kategori. Blok pada diagonal terang; semua yang di luar diagonal mendekati nol.*

Item dalam kategori yang sama mirip; item dari kategori berbeda tidak. Angka-angkanya membenarkan:

KELUARAN TERMINAL

```
=== Mean cosine similarity within / across the three heat-map categories ===
Coffee & Tea    0.752    0.084    0.014
Snacks          0.084    0.633    0.037
Sneakers        0.014    0.037    0.681
```

Baris dan kolomnya adalah Coffee & Tea, Snacks, dan Sneakers. Rata-rata kemiripan antara dua item berbeda dalam kategori yang sama adalah 0.752, 0.633, dan 0.681 pada diagonal; di luar diagonal paling tinggi 0.084. Kotak-kotak samar di luar diagonal berasal dari kata merek yang sama seperti "Aurora".

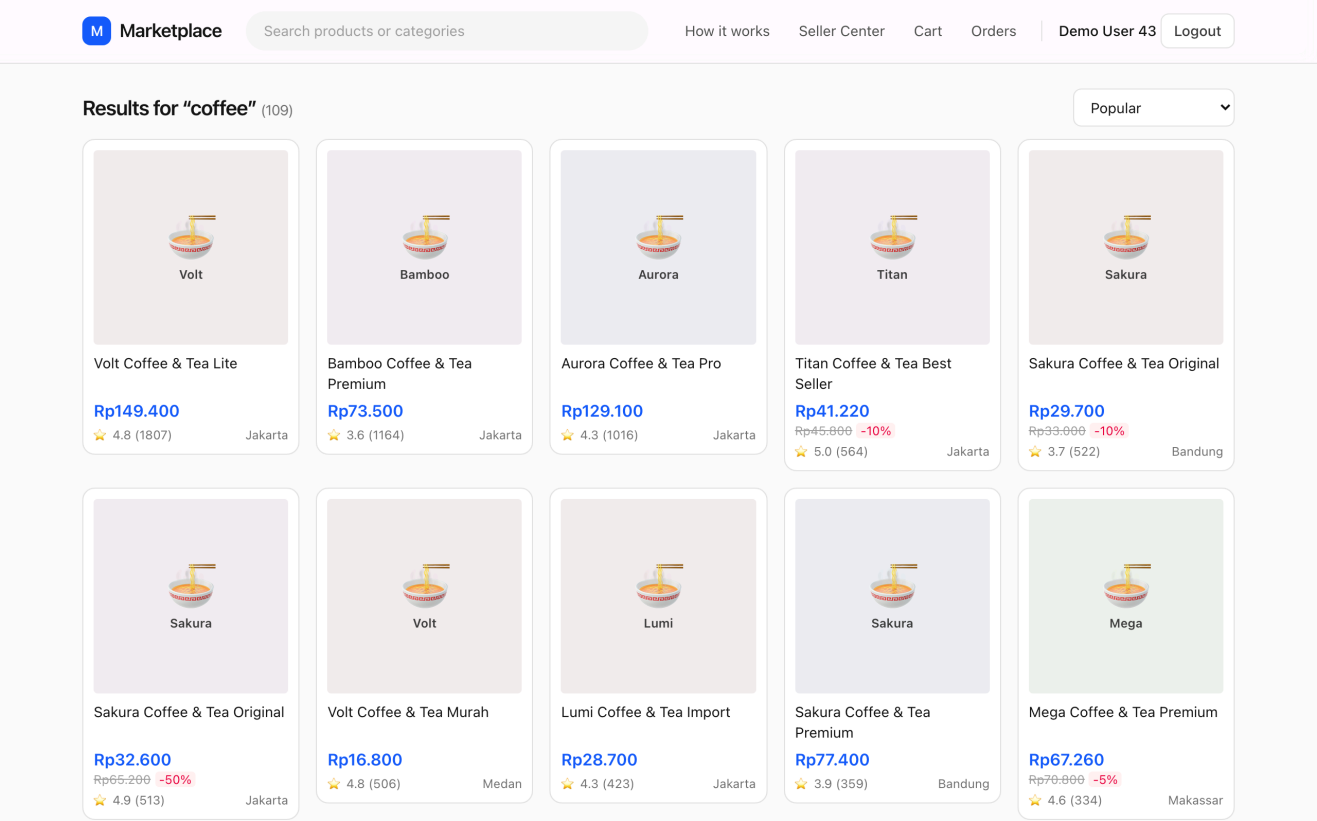
Tahukah kamu?

TF-IDF berusia lebih dari lima puluh tahun. Gagasan memberi bobot pada sebuah term dengan kebalikan dari jumlah dokumen yang memuatnya diterbitkan oleh Karen Spärck Jones pada 1972, dengan judul "a statistical interpretation of term specificity". Sejak itu ia dipakai di mesin pencari dan sistem rekomendasi, dan hingga kini masih menjadi *baseline* yang kuat.

Sumber: Spärck Jones, Journal of Documentation, 1972

> **Tahukah kamu?**
>
> **"Music Genome" milik Pandora adalah content-based filtering yang dikerjakan dengan tangan.** Para pendiri Pandora menghimpun tim musikolog yang menganalisis dan mengklasifikasikan lagu berdasarkan sekitar 400 atribut, seperti gaya vokal atau ritme, lalu memakai vektor atribut itu untuk menemukan lagu yang mirip dengan yang disukai pendengar, alih-alih mengandalkan riwayat pendengar lain.
>
> Sumber: Westergren, NPR interview, 2011

Gagasan yang sama di dalam aplikasi



Gambar 4.3: *Halaman pencarian marketplace untuk kueri "coffee". Hasil dicocokkan pada teks judul dan kategori lalu diurutkan berdasarkan popularitas.*

Aplikasi ini sudah memuat komponen content-based yang sangat kecil: *backend* mencocokkan kueri yang diketik dengan judul produk dan nama kategori (filter `ILIKE` di `backend/src/catalog.rs`), sedangkan urutan "Popular" sama sekali tidak berbasis konten. Pendekatan profil di bab ini mengganti kueri yang diketik dengan *kueri yang dibangun dari perilaku pengguna itu sendiri*.

Jebakan dan praktik di produksi

Spesialisasi berlebihan (*over-specialisation*)

Pustakawan yang hanya membaca sampul akan terus merekomendasikan hal yang sama. Profil pengguna 894 kita didominasi "diapers", sehingga rekomendasinya akan berkisar pada popok; jika pengguna itu berikutnya butuh *stroller*, tak ada kesamaan kata yang menyarankannya. Sistem ini tidak punya **serendipity** (kejutan yang menyenangkan).

Mitigasi: padukan dengan sinyal kolaboratif seperti *co-visitation* (bab 5), *re-rank* demi keberagaman dengan MMR dan batas per kategori (bab 11), dan sisihkan beberapa slot untuk eksplorasi.

Metadata dangkal, teks berisik

Metode berbasis konten hanya sebaik kontennya. Di sini judul berasal dari templat; judul penjual sungguhan dijejali kata kunci ("MURAH!!! Best Seller Sneakers Pria Wanita Original 100%"), salah mengeja merek, dan mencampur bahasa. Sistem produksi membersihkan judul, mengekstrak atribut terstruktur, dan makin sering mengganti TF-IDF dengan *embedding* teks dan gambar hasil pembelajaran (bab 7).

Kekuatan pada cold start

Item yang baru saja didaftarkan tidak punya klik, sehingga popularitas, *co-visitation*, dan faktorisasi matriks (bab 3, 5, 6) tidak bisa merekomendasikannya. Tetapi judul, merek, dan kategorinya ada sejak detik pertama, sehingga penilai berbasis konten dapat menampilkannya kepada pembeli yang tepat *seketika*; kita akan membahasnya lagi di bab 12. *Pengguna* baru dengan satu klik pun terlayani serupa: satu klik sudah menghasilkan profil tak-nol. Ingatlah bahwa konten memberi tahu sistem *ke mana menampilkan* sebuah item; hanya klik nyata yang memberi tahu *apakah item itu bagus*.

Skalabilitas dan penyetelan

Vektor item dihitung secara *offline* dan disimpan dalam indeks *approximate nearest neighbour* (bab 7); profil dicari saat permintaan masuk. Setel waktu paruh, bobot event, dan resep teks pada evaluasi berbasis pemisahan waktu yang sama seperti yang lain; Anda akan memvariasikan waktu paruh pada latihan praktik.

Bagaimana ini cocok dengan sistem pemeringkatan repo

Skor content-based bukan *retriever* di *pipeline* akhir repo; empat *retriever*-nya adalah `recent`, `covisit`, `catpop`, dan `global_trending`. Pengetahuan konten masuk ke *ranker* dengan cara lain: sebagai *feature* seperti `uc_click_share_30d` (porsi klik pengguna yang jatuh pada kategori item itu) dan `item_price_vs_category` (harga dibagi median kategori), yang didefinisikan di `schema.py` dan dihitung di `features.py`. Ini pola yang umum di produksi. Skor konten yang murah menjadi satu masukan di antara banyak masukan bagi *ranker* terlatih, bukan keseluruhan sistem rekomendasi. *Ranker* kemudian belajar dari data seberapa jauh konten layak dipercaya dibandingkan popularitas atau kebaruan.

Ringkasan

- Content-based filtering merepresentasikan setiap item dengan atributnya sendiri (teks, kategori, merek) dan merekomendasikan item yang menyerupai apa yang sudah disukai pengguna.
- TF-IDF memberi bobot kata dengan *term frequency* dikali *inverse document frequency*, sehingga kata spesifik ("sneakers") lebih berarti daripada kata umum ("home"); normalisasi L2 membuat cosine similarity sama dengan *dot product*.
- Profil pengguna adalah rata-rata berbobot dari vektor item yang disentuh pengguna, dengan bobot sama dengan kekuatan event dikali peluruhan waktu eksponensial (waktu paruh 7 hari di

sini).

- Pada marketplace simulasi, metode ini mencapai NDCG@20 sebesar 0.0909, hampir dua kali 0.0475 milik `pop_7d`, tetapi di bawah 0.1302 milik `pop_in_my_categories`, karena kata kategori dan merek tidak dapat memisahkan item di dalam satu kategori.
- Cakupannya sangat tinggi (0.9892 berbanding 0.0087 untuk popularitas), tetapi judul yang nyaris kembar memiliki cosine similarity tepat 1.0, sehingga konten saja tidak dapat membedakan salinan; sinyal perilaku yang harus melakukannya.
- Kekuatan utamanya adalah *cold start* (item dan pengguna baru dapat dinilai seketika); kelemahan utamanya adalah spesialisasi berlebihan, karena metode ini tak pernah menyarankan sesuatu yang tidak mirip dengan riwayat.
- Pada sistem akhir repo, sinyal konten dipakai sebagai *feature* untuk *ranker*, bukan sebagai *retriever* mandiri.

Kuis

Q4.1 Mengapa TF-IDF mengalikan *term frequency* dengan *inverse* document frequency?

A. Agar dokumen panjang mendapat skor lebih tinggi daripada dokumen pendek.
B. Untuk memberi bobot lebih pada term yang muncul di sedikit dokumen dan bobot lebih kecil pada term yang muncul di banyak dokumen.
C. Agar semua vektor saling ortogonal.
D. Untuk menghapus *stop-word* dari kosakata secara otomatis.

Q4.2 Setelah vektor TF-IDF dinormalisasi L2, pernyataan mana tentang cosine similarity yang benar?

A. Nilainya sama dengan jarak Euclidean antara kedua vektor.
B. Nilainya sama dengan *dot product* kedua vektor.
C. Nilainya bisa negatif untuk dua item mana pun.
D. Nilainya bergantung pada banyaknya kata dalam teks tiap item.

Q4.3 Seorang penjual mendaftarkan item baru pukul 09:00 dan belum ada yang mengekliknya. Metode mana yang masih dapat merekomendasikannya pukul 09:01?

A. Popularitas global selama 7 hari terakhir.
B. *Co-visitation* item-ke-item yang dihitung dari sesi.
C. Penilai content-based yang memakai judul, merek, dan kategori item.
D. *Recommender* yang hanya memakai pesanan masa lalu.

Q4.4 Pada hasil, dua item berbeda memiliki cosine similarity tepat 1.000. Apa yang ditunjukkan hal ini tentang teks keduanya, dan mengapa ini menjadi masalah bagi *recommender* yang hanya berbasis konten?

Q4.5 Jelaskan "spesialisasi berlebihan" (*over-specialisation*) dalam content-based filtering dan sebutkan dua cara untuk menguranginya.

Q4.6 Bacalah baris berikut dari skrip:

book/code/ch04_content.py · profile_norm

```
profile = normalize(sp.diags(1 / np.where(wsum > 0, wsum, 1)) @ W @ X)
```

Apa yang akan salah tanpa `np.where(wsum > 0, wsum, 1)`, dan seperti apa baris profil pengguna tanpa event positif setelah baris itu dijalankan?

Q4.7 Seorang pengguna mengeklik item A hari ini (kekuatan 1, umur 0 hari) dan memasukkan item B ke keranjang 14 hari lalu (kekuatan 3). Dengan waktu paruh 7 hari, berapa pecahan dari total bobot profil akhir yang berasal dari item A? (Jawab dengan tiga angka desimal.)

Q4.8 Dua item memiliki vektor berpanjang satu $\mathbf{a} = (0.6,\ 0.8,\ 0)$ dan $\mathbf{b} = (0.8,\ 0,\ 0.6)$. Hitung cosine similarity keduanya.

Q4.9 *Praktik langsung.* Di `book/code/ch04_content.py`, ubah `HALF_LIFE = 7.0` menjadi `1.0`, jalankan skrip, lalu ulangi dengan `30.0`. Bandingkan `ndcg@20` dan `recall@20` pada tabel hasil dengan *baseline* 0.0909 dan 0.1378. Apa yang Anda perkirakan tentang seberapa cepat profil sebaiknya melupakan, dan apakah hasilnya sesuai? (Kembalikan nilainya setelahnya.)

Bab 5 — Collaborative filtering

Tidak ada yang menulis kata "kopi" di kartu itu

Selasa malam, seorang pembeli di Surabaya membuka halaman produk paket hadiah kopi dan teh. Di bawah foto, baris "Produk serupa" terisi lima listing kopi dan teh lainnya. Tidak ada yang menulis aturan "paket X cocok dengan paket Y dan Z", dan tidak ada yang memberi label pada produk-produk itu. Baris tersebut muncul karena, selama tiga puluh hari terakhir, banyak pembeli yang melihat produk ini dalam sesi penjelajahan (*session*) yang sama juga melihat kelima produk tadi.

Itulah trik utama **collaborative filtering** (CF, penyaringan kolaboratif): *kerumunan pengguna sudah melakukan pelabelan untuk Anda*. Setiap klik, keranjang, dan pesanan adalah sebuah suara. Sekalipun Anda tidak pernah melihat satu pun judul, harga, atau foto, selama Anda tahu siapa menyentuh apa dan kapan, Anda tetap bisa mengatakan cukup banyak hal tentang item mana yang cocok bersama dan orang mana yang berselera sama.

Content-based filtering di Bab 4 membutuhkan teks, kategori, dan merek. Collaborative filtering hanya melihat pola perilaku, sehingga ia menghubungkan item yang tidak akan pernah dihubungkan model berbasis konten, dan gagal dengan caranya sendiri ketika data perilaku tipis. Kita membangun empat metode di atas marketplace simulasi dan evaluasi yang sama dengan Bab 3, sehingga angkanya bisa langsung dibandingkan dengan baseline popularitas: user-kNN, item-kNN, ko-kunjungan sesi (*session co-visitation*; dipakai baris "Produk serupa" di aplikasi dan menjadi salah satu dari empat retriever di repo), serta tinjauan soal skalabilitas.

Yang akan Anda pelajari

- Cara membangun matriks user–item dari event implisit.
- User-kNN dan item-kNN, dengan cosine dan Jaccard yang dihitung manual.
- Cara kerja `covisitation_matrix` di repo dan mengapa setiap barisnya dinormalisasi.
- Cara membaca hasil nyata: ndcg, cakupan (*coverage*), dan pengisi popularitas (*popularity padding*).
- Mengapa desain item-to-item bisa diskalakan.

Intuisi: bertanya pada orang yang berselera sama

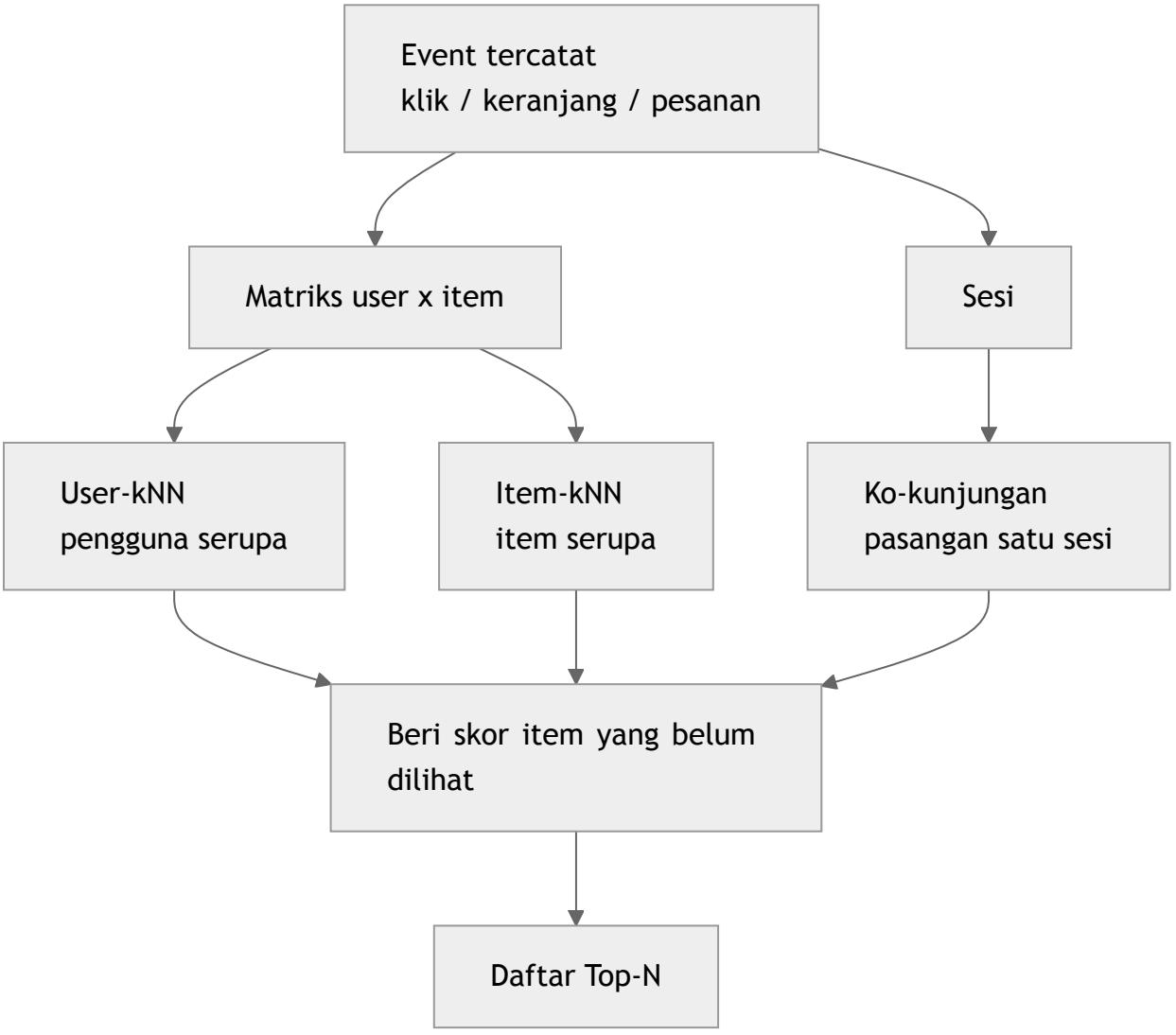
Bayangkan memilih restoran di kota baru. Anda bisa membaca menunya (berbasis konten), atau bertanya pada teman: "Kamu suka tempat ramen itu, apa lagi yang sebaiknya kucoba?" Itu tidak perlu penjelasan *mengapa* Anda menyukainya, cukup ada irisan tempat yang pernah dikunjungi; kunjungan *lain* si teman itulah rekomendasinya.

Itulah **user-based collaborative filtering**, bentuk aslinya. Ide ini mendasari sistem rekomendasi Usenet awal pada 1990-an, dan mudah dijelaskan ke product manager: "pelanggan seperti Anda juga membeli…".

Sekarang balik arahnya. Alih-alih bertanya *orang* mana yang mirip, tanyakan *item* mana yang mirip, di mana dua item dianggap mirip jika orang yang sama berinteraksi dengan keduanya. Jika semua orang

yang membeli French press juga membeli sekantong kopi gilingan kasar, kedua item itu bertetangga, sekalipun judulnya tidak punya kata yang sama. Saat pengguna menunjukkan minat pada French press, kita mencari tetangganya dan menawarkan kopi itu. Inilah **item-based collaborative filtering**. Perannya ditukar, dan, seperti akan kita lihat, pertukaran inilah yang membuatnya praktis pada skala besar.

Varian ketiga mengubah arti "bersama". Pada user-kNN dan item-kNN, dua item terhubung jika *orang* yang sama menyentuh keduanya, mungkin berbulan-bulan berselang. Pada **ko-kunjungan sesi**, dua item terhubung hanya jika disentuh dalam *sesi penjelajahan* yang sama, biasanya dalam hitungan menit. Itu sinyal yang jauh lebih tajam bahwa "kedua item ini sedang dibandingkan sekarang": pembeli yang memilih di antara tiga casing ponsel melihat ketiganya dalam satu kali duduk. Pembeli yang membeli casing ponsel di bulan Maret dan mug di bulan April hampir tidak memberi tahu kita apa-apa tentang apakah casing dan mug berkaitan.



Gambar 5.1: *Tiga cara mengubah log event yang sama menjadi rekomendasi. Dua yang pertama membaca matriks seluruh riwayat; yang ketiga membaca batas sesi.*

Algoritmenya

Matriks interaksi

Semuanya dimulai dari matriks R dengan satu baris per pengguna dan satu kolom per item. Sel R_{ui} menyimpan seberapa kuat pengguna u berinteraksi dengan item i. Log kita bersifat implisit: tidak ada yang memberi rating bintang. Repo mengubah tipe event menjadi kekuatan memakai `schema.EVENT_WEIGHT`: klik bernilai 1, add-to-cart 3, dan pesanan 5. Menjumlahkan semua event seorang pengguna pada suatu item menghasilkan nilai sel. Jika Anda mengklik sebuah item dua kali dan memasukkannya ke keranjang sekali, $R_{ui} = 1 + 1 + 3 = 5$.

Tiga sifat yang penting: R bersifat **jarang** (*sparse*; pengguna menyentuh beberapa lusin item dari ribuan); **non-negatif** (klik yang tidak ada biasanya berarti "belum pernah melihat", bukan "tidak suka"); dan **dibangun pada titik potong (*cutoff*) tertentu**, sehingga evaluasi tidak pernah melihat jawabannya (Bab 2).

User-kNN

Namanya singkatan dari *k nearest neighbours* (k tetangga terdekat). Untuk pengguna target u:

1. Hitung kemiripan antara u dan setiap pengguna lain v. Pilihan klasiknya adalah **cosine similarity** antar baris R mereka:

$$\text{sim}(u, v) = \frac{\sum_i R_{ui}\, R_{vi}}{\sqrt{\sum_i R_{ui}^2}\,\sqrt{\sum_i R_{vi}^2}}$$

2. Ambil k pengguna dengan kemiripan tertinggi, tidak termasuk u sendiri. Merekalah tetangga $N_k(u)$.
3. Beri skor setiap item dengan menjumlahkan kekuatan interaksi para tetangga, dibobot seberapa mirip masing-masing tetangga:

$$\hat{r}_{ui} = \sum_{v \in N_k(u)} \text{sim}(u, v)\; R_{vi}$$

4. Buang item yang tidak boleh dilihat pengguna (stok habis, sudah disembunyikan) dan kembalikan N teratas berdasarkan skor.

Contoh kecil yang dihitung manual. Tiga item A, B, C dan tiga pengguna yang sudah ada. Angka di setiap baris adalah (R_{vA}, R_{vB}, R_{vC}):

user	A	B	C
u_1	1	1	0
u_2	1	3	0
u_3	0	1	1

Pengguna target baru t punya satu klik pada A dan interaksi yang lebih kuat pada C, katakanlah $(1, 0, 2)$. Kita ingin tahu apakah B layak direkomendasikan.

- $\text{sim}(t, u_1) = \frac{1 \cdot 1 + 0 \cdot 1 + 2 \cdot 0}{\sqrt{5}\,\sqrt{2}} = \frac{1}{3.162} = 0.316$
- $\text{sim}(t, u_2) = \frac{1 \cdot 1}{\sqrt{5}\,\sqrt{10}} = \frac{1}{7.071} = 0.141$
- $\text{sim}(t, u_3) = \frac{2 \cdot 1}{\sqrt{5}\,\sqrt{2}} = \frac{2}{3.162} = 0.632$

Dengan $k = 3$ ketiganya menjadi tetangga, dan

$$\hat{r}_{tB} = 0.316 \cdot 1 + 0.141 \cdot 3 + 0.632 \cdot 1 = 1.372$$

Item B adalah satu-satunya item yang belum disentuh t, jadi itulah rekomendasi kita. Perhatikan siapa yang berkontribusi paling besar: u_3, tetangga yang sama-sama punya pesanan pada C, memiliki kemiripan terbesar meskipun satu-satunya bukti darinya untuk B hanyalah sebuah klik. Item yang dipesan mendominasi karena bobot ikut masuk ke cosine.

Cosine versus Jaccard

Cosine memakai *kekuatan* interaksi. **Jaccard similarity** membuangnya dan hanya bertanya siapa yang muncul:

$$J(X, Y) = \frac{|X \cap Y|}{|X \cup Y|}$$

di mana X dan Y adalah *himpunan* pengguna (untuk kemiripan item) atau item (untuk kemiripan pengguna) yang memiliki interaksi apa pun. Ambil item A dan B dari tabel di atas. Pengguna yang menyentuh A adalah $\{u_1, u_2\}$ dan yang menyentuh B adalah $\{u_1, u_2, u_3\}$. Irisannya berisi 2 pengguna dan gabungannya 3, sehingga $J(A, B) = 2/3 = 0.667$. Untuk B dan C: irisannya $\{u_3\}$, gabungannya $\{u_1, u_2, u_3\}$, sehingga $J(B, C) = 1/3 = 0.333$.

Jaccard cocok untuk sinyal yang benar-benar biner ("dalam sesi yang sama atau tidak") dan melindungi dari pengguna berat (*heavy user*) dengan hitungan sangat besar. Cosine adalah default yang lebih baik ketika bobot membawa informasi, seperti pada data kita: satu pesanan bernilai lima klik. Keduanya simetris dan tidak peduli seberapa populer sebuah item.

Item-kNN

Item-kNN adalah mesin yang sama yang diterapkan pada kolom-kolom R. Definisikan kemiripan item i dan j sebagai cosine kolom-kolomnya:

$$\text{sim}(i, j) = \frac{\sum_u R_{ui}\, R_{uj}}{\sqrt{\sum_u R_{ui}^2}\,\sqrt{\sum_u R_{uj}^2}}$$

lalu beri skor item kandidat untuk pengguna u dengan menjumlahkan kemiripannya dengan semua yang pernah disentuh pengguna, dibobot seberapa kuat pengguna menyentuhnya:

$$\hat{r}_{ui} = \sum_j \text{sim}(i, j)\; R_{uj}$$

Dengan matriks mainan yang sama, kolom-kolom atas pengguna (u_1, u_2, u_3) adalah $A = (1, 1, 0)$, $B = (1, 3, 1)$, dan $C = (0, 0, 1)$. Maka:

- $\mathrm{sim}(A,B) = \frac{1+3}{\sqrt{2}\sqrt{11}} = \frac{4}{4.690} = 0.853$
- $\mathrm{sim}(B,C) = \frac{1}{\sqrt{11}\cdot 1} = 0.302$
- $\mathrm{sim}(A,C) = 0$, karena tidak ada yang menyentuh keduanya.

Untuk target $t = (1, 0, 2)$ atas (A, B, C), skor B adalah $0.853 \cdot 1 + 0.302 \cdot 2 = 1.457$. Bentuk komputasinya berbeda: pada user-kNN Anda membandingkan target dengan setiap pengguna saat permintaan datang, sedangkan tabel item–item tidak bergantung pada target, sehingga dihitung sekali di muka dan sebuah permintaan hanyalah pencarian tetangga dari beberapa item yang disentuh pengguna.

Ko-kunjungan sesi

Ko-kunjungan melepaskan pengguna sebagai unit pengelompokan dan memakai **sesi**. Langkah-langkahnya:

1. Ambil hanya event positif dari jendela waktu terbaru (30 hari di repo ini).
2. Dalam setiap sesi, simpan satu bobot per item: event terkuat yang terjadi padanya.
3. Untuk setiap pasangan berurutan item berbeda (a, b) yang berbagi sesi, jumlahkan bobot b dari semua sesi tempat keduanya muncul.
4. **Normalisasi per item sumber**, sehingga bobot keluar setiap item a berjumlah 1:

$$w(a \to b) = \frac{\sum_{s \ni a,b} \omega_{s,b}}{\sum_{c \neq a} \sum_{s \ni a,c} \omega_{s,c}}$$

di mana $\omega_{s,b}$ adalah kekuatan item b dalam sesi s.

Contoh hitung. Tiga sesi: S_1 berisi klik pada A dan klik pada B; S_2 berisi klik pada A, add-to-cart pada B, dan klik pada C; S_3 berisi klik pada A dan klik pada C. Lihat baris untuk A. Jumlah mentah untuk B adalah $1 + 3 = 4$ (klik di S_1, keranjang di S_2). Jumlah mentah untuk C adalah $1 + 1 = 2$. Totalnya 6, sehingga $w(A \to B) = 4/6 = 0.667$ dan $w(A \to C) = 2/6 = 0.333$.

Normalisasi per *sumber* mencegah sumber populer menenggelamkan segalanya (setiap baris berjumlah 1) dan membuat matriks **asimetris**: orang yang melihat casing ponsel sering juga melihat ponselnya, tetapi tidak sebaliknya.

Untuk personalisasi, ambil item terbaru pengguna sebagai *seed* (benih), masing-masing dengan skor kebaruan (*recency*) dari jumlah berpeluruhan waktu (waktu paruh 7 hari, peluruhan yang sama dengan Bab 3). Skor kandidat b adalah $\sum_{a \in \text{seeds}} \text{recent}(u, a) \cdot w(a \to b)$. Jika seed A milik pengguna punya skor kebaruan 0.8, maka pada contoh mainan tadi B mendapat $0.8 \times 0.667 = 0.533$ dan C mendapat $0.8 \times 0.333 = 0.267$.

Tahukah kamu?

Tahukah kamu? Pada 2003, para insinyur Amazon, Greg Linden, Brent Smith, dan Jeremy York, menjelaskan algoritma collaborative filtering item-to-item di balik rekomendasi Amazon.com: alih-alih membandingkan seorang pelanggan dengan jutaan pelanggan lain, mereka membandingkan *item*, dan tabel item serupa yang mahal itu dibangun secara offline. Langkah online-nya hanya mencari item serupa dari judul yang dibeli atau diberi rating pengguna, sehingga skalanya tidak bergantung pada ukuran katalog maupun jumlah pelanggan. Makalah itu membandingkannya dengan collaborative filtering tradisional, yang komputasi online-nya tumbuh seiring keduanya.

Sumber: Linden et al., IEEE Internet Computing, 2003

Tahukah kamu?

Tahukah kamu? Kata "collaborative" dalam collaborative filtering berasal dari 1994. Proyek GroupLens (Resnick dan kolega di MIT dan University of Minnesota) membangun sistem yang memprediksi seberapa besar pembaca akan menyukai artikel netnews Usenet, berdasarkan heuristik bahwa orang yang sepakat di masa lalu kemungkinan akan sepakat lagi. Pengguna memasukkan rating, server rating bernama "Better Bit Bureaus" mengorelasikannya, dan skor prediksi ditampilkan di news reader. Itulah user-kNN di habitat aslinya.

Sumber: Resnick et al., CSCW, 1994

Tahukah kamu?

Tahukah kamu? Pendekatan berbasis pengguna menjadi default selama bertahun-tahun, tetapi pada 2001 Sarwar, Karypis, Konstan, dan Riedl membandingkannya dengan metode berbasis item dan melaporkan bahwa algoritma berbasis item memberikan kinerja yang jauh lebih baik daripada algoritma berbasis pengguna, sekaligus kualitas yang lebih baik daripada algoritma berbasis pengguna terbaik yang tersedia (pada dataset yang mereka uji). Makalah itu membantu menggeser perhatian bidang ini dari "pengguna serupa" ke "item serupa".

Sumber: Sarwar et al., WWW, 2001

Penelusuran kode

Semua angka di bab ini berasal dari `book/code/ch05_cf.py`. Skrip itu memakai setup bersama dari `book/code/common.py`: data simulasi yang sama, cutoff yang sama, evaluasi yang sama. Pertama matriksnya. Helper ini mengubah event positif dari riwayat menjadi matriks sparse user × item berisi jumlah nilai `EVENT_WEIGHT`:

book/code/common.py · interaction_matrix

```
def interaction_matrix(hist: data.History) -> tuple[sp.csr_matrix, pd.Index, pd.Index]:
    """User x item matrix of summed EVENT_WEIGHT (click 1, cart 3, order 5)."""
    pos = hist.interactions()
    user_index = pd.Index(np.sort(pos["user_id"].unique()), name="user_id")
    item_index = pd.Index(np.sort(pos["item_id"].unique()), name="item_id")
    agg = pos.groupby(["user_id", "item_id"])["w"].sum().reset_index()
    m = sp.csr_matrix(
        (agg["w"].to_numpy(), (user_index.get_indexer(agg["user_id"]),
item_index.get_indexer(agg["item_id"]))),
        shape=(len(user_index), len(item_index)),
    )
    return m, user_index, item_index
```

`pos["w"]` adalah bobot per event (1, 3, atau 5), `groupby` menjumlahkannya per pengguna dan item, dan konstruktor `csr_matrix` menaruh setiap jumlah pada baris dan kolom yang tepat. Dua objek `pd.Index` menerjemahkan antara id pengguna dan item yang sebenarnya dengan posisi dalam matriks.

Untuk cosine kita butuh baris dengan panjang satuan, karena cosine dua vektor tak lain adalah hasil kali titiknya setelah keduanya diskalakan ke panjang 1:

book/code/ch05_cf.py · row_normalise

```
def row_normalise(m: np.ndarray) -> np.ndarray:
    norm = np.linalg.norm(m, axis=1, keepdims=True)
    return m / np.maximum(norm, 1e-12)
```

`np.maximum(norm, 1e-12)` menjaga dari pembagian dengan nol untuk pengguna atau item yang barisnya kosong. Setelah ini, "kemiripan semua terhadap semua" menjadi satu perkalian matriks.

User-kNN, langkah demi langkah

Kutipan berikut dari `main()` adalah seluruh model user-kNN. Variabel `R` adalah matriks user × item padat dan `R_eval` adalah baris-baris pengguna evaluasi:

book/code/ch05_cf.py · user_knn_sim

```
Rn = row_normalise(R)
sim_users = row_normalise(R_eval) @ Rn.T  # eval users x all users
```

Lalu, di dalam loop atas k:

book/code/ch05_cf.py · user_knn_topk

```
idx = np.argpartition(-sim_users, k, axis=1)[:, :k]
w = np.zeros_like(sim_users)
np.put_along_axis(w, idx, np.take_along_axis(sim_users, idx, axis=1), axis=1)
scores = to_avail(w @ R)
```

Bacalah sebagai empat langkah algoritma. Perkalian matriks menghitung semua cosine sekaligus. `argpartition` menemukan indeks k kemiripan terbesar per baris *tanpa* mengurutkan seluruh baris (waktunya linear, bukan $n \log n$). Dua baris berikutnya membangun matriks `w` yang menyimpan kemiripan pada posisi tetangga dan nol di tempat lain. Lalu `w @ R` persis rumus $\hat{r}_{ui} = \sum_{v \in N_k(u)} \mathrm{sim}(u, v) R_{vi}$ untuk semua pengguna dan item sekaligus. Skrip ini juga menetapkan entri diagonal setiap pengguna evaluasi menjadi nol agar pengguna bukan tetangga terdekat dirinya

sendiri; jika lupa, setiap pengguna akan "merekomendasikan" riwayatnya sendiri kepada dirinya, yang skornya indah tetapi tidak bermakna.

Item-kNN, langkah demi langkah

Item-kNN lebih pendek karena kerja beratnya adalah perkalian yang sama, diterapkan pada kolom:

book/code/ch05_cf.py · item_knn_sim

```
Cn = row_normalise(R.T)  # items x users, unit rows
S = Cn @ Cn.T
np.fill_diagonal(S, 0.0)
```

dan baris penskorannya:

book/code/ch05_cf.py · item_knn_score

```
scores = to_avail(R_eval @ S)  # score(u, i) = sum_j sim(i, j) * weight(u, j)
```

`R.T` menukar baris dan kolom, sehingga `Cn @ Cn.T` adalah tabel cosine item–item `S`, dengan diagonal dinolkan agar tidak ada item yang menjadi tetangga dirinya sendiri. Baris penskoran menerapkan rumus untuk semua pengguna sekaligus. Sistem produksi tidak pernah membentuk tabel padat seperti ini (lihat skalabilitas).

Matriks ko-kunjungan di repo

Sekarang bagian yang berjalan di dalam sistem sebenarnya. Fungsinya ada di `recommender/recsys/candidates.py`:

recommender/recsys/candidates.py · covisitation_matrix

```
def covisitation_matrix(hist: History) -> pd.DataFrame:
    """(item_a, item_b, weight): normalised co-occurrence of positive actions within a session."""
    pos = hist.interactions()
    pos = pos[pos["age_days"] < COVISIT_WINDOW_DAYS]
    per_session = pos.groupby(["session_id", "item_id"], as_index=False)["w"].max()
    pairs = per_session.merge(per_session, on="session_id", suffixes=("_a", "_b"))
    pairs = pairs[pairs["item_id_a"] != pairs["item_id_b"]]
    m = pairs.groupby(["item_id_a", "item_id_b"], as_index=False)["w_b"].sum()
    # Normalise per source item, so very popular seeds don't dominate everything.
    m["w"] = m["w_b"] / m.groupby("item_id_a")["w_b"].transform("sum")
    return m[["item_id_a", "item_id_b", "w"]]
```

Telusuri baris demi baris. `hist.interactions()` mengembalikan event positif (klik, keranjang, pesanan) beserta bobot `w` dan usianya dalam hari. Baris berikutnya hanya menyimpan event yang lebih baru dari `COVISIT_WINDOW_DAYS` (30). Mengambil `["w"].max()` per `(session_id, item_id)` meringkas aksi berulang pada item yang sama dalam satu sesi menjadi satu aksi terkuat, sehingga pembeli yang menyegarkan halaman sepuluh kali tidak dihitung sepuluh kali. *Self-merge* pada `session_id` membuat setiap pasangan item dalam sesi yang sama (sebuah *self-join*); filter `!=` membuang pasangan trivial item dengan dirinya sendiri. Menjumlahkan `w_b` per pasangan berurutan menghasilkan massa ko-kemunculan mentah, dan `transform("sum")` terakhir membaginya dengan total massa keluar item sumber, yaitu normalisasi dari bagian algoritma.

Self-join adalah pendorong biaya: sesi dengan n item menghasilkan $n(n-1)$ baris, sehingga sebuah bot dengan 5,000 item dalam satu sesi akan menghasilkan sekitar dua puluh lima juta.

Mengubah matriks menjadi kandidat per pengguna adalah tugas `covisit`:

recommender/recsys/candidates.py · covisit

```
def covisit(hist: History, seeds: pd.DataFrame) -> pd.DataFrame:
    top_seeds = _top_per_user(seeds, "cand_recent_score", N_SEEDS)
    m = covisitation_matrix(hist)
    j = top_seeds.merge(m, left_on="item_id", right_on="item_id_a")
    j["s"] = j["cand_recent_score"] * j["w"]
    out = j.groupby(["user_id", "item_id_b"], as_index=False)["s"].sum()
    out = out.rename(columns={"item_id_b": "item_id", "s": "cand_covisit_score"})
    return _top_per_user(out, "cand_covisit_score", N_COVISIT)
```

`top_seeds` memilih sepuluh item terbaru yang paling relevan bagi setiap pengguna (`N_SEEDS`). Merge memasangkan setiap seed dengan barisnya di matriks, hasil kali `cand_recent_score * w` membobot setiap saran sesuai seberapa penting seed bagi pengguna, dan group-by menjumlahkan kontribusi ketika beberapa seed menunjuk kandidat yang sama. Akhirnya `_top_per_user` menyimpan 40 terbaik (`N_COVISIT`). Dua seed yang menunjuk kandidat yang sama saling memperkuat, dan itu perilaku yang benar: item yang di-ko-kunjungi dengan *kedua* hal yang Anda lihat adalah taruhan yang lebih baik daripada item yang hanya di-ko-kunjungi dengan salah satunya.

Seed berasal dari `recent`, yang membobot setiap event dengan `_decay`: event berusia t hari dihitung $0.5^{t/7}$ dari event baru (Bab 3).

Dari mana halaman produk mendapat barisnya

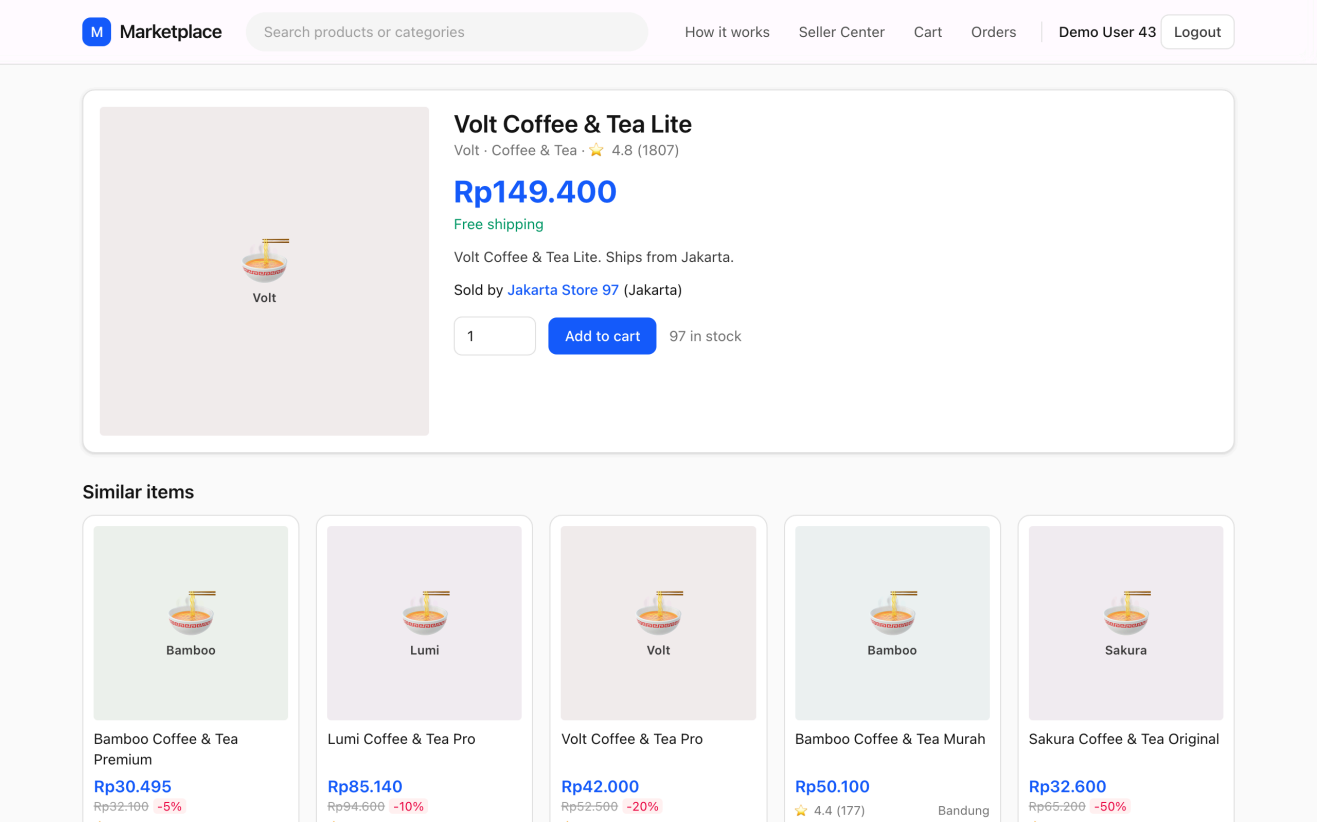
Baris "Produk serupa" dari cerita pembuka bukan algoritma yang berbeda. Metode gRPC `SimilarItems` memanggil `pipeline.similar_items`, yang membaca satu baris dari matriks ko-kunjungan yang sama:

recommender/recsys/pipeline.py · similar_items

```
def similar_items(tables: dict[str, pd.DataFrame], now: pd.Timestamp, item_id: int, k: int) ->
pd.DataFrame:
    """Product page "similar items": co-visited items first, then popular items of the same
category."""
    hist = data.History.at(tables, now)
    empty = pd.DataFrame({"item_id": pd.Series(dtype="int64"), "score":
pd.Series(dtype="float64"), "source": pd.Series(dtype="object")})
    if item_id not in set(hist.items["item_id"]):
        return empty
    in_stock = hist.items.loc[hist.items["in_stock"], ["item_id", "category_id"]]
    m = candidates.covisitation_matrix(hist)
    co = m[(m["item_id_a"] == item_id) & (m["item_id_b"] != item_id) &
m["item_id_b"].isin(in_stock["item_id"])]
    co = co.sort_values("w", ascending=False).head(k)
    out = pd.DataFrame({"item_id": co["item_id_b"].to_numpy(), "score": co["w"].to_numpy(),
"source": "covisit"})

    category = hist.items.loc[hist.items["item_id"] == item_id, "category_id"].iloc[0]
    pop = candidates.item_popularity(hist, 7)
    same = in_stock[(in_stock["category_id"] == category) & (in_stock["item_id"] != item_id) &
~in_stock["item_id"].isin(out["item_id"])]
    same = same.assign(score=same["item_id"].map(pop).fillna(0.0)).sort_values("score",
ascending=False, kind="stable")
    fill = pd.DataFrame({"item_id": same["item_id"].to_numpy(), "score": same["score"].to_numpy(),
"source": "same_category"})
    out = pd.concat([out, fill.head(k - len(out))], ignore_index=True) if len(out) < k else out
    return out.astype({"item_id": "int64", "score": "float64"}).reset_index(drop=True) if len(out)
else empty
```

Paruh pertama membangun baris ko-kunjungan untuk item di halaman itu: baris dengan `item_id_a == item_id`, dibatasi pada item yang tersedia stoknya, diurutkan menurut bobot. Paruh kedua adalah *fallback* untuk item yang jarang terlihat: jika ko-kunjungan menghasilkan kurang dari `k` tetangga, sisa slot diisi item tersedia paling populer dari *kategori yang sama*. Hasilnya membawa label `source` (`covisit` atau `same_category`) sehingga frontend dan log bisa mengetahui mekanisme mana yang menghasilkan tiap kartu.



Gambar 5.2: *Halaman produk listing kopi dan teh dengan baris "Produk serupa" di bawahnya. Setiap kartu dalam baris berasal dari baris ko-kunjungan item di atasnya, dengan item populer sekategori mengisi slot yang kosong.*

Menjalankannya

Dari direktori `book/`:

```
uv run python code/ch05_cf.py
```

Setup-nya:

KELUARAN TERMINAL

```
=== Setup ===
interaction matrix: 1408 users x 2120 items, 18256 non-zero cells, density 0.612%
eval users: 947   recommendable (in-stock) items: 2311
eval users with history before the cutoff: 919  (the rest get 7-day popularity)
```

Selanjutnya, sapuan user-kNN atas ukuran lingkungan (*neighbourhood*) k, dengan contoh seperti apa sebuah lingkungan itu:

KELUARAN TERMINAL

```
=== User-kNN (cosine, neighbours k) ===
k=10   ndcg@20 0.1456  recall@20 0.1740
k=25   ndcg@20 0.1601  recall@20 0.2072
k=50   ndcg@20 0.1662  recall@20 0.2273
k=100  ndcg@20 0.1702  recall@20 0.2419
example: strongest neighbours of the first user with history
  user 1: user 328 (cos 0.703), user 964 (cos 0.380), user 1479 (cos 0.284)
```

Item-kNN, dengan item terdekat dari satu item konkret:

KELUARAN TERMINAL

```
=== Item-kNN (cosine between item columns) ===
item-item similarity: 2120 x 2120, mean non-zero similarity 0.0772, max 1.000
example: most similar items to item 1769 ('Sakura Earphones Original'):
  item 1064  cos 0.538  'Volt Instant Noodles Lokal'
  item 956   cos 0.497  'Bamboo Snacks Best Seller'
  item 2322  cos 0.497  'Sakura Instant Noodles Pro'
```

Ko-kunjungan, dan seberapa banyak pengisi popularitas yang dibutuhkan tiap metode:

KELUARAN TERMINAL

```
=== Session co-visitation (seeded with the user's decayed recent items) ===
co-visitation pairs: 13632   seeds per user <= 10

=== Padding with 7-day popularity (users/slots the method could not fill) ===
pop_7d     no-score users    0  partially padded    0  slots padded 0.0%
user_knn   no-score users   29  partially padded    1  slots padded 3.1%
item_knn   no-score users   29  partially padded    1  slots padded 3.1%
covisit    no-score users   30  partially padded  117  slots padded 8.6%
```

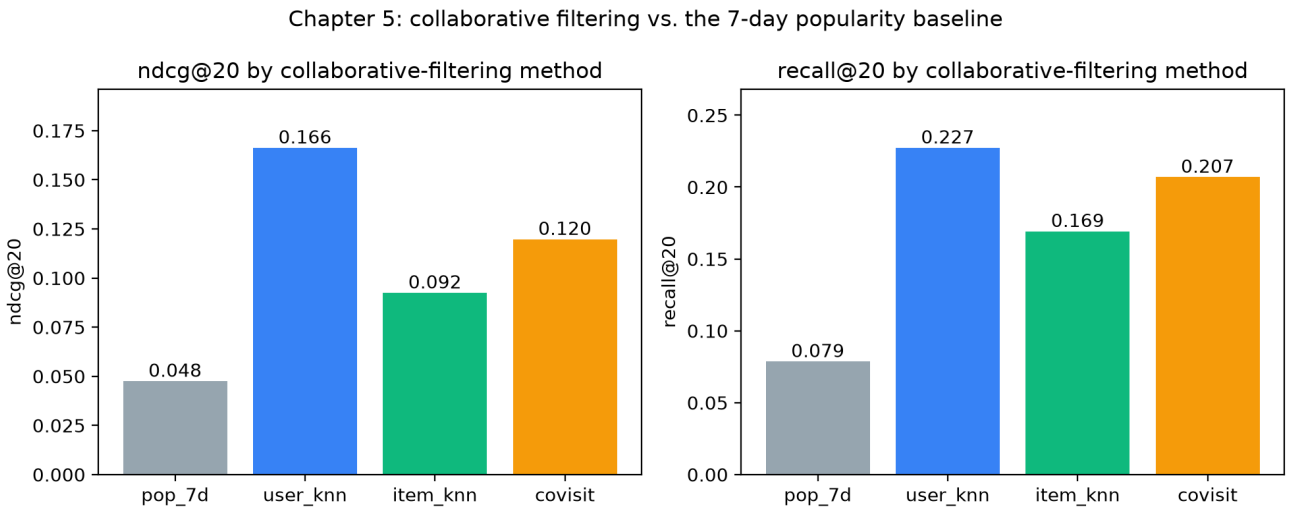
Terakhir tabel ringkasan, dievaluasi pada 50 teratas per pengguna:

KELUARAN TERMINAL

```
=== Results (top-50 lists, evaluated at K=20) ===
           recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20  coverage@20
method
pop_7d       0.0310     0.0493     0.0786     0.1335           0.1066   0.0475       0.0087
user_knn     0.1230     0.1755     0.2273     0.3163           0.3456   0.1662       0.4158
item_knn     0.0460     0.1001     0.1692     0.2934           0.2772   0.0925       0.8399
covisit      0.0795     0.1319     0.2069     0.3220           0.2999   0.1197       0.6460
```

Hasil

Gambaran utamanya: setiap metode kolaboratif mengalahkan baseline popularitas dengan selisih lebar.



Gambar 5.3: *ndcg@20 dan recall@20 untuk baseline popularitas 7 hari dan tiga metode kolaboratif.*

Membaca tabelnya

Baseline `pop_7d` mendapat ndcg@20 sebesar 0.0475 dan recall@20 sebesar 0.0786; coverage@20 sebesar 0.0087 berarti hanya sekitar dua puluh item berbeda dari 2,311. User-kNN menaikkan ndcg@20 menjadi 0.1662, sekitar tiga setengah kali baseline. Bahkan metode terlemah, item-kNN dengan 0.0925, hampir menggandakannya.

User-kNN menang pada ndcg. Dengan 50 tetangga ia mencapai ndcg@20 0.1662 dan order_recall@20 0.3456 (recall khusus pada pesanan, sinyal terkuat). Sapuan menunjukkan hasil yang makin menurun: dari 10 ke 100 tetangga, ndcg@20 naik dari 0.1456 menjadi 0.1702. Dalam simulator, selera mengikuti preferensi kategori, sehingga pengguna dengan riwayat yang beririsan memang berbagi apa yang mereka lakukan berikutnya. Dengan hanya 1,408 pengguna, memindai semuanya murah, sehingga eksperimen ini menyembunyikan apa yang membuat user-kNN menyakitkan pada skala besar. (Kita memakai $k = 50$ tetap; menyetel k pada pengguna evaluasi akan menjadi kebocoran ringan.)

Item-kNN lebih lemah dalam pemeringkatan, lebih kuat dalam keluasan. ndcg@20-nya 0.0925, tetapi coverage@20-nya 0.8399 dibanding 0.4158 milik user-kNN: item-kNN menyebarkan rekomendasinya ke bagian katalog yang sangat besar. Itu sebagian baik (eksposur *long tail*, Bab 1) dan sebagian gejala *noise*, seperti ditunjukkan contoh ini:

Item terdekat dari item 1769, 'Sakura Earphones Original', mencakup 'Volt Instant Noodles Lokal' dengan cosine 0.538. Earphone dan mi tidak berhubungan; kedua item punya sedikit interaksi dan beberapa pengguna kebetulan menyentuh keduanya. Rata-rata kemiripan tak-nol hanya 0.0772 tetapi maksimumnya 1.000, hampir pasti dua item yang disentuh oleh satu pengguna yang sama. Item-kNN produksi karena itu menerapkan **ambang dukungan (*support threshold*)** atau **penyusutan (*shrinkage*)**: abaikan pasangan dengan terlalu sedikit pengguna bersama, atau kalikan kemiripan dengan $n_{ij}/(n_{ij} + \lambda)$.

Ko-kunjungan berada di tengah pada ndcg dan menang pada satu metrik. Ia mencapai ndcg@20 0.1197, tetapi recall@50-nya 0.3220, tertinggi dari semua metode dalam tabel (user-kNN 0.3163). Itulah pola yang Anda inginkan dari sebuah *retriever*: ia mungkin tidak mengurutkan 5 teratasnya sebaik metode terbaik, tetapi daftar yang lebih lebar menjaring sedikit lebih banyak dari apa yang

akhirnya diinginkan pengguna. Recall@5-nya 0.0795 dibanding 0.1230 milik user-kNN, yang persis menunjukkan kelemahan pengurutan. Dalam pipeline produksi, pengurutan adalah tugas ranker; retriever hanya perlu membawa item yang tepat ke meja. Bab 8 mengukur ini pada keempat retriever.

Padding: bagian yang jujur

Baris-baris padding memberi tahu seberapa sering sebuah metode tidak punya apa-apa untuk dikatakan:

- user-kNN dan item-kNN masing-masing memiliki 29 pengguna evaluasi tanpa skor model sama sekali (pada dasarnya pengguna tanpa riwayat yang bisa dipakai sebelum cutoff), ditambah satu pengguna yang terisi sebagian, dan 3.1% dari semua slot diisi daftar popularitas 7 hari.
- Ko-kunjungan memiliki 30 pengguna tanpa skor dan 117 yang terisi sebagian, sehingga 8.6% slot berasal dari fallback popularitas.

Ko-kunjungan lebih banyak melakukan padding karena ia membutuhkan item *terbaru* pengguna pernah di-ko-kunjungi dalam jendela 30 hari: presisi tinggi, cakupan per pengguna rendah. Repo tidak pernah memakainya sendirian; `trending` dan `catpop` menutup celahnya dan ranker melihat keempat skor.

Jebakan dan praktik di produksi

Skalabilitas: mengapa item-to-item menang

Hitung apa yang dilakukan tiap pendekatan saat permintaan datang. User-kNN harus membandingkan target dengan semua U pengguna: sekitar $U \times I$ operasi untuk vektor padat, atau lebih cermat U perkalian titik sparse. Dengan 1,408 pengguna itu memakan milidetik. Dengan puluhan juta pengguna, itu mustahil per permintaan, dan solusi sementara yang umum (sampling pengguna, pengelompokan, reduksi dimensi) semuanya mengorbankan kualitas.

Item-kNN memindahkan biayanya ke offline. Tabel item–item dihitung dalam batch job, misalnya tiap malam, dan disimpan sebagai peta dari setiap item ke m tetangga teratasnya (biasanya 50–200). Saat permintaan datang, sistem mencari daftar tetangga dari beberapa item dalam riwayat pengguna dan menggabungkannya. Kerjanya bergantung pada jumlah item yang disentuh *pengguna*, bukan pada katalog atau basis pengguna. Inilah properti yang ditonjolkan makalah Amazon, dan mengapa begitu banyak industri hidup dengan tabel "juga dilihat".

Komputasi offline-nya tetap perlu kehati-hatian: Anda memanfaatkan sparsity (hanya pasangan dengan pengguna bersama yang penting), membatasi riwayat per pengguna, dan men-sampling item blockbuster. Dalam skrip padat kita tabelnya 2,120 $\times$ 2,120; dengan sepuluh juta item akan ada seratus triliun sel, sehingga produksi menyimpan daftar top-m yang sparse.

Ko-kunjungan punya bentuk yang sama, dan repo hanya menyimpan pasangan yang benar-benar terjadi: 13,632 pasangan, seperti ditunjukkan output di atas. Pendorong biayanya, seperti telah disebut, adalah self-join sesi. Di produksi Anda akan membatasi item per sesi (misalnya 50 terbaru) dan memberi jendela pada data, yang dilakukan repo dengan `COVISIT_WINDOW_DAYS`.

Bias popularitas dan gray sheep

Item populer muncul di banyak daftar tetangga, lebih sering direkomendasikan, dan mengumpulkan lebih banyak klik; normalisasi meredamnya tetapi tidak menghilangkannya (Bab 12). Pengguna yang seleranya beririsan dengan tak seorang pun (*gray sheep*) mendapat sinyal lingkungan yang lemah, dan pengisi popularitas mengambil alih.

Jebakan evaluasi

- **Kemiripan dengan diri sendiri.** Lupa membuang baris pengguna itu sendiri dari daftar tetangganya membuat model merekomendasikan riwayat pengguna sendiri: angka luar biasa, model tak berguna.
- **Menyetel pada pengguna uji.** Ukuran lingkungan k, konstanta shrinkage, dan jumlah seed sebaiknya disetel pada jendela validasi sebelum jendela evaluasi.
- **Cold start.** Item baru tidak punya kolom di R dan tidak punya pasangan sesi, dan pengguna baru punya baris kosong, sehingga CF tidak punya apa-apa untuk dikatakan (Bab 12). Inilah mengapa repo menggabungkan empat retriever.

Yang sebenarnya dilakukan sistem produksi

Sedikit sistem besar yang menjalankan user-kNN murni. Tabel ko-kunjungan atau item-to-item, ditambah embedding hasil pelatihan (Bab 6 dan 7), bertindak sebagai **retriever** ber-recall tinggi yang murah; ranker terlatih mengurutkan gabungannya (Bab 9 dan 10). Di sini `cand_covisit_score` adalah salah satu kolom masukan ranker.

> **Tips**
>
> Saat Anda membangun tabel ko-kunjungan untuk lalu lintas nyata, simpan hitungan mentah dan bobot ternormalisasi sebagai kolom terpisah. Bobot ternormalisasi dipakai untuk pemeringkatan, tetapi pasangan berbobot 0.5 yang didukung dua sesi jauh kurang dapat dipercaya daripada yang didukung dua ribu sesi. Ambang hitungan minimum, diterapkan sebelum normalisasi, adalah peningkatan kualitas termurah yang bisa Anda lakukan.

> **Coba sendiri**
>
> Buka `book/code/ch05_cf.py`, temukan baris `for k in (10, 25, 50, 100):` lalu ubah ukuran lingkungannya. Jalankan ulang dan lihat kolom ndcg@20. Seberapa jauh Anda bisa menaikkan k sebelum kurvanya berhenti membaik? Lihat soal Kuis Q5.10 untuk latihan lengkapnya.

Ringkasan

- Collaborative filtering merekomendasikan dari perilaku semata, sehingga ia menghubungkan item yang tidak akan dihubungkan model konten; nol pada matriks implisit berarti "belum terlihat", bukan "tidak disukai".
- User-kNN memakai suara berbobot kemiripan dari k pengguna terdekat; item-kNN memakai kemiripan dengan item yang disentuh pengguna; keduanya bertumpu pada cosine (atau Jaccard untuk sinyal biner).

- Ketiganya mengalahkan popularitas 7 hari (ndcg@20 0.0475): user-kNN 0.1662, ko-kunjungan 0.1197, item-kNN 0.0925. Item-kNN memiliki cakupan terluas (0.8399) tetapi kemiripan yang berisik di tempat yang sedikit pengguna beririsan.
- Ko-kunjungan sesi adalah CF item-to-item dengan sesi sebagai unit pengelompokan; repo menormalisasi bobot keluar setiap item sumber agar berjumlah 1, yang meredam item populer dan membuat matriks asimetris.
- Ko-kunjungan memiliki recall@50 terbaik dari semua metode dalam tabel (0.3220), yang dibutuhkan sebuah retriever, tetapi bergantung pada pengisi popularitas untuk pengguna dengan sedikit riwayat terbaru (8.6% slot).
- Desain item-to-item bisa diskalakan karena tabel mahalnya dihitung offline dan langkah online hanya bergantung pada ukuran riwayat satu pengguna.
- Di produksi, sinyal kolaboratif adalah retriever dan fitur ranker, bukan kata akhir soal pengurutan.

Kuis

Q5.1 Pada user-based collaborative filtering, asumsi apa yang mendasari rekomendasi?

A. Item dengan judul mirip disukai oleh orang yang sama.
B. Pengguna yang berinteraksi dengan banyak item yang sama di masa lalu kemungkinan akan menyukai item serupa di masa depan.
C. Item populer selalu lebih baik daripada yang tidak populer.
D. Selera pengguna tidak berubah seiring waktu.

Q5.2 Mengapa desain item-to-item ala Amazon bisa diskalakan ke katalog dan basis pelanggan puluhan juta, sedangkan user-kNN klasik tidak?

A. Ia memakai ukuran kemiripan yang lebih akurat.
B. Ia tidak membutuhkan data interaksi sama sekali.
C. Ia menghitung tabel item serupa yang mahal secara offline, sehingga pencarian online hanya bergantung pada jumlah item dalam riwayat pengguna.
D. Ia menyimpan matriks user × item padat penuh di memori agar aksesnya cepat.

Q5.3 Situasi mana yang paling cocok untuk Jaccard similarity dibanding cosine similarity pada matriks berbobot mentah?

A. Sel matriks berisi bobot yang disetel cermat di mana satu pesanan bernilai lima klik dan Anda ingin bobot itu berpengaruh.
B. Sinyalnya biner (sebuah pasangan "dalam sesi yang sama" atau tidak) dan pengguna berat tidak boleh mendominasi lewat hitungan besar.
C. Anda menginginkan kemiripan yang tidak simetris.
D. Anda perlu membandingkan vektor dengan dimensi berbeda.

Q5.4 Jelaskan dalam dua atau tiga kalimat mengapa `covisitation_matrix` membagi bobot setiap pasangan dengan total bobot keluar item sumber. Apa yang terjadi pada daftar kandidat jika tidak dilakukan?

Q5.5 Pada hasil, item terdekat item-kNN dari 'Sakura Earphones Original' mencakup mi. Sebutkan penyebab yang mungkin dan satu perbaikan yang dipakai sistem produksi.

Q5.6 (Membaca kode.) Dalam `covisitation_matrix`, baris `per_session = pos.groupby(["session_id", "item_id"], as_index=False)["w"].max()` dijalankan sebelum self-merge. Apa yang dilakukannya, dan apa yang berubah jika `.max()` diganti `.sum()`?

Q5.7 (Numerik.) Item X disentuh tiga pengguna dengan kekuatan $(2, 0, 1)$ atas pengguna (u_1, u_2, u_3); item Y dengan kekuatan $(1, 1, 0)$. Hitung cosine similarity X dan Y, serta Jaccard similarity himpunan pengguna yang menyentuh keduanya. Berikan keduanya hingga tiga angka desimal.

Q5.8 (Numerik.) Seorang pengguna memiliki dua item seed. Seed A punya skor kebaruan 0.5 dan seed E punya skor kebaruan 1.0. Pada matriks ko-kunjungan, $w(A \to C) = 0.3$ dan $w(E \to C) = 0.4$. Berapa skor kandidat item C untuk pengguna ini, memakai rumus dalam `covisit`?

Q5.9 Berdasarkan tabel hasil, pernyataan mana yang benar?

A. user_knn memiliki ndcg@20 tertinggi dan juga coverage@20 tertinggi.
B. item_knn memiliki coverage@20 tertinggi, tetapi ndcg@20 lebih rendah daripada user_knn maupun covisit.
C. covisit memiliki recall@5 tertinggi.
D. pop_7d mengalahkan item_knn pada ndcg@20.

Q5.10 (Praktik langsung.) Di `book/code/ch05_cf.py`, ubah sapuan lingkungan `for k in (10, 25, 50, 100):` menjadi `for k in (50, 100, 200, 400):` dan jalankan skrip dari salinan sementara (agar tidak menimpa hasil yang sudah di-commit). Bandingkan ndcg@20 dan recall@20 yang tercetak untuk setiap k. Apa yang Anda amati tentang efek menaikkan k melebihi 100, dan mengapa?

Bab 6 — Faktorisasi matriks (*matrix factorisation*)

Pembeli yang tidak mirip siapa pun

Rina pernah mengklik satu set *pour-over* keramik, penggiling kopi manual, dan kertas filter, lalu pada suatu Selasa yang sepi juga mengklik earphone berkabel. Rekomender berbasis pengguna dari bab sebelumnya mencari tetangga (*neighbour*) yang mengklik item yang *sama*, dan di katalog berisi dua ribu produk hampir tidak ada yang melakukannya. Inilah masalah kelangkaan data (*sparsity*) dari Bab 1: metode berbasis tetangga membandingkan baris mentah dari matriks yang hampir kosong, sehingga dua pembeli dengan selera sama tetapi tanpa SKU yang sama akan memiliki cosine nol.

Faktorisasi matriks bertanya dengan cara lain: *himpunan kecil selera tersembunyi apa yang dapat menjelaskan semua klik ini sekaligus?* Setiap pembeli menjadi vektor pendek berisi kekuatan selera, setiap produk menjadi vektor pendek berisi "seberapa banyak dari tiap selera yang saya miliki", dan skor sebuah pasangan adalah hasil kali titik (*dot product*) keduanya. Kita membangunnya dengan numpy biasa agar tidak ada yang tersembunyi di balik pemanggilan pustaka.

Yang akan Anda pelajari

- Apa itu *faktor laten* (*latent factor*), dan mengapa memampatkan matriks sparse raksasa menjadi dua matriks tipis menghasilkan rekomendasi yang berguna.
- Bagaimana faktorisasi klasik berbasis rating eksplisit (yang menggerakkan era Netflix Prize) dilatih dengan *stochastic gradient descent*, dan mengapa ia kurang cocok untuk klik.
- Bagaimana ALS untuk umpan balik implisit (*implicit-feedback ALS*) memandang klik sebagai *preferensi ditambah keyakinan (confidence)*, dan cara menurunkan solusi tertutup (*closed-form*) per barisnya dari nol.
- Bagaimana BPR mengoptimalkan peringkat secara langsung dengan *pairwise loss*, lengkap dengan gradiennya yang dihitung dengan tangan.
- Cara membaca kurva *loss*, menyetel `factors`, `reg`, `alpha`, `lr`, dan membandingkan kedua model dengan baseline dari Bab 3 dan 5 pada label yang persis sama.

Intuisi: dimensi selera yang tidak pernah ditulis siapa pun

Bayangkan Anda menjelaskan para pelanggan tetap kepada barista baru. Anda tidak akan menyerahkan spreadsheet dengan centang untuk setiap item menu; Anda akan berkata "Dewi suka kopi pekat tanpa gula, Bayu suka yang manis dan berkrim susu". Beberapa frasa merangkum ratusan pembelian dan *memprediksi* perilaku.

Faktorisasi matriks mengotomatiskan langkah merangkum itu. Frasa-frasa tersebut ("pekat dan hitam", "manis dan berkrim susu") adalah **faktor laten**: dimensi selera yang tidak diberi label sebelumnya, ditemukan karena membuat data pembelian lebih mudah dijelaskan. Kata *laten* hanya berarti tersembunyi. Algoritma tidak tahu apa arti sebuah faktor; ia hanya tahu bahwa memberi setiap

pengguna sebuah nilai pada faktor 1 dan setiap item sebuah nilai pada faktor 1 membuatnya dapat merekonstruksi matriks klik dengan lebih baik.

Contoh mainan membuat mekanismenya konkret. Misalkan hanya ada dua faktor, dan kebetulan kita bisa menamainya: faktor A adalah "kegadgetan" dan faktor B adalah "kedapuran-dan-makanan". Setiap produk punya nilai pada tiap faktor, begitu pula setiap pembeli:

	faktor A (gadget)	**faktor B (dapur)**
Item		
earphone berkabel	1.0	0.0
power bank	0.9	0.1
mi instan	0.0	1.0
penggiling kopi	0.4	0.8
Pengguna		
Pencinta gadget	1.2	0.2
Pencinta kuliner	0.1	1.1

Skornya adalah hasil kali titik: untuk pencinta gadget dan earphone, $1.2 \times 1.0 + 0.2 \times 0.0 = 1.20$; untuk pencinta kuliner dan mi, $0.1 \times 0.0 + 1.1 \times 1.0 = 1.10$. Setelah diurutkan, pencinta gadget mendapat earphone (1.20), power bank (1.10), penggiling kopi (0.64), mi (0.20); pencinta kuliner mendapat mi, penggiling, power bank. Penggiling adalah *jembatan* yang memiliki sedikit dari kedua selera, sesuatu yang tidak dapat diungkapkan metode berbasis tetangga. Pencinta kuliner juga tidak perlu mengklik mi yang *sama* dengan orang lain: klik pada kacang, minyak goreng, dan penanak nasi menempatkan vektornya dekat $(0.1, 1.1)$, sehingga setiap item dengan faktor B besar akan bernilai tinggi. Satuan transfernya adalah faktor, bukan produk.

Gagasan kedua adalah **peringkat rendah (*low rank*)**. Matriks klik dengan m pengguna dan n item memiliki $m \times n$ sel. Faktorisasi dengan f faktor menggantinya dengan dua matriks yang hanya menyimpan $(m + n) \times f$ angka. Pada data bab ini, itulah selisih antara sekitar tiga juta sel ($1408 \times 2120 = 2{,}984{,}960$) dan $(1408 + 2120) \times 32 = 112{,}896$ parameter. Memeras begitu banyak informasi melalui leher botol yang sempit *memaksa* model menemukan keteraturan dalam data, itulah sebabnya kompresi dan generalisasi di sini adalah hal yang sama.

Algoritma

Kita akan membahas tiga model dengan urutan kerelevanan yang meningkat terhadap repositori ini: faktorisasi eksplisit klasik (untuk melihat bentuk gagasannya), ALS implisit (andalan untuk data klik), dan BPR (alternatif yang sadar peringkat).

Langkah nol: menyiapkan notasi

Misalkan R adalah matriks interaksi berukuran $m \times n$. Dalam buku ini R_{ui} adalah jumlah bobot event pengguna u pada item i: klik bernilai 1, add-to-cart 3, dan order 5, yaitu `schema.EVENT_WEIGHT` dari Bab 1, sehingga pengguna yang mengklik lalu membeli memiliki $R_{ui} = 6$. Sel tanpa interaksi bernilai nol, tetapi, seperti yang akan kita lihat, *nol tidak berarti tidak suka*.

Model memberi setiap pengguna sebuah vektor $x_u \in \mathbb{R}^f$ dan setiap item sebuah vektor $y_i \in \mathbb{R}^f$, disimpan sebagai baris dari dua matriks X (pengguna kali faktor) dan Y (item kali faktor). Skor prediksinya adalah

$$\hat{r}_{ui} = x_u \cdot y_i = \sum_{k=1}^{f} x_{uk}\, y_{ik}.$$

Melatih berarti memilih X dan Y. Ketiga metode hanya berbeda pada *loss yang diminimalkan*.

Faktorisasi eksplisit dan SGD ala Funk

Misalkan, sejenak, kita punya rating bintang, seperti pada Netflix Prize. Hanya sel pada himpunan teramati $\mathcal{K}$ yang membawa informasi, dan kita ingin prediksi yang cocok dengannya sambil menjaga vektor tetap kecil agar tidak menghafal noise:

$$\mathcal{L} = \sum_{(u,i)\in\mathcal{K}} \left(r_{ui} - x_u \cdot y_i\right)^2 + \lambda \left(\|x_u\|^2 + \|y_i\|^2\right).$$

Tuliskan galat prediksi $e_{ui} = r_{ui} - x_u \cdot y_i$. Mendiferensialkan satu suku terhadap x_u menghasilkan $-2\,e_{ui}\,y_i + 2\lambda x_u$, dan serupa untuk y_i. Dengan melipat konstanta 2 ke dalam laju belajar (*learning rate*) γ, satu langkah *stochastic gradient descent* pada satu rating teramati adalah:

$$x_u \leftarrow x_u + \gamma\left(e_{ui}\, y_i - \lambda x_u\right), \qquad y_i \leftarrow y_i + \gamma\left(e_{ui}\, x_u - \lambda y_i\right).$$

Perhatikan betapa masuk akalnya pembaruan ini. Jika model memprediksi terlalu rendah ($e_{ui} > 0$), vektor pengguna bergerak *mendekati* vektor item dan vektor item mendekati vektor pengguna, sehingga hasil kali titiknya membesar. Jika prediksi terlalu tinggi, keduanya saling menjauh.

Contoh hitung 1: satu langkah SGD. Misalkan $x_u = (2.0, 1.0)$, $y_i = (1.5, 0.5)$, $r_{ui} = 5$, $\gamma = 0.05$, $\lambda = 0$. Prediksinya 3.5, jadi $e = 1.5$. Maka $x_u' = (2.1125, 1.0375)$ dan, dengan memakai x_u yang lama, $y_i' = (1.65, 0.575)$. Prediksi barunya adalah $2.1125 \times 1.65 + 1.0375 \times 0.575 \approx 4.08$. Galat menyusut dari 1.5 menjadi sekitar 0.92 dalam satu langkah. Ulangi untuk jutaan rating teramati, beberapa kali putaran, dan vektor-vektornya akan mengendap. Inilah pada dasarnya prosedur yang dipublikasikan Simon Funk selama Netflix Prize, dan alasan metode ini sering disebut "Funk SVD", meskipun bukan *singular value decomposition* sejati: SVD sejati membutuhkan setiap sel terisi, sedangkan di sini kita sengaja mengabaikan sel yang kosong.

> **Tahukah kamu?**
>
> **Melewati sel yang kosong.** Ketika Simon Funk menjelaskan pendekatannya pada Desember 2006, data latih Netflix berupa matriks sekitar 500,000 pengguna kali 17,000 film, sekitar 8.5 miliar sel, dan hanya satu dari delapan puluh lima sel yang berisi rating. Trik utamanya adalah melewati sel kosong sepenuhnya dan menjalankan gradient descent hanya pada sel teramati, sehingga sebuah laptop dapat menyelesaikan satu putaran pelatihan penuh atas 100 juta rating dalam hitungan detik.
>
> Sumber: Funk, "Netflix Update: Try This at Home" (blog sifter.org), 2006

Mengapa faktorisasi eksplisit kurang cocok untuk klik

Umpan balik implisit **tidak memiliki negatif yang teramati**: jika Rina tidak mengklik mouse gaming, bisa jadi ia tidak menyukainya atau memang belum pernah melihatnya. Mencocokkan hanya sel bukan-nol membuat prediksi melayang naik; memperlakukan nol sebagai rating 0 berarti menyatakan sebagian besar katalog dibenci. Dan angkanya bermakna lain: bobot 6 (klik plus order) menyatakan *keyakinan*, bukan besarnya preferensi.

Hu, Koren, dan Volinsky menyelesaikan kedua masalah itu dengan satu pembingkaian ulang. Pecah setiap sel menjadi dua besaran:

- **Preferensi** biner $p_{ui} = 1$ jika $R_{ui} > 0$, dan 0 jika tidak.
- **Keyakinan (*confidence*)** $c_{ui} = 1 + \alpha R_{ui}$.

Preferensi menjawab "apakah pengguna ini menunjukkan minat?". Keyakinan menjawab "seberapa besar loss harus peduli agar sel ini benar?". Sel teramati dengan banyak bukti mendapat keyakinan besar; sel tak teramati tetap pada nilai dasar $c = 1$: kita masih *lemah* meyakini bahwa itu nol, karena pada katalog yang sangat besar kebanyakan hal yang belum terlihat memang tidak menarik, tetapi kita jauh kurang yakin dibanding terhadap sebuah pembelian.

Loss yang baru berjalan pada **semua** $m \times n$ sel:

$$\mathcal{L}_{\text{ALS}} = \sum_{u,i} c_{ui} \left(p_{ui} - x_u \cdot y_i \right)^2 + \lambda \left(\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2 \right).$$

Karena menjumlahkan seluruh sel, SGD harus menyentuh tiga juta suku per epoch padahal hanya 18,256 yang tidak sepele. Trik pada bagian berikutnya adalah bahwa pengoptimal yang berbeda dapat memanfaatkan persis ketimpangan itu.

> **Tahukah kamu?**
>
> **Model faktor berbobot keyakinan dengan solver berwaktu linear.** Makalah ICDM 2008 oleh Hu, Koren, dan Volinsky mengusulkan untuk memperlakukan data implisit sebagai indikasi preferensi positif dan negatif "associated with vastly varying confidence levels", bersama prosedur optimasi yang biayanya tumbuh linear terhadap ukuran data. Mereka menerapkannya pada rekomender acara televisi.
>
> Sumber: Hu et al., ICDM, 2008

ALS: menurunkan solusi tertutup

Loss $\mathcal{L}_{\text{ALS}}$ tidak konveks terhadap (X, Y) secara bersamaan, karena memuat hasil kali $x_u \cdot y_i$. Tetapi tetapkan salah satu sisi, dan ia menjadi masalah kuadrat terkecil berbobot yang sepenuhnya biasa pada sisi lainnya. Pengamatan itulah yang memberi algoritma ini namanya: **alternating least squares** (kuadrat terkecil bergantian). Tahan Y tetap dan selesaikan setiap vektor pengguna secara eksak; lalu tahan X tetap dan selesaikan setiap vektor item secara eksak; ulangi.

Ambil satu pengguna u dengan Y tetap. Satu-satunya suku yang melibatkan x_u adalah

$$J(x_u) = \sum_i c_{ui}\,(p_{ui} - x_u \cdot y_i)^2 + \lambda \|x_u\|^2.$$

Setel gradiennya ke nol:

$$\frac{\partial J}{\partial x_u} = -2\sum_i c_{ui}\,(p_{ui} - x_u \cdot y_i)\,y_i + 2\lambda x_u = 0.$$

Uraikan $x_u \cdot y_i = y_i^\top x_u$ dan kumpulkan suku-suku yang memuat x_u:

$$\Big(\sum_i c_{ui}\, y_i y_i^\top + \lambda I\Big) x_u = \sum_i c_{ui}\, p_{ui}\, y_i.$$

Dalam bentuk matriks, dengan C^u matriks diagonal berisi keyakinan pengguna u atas semua item, dan $p(u)$ vektor preferensinya:

$$\boxed{\,x_u = \left(Y^\top C^u Y + \lambda I\right)^{-1} Y^\top C^u\, p(u)\,}$$

Inilah solusi tertutup. Matriks dalam kurung berukuran $f \times f$, simetris dan, berkat λI, positif definit, sehingga solusinya unik dan setiap solve hanya dapat *menurunkan* loss, tidak pernah menaikkannya. Itulah sebabnya kurva pelatihan yang akan kita lihat nanti turun secara monoton tanpa penyetelan learning rate sama sekali. Vektor item mengikuti secara simetris: tukar peran pengguna dan item lalu gunakan kolom dari C dan P.

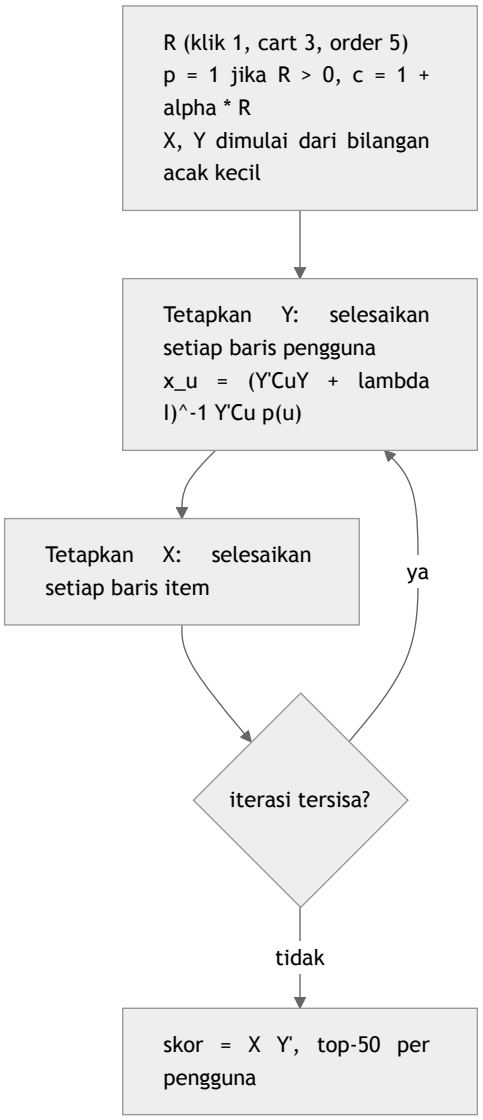
Trik matriks Gram

Menghitung $Y^\top C^u Y$ secara naif membutuhkan biaya $O(f^2 n)$ per pengguna, karena C^u memiliki n entri diagonal. Dengan $n = 2120$ item dan 1408 pengguna, itu pekerjaan yang banyak terbuang, sebab hampir semua entrinya bernilai 1. Hu dkk. memecah matriksnya:

$$Y^\top C^u Y = Y^\top Y + Y^\top (C^u - I)\, Y.$$

Suku pertama, **matriks Gram** $Y^\top Y$, tidak bergantung pada pengguna, sehingga dihitung sekali per sapuan (*sweep*). Suku kedua hanya memiliki entri bukan-nol untuk n_u item yang benar-benar disentuh pengguna (di mana $c_{ui} - 1 = \alpha R_{ui} > 0$), sehingga biayanya $O(f^2 n_u)$. Demikian pula sisi kanan runtuh menjadi jumlah atas item yang disentuh pengguna karena $p_{ui} = 0$ di tempat lain. Total biaya satu sapuan jadi sebanding dengan jumlah sel *teramati*, bukan $m \times n$, dan itulah yang membuat ALS praktis pada data nyata.

Contoh hitung 2: satu solve ALS dengan tangan. Ambil $f = 1$ faktor, faktor item $y = (1, 2, 3)$, seorang pengguna yang menyentuh item 1 dan 2 dengan $R = (1, 2)$ (item 3 tak tersentuh), $\alpha = 2$, $\lambda = 0.5$. Maka $p = (1, 1, 0)$ dan $c = (3, 5, 1)$. Suku Gram-nya $Y^\top Y = 14$, koreksinya $Y^\top (C - I) Y = 2 \cdot 1 + 4 \cdot 4 = 18$, sehingga sisi kiri adalah $14 + 18 + 0.5 = 32.5$ (menjumlahkan $c\,y^2 + \lambda$ secara langsung memberi hasil sama). Sisi kanan adalah $\sum c\,p\,y = 3 + 10 = 13$, jadi $x_u = 13 / 32.5 = 0.4$ dan prediksinya $(0.4, 0.8, 1.2)$. Dengan satu faktor, model tidak dapat memuaskan semua orang, itulah sebabnya model nyata memakai 32; struktur yang perlu diingat adalah bahwa satu pengguna merupakan regresi ridge berbobot kecil atas item yang disentuhnya.



Gambar 6.1: *Satu putaran pelatihan ALS. Setiap kotak di dalam loop adalah solve kuadrat terkecil yang eksak, sehingga tidak ada learning rate yang perlu disetel.*

BPR: optimalkan urutan, bukan nilainya

ALS meminimalkan galat kuadrat pada target 0/1. Tetapi kita tidak pernah menampilkan prediksi 0/1 kepada pengguna; kita menampilkan *daftar berperingkat*. Rendle dan kolega bertanya: mengapa tidak mengoptimalkan peringkat secara langsung? Data latih menjadi **triplet** (u, i, j) dengan i item yang berinteraksi dengan pengguna u dan j item yang tidak. Model harus memberi skor i di atas j.

Misalkan selisihnya $\hat{x}_{uij} = x_u \cdot y_i - x_u \cdot y_j = x_u \cdot (y_i - y_j)$. Tekan dengan fungsi logistik $\sigma(z) = 1/(1+e^{-z})$ untuk mendapatkan probabilitas model mengurutkan pasangan dengan benar. Objektif BPR memaksimalkan log probabilitas itu atas triplet yang disampel, dengan regularisasi L2 yang datang dari prior Gaussian pada parameter, yang merupakan "Bayesian" dalam namanya:

$$\mathcal{L}_{\text{BPR}} = -\sum_{(u,i,j)} \ln \sigma\big(\hat{x}_{uij}\big) + \lambda \lVert \Theta \rVert^2.$$

Ini adalah pengganti AUC yang mulus dan terdiferensialkan, yaitu probabilitas bahwa sebuah positif acak mengungguli sebuah negatif acak untuk pengguna yang sama.

Gradiennya menyenangkan karena pendek. Karena $\frac{d}{dz}\ln\sigma(z) = 1 - \sigma(z) = \sigma(-z)$, dan $\hat{x}_{uij}$ linear terhadap setiap vektor:

$$\frac{\partial \hat{x}_{uij}}{\partial x_u} = y_i - y_j, \qquad \frac{\partial \hat{x}_{uij}}{\partial y_i} = x_u, \qquad \frac{\partial \hat{x}_{uij}}{\partial y_j} = -x_u.$$

Jadi satu langkah ascent dengan learning rate η dan $s = \sigma(-\hat{x}_{uij})$ adalah

$$x_u \mathrel{+}= \eta\big(s\,(y_i - y_j) - \lambda x_u\big), \quad y_i \mathrel{+}= \eta\big(s\,x_u - \lambda y_i\big), \quad y_j \mathrel{+}= \eta\big(-s\,x_u - \lambda y_j\big).$$

Skalar s adalah kuncinya. Ketika pasangan sudah terurut benar dengan selisih lebar, $\hat{x}_{uij}$ besar, $s \approx 0$, dan triplet itu nyaris tidak mengubah apa pun. Ketika pasangan salah atau seri, s mendekati 0.5 atau lebih dan pembaruannya besar. Model mencurahkan upayanya pada pasangan yang saat ini ia dapatkan salah, intuisi yang sama dengan gradien LambdaMART yang akan Anda temui di Bab 10.

Contoh hitung 3: satu langkah BPR. Misalkan $x_u = (0.5, -0.2)$, $y_i = (1.0, 0.5)$ (diklik), $y_j = (0.2, 1.0)$ (acak), $\eta = 0.1$, $\lambda = 0$. Selisihnya $y_i - y_j = (0.8, -0.5)$, sehingga gap-nya $0.5 \cdot 0.8 + (-0.2)(-0.5) = 0.5$. Maka $\sigma(0.5) \approx 0.6225$, loss-nya $-\ln 0.6225 \approx 0.474$ dan $s = 0.3775$. Pembaruannya $x_u \approx (0.5302, -0.2189)$, $y_i \approx (1.0189, 0.4924)$ dan $y_j \approx (0.1811, 1.0076)$. Gap baru sekitar 0.557 dan loss sekitar 0.453: satu langkah kecil menggerakkan pasangan itu ke arah yang benar.

Satu detail praktis. Saat inisialisasi semua vektor mendekati nol, sehingga setiap gap mendekati 0 dan $\sigma(0) = 0.5$, artinya setiap triplet memiliki loss $-\ln 0.5 = \ln 2 \approx 0.693$. Ketika melatih BPR, loss yang *dimulai* di sekitar 0.693 adalah pertanda baik bahwa kodenya terpasang dengan benar.

Hyperparameter dalam satu tempat

- `factors` f: terlalu rendah menyebabkan underfit (selera tidak dapat dipisahkan), terlalu tinggi menyebabkan overfit dan memperlambat pelatihan.
- `reg` λ: terlalu rendah menghafal pengguna yang jarang datanya, terlalu tinggi menyusutkan setiap skor ke arah 0.
- `alpha` α (ALS): terlalu rendah membuat sel teramati dan tak teramati tampak serupa; terlalu tinggi membuat satu klik mendominasi dan menyeret rekomendasi ke item populer.
- `iters` / `epochs`: berhenti ketika loss mendatar; BPR juga membutuhkan `lr` yang wajar (terlalu tinggi berosilasi) dan cukup banyak negatif yang disampel.

Skrip kita memakai `factors = 32`, `reg = 0.1`, `alpha = 20`, 15 iterasi untuk ALS; dan `lr = 0.05`, `reg = 0.01`, 30 epoch, satu negatif untuk BPR. Ini nilai bawaan, bukan optimum hasil penyetelan.

Penjelasan kode

Semua kode berada di `book/code/ch06_mf.py`. Ia melatih kedua model pada matriks pengguna-kali-item yang persis sama dengan Bab 3 dan 5 lalu menyerahkan skornya ke helper evaluasi bersama. Kita mulai dari matriksnya.

Membangun matriks interaksi

book/code/common.py · interaction_matrix

```python
def interaction_matrix(hist: data.History) -> tuple[sp.csr_matrix, pd.Index, pd.Index]:
    """User x item matrix of summed EVENT_WEIGHT (click 1, cart 3, order 5)."""
    pos = hist.interactions()
    user_index = pd.Index(np.sort(pos["user_id"].unique()), name="user_id")
    item_index = pd.Index(np.sort(pos["item_id"].unique()), name="item_id")
    agg = pos.groupby(["user_id", "item_id"])["w"].sum().reset_index()
    m = sp.csr_matrix(
        (agg["w"].to_numpy(), (user_index.get_indexer(agg["user_id"]),
item_index.get_indexer(agg["item_id"]))),
        shape=(len(user_index), len(item_index)),
    )
    return m, user_index, item_index
```

`hist.interactions()` mengembalikan satu baris per event positif (klik, cart, atau order) beserta bobotnya, diambil langsung dari kamus `EVENT_WEIGHT`. Groupby menjumlahkannya per pasangan pengguna-item, sehingga pengguna yang mengklik sebuah item dua kali dan membelinya sekali memiliki $R = 1 + 1 + 5 = 7$. Kita lalu memetakan id mentah ke posisi baris dan kolom yang rapat dengan `pd.Index.get_indexer` dan membangun matriks sparse CSR. Kedua indeks yang dikembalikan sangat penting nanti: keduanya menerjemahkan "baris 17" kembali menjadi "pengguna 61".

Skrip ch06 memadatkan matriks ini dengan `.toarray()`: 1408 pengguna kali 2120 item bertipe float64 hanya beberapa puluh megabyte, jadi tidak masalah untuk skrip pengajaran. Pada skala produksi Anda tidak pernah melakukan ini.

ALS: seluruh algoritma dalam satu fungsi

book/code/ch06_mf.py · als_implicit

```python
def als_implicit(R: np.ndarray, factors=FACTORS, reg=0.1, alpha=20.0, iters=15, seed=0):
    """Minimise  sum_ui c_ui (p_ui - x_u . y_i)^2 + reg (|X|^2 + |Y|^2)
    with p = 1[r > 0] and c = 1 + alpha * r, by alternating exact ridge solves per row."""
    rng = np.random.default_rng(seed)
    n_u, n_i = R.shape
    P = (R > 0).astype(np.float64)
    C = 1.0 + alpha * R
    X = rng.normal(0, 0.1, (n_u, factors))
    Y = rng.normal(0, 0.1, (n_i, factors))
    eye = reg * np.eye(factors)

    def solve(fixed, conf, pref):
        gram = fixed.T @ fixed  # computed once per sweep: the HKV trick
        out = np.empty((conf.shape[0], factors))
        for r in range(conf.shape[0]):
            nz = np.flatnonzero(conf[r] > 1.0)
            Fn = fixed[nz]
            cu = conf[r, nz]
            A = gram + Fn.T @ ((cu - 1.0)[:, None] * Fn) + eye
            b = Fn.T @ (cu * pref[r, nz])
            out[r] = np.linalg.solve(A, b)
        return out

    losses = []
    for _ in range(iters):
        X = solve(Y, C, P)
        Y = solve(X, C.T, P.T)
        err = P - X @ Y.T
        losses.append(float((C * err**2).sum() + reg * ((X**2).sum() + (Y**2).sum())))
    return X, Y, losses
```

Baca fungsi ini dari atas ke bawah, dan cocokkan setiap bagian dengan penurunannya.

- `P` dan `C` adalah matriks preferensi dan keyakinan; `X` dan `Y` dimulai dari noise Gaussian kecil (semua nol akan membuat setiap solve identik).
- `solve(fixed, conf, pref)` adalah solusi tertutup untuk setiap baris pada satu sisi. `gram = fixed.T @ fixed` adalah $Y^\top Y$, dihitung sekali per sapuan: baris itulah trik Hu-Koren-Volinsky.
- Per baris, `nz` menemukan item yang disentuh, `A = gram + Fn.T @ ((cu - 1.0)[:, None] * Fn) + eye` adalah $Y^\top Y + Y^\top (C^u - I) Y + \lambda I$, dan `b` adalah $Y^\top C^u p(u)$. `np.linalg.solve(A, b)` menghindari pembentukan invers.

Loop utama lalu memanggil `solve(Y, C, P)` untuk pengguna dan `solve(X, C.T, P.T)` untuk item. Mentranspos C dan P sudah cukup untuk memakai ulang fungsi yang sama bagi sisi lainnya. Setelah setiap sapuan, kode mencatat objektif penuh dengan operasi numpy padat, `(C * err**2).sum() + reg * (...)`, yang merupakan loss yang diplot nanti.

Satu catatan jujur: `C` dan `err` di sini padat. Itu tidak masalah untuk lab dengan tiga juta sel, tetapi tidak untuk jutaan pengguna. ALS produksi menjaga R tetap sparse sepanjang waktu dan tidak pernah membangun C.

BPR: sampling, gap, dan ketiga gradien

book/code/ch06_mf.py · bpr

```python
def bpr(R: np.ndarray, factors=FACTORS, lr=0.05, reg=0.01, epochs=30, batch=256, seed=0):
    """Bayesian Personalised Ranking: for each positive (u, i) draw a negative j and push
    x_u.y_i above x_u.y_j (maximise log sigmoid of the gap). Mini-batch SGD in numpy."""
    rng = np.random.default_rng(seed)
    n_u, n_i = R.shape
    pos_mask = R > 0
    users, items = np.nonzero(pos_mask)
    X = rng.normal(0, 0.1, (n_u, factors))
    Y = rng.normal(0, 0.1, (n_i, factors))
    losses = []
    for _ in range(epochs):
        order = rng.permutation(len(users))
        total = 0.0
        for s in range(0, len(order), batch):
            b = order[s : s + batch]
            u, i = users[b], items[b]
            j = rng.integers(0, n_i, len(b))
            for _retry in range(5):  # resample negatives that are actually positives
                bad = pos_mask[u, j]
                if not bad.any():
                    break
                j[bad] = rng.integers(0, n_i, bad.sum())
            xu, yi, yj = X[u], Y[i], Y[j]
            gap = (xu * (yi - yj)).sum(1)
            sig = 1.0 / (1.0 + np.exp(gap))  # = 1 - sigmoid(gap) = -d loss / d gap
            total += float(np.log1p(np.exp(-gap)).sum())
            gu = sig[:, None] * (yi - yj) - reg * xu
            gi = sig[:, None] * xu - reg * yi
            gj = -sig[:, None] * xu - reg * yj
            np.add.at(X, u, lr * gu)
            np.add.at(Y, i, lr * gi)
            np.add.at(Y, j, lr * gj)
        losses.append(total / len(users))
    return X, Y, losses
```

Fungsi ini sesuai satu-satu dengan persamaan BPR.

- `np.nonzero(pos_mask)` mendaftar setiap pasangan positif; satu epoch mengunjungi masing-masing sekali dalam mini-batch berukuran 256. `j` adalah item acak seragam, diundi ulang hingga lima kali jika ternyata positif sungguhan.
- `gap` adalah $\hat{x}_{uij}$ untuk batch; `sig = 1.0 / (1.0 + np.exp(gap))` adalah $\sigma(-\text{gap})$, pengali langkah s; `np.log1p(np.exp(-gap))` mengakumulasi loss.
- `gu`, `gi`, `gj` adalah ketiga gradien (tanda minus pada `gj` mendorong negatif menjauh). `np.add.at` diperlukan karena satu pengguna dapat muncul beberapa kali dalam sebuah batch.

Dari skor ke daftar top-50

book/code/dense_recs.py · scores_to_recs

```
def scores_to_recs(scores: np.ndarray, users: np.ndarray, item_ids: np.ndarray, n: int =
common.TOP_N):
    """scores: (len(users), len(item_ids)); only score > 0 counts as 'the model has an opinion'.

    Returns (recs DataFrame, stats dict) where stats describes how much padding was needed.
    """
    scores = np.where(np.isfinite(scores), scores, 0.0)
    flag = scores > 0
    row_max = np.maximum(scores.max(axis=1, keepdims=True), 1e-12)
    pop = pop7_vector(item_ids)[None, :]
    key = flag * 2.0 + np.where(flag, scores / row_max, 0.0) + np.where(flag, 0.0, pop)
    top = np.argsort(-key, axis=1, kind="stable")[:, :n]
    n_model = np.minimum(flag.sum(axis=1), n)
    recs = pd.DataFrame({"user_id": np.repeat(users, n), "item_id": item_ids[top].ravel()})
    stats = {
        "users": int(len(users)),
        "users_without_model_scores": int((n_model == 0).sum()),
        "users_padded_partially": int(((n_model > 0) & (n_model < n)).sum()),
        "share_of_slots_from_popularity_padding": float(1 - n_model.sum() / (n * len(users))),
    }
    return recs, stats
```

Kedua model menghasilkan matriks skor padat `users × items`, `X[users] @ Y[items].T`; helper ini mengubahnya menjadi daftar top-50 yang diharapkan `common.evaluate`, dengan mengisi pengguna tanpa riwayat yang dapat dipakai menggunakan popularitas 7 hari. Ia tidak membuang item yang sudah disentuh pengguna; filter itu menjadi urusan Bab 11.

Jalankan

Dari direktori `book/`:

```
uv run python code/ch06_mf.py
```

Proses berjalan beberapa detik saja karena matriksnya kecil. ALS lebih dulu. Setiap baris mencetak objektif penuh setelah satu sapuan bergantian:

KELUARAN TERMINAL

```
=== ALS (implicit): factors 32, lambda 0.1, alpha 20, 15 iterations ===
trained in 1.7s
iteration  1  loss       225190.5
iteration  2  loss       147262.9
iteration  3  loss       131616.4
iteration  4  loss       124535.6
iteration  5  loss       120478.2
iteration  6  loss       117899.3
iteration  7  loss       116156.7
iteration  8  loss       114928.5
iteration  9  loss       114035.7
iteration 10  loss       113371.0
iteration 11  loss       112865.4
iteration 12  loss       112473.0
iteration 13  loss       112162.2
iteration 14  loss       111911.5
iteration 15  loss       111705.9
```

Lalu epoch pembuka dan penutup BPR. Loss yang dilaporkan adalah rata-rata $-\ln \sigma(\text{gap})$ atas triplet yang disampel pada epoch tersebut, jadi ini besaran waktu pelatihan, bukan skor peringkat:

KELUARAN TERMINAL

```
=== BPR: factors 32, lr 0.05, reg 0.01, 30 epochs, 1 negative per positive ===
trained in 0.8s
epoch  1  mean -log sigmoid(gap) 0.6937
epoch  2  mean -log sigmoid(gap) 0.6864
epoch  3  mean -log sigmoid(gap) 0.6771
epoch  4  mean -log sigmoid(gap) 0.6667
epoch  5  mean -log sigmoid(gap) 0.6530
epoch  6  mean -log sigmoid(gap) 0.6331
epoch  7  mean -log sigmoid(gap) 0.6061
epoch  8  mean -log sigmoid(gap) 0.5707
epoch  9  mean -log sigmoid(gap) 0.5294
epoch 10  mean -log sigmoid(gap) 0.4835
epoch 11  mean -log sigmoid(gap) 0.4398
```

KELUARAN TERMINAL

```
epoch 20  mean -log sigmoid(gap) 0.1918
epoch 21  mean -log sigmoid(gap) 0.1770
epoch 22  mean -log sigmoid(gap) 0.1681
epoch 23  mean -log sigmoid(gap) 0.1555
epoch 24  mean -log sigmoid(gap) 0.1471
epoch 25  mean -log sigmoid(gap) 0.1365
epoch 26  mean -log sigmoid(gap) 0.1320
epoch 27  mean -log sigmoid(gap) 0.1222
epoch 28  mean -log sigmoid(gap) 0.1188
epoch 29  mean -log sigmoid(gap) 0.1132
epoch 30  mean -log sigmoid(gap) 0.1094
```

Terakhir, berapa banyak pengguna evaluasi yang tidak memiliki vektor yang dapat dipakai dan bagaimana kedua model mendapat skor pada minggu uji:

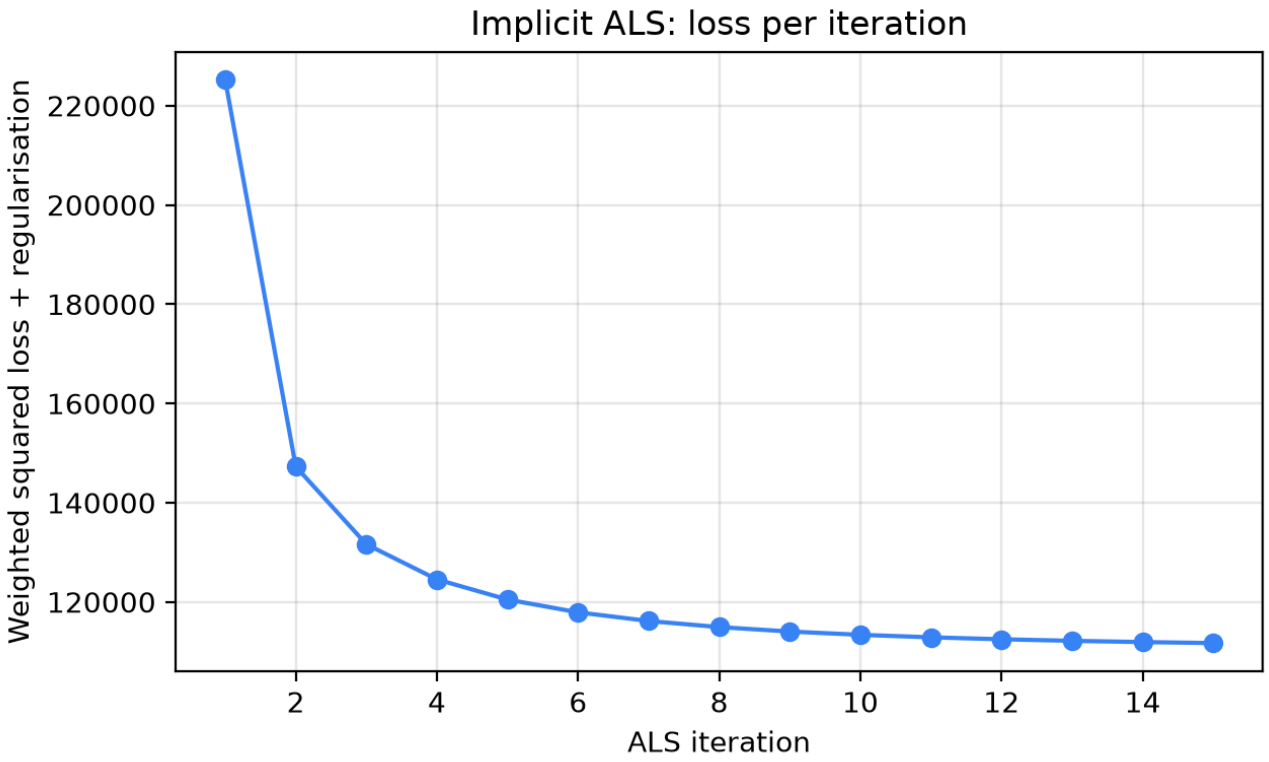
KELUARAN TERMINAL

```
=== Padding with 7-day popularity ===
als_implicit  users without scores   28  slots padded 3.0%
bpr           users without scores   28  slots padded 3.0%

=== Results (top-50 lists, evaluated at K=20) ===
              recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20  coverage@20
method
als_implicit    0.1145     0.1816     0.2715     0.3798           0.4122   0.1581       0.5431
bpr             0.0970     0.1542     0.2218     0.3274           0.3203   0.1379       0.4267
```

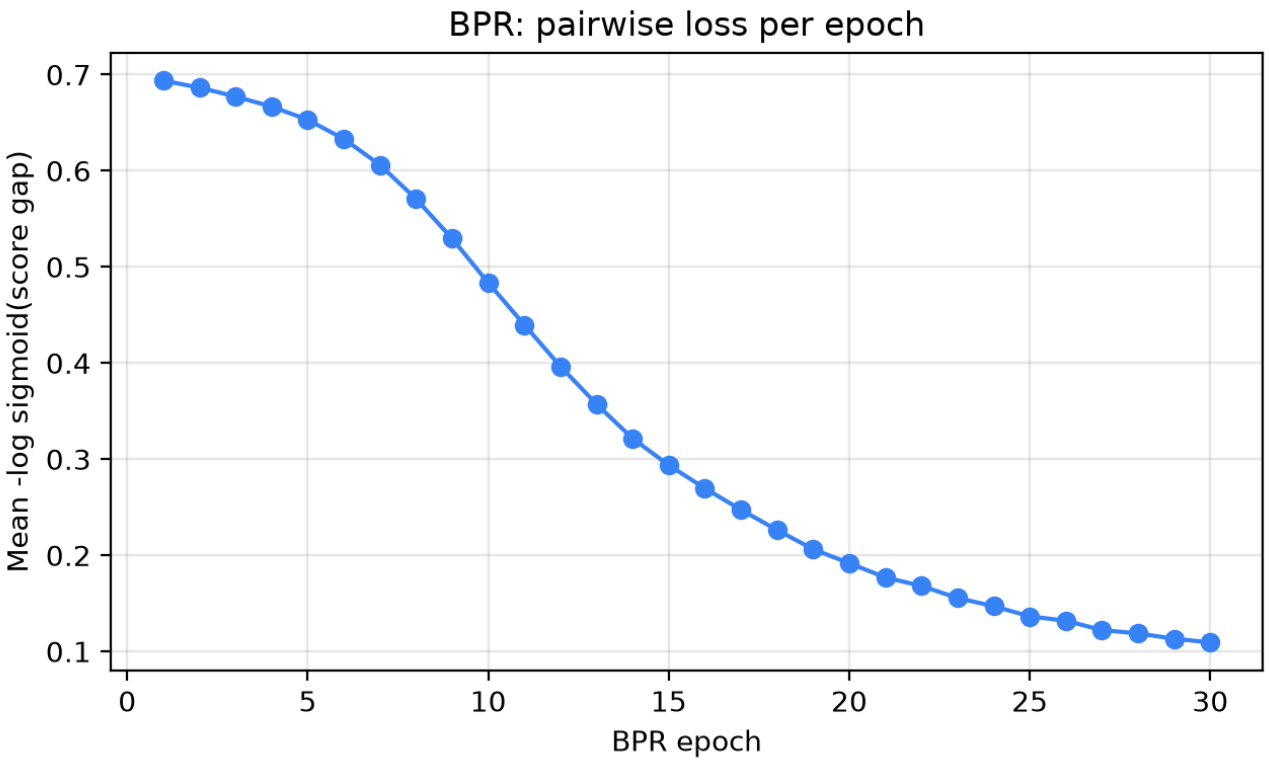
Hasil

Membaca kurva loss



Gambar 6.2: *ALS implisit: loss kuadrat berbobot plus regularisasi per iterasi. Sapuan bergantian pertama menghilangkan sebagian besar loss, lalu kurva mendatar.*

Loss ALS turun pada **setiap** iterasi, seperti yang dijanjikan penurunan rumusnya: dari 225190.5 pada iterasi 1 menjadi 147262.9 pada iterasi 2, 120478.2 pada iterasi 5, dan 111911.5 lalu 111705.9 pada iterasi 14 dan 15 (selisih sekitar 0.2%). Kurva sudah mendatar, jadi 15 iterasi sudah melewati titik tekuknya.



Gambar 6.3: *BPR: rata-rata loss pairwise per epoch. Dimulai dekat ln 2 dan menurun pelan di awal, lalu cepat, lalu pelan lagi.*

Kurva BPR dimulai pada 0.6937, pada dasarnya $\ln 2 \approx 0.693$: saat inisialisasi setiap gap mendekati nol. Nyaris tidak ada yang terjadi selama lima epoch (0.6530 pada epoch 5) karena vektor kecil menghasilkan gradien kecil; lalu loss turun cepat (0.4835 pada epoch 10, 0.2937 pada epoch 15) dan mencapai 0.1094 pada epoch 30. Loss **masih turun** pada epoch 30, jadi model belum konvergen.

Kualitas peringkat dibanding semua yang sudah dibahas

Loss pelatihan bukanlah kualitas peringkat, jadi kita mengevaluasi kedua model pada minggu uji dan metrik yang sama seperti bab-bab sebelumnya (baris ch06 ada di tabel di atas). Sebagai pembanding, metode Bab 5 dan baseline Bab 3 terbaik pada label yang identik:

KELUARAN TERMINAL

```
=== Results (top-50 lists, evaluated at K=20) ===
           recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20  coverage@20
method
pop_7d       0.0310     0.0493     0.0786     0.1335           0.1066   0.0475       0.0087
user_knn     0.1230     0.1755     0.2273     0.3163           0.3456   0.1662       0.4158
item_knn     0.0460     0.1001     0.1692     0.2934           0.2772   0.0925       0.8399
covisit      0.0795     0.1319     0.2069     0.3220           0.2999   0.1197       0.6460
```

KELUARAN TERMINAL

```
pop_in_my_categories    0.0926     0.1425     0.2173     0.3529           0.3050   0.1302
0.2164
```

Beberapa kesimpulan mengikuti langsung dari angka-angka ini.

1. Faktorisasi menghancurkan popularitas. ALS memiliki ndcg@20 0.1581 dibanding 0.0475 untuk `pop_7d`, lebih dari tiga kali lipat.

2. ALS mengalahkan BPR di sini, tetapi tidak meyakinkan. ALS mendapat ndcg@20 0.1581 dan recall@20 0.2715; BPR mendapat 0.1379 dan 0.2218. Loss BPR masih turun dan ia punya lebih banyak knob, jadi bacalah ini sebagai "dengan nilai bawaan ini ALS lebih baik dan lebih murah".

3. ALS tidak mengalahkan user-kNN pada ndcg. User-kNN dengan 50 tetangga mencapai 0.1662, dan bahkan `pop_in_my_categories` mencapai 0.1302, dekat dengan 0.1379 milik BPR. Pada katalog kecil, sinyal tetangga dan kategori adalah baseline yang kuat.

4. Metrik-metrik itu berbeda secara informatif. ALS memiliki recall@20 (0.2715 vs 0.2273) dan order_recall@20 (0.4122 vs 0.3456) yang lebih tinggi daripada user-kNN tetapi ndcg@20 yang lebih rendah. Ia menemukan lebih banyak item yang tepat dan mengurutkannya kurang tajam, itulah sebabnya generator kandidat dinilai dengan recall dan ranker dengan ndcg.

5. Coverage dan pengguna dingin. Coverage@20 adalah 0.5431 untuk ALS dan 0.4267 untuk BPR dibanding 0.0087 untuk `pop_7d`. Tetapi 28 pengguna evaluasi tidak memiliki vektor (tanpa interaksi sebelum cutoff) dan diisi dengan popularitas 7 hari, sekitar 3.0% slot: faktorisasi tidak dapat memberi skor kepada pengguna yang belum pernah ia lihat (Bab 12).

> **Tahukah kamu?**
>
> **Sejuta dolar untuk sepuluh persen.** Netflix Prize diluncurkan pada Oktober 2006 dan menantang dunia untuk meningkatkan akurasi prediksi rating Netflix sebesar 10%. Hadiah diberikan pada September 2009 kepada tim "BellKor's Pragmatic Chaos", yang submisi pemenangnya memperbaiki sistem yang ada sebesar 10.06%, dilaporkan tiba kurang dari setengah jam sebelum tenggat kontes. Deskripsi metode yang dipublikasikan tim itu menonjolkan dua keluarga teknik: model tetangga dan model faktor laten seperti yang ada di bab ini.
>
> Sumber: Sarno, Los Angeles Times, 2009

> **Tahukah kamu?**
>
> **Mengoptimalkan peringkat secara langsung.** Dalam makalah UAI 2009 mereka, Rendle dan kolega mencatat bahwa metode populer untuk rekomendasi item dari umpan balik implisit, seperti faktorisasi matriks dan adaptive k-nearest-neighbour, "are designed for the item prediction task of personalized ranking" namun "none of them is directly optimized for ranking". Kriteria BPR-Opt mereka, yang diturunkan sebagai penaksir maximum posterior, dilatih dengan stochastic gradient descent menggunakan bootstrap sampling dan dapat diterapkan pada kedua keluarga model.
>
> Sumber: Rendle et al., UAI, 2009

Jebakan dan cara produksi melakukannya

Menyetel pada set uji Anda. Menyapu `alpha`, `factors`, dan `reg` terhadap label evaluasi berarti menyesuaikan diri pada data evaluasi. Setel pada jendela validasi yang lebih awal dan evaluasi sekali pada minggu uji; kami membiarkan nilai bawaan tanpa penyetelan karena alasan ini.

Tidak teramati bukan berarti negatif. ALS memberi bobot 1 pada sel kosong dan BPR menyampel sel kosong secara seragam, tetapi item bagus yang tidak pernah dilihat pengguna tetap dihukum. Sistem produksi memakai negatif yang *sadar paparan* (item yang ditampilkan namun diabaikan, yaitu event

`impression`, yang dicatat repositori ini) dan sampling berbobot popularitas, karena sampling seragam memilih item langka sebagai negatif jauh lebih sering daripada item populer.

Matriks padat tidak skalabel. Kode kita membangun matriks C, P, dan galat yang padat. Pada skala Shopee, produksi menjaga R tetap sparse, memproses pengguna dalam blok seukuran GPU, dan men-*shard* X dan Y; pustaka sumber terbuka `implicit` dan Spark MLlib mengimplementasikan bentuk sparse-nya.

Fold-in. Dengan Y dibekukan, vektor seorang pengguna adalah solusi tertutup yang dapat dihitung ulang jauh di bawah satu milidetik dari klik terbarunya, sehingga model yang dilatih tiap malam tetap dapat bereaksi pada perilaku segar (bandingkan desain on-request repositori ini).

Dot product, bukan jarak. Panjang vektor berpengaruh pada $x_u \cdot y_i$; pustaka ANN membutuhkan varian *maximum-inner-product* (Bab 7).

Posisinya dalam repositori ini. Layanan rekomender tidak memakai ALS maupun BPR saat ini: sumber kandidatnya adalah empat retriever dari Bab 8 dan rankernya LightGBM (Bab 10). Perluasan yang wajar adalah melatih ALS saat retrain dan memakai dot product sebagai sumber kandidat tambahan sekaligus fitur ranker (lihat daftar periksa di Bab 14).

Ringkasan

- Faktorisasi matriks memberi setiap pengguna dan item vektor pendek berisi **faktor laten** dan menilai sebuah pasangan dengan dot product-nya, memampatkan sekitar tiga juta sel menjadi kira-kira seratus ribu parameter.
- Faktorisasi eksplisit (Funk SVD) mencocokkan rating teramati dengan SGD; ia kurang cocok untuk klik, di mana sel kosong bukan nol dan bobot menyatakan keyakinan.
- **ALS implisit** memecah setiap sel menjadi preferensi $p = 1[R > 0]$ dan keyakinan $c = 1 + \alpha R$ lalu meminimalkan galat kuadrat berbobot atas semua sel.
- Dengan satu sisi tetap, setiap baris punya solusi tertutup, $x_u = (Y^\top C^u Y + \lambda I)^{-1} Y^\top C^u p(u)$; trik Gram membuat biaya satu sapuan sebanding dengan sel teramati, dan loss turun pada setiap iterasi.
- **BPR** mendorong item yang diklik di atas item acak, dengan gradien diskalakan oleh $\sigma(-\text{gap})$; run yang sehat dimulai dekat $\ln 2 \approx 0.693$.
- ALS mencapai ndcg@20 0.1581 dan BPR 0.1379 dibanding 0.0475 untuk `pop_7d`; 0.1662 milik user-kNN masih lebih tinggi, meski ALS memiliki recall@20 lebih tinggi, sehingga faktorisasi adalah model retrieval, bukan ranker yang tajam.
- 28 pengguna evaluasi tidak memiliki vektor dan perlu diisi dengan popularitas; nilai jangka panjang dari faktor adalah representasinya, yang dikembangkan lebih jauh di Bab 7.

Kuis

Q6.1 Dalam model faktorisasi dengan 32 faktor, apa itu *faktor laten*?

- A. Satu kolom dari matriks interaksi mentah, seperti klik pada satu produk.

- B. Salah satu dimensi terpelajar dari vektor pengguna dan item; ia tidak diberi nama sebelumnya dan ditemukan karena membantu merekonstruksi interaksi.
- C. Sebuah hyperparameter yang dipilih dengan cross-validation, seperti learning rate.
- D. Sebuah fitur seperti harga atau kategori yang diketik penjual saat membuat produk.

Q6.2 Pada model ALS implisit Hu-Koren-Volinsky, apa peran p_{ui} dan c_{ui}?

- A. p_{ui} adalah skor prediksi dan c_{ui} adalah rating sebenarnya.
- B. p_{ui} adalah target biner "menunjukkan minat" dan $c_{ui} = 1 + \alpha R_{ui}$ menyatakan seberapa besar loss harus peduli agar sel itu benar.
- C. p_{ui} adalah popularitas item dan c_{ui} adalah kategori item.
- D. p_{ui} adalah learning rate dan c_{ui} adalah kekuatan regularisasi.

Q6.3 Mengapa trik matriks Gram $Y^\top C^u Y = Y^\top Y + Y^\top (C^u - I) Y$ penting bagi ALS?

- A. Ia membuat loss konveks terhadap X dan Y secara bersamaan, sehingga ALS selalu menemukan minimum global.
- B. Ia menghilangkan kebutuhan akan regularisasi.
- C. $Y^\top Y$ dapat dihitung sekali per sapuan dan dibagi ke semua pengguna, sehingga biaya per pengguna hanya bergantung pada item yang disentuh pengguna itu, bukan seluruh katalog.
- D. Ia memungkinkan ALS menangani rating bintang eksplisit.

Q6.4 Jelaskan dalam dua atau tiga kalimat mengapa memperlakukan setiap sel kosong sebagai rating 0 yang sebenarnya adalah gagasan buruk untuk data klik, dan bagaimana ALS implisit melunakkannya.

Q6.5 Loss pelatihan BPR dalam bab ini dimulai sekitar 0.693 pada epoch 1. Mengapa itu nilai yang diharapkan ketika vektor hampir nol?

Q6.6 (Perhitungan) Seorang pengguna menyentuh dua item. Model satu faktor memiliki faktor item $y = (2, 1)$ untuk kedua item itu (dan tidak ada item lain). Bobot interaksi mentah pengguna adalah $R = (1, 3)$. Gunakan $\alpha = 4$ dan $\lambda = 1$. Hitung faktor pengguna x_u dengan solusi tertutup ALS (kedua item memiliki preferensi 1).

Q6.7 (Perhitungan) Pada BPR, misalkan untuk satu triplet $\hat{x}_{uij} = -\ln 3 \approx -1.0986$. Hitung $\sigma(\hat{x}_{uij})$, loss $-\ln \sigma(\hat{x}_{uij})$, dan pengali gradien $s = \sigma(-\hat{x}_{uij})$. Apakah triplet itu terurut dengan benar? Apakah pembaruannya besar atau kecil?

Q6.8 (Membaca kode) Pada fungsi `bpr` yang ditampilkan di penjelasan kode, perhatikan `sig = 1.0 / (1.0 + np.exp(gap))` dan `gu`, `gi`, serta `gj`. Berapakah nilai `sig` secara matematis, mengapa ia mengalikan ketiga gradien, dan mengapa `gj` membawa tanda negatif?

Q6.9 ALS memiliki recall@20 yang lebih tinggi daripada user-kNN (0.2715 dibanding 0.2273) tetapi ndcg@20 yang lebih rendah (0.1581 dibanding 0.1662). Apa yang ditunjukkan perbedaan kedua metrik itu tentang hal yang dilakukan baik oleh masing-masing metode? Di mana dalam sistem multi-tahap Anda akan memakai ALS?

Q6.10 (Praktik) Buka `book/code/ch06_mf.py`. Di `main()`, ubah pemanggilan ALS menjadi `als_implicit(R, alpha=1.0)`, dan pada run kedua menjadi `als_implicit(R, factors=8)`. Jalankan skrip setelah setiap perubahan. Catat `ndcg@20` untuk `als_implicit` dan loss ALS akhir setiap kali, lalu bandingkan dengan hasil di bab ini. Nyatakan perubahan mana yang Anda perkirakan lebih merugikan dan mengapa, lalu periksa. Apa yang disarankan eksperimen ini tentang memvalidasi hyperparameter pada label yang sama dengan yang Anda laporkan?

Bab 7 — Embedding dan retrieval two-tower

Malam puncak 9.9 sale, tim marketplace memelototi sebuah dasbor. Item-to-item co-visitation, *retriever* yang Anda kenal di Bab 5, sudah berbulan-bulan melayani strip "produk serupa", dan ia hebat di satu hal: memberi tahu apa saja yang dilihat orang-orang yang membuka ponsel *ini*. Namun Dewi, engineer terbaru di tim, punya keluhan lain. Seorang pengguna yang seminggu penuh menelusuri sepatu lari, jam tangan olahraga, dan minuman elektrolit membuka beranda dan disuguhi casing ponsel. "Sistem ini tidak tahu apa-apa soal *siapa* dia," kata Dewi. "Ia hanya tahu halaman produk mana yang disentuhnya. Tidak ada konsep selera yang menghubungkan sepatu, jam tangan, dan bubuk elektrolit."

Yang Dewi inginkan adalah sebuah *representasi*: daftar pendek angka untuk setiap pengguna dan setiap item, yang dipelajari sedemikian rupa sehingga "cocok dengan pengguna ini" berubah menjadi satu operasi aritmetika. Jika angkanya bagus, pengguna yang gemar olahraga akan mendarat di dekat item olahraga, bahkan ketika tak satu pun item itu pernah muncul bersamaan dengan riwayatnya. Dan jika operasinya murah, tim dapat menilai jutaan item tanpa *ranker* yang berat. Itulah janji **embedding** dan **model two-tower**, keluarga arsitektur yang menjadi dasar sistem retrieval skala besar seperti milik YouTube (lihat kotak fakta di bawah).

Di bab ini kita membangunnya dari nol di PyTorch pada marketplace simulasi yang sama dengan bab-bab lain, melatihnya, dan mengevaluasinya terhadap *baseline* popularitas. Hasilnya tidak seperti yang dijanjikan buku teks: model yang mengikuti resep dengan setia justru mencetak skor *di bawah* baseline sepele "populer dalam 7 hari terakhir". Mencari tahu sebabnya, lalu memperbaikinya dengan koreksi satu baris dari makalah produksi sungguhan, adalah bagian paling instruktif dari bab ini. Kita tutup dengan membangun indeks pencarian tetangga terdekat hampiran (*approximate nearest-neighbour*, IVF) hanya dengan numpy, sehingga Anda paham mengapa retrieval pada katalog raksasa tidak perlu menyentuh setiap item.

Yang akan Anda pelajari

- Apa itu embedding (*embedding*), dan mengapa "kedekatan dalam ruang vektor" adalah pengganti yang berguna untuk aturan kemiripan yang ditulis tangan.
- Bagaimana model two-tower disusun: menara pengguna, menara item, perkalian titik (*dot product*), dan mengapa pemisahan itu penting untuk *serving*.
- Bagaimana *loss* softmax in-batch dengan suhu (*temperature*) melatih kedua menara, dan mengapa negatif in-batch bias terhadap item populer.
- Bagaimana koreksi logQ menghilangkan bias itu, dan apa dampaknya pada angka offline kita.
- Bagaimana pencarian tetangga terdekat hampiran (IVF dari nol; HNSW, ScaNN, dan FAISS di dunia nyata) menukar *recall* dengan kecepatan.
- Bagaimana membaca gambar PCA dari embedding tanpa menafsirkannya berlebihan.

Intuisi: dari nama menjadi koordinat

Bayangkan cara kerja asisten toko buku yang baik. Ia tidak menghafal tabel "pelanggan yang membeli X juga membeli Y". Ia punya *peta di kepalanya*: buku masak di satu sisi toko, novel thriller di sisi lain, dan "memoar kuliner" di suatu tempat di antaranya. Pelanggan baru masuk, menyebut dua kalimat tentang apa yang disukainya, dan sang asisten menempatkannya di peta. Merekomendasikan kemudian hanyalah "berjalan ke rak terdekat yang belum diacak-acak".

Embedding adalah peta itu yang dibuat eksplisit. Setiap pengguna dan setiap item menjadi sebuah titik dalam ruang vektor, misalnya 32 dimensi. Tak seorang pun memberi tahu model arti dimensi-dimensinya. Pelatihan mendorong pengguna ke arah item yang berinteraksi dengannya dan menjauh dari item yang diabaikan, dan koordinat yang memungkinkan hal itu adalah embedding-nya. Faktorisasi matriks (Bab 6) sudah menghasilkan vektor semacam itu: satu baris P per pengguna, satu baris Q per item. Lalu apa yang baru?

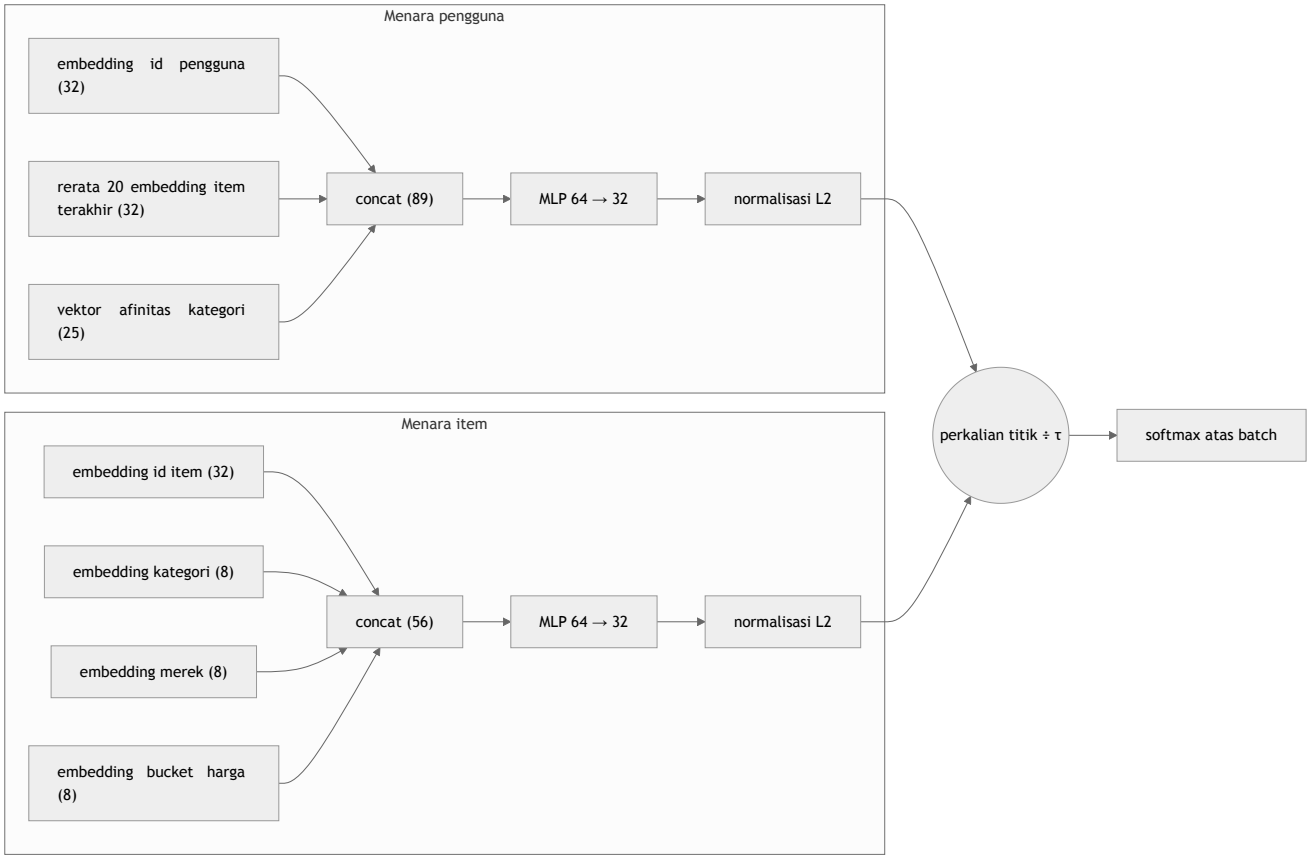
Tiga hal.

1. **Vektor dihitung oleh sebuah fungsi, bukan dicari di tabel.** Pada faktorisasi matriks setiap item punya vektor parameter bebas. Item tanpa interaksi tidak mendapat vektor. Pada model two-tower, vektor item adalah *keluaran jaringan saraf kecil* yang inputnya mencakup informasi sampingan: kategori, merek, *bucket* harga. Item baru dengan kategori dan merek yang diketahui mendapat vektor yang masuk akal sejak hari pertama. Hal yang sama berlaku untuk pengguna: menara pengguna dapat membaca item terakhir yang disentuh pengguna beserta afinitas kategorinya, sehingga vektornya bereaksi terhadap apa yang ia lakukan lima menit lalu, selama fitur riwayat masih segar saat Anda menghitungnya.
2. **Loss-nya berupa softmax bercita rasa pemeringkatan, bukan galat kuadrat.** Kita tidak akan memprediksi rating. Kita bertanya: dari semua item di batch ini, mana yang benar-benar berinteraksi dengan pengguna? Itu masalah klasifikasi, dan ia skalabel.
3. **Serving berupa pencarian tetangga terdekat.** Karena skor akhirnya adalah perkalian titik antara vektor pengguna dan vektor item, dan vektor item tidak bergantung pada pengguna, kita dapat menghitung semua vektor item sekali, memasukkannya ke indeks, dan menjawab "50 item teratas untuk pengguna ini" tanpa *forward pass* jaringan saraf per item.

> **Tahukah kamu?**
>
> **Vektor yang bisa berhitung.** Makalah Google 2013 tentang pelatihan vektor kata yang efisien mengingat kembali sebuah temuan awal: dengan aljabar sederhana pada vektor kata, vector("King") − vector("Man") + vector("Woman") menghasilkan vektor yang paling dekat dengan vektor "Queen". Tak ada yang memprogram keteraturan itu ke dalam vektornya. Embedding rekomendasi mengikuti ide yang sama dengan item dan pengguna sebagai pengganti kata.
>
> Sumber: Mikolov et al., arXiv, 2013

Arsitektur two-tower

Tower (menara) hanyalah fungsi yang mengubah input mentah menjadi sebuah vektor. Model two-tower memiliki dua menara yang tidak pernah saling melihat input masing-masing.



Gambar 7.1: *Dua menara pada model kita. Satu-satunya tempat sisi pengguna dan sisi item bertemu adalah perkalian titik terakhir.*

Menara pengguna menggabungkan (*concat*) embedding id pengguna yang dipelajari (32 angka), *rerata* embedding dari 20 item terakhir yang berinteraksi dengan pengguna (32, dari tabel id item yang sama dengan yang dipakai menara item), dan vektor afinitas kategori 25 dimensi dari `recsys.candidates.category_affinity`; MLP dua lapis memetakan 89 angka ini menjadi 32. **Menara item** menggabungkan embedding id item (32), kategori (8), merek (8), dan *bucket* harga (8, kuintil harga relatif terhadap median kategori); MLP lain memetakan 56 angka ini menjadi 32.

Kedua keluaran dinormalisasi **L2**, sehingga perkalian titik vektor pengguna dan vektor item persis sama dengan *kemiripan kosinus*-nya, antara -1 dan 1. Ini mencegah model menggelembungkan norma item populer dan membatasi skor, yang membuat kita bisa membahas suhu berikutnya.

Mengapa dua menara dan bukan satu jaringan yang melihat pengguna dan item sekaligus? Model gabungan (*cross*) dapat mempelajari interaksi seperti "pengguna ini suka item murah *hanya* di Beauty" dan biasanya lebih akurat, tetapi ia harus berjalan sekali per pasangan (pengguna, item): layak untuk 2,311 item, tidak untuk 50 juta. Desain two-tower mengorbankan interaksi yang ekspresif demi kemampuan menghitung dan mengindeks salah satu sisi terlebih dahulu, kompromi yang tepat untuk tahap pertama, karena *ranker* di Bab 9 dan 10 menilai ulang item yang lolos dengan semua fitur silang yang Anda mau.

Tahukah kamu?

Dua tahap di YouTube. Makalah YouTube 2016 membagi sistem rekomendasinya menjadi dua jaringan: pembangkitan kandidat, yang menyaring korpus raksasa menjadi ratusan video, dan pemeringkatan, yang menilainya. Pembangkitan kandidat dirumuskan sebagai klasifikasi multikelas ekstrem atas jutaan video menggunakan embedding pengguna dan video yang dipelajari. Karena menilai jutaan video dalam hitungan puluhan milidetik membutuhkan skema hampiran yang sublinear terhadap ukuran korpus, serving direduksi menjadi pencarian tetangga terdekat dalam ruang perkalian titik. Struktur sistem di buku ini mengikuti pembagian dua tahap yang sama.

Sumber: Covington et al., RecSys, 2016

Algoritma: pelatihan dengan softmax in-batch

Langkah demi langkah

Data latih berupa pasangan positif: pengguna dan item yang berinteraksi dengannya sebelum *cutoff* (klik, keranjang, atau pesanan, dari `common.interaction_matrix` dan `History.interactions`). Kita tidak punya negatif berlabel; "pengguna tidak mengklik" tidak sama dengan "pengguna tidak menyukainya". Triknya yang baku adalah membuat negatif secara gratis dari batch itu sendiri.

1. Ambil sampel satu batch berisi B pasangan positif $(u_1, i_1), \ldots, (u_B, i_B)$.
2. Jalankan menara pengguna pada setiap u_b untuk mendapat $\mathbf{u}_b$, dan menara item pada setiap i_b untuk mendapat $\mathbf{v}_b$. Keduanya vektor satuan.
3. Bangun matriks skor terskala $B \times B$, $s_{bj} = \mathbf{u}_b \cdot \mathbf{v}_j / \tau$, dengan τ adalah **suhu** (*temperature*).
4. Baris b adalah masalah klasifikasi B kelas: jawaban yang benar adalah kolom b (item yang benar-benar disentuh pengguna itu); $B - 1$ item lain dalam batch diperlakukan sebagai negatif.
5. Minimalkan *cross-entropy* klasifikasi itu, dirata-ratakan atas batch:

$$\mathcal{L} = -\frac{1}{B} \sum_{b=1}^{B} \log \frac{\exp(s_{bb})}{\sum_{j=1}^{B} \exp(s_{bj})}$$

Inilah ide **sampled softmax**: softmax penuh atas seluruh katalog adalah objektif yang tepat tetapi terlalu mahal, sehingga kita mengaproksimasi penyebutnya dengan item-item lain dalam batch.

Suhu

Karena kosinus berada dalam $[-1, 1]$, selisih mentah antara item bagus (0.8) dan item biasa (0.3) sangat kecil bagi softmax: $e^{0.8}$ berbanding $e^{0.3}$ hanya sekitar 1.6 kali. Membagi dengan τ kecil memperlebar skalanya. Dengan $\tau = 0.05$, seperti di skrip kita, selisih yang sama menjadi 16 satuan logit dan softmax menjadi tegas. Suhu rendah membuat loss peduli pada ketepatan *puncak* peringkat dan menghukum negatif sulit dengan berat; suhu tinggi menyebarkan probabilitas lebih merata dan belajar lebih lembut. Ini hiperparameter yang Anda *tuning*, biasanya antara 0.05 dan 0.2.

Contoh hitung dengan angka sederhana

Ambil $B = 3$ dan, agar aritmetikanya ramah, $\tau = 0.5$. Misalkan kosinus pengguna 1 dengan tiga item dalam batch adalah $0.8, 0.3, 0.1$, dan item 1 adalah positif sebenarnya.

- Logit terskala: $1.6,\ 0.6,\ 0.2$.
- Eksponensial: $4.95,\ 1.82,\ 1.22$, jumlah ≈ 8.00.
- Probabilitas item yang benar: $4.95/8.00 \approx 0.62$. Loss $= -\ln 0.62 \approx 0.48$.

Jika pelatihan menggeser kosinusnya dengan item 1 menjadi 0.9, probabilitas naik menjadi sekitar 0.67 dan loss turun. Gradien menarik $\mathbf{u}_1$ ke arah $\mathbf{v}_1$ dan mendorongnya menjauh dari $\mathbf{v}_2, \mathbf{v}_3$; karena setiap item adalah positif bagi sebagian baris dan negatif bagi baris lain, vektor item terus ditarik dan didorong sepanjang pelatihan.

Bias tersembunyi: siapa yang jadi negatif?

Batch adalah sampel acak pasangan positif, sehingga sebuah item muncul di dalamnya sebanding dengan jumlah interaksinya. Item dengan 2% dari seluruh interaksi menjadi positif bagi sekitar satu dari lima puluh baris, dan menjadi *negatif* di semua baris lainnya; item langka nyaris tak pernah ada di batch. Item populer jadi terlalu banyak terwakili sebagai negatif dibanding item katalog yang dipilih seragam, dan model belajar bahwa "item populer biasanya jawaban yang salah", sehingga skor mereka tergeser turun. Padahal saat serving kita menilai setiap item secara seragam, jadi penalti popularitas yang dipelajari itu adalah galat sistematis yang menekan *best-seller* yang sebenarnya relevan bagi banyak pengguna.

Koreksi logQ

Yi et al. menjelaskan perbaikannya di Google: kurangi setiap logit dengan log probabilitas bahwa item itu terambil ke dalam batch. Jika q_j adalah probabilitas sampling item j, definisikan logit terkoreksi

$$s^c_{bj} = \frac{\mathbf{u}_b \cdot \mathbf{v}_j}{\tau} - \log q_j$$

dan pakai s^c dalam softmax. Intuisinya: item yang sering terambil sebagai negatif (q_j besar, $\log q_j$ mendekati nol) mendapat dorongan kecil; item yang jarang terambil ($\log q_j$ sangat negatif) mendapat dorongan besar. Ini persis membatalkan dorongan ke bawah ekstra yang akan diterima item populer, sehingga perkalian titik itu sendiri belajar "kecocokan", bukan "kecocokan dikurangi pajak popularitas". Pada makalahnya, frekuensi diestimasi *online* dari aliran data; pada skrip offline kita, kita memakai frekuensi empiris item di antara pasangan latih.

Versi numerik dari contoh yang sama: misalkan item 1 populer ($q = 0.5$) dan item 2 dan 3 lebih jarang ($q = 0.25$). Maka $-\log q = 0.69,\ 1.39,\ 1.39$, logit terkoreksinya $2.29,\ 1.99,\ 1.59$, dan probabilitas item yang benar turun dari 0.62 menjadi sekitar 0.45 (loss dari 0.48 menjadi 0.80). Model kini harus memisahkan positif dengan lebih bersih di ruang kosinus alih-alih bersandar pada popularitas. Saat serving, suku $-\log q$ dibuang.

> **Tahukah kamu?**
>
> **Mengoreksi bias sampling pada retrieval YouTube.** Yi et al. meninjau model retrieval two-tower yang dilatih dengan negatif in-batch, yaitu item yang terambil dalam batch acak. Mereka berargumen bahwa loss batch ini mengalami bias sampling yang dapat sangat membatasi performa, terutama pada distribusi item berhukum pangkat (*power-law*). Mereka mengusulkan algoritma yang mengestimasi frekuensi item dari data aliran tanpa kosakata item yang tetap, dan menerapkan pendekatan terkoreksi bias itu pada Neural Deep Retrieval, sistem YouTube yang melakukan retrieval dari korpus puluhan juta video.
>
> Sumber: Yi et al., RecSys, 2019

Penelusuran kode

Semua kode di bawah berasal dari `book/code/ch07_two_tower.py`, yang melatih empat varian (10 atau 30 epoch, dengan dan tanpa logQ) lalu menjalankan demo ANN.

Kedua menara

book/code/ch07_two_tower.py · Towers

```python
class Towers(nn.Module):
    def __init__(self, n_users, n_items, n_cats, n_brands, n_buckets):
        super().__init__()
        self.user_emb = nn.Embedding(n_users, 32)
        self.item_emb = nn.Embedding(n_items + 1, 32, padding_idx=n_items)  # shared; last row =
padding
        self.cat_emb = nn.Embedding(n_cats, 8)
        self.brand_emb = nn.Embedding(n_brands, 8)
        self.price_emb = nn.Embedding(n_buckets, 8)
        self.pad = n_items
        self.user_mlp = nn.Sequential(nn.Linear(32 + 32 + n_cats, 64), nn.ReLU(), nn.Linear(64,
32))
        self.item_mlp = nn.Sequential(nn.Linear(32 + 8 + 8 + 8, 64), nn.ReLU(), nn.Linear(64, 32))

    def user_vec(self, uid, hist_items, aff):
        mask = (hist_items != self.pad).float().unsqueeze(-1)
        h = self.item_emb(hist_items)
        mean = (h * mask).sum(1) / mask.sum(1).clamp(min=1.0)
        x = torch.cat([self.user_emb(uid), mean, aff], dim=-1)
        return F.normalize(self.user_mlp(x), dim=-1)

    def item_vec(self, iid, cat, brand, bucket):
        x = torch.cat([self.item_emb(iid), self.cat_emb(cat), self.brand_emb(brand),
self.price_emb(bucket)], dim=-1)
        return F.normalize(self.item_mlp(x), dim=-1)
```

Baca kelas ini dalam empat bagian.

- **Tabel embedding.** `user_emb` punya satu baris 32 dimensi per pengguna; `item_emb` punya satu baris per item *ditambah satu baris ekstra* sebagai *padding* (`padding_idx=n_items`), sehingga pengguna dengan riwayat kurang dari 20 item tetap dapat berbagi tensor berlebar tetap. `cat_emb`, `brand_emb`, dan `price_emb` adalah tabel kecil 8 dimensi. Perhatikan bahwa `item_emb` **dibagi bersama**: menara item mengambil baris item itu sendiri, dan menara pengguna mengambil baris item-item riwayat pengguna dari tabel yang sama. Berbagi berarti vektor untuk "sepatu lari A" punya satu makna di kedua menara.

- **MLP.** `user_mlp` menerima `32 + 32 + n_cats` input (32 untuk id pengguna, 32 untuk rerata riwayat, 25 untuk afinitas), yaitu angka 89 di diagram. `item_mlp` menerima `32 + 8 + 8 + 8 = 56`. Masing-masing berbentuk `Linear → ReLU → Linear` dengan lapis tersembunyi selebar 64 dan keluaran selebar 32.
- **`user_vec` dan `item_vec`.** Pada `user_vec`, `mask` bernilai 1 di slot riwayat yang berisi item sungguhan dan 0 di slot padding; embedding di-*mask* sebelum dijumlahkan lalu dibagi dengan jumlah item sungguhan (dibatasi minimal 1 agar riwayat kosong dibagi 1). Tanpa mask, padding akan menyeret setiap pengguna berriwayat pendek ke arah "kosong" yang sama. Rerata itu digabung dengan embedding id pengguna dan afinitas, dilewatkan ke MLP, dan dinormalisasi dengan `F.normalize`. `item_vec` lebih sederhana: gabungkan empat embedding, MLP, normalisasi. Yang krusial, ia hanya menerima atribut item, tanpa input pengguna, dan itulah yang memungkinkan kita menghitung vektor seluruh katalog sekali, secara offline, lalu menyimpannya.

Putaran pelatihan

Fungsi pelatihan bersarang di dalam `main`; proses build mendukung simbol bersarang, sehingga kita bisa menampilkannya tersendiri.

book/code/ch07_two_tower.py · main.train

```
def train(epochs: int, logq: bool, tag: str):
    torch.manual_seed(0)
    model = Towers(len(all_users), len(items), n_cats, len(brands), 5)
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    g = torch.Generator().manual_seed(0)
    counts = torch.bincount(p_i, minlength=len(items)).float()
    log_q = torch.log(counts.clamp(min=1.0) / len(p_i))  # item sampling probability in a batch
    losses = []
    for ep in range(1, epochs + 1):
        model.train()
        perm = torch.randperm(len(p_u), generator=g)
        total, nb = 0.0, 0
        for s0 in range(0, len(perm), BATCH):
            b = perm[s0 : s0 + BATCH]
            u, i = p_u[b], p_i[b]
            h = t_hist[u].clone()
            # book:start leak_mask
            h[h == i[:, None]] = model.pad  # hide the target item from its own history (no
leakage)
            # book:end leak_mask
            uv = model.user_vec(u, h, t_aff[u])
            iv = model.item_vec(i, t_cat[i], t_brand[i], t_bucket[i])
            logits = uv @ iv.T / TEMP
            if logq:
                logits = logits - log_q[i][None, :]  # logQ correction (Yi et al., RecSys 2019)
            same = i[:, None] == i[None, :]
            logits = logits.masked_fill(same & ~torch.eye(len(b), dtype=torch.bool), -1e9)  #
duplicates are not negatives
            loss = F.cross_entropy(logits, torch.arange(len(b)))
            opt.zero_grad()
            loss.backward()
            opt.step()
            total += loss.item()
            nb += 1
        losses.append(total / nb)
        if ep in (1, 5, 10, 20, 30) or ep == epochs:
            print(f"[{tag}] epoch {ep:>2d}  loss {losses[-1]:.4f}")
    model.eval()
    with torch.no_grad():
        iv_all = model.item_vec(t_item, t_cat, t_brand, t_bucket).numpy()
        uv_eval = model.user_vec(torch.tensor(eval_rows), t_hist[eval_rows],
t_aff[eval_rows]).numpy()
    return model, iv_all, uv_eval, losses
```

Telusuri putaran dalamnya.

1. `perm = torch.randperm(...)` mengocok pasangan positif tiap epoch; `b` adalah irisan 512 di antaranya (`BATCH`).
2. `h = t_hist[u].clone()` mengambil matriks 20 item terakhir setiap pengguna, dan baris berikutnya, `h[h == i[:, None]] = model.pad`, **mengosongkan item target di dalam riwayat penggunanya sendiri**. Tanpa itu, menara pengguna akan melihat jawaban yang seharusnya ia prediksi, jenis kebocoran data (*leakage*) yang sama dengan yang akan Anda pelajari di Bab 9. Model akan mencapai loss latih rendah dengan menyalin, lalu gagal pada minggu *held-out* yang tidak punya jalan pintas semacam itu.
3. `uv` dan `iv` adalah vektor pengguna dan item untuk batch. `logits = uv @ iv.T / TEMP` adalah matriks $B \times B$ s_{bj} dari algoritma.
4. `if logq:` mengurangkan `log_q[i][None, :]`, dengan `log_q` dihitung di awal `train` sebagai `log(count / total)` tiap item di antara pasangan latih. *Broadcasting* mengurangkan $\log q_j$ dari kolom j. Satu baris itulah seluruh koreksi logQ.

5. Mask `same` menangani duplikat: jika dua pengguna dalam batch mengklik item yang sama, kolom itu akan menjadi "negatif" yang sebenarnya positif, sehingga entri off-diagonal tersebut diset ke -10^9 dan lenyap dari softmax.
6. `F.cross_entropy(logits, torch.arange(len(b)))` adalah loss di atas (kolom yang benar untuk baris b adalah b), diikuti langkah Adam dengan *learning rate* 10^{-3}.

Saat putaran selesai, `train` beralih ke mode eval dan menghitung vektor item untuk seluruh katalog serta vektor pengguna untuk pengguna evaluasi.

Penilaian dan evaluasi

Penilaian (`main.evaluate_variant` di skrip) adalah satu perkalian matriks, `uv_eval @ V.T`, dari 947 pengguna evaluasi terhadap item yang tersedia. Pengguna tanpa riwayat sebelum *cutoff* tidak mendapat skor model dan diisi dengan popularitas 7 hari, karena model "tidak punya pendapat". Lalu `common.evaluate` menghitung metrik yang dipakai setiap bab: recall@K, order recall, NDCG@20, dan coverage@20.

Indeks ANN, dari nol

Separuh kedua skrip tidak melibatkan *deep learning* sama sekali. IVF, *inverted file index*, hanya butuh rutin *clustering*.

book/code/ch07_two_tower.py · kmeans

```
def kmeans(x: np.ndarray, k: int, iters: int, seed: int = 0):
    rng = np.random.default_rng(seed)
    cent = x[rng.choice(len(x), k, replace=False)].copy()
    for _ in range(iters):
        assign = np.argmax(x @ cent.T - 0.5 * (cent**2).sum(1), axis=1)  # nearest centroid
(euclidean)
        for c in range(k):
            pts = x[assign == c]
            if len(pts):
                cent[c] = pts.mean(0)
    assign = np.argmax(x @ cent.T - 0.5 * (cent**2).sum(1), axis=1)
    return cent, assign
```

Ini k-means Lloyd dengan satu trik kecil. Mencari centroid terdekat menurut jarak Euklides sama dengan `argmax(x @ cent.T - 0.5 * ||cent||²)`: dengan mengembangkan $\|x - c\|^2 = \|x\|^2 - 2x \cdot c + \|c\|^2$, suku $\|x\|^2$ sama untuk setiap centroid sehingga dapat dibuang, dan membalik tanda mengubah "minimalkan" menjadi "maksimalkan". Satu perkalian matriks menetapkan semua titik sekaligus. Lalu tiap centroid berpindah ke rerata titik-titiknya, dan kita ulangi 20 iterasi.

Pencarian tetangga terdekat hampiran

Mengapa brute force berhenti bekerja

Brute force menghitung perkalian titik query dengan setiap vektor item dan menyimpan K teratas. Untuk 2,311 item dalam stok kita itu sepele; untuk 50 juta item, itu sekitar $50\text{M} \times 32 \approx 1.6$ miliar operasi kali-tambah per permintaan, jauh melampaui anggaran latensi puluhan milidetik. Tetapi kita tidak butuh top 50 yang *persis*, karena *ranker* toh menilai ulang semuanya; kita hanya butuh daftar kandidat yang memuat *sebagian besar* tetangga sebenarnya.

IVF dalam empat langkah

1. **Latih.** Jalankan k-means pada vektor item untuk mendapat C centroid ($C = 32$ di sini). Tetapkan setiap item ke centroid terdekatnya. Himpunan item untuk satu centroid adalah *inverted list*-nya ("inverted file" pada namanya).
2. **Query: pilih cluster.** Untuk vektor query $\mathbf{q}$, nilai terhadap C centroid (murah: C perkalian titik) dan simpan `n_probe` terbaik.
3. **Query: pindai.** Gabungkan inverted list dari cluster-cluster itu dan hitung perkalian titik eksak hanya pada item-item tersebut.
4. **Kembalikan K terbaik** dari item yang dipindai.

Biayanya rata-rata $C + n_{\text{probe}} \cdot N/C$ perkalian titik, bukan N. Untuk $N = 2{,}311$ dan $C = 32$, memeriksa 4 cluster memindai sekitar $4 \times 72 \approx 289$ item ditambah 32 centroid, dibanding 2,311. Harganya: tetangga sejati yang berada di cluster yang tidak Anda periksa akan hilang; dari situlah *recall terhadap brute force* berasal.

Bayangkan dua gugus berisi masing-masing empat item, di sekitar $(1, 1)$ dan $(5, 5)$. Query di $(1.5, 1.2)$ dengan `n_probe = 1` hanya memindai gugus pertama dan hasilnya eksak. Query di $(3, 3)$ punya tetangga di kedua gugus, jadi memeriksa satu gugus kehilangan sebagian; efek perbatasan itulah yang diperbaiki oleh lebih banyak probe.

Demo dalam kode

Putaran query IVF ditulis inline di `main`. Untuk setiap query, ia menilai centroid, memilih `n_probe` teratas, mengumpulkan anggotanya, dan memeringkat hanya anggota itu secara eksak:

book/code/ch07_two_tower.py · ivf_search

```
for qi, ex in zip(q, exact):
    probe = np.argsort(-(cent @ qi))[:n_probe]
    cand = np.concatenate([members[c] for c in probe])
    top = cand[np.argsort(-(V[cand] @ qi))[:20]]
    rec_hits.append(len(set(top) & set(ex)) / 20)
    scanned.append(len(cand))
```

Baris `probe` adalah pemilihan cluster (`cent @ qi`, hanya 32 perkalian titik), `cand` menggabungkan inverted list, `top` adalah pemeringkatan ulang eksak pada item yang dipindai, dan dua baris terakhir mencatat recall terhadap jawaban brute force `ex` serta jumlah item yang dipindai. Di bawah kita tampilkan apa yang dicetak putaran ini, dengan 400 pengguna evaluasi sebagai query.

Kita menjalankan demo ANN pada model spesifikasi (`two_tower`); kualitas indeks bergantung pada geometri vektor, bukan pada kualitas rekomendasi.

> **Tahukah kamu?**
>
> **Indeks berbasis graf.** Selain indeks berbasis cluster seperti IVF, pencarian tetangga terdekat hampiran dapat memakai graf kedekatan. HNSW (Hierarchical Navigable Small World graphs) secara bertahap membangun struktur multi-lapis graf kedekatan di atas himpunan bagian bersarang dari vektor yang tersimpan. Memulai pencarian dari lapisan atas memungkinkan algoritma memanfaatkan pemisahan skala antar tautan, dan para penulis melaporkan skala kompleksitas logaritmik.
>
> Sumber: Malkov et al., IEEE TPAMI, 2020

Indeks graf bukan satu-satunya pilihan. Pustaka FAISS, yang dijelaskan oleh Johnson et al. (2021), menyediakan desain optimal untuk pencarian brute force, hampiran, dan product-quantisation di GPU; makalah mereka melaporkan pembangunan graf k-NN atas 1 miliar vektor dalam waktu kurang dari 12 jam pada empat GPU. Guo et al. (2020) menargetkan *maximum inner product search* dengan loss kuantisasi *anisotropik*, yang menghukum galat yang penting bagi pemeringkatan inner-product alih-alih galat rekonstruksi biasa; pustaka ScaNN milik Google dibangun di sekitar lini riset ini. Demo 2,311 item kita tidak butuh semua itu, tetapi inilah nama-nama yang perlu Anda cari ketika katalog melampaui satu perkalian matriks.

Jalankan

```
cd book
uv run python code/ch07_two_tower.py
```

Skrip ini deterministik (semua seed dikunci) dan memakan waktu beberapa menit di CPU laptop.

KELUARAN TERMINAL

```
users 1500  items 2495  categories 25  brands 10  price buckets 5
training pairs (distinct user-item positives before cutoff): 18256
user history window: last 20 items; temperature 0.05; batch 512; epochs 10
```

Kita punya 18,256 pasangan positif (pengguna, item) yang berbeda sebelum *cutoff* dan 1,500 pengguna. Sebelum menyalahkan model atas skor yang lemah, skrip juga menjalankan diagnostik (pemetaan indeks yang konsisten, riwayat sepenuhnya sebelum cutoff, vektor afinitas tidak kosong, item yang habis stok dikecualikan); semuanya lolos. Lalu pelatihan dimulai.

KELUARAN TERMINAL

```
=== Training two_tower: 10 epochs, logQ correction off ===
[two_tower] epoch  1  loss 6.8075
[two_tower] epoch  5  loss 5.8995
[two_tower] epoch 10  loss 5.1991
sanity: true training item is in the user's top-20 of 2495 items for 14.7% of training pairs
```

Loss model spesifikasi turun stabil dari 6.8075 ke 5.1991, jadi optimisasi bekerja. Baris sanity memberi tahu kita bahwa untuk 14.7% pasangan *latih*, item yang benar berada di 20 teratas pengguna dari 2,495 item, jauh di atas peluang acak (sekitar 0.8%), sehingga model telah mempelajari struktur yang nyata. Loss pada run logQ tidak dapat dibandingkan, karena logit-nya menyertakan suku ekstra $-\log q_j$.

Hasil

KELUARAN TERMINAL

```
=== Evaluation: brute-force top 50 over in-stock items ===
users without history (padded with 7-day popularity): 28 of 947
                     recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20
coverage@20
method
two_tower              0.0183     0.0399     0.0818     0.1932           0.0912   0.0361
0.9766
two_tower_logq         0.0747     0.1141     0.1766     0.2846           0.2243   0.1053
0.4344
two_tower_30ep         0.0182     0.0352     0.0754     0.1804           0.0893   0.0357
0.9965
two_tower_logq_30ep    0.0608     0.1074     0.1779     0.2936           0.2129   0.0982
0.6525
pop_7d                 0.0310     0.0493     0.0786     0.1335           0.1066   0.0475
0.0087
best two-tower variant: two_tower_logq; the ANN demo below uses the spec model 'two_tower'
```

Inilah tabel yang mengejutkan kita. Bacalah baris demi baris.

Model spesifikasi kalah dari popularitas. `two_tower` mencapai NDCG@20 sebesar 0.0361, di bawah baseline popularitas 7 hari yang 0.0475. Recall@20-nya 0.0818 kurang lebih setara dengan `pop_7d` (0.0786), dan recall@50-nya lebih baik (0.1932 vs. 0.1335), jadi ia memang mengambil item berguna lebih dalam di daftar, tetapi menempatkan yang benar di peringkat rendah. Pada order recall, yang hanya menghitung item yang benar-benar dibeli, ia tertinggal: 0.0912 melawan 0.1066. Pelatihan 30 epoch (`two_tower_30ep`) tidak membantu: NDCG@20 0.0357, dan loss terus turun, jadi optimisasi yang lebih banyak bukan jawabannya.

Kolom `coverage@20` membongkar penyebabnya. Model spesifikasi merekomendasikan sebaran katalog yang sangat lebar, dengan coverage@20 0.9766, sedangkan `pop_7d` hanya 0.0087, karena ia menampilkan hampir item yang sama kepada semua orang. Model spesifikasi telah belajar *untuk tidak* merekomendasikan item populer, tepat patologi yang diprediksi bagian logQ. Di marketplace ini popularitas adalah sinyal yang kuat dan sah, seperti ditunjukkan baris `pop_7d`: satu daftar bersama berisi beberapa item mengalahkan NDCG model. Model yang sengaja menghindari item populer membuang prior terbaiknya.

Koreksi logQ memperbaikinya. `two_tower_logq`, identik dalam segala hal kecuali satu baris pengurangan, mencapai NDCG@20 0.1053, lebih dari dua kali baseline popularitas 0.0475, dan sekitar tiga kali model tanpa koreksi yang 0.0361. Recall@20 melonjak dari 0.0818 ke 0.1766 dan order recall dari 0.0912 ke 0.2243. Coverage turun ke 0.4344: model terkoreksi terkonsentrasi pada himpunan bagian yang lebih masuk akal, masih jauh lebih luas daripada baseline popularitas 0.0087, tetapi tidak lagi menebar rekomendasi ke seluruh katalog. Itulah kompromi yang Anda inginkan pada sebuah retriever: terpersonalisasi *dan* sadar popularitas.

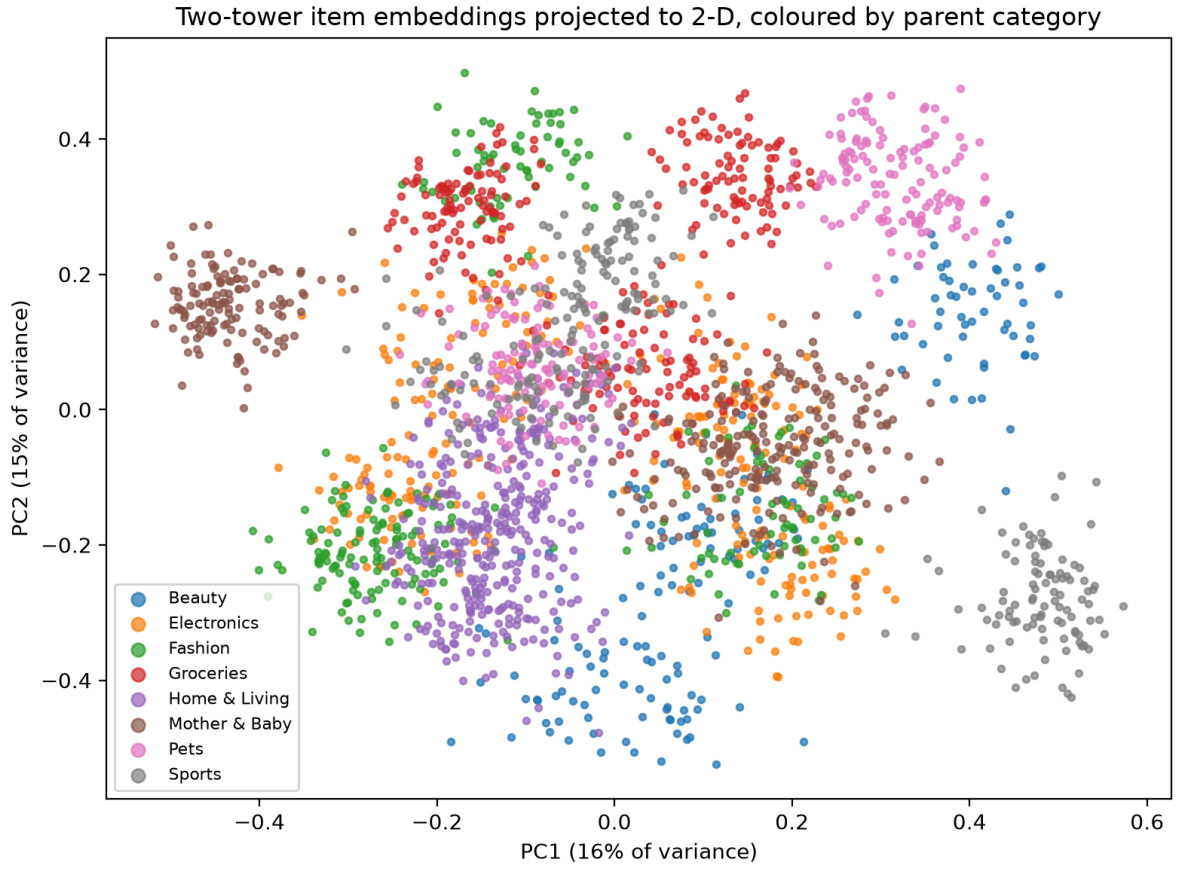
Lebih banyak epoch juga bukan obatnya. Dengan koreksi, 30 epoch menghasilkan NDCG@20 0.0982, sedikit *lebih rendah* daripada 0.1053 pada 10 epoch, awal *overfitting* yang lazim dengan hanya 18,256 pasangan latih. Evaluasilah pada himpunan *held-out* berdasarkan waktu dengan beberapa jumlah epoch, dan hentikan lebih awal (*early stopping*).

Catatan jujur: angka-angka ini berasal dari satu seed pada satu marketplace simulasi. Selisih antara model spesifikasi dan model terkoreksi (sekitar tiga kali lipat pada NDCG) cukup besar untuk

memercayai arahnya, tetapi kita tidak mengklaim bahwa `two_tower_logq` mengalahkan `two_tower_logq_30ep` secara kokoh. Bab 10 menunjukkan cara mengukur derau seed.

Seperti apa tampilan embedding-nya

Selanjutnya, vektor item dari model spesifikasi, dipadatkan dari 32 dimensi menjadi dua dengan PCA (dekomposisi nilai singular dari matriks yang dipusatkan, di numpy).



Gambar 7.2: *Embedding item two-tower diproyeksikan ke dua komponen utama pertama dan diwarnai menurut kategori induk. Pets, Sports, dan Mother & Baby membentuk pulau-pulau terpisah; Fashion, Electronics, Home & Living, dan lainnya saling tumpang tindih di tengah.*

KELUARAN TERMINAL

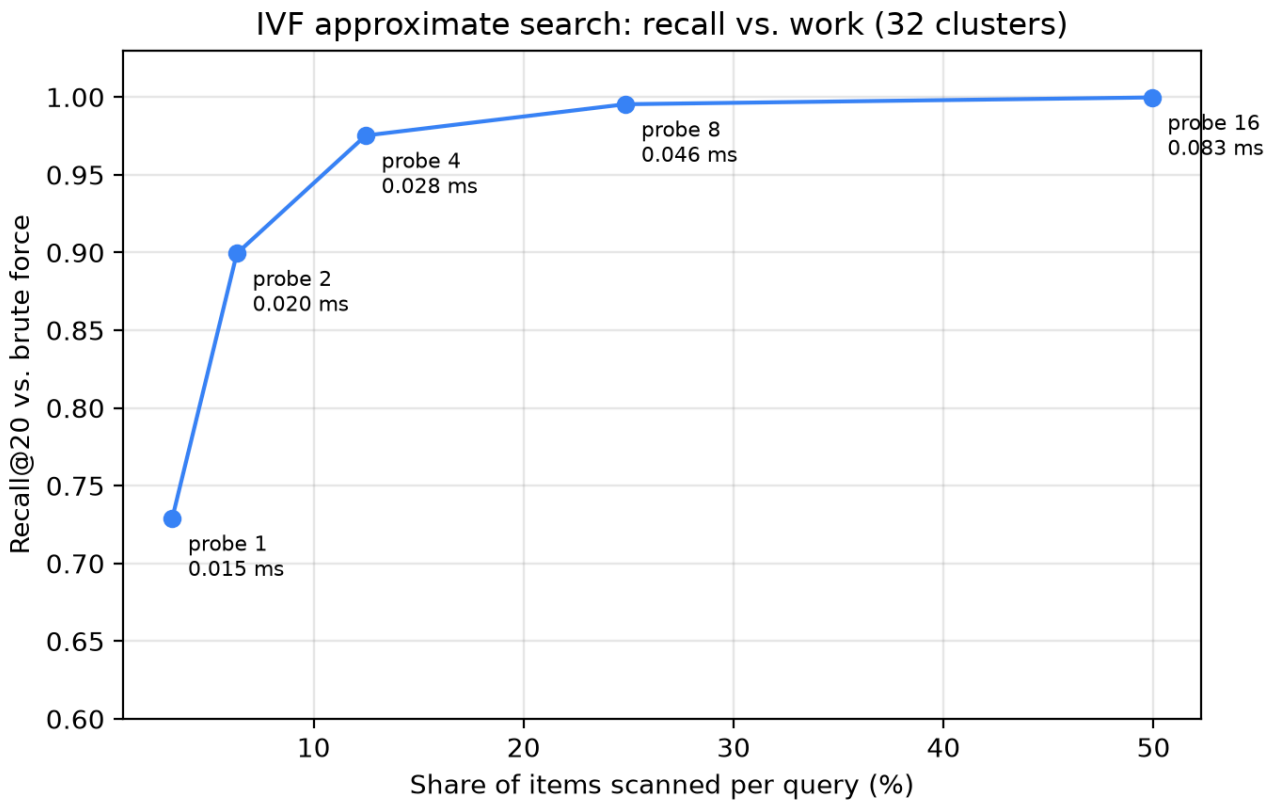
```
PCA: first two components explain 30.6% of item-vector variance
```

Model tidak pernah diberi kategori induk, hanya kategori daun, merek, dan *bucket* harga, namun item mengelompok menurut induknya: Pets berada di kanan atas, Sports membentuk pulau di kanan bawah, Mother & Baby awan di kiri, dan kategori dengan pembeli yang mirip bertumpang tindih di tengah. Tetapi hati-hati: dua komponen hanya menjelaskan 30.6% varians, sehingga sekitar 70% struktur tidak terlihat di sini, dan titik yang tampak dekat di 2-D bisa jauh di 32-D. PCA adalah uji kewajaran, bukan pengukuran.

Kompromi recall versus kecepatan

KELUARAN TERMINAL

```
=== Approximate nearest neighbours: IVF with 32 k-means clusters (on two_tower) ===
cluster sizes: min 24  median 68  max 127
brute force: 0.140 ms/query over 2311 items  (400 queries)
probe  1 clusters  recall@20 vs brute force 0.729  0.015 ms/query  scanned 3.3% of items
probe  2 clusters  recall@20 vs brute force 0.899  0.020 ms/query  scanned 6.3% of items
probe  4 clusters  recall@20 vs brute force 0.975  0.028 ms/query  scanned 12.5% of items
probe  8 clusters  recall@20 vs brute force 0.995  0.046 ms/query  scanned 24.9% of items
probe 16 clusters  recall@20 vs brute force 1.000  0.083 ms/query  scanned 49.9% of items
```



Gambar 7.3: *Pencarian hampiran IVF pada vektor item model spesifikasi: recall@20 relatif terhadap brute force terhadap porsi katalog yang dipindai, untuk 1, 2, 4, 8, dan 16 cluster yang diperiksa dari 32.*

Ke-32 cluster berisi antara 24 dan 127 item (median 68), tidak merata seperti cluster k-means pada umumnya. Memeriksa satu cluster saja hanya memindai 3.3% item dan sudah memulihkan 0.729 dari top 20 eksak. Dua cluster membawa recall ke 0.899 dengan 6.3% pekerjaan, empat ke 0.975 dengan 12.5%, dan delapan ke 0.995 dengan 24.9%; pada 16 probe, separuh katalog, recall-nya 1.000. Beberapa probe pertama membeli hampir seluruh recall, sebab itulah sistem produksi jarang memeriksa lebih dari beberapa persen clusternya.

Kolom waktu perlu hati-hati: pada 2,311 item, brute force hanya butuh 0.140 ms per query, dan overhead Python mendominasi waktu IVF. Yang berpindah ke katalog jutaan item adalah *bentuknya*: kerja tumbuh seiring porsi item yang dipindai sementara recall cepat jenuh, jadi ekstrapolasikan dari kolom "porsi dipindai", bukan dari milidetiknya.

Jebakan dan praktik di produksi

Training-serving skew. Menara pengguna memakai 20 item terakhir dan vektor afinitas. Di produksi, *feature pipeline* harus menghasilkan fitur yang identik secara online dan offline; ketidakcocokan kecil dalam definisi "20 item terakhir" secara diam-diam menurunkan kualitas model (Bab 9 membahas ketepatan titik waktu, *point-in-time correctness*).

Embedding bergeser; bangun ulang indeks bersama-sama. Vektor item dari model versi 17 dan vektor pengguna dari versi 18 hidup di ruang yang tidak berhubungan, sehingga menara pengguna dan indeks item dikirim sebagai satu unit berversi, dan indeks dibangun ulang secara offline setiap kali model berubah.

Bias popularitas bersembunyi di banyak tempat. Sampling negatif yang tidak seragam, atau set latih yang terlalu banyak memuat item populer, menghasilkan efek yang sama. Obatnya mirip: beri bobot ulang, koreksi logit, atau campurkan negatif yang disampel seragam.

Negatif palsu (*false negatives*). Mask `same` kita menghapus kasus yang jelas, tetapi sebuah item masih bisa muncul sebagai "negatif" bagi pengguna yang sebenarnya akan menyukainya dan hanya belum pernah melihatnya; batch besar membuat hal ini lebih mungkin. Penambangan negatif sulit (*hard negative mining*, item yang ditampilkan kepada pengguna tetapi diabaikan) membutuhkan kehati-hatian yang sama.

Cold start baru setengah terselesaikan. Item baru mendapat vektor yang masuk akal dari kategori, merek, dan *bucket* harganya, tetapi embedding id item-nya masih derau yang belum terlatih. Tim biasanya membuang embedding id di bawah ambang interaksi tertentu atau meng-*hash* id langka ke *bucket* bersama. Bab 12 kembali membahas cold start.

Memilih indeks. IVF adalah baseline termudah dan murah dibangun serta diperbarui. HNSW biasanya memberi recall lebih baik pada latensi yang sama tetapi memakai lebih banyak memori dan canggung menangani penghapusan; product quantisation memampatkan vektor. Pada skala kita, satu perkalian matriks adalah jawaban yang tepat: jangan menambah dependensi ANN sebelum brute force benar-benar menyakitkan.

Bagaimana repo ini melakukannya. Sistem ala Shopee di bab-bab selanjutnya tidak memiliki tahap two-tower. Tahap pertamanya adalah empat retriever tulisan tangan yang digabungkan di Bab 8, dan ranker LightGBM di Bab 10 yang mengerjakan beban berat, pilihan yang masuk akal untuk 2,495 item. Retriever yang dipelajari layak dipakai ketika katalog melampaui retriever berbasis aturan atau ketika Anda menginginkan kandidat di luar jangkauan co-visitation; Bab 14 menunjukkan di mana ia akan ditempatkan. Ukurlah dengan *recall* marjinalnya terhadap sumber yang sudah ada, bukan dengan NDCG semata.

Ringkasan

- Embedding adalah vektor yang dipelajari; kedekatan menggantikan aturan kemiripan tulisan tangan, dan informasi sampingan memberi item serta pengguna baru vektor yang bermakna.
- Model two-tower menghitung vektor pengguna dan item dengan jaringan terpisah; skornya adalah perkalian titik mereka (kosinus setelah normalisasi L2).

- Softmax in-batch memperlakukan item lain dalam batch sebagai negatif; suhu τ menskalakan kosinus, dan τ kecil mempertajam softmax.
- Negatif in-batch terambil sebanding dengan popularitas, sehingga loss mendorong item populer ke bawah: model spesifikasi mencetak NDCG@20 0.0361, di bawah baseline popularitas 7 hari yang 0.0475.
- Mengurangkan $\log q_j$ dari logit (Yi et al., 2019) mengangkat NDCG@20 ke 0.1053 dan recall@20 dari 0.0818 ke 0.1766; pelatihan lebih lama tidak membantu.
- Vektor item tidak bergantung pada pengguna, sehingga dapat dihitung lebih dulu dan diindeks; IVF hanya memindai beberapa cluster k-means per query, dan memindai 12.5% item mempertahankan 0.975 dari top 20 brute force.
- Pada ukuran katalog ini brute force sudah cukup; ANN, indeks berversi, dan penanganan cold start baru penting pada skala produksi.

Kuis

Q7.1 Apa yang terjadi ketika suhu τ pada softmax in-batch diturunkan?

A. Softmax menjadi lebih datar, sehingga semua item dalam batch diperlakukan sama.
B. Selisih antar skor kosinus diregangkan, sehingga loss berfokus pada ketepatan puncak peringkat dan menghukum negatif sulit lebih berat.
C. Dimensi embedding bertambah.
D. Item populer dihapus dari batch.

Q7.2 Mengapa negatif in-batch membuat model two-tower bias terhadap item populer?

A. Item populer punya judul lebih panjang, yang membingungkan menara item.
B. Item populer selalu kehabisan stok.
C. Item masuk ke batch sebanding dengan jumlah interaksinya, sehingga item populer muncul sebagai negatif pada banyak baris pengguna lain dan didorong turun lebih sering.
D. Normalisasi L2 memberi item populer norma yang lebih kecil.

Q7.3 Mengapa desain two-tower memungkinkan pencarian tetangga terdekat hampiran saat serving?

A. Karena vektor item hanya bergantung pada atribut item, sehingga dapat dihitung sekali dan diindeks, dan skor pengguna-item adalah inner product.
B. Karena menara pengguna berjalan di GPU.
C. Karena suhu selalu bernilai 1 saat serving.
D. Karena menara item menerima riwayat pengguna sebagai input.

Q7.4 Dalam satu atau dua kalimat, jelaskan apa yang dikurangkan koreksi logQ dari setiap logit, dan mengapa pengurangan itu menghilangkan penalti popularitas.

Q7.5 Demo IVF kita mengukur "recall@20 terhadap brute force", bukan NDCG. Pertanyaan apa yang dijawab metrik itu, dan mengapa itu yang tepat untuk indeks ANN?

Q7.6 Bacalah baris dari putaran pelatihan ini:

book/code/ch07_two_tower.py · leak_mask

```
h[h == i[:, None]] = model.pad  # hide the target item from its own history (no leakage)
```

Apa yang dilakukannya, dan apa yang kemungkinan terjadi pada loss latih dan hasil evaluasi jika baris itu dihapus?

Q7.7 Sebuah katalog berisi 10,000,000 item. Anda membangun indeks IVF dengan 4,096 cluster (anggap ukuran cluster sama) dan memeriksa 16 cluster per query. Kira-kira berapa perkalian titik yang dibutuhkan satu query (penilaian centroid ditambah pemindaian), dan berapa pecahannya dibanding brute force?

Q7.8 Dua item memiliki probabilitas sampling $q = 0.04$ dan $q = 0.0004$. Seberapa besar koreksi logQ menaikkan logit item yang lebih langka relatif terhadap item yang lebih populer?

Q7.9 Praktik langsung. Di `book/code/ch07_two_tower.py`, ubah `TEMP = 0.05` menjadi `TEMP = 0.2`, jalankan skrip, dan bandingkan baris `two_tower` dan `two_tower_logq` (ndcg@20 dan coverage@20) dengan tabel di bab ini. Lalu ulangi dengan `TEMP = 0.02`. Nyatakan apa yang Anda amati tentang pengaruh suhu dan apakah koreksi logQ masih membantu. (Skrip menimpa `results/ch07_two_tower.*`; pulihkan dengan `git checkout book/results` setelahnya.)

Bab 8 — Pembangkitan kandidat multi-sumber

Pukul 12.05 pada hari gajian, halaman utama sebuah marketplace ala Shopee akan dibuka oleh beberapa ratus ribu orang dalam menit yang sama. Masing-masing butuh dua puluh produk dalam sepersekian detik, dari katalog berisi jutaan barang. Seseorang mengusulkan: "Jalankan saja model pemeringkatan terbaik kita pada setiap barang untuk setiap pengguna, lalu urutkan." Hitung-hitungan kasar langsung mengakhiri rapat: bahkan 20 mikrodetik per pasangan (pengguna, barang) berarti puluhan detik waktu CPU per permintaan, dan model yang benar-benar kita inginkan, dengan puluhan fitur per pasangan, jauh lebih lambat.

Maka tim memecah masalah menjadi dua. Tahap yang murah dan luas memilih sekitar seratus barang yang *mungkin* menarik bagi pengguna ini; tahap yang mahal dan presisi hanya meranking barang-barang itu. Bab ini membahas tahap pertama, yaitu **pembangkitan kandidat** (*candidate generation*) atau **retrieval**. Tahap ini menentukan apa saja yang boleh dilihat oleh *ranker*, dan menetapkan batas atas yang keras untuk semua yang terjadi sesudahnya.

Di repositori ini, tahap pertama adalah `recommender/recsys/candidates.py`: empat *retriever* kecil, masing-masing berupa alasan berbeda untuk meyakini bahwa pengguna mungkin menyukai suatu barang, ditambah beberapa filter keras. Simulator memungkinkan kita memberi angka pada setiap klaim.

Yang akan Anda pelajari

- Mengapa sistem rekomendasi disusun sebagai corong (*funnel*) bertahap, dan mengapa *recall*, bukan *precision*, yang menjadi tujuan tahap pertama.
- Cara kerja empat *retriever* di repo (`recent`, `covisit`, `catpop`, `global_trending`) dan perilaku nyata apa yang ditiru masing-masing.
- Cara menggabungkan *retriever* dengan *outer join*, menyimpan skor tiap *retriever* sebagai fitur, dan menghitung berapa sumber yang mengusulkan suatu barang.
- Cara mengukur *retriever* dengan *recall* dan dengan *recall* yang *unik*, serta apa arti "batas atas *recall*" (*recall ceiling*) bagi seluruh sistem.
- Mengapa barang yang disembunyikan dan barang yang stoknya habis harus dibuang dengan filter keras di tahap ini, bukan diserahkan kepada *ranker*.

Intuisi: pustakawan dan daftar pendek

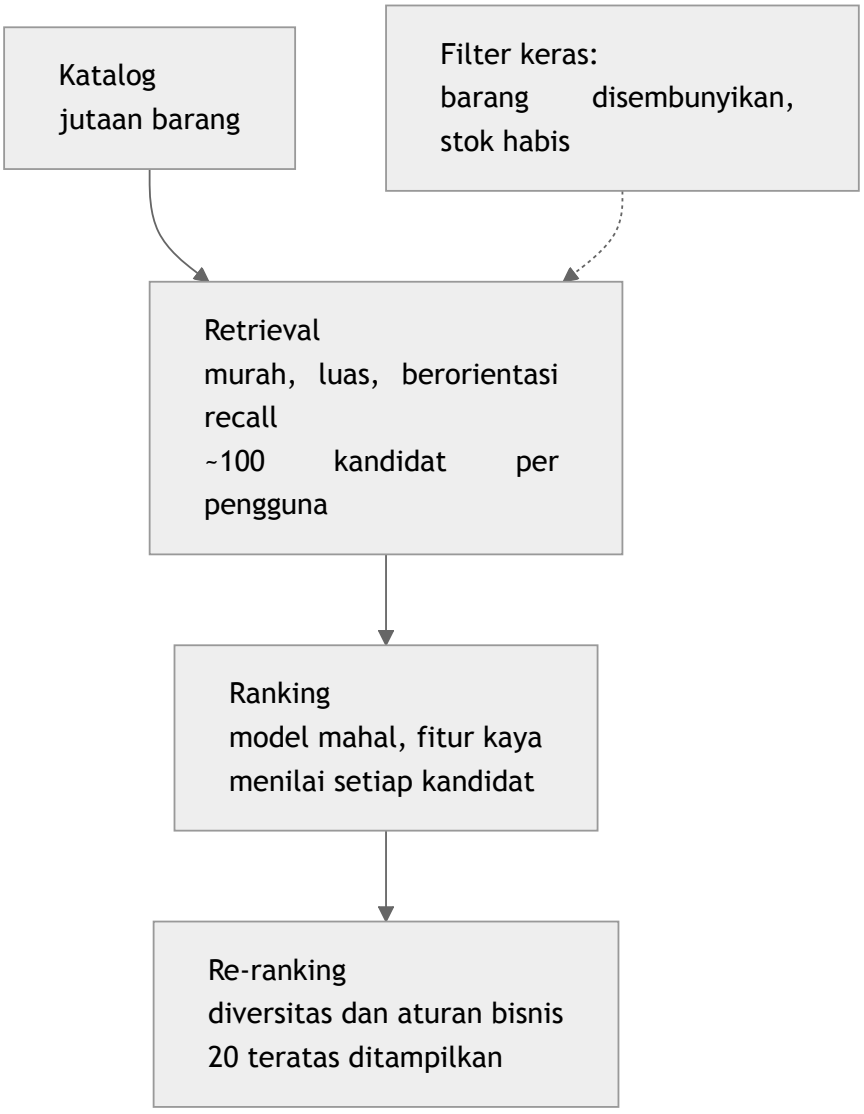
Bayangkan Anda meminta pustakawan, "Carikan sesuatu yang bagus untuk dibaca malam ini." Ia tidak membaca semua buku di gedung itu. Ia membuat daftar pendek dengan beberapa naluri: apa yang Anda pinjam minggu lalu, apa yang juga dipinjam pembaca buku-buku itu, rak mana yang sering Anda jelajahi dan buku mana di rak itu yang paling banyak dipinjam bulan ini, dan, jika Anda pengunjung baru, apa yang sedang dipinjam semua orang saat ini. Baru setelah ada dua puluh atau tiga puluh buku di troli, ia duduk, memeriksa setiap buku dengan teliti, dan menentukan urutannya.

Perhatikan bahwa pustakawan memakai *beberapa* naluri, karena masing-masing gagal dengan cara berbeda: pinjaman terakhir Anda tak berguna jika Anda anggota baru, dan "populer bulan ini" tak berguna untuk selera yang niche. Menggabungkannya menutup titik buta. Itulah **pembangkitan kandidat multi-sumber**, dan *docstring* modul `candidates.py` menyebut setiap *retriever* sebagai "alasan berbeda untuk menyukai suatu barang".

Mengapa dua tahap: corong

Misalkan katalog berisi N barang dan kita melayani U permintaan per detik. Menilai setiap barang dengan *ranker* yang butuh c detik per pasangan memerlukan $U \cdot N \cdot c$ detik komputasi setiap detiknya. Dengan N jutaan, bahkan c yang murah pun tak terjangkau, karena *ranker* yang bagus membutuhkan fitur yang dihitung *per pasangan*: riwayat pengguna dengan barang ini, *click-through rate* terkininya, kualitas toko.

Corong menyelesaikan ini dengan membelanjakan komputasi di tempat yang berguna. Setiap tahap melihat lebih sedikit barang sehingga mampu menghabiskan lebih banyak komputasi per barang:



Gambar 8.1: *Corong rekomendasi. Setiap tahap melihat lebih sedikit barang dan dapat menghabiskan lebih banyak komputasi per barang. Filter keras bekerja di titik paling awal.*

Tiga sifat menyusul. **Recall adalah metrik tahap pertama:** dari barang yang nantinya disukai pengguna, berapa banyak yang ada dalam daftar pendek saya? Urutan di dalam daftar pendek tidak penting dan *precision* bersifat sekunder, karena *ranker* yang membersihkan positif palsu. **Daftar pendek adalah batas atas:** jika klik di masa depan tidak ada dalam himpunan kandidat, tidak ada *ranker* yang dapat merekomendasikannya. Inilah *recall ceiling*. **Murah lebih penting daripada elegan:** sebuah *retriever* sebaiknya berupa pencarian indeks, *group-by*, atau kueri tetangga terdekat; apa pun yang membutuhkan fitur per pasangan termasuk tahap kedua.

> **Tahukah kamu?**
>
> **Dua jaringan, satu corong.** Sistem rekomendasi YouTube, sebagaimana dipublikasikan oleh Covington, Adams, dan Sargin, dibagi menjadi tepat dua jaringan saraf: satu untuk pembangkitan kandidat dan satu untuk pemeringkatan. Jaringan pembangkit kandidat menerima riwayat aktivitas pengguna dan mengambil sebagian kecil, "ratusan" video, dari korpus berisi jutaan; jaringan pemeringkat kemudian menilai himpunan kecil itu dengan himpunan fitur yang jauh lebih kaya. Para penulis menunjukkan bahwa rancangan dua tahap ini memungkinkan mereka merekomendasikan dari korpus yang sangat besar sambil tetap yakin bahwa beberapa video yang ditampilkan bersifat personal, dan juga memungkinkan mereka memadukan kandidat yang dihasilkan oleh sumber lain.
>
> Sumber: Covington et al., RecSys, 2016

Makalah yang sama membuat pengamatan kedua yang ditiru repo ini: informasi dari tahap kandidat sebaiknya diteruskan ke *ranker* sebagai fitur, seperti sumber mana yang menominasikan suatu kandidat dan dengan skor berapa. `generate` melakukannya lewat `cand_recent_score`, `cand_covisit_score`, `cand_catpop_score`, `cand_global_pop_rank`, dan `cand_n_sources`.

Empat retriever

Setiap *retriever* menjawab pertanyaan berbeda dan diam untuk kelompok pengguna yang berbeda.

Retriever	Pertanyaan yang dijawab	Analogi	Diam untuk
`recent`	Apa yang *Anda* sentuh belakangan ini?	"Pinjaman terakhir Anda"	Pengguna tanpa riwayat
`covisit`	Apa yang juga disentuh orang yang menyentuh barang *Anda*?	"Pembaca buku ini juga membaca…"	Pengguna tanpa riwayat, atau yang barangnya tak dikunjungi orang lain
`catpop`	Apa yang populer di kategori yang *Anda* sukai?	"Rak terlaris yang Anda jelajahi"	Pengguna tanpa klik *dan* tanpa pencarian
`global_trending`	Apa yang sedang ramai bagi semua orang saat ini?	"Etalase di depan toko"	Tak seorang pun. Daftarnya sama untuk semua

Semuanya berbagi peluruhan waktu eksponensial (gagasan dari Bab 3) dan pemotongan "top-n per pengguna" agar tidak ada sumber yang membanjiri kolam kandidat:

recommender/recsys/candidates.py · _decay

```
def _decay(age_days: pd.Series) -> pd.Series:
    return np.power(0.5, age_days / HALF_LIFE_DAYS)
```

recommender/recsys/candidates.py · _top_per_user

```
def _top_per_user(df: pd.DataFrame, score: str, n: int) -> pd.DataFrame:
    df = df.sort_values(["user_id", score], ascending=[True, False])
    return df.groupby("user_id").head(n)
```

Dengan `HALF_LIFE_DAYS = 7.0`, untuk sebuah aksi berbobot w yang terjadi a hari sebelum *cutoff*:

$$s = w \cdot 2^{-a/7}.$$

Sebuah pesanan ($w = 5$) dari seminggu lalu bernilai 2.5, lima kali lipat klik berumur seminggu ($w = 1$ menghasilkan 0.5). Bobotnya berasal dari `schema.EVENT_WEIGHT`: klik 1, tambah ke keranjang 3, pesanan 5.

Retriever 1: `recent`

Retriever paling sederhana adalah ingatan. Ia menjumlahkan bobot yang telah meluruh dari semua aksi masa lalu pengguna pada setiap barang dan menyimpan 20 barang teratas per pengguna.

recommender/recsys/candidates.py · recent

```
def recent(hist: History) -> pd.DataFrame:
    pos = hist.interactions()
    pos["s"] = pos["w"] * _decay(pos["age_days"])
    out = pos.groupby(["user_id", "item_id"], as_index=False)["s"].sum()
    return _top_per_user(out.rename(columns={"s": "cand_recent_score"}), "cand_recent_score",
N_RECENT)
```

Ini tampak terlalu sepele untuk berguna, tetapi pembeli memang kembali ke barang yang sama: mereka melihat sebuah ponsel, pergi, dan kembali dua hari kemudian. Ini juga daftar benih (*seed*) untuk `covisit`.

Retriever 2: `covisit`

Ko-kunjungan (*co-visitation*) adalah *collaborative filtering* item-ke-item berbasis sesi dari Bab 5. `covisitation_matrix` menghitung seberapa sering dua barang menerima aksi positif dalam sesi yang sama dalam 30 hari dan menormalkannya per barang sumber, sehingga setiap baris berjumlah 1: distribusi $w(a \to b)$ atas barang b yang biasanya menyertai a.

Retriever `covisit` kemudian bergerak dari barang-barang terbaru terbaik milik pengguna ke tetangganya:

recommender/recsys/candidates.py · covisit

```
def covisit(hist: History, seeds: pd.DataFrame) -> pd.DataFrame:
    top_seeds = _top_per_user(seeds, "cand_recent_score", N_SEEDS)
    m = covisitation_matrix(hist)
    j = top_seeds.merge(m, left_on="item_id", right_on="item_id_a")
    j["s"] = j["cand_recent_score"] * j["w"]
    out = j.groupby(["user_id", "item_id_b"], as_index=False)["s"].sum()
    out = out.rename(columns={"item_id_b": "item_id", "s": "cand_covisit_score"})
    return _top_per_user(out, "cand_covisit_score", N_COVISIT)
```

Skor sebuah tetangga b adalah suara berbobot dari para *seed* pengguna:

$$\text{covisit}(u, b) = \sum_{a \in \text{seeds}(u)} \text{recent}(u, a)\; w(a \to b),$$

dengan `seeds(u)` adalah sepuluh barang terbaru terkuat milik pengguna (`N_SEEDS = 10`) dan kita menyimpan 40 tetangga teratas (`N_COVISIT = 40`). Perhatikan ketergantungannya: `covisit` mengambil *keluaran* `recent` sebagai masukannya. Jika pengguna tidak punya riwayat, `recent` kosong, sehingga `covisit` juga kosong.

Retriever 3: `catpop`

Retriever ketiga bekerja pada *kategori*, bukan barang. Pertama ia membangun tabel minat pengguna-per-kategori, `category_affinity`:

recommender/recsys/candidates.py · category_affinity

```
def category_affinity(hist: History) -> pd.DataFrame:
    """User x category interest from clicks/carts/orders AND searches (both decayed), as a
share."""
    pos = hist.interactions().merge(hist.items[["item_id", "category_id"]], on="item_id")
    pos["s"] = pos["w"] * _decay(pos["age_days"])
    se = hist.searched_categories()
    se["s"] = 2.0 * _decay(se["age_days"])  # a search ~ two clicks of intent
    both = pd.concat([pos[["user_id", "category_id", "s"]], se[["user_id", "category_id", "s"]]])
    aff = both.groupby(["user_id", "category_id"], as_index=False)["s"].sum()
    aff["share"] = aff["s"] / aff.groupby("user_id")["s"].transform("sum")
    return aff
```

Dua detail penting. Pencarian dihitung: pencarian "coffee" menjadi *hit* kategori lewat pencocokan substring yang naif dan berbobot 2.0 ("sekitar dua klik niat"), sehingga pembeli yang mencari tetapi tak pernah mengklik tetap punya profil. Dan hasilnya berupa *porsi* (*share*): bobot setiap pengguna berjumlah 1, sehingga pengguna berat tidak tampak "lebih berminat" daripada pengguna ringan.

`catpop` mengambil tiga kategori teratas tiap pengguna dan menggabungkannya (*join*) dengan barang tersedia paling populer di kategori itu:

recommender/recsys/candidates.py · catpop

```
def catpop(hist: History, available: pd.DataFrame) -> pd.DataFrame:
    aff = category_affinity(hist)
    aff = aff.sort_values(["user_id", "share"], ascending=[True,
False]).groupby("user_id").head(N_TOP_CATEGORIES)
    pop = item_popularity(hist, POPULARITY_WINDOW_DAYS).rename("pop")
    items = available[["item_id", "category_id"]].join(pop, on="item_id").fillna({"pop": 0.0})
    items = items.sort_values("pop", ascending=False).groupby("category_id").head(N_PER_CATEGORY)
    items["pop_norm"] = items["pop"] / items.groupby("category_id")
["pop"].transform("max").clip(lower=1e-9)
    j = aff.merge(items, on="category_id")
    j["cand_catpop_score"] = j["share"] * (0.1 + j["pop_norm"])
    return j[["user_id", "item_id", "cand_catpop_score"]]
```

Untuk setiap kategori hanya 20 barang terpopuler dalam 7 hari terakhir yang disimpan (`N_PER_CATEGORY = 20`), dan popularitas dinormalkan terhadap nilai maksimum kategori, menghasilkan `pop_norm` dalam $[0, 1]$. Skornya adalah

$$\text{catpop}(u, i) = \text{share}(u, c(i)) \cdot \big(0.1 + \text{pop_norm}(i)\big).$$

Konstanta 0.1 itu penting: tanpanya, barang dengan popularitas terkini nol akan bernilai tepat nol dan tampak "tidak diusulkan". Dengannya, bahkan listing baru di kategori favorit pengguna tetap memenuhi syarat.

Retriever 4: `global_trending`

Retriever terakhir sengaja dibuat membosankan:

recommender/recsys/candidates.py · global_trending

```
def global_trending(hist: History, available: pd.DataFrame) -> pd.DataFrame:
    pop = item_popularity(hist, 3.0)
    pop = pop[pop.index.isin(available["item_id"])].nlargest(N_GLOBAL)
    return pd.DataFrame({"item_id": pop.index, "cand_global_pop_rank": np.arange(len(pop))})
```

Ia mengambil popularitas setiap barang dalam 3 hari terakhir (jendela yang lebih pendek daripada 7 milik `catpop`, sebab "trending" berarti *saat ini*), hanya menyimpan barang yang stoknya ada, dan mengembalikan 30 teratas beserta peringkatnya. Daftarnya sama untuk setiap pengguna. Nilainya bukan personalisasi, melainkan *cakupan* (*coverage*): ia menjamin setiap pengguna, termasuk pengunjung baru tanpa event apa pun, menerima setidaknya 30 kandidat. Urutan operasinya penting: filter stok terjadi **sebelum** `nlargest(N_GLOBAL)`. Jika diterapkan sesudahnya, setiap barang habis di 30 teratas akan meninggalkan lubang, dan daftar akan menyusut tanpa disadari.

Contoh hitung sederhana

Mari kita hitung satu pengguna dengan tangan. Pada *cutoff*, pengguna u punya riwayat berikut (peluruhan dengan waktu paruh 7 hari):

- Mengklik barang A tujuh hari lalu: $1 \cdot 2^{-1} = 0.5$.
- Memesan barang B empat belas hari lalu: $5 \cdot 2^{-2} = 1.25$.

Maka `recent` mengusulkan B dengan skor 1.25 dan A dengan skor 0.5. Sekarang anggap baris ko-kunjungannya adalah:

seed	tetangga	w
B	C	0.4
B	D	0.6
A	C	0.5
A	E	0.5

Maka skor `covisit` adalah $C = 1.25 \cdot 0.4 + 0.5 \cdot 0.5 = 0.75$, $D = 1.25 \cdot 0.6 = 0.75$, dan $E = 0.5 \cdot 0.5 = 0.25$. Barang C menerima suara dari dua *seed*, yang persis jenis kesepakatan yang ingin kita beri ganjaran.

Terakhir, anggap afinitas pengguna adalah 60% Pets dan 40% Beauty. Sebuah barang makanan hewan dengan `pop_norm = 0.5` mendapat $0.6 \cdot (0.1 + 0.5) = 0.36$, sedangkan barang kecantikan dengan popularitas 0.9 mendapat $0.4 \cdot (0.1 + 0.9) = 0.40$. Barang yang kurang populer di kategori yang lebih disukai bisa kalah dari barang populer di tempat lain; *ranker* yang memutuskan nanti.

Menggabungkan sumber: `generate`

Kini kita punya empat tabel dengan bentuk berbeda: `recent`, `covisit`, dan `catpop` berkunci (pengguna, barang) dengan kolom skornya sendiri, sedangkan `global_trending` adalah daftar yang tak bergantung pada pengguna. `generate` menjahitnya:

recommender/recsys/candidates.py · generate

```
def generate(hist: History, user_ids: np.ndarray) -> pd.DataFrame:
    """Union of all retrievers -> one row per (user_id, item_id) with per-source scores."""
    available = hist.items[hist.items["in_stock"]]
    rec = recent(hist)
    sources = [rec, covisit(hist, rec), catpop(hist, available)]
    users = pd.DataFrame({"user_id": user_ids})
    trending = users.merge(global_trending(hist, available), how="cross")
    sources.append(trending)

    cands = users.merge(sources[0], on="user_id")
    for s in sources[1:]:
        cands = cands.merge(s, on=["user_id", "item_id"], how="outer")
    cands = cands[cands["user_id"].isin(user_ids) & cands["item_id"].isin(available["item_id"])]

    # Negative feedback is a hard filter: never re-show an item the user hid.
    hidden = hist.events.loc[hist.events["event_type"] == "hide", ["user_id",
"item_id"]].drop_duplicates()
    cands = cands.merge(hidden.assign(_hidden=True), on=["user_id", "item_id"], how="left")
    cands = cands[cands["_hidden"].isna()].drop(columns="_hidden")

    score_cols = ["cand_recent_score", "cand_covisit_score", "cand_catpop_score"]
    cands["cand_n_sources"] = cands[score_cols].notna().sum(axis=1) +
cands["cand_global_pop_rank"].notna()
    cands[score_cols] = cands[score_cols].fillna(0.0)
    cands["cand_global_pop_rank"] = cands["cand_global_pop_rank"].fillna(999)
    return cands.reset_index(drop=True)
```

Telusuri dalam empat gerakan.

1. Gabungan lewat outer join. Daftar global di-*cross-join* dengan daftar pengguna sehingga setiap pengguna mendapat 30 baris yang sama. Lalu sumber-sumber digabung dengan `how="outer"` pada `(user_id, item_id)`. Barang yang diusulkan sumber mana pun bertahan. Barang yang diusulkan dua sumber menjadi *satu baris dengan dua kolom skor tak-null*, bukan dua baris. Inilah keputusan desain terpenting dalam fungsi ini: unit tahap pemeringkatan adalah pasangan (pengguna, barang), dan sumber-sumber itu adalah *fitur dari pasangan tersebut*.

2. Filter ketersediaan. `cands["item_id"].isin(available["item_id"])` membuang apa pun yang stoknya habis *pada saat cutoff*. `History.at` membuang waktu kehabisan stok di masa depan, sehingga filter hanya memakai informasi yang tersedia saat permintaan.

3. Filter barang tersembunyi. Tombol "Not interested" mencatat event `hide`. `generate` melakukan *left join* untuk setiap pasangan `(user, item)` yang memiliki event tersebut sebagai kolom penanda, lalu membuang yang cocok: sebuah *filter keras* pada kolam, bukan fitur.

4. Pembukuan sumber. Terakhir fungsi menghitung `cand_n_sources`, jumlah sumber yang mengusulkan pasangan itu, *sebelum* ia mengganti skor yang hilang dengan 0.0 (dan peringkat global yang hilang dengan 999). Urutan penting di sini, dan kita akan memakainya sebagai soal kuis: begitu `fillna` dijalankan pada ketiga kolom skor, kolom-kolom itu tidak pernah null lagi, sehingga masing-masing akan dihitung sebagai sumber untuk setiap baris.

Mengukur retriever: recall, pair recall, dan unique recall

Bagaimana kita tahu bahwa sebuah *retriever* bagus? Dengan aturan dari Bab 2 kita butuh pemisahan waktu: kita membangun kandidat dari riwayat sebelum *cutoff*, lalu melihat apa yang sebenarnya dilakukan pengguna dalam tujuh hari berikutnya (yaitu *label*, di sini 947 pengguna evaluasi dengan 3,357 pasangan (pengguna, barang) positif). Pertanyaan untuk setiap *retriever* kemudian: berapa porsi positif masa depan itu yang ia cakup?

Skrip pendamping `book/code/ch08_candidates.py` menghitung tiga angka per sumber:

- **recall**: untuk setiap pengguna, porsi positif masa depannya yang ditemukan dalam kandidat sumber itu, dirata-ratakan atas pengguna. Ini adalah `evaluate.candidate_recall`.
- **pair recall**: porsi yang sama, tetapi dikumpulkan (*pooled*) atas semua pasangan positif, sehingga pengguna dengan banyak positif berbobot lebih besar.
- **unique pair recall**: porsi pasangan terkumpul yang ditemukan oleh sumber *ini* dan oleh *tidak ada* sumber lain. Angka inilah yang membenarkan keberadaan sumber tersebut.

Unique recall menjawab pertanyaan rekayasa "apa yang hilang jika kita menghapus *retriever* ini?" *Retriever* dengan *recall* tinggi tetapi *unique recall* nol adalah *retriever* yang redundan, dan kita membayarnya dengan latensi dan derau fitur tanpa hasil apa pun.

Jalankan

Dari direktori `book/`:

```
uv run python code/ch08_candidates.py
```

Bagian pertama keluarannya adalah tabel per sumber:

KELUARAN TERMINAL

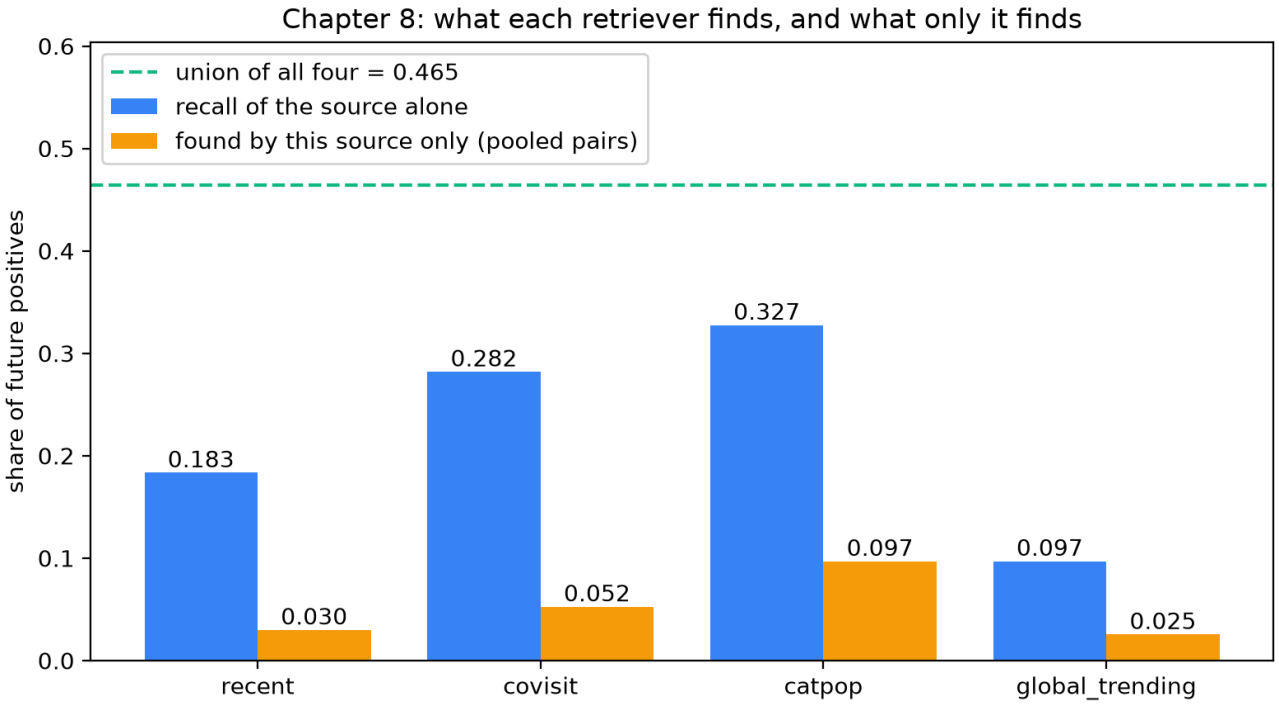
```
=== Retrievers one by one (eval users: 947, label pairs: 3357) ===
                 cands_per_user  users_with_cands  recall  pair_recall  unique_pair_recall
source
recent                 12.1880            0.9704  0.1833       0.2049              0.0298
covisit                35.8321            0.9683  0.2818       0.3125              0.0524
catpop                 54.9102            0.9778  0.3274       0.3545              0.0968
global_trending        30.0000            1.0000  0.0969       0.0983              0.0253
UNION (generate)      103.9366            1.0000  0.4645       0.5019                 NaN

recall = mean over users of (their future positives that are in the set);
pair_recall = same, pooled over all (user, item) positives; unique = found by this source only.
```

`cands_per_user` adalah ukuran rata-rata daftar tiap sumber. `users_with_cands` menunjukkan `recent` dan `covisit` sekitar 0.97, jadi beberapa persen pengguna tidak punya riwayat, sedangkan `global_trending` menjangkau semua orang. Gabungannya rata-rata berisi 103.94 kandidat per pengguna, yaitu "sekitar seratus" dari corong tadi.

Hasil

Gambar di bawah memperlihatkan *recall* per sumber dibandingkan dengan gabungan, serta *recall* pasangan yang ditemukan *hanya* oleh sumber itu.



Gambar 8.2: *Apa yang ditemukan tiap retriever sendirian (biru) dan apa yang hanya ditemukan olehnya (oranye), dibandingkan dengan recall gabungan keempatnya (garis putus-putus).*

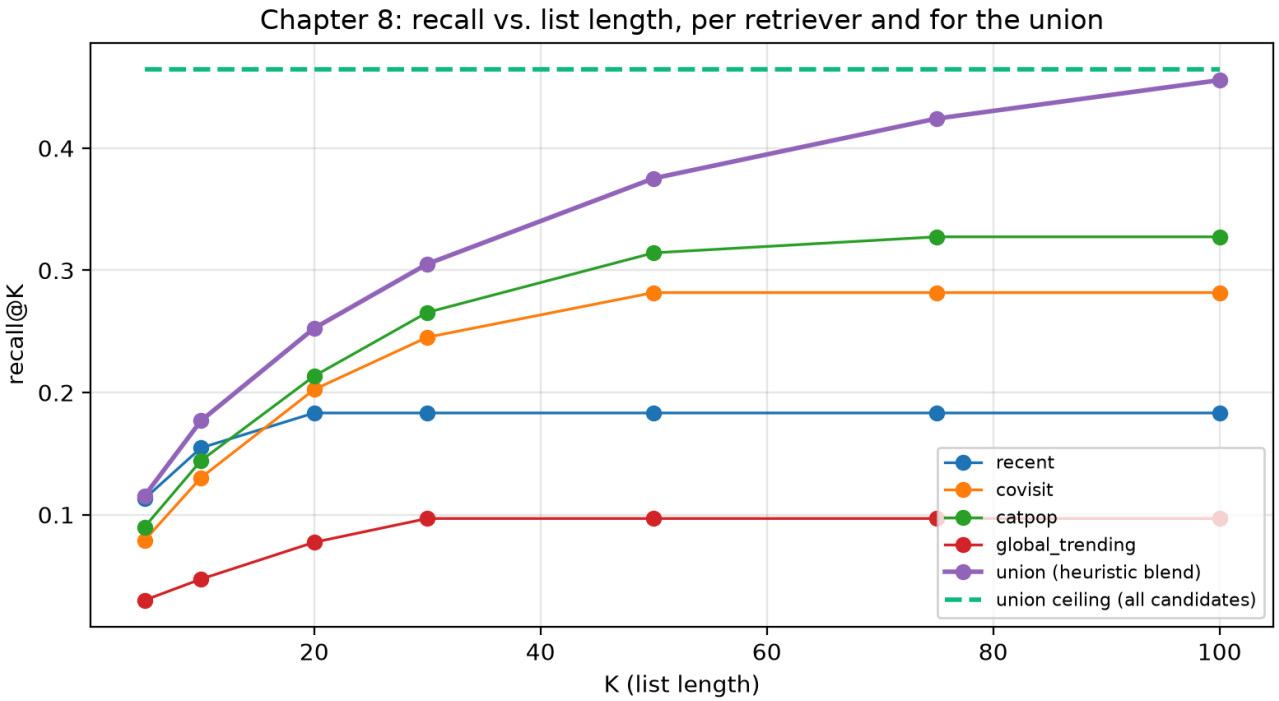
Empat pengamatan, dikaitkan dengan angkanya.

Tidak ada satu pun retriever yang cukup. Sumber tunggal terbaik, `catpop`, mencapai *recall* 0.3274; gabungannya mencapai 0.4645, sekitar 14 poin (kira-kira 42% secara relatif) lebih tinggi. **Tumpang tindih adalah informasi, bukan pemborosan.** Keempat nilai `cands_per_user` berjumlah $12.19 + 35.83 + 54.91 + 30.00 \approx 132.9$, tetapi gabungannya hanya 103.94, jadi sekitar 29 kandidat per pengguna (seperlima) diusulkan oleh lebih dari satu sumber.

Setiap sumber membawa nilai unik, tetapi tidak sama besar. `catpop` menemukan 9.68% dari semua pasangan positif yang tidak ditemukan siapa pun, `covisit` 5.24%, `recent` 2.98%, dan `global_trending` 2.53%. Daftar *trending* punya *recall* terendah tetapi nilai unik hampir sama besar dengan `recent`: ia jaring pengaman murah untuk positif yang tak dijangkau sumber personal mana pun.

Bahkan gabungan pun melewatkan lebih dari separuh masa depan. Batas atas 0.4645 berarti 53.55% dari apa yang diklik, ditambahkan ke keranjang, atau dipesan pengguna dalam seminggu tidak pernah ditawarkan kepada *ranker*. Sebagian tak terhindarkan (penemuan lewat pencarian, iklan, teman); sebagian dapat ditangani dengan lebih banyak sumber, seperti *retrieval two-tower* dari Bab 7.

Recall sebagai fungsi dari berapa banyak kandidat yang kita simpan digambarkan di bawah (nilainya ada di `results/ch08_candidates.json`). Setiap sumber diurutkan menurut skornya sendiri dan gabungan menurut *heuristic blend* milik repo.



Gambar 8.3: *Recall@K untuk setiap retriever (menurut skornya sendiri) dan untuk gabungan yang diurutkan dengan heuristic blend. Sumber tunggal mendatar ketika kehabisan kandidat; gabungan terus menanjak menuju batas atas.*

Ekor yang datar adalah ciri khas *retriever* berukuran tetap. `recent` berada di 0.1833 sejak $K = 20$ karena ia tidak pernah mengusulkan lebih dari 20 barang per pengguna. `covisit` berhenti di 0.2818 sejak $K = 50$, dan `catpop` di 0.3274 sejak $K = 75$. Gabungan yang diurutkan dengan *heuristic blend* terus menanjak, dan pada $K = 100$ mencapai 0.4557, dekat dengan batas atas 0.4645. Ia tidak mencapainya tepat karena rata-rata kolam berisi 103.94 kandidat per pengguna, sehingga pemotongan di 100 masih membuang beberapa positif yang berada di ujung daftar sebagian pengguna. Batas atas adalah *recall* yang Anda dapat ketika tidak ada yang dipotong sama sekali.

Pada $K = 20$, *heuristic blend* mencapai *recall* 0.2526 dari 0.4645 yang tersedia. Selisih 0.21 itu adalah ruang perbaikan yang dapat ditangkap oleh *ranker* yang lebih baik (Bab 10); ia tidak bisa melampaui batas atas, tetapi bisa mendekatinya jauh lebih rapat.

> **Tahukah kamu?**
>
> **Mengapa lebih banyak sumber membantu: kesepakatan adalah bukti.** Kandidat yang diusulkan beberapa *retriever* adalah taruhan yang lebih baik. Pada proses kita, 76,612 kandidat diusulkan oleh tepat satu sumber dan hanya 0.9% di antaranya ternyata positif pada minggu berikutnya. Untuk kandidat yang diusulkan keempat sumber (622 kandidat), tingkat positifnya 18.2%, sekitar dua puluh kali lebih tinggi. Inilah alasan `cand_n_sources` ada sebagai fitur pemeringkatan, mencerminkan praktik yang dijelaskan Covington et al. yaitu meneruskan sumber-sumber penominasi beserta skornya dari pembangkitan kandidat ke *ranker*.
>
> Sumber: Covington et al., RecSys, 2016

Tingkat itu naik secara monoton menurut jumlah sumber (tabel per jumlah ada di `results/ch08_candidates.txt`): 0.0090, 0.0319, 0.0806, dan 0.1817. Setiap *retriever* tambahan yang

secara independen mengusulkan sebuah barang menaikkan peluang bahwa pengguna menginginkannya, dan *ranker* berbasis pohon memakai sinyal ini dengan intens.

Diurutkan dengan *heuristic blend* buatan tangan milik repo tanpa pembelajaran sama sekali, gabungan itu sudah meraih NDCG@20 sebesar 0.1674 dengan *coverage* 0.5682 (porsi katalog berstok yang muncul di 20 teratas seseorang). *Ranker* terlatih di Bab 10 memperbaikinya pada kolam yang sama, selalu *di dalam batas atas yang digambar bab ini*.

> **Tahukah kamu?**
>
> **Dibangun untuk skala dan waktu nyata.** Linden, Smith, dan York menggambarkan *collaborative filtering* item-ke-item milik Amazon sebagai jawaban atas pengecer dengan "puluhan juta pelanggan dan jutaan barang katalog yang berbeda" yang halamannya harus dikembalikan secara *real time*, tidak lebih dari setengah detik. Komputasi daring algoritmanya berskala independen dari jumlah pelanggan dan jumlah barang katalog, karena kerja item-ke-item yang mahal tidak dilakukan per permintaan. *Retriever* `covisit` punya semangat yang sama: matriks ko-kunjungan merangkum 30 hari sesi, dan saat melayani ia hanya digabungkan dengan sepuluh barang *seed* pengguna.
>
> Sumber: Linden et al., IEEE Internet Computing, 2003

Filter keras: yang tidak boleh sampai ke ranker

Beberapa aturan bersifat mutlak, bukan "pelajari sebuah bobot". `generate` menerapkan dua selagi kolam dirakit.

Barang yang stoknya habis

Barang yang tidak bisa dibeli tidak seharusnya ditampilkan, maka `generate` membatasi setiap sumber ke `hist.items[hist.items["in_stock"]]`. Filter mengalahkan fitur di sini karena asimetri kesalahannya: barang habis di posisi 3 mengajari pembeli bahwa rekomendasi tidak dapat diandalkan, sedangkan kehilangannya dari kolam tidak merugikan apa-apa, karena ia memang tak mungkin menghasilkan penjualan.

Barang yang disembunyikan

Ketika pengguna menekan "Not interested", *frontend* mengirim event `hide`; `generate` membuang setiap baris yang memilikinya. Mengapa tidak memberi *ranker* sebuah fitur? Model terlatih bersifat statistik dan mungkin masih menampilkan barang tersembunyi itu sekali dalam seratus permintaan, dan pengguna melihat persis yang satu itu. Pengguna memaafkan rekomendasi yang tidak relevan, tetapi tidak yang secara eksplisit sudah mereka tolak.

Penelusuran kode: bagaimana skrip evaluasi dibangun

Angka-angka *recall* berasal dari `book/code/ch08_candidates.py`. Skrip ini memanggil setiap *retriever*, membatasi hasilnya pada pengguna evaluasi dan barang berstok, lalu membangun himpunan pasangan `(user_id, item_id)` per sumber:

book/code/ch08_candidates.py · unique_recall

```
covered = {}
for name, df in per_source.items():
    covered[name] = set(zip(df["user_id"], df["item_id"]))
label_pairs = set(zip(pairs["user_id"], pairs["item_id"]))
```

Unique recall adalah selisih himpunan. Untuk sumber s, dengan L himpunan pasangan label positif dan C_s pasangan yang dicakup oleh s:

$$\text{unique}(s) = \frac{\left|(L \cap C_s) \setminus \bigcup_{o \neq s} C_o\right|}{|L|}.$$

Baris *union* memanggil `candidates.generate` yang asli, sehingga skrip mengukur jalur produksi. *Global trending* tidak punya skor per pengguna, jadi skrip menegasikan peringkatnya untuk mengubah "peringkat 0 terbaik" menjadi "lebih besar lebih baik". Setiap sumber dievaluasi pada 947 pengguna, label, dan filter ketersediaan yang sama; membandingkan *retriever* pada populasi yang berbeda adalah kesalahan yang umum.

Jebakan dan cara produksi melakukannya

Jebakan 1: mengoptimalkan precision di retrieval. *Ranker* menyingkirkan positif palsu lebih baik daripada *retriever*, dan kandidat yang dibuang hilang selamanya. Optimalkan *recall* pada anggaran kandidat yang tetap.

Jebakan 2: anggaran kandidat adalah anggaran latensi. Setiap kandidat tambahan membebani komputasi fitur dan inferensi. Konstanta di bagian atas `candidates.py` (`N_RECENT`, `N_COVISIT`, `N_PER_CATEGORY`, `N_GLOBAL`) adalah tuasnya; plot *recall* terhadap ukuran kolam (gambar kedua kita) dan berhentilah di titik kurva mendatar.

> **Peringatan**
>
> *Bug* di tahap kandidat sulit terlihat: rekomendasi tetap tampak masuk akal tetapi berasal dari kolam yang lebih miskin. Pantau *candidate recall* secara offline dan `candidates_considered` secara online seperti metrik kesehatan lain; penurunan mendadak biasanya berarti *pipeline* rusak, bukan perilaku pengguna.

Cara sistem produksi melakukannya. Marketplace sungguhan menjalankan jauh lebih banyak *retriever* secara paralel, masing-masing dengan anggaran latensinya sendiri: *retrieval* embedding dengan indeks tetangga terdekat aproksimasi (Bab 7), *retrieval* berbasis sesi, "toko yang sama dengan barang yang Anda pesan", "sering dibeli bersama", daftar kampanye. Langkah penggabungan menerapkan kuota per sumber dan mendeduplikasi menurut barang; filter keras dijalankan terhadap penyimpanan cepat seperti daftar blokir *key-value*; dan *recall* offline pada ukuran kolam adalah pekerjaan harian yang dipantau.

Ringkasan

- Corong dua tahap ada karena meranking setiap barang untuk setiap pengguna dengan model yang kaya tidak terjangkau biayanya.
- Tahap pertama dinilai dengan *recall* dan menetapkan batas atas: gabungan semua sumber berisi 46.45% positif masa depan pengguna, dan tidak ada *ranker* yang bisa melampauinya.
- Repo memakai empat *retriever*: `recent` (riwayat sendiri yang meluruh), `covisit` (barang yang dilihat dalam sesi yang sama), `catpop` (populer di kategori favorit, dari klik dan pencarian), dan `global_trending` (cadangan yang tak bergantung pengguna).
- Sumber digabung dengan *outer join* pada (pengguna, barang); setiap skor disimpan sebagai fitur dan `cand_n_sources` menghitung sumber yang sepakat (tingkat positif 0.9% untuk satu sumber, 18.2% untuk empat).
- Nilai *retriever* dengan *recall* dan *unique recall*; tumpang tindih itu umum (sekitar 133 usulan per pengguna berbanding 103.94 dalam gabungan).
- Barang tersembunyi dan barang habis adalah filter keras pada kolam, bukan fitur.

Kuis

Q8.1 Mengapa *recall*, bukan *precision*, menjadi metrik utama untuk tahap pembangkitan kandidat?

A. Karena *precision* tidak dapat dihitung pada *feedback* implisit.
B. Karena barang yang tidak ada di himpunan kandidat tidak akan pernah bisa dipulihkan oleh *ranker*, sedangkan positif palsu dapat dibuang olehnya.
C. Karena *recall* lebih murah dihitung daripada *precision*.
D. Karena *ranker* hanya bekerja bila masukannya tidak berisi barang yang tidak relevan.

Q8.2 Menurut tabel per sumber di `results/ch08_candidates.txt`, *retriever* mana yang menemukan porsi terbesar pasangan positif yang tidak ditemukan *retriever* lain?

A. `recent`
B. `covisit`
C. `catpop`
D. `global_trending`

Q8.3 Pada keluaran, tingkat positif naik dari 0.0090 untuk kandidat dengan satu sumber menjadi 0.1817 untuk kandidat dengan empat sumber. Apa pemanfaatan temuan ini yang paling tepat dalam sistem?

A. Membuang kandidat yang hanya diusulkan satu sumber.
B. Meneruskan `cand_n_sources` (dan skor per sumber) ke *ranker* sebagai fitur.
C. Hanya mengembalikan kandidat yang diusulkan oleh keempat sumber.
D. Mengganti *ranker* dengan menghitung jumlah sumber.

Q8.4 Seorang pengunjung tanpa riwayat membuka halaman utama. *Retriever* mana yang masih menghasilkan kandidat untuknya?

A. Keempatnya, karena *outer join* mengisi nilai yang hilang.
B. Hanya `recent` dan `covisit`.
C. Hanya `global_trending`.
D. `catpop` dan `global_trending`, karena `catpop` beralih ke barang populer secara keseluruhan.

Q8.5 Gabungan sumber memiliki batas atas *recall* 0.4645, namun gabungan yang diurutkan dengan *heuristic blend* hanya mencapai 0.4557 pada K=100. Jelaskan mengapa angka pada K=100 tidak persis sama dengan batas atas.

Q8.6 Mengapa repo memperlakukan barang "tersembunyi" sebagai filter keras di `generate`, dan tidak memberi *ranker* fitur `was_hidden` lalu membiarkannya belajar menghindarinya?

Q8.7 Baca bagian akhir `generate`: baris `cands["cand_n_sources"] = ...` berada sebelum `cands[score_cols] = cands[score_cols].fillna(0.0)`. Misalkan kedua pernyataan itu ditukar. Nilai berapa yang akan diambil `cand_n_sources` untuk kandidat yang hanya diusulkan `covisit` dan tidak ada dalam daftar *trending*, dan mengapa itu masalah?

Q8.8 Baca `global_trending`. Kode menerapkan `pop[pop.index.isin(available["item_id"])]` sebelum `.nlargest(N_GLOBAL)`. Apa masalah yang akan teramati jika urutannya dibalik?

Q8.9 Dengan memakai kolom `cands_per_user` pada tabel *retriever*, hitung rata-rata jumlah pasangan (pengguna, barang) per pengguna yang diusulkan oleh lebih dari satu sumber, yaitu jumlah keempat ukuran per sumber dikurangi ukuran gabungan. Berikan nilainya hingga satu angka desimal dan sebagai persentase dari jumlah tersebut.

Q8.10 Praktik langsung. Buka `book/code/ch08_candidates.py` dan, di baris paling atas `main()`, tambahkan baris `candidates.N_TOP_CATEGORIES = 5` (nilai bawaannya 3). Jalankan skrip lagi, lalu kembalikan ke semula (`git checkout book/code/ch08_candidates.py book/results`). Perubahan apa yang Anda harapkan pada baris `catpop` (`cands_per_user`, `recall`) dan pada baris gabungan, dan apa yang terjadi pada rata-rata jumlah kandidat per pengguna? Jelaskan dalam satu atau dua kalimat mengapa tugas *ranker* menjadi lebih sulit meskipun *recall* membaik.

Bab 9 — Rekayasa fitur untuk pemeringkatan

Dewi bekerja di tim growth sebuah marketplace yang mirip Shopee. Senin pagi ia menunjukkan kepada manajernya sebuah model pemeringkatan (*ranking*) dengan NDCG offline 0.50, lebih dari dua kali lipat model sebelumnya. Sang manajer senang dan menanyakan tanggal peluncuran. Kamis, model masuk A/B test 5%, dan pada hari Jumat dasbor menunjukkan bahwa performanya tidak lebih baik dari model lama. Di beberapa segmen malah lebih buruk. Dewi menghabiskan akhir pekan membaca *pipeline* fitur barisnya demi baris, dan pukul 2 dini hari Minggu ia menemukannya: salah satu kolom, "jumlah klik pada item ini minggu ini", dihitung dari seluruh minggu *termasuk hari-hari yang seharusnya diprediksi oleh model*. Model itu tidak belajar merekomendasikan. Ia belajar membaca lembar jawaban.

Cerita seperti ini, dalam satu bentuk atau lainnya, terjadi di hampir setiap perusahaan yang membangun sistem rekomendasi. Penyebabnya, fitur (*feature*) adalah tempat sistem pemeringkatan bersentuhan dengan dunia nyata, dan dunia nyata punya sumbu waktu yang tidak dimiliki sebuah *spreadsheet*. Di Bab 8 kita membangun mesin yang mengusulkan beberapa ratus item yang masuk akal untuk setiap pengguna. Di bab ini kita mengubah setiap pasangan (pengguna, item) tersebut menjadi sebaris angka yang bisa dipelajari oleh model pemeringkatan. Cara Anda membangun angka-angka itu lebih penting daripada model mana yang Anda beri makan dengannya.

Yang akan Anda pelajari

- Mengapa sebuah fitur hanya valid jika dihitung *per* waktu pemotongan (*cutoff*), dan bagaimana `History.at` menegakkan aturan itu untuk seluruh basis kode.
- Delapan kelompok fitur yang dipakai di repo ini (total 46 fitur), kegunaan tiap kelompok, dan kelompok mana yang membawa personalisasi dibanding kualitas.
- Bagaimana jendela waktu (3 / 7 / 14 / 30 hari) dan pelicinan Bayesian (*Bayesian smoothing*) mengubah hitungan mentah menjadi rasio yang stabil.
- Bagaimana fitur silang (*cross feature*) seperti pengguna × item, pengguna × kategori, dan pengguna × toko dibangun dengan beberapa panggilan `groupby`.
- Bagaimana membaca satu baris fitur lengkap untuk kandidat nyata, dan membandingkan contoh positif dengan negatif.
- Bagaimana menulis tes kebocoran (*leakage*) otomatis, dan seberapa besar kerusakannya jika Anda benar-benar bocor (NDCG offline 0.50 versus 0.21 yang jujur).

Intuisi: fitur adalah catatan briefing

Bayangkan Anda adalah *personal shopper* yang disewa untuk memilih sepuluh produk bagi seorang pelanggan. Sebelum mulai, seorang asisten menyerahkan catatan briefing satu halaman untuk setiap produk kandidat. Catatan itu berisi hal-hal seperti: "Pelanggan ini melihat produk persis ini tiga kali minggu lalu", "produk ini 40% lebih murah daripada produk sejenis", "toko ini membalas pesan dalam waktu satu jam", "kategori ini mencakup 31% dari yang dijelajahi pelanggan bulan ini". Anda tidak

melihat log mentah jutaan klik. Anda melihat catatannya. Jika catatannya bagus, *shopper* yang biasa-biasa saja pun akan memilih dengan baik. Jika catatannya buruk, *shopper* tercerdas sedunia pun hanya menebak.

Model pemeringkatan adalah *shopper* itu, dan baris fitur adalah catatan briefing-nya. Pohon yang di-*boost* dengan gradien (Bab 10) pandai menggabungkan beberapa lusin angka yang informatif, tetapi tidak bisa mengarang informasi yang tidak ada di dalam baris. Inilah sebabnya para praktisi berkata bahwa modelnya bagian yang mudah, dan fiturnya tempat kerja sebenarnya.

Ada satu aturan lagi untuk catatan briefing, dan inilah aturan yang dilanggar Dewi: **catatan hanya boleh berisi apa yang diketahui pada saat keputusan dibuat**. Seorang *personal shopper* pada hari Selasa tidak bisa diberi tahu apa yang akan dibeli pelanggan pada hari Rabu. Dalam bahasa *machine learning*, fitur untuk sebuah contoh pelatihan harus dihitung pada waktu pemotongan T, hanya dengan peristiwa yang cap waktunya sebelum T (benar-benar sebelum), sedangkan label berasal dari peristiwa pada atau setelah T. Kita menyebut properti ini *kebenaran pada titik waktu* (*point-in-time correctness*). Pelanggarannya disebut kebocoran data (*leakage*).

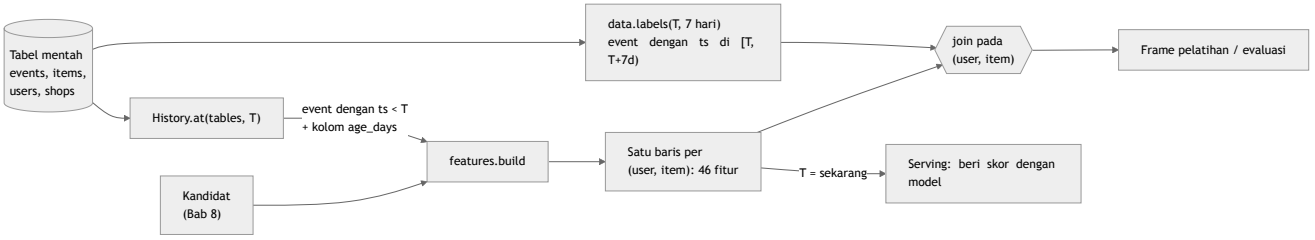
Algoritma: dari peristiwa mentah ke baris fitur

Langkah 0: tetapkan cutoff dan bagi dunia menjadi dua

Misalkan T adalah cap waktu pemotongan. Segala yang boleh dipakai sistem adalah *riwayat* (*history*) H_T:

$$H_T = \{\, e \in \text{events} : e.\text{ts} < T \,\} \cup \{\, i \in \text{items} : i.\text{listed_at} \le T \,\}$$

Label berasal dari jendela masa depan $[T, T + 7\ \text{hari})$. Kode yang sama dipakai saat pelatihan (dengan T di masa lalu, sehingga masa depan dapat diamati dan kita bisa memberi label) dan saat penyajian (*serving*) (dengan T = sekarang, sehingga masa depan kosong). Kesimetrisan ini adalah keputusan desain terpenting dalam *pipeline* fitur. Jika pelatihan dan *serving* berbagi satu jalur kode, satu kelas bug yang dikenal sebagai *training/serving skew* tidak mungkin terjadi.



Gambar 9.1: *Fitur dihitung dari masa lalu cutoff, label dari masa depannya; saat serving, bagian masa depan itu kosong.*

Langkah 1: tambahkan umur pada setiap peristiwa

Alih-alih menyimpan cap waktu absolut, `History.at` menghitung di muka `age_days = (T − ts) / 1 hari` untuk setiap peristiwa yang lolos. Sebuah jendela lalu menjadi filter sederhana: "klik dalam 7 hari terakhir" adalah `age_days < 7`. Kolom umur ini juga membuat mustahil menulis fitur yang bergantung pada jam dinding saat ini, karena satu-satunya jam adalah T.

Langkah 2: agregasi di atas jendela

Untuk sebuah entitas e (pengguna, item, atau pasangan) dan jenis peristiwa t, hitungan berjendela adalah

$$n_{e,t}(w) = \left|\{\,\text{events of type } t \text{ for } e \text{ with age} < w\,\}\right|.$$

Kita memakai beberapa jendela karena masing-masing menjawab pertanyaan yang berbeda. Jendela 3 hari berkata "apa yang sedang ramai sekarang", jendela 30 hari berkata "apa yang stabil bagus", dan rasio keduanya menunjukkan apakah sesuatu sedang menghangat atau mendingin. Fitur *trend* di repo adalah

$$\text{trend} = \frac{n_{\text{click}}(3)/3 + 0.1}{n_{\text{click}}(14)/14 + 0.1}.$$

Kedua suku adalah *laju klik per hari*. Nilai di atas 1 berarti tiga hari terakhir rata-rata lebih ramai daripada dua minggu terakhir. Konstanta $+0.1$ adalah bilangan kecil yang mencegah pembagian dengan nol dan menjaga hitungan yang sangat kecil agar tidak menghasilkan rasio yang konyol.

Langkah 3: haluskan rasio

Click-through rate (CTR) mentah adalah $\text{clicks}/\text{impressions}$. Item yang ditampilkan sekali dan diklik sekali punya CTR 100%, yang tidak masuk akal. Obatnya adalah pelicinan Bayesian: anggap setiap item memulai dengan s tayangan khayalan pada laju rata-rata global p_0, lalu biarkan data nyata berangsur mengambil alih:

$$\widehat{\text{CTR}} = \frac{\text{clicks} + s \cdot p_0}{\text{impressions} + s}.$$

Dengan $s = 50$ dan CTR global $p_0 = 0.14$, 50 tayangan khayalan pertama menyumbang $50 \times 0.14 = 7$ klik khayalan. Ambil dua item:

Item	Klik	Tayangan	CTR mentah	CTR yang dihaluskan
A (baru, beruntung)	1	1	1.000	$(1+7)/(1+50) = 0.157$
B (mapan)	30	100	0.300	$(30+7)/(100+50) = 0.247$

Item baru ditarik hampir sepenuhnya kembali ke rata-rata, sementara item mapan mempertahankan sebagian besar keunggulannya. Parameter s adalah "kekuatan keyakinan" pada prior: makin besar nilainya, makin banyak bukti yang dibutuhkan sebelum data milik item itu sendiri menang. Gagasan yang sama muncul di Bab 3 untuk rating.

Langkah 4: bangun fitur silang

Fitur item menggambarkan item untuk semua orang, dan fitur pengguna menggambarkan pengguna untuk semua item. Tak satu pun menyatakan apakah *pengguna ini* akan menyukai *item ini*. Informasi itu ada di *fitur silang* (*cross feature*), yaitu agregat atas sepasang kunci:

- **pengguna × item**: seberapa sering pengguna ini mengklik, memasukkan ke keranjang, atau memesan item persis ini? Berapa jam lalu? Berapa kali item itu ditampilkan kepadanya lalu

diabaikan?

- **pengguna × kategori**: berapa porsi klik terbaru pengguna ini yang jatuh pada kategori item ini? Berapa pesanan yang ia buat di sana, dan berapa hari lalu?
- **pengguna × toko**: apakah pengguna ini pernah membeli dari penjual ini?
- **level harga pengguna × harga item**: apakah item ini murah atau mahal *bagi pengguna ini*?

Fitur silang adalah tempat personalisasi terjadi. Sebagian besar daya prediksi mentah dalam sebuah *ranker* marketplace berasal dari kombinasi "pengguna ini punya riwayat dengan sesuatu yang dekat dengan item ini".

Langkah 5: tangani nilai yang hilang secara sengaja

Pasangan yang tidak pernah berinteraksi tidak punya baris di tabel agregat, sehingga *join* menghasilkan nilai yang hilang (*missing*). Nilai yang hilang bisa berarti dua hal berbeda: "nol" (tidak ada klik) atau "tidak diketahui atau tidak pernah" (belum pernah memesan, sehingga jumlah hari sejak pesanan terakhir tidak ada). Repo ini memilih per fitur. Hitungan menjadi 0. Fitur keterkinian (*recency*), yang nilai 0-nya berarti "baru saja", diberi nilai sentinel yang besar: 9999 jam untuk "tidak pernah disentuh" atau 999 hari untuk "tidak pernah memesan". Pohon membelah pada ambang batas, sehingga sentinel yang jauh dari nilai nyata memisahkan kedua kasus dengan bersih.

> **Tahukah kamu?**
>
> **Apakah fitur teratas mendominasi model pemeringkatan?** Dalam studi prediksi klik di Facebook yang banyak dikutip, para penulis mem-*boost* pohon atas ratusan fitur dan menemukan bahwa 10 fitur teratas bertanggung jawab atas sekitar separuh dari total kepentingan fitur, sedangkan 300 fitur terakhir menyumbang kurang dari 1%. Mereka juga menemukan bahwa fitur *historis* (seperti klik kumulatif pada sebuah iklan) memiliki daya jelas yang jauh lebih besar daripada fitur kontekstual: 10 fitur teratas semuanya historis. Fitur kontekstual terutama penting untuk *cold start*.
>
> Sumber: He et al., ADKDD (KDD workshop), 2014

Penelusuran kode

Penjaga gerbang: `History.at`

Seluruh logika titik-waktu proyek ini berada dalam satu *class method*. Bacalah dengan cermat, karena setiap fitur dalam sistem bergantung padanya.

recommender/recsys/data.py · History.at

```python
@classmethod
def at(cls, tables: dict[str, pd.DataFrame], cutoff: pd.Timestamp) -> History:
    ev = tables["events"]
    ev = ev[ev["ts"] < cutoff].copy()
    ev["age_days"] = (cutoff - ev["ts"]) / pd.Timedelta(days=1)
    se = tables["searches"]
    se = se[se["ts"] < cutoff].copy()
    se["age_days"] = (cutoff - se["ts"]) / pd.Timedelta(days=1)
    cats = tables["categories"]
    items = tables["items"]
    items = items[items["listed_at"] <= cutoff].merge(
        cats[["category_id", "is_consumable", "median_price"]], on="category_id"
    )
    # Stock status is known at cutoff; the *future* stock-out time is not.
    items["in_stock"] = items["out_of_stock_at"].isna() | (items["out_of_stock_at"] > cutoff)
    items = items.drop(columns="out_of_stock_at")
    return cls(cutoff, ev, se, items, tables["users"], tables["shops"], cats)
```

Telusuri baris demi baris:

1. `ev["ts"] < cutoff` menyimpan peristiwa yang benar-benar *sebelum* cutoff. Pertidaksamaannya ketat, dan itu penting: peristiwa bercap tepat T termasuk jendela label, bukan riwayat. Fungsi label memakai `ts >= cutoff`, sehingga setiap peristiwa jatuh tepat di satu sisi.
2. `age_days` dihitung terhadap cutoff, tidak pernah terhadap `now()`.
3. Pencarian (*search*) mendapat perlakuan yang sama, karena kueri pencarian sama berharganya sebagai sinyal perilaku seperti klik.
4. Katalog difilter dengan `listed_at <= cutoff`. Item yang baru didaftarkan Selasa depan tidak ada bagi model yang berdiri pada hari Jumat. Ini versi katalog dari aturan yang sama, dan inilah sebabnya produk yang benar-benar baru tidak bisa muncul dalam kandidat pelatihan sebelum peluncurannya.
5. `in_stock` bernilai benar jika item tidak punya waktu kehabisan stok *atau* kehabisan stok terjadi setelah cutoff, dan waktu kehabisan stok di masa depan dibuang, sehingga model tidak bisa belajar "item ini akan habis dalam dua hari". Harga, rating, dan jumlah rating ditampilkan sebagaimana adanya *saat ini*; dalam data simulasi ini nilainya tidak pernah berubah, sehingga laboratorium tetap jujur. Katalog sungguhan berubah, itulah sebabnya produksi menyimpan *snapshot* (lihat bagian jebakan).

Dataclass `History` bersifat *frozen* dan tidak ada kode lain yang membaca tabel `events` mentah, jadi fungsi fitur yang menerima `History` tidak dapat melihat masa depan. Satu-satunya kode yang menengok ke depan adalah `data.labels`: ia memilih peristiwa di $[T, T + 7d)$, memetakan click / add_to_cart / order ke label bertingkat 1 / 2 / 3 lewat `schema.LABEL_GAIN`, dan menyimpan nilai maksimum per (pengguna, item).

Blok bangunan

Dua pembantu kecil di `features.py` melakukan sebagian besar agregasi. `_count` memfilter satu jenis peristiwa, opsional ke sebuah jendela dengan `age_days < max_age`, lalu mengelompokkan berdasarkan daftar kunci apa pun (pengguna, item, atau pasangan). `_smoothed` adalah rumus Langkah 3 pada seluruh kolom: `num` adalah klik, `den` tayangan, `prior` adalah p_0, `strength` adalah s. Fitur pengguna (`user_features`) memakai pembantu yang sama: empat penghitung berjendela, CTR

yang dihaluskan dengan kekuatan 20, serta keterkinian dan umur akun yang diukur terhadap cutoff, tidak pernah terhadap jam dinding.

Fitur item

recommender/recsys/features.py · item_features

```
def item_features(hist: History) -> pd.DataFrame:
    ev = hist.events
    it = hist.items.copy()
    it["item_price"] = it["price"]
    it["item_discount_pct"] = it["discount_pct"]
    it["item_price_vs_category"] = it["price"] / it["median_price"]
    it["item_free_shipping"] = it["free_shipping"].astype(int)
    it["item_rating_avg"] = it["rating_avg"]
    it["item_rating_count"] = np.log1p(it["rating_count"])
    it["item_age_days"] = (hist.cutoff - it["listed_at"]) / pd.Timedelta(days=1)
    it["item_is_consumable"] = it["is_consumable"].astype(int)

    counts = [
        ("item_impressions_7d", "impression", 7),
        ("item_clicks_7d", "click", 7),
        ("item_orders_7d", "order", 7),
        ("_impr_30d", "impression", 30),
        ("_clicks_30d", "click", 30),
        ("_orders_30d", "order", 30),
        ("_clicks_3d", "click", 3),
        ("_clicks_14d", "click", 14),
        ("_orders_all", "order", None),
        ("_returns_all", "return", None),
    ]
    for name, et, age in counts:
        it = it.merge(_count(ev, ["item_id"], et, age, name), on="item_id", how="left")
    it = it.fillna({name: 0 for name, _, _ in counts})

    ctr = it["_clicks_30d"].sum() / max(it["_impr_30d"].sum(), 1)
    cvr = it["_orders_30d"].sum() / max(it["_clicks_30d"].sum(), 1)
    ret = it["_returns_all"].sum() / max(it["_orders_all"].sum(), 1)
    it["item_ctr_30d"] = _smoothed(it["_clicks_30d"], it["_impr_30d"], ctr, 50)
    it["item_cvr_30d"] = _smoothed(it["_orders_30d"], it["_clicks_30d"], cvr, 20)
    it["item_return_rate"] = _smoothed(it["_returns_all"], it["_orders_all"], ret, 10)
    it["item_trend"] = (it["_clicks_3d"] / 3 + 0.1) / (it["_clicks_14d"] / 14 + 0.1)

    sh = hist.shops.rename(columns={"is_mall": "shop_is_mall", "cancel_rate": "shop_cancel_rate",
"response_rate": "shop_response_rate"})
    sh["shop_is_mall"] = sh["shop_is_mall"].astype(int)
    it = it.merge(sh[["shop_id", "city", "shop_rating", "shop_is_mall", "shop_cancel_rate",
"shop_response_rate"]], on="shop_id")
    return it.rename(columns={"city": "shop_city"})
```

Blok pertama penugasan menyalin atribut katalog statis ke kolom fitur. `item_rating_count` melewati `np.log1p` karena hitungan berekor berat (5 versus 50 rating lebih bermakna daripada 5,005 versus 5,050), dan `item_price_vs_category` membagi dengan median kategori sehingga sarung ponsel dan laptop dibandingkan pada "murah atau premium untuk ukurannya". Blok tengah mendeklarasikan sepuluh penghitung sebagai tripel `(name, event_type, window)`; nama berawalan garis bawah adalah masukan sementara untuk rasio yang dihaluskan (kekuatan 50 untuk CTR, 20 untuk konversi, 10 untuk retur, lebih rendah untuk peristiwa yang lebih jarang). Setelah itu datang rumus *trend* dari Langkah 2 dan *join* dengan kualitas toko.

Fitur silang

Tiga fungsi membangun fitur silang. Tampilannya berbeda tetapi berbagi satu pola: *gabungkan peristiwa ke kunci kedua, lalu kelompokkan berdasarkan pasangan*.

recommender/recsys/features.py · user_item_features

```
def user_item_features(hist: History) -> pd.DataFrame:
    ev = hist.events
    keys = ["user_id", "item_id"]
    ui = ev[keys].drop_duplicates()
    for name, et in [("ui_clicks", "click"), ("ui_carts", "add_to_cart"), ("ui_orders", "order"),
("_impr", "impression")]:
        ui = ui.merge(_count(ev, keys, et, None, name), on=keys, how="left")
    ui = ui.fillna(0)
    pos = ev[ev["event_type"].isin(schema.EVENT_WEIGHT)]
    last = (pos.groupby(keys)["age_days"].min() * 24).rename("ui_hours_since_last")
    dwell = ev.groupby(keys)["dwell_sec"].max().rename("ui_max_dwell_sec")
    ui = ui.join(last, on=keys).join(dwell, on=keys)
    ui["ui_ignored_impressions"] = np.where(ui["ui_clicks"] == 0, ui["_impr"], 0)
    return ui.drop(columns="_impr")
```

`user_item_features` menghitung klik, keranjang, pesanan, dan tayangan per pasangan. `ui_ignored_impressions` adalah berapa kali pasangan itu ditampilkan *hanya jika pengguna tidak pernah mengkliknya*, yaitu sinyal kejenuhan: item yang ditampilkan sepuluh kali tanpa klik kemungkinan besar tidak menarik. `ui_hours_since_last` hanya memakai aksi positif (klik, keranjang, pesanan) dan mengubah hari menjadi jam. `ui_max_dwell_sec` adalah lama tinggal terpanjang di halaman produk: pengguna yang menghabiskan tujuh menit di halaman produk memberi tahu kita lebih banyak daripada yang pergi setelah dua detik.

Blok pengguna × kategori menghitung *porsi* klik (`transform("sum")` membagi dengan total pengguna, sehingga pengklik berat dan ringan dapat dibandingkan), ditambah pencarian yang dipetakan ke kategori dan keterkinian pesanan. Blok pengguna × toko memakai pola yang sama. `user_price_level` adalah median dari `price / category_median_price` atas klik pengguna; membagi rasio milik item dengannya menghasilkan `price_ratio_to_user`, sehingga 0.62 berarti item itu berharga 62% dari yang biasa dibayar pengguna ini relatif terhadap norma kategori.

Menyatukan semuanya: `build`

recommender/recsys/features.py · build

```
def build(cands: pd.DataFrame, hist: History) -> pd.DataFrame:
    """Attach every feature in schema.FEATURES to the candidate rows."""
    df = cands.merge(item_features(hist), on="item_id", how="left")
    df = df.merge(user_features(hist), on="user_id", how="left")
    df = df.merge(user_item_features(hist), on=["user_id", "item_id"], how="left")
    df = df.merge(user_category_features(hist), on=["user_id", "category_id"], how="left")
    df = df.merge(user_shop_features(hist), on=["user_id", "shop_id"], how="left")
    df = df.join(user_price_level(hist), on="user_id")
    df = df.merge(hist.users[["user_id", "city"]].rename(columns={"city": "user_city"}),
on="user_id", how="left")

    df["price_ratio_to_user"] = df["item_price_vs_category"] / df["_user_price_level"].fillna(1.0)
    df["same_city"] = (df["user_city"] == df["shop_city"]).astype(int)
    fill = {
        "ui_hours_since_last": NEVER_HOURS,
        "uc_days_since_order": NEVER_DAYS,
        "user_days_since_active": NEVER_DAYS,
    }
    df = df.fillna(fill)
    zero = ["ui_clicks", "ui_carts", "ui_orders", "ui_ignored_impressions", "ui_max_dwell_sec",
"uc_click_share_30d",
            "uc_searches_30d", "uc_orders", "us_clicks", "us_orders", "user_clicks_7d",
"user_clicks_30d",
            "user_orders_30d"]  # fmt: skip
    df[zero] = df[zero].fillna(0)

    # Keep ids for grouping/re-ranking, then exactly the registered features, in order.
    out = df[["user_id", "item_id", "category_id", "shop_id"] + schema.FEATURE_NAMES]
    assert list(out.columns[4:]) == schema.FEATURE_NAMES
    return out.reset_index(drop=True)
```

`build` menerima *frame* kandidat dari Bab 8 (`user_id`, `item_id`, lima skor sumber kandidat) dan melakukan *left join*: kandidat dipertahankan, fitur ditempelkan. Dua detail mudah terlewat. `assert` di akhir menjamin kolom keluaran sama dengan `schema.FEATURE_NAMES` dengan urutan yang sama, sehingga fitur tidak bisa ditambahkan ke kode lalu terlupa di skema, atau sebaliknya. Dan kamus `fill` dan `zero` yang eksplisit adalah tempat kebijakan nilai hilang pada Langkah 5 dituliskan di satu tempat. Karena fungsi yang sama berjalan offline dan di dalam layanan gRPC, model melihat semantik kolom yang identik di keduanya.

Delapan kelompok fitur

Skema mendaftarkan setiap fitur dengan nama, kelompok, dan deskripsi. Kelompok beserta ukurannya:

Kelompok	Jumlah	Yang digambarkan	Contoh fitur
`candidate`	5	Retriever mana yang mengusulkan item dan seberapa kuat	`cand_recent_score`, `cand_covisit_score`, `cand_n_sources`
`user`	8	Tingkat aktivitas dan profil pengguna	`user_clicks_30d`, `user_ctr_30d`, `user_days_since_active`
`item`	15	Harga, kualitas, popularitas, kesegaran, tren	`item_ctr_30d`, `item_cvr_30d`, `item_trend`, `item_age_days`
`shop`	4	Keandalan penjual	`shop_rating`, `shop_is_mall`, `shop_cancel_rate`
`user_item`	7	Riwayat pengguna ini dengan item persis ini	`ui_clicks`, `ui_hours_since_last`, `ui_max_dwell_sec`, `price_ratio_to_user`
`user_category`	4	Hubungan pengguna dengan kategori item	`uc_click_share_30d`, `uc_days_since_order`
`user_shop`	2	Hubungan pengguna dengan penjual	`us_clicks`, `us_orders`
`context`	1	Konteks permintaan atau pasangan	`same_city`

Jumlahnya genap 46. Perhatikan proporsinya. Lima belas fitur menggambarkan item saja, dan itulah kelompok terbesar. Tetapi kelompok `user_item` (7), `user_category` (4), `user_shop` (2), dan `context` (1) bersama-sama memuat 14 fitur tingkat pasangan. Keempat belas fitur itulah yang membuat dua pengguna melihat daftar yang berbeda.

> **Tahukah kamu?**
>
> **Kebocoran punya definisi formal.** Kaufman, Rosset, dan Perlich memberi kebocoran sebuah pernyataan yang presisi dalam *data mining*: kebocoran adalah masuknya informasi tentang target yang seharusnya tidak tersedia secara sah bagi model pada saat prediksi. Mereka mengkatalogkan cara kebocoran menyelinap, lewat fitur yang menjadi proksi label, lewat cara sebuah *data set* disampel, dan lewat waktu, serta memakai *data set* kompetisi untuk menunjukkan seberapa sering hal itu menentukan pemenang dan pecundang.
>
> Sumber: Kaufman et al., ACM Transactions on Knowledge Discovery from Data, 2012

Tes kebocoran

Disiplin itu baik, tetapi tes otomatis lebih baik. Repo ini menyertakan tes yang menulis ulang masa depan dan memeriksa bahwa tidak ada yang berubah pada fitur.

`recommender/tests/test_point_in_time.py · _build`

```python
def _build(tables, cutoff):
    hist = data.History.at(tables, cutoff)
    users = tables["users"]["user_id"].to_numpy()
    feats = features.build(candidates.generate(hist, users), hist)
    return feats.sort_values(["user_id", "item_id"]).reset_index(drop=True)
```

Pembantu ini membangun seluruh *frame* fitur pada suatu cutoff, termasuk kandidat, dan mengurutkannya agar urutan baris tidak menyebabkan kegagalan palsu.

recommender/tests/test_point_in_time.py · test_features_ignore_events_after_cutoff

```python
def test_features_ignore_events_after_cutoff(tables):
    cutoff = pd.Timestamp(SimConfig().start_date) + pd.Timedelta(days=CUTOFF_DAY)
    before = _build(tables, cutoff)

    # Rewrite the future: every post-cutoff event becomes an order of item 0, and
    # every post-cutoff search becomes a query for a real category.
    future = tables["events"]["ts"] >= cutoff
    assert future.any()
    tampered = dict(tables)
    ev = tables["events"].copy()
    ev.loc[future, ["event_type", "item_id"]] = ["order", 0]
    tampered["events"] = ev
    se = tables["searches"].copy()
    se.loc[se["ts"] >= cutoff, "query"] = "diapers"
    tampered["searches"] = se

    pd.testing.assert_frame_equal(before, _build(tampered, cutoff))
```

Tes berjalan dalam empat langkah. Ia membangun fitur pada hari ke-30 simulasi. Lalu ia mengutak-atik masa depan: *setiap* peristiwa setelah cutoff menjadi pesanan item 0, dan setiap pencarian sesudahnya menjadi kueri "diapers". Kemudian ia membangun ulang, dan `assert_frame_equal` menuntut kedua *frame* identik, sel demi sel. Jika sebuah fitur baru membaca satu peristiwa masa depan saja, hasil yang diutak-atik akan berbeda dan tes gagal. Tes ini sengaja dibuat brutal. Kebocoran yang halus, misalnya jendela 30 hari yang tanpa sengaja merentang ke depan lewat `abs(age_days)`, akan menghasilkan perbedaan kecil di segelintir sel, dan tetap akan tertangkap.

Jalankan

Skrip pendamping `ch09_features.py` membangun *frame*, mencetak satu baris fitur lengkap, membandingkan contoh positif dan negatif, lalu menjalankan eksperimen kebocoran. Dari direktori `book/`:

```
uv run python code/ch09_features.py
```

Pertama, *frame*-nya. *Frame* pelatihan memakai cutoff satu minggu lebih awal daripada *frame* evaluasi, dengan label dari tujuh hari berikutnya pada masing-masing, meniru *pipeline* pelatihan yang sebenarnya.

KELUARAN TERMINAL

```
=== Frames ===
train frame:   78793 rows, 781 users, 1221 positive rows  (cutoff 2026-09-16 00:00:00)
eval  frame:   98428 rows, 947 users, 1685 positive rows  (cutoff 2026-09-23 00:00:00)
number of features: 46 in 8 groups
```

Sekitar 78,800 baris pelatihan dan 98,400 baris evaluasi berasal dari masing-masing 781 dan 947 pengguna. Tingkat positifnya sedikit di atas 1.7%, yang lazim: pada sebagian besar kandidat, pengguna tidak melakukan apa-apa dalam seminggu ke depan. Kita akan kembali ke ketidakseimbangan kelas ini di Bab 10.

Sekarang satu baris kandidat dengan seluruh 46 fitur: kandidat dengan skor heuristik terbaik untuk pengguna 42, diringkas di sini menjadi kelompok kandidat dan pengguna (baris lengkapnya ada di berkas hasil).

KELUARAN TERMINAL

```
=== One candidate row, every feature (user 42, best heuristic score) ===
user_id=42 item_id=2381 category_id=9 shop_id=10 label=0
  [candidate]
    cand_recent_score      =     11.629   Time-decayed weight of the user's own past actions on
this item
    cand_covisit_score     =      3.564   Co-visitation: how often this item is clicked in the
same session as the user's recent items
    cand_catpop_score      =      0.381   Popularity of the item inside the user's favourite
categories
    cand_global_pop_rank   =      0.000   Rank in global trending list (999 = not in list)
    cand_n_sources         =      4.000   How many retrievers proposed this item
  [user]
    user_clicks_7d         =      9.000   User's clicks in the last 7 days (activity level)
    user_clicks_30d        =     45.000   User's clicks in the last 30 days
    user_orders_30d        =     14.000   User's orders in the last 30 days
    user_ctr_30d           =      0.132   User's click-through rate (smoothed)
    user_days_since_active =      1.341   Days since the user's last event
    user_account_age_days  =    336.000   Days since signup
    user_device_tier       =      3.000   Phone tier 1-3
    user_age_bucket        =      3.000   Age bucket 0-3
  [item]
```

Hasil: membaca satu baris fitur

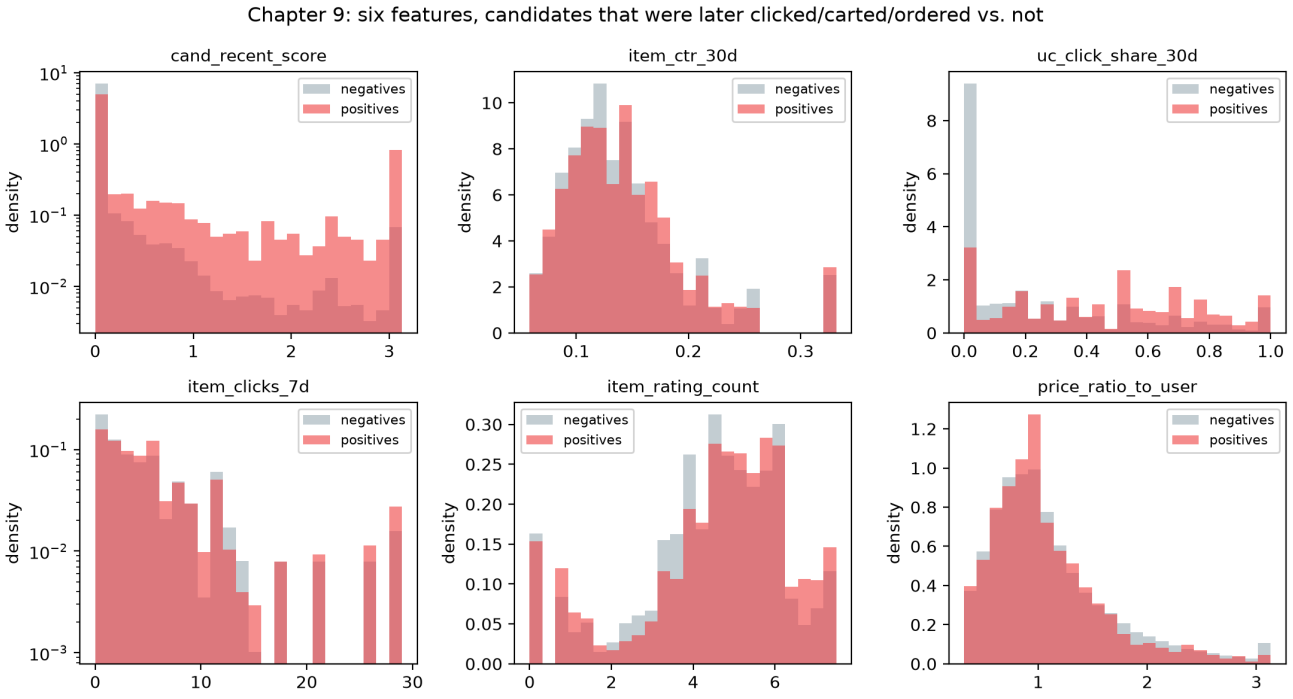
Bacalah baris itu seperti detektif. Kandidatnya adalah item 2381, dan catatan briefing berkata: pengguna telah berinteraksi langsung dengannya (`cand_recent_score` bernilai 11.63, bobot peluruhan yang besar), item itu diusulkan oleh keempat retriever (`cand_n_sources` = 4) dan menempati nomor 1 dalam daftar *trending* global (`cand_global_pop_rank` = 0). Pengguna aktif (45 klik dalam 30 hari, 14 pesanan) dan terakhir terlihat 1.3 hari lalu. Item itu murah untuk kategorinya (`item_price_vs_category` = 0.60), punya rating terbaik (5.0) dengan $e^{5.673} - 1 \approx 290$ rating, dan sedang populer sekarang (29 klik dalam tujuh hari). Fitur silang menceritakan kisah personalnya: enam klik, enam penambahan ke keranjang, dan lima pesanan pada item persis ini, dan 31% klik kategori pengguna jatuh pada kategori ini.

Apakah contoh positif tampak berbeda dari negatif?

Sebuah fitur hanya berguna jika distribusinya berbeda antara kandidat yang kemudian dilibati dan yang tidak. Skrip mencetak rata-rata dan median untuk enam fitur representatif.

KELUARAN TERMINAL

```
=== Positives vs negatives (eval frame): mean of selected features ===
                     mean_positive  mean_negative  median_positive  median_negative
feature
cand_recent_score            0.847          0.099            0.000            0.000
item_ctr_30d                 0.144          0.141            0.135            0.129
uc_click_share_30d           0.447          0.245            0.462            0.143
item_clicks_7d               5.893          5.204            4.000            3.000
item_rating_count            4.555          4.414            4.920            4.682
price_ratio_to_user          1.073          1.118            0.980            0.982
base rate of positives among candidates: 0.0171
```



Gambar 9.2: *Distribusi enam fitur untuk contoh positif dan negatif*

`cand_recent_score` memisahkan kedua kelas dengan tajam (rata-rata 0.847 untuk positif versus 0.099; aksi terbaru pengguna sendiri adalah prediktor terkuat), dan `uc_click_share_30d` melakukan hal yang sama secara personal (0.447 versus 0.245; contoh negatif menumpuk di nol). `item_ctr_30d` (0.144 versus 0.141), `item_rating_count` (4.555 versus 4.414), dan `price_ratio_to_user` (1.073 versus 1.118) nyaris tidak berbeda, dan `item_clicks_7d` hanya sedikit lebih tinggi (5.893 versus 5.204). Fitur yang tampak lemah masih bisa membantu pohon dalam interaksi dengan fitur lain, jadi jangan pernah menghapus fitur hanya berdasarkan histogram.

Seperti apa bentuk kebocoran

Sekarang eksperimen yang menjelaskan akhir pekan Dewi. Skrip menambahkan *satu* kolom ekstra, `leaky_future_clicks`, yang menghitung klik pengguna pada item di dalam jendela label. Berikut baris persis yang menciptakan kebocoran itu:

book/code/ch09_features.py · label_window_clicks

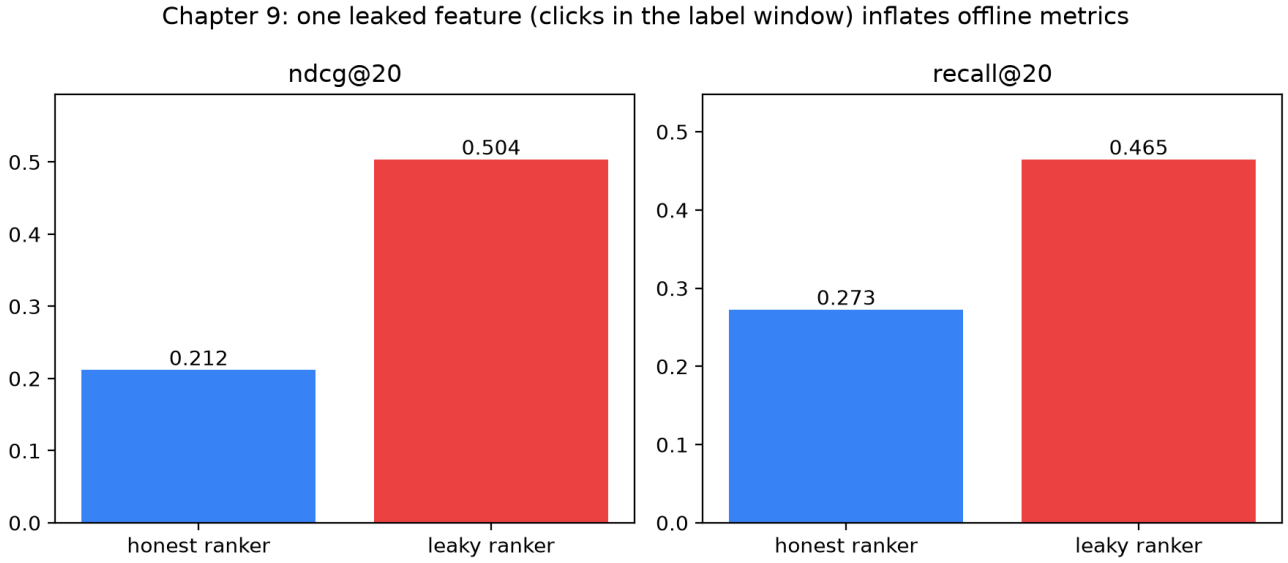
```
def label_window_clicks(frame: pd.DataFrame, start: pd.Timestamp) -> pd.Series:
    """The LEAK: how often the user clicked this item inside [start, start+7d) - the future."""
    ev = common.load_tables()["events"]
    win = ev[(ev["ts"] >= start) & (ev["ts"] < start + pd.Timedelta(days=7)) & (ev["event_type"]
== "click")]
    n = win.groupby(["user_id", "item_id"]).size().rename("n").reset_index()
    m = frame[["user_id", "item_id"]].merge(n, on=["user_id", "item_id"], how="left")
    return pd.Series(m["n"].fillna(0).to_numpy(), index=frame.index)
```

Kelihatannya polos: sebuah `groupby` biasa yang satu-satunya unsur tak lazim adalah filter waktu `ts >= start`, kebalikan dari yang ditegakkan `History.at`. *Ranker* LightGBM yang sama dilatih dua kali, dengan 46 fitur jujur dan dengan 47 (fitur jujur ditambah kebocoran), lalu keduanya dievaluasi pada *frame* yang sama.

KELUARAN TERMINAL

```
=== Leakage demo: add the user's clicks on the item *inside the label window* ===
               recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20  coverage@20
method
honest ranker    0.1503     0.2015     0.2725     0.3824            0.421   0.2116       0.4245
leaky ranker     0.4538     0.4641     0.4645     0.4645            0.616   0.5035       0.5907

leaky_future_clicks share of total gain: 88.5% (rank 1 of 47)
ndcg@20 inflation: 2.38x
The leaky model looks brilliant offline and would be useless live: in production the future is not
available.
```



Gambar 9.3: *Metrik offline dengan dan tanpa fitur yang bocor*

Angkanya dramatis. NDCG@20 melonjak dari 0.2116 ke 0.5035, faktor 2.38, dan recall@20 dari 0.2725 ke 0.4645. Kolom bocor itu mengambil 88.5% dari seluruh *gain* dalam model, menempati peringkat pertama dari 47 fitur. Perhatikan kolom recall untuk *ranker* yang bocor: recall@10, @20, dan @50 semuanya sekitar 0.464, sehingga pada dasarnya setiap contoh positif yang terjangkau sudah ditemukan dalam sepuluh posisi pertama. Sebuah *ranker* hanya bisa menyusun ulang kandidat yang diberikan kepadanya, jadi dataran ini adalah batas atas *recall* kandidat yang kita bahas di Bab 8: contoh positif di luar batas itu tidak pernah diusulkan oleh retriever mana pun. Model yang memegang jawaban langsung menabrak batas atas tersebut.

Peningkatan 2.4× dari satu kolom seharusnya membuat Anda curiga, bukan senang. Tanda bahaya: satu fitur baru mengambil sebagian besar *gain*; metrik offline melonjak jauh melebihi perubahan sebelumnya mana pun; *recall* jenuh pada batas atas kandidat pada K yang kecil; atau kepentingan terkonsentrasi pada fitur yang menggambarkan hasil (`total_orders`, apa pun yang dihitung `all_time` atas seluruh tabel).

Jebakan dan cara produksi menanganinya

1. Men-*snapshot* atribut yang berubah lambat. Di repo ini katalog adalah satu tabel, dan perubahan harga atau rating akan menimpa riwayat. Sistem sungguhan membutuhkan *join titik-waktu* (*point-in-time join*): nilai yang berlaku pada T. *Feature store* (Bab 14) mencatat setiap nilai fitur dengan cap

waktu peristiwa dan menerapkan "*as-of join*" yang memilih nilai terbaru yang tidak melewati T. Tanpanya, rating sebuah item pada saat pelatihan menyertakan ulasan yang ditulis *karena* orang membelinya setelah cutoff. Itulah kebocoran yang akan ditemukan Dewi berikutnya.

2. Agregat yang berdekatan dengan label. "Total pesanan item ini" atas seluruh tabel adalah kebocoran klasik, karena menyertakan pesanan yang sedang Anda coba prediksi. Jebakan yang sama bersembunyi dalam rasio, dalam "tanggal pembelian terakhir", dan dalam `max` atau `min` apa pun atas kolom tak terbatas. Filter `age_days < w` di dalam `_count` adalah pengamannya: setiap agregat berjendela dihitung hanya atas `History`.

3. *Training/serving skew*. Pertahanan terbaik adalah satu jalur kode: `features.build` melayani pelatihan, evaluasi, dan layanan gRPC langsung. Sistem dengan *pipeline* batch dan online terpisah membutuhkan tes paritas yang menjalankan keduanya pada masukan yang sama, karena perbedaan pembulatan, zona waktu, atau aturan nilai hilang diam-diam menurunkan kualitas model.

4. *Feature creep*. Setiap fitur adalah dependensi yang memakan waktu *serving* dan bisa rusak ketika sumbernya berubah. Tambahkan satu kelompok, jalankan ablasi (Bab 10), dan buang kelompok yang tidak membantu.

> **Tahukah kamu?**
>
> **Dependensi data adalah utang teknis.** Dalam "Hidden Technical Debt in Machine Learning Systems", Sculley dan rekan-rekannya menguraikan dependensi data yang *tidak stabil* (*unstable*) dan *kurang dimanfaatkan* (*underutilized*): sinyal masukan yang perilakunya berubah seiring waktu, atau yang memberi manfaat pemodelan tambahan sangat kecil namun membuat sistem rentan terhadap perubahan di hulu. Mereka menyarankan mendeteksi dependensi yang kurang dimanfaatkan dengan evaluasi *leave-one-feature-out* yang menyeluruh dan rutin, persis seperti ablasi kelompok yang kita jalankan di Bab 10.
>
> Sumber: Sculley et al., NeurIPS, 2015

5. Biaya saat *serving*. Kode bab ini menghitung ulang setiap tabel fitur pada setiap permintaan `Recommend`, itulah sebabnya satu permintaan memakan waktu kira-kira satu detik (diukur di Bab 13). Produksi menghitung di muka bagian yang berat (statistik item oleh *streaming job*, fitur pengguna di penyimpanan *key-value*) dan hanya menghitung fitur pasangan yang murah seperti `same_city` pada saat permintaan. *Definisi*-nya tetap identik.

Ringkasan

- Sebuah fitur valid hanya jika dihitung dari peristiwa yang benar-benar sebelum cutoff T; label berasal dari $[T, T + 7d)$. Melanggar ini adalah kebocoran.
- `History.at` adalah satu-satunya gerbang: ia memfilter peristiwa, pencarian, dan katalog berdasarkan cutoff, menambahkan `age_days`, dan menyembunyikan kehabisan stok di masa depan. Semua kode fitur hanya membaca sebuah `History`.
- Jendela (3/7/14/30 hari) memisahkan "ramai sekarang" dari "stabil bagus"; fitur *trend* membandingkan laju per hari dan memakai konstanta kecil agar tetap stabil.
- Pelicinan, $(\text{num} + s\,p_0)/(\text{den} + s)$, menjaga rasio dari nilai ekstrem ketika data tipis; s adalah seberapa banyak bukti yang Anda tuntut.

- Ke-46 fitur terbagi dalam delapan kelompok: candidate, user, item, shop, user_item, user_category, user_shop, dan context. Ke-14 fitur silang tingkat pasangan membawa personalisasi.
- Nilai hilang adalah keputusan desain: hitungan menjadi 0, fitur keterkinian mendapat nilai sentinel (9999 jam, 999 hari) yang bisa dibelah oleh pohon.
- Satu kolom bocor mengambil 88.5% *gain* model dan menaikkan NDCG@20 offline dari 0.2116 ke 0.5035; hasil sebagus itu dari satu fitur adalah tanda peringatan, bukan keberhasilan.
- Otomatiskan pengamannya: tes yang menulis ulang masa depan dan menuntut fitur identik menangkap kebocoran sebelum sampai ke dasbor.

Kuis

Q9.1 Apa yang disyaratkan oleh kebenaran titik-waktu (*point-in-time correctness*) pada sebuah fitur yang dipakai untuk melatih *ranker*?

A. Fitur harus dinormalisasi ke rata-rata nol dan varians satu.
B. Fitur harus dihitung hanya dari informasi yang tersedia sebelum waktu cutoff contoh pelatihan.
C. Fitur harus dihitung dari jumlah peristiwa yang sama untuk setiap pengguna.
D. Fitur harus dihitung ulang setiap jam di produksi.

Q9.2 Pada `History.at`, mengapa filter `ev["ts"] < cutoff` bersifat ketat dan bukan `<=`?

A. Karena cap waktu *floating-point* tidak bisa dibandingkan dengan `<=`.
B. Karena peristiwa tepat pada cutoff termasuk jendela label `[cutoff, cutoff + days)`, sehingga setiap peristiwa dipakai tepat di satu sisi.
C. Karena basis data menyimpan cap waktu dengan resolusi satu detik.
D. Karena tayangan (*impression*) tidak dicatat tepat pada cutoff.

Q9.3 Sebuah item ditampilkan sekali dan diklik sekali sehingga CTR mentahnya 100%. Pernyataan mana tentang CTR yang dihaluskan $(\text{clicks} + s\,p_0)/(\text{impressions} + s)$ yang benar?

A. Tetap 100% karena item itu tidak punya contoh negatif.
B. Bergeser mendekati rata-rata global p_0, dan menjauh darinya hanya seiring item mengumpulkan lebih banyak tayangan.
C. Menjadi tepat nol sampai item itu memiliki setidaknya s tayangan.
D. Tidak lagi bergantung pada klik milik item itu sendiri.

Q9.4 Dalam satu atau dua kalimat: mengapa kode memakai sentinel seperti 9999 untuk `ui_hours_since_last` dan bukan 0 ketika pengguna tidak pernah menyentuh item?

Q9.5 Jelaskan perbedaan antara fitur item, fitur pengguna, dan fitur silang, dan sebutkan satu contoh masing-masing dari bab ini. Jenis mana yang membawa sebagian besar personalisasi?

Q9.6 Bacalah tes di bawah (disertakan lebih awal di bab ini sebagai `test_features_ignore_events_after_cutoff`). Misalkan seseorang menambahkan fitur `item_orders_all_time` yang menghitung pesanan per item dari variabel tingkat modul yang

menyimpan tabel peristiwa lengkap tanpa filter, bukan dari `hist.events`. Apakah tes akan gagal, dan mengapa? Apa persisnya dalam tes yang membuat kegagalan itu terlihat?

```
ev.loc[future, ["event_type", "item_id"]] = ["order", 0]
pd.testing.assert_frame_equal(before, _build(tampered, cutoff))
```

Q9.7 Sebuah item punya 3 klik dan 10 tayangan. CTR global adalah 0.10 dan kekuatan pelicinan $s = 20$. Hitung CTR yang dihaluskan, dan berikan CTR mentah sebagai pembanding.

Q9.8 Sebuah item mendapat 6 klik dalam 3 hari terakhir dan 14 klik dalam 14 hari terakhir. Dengan rumus repo dan konstanta 0.1, hitung `item_trend`. Apakah item itu sedang menghangat atau mendingin?

Q9.9 *Praktik langsung.* Di `book/code/ch09_features.py`, ubah jendela pada `label_window_clicks` dari `pd.Timedelta(days=7)` menjadi `pd.Timedelta(days=1)` (hanya untuk fitur bocor, bukan label), lalu jalankan skrip lagi. Sebelum menjalankan, prediksikan apa yang akan terjadi pada NDCG@20 *ranker* yang bocor dan pada porsi *gain* yang diambil oleh `leaky_future_clicks`. Apa yang Anda amati, dan apa yang ditunjukkannya tentang seberapa banyak masa depan yang perlu bocor dalam sebuah fitur sebelum metrik offline tidak lagi dapat dipercaya?

Bab 10 — Learning to rank

Dina adalah *ranking engineer* di tim sebuah marketplace. Senin pagi ia membuka tiket yang berbunyi: "Home feed aneh. Kita menampilkan ketel di urutan pertama kepada pengguna yang baru kemarin membeli ketel, dan casing ponsel di slot 9 untuk orang yang seminggu penuh menatap casing ponsel." Tidak ada yang salah dengan tahap retrieval. Pembangkit kandidat (*candidate generator*) dari bab-bab sebelumnya sudah bekerja dengan baik: produk yang tepat memang ada di suatu tempat dalam kumpulan sekitar seratus kandidat untuk pengguna itu. Masalahnya ada pada *urutan*. Kumpulan itu tiba dalam urutan sesuai retriever mana pun yang kebetulan menemukan item lebih dulu, padahal halaman hanya punya ruang untuk beberapa slot teratas bagi setiap pengguna.

Solusi sementara timnya adalah rumus tulisan tangan yang masih ada di repo sebagai `pipeline.heuristic_score`: menormalkan tiga skor retriever per pengguna lalu menjumlahkannya. Bab ini mengganti jalan pintas itu dengan algoritma pembelajaran: 46 fitur (*feature*) per pasangan `(user, item)` (Bab 9), label bertingkat (*graded labels*) dari tujuh hari berikutnya, dan sebuah model yang menaruh item terbaik di urutan pertama. Di akhir bab, Anda akan membuka model di balik *popover* "Why this?" pada aplikasi.

Apa yang akan Anda pelajari

- Tiga keluarga tujuan pelatihan *learning-to-rank* (pointwise, pairwise, listwise) dan kapan masing-masing masuk akal.
- Cara kerja *gradient-boosted decision tree* dari prinsip dasar, dengan contoh kecil satu *split* dan satu putaran residual.
- Apa yang ditambahkan LambdaMART di atas *boosting*: gradien *lambda* dan *delta* pertukaran NDCG.
- Bagaimana `ranker.train` milik repo mengonfigurasi LightGBM, termasuk label bertingkat dan tabel `label_gain`.
- Cara membaca *feature importance*, kontribusi SHAP, dan ablasi kelompok fitur, serta cara agar tidak tertipu oleh derau *seed*.
- Hasil yang jujur: pada dataset simulasi kecil ini, model pointwise biasa mendapat skor *sedikit lebih tinggi* daripada LambdaMART, dan mengapa hal itu terjadi.

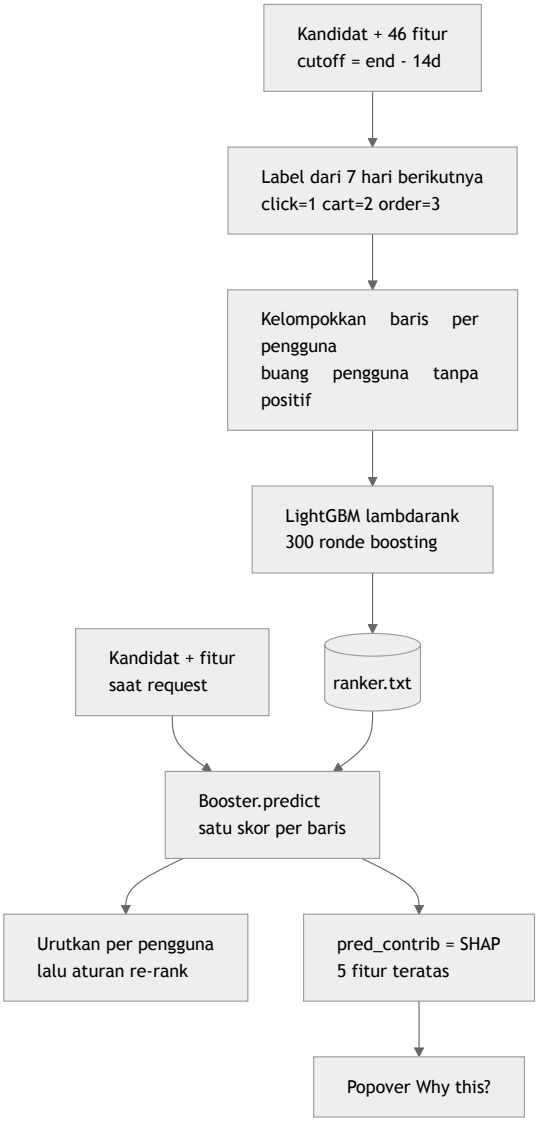
Intuisi: mengurutkan berdasarkan skor yang dipelajari seseorang

Bayangkan seorang perekrut yang menyusun daftar pendek kandidat. Perekrut junior mengikuti daftar periksa yang bersifat aditif (heuristic blend kita); perekrut senior punya naluri non-aditif yang dipelajari dari ribuan perekrutan. Learning to rank mengekstrak penilaian itu dari hasil-hasil historis. Dua pertanyaan dapat dipelajari dari riwayat yang sama: "seberapa besar peluang kandidat ini diterima?" (sebuah probabilitas) dan "di antara dua orang ini, siapa yang didahulukan?" (sebuah urutan). Perekrut dengan lima slot wawancara hanya memerlukan yang kedua. Literatur ranking memiliki tiga keluarga:

Keluarga	Model dilatih untuk…	Loss melihat	Contoh
Pointwise	memprediksi label setiap baris secara terpisah	satu baris	regresi logistik, `objective="binary"`
Pairwise	menaruh item yang lebih relevan di atas item yang kurang relevan	pasangan baris dari pengguna yang sama	RankNet, Ranking SVM
Listwise	langsung memperbaiki metrik tingkat daftar seperti NDCG	seluruh daftar untuk satu pengguna	LambdaMART, `objective="lambdarank"`

Karena NDCG@20 (Bab 2) adalah metrik tingkat daftar yang mendiskon slot-slot bawah, metode listwise seharusnya menjadi pilihan yang alami. Praktiknya ternyata tidak serapi itu.

Alur keseluruhan tahap ini di repo tampak seperti berikut.



Gambar 10.1: *bagaimana ranker dilatih secara offline dan dipakai saat serving, termasuk jalur penjelasan SHAP.*

Algoritma

Kita akan menyusun bagian-bagiannya secara berurutan: pertama *gradient boosting* biasa, lalu *lambda* yang mengubahnya menjadi ranker, lalu label bertingkat yang memberi tahu model apa arti "bagus".

Langkah 1: regression tree dan satu split

Decision tree mengubah satu baris fitur menjadi angka dengan mengajukan serangkaian pertanyaan ya/tidak. Pohon *regresi* menyimpan sebuah angka di setiap daun. Melatihnya berarti memilih, di setiap simpul, pertanyaan (fitur dan ambang) yang membuat baris-baris di tiap sisi sesama mirip sebisa mungkin. "Mirip" diukur dengan jumlah kuadrat galat (*sum of squared errors*, SSE) di sekitar rerata tiap sisi.

Ambil set pelatihan mainan berisi enam baris kandidat untuk satu fitur, `ui_max_dwell_sec` (waktu terlama pengguna bertahan di halaman item ini), beserta label bertingkatnya.

(detik dwell, label) = (0, 0), (0, 0), (5, 0), (30, 1), (60, 2), (90, 3), satu baris masing-masing, bernomor 1 sampai 6.

Boosting dimulai dari prediksi konstan, yaitu rerata label: $F_0 = (0+0+0+1+2+3)/6 = 1.0$. **Residual** setiap baris adalah sisa yang belum terjelaskan, $r_i = y_i - F_0$, yang menghasilkan $-1, -1, -1, 0, 1, 2$. Jumlah kuadratnya adalah $1+1+1+0+1+4 = 8$.

Sekarang kita coba kandidat split pada waktu dwell dan menilai masing-masing dari seberapa besar SSE yang dihilangkan. Untuk split menjadi himpunan kiri L dan kanan R dengan residual r,

$$\mathrm{SSE}(S) = \sum_{i \in S} \left(r_i - \bar{r}_S\right)^2, \qquad \text{gain} = \mathrm{SSE}(\text{all}) - \mathrm{SSE}(L) - \mathrm{SSE}(R).$$

split	baris kiri	baris kanan	SSE(L)	SSE(R)	gain
dwell ≤ 2.5	1, 2	3–6	0	5.0	3.0
dwell ≤ 17.5	1–3	4–6	0	2.0	6.0
dwell ≤ 45	1–4	5, 6	0.75	0.5	**6.75**
dwell ≤ 75	1–5	6	3.2	0	4.8

Split pemenangnya adalah `dwell ≤ 45`. Nilai daun adalah rerata residual di tiap sisi: -0.75 di kiri, $+1.5$ di kanan. Pohon seperti ini adalah "weak learner": satu pertanyaan, dua angka, dan jelas keliru untuk baris 4 dan 5 secara individual. Itu tidak masalah. Boosting tidak memerlukan satu pohon yang hebat; ia memerlukan ratusan pohon biasa-biasa saja yang masing-masing memperbaiki sebagian galat sebelumnya.

Kolom *gain* juga menjadi asal **feature importance** nanti: setiap kali sebuah fitur dipilih untuk split, gain split itu dikreditkan kepadanya. Menjumlahkan kredit itu atas semua split dari semua pohon lalu membaginya dengan total menghasilkan "porsi dari total gain" yang akan Anda lihat pada hasil.

Langkah 2: boosting = pencocokan residual berulang

Aturan pembaruannya sederhana. Dengan laju pembelajaran (*learning rate*) η (LightGBM menyebutnya `learning_rate`), prediksi setiap baris digeser sebagian jarak menuju nilai daun pohonnya:

$$F_m(x) = F_{m-1}(x) + \eta \, h_m(x).$$

Ambil $\eta = 0.5$ untuk contoh satu ronde. Baris 1–4 jatuh ke kiri dan bergeser sebesar $0.5 \times (-0.75) = -0.375$, dari 1.0 menjadi 0.625. Baris 5 dan 6 bergeser sebesar $0.5 \times 1.5 = +0.75$, menjadi 1.75. Residual barunya adalah:

Baris 1-3 kini punya residual -0.625, baris 4 punya +0.375, baris 5 punya +0.25 dan baris 6 punya +1.25. Total galat kuadrat turun dari 8 menjadi $3 \times 0.3906 + 0.1406 + 0.0625 + 1.5625 \approx 2.94$. Pohon berikutnya dilatih pada residual baru ini, dan seterusnya. Setelah 300 ronde dengan η kecil, model telah mengambil 300 langkah koreksi kecil.

Lalu mengapa disebut "gradient" boosting? Untuk galat kuadrat, residual $y - F$ persis sama dengan gradien negatif dari $\frac{1}{2}(y - F)^2$ terhadap prediksi F. Wawasan Friedman adalah bahwa Anda dapat memasukkan loss *apa pun* yang dapat diturunkan, lalu mencocokkan setiap pohon baru dengan gradien negatif loss itu. Ganti galat kuadrat dengan log-loss dan Anda mendapat pengklasifikasi; ganti dengan loss ranking dan Anda mendapat ranker.

> **Tahukah kamu?**
>
> **Boosting adalah gradient descent dalam ruang fungsi.** Makalah Jerome Friedman tahun 2001, "Greedy function approximation: A gradient boosting machine", membingkai boosting sebagai gradient descent bertahap pada loss apa pun yang dapat diturunkan, dengan setiap pohon baru dicocokkan ke gradien negatif loss pada prediksi saat ini. Satu gagasan itulah yang membuat pustaka yang sama dapat melatih regressor, classifier, dan ranker hanya dengan mengganti objektifnya.
>
> Sumber: Friedman, Annals of Statistics, 2001

Langkah 3: dari galat per baris ke kualitas per daftar

Objektif pointwise (misalnya log-loss pada "apakah baris ini diklik atau lebih?") memperlakukan setiap baris sebagai contoh yang independen. Ia dengan senang hati menghabiskan kapasitas model untuk menebak probabilitas item ke-95 dengan tepat. Padahal halaman Dina hanya menampilkan 20 posisi teratas, dan NDCG mendiskon posisi-posisi akhirnya dengan kuat.

Ingat dari Bab 2, dengan relevansi bertingkat rel_i pada posisi i:

$$\mathrm{DCG@}K = \sum_{i=1}^{K} \frac{2^{rel_i} - 1}{\log_2(i+1)}, \qquad \mathrm{NDCG@}K = \frac{\mathrm{DCG@}K}{\mathrm{IDCG@}K},$$

dengan IDCG adalah DCG dari urutan yang sempurna. Pembilang $2^{rel} - 1$ adalah **gain**: relevansi 0, 1, 2, 3 menghasilkan gain 0, 1, 3, 7. Satu order bernilai tujuh klik. Penyebutnya adalah **discount**: posisi 1 berbobot 1, posisi 2 berbobot 0.631, posisi 3 berbobot 0.5, dan seterusnya.

NDCG bergantung pada *urutan terurut*, yang tidak dapat diturunkan, sehingga kita tidak bisa mengambil gradiennya. Trik LambdaRank (Burges, 2010) bersifat pragmatis: *lewati loss, definisikan gradiennya secara langsung*.

Langkah 4: lambda dan pertukaran NDCG

Mulailah dengan loss pairwise, seperti pada RankNet. Untuk dua item i dan j dari pengguna yang sama, dengan i lebih relevan, skor model saat ini adalah s_i dan s_j. Probabilitas yang diberikan model

untuk "i berada di atas j" adalah fungsi logistik dari selisih skor, dan loss pairwise mendorong $s_i - s_j$ membesar. Gradiennya terhadap s_i adalah

$$\lambda_{ij} = \frac{-\sigma}{1 + e^{\sigma(s_i - s_j)}}.$$

Ketika i sudah jauh di atas j, penyebutnya sangat besar dan dorongannya lenyap. Ketika i keliru berada di bawah j, $s_i - s_j$ negatif, penyebutnya mendekati 1, dan dorongannya mendekati maksimum σ. Sejauh ini ini hanyalah logika pairwise biasa: semua pasangan yang salah urutan sama mendesaknya.

Langkah LambdaRank adalah **mengalikannya dengan perubahan NDCG yang akan terjadi bila kedua item ditukar**:

$$\lambda_{ij} = \frac{-\sigma}{1 + e^{\sigma(s_i - s_j)}} \left|\Delta \mathrm{NDCG}_{ij}\right|.$$

Menukar item pada posisi p_i dan p_j mengubah DCG sebesar $(g_i - g_j)\left(d_{p_i} - d_{p_j}\right)$ dalam nilai mutlak, dengan g adalah gain dan $d_p = 1/\log_2(p+1)$ adalah discount. Membaginya dengan IDCG menghasilkan $|\Delta\mathrm{NDCG}|$. Pasangan yang salah urutan *dan* berada di puncak daftar dengan selisih gain besar mendapat lambda besar. Pasangan yang salah urutan di posisi 80 dan 81, atau yang gain-nya sama, nyaris tidak mendapat apa-apa. Begitulah algoritma "memperhatikan" puncak daftar tanpa pernah menurunkan NDCG.

Lambda setiap baris dijumlahkan atas semua pasangannya, bertanda positif untuk anggota yang lebih relevan dan negatif untuk yang kurang relevan. Lambda ini menggantikan gradien negatif ketika mencocokkan pohon berikutnya: itulah **LambdaMART** (**MART**, *multiple additive regression trees*, adalah nama lama untuk gradient-boosted tree).

Contoh hitung delta pertukaran

Ambil tiga kandidat untuk satu pengguna, saat ini berurutan A, B, C dengan label bertingkat 0, 3, 1. Gain-nya adalah $0, 7, 1$.

- $\mathrm{DCG} = 0 \cdot 1 + 7 \cdot 0.631 + 1 \cdot 0.5 = 4.917$.
- Urutan ideal adalah B, C, A, sehingga $\mathrm{IDCG} = 7 \cdot 1 + 1 \cdot 0.631 = 7.631$.
- $\mathrm{NDCG} = 4.917 / 7.631 = 0.644$.

Menukar A dan B (posisi 1, 2) mengubah NDCG sebesar $7 \times (1 - 0.631)/7.631 = 0.339$; menukar A dan C (posisi 1, 3) hanya sebesar $1 \times 0.5/7.631 = 0.066$. Kesalahan pertama, item yang tak pernah diklik berada di atas item tingkat order, lima kali lebih penting. Sekarang anggaplah skor mentah model adalah $s_A = 2.0$ dan $s_B = 1.0$ dengan $\sigma = 1$. Untuk pasangan "B seharusnya di atas A", $s_B - s_A = -1$, sehingga faktor logistiknya $1/(1 + e^{-1}) = 0.731$. Besar lambda untuk pasangan itu adalah $0.731 \times 0.339 = 0.248$: dorongan ke atas yang tegas pada B dan dorongan ke bawah yang sama besar pada A. Untuk pasangan (C, A) dengan faktor logistik yang sama, besarnya hanya $0.731 \times 0.066 = 0.048$. Karena itu pohon berikutnya akan mencurahkan sebagian besar upaya split-nya pada fitur yang membedakan B dari A.

> **Tahukah kamu?**
>
> **LambdaMART memenangi Yahoo! Learning to Rank Challenge.** Sistem yang memenangi Track 1 dari Yahoo! Learning to Rank Challenge (2010) adalah ensemble model gradien-lambda, yang dibangun di atas boosted tree yang dilatih dengan gradien ala LambdaRank. Makalah tinjauan Chris Burges sebelumnya, "From RankNet to LambdaRank to LambdaMART", adalah rujukan standar tentang bagaimana ketiga algoritma itu saling terkait.
>
> Sumber: Burges et al., Proceedings of the Learning to Rank Challenge (PMLR), 2011

Langkah 5: label bertingkat

Di repo ini label bukan sekadar 0 atau 1. `schema.LABEL_GAIN` menyatakan bahwa klik bernilai 1, add-to-cart bernilai 2, dan order bernilai 3. Jika pengguna melakukan beberapa hal pada item yang sama dalam jendela label, `data.labels` menyimpan yang *terkuat*. Ini penting dalam dua hal:

1. Model diberi imbalan karena menaruh item yang dipesan pengguna di atas item yang hanya diklik.
2. Parameter `label_gain=[0, 1, 3, 7]` di LightGBM mengubah label bilangan bulat menjadi gain $2^{rel} - 1$ yang dipakai dalam NDCG. Tabel bawaan LightGBM kebetulan sama, yaitu $2^{rel} - 1$ untuk label-label ini, tetapi menuliskannya secara eksplisit di `PARAMS` menjaga agar objektif dan evaluator offline tidak diam-diam menyimpang; artinya objektif yang dioptimalkan dan metrik yang dilaporkan adalah fungsi yang sama.

Pengguna tanpa positif dalam jendela tidak membawa informasi urutan (semua $|\Delta\mathrm{NDCG}|$ mereka bernilai 0), sehingga `ranker.train` membuangnya.

Penjelasan kode

Ranker itu sendiri

Seluruh konfigurasi LambdaMART berada di kamus `PARAMS` di bagian atas `recommender/recsys/ranker.py` (ditambah konstanta `NUM_ROUNDS`):

Parameter	Nilai	Peran
`objective`	`lambdarank`	gradien lambda (Langkah 4)
`learning_rate`	0.05	ukuran langkah η
`num_leaves`	31	batas ukuran pohon
`min_data_in_leaf`	50	jumlah baris minimum per daun
`bagging_fraction` / `bagging_freq`	0.8 / 1	sampel baris 80% yang baru untuk tiap pohon
`feature_fraction`	0.8	sampel kolom 80% per pohon
`label_gain`	`[0, 1, 3, 7]`	gain untuk label 0..3
`lambdarank_truncation_level`	20	fokus pada 20 teratas
`NUM_ROUNDS`	300	jumlah pohon

Langkah kecil dengan banyak ronde menggeneralisasi lebih baik daripada sedikit langkah besar. LightGBM menumbuhkan pohon secara *leaf-wise*, sehingga `num_leaves` (bukan kedalaman) yang mengendalikan kompleksitas, dan `min_data_in_leaf` mencegah sebuah daun menghafal satu pengguna. Bagging dan pengambilan sampel fitur menyuntikkan keacakan, itulah sebabnya `seed` mengubah hasil (lihat ablasi). Pemotongan pada 20 sesuai dengan `K = 20`: pasangan yang jauh di dalam daftar memiliki $|\Delta\mathrm{NDCG}|$ yang dapat diabaikan.

> **Tahukah kamu?**
>
> **LightGBM cepat karena mengerjakan lebih sedikit per split.** Makalah LightGBM memperkenalkan Gradient-based One-Side Sampling, yang melewati sebagian besar baris bergradien kecil saat memperkirakan gain split, dan Exclusive Feature Bundling, yang menggabungkan fitur jarang yang hampir tidak pernah bernilai bukan-nol secara bersamaan. Pada dataset publik yang diuji penulisnya, ini mempercepat pelatihan gradient-boosted tree konvensional hingga lebih dari 20 kali dengan akurasi yang nyaris sama.
>
> Sumber: Ke et al., NeurIPS, 2017

Pelatihan hanya memerlukan delapan baris. Bacalah fungsi ini pelan-pelan: strukturnya menyandikan semua yang diinginkan LightGBM dari data ranking.

recommender/recsys/ranker.py · train

```python
def train(frame: pd.DataFrame, feature_names: list[str] = schema.FEATURE_NAMES, seed: int = 0) ->
lgb.Booster:
    """`frame` = features + `label`. Groups without any positive teach nothing; drop them."""
    has_pos = frame.groupby("user_id")["label"].transform("max") > 0
    frame = frame[has_pos].sort_values("user_id", kind="stable")
    groups = frame.groupby("user_id", sort=False).size().to_numpy()
    ds = lgb.Dataset(frame[feature_names], label=frame["label"], group=groups)
    return lgb.train({**PARAMS, "seed": seed}, ds, num_boost_round=NUM_ROUNDS)
```

Ada tiga hal yang perlu diperhatikan. Pertama, mask `has_pos` hanya menyimpan pengguna yang punya setidaknya satu positif dalam jendela label; sisanya dibuang. Kedua, `sort_values("user_id", kind="stable")` menempatkan baris-baris tiap pengguna berdampingan, karena argumen `group=` milik LightGBM adalah daftar *ukuran grup yang berurutan*, bukan id pengguna. Jika baris tidak berdampingan, batas grup akan diam-diam memotong pengguna yang salah dan model akan berlatih pada data sampah tanpa memunculkan galat apa pun. Ketiga, seed dapat disuntikkan: `train(frame, names, seed)` inilah yang memungkinkan ablasi melatih ulang model yang sama dengan seed acak berbeda.

Melatih dan mengevaluasi dari ujung ke ujung

`pipeline.train_and_evaluate` menyatukan pemisahan data dari Bab 2, pembangun frame dari Bab 9, dan ranker di atas:

recommender/recsys/pipeline.py · train_and_evaluate

```
def train_and_evaluate(tables: dict[str, pd.DataFrame], end: pd.Timestamp, ablation: bool =
False):
    train_cutoff = end - pd.Timedelta(days=2 * LABEL_DAYS)
    eval_cutoff = end - pd.Timedelta(days=LABEL_DAYS)
    train_frame, _ = build_frame(tables, train_cutoff)
    eval_frame, eval_labels = build_frame(tables, eval_cutoff)
    model = ranker.train(train_frame)

    eval_scored = eval_frame.assign(score=ranker.score(model, eval_frame))
    popular = eval_frame[eval_frame["cand_global_pop_rank"] < 999]
    results = {
        "global popular": evaluate.metrics(_top(popular, -popular["cand_global_pop_rank"]),
eval_labels, K),
        "heuristic blend": evaluate.metrics(_top(eval_frame, heuristic_score(eval_frame)),
eval_labels, K),
        "lightgbm ranker": evaluate.metrics(_top(eval_frame, eval_scored["score"]), eval_labels,
K),
        "lightgbm + rerank": evaluate.metrics(rerank.rerank(eval_scored, K), eval_labels, K),
    }
    report = {
        "train_rows": len(train_frame),
        "eval_rows": len(eval_frame),
        "eval_users": eval_labels["user_id"].nunique(),
        "candidate_recall": evaluate.candidate_recall(eval_frame, eval_labels),
        "results": pd.DataFrame(results).T,
        "importance": ranker.importance(model),
    }
    if ablation:
        report["ablation"] = group_ablation(train_frame, eval_frame, eval_labels,
report["results"].loc["lightgbm ranker"])
    return model, report
```

Telusuri secara berurutan. Kedua cutoff berada 14 dan 7 hari sebelum `end`. Frame pelatihan dibangun per `end - 14d`, dan labelnya berasal dari 7 hari berikutnya. Frame evaluasi dibangun per `end - 7d`, dengan label dari 7 hari terakhir; model tidak pernah melihatnya. Ranker dilatih hanya pada frame pertama, dan dinilai pada frame kedua.

Kamus `results` membandingkan empat metode pada frame evaluasi yang sama, memakai fungsi `evaluate.metrics` yang sama. "global popular" adalah baseline Bab 3. "heuristic blend" adalah rumus tulisan tangan Dina. "lightgbm ranker" adalah yang baru saja kita latih. "lightgbm + rerank" menerapkan aturan bisnis dari Bab 11 di atasnya. Tanpa baris-baris ini dalam tabel yang sama, model ML tidak punya tolok ukur, dan sebuah model bisa saja dirilis padahal tidak lebih baik daripada blend bila tak seorang pun memeriksanya.

Service, di `recommender/recsys/service.py`, memanggil mesin `retrain` yang sama menurut jadwal timer dan lewat tombol admin, sehingga yang Anda baca di sini adalah kode yang benar-benar berjalan di produksi untuk repo ini.

Menjelaskan skor dengan SHAP

Model boosted adalah jumlah dari 300 pohon. Satu prediksi melewati semuanya, sehingga membaca pohon secara langsung tidak ada harapan. Nilai **SHAP** (Lundberg dan Lee, 2017) mengatasi ini dengan gagasan dari teori permainan kooperatif: perlakukan fitur sebagai pemain dalam permainan yang hadiahnya adalah skor model. Nilai Shapley sebuah fitur adalah rerata kontribusi marginalnya atas semua urutan kemungkinan penambahan fitur. Untuk satu baris, kontribusi semua fitur ditambah skor dasar (*base score*) berjumlah *persis* sama dengan keluaran mentah model.

Menghitung nilai Shapley secara naif memerlukan himpunan bagian fitur dalam jumlah eksponensial. Untuk ensemble pohon ada algoritma eksak yang cepat, dan LightGBM mengimplementasikannya di balik satu flag, `pred_contrib=True`. Repo membungkusnya:

recommender/recsys/ranker.py · contributions

```
def contributions(model: lgb.Booster, frame: pd.DataFrame, top: int = 5) -> list[list[tuple[str,
float, float]]]:
    """Per row: the `top` features by |SHAP contribution|, as (feature, value, contribution).

    LightGBM's pred_contrib returns one column per feature plus a final bias column;
    contributions + bias sum to the raw score.
    """
    names = model.feature_name()
    x = frame[names]
    contrib = model.predict(x, pred_contrib=True)[:, :-1]
    values = x.to_numpy(dtype=float)
    out = []
    for row_c, row_v in zip(contrib, values):
        order = np.argsort(-np.abs(row_c))[:top]
        out.append([(names[j], float(row_v[j]), float(row_c[j])) for j in order])
    return out
```

`model.predict(x, pred_contrib=True)` mengembalikan matriks dengan satu kolom per fitur ditambah satu kolom terakhir yang memuat **bias**, yaitu keluaran rerata model ketika ia tidak tahu apa-apa tentang baris tersebut. Irisan `[:, :-1]` membuang kolom terakhir itu, sehingga yang tersisa adalah kontribusi murni per fitur. Untuk setiap baris kita lalu mengurutkan fitur berdasarkan $|\text{contribution}|$ dan menyimpan `top` teratas sebagai triplet `(name, value, contribution)`. Service gRPC memanggil ini di `recommend_rpc` dan menaruh lima teratas di setiap respons; API Rust meneruskannya tanpa diubah, dan `WhyThis.vue` menggambarkannya sebagai batang.

> **Tahukah kamu?**
>
> **SHAP eksak untuk pohon berjalan dalam waktu polinomial.** Makalah TreeExplainer menyajikan algoritma waktu polinomial pertama untuk menghitung penjelasan optimal berbasis teori permainan bagi model berbasis pohon seperti random forest dan gradient-boosted tree. Sebelumnya, nilai Shapley eksak memerlukan pencacahan himpunan bagian fitur yang jumlahnya eksponensial, sehingga tidak praktis untuk model dengan puluhan fitur dan ratusan pohon.
>
> Sumber: Lundberg et al., Nature Machine Intelligence, 2020

Jalankan

Skrip pendamping melatih tiga ranker pada frame kandidat yang persis sama, menjalankan pemeriksaan empat seed, mencetak importance dan rincian SHAP, serta menjalankan ablasi kelompok. Dari direktori `book/`:

```
uv run python code/ch10_ltr.py
```

Skrip membangun frame pelatihan (cutoff 14 hari sebelum akhir) dan frame evaluasi (7 hari sebelum akhir) satu kali, lalu memakainya kembali untuk setiap model. Blok pertama membandingkan ketiga urutan.

KELUARAN TERMINAL

```
=== Three ways to order the same candidates ===
                    recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20
coverage@20
method
heuristic blend       0.1153     0.1769     0.2526     0.3753           0.3978   0.1674
0.5682
pointwise LightGBM    0.1577     0.2176     0.2836     0.3915           0.4250   0.2146
0.4777
LambdaMART            0.1503     0.2015     0.2725     0.3824           0.4210   0.2116
0.4245

candidate recall ceiling: 0.4645
```

Bacalah tabel dari atas ke bawah. Heuristic blend mencapai ndcg@20 sebesar 0.1674. Kedua model terlatih mencapai 0.2146 (pointwise) dan 0.2116 (LambdaMART): sekitar 26–28% lebih tinggi, dari kandidat dan label yang sama. Baris terakhir, 0.4645, adalah *batas atas recall kandidat* (*candidate recall ceiling*), yaitu fraksi positif sebenarnya milik pengguna yang memang ada dalam kumpulan kandidat. Recall@20 dari ranker mana pun dibatasi olehnya. Recall@20 LambdaMART sebesar 0.2725 adalah 59% dari yang dapat dicapai oleh pengurutan ulang oracle atas kumpulan yang sama.

Hasil

Pointwise versus LambdaMART: perbandingan yang jujur

Blok kedua keluaran adalah yang dilewatkan banyak tutorial ranking. Sebuah LightGBM pointwise (`objective="binary"`, label = "apakah ada aksi positif?") dilatih pada baris yang sama dengan hyperparameter yang sama, dan kedua model dilatih ulang dengan empat seed acak berbeda.

KELUARAN TERMINAL

```
=== Seed check: pointwise vs LambdaMART over 4 seeds (ndcg@20) ===
      pointwise  lambdamart
seed
0        0.2146      0.2116
1        0.2134      0.2108
2        0.2167      0.2155
3        0.2142      0.2121
mean: {'pointwise': 0.2147, 'lambdamart': 0.2125}  std: {'pointwise': 0.0014, 'lambdamart':
0.0021}
At this data size pointwise is marginally ahead (about one seed-std): with ~100 candidates per
user and few
positives, the list-aware objective has little extra to exploit. Both beat the heuristic blend by
a wide margin.
```

Model pointwise unggul pada setiap seed. Rerata ndcg@20 adalah 0.2147 berbanding 0.2125, selisih 0.0022, dan simpangan baku antar seed adalah 0.0014 untuk pointwise dan 0.0021 untuk LambdaMART. Selisih itu dengan demikian sekitar satu simpangan baku seed: tandanya konsisten pada keempat seed, tetapi kecil. Pointwise juga unggul pada recall@5 (0.1577 berbanding 0.1503) dan recall@10 (0.2176 berbanding 0.2015). Coverage@20 LambdaMART, yaitu porsi katalog yang muncul di 20 teratas seseorang, adalah yang terendah dari ketiganya (0.4245, dibandingkan 0.4777 untuk pointwise dan 0.5682 untuk heuristic blend).

Ini bertentangan dengan prinsip bahwa objektif listwise cocok untuk metrik tingkat daftar. Alasan yang masuk akal:

1. **Sedikit positif per daftar.** Sekitar seratus kandidat dan segelintir positif menghasilkan sedikit pasangan yang informatif; label pointwise biner adalah sinyal yang lebih sederhana yang langsung dipelajari pohon.
2. **Batas atas yang rendah.** Dengan recall kandidat 0.4645, ruang untuk keuntungan dari ranking kecil, sehingga dua ranker yang baik berakhir berdekatan.
3. **Label bertingkat yang berderau.** Order jarang terjadi dan bergantung pada suasana hati dan stok; `label_gain` memperkuat derau persis itu bagi LambdaMART.

Semua ini tidak menjadikan LambdaMART pilihan yang salah. Selisihnya sekitar satu simpangan baku seed, pada dataset yang jauh lebih kecil daripada hasil-hasil yang dipublikasikan, dan Anda tidak akan meluncurkan produk berdasarkan itu. Yang tak terbantahkan adalah selisih terhadap heuristic blend: kedua model terlatih mengalahkannya sekitar 0.045 ndcg@20. Repo mempertahankan `lambdarank` karena ia semestinya meningkat seiring data dan menangani label bertingkat; jika A/B test Anda berkata lain, `objective="binary"` adalah perubahan satu baris. Pelajarannya ada pada prosesnya: *simpan baseline sederhana di dalam tabel dan jalankan ulang dengan beberapa seed sebelum menyatakan pemenang.*

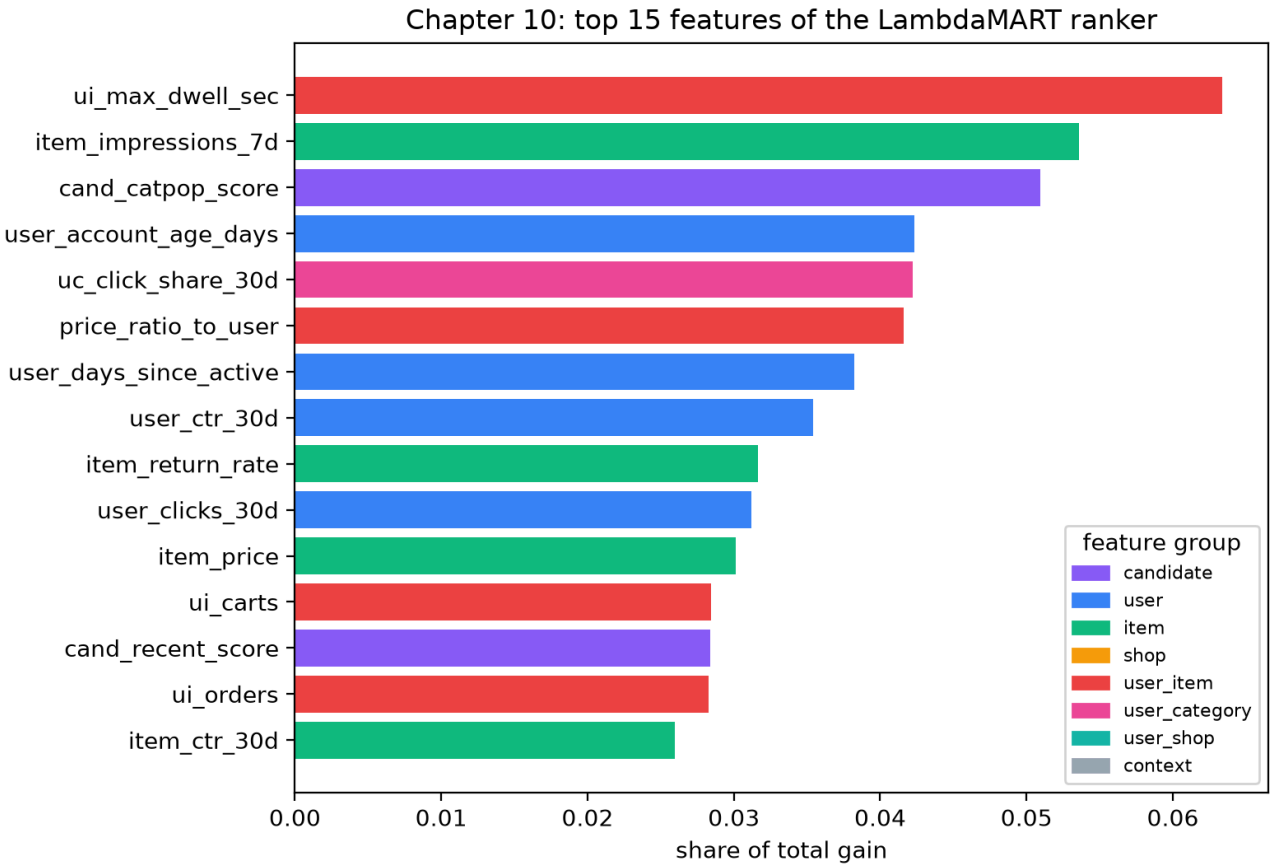
Apa yang dipelajari model? Feature importance

Pertanyaan paling sederhana untuk model boosted adalah fitur mana yang paling banyak ia pakai. LightGBM dapat melaporkan total gain per fitur (jumlah gain split dari Langkah 1). `ranker.importance` menormalkan gain menjadi porsi yang berjumlah satu.

`ranker.importance` menormalkan total gain per fitur menjadi porsi yang berjumlah satu. Grafik menampilkan 15 fitur teratas; keluaran memberikan porsi gain per kelompok.

KELUARAN TERMINAL

```
Gain share by feature group:
group
item             0.309
user_item        0.198
user             0.189
candidate        0.134
user_category    0.086
shop             0.065
user_shop        0.014
context          0.004
```



Gambar 10.2: *15 fitur teratas ranker LambdaMART berdasarkan porsi total gain, diwarnai menurut kelompok fitur.*

Importance-nya *tersebar*: fitur teratas, `ui_max_dwell_sec`, hanya memiliki 6.3% gain. Menurut kelompok, fitur **item** memimpin (0.309), lalu user-item (0.198), user (0.189) dan candidate (0.134); **context** mendapat 0.004. Satu peringatan: gain importance menggambarkan bagaimana pohon dibangun, bukan seberapa besar model akan menderita tanpa suatu fitur; fitur yang berkorelasi berbagi kredit, dan fitur dengan banyak nilai berbeda mendapat lebih banyak kesempatan split. Ablasi memperlihatkan ketidaksesuaian itu.

Why this? Satu prediksi di bawah mikroskop

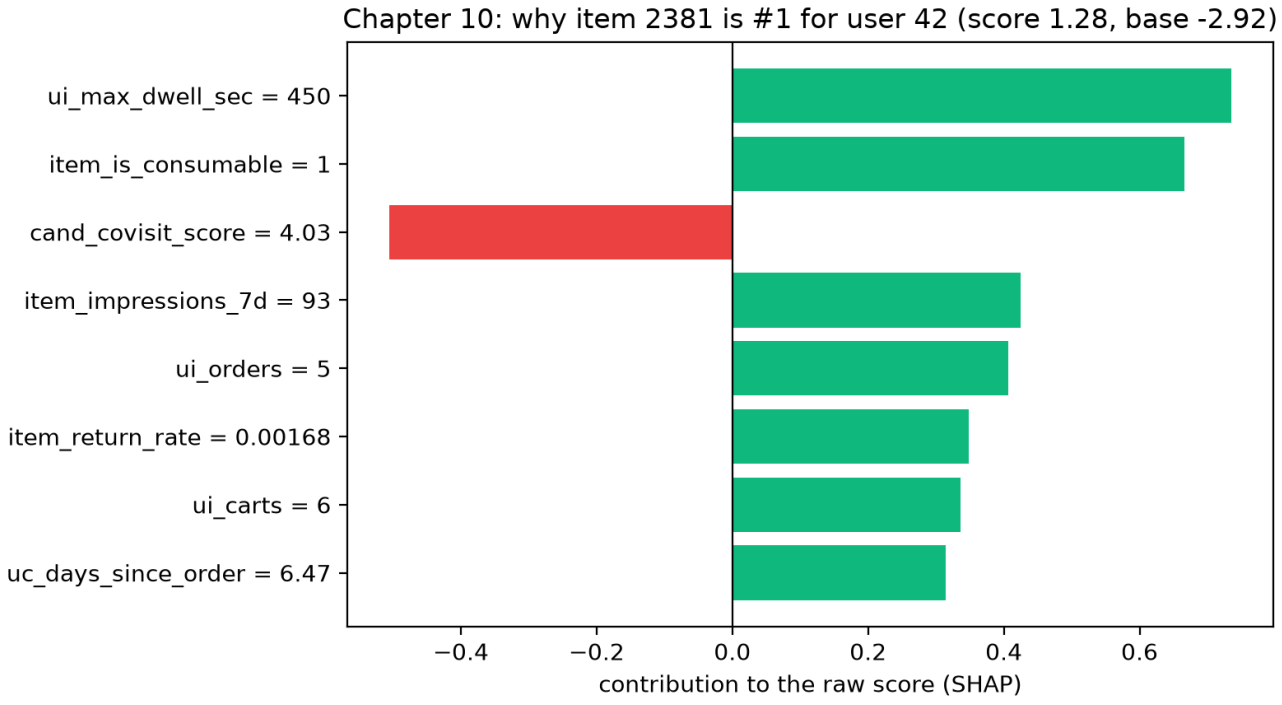
Importance merata-ratakan seluruh prediksi; SHAP menjelaskan satu prediksi. Skrip menilai 106 kandidat untuk pengguna 42 dan mencetak kontribusi item teratas (id 2381):

KELUARAN TERMINAL

```
=== SHAP breakdown: first recommendation for user 42 ===
candidates scored: 106; top item_id=2381 score=1.277
  ui_max_dwell_sec         value=   449.900  contribution=+0.735   Longest product-page dwell on
this item
  item_is_consumable       value=     1.000  contribution=+0.666   Category is repeat-purchase
  cand_covisit_score       value=     4.028  contribution=-0.505   Co-visitation: how often this
item is clicked in the same session as the user's recent items
  item_impressions_7d      value=    93.000  contribution=+0.425   Times shown in last 7 days
  ui_orders                value=     5.000  contribution=+0.407   Times this user ordered this
item
  item_return_rate         value=     0.002  contribution=+0.348   Returns / orders, smoothed
(quality signal)
  ui_carts                 value=     6.000  contribution=+0.336   Times this user carted this
item
  uc_days_since_order      value=     6.465  contribution=+0.314   Days since last order in the
category (re-purchase cycle; 999 = never)
  (bias / base score: -2.917; contributions + bias = raw score)
  sum of all contributions + bias = 1.277  (model score 1.277)

Top 10 recommended item ids for user 42: [2381, 2200, 1872, 2015, 87, 873, 1743, 30, 2482, 414]
```

Skor mentahnya 1.277 dan bias (baseline untuk kandidat yang biasa-biasa saja) adalah -2.917; baris-baris terakhir mengonfirmasi sifat aditif: kontribusi ditambah bias sama dengan skor model. Delapan kontribusi terbesar berjumlah sekitar +2.73, sehingga 38 fitur lainnya menambahkan sisa +1.47.



Gambar 10.3: *Mengapa item 2381 berada di peringkat pertama untuk pengguna 42: delapan kontribusi SHAP terbesar, positif berwarna hijau dan negatif berwarna merah.*

Membaca grafik sebagai cerita: pengguna 42 bertahan pada item ini selama 450 detik (+0.735), item ini adalah barang habis pakai (+0.666), memiliki banyak impresi baru-baru ini (+0.425), dan pengguna memesannya lima kali serta memasukkannya ke keranjang enam kali (+0.407, +0.336). Satu batang negatif ini sangat instruktif: `cand_covisit_score` = 4.03 *menurunkan* skor sebesar 0.505, karena item yang ditemukan hanya lewat co-visitation adalah taruhan yang lebih lemah daripada

bukti langsung dari pengguna ini. Jumlah skor retriever tulisan tangan tidak dapat mengungkapkan hal itu.

Popover "Why this?" pada aplikasi (`WhyThis.vue`, tangkapan layar di Bab 13) menampilkan kontribusi ini untuk model yang hidup; baris sumbernya berasal dari `pipeline.source_of`.

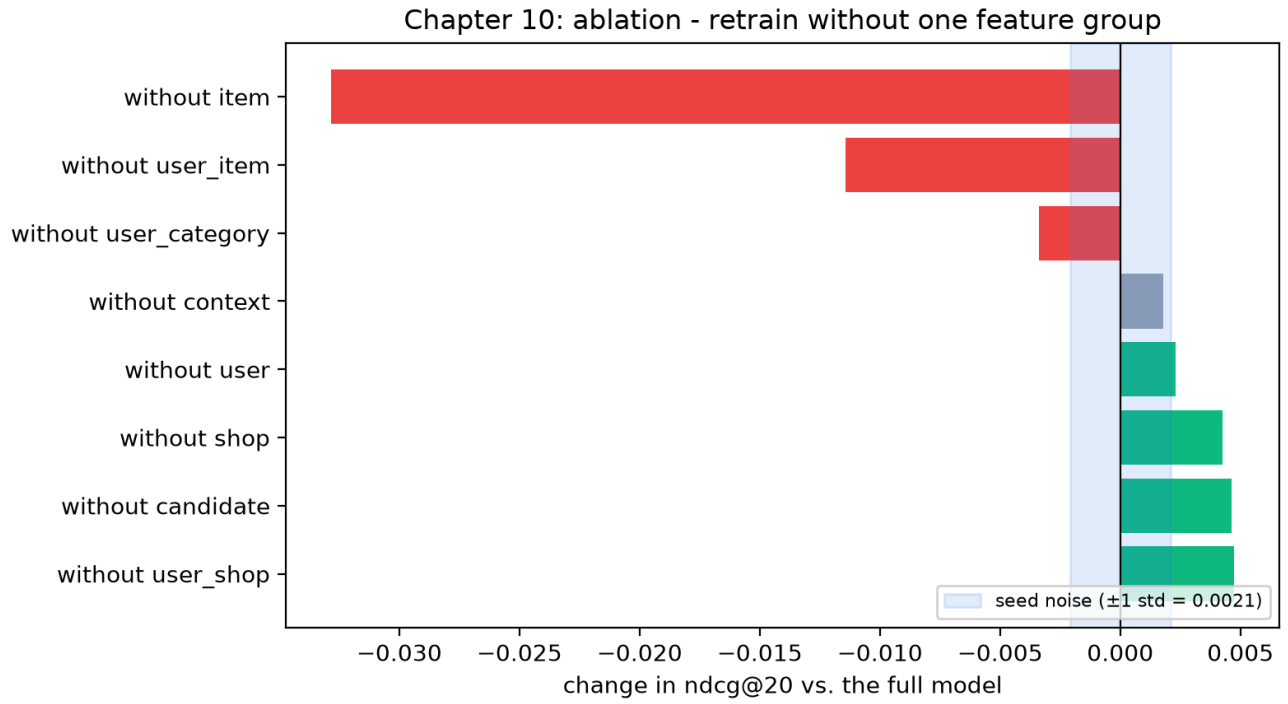
Ablasi: kelompok fitur mana yang penting, dan dibandingkan dengan apa?

Importance bertanya tentang model yang sudah dilatih. **Ablasi** (*ablation*) mengajukan pertanyaan kontrafaktual: andaikan kita tidak pernah memiliki fitur-fitur ini, apakah modelnya akan lebih buruk? Anda menjawabnya dengan melatih ulang. `pipeline.group_ablation` menghapus satu kelompok fitur pada satu waktu, melatih ulang dengan pengaturan yang sama, dan melaporkan perubahan tiap metrik terhadap model penuh.

`pipeline.group_ablation` melatih ulang tanpa tiap kelompok fitur, dan juga melatih ulang model *penuh* dengan seed 1, 2 dan 3. Simpangan baku mereka adalah **noise floor** (batas derau): delta yang lebih kecil darinya tidak memberi tahu apa-apa, karena seed lain dapat membalik tandanya.

KELUARAN TERMINAL

```
                       recall@20  order_recall@20  ndcg@20
without item             -0.0321          -0.0492  -0.0328
without user_item        -0.0069          -0.0235  -0.0114
without user_category    -0.0159          -0.0148  -0.0034
without context           0.0011          -0.0022   0.0018
without user             -0.0037           0.0032   0.0023
without shop             -0.0017           0.0023   0.0042
without candidate         0.0045           0.0144   0.0046
without user_shop         0.0055           0.0043   0.0047
seed noise (±1 std)       0.0021           0.0059   0.0021
```



Gambar 10.4: *Perubahan ndcg@20 ketika tiap kelompok fitur dihapus, dengan pita berarsir setebal satu simpangan baku seed.*

Derau seed pada ndcg@20 adalah 0.0021 (pita berarsir). Bandingkan perubahan ndcg@20 tiap baris dengan pita itu:

- **tanpa item: −0.0328.** Lima belas kali derau. Kelompok item (harga, popularitas, CTR, tingkat retur, stok, dan seterusnya) adalah bukti paling berharga, dan tidak ada kelompok lain yang dapat menggantikannya.
- **tanpa user_item: −0.0114.** Sekitar lima kali derau. Apa yang dilakukan pengguna ini pada item ini penting, dan kelompok lain tidak menggantikannya.
- **tanpa user_category: −0.0034.** Melampaui pita derau pada ndcg, dan jelas melampauinya pada recall@20 (−0.0159 berbanding derau 0.0021), sehingga kita menghitungnya sebagai kontribusi yang nyata, meski lebih kecil.
- **context, user, shop, candidate, user_shop:** delta antara +0.0018 dan +0.0047. Nilai-nilai ini *positif*, artinya model tanpa kelompok-kelompok ini mendapat skor *lebih tinggi*, dan semuanya berada dalam kisaran sekitar dua simpangan baku derau dari nol. Pembacaan yang jujur adalah "tidak ada bukti bahwa mereka membantu pada data ini", bukan "menghapusnya memperbaiki model".

Bandingkan dengan gain importance: kelompok candidate memiliki 13.4% gain dan kelompok user 18.9%, namun menghapus salah satunya tidak merugikan, karena fitur yang berkorelasi (bukti user-item menggemakan skor retriever) mengambil alih. Gain importance tidak pernah mengungkapkan hal itu.

Jebakan dan praktik produksi

- **Kebocoran (*leakage*).** Sebuah fitur bocor menaikkan ndcg@20 offline dari 0.2116 yang jujur menjadi 0.5035 di Bab 9. Bangun fitur per cutoff lewat `History.at` dan pertahankan `tests/test_point_in_time.py`.
- **Batas grup.** `group=` milik LightGBM adalah daftar ukuran berurutan; baris yang tidak terurut berjalan tanpa peringatan dan melatih sampah. Urutkan dulu, seperti yang dilakukan `ranker.train`.
- **Skor tidak punya skala.** Jangan menerapkan ambang pada skor LambdaMART atau mencampurnya dengan probabilitas; gunakan model pointwise atau kalibrasi bila probabilitas diperlukan.
- **Desain label adalah desain produk.** Tabel click 1 / cart 2 / order 3 adalah opini bisnis; tim produksi sering melatih beberapa *head* terpisah lalu menggabungkannya (Bab 14).
- **Retrain sebagai deploy.** Service melatih ulang setiap enam jam dan dari tombol admin; tolak menukar dengan model yang lebih buruk daripada baseline heuristik.
- **Penjelasan bukan sebab-akibat.** Batang SHAP hanya menyatakan bahwa sebuah nilai mendorong skor naik relatif terhadap baseline, tidak lebih.

Ringkasan

- Learning to rank mengubah kumpulan kandidat dan fiturnya menjadi sebuah urutan. Metode pointwise memprediksi label tiap baris, metode pairwise mengurutkan pasangan, metode

listwise menargetkan metrik daftar seperti NDCG.

- Gradient boosting membangun model sebagai jumlah pohon kecil, masing-masing dicocokkan dengan residual (gradien negatif) ensemble saat ini; gain sebuah split adalah SSE yang dihilangkannya.
- LambdaMART mengganti gradien NDCG yang tak dapat dipakai dengan *lambda*: dorongan logistik pairwise yang diskalakan oleh perubahan NDCG $|\Delta\mathrm{NDCG}|$ yang akan terjadi bila pasangan ditukar, sehingga kesalahan di dekat puncak dengan selisih gain besar mendominasi.
- `ranker.PARAMS` milik repo memakai `lambdarank`, 300 ronde dengan learning rate 0.05, `label_gain=[0, 1, 3, 7]` untuk label bertingkat click/cart/order, dan pemotongan pada 20. Baris harus dikelompokkan berdampingan per pengguna, dan pengguna tanpa positif dibuang.
- Kedua ranker terlatih mengalahkan heuristic blend dengan selisih lebar (ndcg@20 0.2146 dan 0.2116 dibandingkan 0.1674), tetapi di sini model pointwise sedikit mengungguli LambdaMART pada keempat seed, dengan selisih sekitar satu simpangan baku seed. Objektif yang sadar daftar baru terbayar dengan lebih banyak data per query.
- Gain importance tersebar (fitur teratas hanya 6.3%); ia memberi tahu bagaimana pohon dibangun, bukan apa yang dibutuhkan model.
- SHAP memberikan penjelasan per prediksi yang eksak dan aditif (kontribusi ditambah bias sama dengan skor) dan menggerakkan popover "Why this?" pada aplikasi lewat `ranker.contributions`.
- Ablasi kelompok fitur menjawab pertanyaan kontrafaktual, tetapi hanya delta yang lebih besar dari derau seed yang dihitung: di sini hanya kelompok item, user–item, dan user–category yang melampauinya.

Kuis

Q10.1 Pernyataan mana yang paling tepat membedakan learning to rank pointwise, pairwise, dan listwise?

A. Pointwise memakai pohon, pairwise memakai jaringan saraf, listwise memakai model linear.
B. Loss pointwise menilai setiap baris sendiri-sendiri, loss pairwise membandingkan dua item dari query yang sama, loss listwise melihat seluruh daftar terurut.
C. Metode pointwise memerlukan label bertingkat; metode pairwise dan listwise memerlukan label biner.
D. Metode pointwise dilatih secara online, yang lain secara offline.

Q10.2 Pada gradient boosting dengan galat kuadrat, apa yang berusaha diprediksi oleh setiap pohon baru?

A. Label asli setiap baris.
B. Rerata label seluruh set pelatihan.
C. Residual ensemble saat ini, yaitu gradien negatif dari loss.
D. Fitur dengan gain tertinggi.

Q10.3 Pada LambdaMART, mengapa setiap dorongan pairwise dikalikan dengan $|\Delta\mathrm{NDCG}|$?

A. Agar loss dapat diturunkan.
B. Agar skor dapat dibandingkan antar pengguna.
C. Untuk mempercepat pelatihan dengan melewati pasangan.
D. Agar kesalahan urutan yang akan mengubah NDCG banyak (di dekat puncak, dengan selisih gain besar) dihitung lebih berat daripada yang hanya mengubahnya sedikit.

Q10.4 `ranker.train` mengurutkan frame berdasarkan `user_id` dengan sort stabil sebelum membangun dataset LightGBM. Mengapa baris-baris satu pengguna harus berdampingan, dan apa yang salah jika tidak?

Q10.5 Pada hasil kita, LightGBM pointwise mendapat skor ndcg@20 sedikit lebih tinggi daripada LambdaMART. Berikan dua alasan yang masuk akal, dan sebutkan bukti apa yang akan membuat Anda mengganti objektif produksi.

Q10.6 Baca kode ini dari `recommender/recsys/ranker.py`:

recommender/recsys/ranker.py · contributions

```
def contributions(model: lgb.Booster, frame: pd.DataFrame, top: int = 5) -> list[list[tuple[str,
float, float]]]:
    """Per row: the `top` features by |SHAP contribution|, as (feature, value, contribution).

    LightGBM's pred_contrib returns one column per feature plus a final bias column;
    contributions + bias sum to the raw score.
    """
    names = model.feature_name()
    x = frame[names]
    contrib = model.predict(x, pred_contrib=True)[:, :-1]
    values = x.to_numpy(dtype=float)
    out = []
    for row_c, row_v in zip(contrib, values):
        order = np.argsort(-np.abs(row_c))[:top]
        out.append([(names[j], float(row_v[j]), float(row_c[j])) for j in order])
    return out
```

Mengapa fungsi ini mengiris matriks prediksi dengan `[:, :-1]`? Apa yang terjadi pada "penjelasan" jika irisan itu terlupa, dengan asumsi daftar kemudian diurutkan menurut $|\text{contribution}|$ untuk setiap baris?

Q10.7 Kandidat seorang pengguna diberi peringkat dengan label bertingkat $1, 0, 3$ pada posisi 1, 2, 3 (gain $2^{rel} - 1 = 1, 0, 7$). Dengan $d_p = 1/\log_2(p+1)$, hitung $|\Delta\text{NDCG}|$ untuk menukar item pada posisi 1 dan 3. Berikan tiga desimal.

Q10.8 Untuk item yang dijelaskan dalam bab ini, skor model adalah 1.277 dan bias adalah -2.917. Kedelapan kontribusi yang ditampilkan berjumlah sekitar $+2.726$. Berapa kontribusi gabungan dari fitur-fitur sisanya?

Q10.9 Sebuah ablasi melaporkan "tanpa user_category: -0.0034 ndcg@20" dan derau seed adalah ± 0.0021 (satu simpangan baku). Kesimpulan mana yang paling didukung?

A. Menghapus kelompok itu berdampak pasti sebesar tepat -0.0034.
B. Kelompok itu tidak berguna karena efeknya di bawah 0.01.
C. Efeknya lebih besar daripada satu simpangan baku derau, sehingga ada bukti ringan bahwa

kelompok itu membantu; memeriksa recall@20 atau lebih banyak seed akan memperkuat kesimpulan. D. Hasilnya tidak bermakna karena delta-nya negatif.

Q10.10 *Praktik langsung.* Di `book/code/ch10_ltr.py`, tepat sebelum baris `model = ranker.train(train)`, tambahkan `ranker.PARAMS["label_gain"] = [0, 1, 1, 1]` agar sebuah order bernilai tidak lebih dari sebuah klik, lalu jalankan ulang skrip. Amati `ndcg@20` dan `order_recall@20` untuk LambdaMART pada tabel pertama dan bandingkan dengan nilai di bab ini, dengan memakai simpangan baku seed (0.0021) sebagai tolok ukur apakah perubahan apa pun nyata. Kembalikan perubahan itu setelahnya.

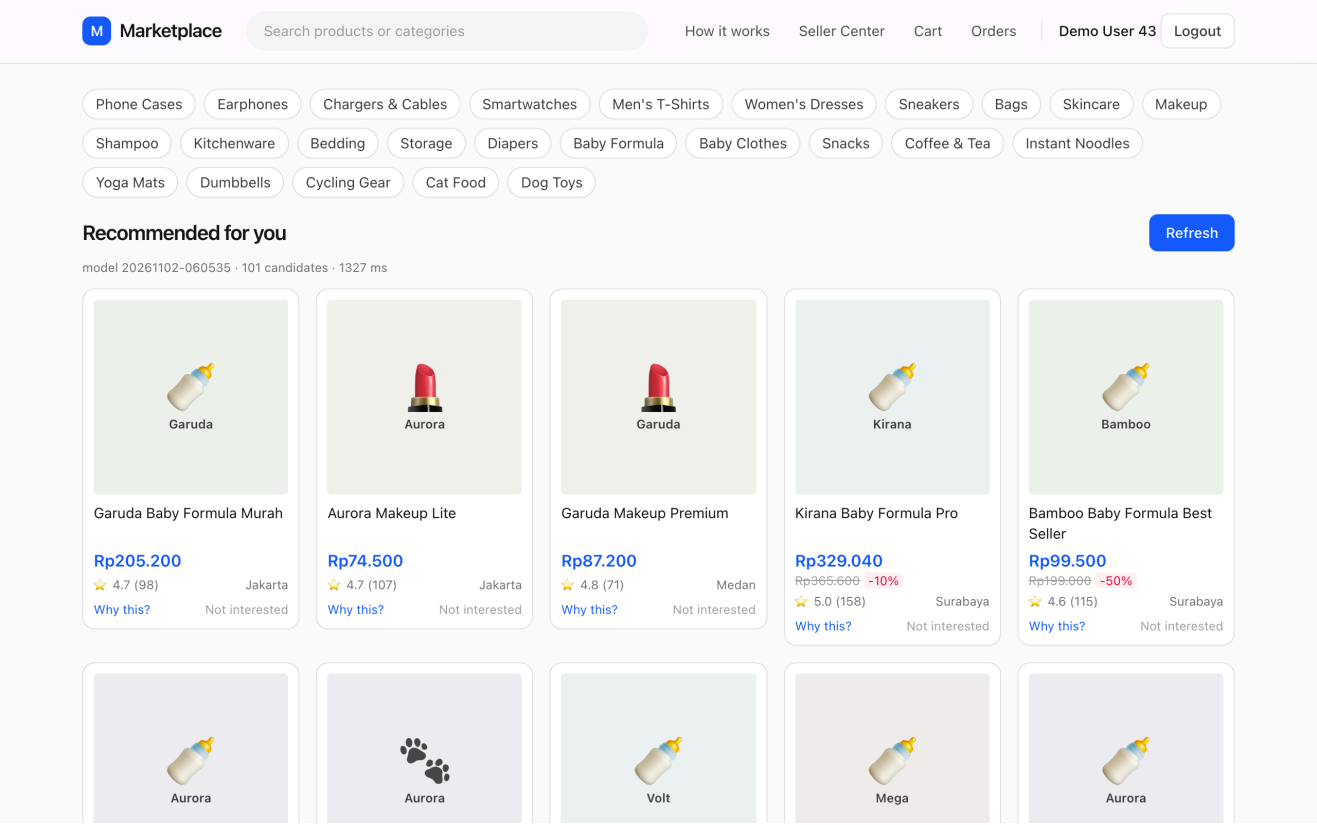
Bab 11 — Re-ranking, diversitas, dan aturan bisnis

Halaman yang isinya susu formula semua

Bayangkan Anda membuka beranda aplikasi marketplace besar pada Selasa malam. Minggu lalu Anda membeli sebungkus popok dan mengeklik tiga kaleng susu formula untuk membandingkan harga. Sistem rekomendasi Anda tergolong bagus: pemeringkat (*ranker*) berbasis gradient boosting dari bab sebelumnya, dilatih dengan puluhan ribu contoh, dengan NDCG offline yang sehat. Ia membaca riwayat Anda dan menyimpulkan, dengan benar, bahwa Anda orang tua baru. Maka layar pertama berisi susu formula, susu formula, susu formula, botol bayi, susu formula lagi, dan satu lipstik yang nyasar di posisi lima.

Tidak ada yang *salah* dengan tiap kartu; masing-masing adalah taruhan yang masuk akal. Tetapi *halamannya* buruk. Kalau Anda ingin menjelajah, tidak ada yang bisa dijelajahi, dan kalau kemarin Anda membeli kulkas, halaman itu mungkin menawarkan tiga kulkas lagi. Ranker Anda menilai item satu per satu, dan ia tidak tahu bahwa dua puluh taruhan bagus yang mirip tidak bernilai dua puluh kali lipat satu taruhan.

Inilah tugas tahap terakhir pipeline: **re-ranking** (pemeringkatan ulang). Tahap ini menerima daftar yang sudah diberi skor lalu memperbaiki properti yang hanya ada pada level *seluruh daftar*: keragaman, keadilan antarpenjual, item yang tidak boleh muncul lagi, item yang wajib ditampilkan bisnis, dan sedikit keacakan yang disengaja agar sistem terus belajar. Kita akan melihat re-ranker di repo (beberapa baris pandas), lalu Maximal Marginal Relevance (MMR), mengukur berapa harga diversitas dalam akurasi offline, dan menutup dengan epsilon-greedy serta Thompson sampling.



Gambar 11.1: *Beranda "Recommended for you" di aplikasi kita: baris pertama didominasi susu formula, dengan make-up terselip di antaranya. Inilah halaman yang dirancang untuk diperbaiki oleh re-ranker.*

Yang akan Anda pelajari

- Mengapa model yang menilai item secara independen menghasilkan daftar yang buruk sebagai daftar, dan apa itu tujuan "level daftar" (*list-level objectives*).
- Bagaimana Maximal Marginal Relevance menyeimbangkan relevansi terhadap redundansi, lengkap dengan rumus dan contoh hitungan tangan.
- Bagaimana `rerank.rerank` di repo mengimplementasikan filter sudah-dibeli dan batas (*cap*) per kategori serta per toko dalam lima baris pandas.
- Berapa banyak akurasi offline yang dikorbankan tiap aturan ini pada data kita, memakai angka nyata dari skrip pendamping.
- Bagaimana keadilan bagi penjual, item bersponsor, dan dorongan (*boost*) manual masuk ke tahap re-ranking tanpa merusak ranker.
- Bagaimana epsilon-greedy dan Thompson sampling memilih item yang dieksplorasi, dengan contoh hitungan distribusi Beta.

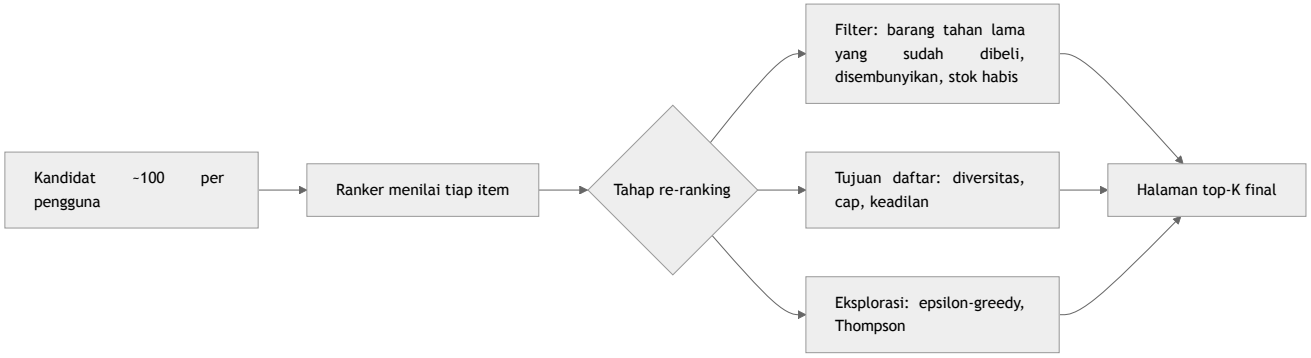
Intuisi: menata etalase toko, bukan memilih produk terbaik

Pikirkan cara pemilik toko menata etalase. Ia tidak mengurutkan stok berdasarkan margin keuntungan lalu memajang dua belas item teratas berjajar. Kalau dua belas produk terlaris semuanya sepatu lari, etalase itu berkata "kami jual sepatu lari", dan pejalan kaki yang butuh jaket akan lewat begitu saja. Ia memilih *himpunan* item yang bersama-sama menyampaikan sesuatu: sepatu, jaket, tas, hadiah. Setiap item di etalase sedikit lebih buruk daripada item terbaik yang bisa ia pilih untuk slot itu, tetapi *etalasenya* lebih baik, karena item-itemnya saling melengkapi, bukan saling menggandakan.

Beranda rekomendasi adalah sebuah etalase. Ranker di Bab 10 adalah pemilik toko yang mengurutkan berdasarkan margin. Ia menjawab "seberapa besar kemungkinan pengguna ini berinteraksi dengan item ini?" untuk tiap item secara independen, dan independensi itulah masalahnya: setelah Anda menampilkan satu kaleng susu formula, probabilitas bahwa kaleng kedua berguna tidak sama dengan probabilitas awalnya. Ekonom menyebutnya *diminishing marginal utility* (utilitas marjinal yang menurun); dalam information retrieval disebut *redundansi*.

Ada tiga keluarga logika re-ranking dan kita akan bertemu semuanya:

1. **Filter** membuang item yang tidak boleh muncul (sudah dibeli, disembunyikan, stok habis).
2. **Tujuan level daftar** seperti diversitas: nilai sebuah item bergantung pada apa yang sudah ada di halaman.
3. **Eksplorasi**: kadang sebuah item ditaruh di halaman karena sistem *belum yakin* tentangnya, bukan karena ia taruhan terbaik.



Gambar 11.2: *Re-ranking adalah tahap terakhir. Ia menerima kandidat yang sudah diberi skor dan menghasilkan halaman; ia tidak pernah melatih ulang model.*

Algoritmanya

Filter: aturan sudah-dibeli

Aturan paling sederhana adalah *filter sudah-dibeli* (*already-bought filter*). Kita membuang item jika pengguna pernah memesannya dan kategorinya bukan barang habis pakai (*consumable*). Orang yang bulan lalu membeli kulkas tidak perlu melihat kulkas yang sama lagi; orang yang bulan lalu membeli kopi sangat mungkin perlu. Di simulator (dan di skema pada Bab 1) setiap kategori memiliki flag `is_consumable`, yang di tabel fitur muncul sebagai `item_is_consumable`, dan fitur riwayat `ui_orders` menghitung berapa kali pengguna ini memesan item ini. Aturannya:

$$\text{drop}(u, i) \iff \text{ui_orders}(u, i) > 0 \ \wedge \ \text{item_is_consumable}(i) = 0.$$

Item yang disembunyikan (pengguna menekan "Not interested") dan item yang stoknya habis dibuang lebih awal, di `candidates.generate`, yang lebih murah karena kita tidak menghitung fitur untuk baris yang tidak akan pernah ditampilkan.

Cap: diversitas ala kadarnya

Aturan kedua membatasi berapa banyak item dari satu kategori dan satu toko yang boleh muncul. Urutkan item pengguna berdasarkan skor. Beri nomor $0, 1, 2, \ldots$ pada item tiap kategori menurut

urutan skor, dan lakukan hal yang sama untuk tiap toko. Pertahankan sebuah item hanya jika posisinya dalam kategori di bawah C_{cat} dan posisinya dalam toko di bawah C_{shop}. Di repo $C_{\text{cat}} = 8$ dan $C_{\text{shop}} = 4$. Lalu ambil K item pertama yang lolos.

Cap itu kasar tetapi ampuh. Ia memberi jaminan keras (tidak ada halaman yang memuat lebih dari empat item dari satu toko) dengan harga sama sekali tidak peka terhadap seberapa *mirip* item-itemnya. Dua casing ponsel dari dua toko berbeda, keduanya di kategori "Phone Cases", bagi manusia nyaris kembar, namun cap mengizinkan delapan di antaranya.

Maximal Marginal Relevance

MMR, diusulkan oleh Carbonell dan Goldstein untuk mengurutkan ulang hasil pencarian dan ringkasan, adalah jawaban klasik untuk "bagaimana menghukum kemiripan terhadap apa yang sudah ada di halaman?". Algoritmanya serakah (*greedy*). Mulai dari daftar kosong S. Pada tiap langkah pilih item i dari kandidat tersisa R yang memaksimalkan

$$\text{MMR}(i) = \lambda \cdot \text{rel}(i) - (1 - \lambda) \cdot \max_{j \in S} \text{sim}(i, j),$$

dengan $\text{rel}(i)$ adalah relevansi (di sini, skor ranker yang diskalakan ke $[0, 1]$ per pengguna) dan $\text{sim}(i, j) \in [0, 1]$ mengukur seberapa mirip dua item. Parameter $\lambda \in [0, 1]$ adalah kenop pengatur. Pada $\lambda = 1$ suku kedua lenyap dan MMR mereproduksi peringkat asli. Pada $\lambda = 0$ relevansi diabaikan dan Anda mendapat daftar yang tersebar maksimal. Untuk pilihan pertama himpunan S kosong, sehingga penalti nol menurut konvensi, dan item peringkat teratas selalu muncul pertama.

Algoritma dalam langkah bernomor:

1. Skalakan skor tiap pengguna ke $[0, 1]$: $\text{rel}(i) = (s_i - s_{\min}) / (s_{\max} - s_{\min})$.
2. Hitung lebih dulu matriks kemiripan berpasangan untuk kandidat pengguna ini. Di skrip pendamping kita, $\text{sim} = 1$ untuk kategori yang sama, 0.5 untuk kategori induk yang sama, dan 0 selain itu.
3. Untuk setiap kandidat tersisa, simpan kemiripan terbesar terhadap item mana pun yang sudah terpilih.
4. Ulangi K kali: hitung skor MMR setiap item tersisa, pilih argmax-nya, tandai sebagai terambil, dan perbarui nilai maksimum berjalan dari kemiripan.

Biayanya $O(K \cdot n)$ per pengguna setelah matriks kemiripan ada, dan membangun matriks itu $O(n^2)$. Dengan $n \approx 100$ kandidat, itu beberapa ribu operasi; sepele.

Contoh hitungan dengan empat item

Ambil $\lambda = 0.7$ dan empat kandidat dengan relevansi ternormalisasi dan kategori:

item	rel	kategori
A	1.0	Phones
B	0.9	Phones
C	0.6	Shoes
D	0.5	Beauty

Item pada kategori yang sama memiliki kemiripan 1; semua pasangan lain kemiripannya 0 (kita abaikan level induk di sini demi kesederhanaan).

- **Pilihan 1.** Belum ada yang terpilih. Skornya $0.7\times$ rel: A = 0.70, B = 0.63, C = 0.42, D = 0.35. Kita pilih **A**.
- **Pilihan 2.** B adalah Phone, jadi kemiripannya dengan A adalah 1: $0.7 \times 0.9 - 0.3 \times 1 = 0.33$. C: $0.7 \times 0.6 - 0.3 \times 0 = 0.42$. D: 0.35. Kita pilih **C**, meskipun B jauh lebih relevan.
- **Pilihan 3.** B masih memiliki kemiripan 1 dengan A (maksimum atas himpunan terpilih $\{A, C\}$): 0.33. D memiliki kemiripan 0 dengan keduanya: 0.35. Kita pilih **D**.
- **Pilihan 4.** Tinggal B.

Urutan akhirnya A, C, D, B. B, kembaran Phones itu, turun dari tempat kedua ke terakhir. Dengan $\lambda = 0.9$ suku penaltinya 0.1, sehingga B akan mendapat $0.81 - 0.10 = 0.71$ melawan 0.54 milik C, dan B tetap di tempat kedua. Inilah seluruh cerita parameter itu: λ menentukan seberapa besar selisih relevansi yang masih bisa "dimaafkan" bagi sebuah duplikat.

Eksplorasi: memberi kesempatan pada item yang belum pasti

Semua yang di atas mengasumsikan skor ranker benar, padahal ranker hanya tahu apa yang pernah dilihatnya. Item yang tidak pernah ditampilkan tidak punya klik, sehingga model memberinya skor rendah, sehingga ia tidak pernah ditampilkan: sebuah *feedback loop* (lingkaran umpan balik) yang kita pelajari di Bab 12. Obat buku teksnya adalah kerangka **multi-armed bandit**: tiap item adalah sebuah "lengan" dengan probabilitas klik yang tidak diketahui, dan setiap impresi sekaligus mendatangkan reward dan mengajari kita tentang lengan itu.

Epsilon-greedy adalah kebijakan paling sederhana: dengan probabilitas $1 - \varepsilon$ tampilkan item terbaik menurut model, selain itu tampilkan item acak dari sebuah kolam. Dengan $\varepsilon = 0.1$ pada halaman 20 slot, rata-rata dua slot bersifat eksploratif. Implementasinya sepele dan boros: slot acak sama mungkin memilih item dengan 50 impresi dan klik rate buruk seperti item dengan 2 impresi dan potensi yang belum diketahui.

Thompson sampling lebih cerdas soal *di mana* bereksplorasi. Untuk setiap item, pertahankan distribusi probabilitas atas klik rate θ-nya. Dengan prior Beta dan umpan balik klik/tidak-klik, ini bersifat konjugat: setelah c klik dan $n - c$ non-klik, dimulai dari prior seragam $\mathrm{Beta}(1, 1)$,

$$\theta \mid \text{data} \sim \mathrm{Beta}(1 + c,\ 1 + n - c), \qquad \mathbb{E}[\theta] = \frac{1 + c}{2 + n}, \qquad \mathrm{Var}[\theta] = \frac{ab}{(a+b)^2(a+b+1)}.$$

Untuk memilih apa yang ditampilkan, *tarik satu sampel acak dari distribusi Beta tiap item dan urutkan berdasarkan hasil tarikan*. Item dengan banyak bukti memiliki distribusi sempit; item dengan sedikit bukti memiliki distribusi lebar dan kadang menarik angka tinggi, yang memberinya sebuah impresi. Seiring bukti terkumpul, eksplorasi padam dengan sendirinya, tanpa ε yang harus disetel.

Contoh Thompson yang dihitung

Tiga item, prior seragam:

item	impresi	klik	posterior	rerata	simpangan baku
A (mapan)	10	3	Beta(4, 8)	0.333	0.131
B (tampil dua kali, tanpa klik)	2	0	Beta(1, 3)	0.250	0.194
C (baru sama sekali)	0	0	Beta(1, 1)	0.500	0.289

Rerata-nya adalah $4/12$, $1/4$, dan $1/2$. Variansnya mengikuti rumus: untuk A, $ab/((a+b)^2(a+b+1)) = 32/(144 \times 13) \approx 0.0171$, sehingga simpangan bakunya sekitar 0.131; untuk B adalah $3/(16 \times 5) = 0.0375$ (simpangan baku 0.194); untuk C adalah $1/(4 \times 3) = 0.0833$ (simpangan baku 0.289).

Misalkan satu ronde sampling menghasilkan tarikan (ilustratif) A = 0.31, B = 0.18, C = 0.77. Urutannya C, A, B: pendatang baru yang belum dikenal tampil pertama. Jika C kemudian mendapat 1 klik dari 1 impresi, ia menjadi Beta(2, 1) dengan rerata 0.667; jika ia mendapat nol klik dalam 5 impresi, ia menjadi Beta(1, 6), rerata 0.143, dan jarang menang tarikan lagi. Sistem bereksplorasi persis sebanyak ketidakpastiannya.

Tahukah kamu?

Maximal Marginal Relevance adalah gagasan tahun 1998 yang melampaui tugas aslinya. MMR diperkenalkan oleh Carbonell dan Goldstein di SIGIR 1998 untuk mengurutkan ulang dokumen hasil pencarian dan untuk membuat ringkasan, dengan menggabungkan suku relevansi terhadap kueri dan suku kebaruan yang menghukum kemiripan terhadap dokumen yang sudah dipilih. Rumus greedy yang sama sampai kini masih menjadi titik awal baku untuk mendiversifikasi slate rekomendasi.

Sumber: Carbonell et al., SIGIR, 1998

Tahukah kamu?

Diversitas bisa menurunkan akurasi rata-rata namun tetap meningkatkan kepuasan. Ziegler dan rekan-rekan memperkenalkan "topic diversification" dan metrik intra-list similarity untuk daftar rekomendasi. Mereka melaporkan bahwa, meskipun merugikan akurasi rata-rata, metode itu meningkatkan kepuasan pengguna, yang mereka uji dalam studi daring dengan lebih dari 2,100 subjek, terutama untuk daftar yang dihasilkan oleh collaborative filtering berbasis item.

Sumber: Ziegler et al., WWW, 2005

Tahukah kamu?

Thompson sampling lebih tua daripada komputer, dan mengejutkan kuatnya. William R. Thompson menjelaskan gagasan memilih di antara perlakuan secara proporsional terhadap probabilitas posterior bahwa perlakuan itu yang terbaik dalam makalah Biometrika tahun 1933. Pada 2011, Chapelle dan Li melaporkan bahwa pada iklan display dan rekomendasi artikel berita, metode ini sangat kompetitif dibanding metode bandit lain, dan berargumen bahwa ia seharusnya menjadi baseline standar.

Sumber: Chapelle et al., NeurIPS, 2011

Penelusuran kode

Re-ranker di repo

Re-ranker produksi di repo ini adalah seluruh file `recommender/recsys/rerank.py`. Berikut satu-satunya fungsinya.

`recommender/recsys/rerank.py · rerank`

```python
def rerank(scored: pd.DataFrame, k: int) -> pd.DataFrame:
    """`scored` needs user_id, item_id, category_id, shop_id, ui_orders, item_is_consumable,
score."""
    bought_durable = (scored["ui_orders"] > 0) & (scored["item_is_consumable"] == 0)
    scored = scored[~bought_durable].sort_values(["user_id", "score"], ascending=[True, False])
    # Keep an item only if it is within the user's top-N for its category AND top-M for its shop.
    cat_rank = scored.groupby(["user_id", "category_id"]).cumcount()
    shop_rank = scored.groupby(["user_id", "shop_id"]).cumcount()
    kept = scored[(cat_rank < MAX_PER_CATEGORY) & (shop_rank < MAX_PER_SHOP)]
    return kept.groupby("user_id").head(k)
```

Bacalah baris demi baris. `scored` memiliki satu baris per kandidat `(user_id, item_id)` dengan `score` dari ranker ditambah kolom-kolom yang dibutuhkan aturan. Pernyataan pertama membangun mask boolean `bought_durable`; yang kedua membuang baris-baris itu dan mengurutkan berdasarkan pengguna lalu skor menurun, yang diandalkan oleh baris-baris berikutnya.

Lalu datang triknya. `groupby(["user_id", "category_id"]).cumcount()` menomori baris di dalam tiap grup menurut urutan saat ini, mulai dari nol. Karena frame sudah terurut berdasarkan skor, nomor itu persis berarti "berapa banyak item berskor lebih tinggi dari kategori ini yang sudah dimiliki pengguna ini". Hal yang sama dilakukan per toko. Sebuah baris lolos bila kedua penghitung berada di bawah batasnya (`MAX_PER_CATEGORY = 8`, `MAX_PER_SHOP = 4`, didefinisikan di awal modul). Terakhir `groupby("user_id").head(k)` memotong daftar tiap pengguna menjadi k item. Fungsi ini sepenuhnya tervektorisasi, dan itu penting: saat evaluasi training ia berjalan untuk ribuan pengguna sekaligus, dan di RPC `Recommend` langsung ia berjalan pada frame satu pengguna.

Skrip pendamping MMR

Skrip pendamping `book/code/ch11_rerank.py` mengimplementasikan MMR di atas skor LambdaMART dari Bab 10 (ia melatih ulang ranker yang sama melalui helper bersama yang kecil, sehingga angkanya sebanding dengan bab sebelumnya). Pertama, fungsi kemiripan.

`book/code/ch11_rerank.py · similarity`

```python
def similarity(cat: np.ndarray, parent: np.ndarray) -> np.ndarray:
    """1 if same category, 0.5 if same parent category, else 0."""
    same_cat = cat[:, None] == cat[None, :]
    same_par = parent[:, None] == parent[None, :]
    return np.where(same_cat, 1.0, np.where(same_par, 0.5, 0.0))
```

Fungsi ini mem-broadcast array kategori dan kategori induk menjadi matriks $n \times n$ dengan `cat[:, None] == cat[None, :]`; `np.where` bersarang memberi 1 untuk kategori yang sama, 0.5 untuk induk yang sama, selain itu 0. Nilai parsial untuk induk itu penting karena "Dog Toys" dan "Cat Food" adalah kategori berbeda tetapi sama-sama Pets, dan halaman penuh item Pets tetap repetitif.

Sekarang loop greedy-nya sendiri.

book/code/ch11_rerank.py · mmr

```python
def mmr(scored: pd.DataFrame, parent_of: dict[int, str], lam: float, n: int = common.TOP_N) ->
pd.DataFrame:
    """Greedy MMR per user: pick argmax  lam * rel - (1 - lam) * max_sim_to_selected."""
    rows = []
    for uid, g in scored.groupby("user_id", sort=True):
        s = g["score"].to_numpy()
        rel = (s - s.min()) / max(s.max() - s.min(), 1e-9)
        cat = g["category_id"].to_numpy()
        par = np.array([parent_of[c] for c in cat])
        sim = similarity(cat, par)
        chosen: list[int] = []
        max_sim = np.zeros(len(g))
        avail = np.ones(len(g), dtype=bool)
        for _ in range(min(n, len(g))):
            mmr_score = np.where(avail, lam * rel - (1 - lam) * max_sim, -np.inf)
            j = int(np.argmax(mmr_score))
            chosen.append(j)
            avail[j] = False
            max_sim = np.maximum(max_sim, sim[j])
        sel = g.iloc[chosen]
        rows.append(sel[["user_id", "item_id", "category_id"]])
    return pd.concat(rows, ignore_index=True)
```

Telusuri berurutan. Skor dinormalisasi ke $[0, 1]$ per pengguna (`1e-9` menjaga dari skor yang semuanya sama). `max_sim` menyimpan, untuk tiap kandidat, kemiripan terbesar terhadap apa pun yang sudah dipilih; nol berarti "himpunan kosong, tanpa penalti". Di dalam loop, `mmr_score` persis $\lambda \cdot \text{rel} - (1 - \lambda) \cdot \max_\text{sim}$, dengan item yang sudah terambil diset ke $-\infty$. Setelah memilih `j`, satu baris matriks kemiripan memperbarui maksimum berjalan dengan `np.maximum(max_sim, sim[j])`, itulah yang menjaga algoritma tetap murah.

Loop mengembalikan hanya `user_id`, `item_id`, dan `category_id` menurut urutan terpilih, format yang diharapkan `common.evaluate`. Sebuah helper, `diversity`, menghitung kategori berbeda di 20 teratas tiap pengguna dan merata-ratakannya atas pengguna; inilah *intra-list diversity* yang digambarkan di bawah.

Fungsi `main` mengulang $\lambda \in \{1.0, 0.9, 0.7, 0.5, 0.3\}$, lalu mengevaluasi dua varian lagi: `rerank.rerank` milik repo, dan filter sudah-dibeli saja, sehingga kita bisa melihat aturan mana yang menelan biaya berapa.

Jalankan

Dari direktori `book/`:

```
uv run python code/ch11_rerank.py
```

Skrip mencetak tabel untuk semua metode. Berikut hasil yang ditangkap: blok pertama menampilkan lima pengaturan MMR dan aturan repo.

KELUARAN TERMINAL

```
                          recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20
coverage@20  distinct_categories@20
method
MMR lambda=1.0                0.1503     0.2015     0.2725     0.3824           0.4210   0.2116
0.4245                  5.3263
MMR lambda=0.9                0.1459     0.1921     0.2648     0.3788           0.4149   0.2081
0.4063                  6.7782
MMR lambda=0.7                0.1163     0.1674     0.2398     0.3688           0.3927   0.1884
0.3726                 11.0317
MMR lambda=0.5                0.0930     0.1169     0.1907     0.3582           0.3164   0.1603
0.3354                 16.2777
MMR lambda=0.3                0.0927     0.1030     0.1430     0.3580           0.2351   0.1401
0.3129                 18.6864
rerank.rerank caps            0.1496     0.1971     0.2505     0.2956           0.3955   0.2044
0.3825                  6.5787
```

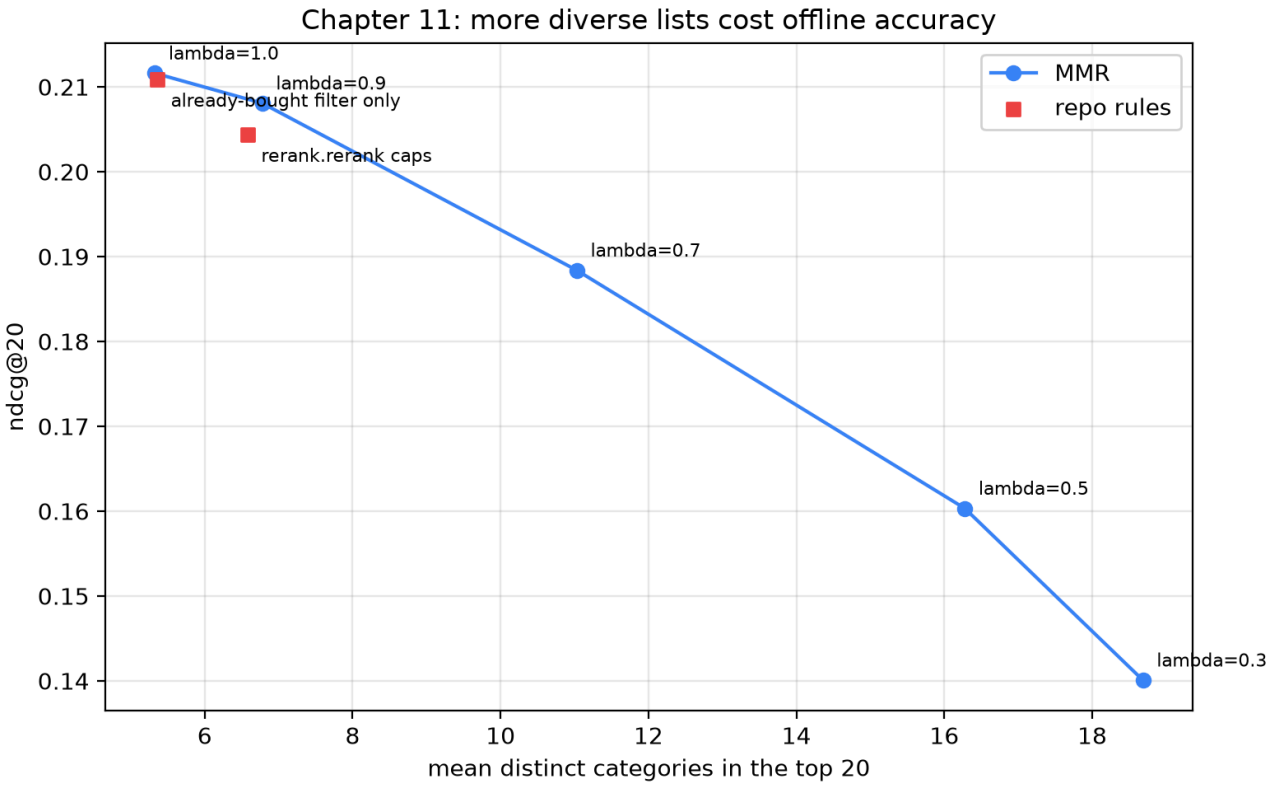
Dan ringkasan dari skrip itu sendiri tentang efek tiap λ, relatif terhadap ranker yang tidak dimodifikasi:

KELUARAN TERMINAL

```
LambdaMART alone: ndcg@20 0.2116, 5.33 distinct categories in the top 20
  MMR lambda=0.9     ndcg change −0.0035, distinct categories +1.45
  MMR lambda=0.7     ndcg change −0.0232, distinct categories +5.71
  MMR lambda=0.5     ndcg change −0.0513, distinct categories +10.95
  MMR lambda=0.3     ndcg change −0.0715, distinct categories +13.36
```

Hasil: berapa harga diversitas?

Mulai dari baris pertama. `MMR lambda=1.0` adalah peringkat LambdaMART tanpa diubah (dengan $\lambda = 1$ penalti dikalikan nol): ndcg@20 sebesar 0.2116 dan recall@20 sebesar 0.2725. Top 20 berisi rata-rata 5.33 kategori berbeda. Itulah baseline kita: kira-kira lima jenis produk berbeda pada halaman 20 kartu. Tidak buruk, tetapi belum etalase.



Gambar 11.3: *Trade-off antara ndcg@20 offline dan rata-rata jumlah kategori berbeda di 20 teratas, untuk lima pengaturan MMR dan untuk varian berbasis aturan di repo.*

Grafik itu memplot tabel yang sama. Bergerak menyusuri garis MMR dari kiri atas berarti bergerak dari "relevansi murni" ke "variasi murni". Membaca angkanya:

- $\lambda = 0.9$: ndcg turun dari 0.2116 ke 0.2081, perubahan -0.0035, kerugian relatif di bawah 2%. Kategori berbeda naik dari 5.33 ke 6.78, kenaikan 1.45. Ini jauh nilai tukar terbaik di kurva: satu setengah kategori tambahan dengan harga sepertiga poin NDCG.
- $\lambda = 0.7$: ndcg turun 0.0232 menjadi 0.1884, sementara halaman kini mencakup 11.03 kategori: 5.71 lebih banyak daripada baseline. Sedikit lebih dari dua kategori tambahan untuk setiap 0.01 ndcg yang hilang, dan kurva mulai menukik.
- $\lambda = 0.5$ **dan** 0.3: ndcg adalah 0.1603 dan 0.1401, dan jumlah kategori berbeda mencapai 16.28 dan 18.69. Halaman 20 kartu dengan 18.69 kategori hampir seluruhnya kategori unik. Kita telah merelakan kira-kira seperempat dan sepertiga ndcg ranker untuk mendapatkannya.

Ujung dalamnya instruktif. `recall@5` turun dari 0.1503 pada $\lambda = 1.0$ menjadi 0.0927 pada $\lambda = 0.3$, karena MMR menurunkan item kuat yang mirip dengan yang sudah ditempatkan, sehingga *puncak* daftar paling menderita. Namun `recall@50` nyaris tidak bergerak, dari 0.3824 ke 0.3580: item yang sama masih ada di daftar, hanya lebih rendah. Re-ranking mengubah urutan dan seleksi dari kolam yang tetap; ia tidak pernah menciptakan kandidat bagus yang baru.

Coverage menceritakan kisah terkait: coverage@20 turun dari 0.4245 ke 0.3129 seiring λ menurun. MMR mendiversifikasi *kategori di dalam satu daftar pengguna*, dan untuk itu ia menyelam ke item terbaik tiap kategori, yang merupakan item populer yang sama bagi semua orang. Halaman jadi lebih bervariasi, namun sistem menampilkan irisan katalog yang lebih sempit. Diversitas per daftar dan

coverage tingkat sistem adalah tujuan berbeda yang membutuhkan alat berbeda, misalnya batas eksposur per item.

Aturan bawaan repo

Sekarang dua kotak merah di grafik. `already-bought filter only` mendapat ndcg@20 sebesar 0.2109 dibanding 0.2116, kerugian 0.0007, jauh lebih kecil daripada langkah MMR mana pun, dengan rata-rata 5.36 kategori: nyaris gratis secara offline. `rerank.rerank caps` (filter ditambah 8 per kategori dan 4 per toko) menghasilkan 0.2044 dan 6.58 kategori berbeda. MMR pada $\lambda = 0.9$ sedikit lebih baik (0.2081 dengan 6.78 kategori), karena ia menurunkan item berdasarkan kemiripannya dengan sisa halaman, sedangkan cap hanya bereaksi setelah hitungan melewati ambang.

Ada perbedaan yang lebih tajam pada `recall@50`: 0.2956 untuk cap dibanding 0.3789 untuk filter saja. `rerank` membuang item tanpa menggantinya, sehingga ketika kolam kandidat pengguna terkonsentrasi di beberapa kategori, daftar yang tersisa bisa lebih pendek dari 50 (dugaan terkuat kami; tidak kami instrumentasi). Cap yang ketat karena itu membutuhkan variasi yang cukup dari candidate generator di hulu.

Satu peringatan: label offline berasal dari pengguna yang melihat apa pun yang ditampilkan oleh sistem *lama*. Label itu tidak bisa memberi imbalan pada variasi yang tidak pernah dilihat pengguna, dan kebosanan jangka panjang akibat halaman monoton tidak tampak dalam jendela tujuh hari. Ziegler dan rekan-rekan membutuhkan studi daring untuk menunjukkan manfaatnya, dan Anda pun begitu. Perlakukan kurva offline sebagai *daftar harga* dan A/B test (Bab 14) sebagai tempat memutuskan berapa yang mau dibayar.

Melampaui diversitas: aturan bisnis di re-ranker

Keadilan bagi penjual

Marketplace itu dua sisi: pembeli menginginkan relevansi, penjual ingin terlihat. Ranker yang dilatih pada engagement memberi toko mapan lebih banyak impresi, maka lebih banyak ulasan: lingkaran yang kaya makin kaya yang bisa mematikan penjual baru yang bagus. Cap per toko adalah aturan keadilan yang kasar. Pendekatan yang lebih berprinsip bekerja dengan *eksposur*: karena perhatian menurun tajam menurut peringkat (Bab 12), slot 15 bukanlah slot 1. Singh dan Joachims memformalkan batasan atas eksposur yang diterima item atau kelompok relatif terhadap merit mereka, dan memberikan algoritma yang memaksimalkan utilitas pengguna dengan batasan itu. Dalam praktik, marketplace mengimplementasikan versi sederhana, seperti slot minimum bagi penjual baru atau anggaran eksposur harian per toko.

Iklan, boost, dan slot bersponsor

Hampir setiap marketplace menjual penempatan iklan. Aturan desainnya: **jangan mengotori ranker dengan uang iklan.** Iklan disisipkan *setelah* ranker, di posisi tetap (katakanlah slot 3, 8, dan 13), dipilih oleh lelang terpisah yang menimbang tawaran dan prediksi klik rate, dan diberi label "Sponsored". Boost kampanye ("semua item sale 9.9 mendapat $+0.05$") juga sebaiknya menjadi suku yang terlihat di re-ranker dengan tanggal mulai dan selesai, bukan perubahan pada data training;

kalau tidak, pelatihan ulang berikutnya memperlakukan impresi hasil boost sebagai bukti organik dan menanamkan kampanye itu ke dalam model.

Eksplorasi di re-ranker

Dalam praktik, jalankan pipeline seperti biasa, lalu tukar item pada sebagian ε dari *slot* (bukan pengguna) dengan kandidat yang kurang terekspos, sebaiknya di separuh bawah halaman; untuk Thompson sampling, simpan hitungan klik dan impresi per item di feature store dan campurkan satu tarikan Beta per kandidat ke dalam skor. Bagaimanapun caranya, **catat slot mana yang eksploratif dan dengan probabilitas berapa**; kalau tidak, datanya tetap bias seperti sebelumnya dan tidak bisa dikoreksi (inverse propensity weighting, Bab 12).

Jebakan dan praktik produksi

Normalisasikan relevansi sebelum MMR. Rumus mencampur skor dengan kemiripan di $[0, 1]$, tetapi skor LightGBM tidak terbatas dan berbeda tiap pengguna. Tanpa penskalaan ulang (kita memakai min-max per pengguna), λ bermakna lain untuk setiap pengguna.

Jangan menyetel λ hanya secara offline. Kurva di atas adalah daftar harga, bukan resep. Mulailah dengan ringan (0.9 kita memberi sebagian besar keuntungan awal), A/B test pada metrik level sesi (pesanan per pengguna, tingkat retur), dan geser kenopnya dari sana.

Urutan operasi penting. Filter dulu, diversitas berikutnya, penyisipan iklan terakhir; sisihkan slot eksplorasi *sebelum* cap, kalau tidak sebuah cap bisa diam-diam membuang item yang dieksplorasi.

Cap bisa memperpendek halaman, dan re-ranking tidak dapat memperbaiki kolam yang buruk. Putuskan apakah akan backfill, melonggarkan cap, atau menerima halaman pendek (repo menerimanya). Kalau semua isi kolam adalah susu formula, MMR hanya memilih susu formula yang paling tidak mirip, jadi tegakkan diversitas di hulu juga, misalnya dengan retriever terpisah per klaster minat.

Ringkasan

- Ranker menilai item secara independen, sehingga daftar dengan skor terbaik bisa repetitif; re-ranking mengoptimalkan seluruh daftar.
- Filter (barang tahan lama yang sudah dibeli, disembunyikan, stok habis) nyaris tidak berbiaya secara offline: 0.2109 versus 0.2116 ndcg@20.
- Cap memberi jaminan keras tetapi buta terhadap kemiripan; di sini cap menelan akurasi lebih banyak daripada MMR ringan untuk variasi yang serupa.
- MMR memilih secara greedy dengan $\lambda \cdot \text{rel} - (1 - \lambda) \cdot \max \text{sim}$; $\lambda = 0.9$ menambah sekitar 1.45 kategori dengan harga -0.0035 ndcg, sementara nilai yang lebih rendah kehilangan akurasi dengan cepat. Setel λ lewat A/B test.
- Simpan aturan keadilan, slot iklan, dan boost di re-ranker, di luar model, agar bisa diaudit dan dimatikan.
- Epsilon-greedy mengeksplorasi dengan laju tetap; Thompson sampling mengeksplorasi sebanding dengan ketidakpastian. Catat slot mana yang eksploratif.

Kuis

Q11.1 Pada MMR dengan $\lambda = 1$, apa yang dikembalikan algoritma?

A. Daftar dengan diversitas maksimal, mengabaikan relevansi.
B. Urutan relevansi asli, karena penalti kemiripan dikalikan nol.
C. Kebalikan dari urutan asli.
D. Permutasi acak.

Q11.2 Mengapa `rerank.rerank` mengecualikan kategori consumable dari filter sudah-dibeli?

A. Consumable lebih murah, sehingga bisnis tidak peduli.
B. Consumable tidak dapat disembunyikan oleh pengguna.
C. Membeli consumable lagi (kopi, popok) adalah pembelian ulang yang normal dan diinginkan, tidak seperti barang tahan lama seperti kulkas.
D. Consumable tidak memiliki `shop_id`.

Q11.3 Apa keunggulan utama Thompson sampling dibanding epsilon-greedy?

A. Ia tidak memerlukan pencatatan impresi.
B. Ia tidak pernah menampilkan item teratas model.
C. Ia mengeksplorasi sebanding dengan ketidakpastian, sehingga item buruk yang sudah terukur dengan baik berhenti ditampilkan dan item baru mendapat kesempatan tanpa ε tetap.
D. Ia dijamin memaksimalkan NDCG offline.

Q11.4 Jelaskan dalam dua atau tiga kalimat mengapa aturan bisnis seperti slot bersponsor dan boost kampanye sebaiknya diterapkan di re-ranker dan bukan sebagai fitur tambahan atau perubahan label pada data training ranker.

Q11.5 Pada hasil, `recall@50` untuk `rerank.rerank caps` jauh lebih rendah daripada untuk filter sudah-dibeli saja, sementara `recall@5` nyaris sama. Berikan penjelasan yang paling mungkin.

Q11.6 Baca fungsi `rerank` yang disertakan di bab ini. Seorang pengguna memiliki 30 kandidat berskor, semuanya dari kategori yang sama, berasal dari 10 toko berbeda (masing-masing 3 item), dan tidak satu pun adalah barang tahan lama yang sudah dibeli. Dengan `MAX_PER_CATEGORY = 8`, `MAX_PER_SHOP = 4` dan $k = 20$, berapa item yang dikembalikan fungsi untuk pengguna ini, dan aturan mana yang bertanggung jawab?

Q11.7 Gunakan MMR dengan $\lambda = 0.5$ pada tiga item dengan relevansi ternormalisasi 0.9 (kategori X), 0.8 (kategori X), dan 0.5 (kategori Y). Kemiripan adalah 1 untuk kategori yang sama dan 0 selain itu. Berikan urutan akhir, dengan menunjukkan skor MMR pada tiap langkah.

Q11.8 Sebuah item dimulai dengan prior seragam Beta(1, 1) lalu menerima 20 impresi dan 5 klik. Apa distribusi posteriornya, rerata klik rate-nya, dan simpangan bakunya (tiga desimal)? Bandingkan simpangan baku itu dengan item baru (tanpa impresi) dan jelaskan artinya bagi Thompson sampling.

Q11.9 Pernyataan mana yang paling sesuai dengan hasil terukur di bab ini?

A. MMR dengan $\lambda = 0.9$ menelan sekitar 0.0035 ndcg@20 dan menambah sekitar 1.45 kategori berbeda; $\lambda = 0.3$ menelan sekitar 0.0715 ndcg@20.
B. MMR pada setiap λ menaikkan coverage@20 di atas baseline.
C. Filter sudah-dibeli saja menelan sekitar 0.05 ndcg@20.
D. Cap menurunkan jumlah kategori berbeda di bawah baseline.

Q11.10 Hands-on: salin `book/code/ch11_rerank.py` ke file sementara dan ubah `similarity` sehingga berbagi kategori induk memberi 0.0 alih-alih 0.5 (hanya kategori yang persis sama yang dianggap mirip). Jalankan dan bandingkan ndcg@20 dan `distinct_categories@20` pada $\lambda = 0.5$ dengan tabel di bab ini. Nyatakan apa yang Anda harapkan teramati (petunjuk: bagaimana penalti berubah untuk item di kategori bersaudara?) dan apakah hasilnya sesuai.

Bab 12 — Cold start, konteks, dan bias

Pukul 00:01 pada malam sale 12.12. Hanya dalam satu menit, puluhan ribu orang yang belum pernah membuka marketplace ini mengetuk ikon aplikasi, karena seorang teman mengirim tautan flash deal. Bagi sistem rekomendasi, para pengunjung ini adalah hantu: tanpa klik, tanpa pencarian, tanpa riwayat. Pada saat yang sama, tiga ratus penjual menayangkan produk baru yang belum pernah ditampilkan kepada siapa pun, sehingga tidak ada yang pernah mengkliknya. Di latar belakang, seorang data scientist menatap dashboard dengan bingung: item di posisi teratas setiap beranda memiliki *click-through rate* lebih dari sepuluh kali lipat dibandingkan item di slot terakhir, dan tim tidak bisa menentukan seberapa besar selisih itu berasal dari "item teratas memang lebih bagus" dan seberapa besar dari "item teratas memang berada di atas".

Tiga masalah bersembunyi dalam cerita itu, dan bab ini membahasnya satu per satu:

- **Cold start**: model ditanya tentang pengguna atau item yang tidak memiliki data perilaku sama sekali.
- **Konteks (*context*)**: jawaban yang tepat bergantung pada *kapan* dan *di mana* permintaan terjadi, bukan hanya pada *siapa* yang meminta.
- **Bias**: klik yang kita gunakan untuk melatih model dihasilkan oleh sistem yang justru ingin kita perbaiki, sehingga klik itu menggambarkan keinginan pengguna secara terdistorsi.

Yang akan Anda pelajari:

- Tiga jenis cold start (pengguna baru, item baru, sistem baru) dan tangga *fallback* yang praktis untuk masing-masing.
- Bagaimana repositori ini menurunkan kualitas secara anggun (*graceful degradation*): dari ranker yang dipersonalisasi, ke item *trending* untuk pengguna anonim, hingga query SQL biasa ketika layanan Python mati.
- Apa itu bias posisi (*position bias*), bagaimana melihatnya di log impresi, dan bagaimana *inverse propensity weighting* mengoreksinya.
- Bagaimana *feedback loop* mengubah keunggulan popularitas yang kecil menjadi besar, dan cara termurah untuk melawannya.
- Mengapa konteks saat permintaan (jam, perangkat, *surface*) *bukan* fitur dari ranker mingguan di buku ini, dan seperti apa model pendamping *real-time*-nya.
- Cara mengukur semuanya: kualitas ranker per kelompok keaktifan pengguna, dan seberapa sering item baru mencapai 20 teratas.

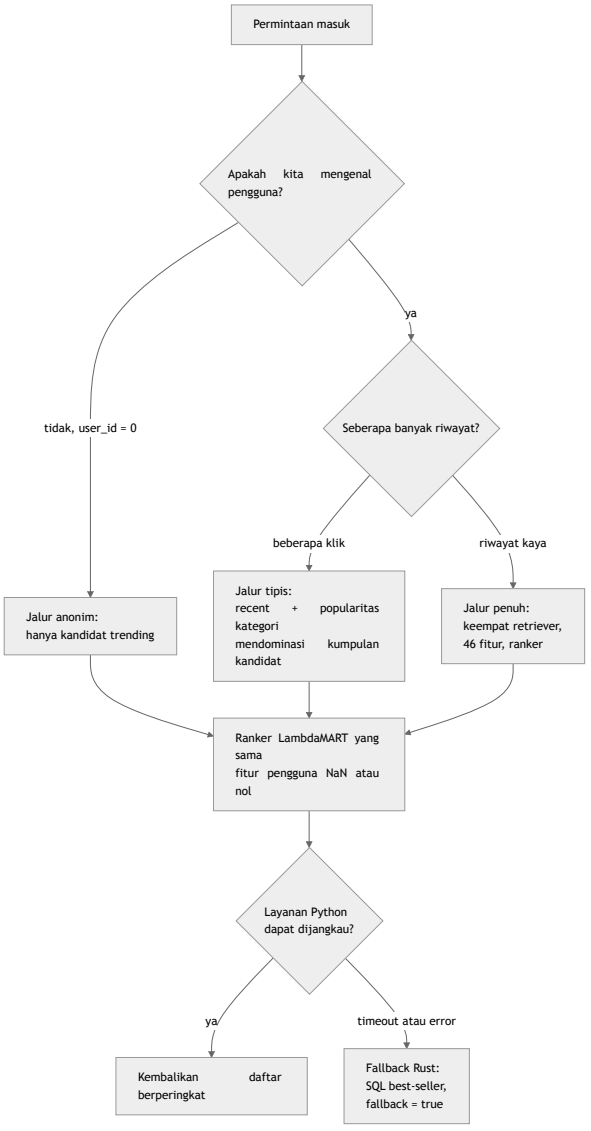
Intuisi: kolega baru, menu restoran, dan eskalator

Bayangkan Anda orang pertama yang menyapa pelanggan di sebuah toko buku. Seorang asing masuk. Anda tidak tahu apa pun tentangnya. Apa yang Anda lakukan? Anda tidak diam. Anda menunjuk meja buku terlaris, karena buku terlaris adalah tebakan terbaik untuk orang yang tidak dikenal. Setelah orang itu mengambil dua buku masak, Anda mengantarnya ke rak masakan. Sepuluh menit kemudian Anda sudah merekomendasikan penulis tertentu. Cold start persis seperti tangga ini: *mulailah dari apa*

yang berlaku untuk semua orang, lalu menyempit seiring bukti bertambah. Sistem rekomendasi yang melewatkan tangga ini akan diam saja (halaman kosong) atau berpura-pura mengenal orang yang belum pernah ia temui.

Item baru adalah bayangan cerminnya: pelayan yang hanya merekomendasikan hidangan yang sudah dipesan orang lain tidak akan pernah mempromosikan hidangan baru yang luar biasa, sehingga hidangan itu tidak pernah dipesan. Seseorang harus "menghabiskan" satu rekomendasi untuk sesuatu yang belum terbukti (masalah eksplorasi di bab 11).

Bias posisi punya analogi yang paling jelas. Ambil dua pintu identik: satu setinggi mata di depan lorong, satu lagi di ujung lorong di balik tikungan. Lebih banyak orang akan melewati pintu pertama, dan bukan karena pintu itu lebih baik. Rak supermarket menunjukkan efek yang sama: produk di setinggi mata lebih laku daripada yang di dekat lantai, bahkan jika posisinya ditukar. Jika Anda menilai produk hanya dari penjualan, Anda akan "belajar" bahwa produk setinggi mata lebih baik, lalu menaruhnya lagi di setinggi mata. Lingkaran yang membenarkan dirinya sendiri itu adalah **feedback loop**, dan kita akan melihatnya dua kali di bab ini, sekali untuk posisi dan sekali untuk popularitas.



Gambar 12.1: *tangga cold start. Ranker yang sama melayani setiap pengguna; yang berubah adalah seberapa banyak bukti yang dapat disumbangkan oleh generator kandidat dan fitur, dan jaring pengaman di luar menangkap kegagalan layanan.*

Algoritma: tangga, propensity, dan loop

Bagian ini terdiri dari empat bagian. Pertama kita lihat di mana cold start menggigit dalam pipeline yang sudah kita bangun. Lalu kita bahas bias posisi dan *inverse propensity weighting* dengan contoh kecil. Kemudian kita memodelkan feedback loop, dan terakhir kita menentukan di mana konteks seharusnya berada.

Di mana cold start menggigit

Telusuri pipeline dari bab 8–10 dan tanyakan di setiap tahap: *apa yang terjadi jika tidak ada data?*

1. **Pembangkitan kandidat (*candidate generation*).** Keempat retriever adalah `recent` (item yang pernah dilihat pengguna), `covisit` (item yang dilihat bersama item-item itu), `catpop` (item populer di kategori yang disukai pengguna), dan `global_trending`. Untuk pengguna tanpa riwayat, tiga yang pertama tidak mengembalikan apa-apa. `global_trending` tidak bergantung pada pengguna, sehingga *setiap* pengguna, termasuk yang benar-benar baru, menerima setidaknya item yang sedang trending secara global. Itulah anak tangga pertama, dan sudah tertanam di `candidates.generate`.
2. **Fitur.** Fitur tingkat pengguna dibangun dengan mengagregasi event. Pengguna tanpa event tidak memiliki baris di agregat event, sehingga *left join* menghasilkan nilai yang hilang. Builder mengisinya dengan sentinel eksplisit: penghitung menjadi 0, dan "hari sejak aktif" menjadi konstanta besar (`NEVER_DAYS = 999`). Untuk sisanya, LightGBM menerima nilai hilang secara native, dan mempelajari di setiap pemisahan (*split*) ke sisi mana nilai hilang harus pergi.
3. **Pemeringkatan (*ranking*).** Ranker adalah satu model untuk semua orang. Ia sudah melihat contoh latih dengan riwayat tipis maupun kosong, sehingga ia belajar apa yang harus dilakukan ketika `user_clicks_30d` bernilai nol: bersandar pada fitur tingkat item seperti CTR dan tingkat konversi yang dihaluskan. **Item baru** adalah kasus yang lebih sulit, karena sinyal kuat yang biasa semuanya bersifat perilaku. Produk yang ditayangkan satu jam lalu tidak punya klik, pesanan, maupun co-visit, dan CTR tingkat itemnya adalah prior hasil *Bayesian smoothing*. Rumus pemulusan dari bab 3 adalah alat yang tepat: dengan laju prior p_0 dan kekuatan prior s,

$$\hat{p} = \frac{x + p_0\, s}{n + s}$$

dengan x adalah jumlah klik dan n jumlah impresi. Dengan $x = 0$ dan $n = 0$ estimasinya adalah p_0, rata-rata katalog, bukan nol. Item baru dengan demikian diperlakukan sebagai "rata-rata sampai terbukti sebaliknya", bukan "buruk karena tidak ada yang mengkliknya". Tanpa pemulusan, item baru akan memiliki CTR $0/0$ (tidak terdefinisi) atau 0 (sangat rendah tanpa harapan). Ranker juga menerima `item_age_days`, sehingga ia dapat mempelajari efek kesegaran secara eksplisit.

Bias posisi dan inverse propensity weighting

Sekarang bias. Pengguna mengklik item hanya jika mereka *memperhatikannya* dan *menyukainya*. **Hipotesis pemeriksaan (*examination hypothesis*)** klasik menuliskannya sebagai perkalian:

$$P(\text{click} = 1 \mid \text{item } i, \text{position } k) = \underbrace{\theta_k}_{\text{examination}} \cdot \underbrace{r_i}_{\text{relevance}}$$

Di sini $\theta_k \in [0, 1]$ adalah probabilitas bahwa pengguna bahkan melihat posisi k (**propensity** posisi itu) dan r_i adalah seberapa besar pengguna menyukai item i jika mereka melihatnya. Kita ingin mempelajari r_i, tetapi log hanya berisi klik, yang mencampurkan θ_k dan r_i. Melatih ranker langsung pada klik mengajarkannya untuk menaruh *item yang ditampilkan tinggi* di atas, dan itu sirkular.

Perbaikan **inverse propensity weighting** (IPW) hanya satu baris: bobot setiap klik dengan $1/\theta_k$. Secara ekspektasi,

$$\mathbb{E}\left[\frac{c}{\theta_k}\right] = \frac{\theta_k \, r_i}{\theta_k} = r_i,$$

sehingga laju klik berbobot adalah estimasi relevansi yang tak bias (*unbiased*) terlepas dari di mana item ditampilkan. Berikut contoh hitung dengan angka mainan. Misalkan propensity-nya adalah $\theta_0 = 1.0$, $\theta_1 = 0.5$, $\theta_2 = 0.25$. Item A ditampilkan 100 kali di posisi 0 dan mendapat 20 klik. Item B ditampilkan 100 kali di posisi 2 dan mendapat 10 klik.

1. CTR mentah: A adalah $20/100 = 0.20$, B adalah $10/100 = 0.10$. Klik mentah mengatakan A dua kali lebih baik.
2. Bobot setiap klik: setiap klik pada A bernilai $1/1.0 = 1$, setiap klik pada B bernilai $1/0.25 = 4$.
3. Klik berbobot: A adalah $20 \times 1 = 20$, B adalah $10 \times 4 = 40$.
4. Relevansi yang sudah dikoreksi: A adalah $20/100 = 0.20$, B adalah $40/100 = 0.40$.

Setelah koreksi, B dua kali lebih relevan daripada A. Klik mentah justru terbalik, karena B "dihukum" akibat ditampilkan di slot ketiga.

IPW punya harga: variansi. Satu klik di posisi dengan $\theta_k = 0.02$ dihitung 50 kali. Satu klik yang kebetulan bisa mendominasi set latih. Dalam praktik kita melakukan **clipping** pada bobot, misalnya $w = \min(1/\theta_k, M)$ dengan M sekitar 10–50, menerima sedikit bias demi variansi yang jauh lebih kecil.

Masalah yang lebih sulit adalah mengestimasi θ_k. Jalan pintas yang menggoda adalah membacanya dari tabel CTR per posisi: $\hat{\theta}_k \propto \mathrm{CTR}_k / \mathrm{CTR}_0$. Itu hanya berhasil jika item dengan *kualitas yang sama* muncul di setiap posisi. Dalam sistem yang di-log, ranker sengaja menaruh tebakan terbaiknya di atas, sehingga tabel itu mencampuradukkan pemeriksaan dengan relevansi. Estimasi propensity yang jujur membutuhkan pengacakan: tukar dua item bersebelahan pada sebagian kecil lalu lintas, atau acak sebagian kecil halaman, lalu bandingkan. Alternatif yang lebih murah adalah memperlakukan posisi sebagai **fitur model** saat pelatihan (sehingga model dapat menisbahkan sebagian klik ke posisi) lalu menetapkannya ke konstanta saat *serving*, yang disarankan oleh latihan 3 di `recommender/README.md`.

Feedback loop

Bias posisi adalah feedback loop *lokal*; yang *global* bekerja di tingkat katalog. Katakanlah beranda menampilkan item A, yang memiliki 10 klik lebih banyak daripada item B minggu ini. Ranker, yang dilatih pada klik, memberi A skor lebih tinggi. A lebih sering ditampilkan, mengumpulkan lebih banyak klik, dan memperlebar selisih. Model mainan membuat dinamikanya konkret. Misalkan porsi

tampil setiap item pada minggu $t+1$ sebanding dengan kliknya pada minggu t, dan klik sebanding dengan porsi tampil dikali relevansi sebenarnya r. Maka

$$s_i^{(t+1)} = \frac{s_i^{(t)}\, r_i}{\sum_j s_j^{(t)}\, r_j}.$$

Ambil $r_A = 0.12$, $r_B = 0.10$ dan awal 50/50: minggu 1 menghasilkan $s_A = 0.06/0.11 = 0.545$ dan minggu 2 menghasilkan 0.590. Item yang sedikit lebih baik terus menambah porsi, dan item C yang tidak pernah ditampilkan dengan $r_C = 0.30$ tidak pernah ditemukan. Loop ini memperkuat apa yang sudah diyakini sistem; ia tidak mempertanyakannya.

Di mana konteks seharusnya berada?

Ranker memprediksi hasil 7 hari per pengguna, sehingga jam permintaan identik untuk semua kandidat milik satu pengguna dan tidak memiliki label per permintaan (`recommender/README.md`, "What's deliberately simplified"). Chaney dkk. (RecSys 2018) menunjukkan lewat simulasi bahwa melatih pada data yang sudah direkomendasikan menyeragamkan perilaku tanpa menambah utilitas; penawarnya adalah eksplorasi, diversitas, dan pencatatan impresi beserta posisinya.

Terakhir, konteks saat permintaan: jam, hari dalam minggu, perangkat, *surface* masuk. Repositori ini mencatat semuanya (tabel `sessions` menyimpan `surface` dan `device_os`; `events` memiliki `ts`, `position`, dan `surface`), namun ranker LightGBM dari bab 9–10 **tidak** menggunakannya sebagai fitur. Itu disengaja, dan `recommender/README.md` menjelaskan alasannya di bagian "What's deliberately simplified".

Tempat yang tepat untuk konteks adalah **model kedua yang real-time** yang dilatih pada log impresi: setiap baris adalah satu item yang ditampilkan, labelnya adalah apakah item itu diklik, dan fiturnya mencakup jam, surface, perangkat, dan *posisi saat item ditampilkan*. Model itu me-*re-score* beberapa lusin item yang sudah dipilih oleh ranker mingguan. **Fitur sesi (*session features*)** real-time termasuk dalam lapisan yang sama: "kategori yang diklik dalam 30 menit terakhir", "kueri pencarian terakhir", "item yang dimasukkan ke keranjang pada sesi ini". Retriever `recent` dan `Store.refresh()` sudah memberi repositori ini sedikit gambaran, karena klik yang dikirim dua detik lalu mengubah respons berikutnya; tetapi ranker itu sendiri hanya dilatih ulang secara berkala.

> **Tahukah kamu?**
>
> **Pengguna melihat dua hasil teratas hampir sama sering, tetapi mengklik yang pertama jauh lebih sering.** Joachims dan rekan-rekannya melacak mata mahasiswa yang mencari di web dan menemukan bahwa abstrak di peringkat 1 dan 2 dilihat dengan frekuensi yang hampir sama, namun pengguna mengklik tautan pertama jauh lebih sering daripada yang kedua. Dalam eksperimen lanjutan di mana dua hasil teratas ditukar tanpa sepengetahuan pengguna, pengguna tetap sangat cenderung mengklik apa pun yang ada di posisi pertama, bahkan ketika hasil kedua dinilai lebih relevan. Para penulis menyebutnya *trust bias* dan menyimpulkan bahwa klik "informative but biased" (informatif tetapi bias). Sumber: Joachims et al., SIGIR, 2005

> **Tahukah kamu?**
>
> **Heuristik ternyata sangat sulit dikalahkan ketika Anda tidak tahu apa-apa.** Schein dan rekan-rekannya membandingkan model probabilistik untuk masalah cold start, yaitu merekomendasikan item yang belum pernah dinilai siapa pun, dengan "heuristic recommender" yang sepele. Pada tugas umpan balik implisit, sekadar mengurutkan pengguna berdasarkan banyaknya film yang telah mereka nilai memberikan performa global yang hampir sama dengan *aspect model* mereka, sehingga para penulis berargumen bahwa baseline heuristik semacam itu selalu perlu disertakan dalam evaluasi jenis ini. Sumber: Schein et al., SIGIR, 2002

Penelusuran kode

Kita akan mengikuti tangga yang sama dengan pipeline: bagaimana retriever menghadapi riwayat yang hilang, bagaimana fitur menutup celah, bagaimana halaman produk menangani item tanpa co-view, dan terakhir apa yang dilakukan API Rust ketika seluruh layanan Python menghilang.

Kandidat untuk semua orang

Mulai dari satu-satunya retriever yang sama sekali tidak membutuhkan pengguna:

`recommender/recsys/candidates.py · global_trending`

```
def global_trending(hist: History, available: pd.DataFrame) -> pd.DataFrame:
    pop = item_popularity(hist, 3.0)
    pop = pop[pop.index.isin(available["item_id"])].nlargest(N_GLOBAL)
    return pd.DataFrame({"item_id": pop.index, "cand_global_pop_rank": np.arange(len(pop))})
```

`global_trending` mengambil interaksi berbobot tiga hari terakhir (event klik, keranjang, dan pesanan, masing-masing dengan bobotnya sendiri), mempertahankan item yang tersedia stoknya, dan mengembalikan `N_GLOBAL` teratas. Fungsi ini tidak menerima argumen pengguna. Satu fakta itu adalah seluruh cerita pengguna anonim: fungsi ini menghasilkan kandidat yang sama untuk semua orang.

Setiap id pengguna di `generate` di-*cross-join* dengan frame trending, sehingga tidak ada pengguna yang memiliki nol kandidat dan ranker tidak pernah menerima grup kosong. Skor sumber yang hilang menjadi 0.0 dan peringkat trending yang hilang menjadi 999; semua kandidat pengguna cold memiliki `cand_n_sources` sama dengan 1, pola yang dipelajari ranker untuk dikenali.

Fitur yang toleran terhadap kekosongan

Di `features.user_features` setiap pengguna memiliki baris (dimulai dari tabel `users` penuh), penggabungan hitungan berupa left join, dan `fillna` mengubah hitungan yang hilang menjadi nol. `_smoothed`, rumus $\hat{p} = (x + p_0 s)/(n + s)$, dipanggil di `user_features` sebagai `_smoothed(u["user_clicks_30d"], u["_impr_30d"], global_ctr, 20)`: pengguna baru mendapat tepat `global_ctr`, dan $s = 20$ bertindak seperti dua puluh impresi imajiner pada laju rata-rata.

Setelah join, `user_days_since_active` untuk pengguna tanpa event diisi dengan `NEVER_DAYS = 999`. Ini sentinel yang lebih baik daripada nol, karena nol berarti "baru saja aktif", kebalikan dari kenyataan. Fitur item mengikuti filosofi yang sama di `item_features`: hitungan menjadi nol, laju dihaluskan ke

rata-rata katalog, dan `item_age_days` dihitung langsung dari `listed_at`, yang ada bahkan untuk item yang belum pernah dilihat siapa pun.

Halaman produk tanpa co-view

Strip "Similar items" di halaman produk menghadapi versi masalah ini untuk setiap produk yang baru ditayangkan:

recommender/recsys/pipeline.py · similar_items

```
def similar_items(tables: dict[str, pd.DataFrame], now: pd.Timestamp, item_id: int, k: int) ->
pd.DataFrame:
    """Product page "similar items": co-visited items first, then popular items of the same
category."""
    hist = data.History.at(tables, now)
    empty = pd.DataFrame({"item_id": pd.Series(dtype="int64"), "score":
pd.Series(dtype="float64"), "source": pd.Series(dtype="object")})
    if item_id not in set(hist.items["item_id"]):
        return empty
    in_stock = hist.items.loc[hist.items["in_stock"], ["item_id", "category_id"]]
    m = candidates.covisitation_matrix(hist)
    co = m[(m["item_id_a"] == item_id) & (m["item_id_b"] != item_id) &
m["item_id_b"].isin(in_stock["item_id"])]
    co = co.sort_values("w", ascending=False).head(k)
    out = pd.DataFrame({"item_id": co["item_id_b"].to_numpy(), "score": co["w"].to_numpy(),
"source": "covisit"})

    category = hist.items.loc[hist.items["item_id"] == item_id, "category_id"].iloc[0]
    pop = candidates.item_popularity(hist, 7)
    same = in_stock[(in_stock["category_id"] == category) & (in_stock["item_id"] != item_id) &
~in_stock["item_id"].isin(out["item_id"])]
    same = same.assign(score=same["item_id"].map(pop).fillna(0.0)).sort_values("score",
ascending=False, kind="stable")
    fill = pd.DataFrame({"item_id": same["item_id"].to_numpy(), "score": same["score"].to_numpy(),
"source": "same_category"})
    out = pd.concat([out, fill.head(k - len(out))], ignore_index=True) if len(out) < k else out
    return out.astype({"item_id": "int64", "score": "float64"}).reset_index(drop=True) if len(out)
else empty
```

Ini adalah tangga dua anak tangga dalam sekitar dua puluh baris. Pertama ia meminta matriks co-visitation untuk item yang dilihat bersama `item_id`. Item baru tidak punya baris di sana, sehingga `co` kosong. Lalu ia mengisi slot yang tersisa dengan item tersedia dari *kategori yang sama*, diurutkan berdasarkan popularitas tujuh hari, dan melabelinya `same_category`. Perhatikan `.fillna(0.0)`: item tanpa popularitas sama sekali tetap boleh muncul, di peringkat terakhir. Kolom `source` digunakan UI untuk memberi tahu pengguna (dan Anda) anak tangga mana yang menjawab.

Jaring pengaman: ketika Python mati

Anak tangga terakhir sama sekali tidak ada di Python. API Rust memperlakukan recommender sebagai dependensi opsional. Pertama, fallback SQL:

backend/src/recommender.rs · fallback

```rust
/// Used when the recommender is down: recent best-sellers, padded by rating count.
async fn fallback(
    pool: &PgPool,
    k: i64,
    category_id: Option<i64>,
    exclude_id: Option<i64>,
) -> Result<Vec<RecCard>, sqlx::Error> {
    let mut qb = QueryBuilder::new(format!("SELECT {CARD_COLS} {CARD_FROM} WHERE i.is_active AND
i.stock > 0"));
    if let Some(c) = category_id {
        qb.push(" AND i.category_id = ").push_bind(c);
    }
    if let Some(x) = exclude_id {
        qb.push(" AND i.id <> ").push_bind(x);
    }
    qb.push(
        " ORDER BY (SELECT count(*) FROM events e WHERE e.item_id = i.id AND e.event_type =
'order'
                    AND e.ts > now() - interval '7 days') DESC, i.rating_count DESC, i.id LIMIT
",
    )
    .push_bind(k);
    let cards = qb.build_query_as::<ProductCard>().fetch_all(pool).await?;
    Ok(cards
        .into_iter()
        .enumerate()
        .map(|(i, card)| RecCard {
            card,
            score: 0.0,
            source: "trending".into(),
            rank: i as i32 + 1,
            contributions: vec![],
        })
        .collect())
}
```

Fungsi ini menerima `category_id` opsional (dipakai oleh "Similar items") dan `exclude_id` opsional, lalu mengurutkan item aktif yang tersedia stoknya berdasarkan jumlah event `order` dalam tujuh hari terakhir, memutus seri dengan `rating_count`, lalu dengan id agar keluarannya deterministik. Setiap baris mendapat `source: "trending"`, `score: 0.0`, dan vektor `contributions` kosong, melaporkan dengan jujur bahwa tidak ada yang dipersonalisasi. Sekarang rute yang memutuskan kapan menggunakannya:

backend/src/recommender.rs · recommend_route

```
async fn recommendations(
    State(st): State<AppState>,
    MaybeUser(user): MaybeUser,
    Query(q): Query<RecsQuery>,
) -> AppResult<Json<Value>> {
    let k = q.k.unwrap_or(20).clamp(1, 50);
    let req = pb::RecommendRequest {
        user_id: user.unwrap_or(0),
        k: k as i32,
        surface: q.surface.unwrap_or_else(|| "home_feed".into()),
    };
    let mut rec = st.rec.clone();
    let res = tokio::time::timeout(Duration::from_secs(5), rec.recommend(req)).await;
    match res {
        Ok(Ok(resp)) => {
            let r = resp.into_inner();
            let items = hydrate(&st.pool, r.items).await?;
            Ok(Json(json!({
                "items": items,
                "model_version": r.model_version,
                "latency_ms": r.latency_ms,
                "candidates_considered": r.candidates_considered,
                "fallback": false,
            })))
        }
        other => {
            match other {
                Ok(Err(e)) => tracing::warn!("recommender error, using fallback: {e}"),
                _ => tracing::warn!("recommender timed out, using fallback"),
            }
            let items = fallback(&st.pool, k, None, None).await?;
            Ok(Json(json!({
                "items": items,
                "model_version": "fallback",
                "latency_ms": 0,
                "candidates_considered": 0,
                "fallback": true,
            })))
        }
    }
}
```

Tiga detail layak diperhatikan. Panggilan gRPC dibungkus dengan `tokio::time::timeout(Duration::from_secs(5), ...)`, sehingga layanan yang *lambat* diperlakukan sama seperti yang *mati*: pengguna tidak pernah dibiarkan menatap spinner lebih dari lima detik. `match` hanya memiliki satu cabang sukses; cabang *catch-all* `other` mencakup baik error gRPC maupun timeout yang habis, mencatat peringatan (membedakan error dari timeout), dan jatuh ke `fallback(&st.pool, k, None, None)`. Dan JSON yang dikembalikannya memiliki bentuk yang persis sama dengan jalur sukses, ditambah `"fallback": true` dan `"model_version": "fallback"`. Frontend tidak memerlukan jalur kode kedua; ia hanya perlu membaca flag itu dan menampilkan banner.

Banner kuning di beranda (tangkapan layar `fallback-banner.png`, ditampilkan dalam tur di bab 13) adalah satu-satunya hal yang perlu diubah frontend.

Tabel posisi di bawah ini ada hanya karena browser mencatat impresi *beserta posisinya* (`ProductGrid.vue`: `position` adalah indeks array, satu impresi per item per grid). Tanpa log itu Anda tidak dapat menghitung CTR maupun propensity.

Jalankan

Skrip pendamping `book/code/ch12_coldstart_bias.py` menghitung tiga hal pada data simulasi yang sama dengan bab-bab lain: CTR per posisi tampil di atas log impresi, kualitas ranker per kelompok keaktifan pengguna, dan porsi item baru di 20 teratas setiap metode. Jalankan dari direktori `book/`:

```
uv run python code/ch12_coldstart_bias.py
```

Stdout-nya disimpan di `results/ch12_coldstart_bias.txt`. Kita membacanya bagian demi bagian. Pertama, tabel posisi:

KELUARAN TERMINAL

```
position
0               16770    3999  0.2385
1               16770    2908  0.1734
2               16770    2770  0.1652
3               16770    2490  0.1485
4               16770    2289  0.1365
5               16770    2093  0.1248
6               16770    1128  0.0673
7               16770     933  0.0556
8               16770     881  0.0525
9               16770     818  0.0488
10              16770     702  0.0419
11              16770     262  0.0156

position 0 vs position 11: 15.3x higher CTR
```

Lalu kelompok keaktifan, yang membandingkan baseline popularitas `pop_7d` dari bab 3 dengan ranker LambdaMART dari bab 10:

KELUARAN TERMINAL

```
=== Metrics by user activity (clicks before the cutoff) ===
                        users  recall@20  order_recall@20  ndcg@20
bucket      method
0–2 clicks  pop_7d         78     0.0795           0.0753   0.0241
            LambdaMART     78     0.1108           0.1774   0.0666
3–10 clicks pop_7d        258     0.0785           0.0938   0.0489
            LambdaMART    258     0.2143           0.3184   0.1395
>10 clicks  pop_7d        611     0.0785           0.1136   0.0499
            LambdaMART    611     0.3177           0.4764   0.2606
  0–2 clicks   LambdaMART / pop_7d ndcg@20 = 2.76x
  3–10 clicks  LambdaMART / pop_7d ndcg@20 = 2.86x
  >10 clicks   LambdaMART / pop_7d ndcg@20 = 5.22x
```

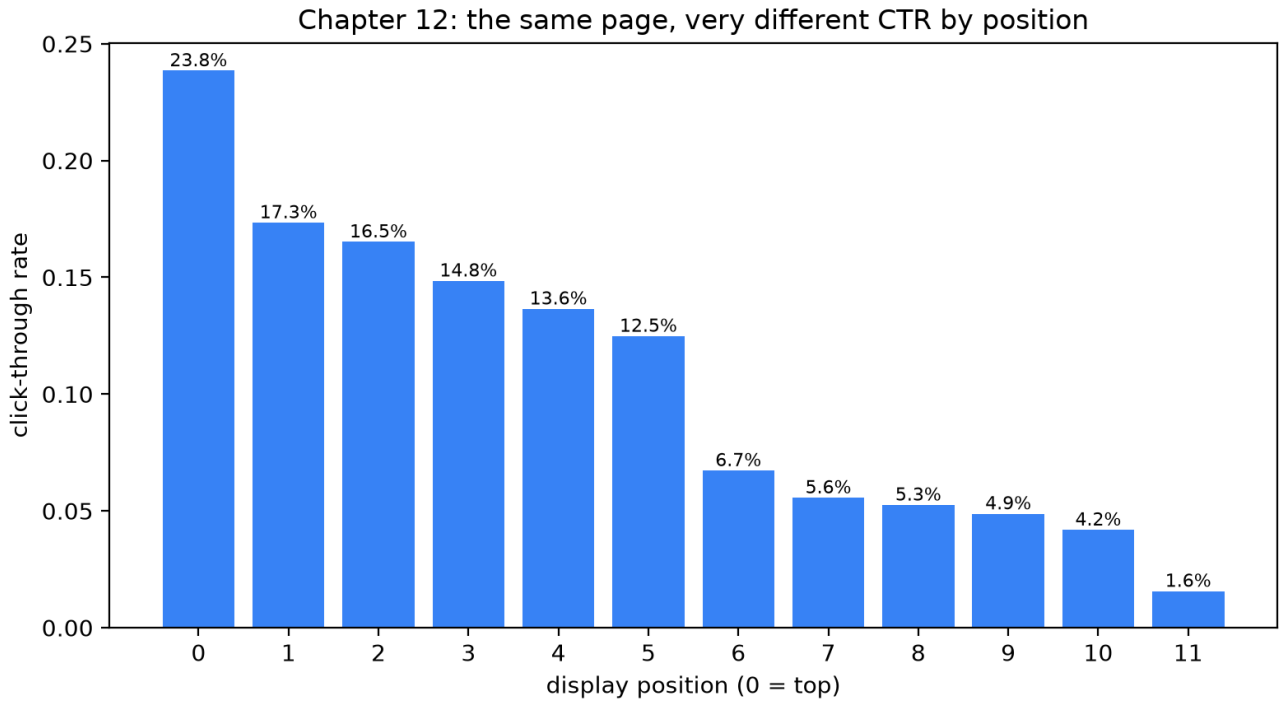
Dan terakhir, eksposur item baru:

KELUARAN TERMINAL

```
=== Share of NEW items (listed < 7 days before the cutoff) in the top 20 ===
  catalogue (available items)   2.16%
  pop_7d                        0.00%
  LambdaMART                    0.25%
  future positives              2.68%
```

Hasil

Tabel posisi

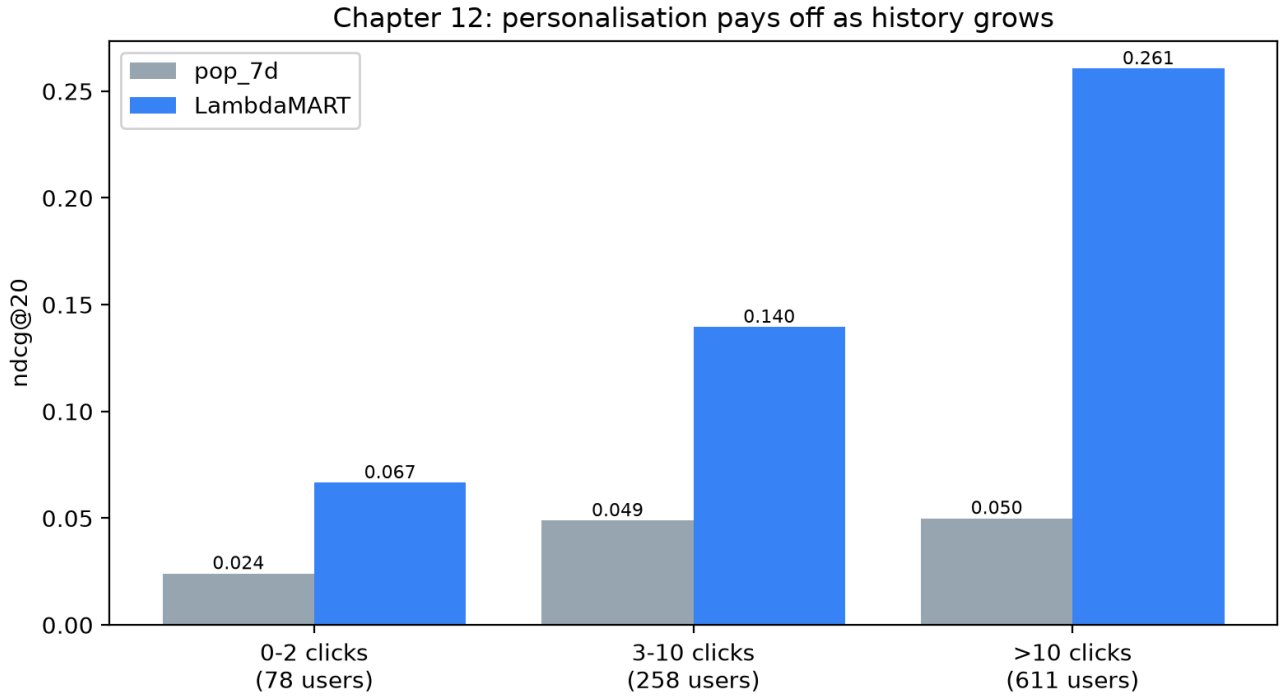


Gambar 12.2: *CTR menurut posisi tampil pada seluruh impresi sebelum cutoff. Posisi 0 berkonversi 23.8% dan posisi 11 sebesar 1.6%.*

Gambar: click-through rate menurut posisi tampil. Setiap posisi memiliki jumlah impresi yang persis sama, sehingga batang-batangnya dapat dibandingkan langsung.

Masing-masing dari dua belas posisi memiliki tepat 16,770 impresi, yang berarti penyebutnya sama dan tinggi batang adalah laju yang jujur. Posisi 0 memiliki CTR 0.2385 dan posisi 11 memiliki 0.0156, rasio 15.3 kali. Penurunannya tidak mulus. Dari posisi 0 ke posisi 1, CTR turun dari 0.2385 ke 0.1734; posisi 1 sampai 5 menurun perlahan dari 0.1734 ke 0.1248; lalu antara posisi 5 (0.1248) dan posisi 6 (0.0673) nilainya hampir separuh dalam satu langkah. Tebing itu konsisten dengan halaman yang enam item pertamanya terlihat tanpa menggulir. Joachims et al. mengamati patahan serupa di batas lipatan layar (*fold*) pada data eye-tracking mereka. Posisi 6 sampai 10 kemudian menurun perlahan, dan posisi 11 turun tajam lagi.

Personalisasi membuahkan hasil seiring riwayat bertambah



Gambar 12.3: *ndcg@20 untuk pop_7d dan LambdaMART pada tiga kelompok keaktifan pengguna.*

Gambar: ndcg@20 menurut kelompok jumlah klik sebelumnya. Baseline popularitas nyaris tidak bergerak terhadap riwayat pengguna; ranker naik tajam.

Tabel kedua membagi 947 pengguna evaluasi (78 + 258 + 611) berdasarkan jumlah klik yang mereka miliki sebelum cutoff. Lihat baris `pop_7d` terlebih dulu. ndcg@20-nya adalah 0.0241 untuk pengguna dengan 0–2 klik, 0.0489 untuk 3–10 klik, dan 0.0499 untuk lebih dari 10. Popularitas mengabaikan pengguna, sehingga riwayat tidak membantunya; recall@20-nya sama, 0.08, di ketiga kelompok (0.0795, 0.0785, 0.0785).

Baris LambdaMART naik tajam: ndcg@20 sebesar 0.0666, 0.1395, dan 0.2606. Untuk pengguna terberat, rasio terhadap `pop_7d` adalah 5.22x. Untuk kelompok paling dingin, 2.76x. Dua hal layak ditekankan.

Pertama, **ranker sudah mengalahkan popularitas bahkan untuk pengguna dengan paling banyak dua klik**. Alasan yang masuk akal (tidak kita isolasi di sini) adalah bahwa kandidat trending ditambah fitur tingkat item seperti CTR dan tingkat konversi yang dihaluskan menghasilkan model popularitas yang lebih baik daripada hitungan tujuh hari mentah. Rasio untuk kelompok 3–10 (2.86x) mirip dengan kelompok cold (2.76x); lonjakan besar baru terjadi di atas sepuluh klik, ketika retriever personal (`recent`, `covisit`, `catpop`) dan fitur pengguna-item mulai mendominasi kumpulan kandidat dan pemeringkatan.

Tabel eksposur item baru

Tabel ketiga adalah yang paling tidak nyaman. Item yang ditayangkan kurang dari tujuh hari sebelum cutoff membentuk 2.16% dari katalog yang tersedia. Di antara *positif masa depan* (item yang benar-benar berinteraksi dengan pengguna selama pekan label), mereka membentuk 2.68%, sedikit lebih

besar daripada porsinya di katalog, jadi pengguna memang menginginkan item segar. Apa yang ditampilkan metode-metode itu? `pop_7d` menaruh tepat 0.00% item baru di 20 teratasnya. LambdaMART menaruh 0.25%. Keduanya satu orde besaran di bawah 2.68% yang akan disambut pengguna.

Ini adalah feedback loop yang beraksi, tertangkap dalam satu tabel. Ranker tidak jahat: item baru memiliki fitur perilaku yang lemah, sehingga diberi skor rendah, sehingga tidak ditampilkan, sehingga tetap lemah. Model dengan ndcg offline lebih baik daripada baseline popularitas tetap bisa menjadi "warga" yang lebih buruk bagi para penjual. Ini pengingat bahwa metrik offline memberi penghargaan pada prediksi masa lalu. Metrik itu tidak menghargai *memberi kesempatan kepada masa depan*.

Jebakan dan praktik di produksi

Memperlakukan "pengguna baru" sebagai satu kasus. Setidaknya ada tiga: pengunjung yang belum login, pengguna yang baru mendaftar dengan profil tetapi tanpa perilaku, dan pengguna yang kembali setelah pergi berbulan-bulan. Repositori ini menangani yang pertama dengan `user_id = 0`, yang sampai ke layanan melalui `Recommend` sebagai permintaan biasa dan mendapat kandidat trending. Yang kedua dapat memanfaatkan kolom `city`, `age_bucket`, dan `device_tier` yang sudah dikumpulkan formulir pendaftaran. Yang ketiga paling baik ditangani dengan fitur seperti `user_days_since_active` yang memperlihatkan jeda itu kepada model.

Konteks di tempat yang salah. Menambahkan "jam dalam sehari" ke ranker mingguan memberi Anda fitur yang konstan dalam setiap grup; paling baik ia diabaikan, paling buruk ia membuat pohon menyesuaikan diri dengan noise. Taruh konteks di lapisan real-time, yang memiliki baris dan label per impresi. README repositori menjelaskan versi produksi dari desain ini: ranker kedua yang real-time, dilatih pada log `impression`, yang dapat menggunakan jam, surface, dan posisi, ditempatkan setelah tumpukan selera jangka panjang.

Ringkasan

- Cold start hadir dalam tiga jenis (pengguna baru, item baru, sistem baru); obatnya untuk masing-masing adalah *tangga* fallback yang menyempit dari global ke personal seiring bukti bertambah.
- Di sini `global_trending` menjamin setiap pengguna, termasuk `user_id = 0`, mendapat setidaknya sebagian kandidat, dan pemulusan serta sentinel eksplisit (`NEVER_DAYS`) menjaga fitur tetap bermakna bagi pengguna tanpa event.
- Jika layanan Python lambat atau mati, API Rust mengembalikan daftar best-seller SQL dengan `fallback: true` setelah timeout lima detik, dengan bentuk JSON yang sama seperti respons normal.
- Bias posisi terlihat di log impresi: CTR turun dari 0.2385 di posisi 0 menjadi 0.0156 di posisi 11, tetapi CTR mentah per posisi mencampurkan pemeriksaan dengan relevansi dan bukan kurva propensity.
- Inverse propensity weighting membagi setiap klik dengan probabilitas bahwa posisinya diperiksa; ia tak bias tetapi bervariansi tinggi, sehingga bobot di-clip dan propensity diestimasi dengan pengacakan.

- Ranker mengalahkan baseline popularitas di semua kelompok keaktifan, tetapi marginnya tumbuh dari 2.76x (0–2 klik) ke 5.22x (lebih dari 10 klik); kohort cold berukuran kecil, sehingga metriknya berisik.
- Item baru adalah 2.16% dari katalog dan 2.68% dari positif masa depan, tetapi hanya 0.25% dari 20 teratas LambdaMART: model yang lebih baik secara offline tetap dapat membuat katalog kelaparan, dan itu adalah feedback loop.
- Konteks saat permintaan seharusnya berada di model real-time tingkat impresi, bukan di ranker mingguan "7 hari ke depan".

Kuis

Q12.1 Pernyataan mana tentang `candidates.global_trending` yang benar?

A. Ia mengembalikan item yang dipersonalisasi dari sesi terbaru pengguna.
B. Ia tidak bergantung pada pengguna, sehingga menjamin setiap pengguna, termasuk `user_id = 0`, mendapat sebagian kandidat.
C. Ia hanya dipanggil ketika layanan Python mati.
D. Ia mengurutkan item berdasarkan diskon harganya.

Q12.2 Item baru memiliki 0 impresi dan 0 klik. CTR yang dihaluskan di `features._smoothed` dihitung sebagai $(x + p_0 s)/(n + s)$. Apa yang dilihat model sebagai CTR item ini?

A. 0, karena tidak ada yang mengkliknya.
B. Tidak terdefinisi, yang membuat builder crash.
C. Prior p_0, yaitu CTR rata-rata katalog.
D. 1, karena item baru diberi dorongan.

Q12.3 Mengapa "jam dalam sehari" *bukan* fitur di ranker LightGBM buku ini?

A. Jam dalam sehari tidak memiliki daya prediksi di sistem e-commerce mana pun.
B. LightGBM tidak dapat menangani fitur numerik yang berputar kembali pada 24.
C. Ranker memprediksi hasil 7 hari per pengguna, sehingga jam permintaan sama untuk semua kandidat milik satu pengguna dan tidak ada label per permintaan untuk dipelajari.
D. Tabel `events` tidak menyimpan timestamp.

Q12.4 Dengan kata-kata Anda sendiri, jelaskan bagaimana feedback loop popularitas terbentuk pada recommender yang dilatih dengan klik, dan sebutkan dua teknik yang mengurangi efeknya.

Q12.5 Rute Rust `recommendations` beralih ke SQL setelah timeout 5 detik *dan* ketika panggilan gRPC mengembalikan error. Mengapa masuk akal memperlakukan respons yang lambat sama seperti yang gagal? Dua field apa dalam JSON yang memberi tahu frontend bahwa fallback digunakan?

Q12.6 Baca fungsi `fallback` yang disertakan. Item P memiliki 0 event `order` dalam tujuh hari terakhir dan `rating_count = 500`. Item Q memiliki 1 event `order` dalam tujuh hari terakhir dan `rating_count = 5`. Keduanya aktif dan tersedia stoknya. Mana yang peringkatnya lebih tinggi, dan mengapa query berakhir dengan `i.id`?

Q12.7 Propensity mainan adalah $\theta_0 = 0.8$ dan $\theta_3 = 0.2$. Item A ditampilkan 200 kali di posisi 0 dan menerima 40 klik. Item B ditampilkan 200 kali di posisi 3 dan menerima 20 klik. Hitung CTR mentah dan estimasi relevansi hasil koreksi IPW untuk masing-masing item, dan sebutkan item mana yang lebih relevan.

Q12.8 *Praktik langsung.* Buka `book/code/ch12_coldstart_bias.py` dan ubah `BUCKETS` sehingga juga memuat kelompok nol klik, misalnya `("0 clicks", 0, 0)` di urutan pertama (dan atur kelompok berikutnya mulai dari 1). Jalankan ulang skripnya. Apa yang Anda perkirakan akan teramati tentang jumlah pengguna di kelompok baru, tentang rasio `LambdaMART / pop_7d`, dan tentang seberapa besar Anda mempercayai rasio itu? Laporkan apa yang sebenarnya Anda lihat.

Q12.9 Mengapa praktisi melakukan *clipping* pada bobot inverse propensity, misalnya pada $w = \min(1/\theta_k, 20)$?

A. Karena propensity bisa negatif.
B. Untuk mengurangi variansi: klik di posisi berpropensity sangat rendah jika tidak di-clip akan dihitung seperti puluhan klik dan mendominasi pelatihan, dengan bayaran bias yang kecil.
C. Agar bobot berjumlah satu.
D. Karena LightGBM menolak bobot di atas 20.

Bab 13 — Membangun sistemnya

Selasa malam, beberapa menit setelah gajian, Dewi membuka aplikasi marketplace di ponselnya. Ia mengetuk listing makanan kucing, menggulir foto-fotonya, memutuskan harganya agak mahal, lalu menekan tombol kembali. Ia tidak membeli apa pun. Ia tidak menekan "suka" pada apa pun. Ia bahkan tidak mengetik satu kata pun. Namun, jika tiga puluh detik kemudian ia kembali ke halaman beranda, feed-nya seharusnya sudah tampak sedikit berbeda: lebih banyak perlengkapan hewan peliharaan, lebih sedikit casing ponsel. Halaman yang baru saja ia tinggalkan harus mengubah apa yang diyakini sistem tentang dirinya.

Yang akan Anda pelajari

- Bagaimana empat proses (Vue, Rust/Axum, PostgreSQL, Python gRPC) membagi tanggung jawab, dan mengapa recommender hanya *membaca* database.
- Perjalanan satu klik, dari sebuah impresi hingga rekomendasi yang berubah, dengan kode di setiap lompatan.
- Bagaimana impresi, klik, dan dwell (lama berada di halaman) ditangkap, dan mengapa urutan dua *listener* `pagehide` menentukan apakah sebuah klik tercatat.
- Bagaimana sesi diturunkan, bagaimana recommender menyegarkan salinan database di memori, serta bagaimana hidrasi (*hydration*), *fallback*, dan pelatihan ulang bekerja.
- Bagaimana satu berkas `.proto` menghasilkan kedua sisi, dan berapa latensi *end-to-end*-nya (kita mengukur dan memprofilnya).

Intuisi: restoran dengan buku besar

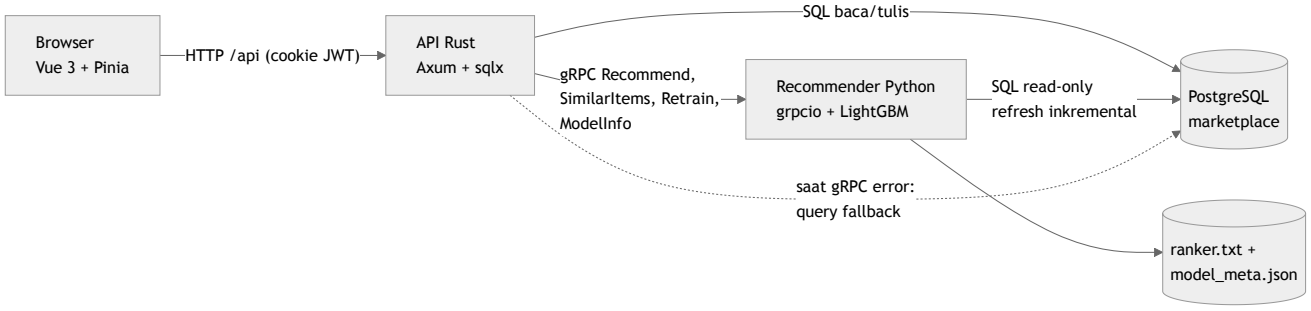
Bayangkan restoran yang sibuk. Para pelayan mencatat pesanan, mengantar piring, dan menulis aktivitas setiap meja ke dalam sebuah buku besar (*ledger*): siapa yang duduk, apa yang mereka lihat di menu, apa yang mereka pesan. Dapur punya seorang koki yang sangat pandai menyarankan hidangan yang tepat, tetapi koki itu tidak pernah berbicara dengan pelanggan dan tidak pernah menulis di buku besar. Setiap kali pelayan bertanya, "apa yang sebaiknya kita sarankan untuk meja dua belas?", koki melirik buku besar, berpikir, lalu menjawab dengan daftar pendek. Jika koki sedang sakit, pelayan tetap memegang kartu cetak berisi lima hidangan terlaris minggu ini dan menyerahkannya ke meja itu. Tidak ada yang pulang dengan lapar.

Itulah persis arsitektur ini. **Buku besar** adalah PostgreSQL. **Para pelayan** adalah API Rust, satu-satunya penulis: ia mendaftarkan pengguna, menangani aksi keranjang dan *checkout*, serta menambahkan event perilaku. **Koki** adalah layanan recommender Python, yang membaca buku besar lewat koneksi *read-only* dan mengembalikan saran berperingkat melalui gRPC. **Menu** yang dilihat pelanggan adalah *single-page application* Vue. **Kartu cetak** adalah query *fallback* di dalam API Rust.

Pembagian ini disengaja, dengan tiga alasan.

1. **Satu penulis berarti satu set aturan.** Stok tidak pernah negatif, sebuah pesanan hanya bergerak `paid → shipped → completed`, dan klien tidak dapat memalsukan event `order`, semuanya di satu tempat.

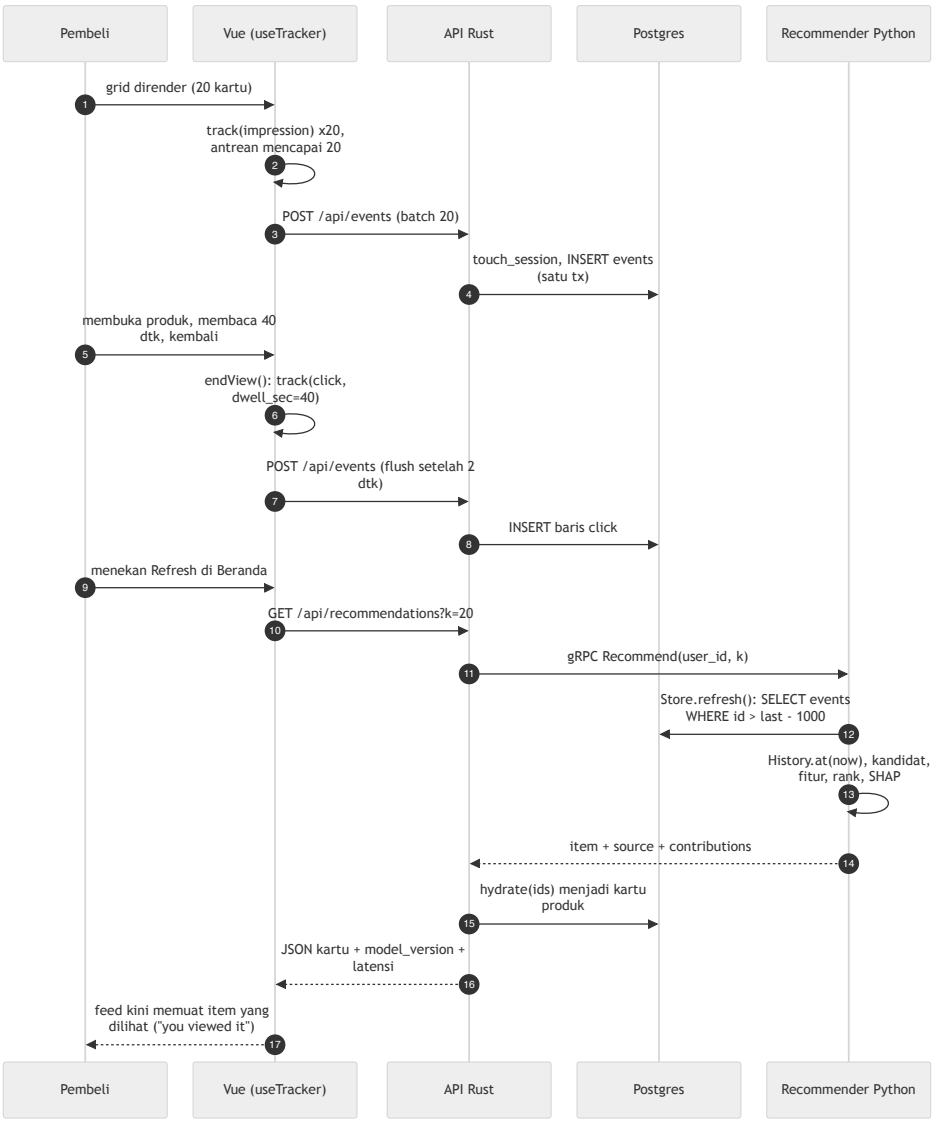
2. **Recommender boleh salah atau mati tanpa menjatuhkan toko.** Ia berada di balik batas jaringan dengan *timeout* dan tidak memegang koneksi yang bisa menulis: di `recsys/db.py`, `connect(read_only=True)` menyetel `default_transaction_read_only=on`, sehingga `INSERT` yang tidak disengaja akan gagal dengan keras lewat `ReadOnlySqlTransaction`.
3. **Setiap sisi memakai alat yang tepat.** Rust untuk API konkuren dengan pengecekan tipe dan transaksi; Python untuk pandas, LightGBM, dan kode riset.



Gambar 13.1: *Empat proses. Rust adalah satu-satunya penulis; Python membaca Postgres lewat koneksi read-only dan dipanggil melalui gRPC.*

Algoritma: perjalanan satu klik

"Algoritma" bab ini adalah sebuah protokol: urutan langkah yang mengubah aksi pengguna menjadi peringkat yang berubah. Mari kita tuliskan dulu sebagai diagram urutan (*sequence diagram*), lalu menelusurinya langkah demi langkah.



Gambar 13.2: *Satu klik, dari ujung ke ujung. Paruh pertama mencatat impresi dan klik beserta dwell time-nya; paruh kedua membacanya kembali menjadi feed baru.*

Langkah-langkah bernomor:

1. **Render dan amati.** `ProductGrid.vue` memanggil `track` dengan `impression` satu kali per item per grid, dengan indeksnya sebagai `position` dan nama surface sebagai konteks.
2. **Antre dan flush.** `useTracker.ts` melakukan flush pada 20 event yang mengantre atau 2 detik setelah event pertama, dengan satu `POST /api/events`.
3. **Validasi dan simpan.** `post_events` hanya menerima `impression`, `click`, dan `hide`, paling banyak 100 per batch, dan memasukkannya dalam satu transaksi dengan `session_id` pengguna saat ini.
4. **Dwell menjadi klik.** Ketika pembeli meninggalkan halaman produk, `Product.vue` mengirim satu `click` yang membawa `dwell_sec`, sinyal implisit terkuat: `ui_max_dwell_sec` adalah fitur teratas berdasarkan *gain* pada halaman model.
5. **Bertanya.** Halaman beranda memanggil `GET /api/recommendations`; Rust memanggil layanan Python dengan *timeout* 5 detik.
6. **Refresh.** `Recommend` pertama-tama memanggil `Store.refresh()`, yang hanya membaca event yang lebih baru daripada yang sudah dilihatnya.

7. **Jalankan Bab 8-11.** `History.at(now)`, kandidat, `features.build`, penilaian LightGBM, `rerank.rerank`, SHAP untuk item yang lolos.
8. **Hidrasi dan respons.** gRPC mengembalikan id, skor, source, dan contributions; Rust mengambil kartunya dalam satu query.

Contoh hitungan dengan angka sederhana

Seberapa cepat peringkat berubah setelah sebuah klik? Misalkan T_{flush} adalah jeda flush di klien, T_{refresh} *throttle* refresh milik recommender, dan L waktu permintaan rekomendasi itu sendiri. Pada kasus terburuk, baris baru tiba tepat setelah recommender melakukan refresh, sehingga

$$T_{\text{stale}} \leq T_{\text{flush}} + T_{\text{refresh}} + L.$$

Dalam basis kode ini, `FLUSH_MS = 2000` di `useTracker.ts` dan `MIN_REFRESH_SEC = 1.0` di `db.py`. Tambahkan median waktu permintaan yang diukur nanti di bab ini, sekitar 0.84 dtk, dan kasus terburuknya sekitar $2 + 1 + 0.84 = 3.84$ detik antara "pembeli menekan kembali" dan "pembeli melihat feed yang sudah mengetahuinya". Bagi pembeli tipikal yang menekan Refresh beberapa detik setelah kembali, item baru itu sudah ada di feed. Itulah keputusan "rekomendasi dihitung saat diminta" dari rencana proyek yang membuahkan hasil: tidak ada batch malam hari di antara perilaku dan dampaknya.

> **Tahukah kamu?**
>
> **Mengapa loop ini tidak bisa diserahkan kepada sebuah notebook.** Dalam "Hidden Technical Debt in Machine Learning Systems", Sculley et al. mengamati bahwa hanya sebagian kecil dari sistem ML dunia nyata adalah kode ML itu sendiri; infrastruktur di sekelilingnya untuk pengumpulan data, *serving*, dan *monitoring* itu "vast and complex" (luas dan kompleks). Di repositori ini proporsi yang sama terlihat: model ranking berada dalam segelintir berkas Python, sedangkan pengumpulan, penyimpanan, dan pengantaran input serta outputnya mencakup frontend, API, skema, dan kontrak RPC.
>
> Sumber: Sculley et al., NeurIPS, 2015

Penelusuran kode, bagian 1: menangkap perilaku di browser

Tracker

`track()` pertama-tama memeriksa `if (!useAuth().user) return`: pengunjung anonim tidak menghasilkan event (server juga menjawab `204` tanpa menyimpan), sehingga pengunjung baru hanya mendapat item *trending*, jalur *cold start* di Bab 12. Lalu datang kebijakan *batching*: kirim pada `FLUSH_AT = 20`, kalau tidak, mulai *timer* 2 detik. `flush()` sengaja menelan error jaringan: pelacakan bersifat *best-effort*. Bagian bawah region menangani kepergian dari halaman, ketika browser dapat membatalkan panggilan `fetch` yang sedang berjalan; `navigator.sendBeacon` mengantrekan POST kecil yang dikirim browser setelah halaman hilang. Himpunan `pageHideHooks` membawa cerita sebuah bug, di bawah.

frontend/src/composables/useTracker.ts · tracker

```ts
export interface TrackEvent {
  item_id: number
  event_type: 'impression' | 'click' | 'hide'
  position?: number
  surface?: string
  dwell_sec?: number
}

const FLUSH_MS = 2000
const FLUSH_AT = 20
let queue: TrackEvent[] = []
let timer: number | null = null

function take(): TrackEvent[] {
  const batch = queue
  queue = []
  if (timer !== null) {
    clearTimeout(timer)
    timer = null
  }
  return batch
}

async function flush() {
  const events = take()
  if (!events.length) return
  try {
    await fetch('/api/events', {
      method: 'POST',
      credentials: 'same-origin',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ events }),
    })
  } catch {
    /* tracking is best-effort */
  }
}

export function track(e: TrackEvent) {
  if (!useAuth().user) return
  queue.push(e)
  if (queue.length >= FLUSH_AT) void flush()
  else if (timer === null) timer = window.setTimeout(() => void flush(), FLUSH_MS)
}

// Views that still need to record something when the tab closes (e.g. Product.vue's
// click + dwell) register here, so their events are queued before the beacon is sent.
const pageHideHooks = new Set<() => void>()

export function onPageHide(fn: () => void): () => void {
  pageHideHooks.add(fn)
  return () => pageHideHooks.delete(fn)
}

window.addEventListener('pagehide', () => {
  for (const fn of pageHideHooks) fn()
  const events = take()
  if (!events.length) return
  navigator.sendBeacon('/api/events', new Blob([JSON.stringify({ events })], { type:
'application/json' }))
})

export function useTracker() {
  return { track, flush }
}
```

Impresi

Impresi berarti "item ini dirender untuk pengguna ini pada posisi ini di surface ini". Ia adalah penyebut dari setiap *click-through rate* di Bab 9 (`item_impressions_7d`, `user_ctr_30d`) dan bahan mentah untuk analisis bias posisi di Bab 12. Tanpa impresi yang tercatat, Anda hanya bisa mengatakan apa yang diklik orang, tidak pernah apa yang mereka abaikan.

frontend/src/components/ProductGrid.vue · impressions

```
const seen = new Set<number>() // one impression per item id per grid instance
const idsKey = computed(() => props.items.map((i) => i.id).join(','))
watch(
  idsKey,
  () => {
    props.items.forEach((item, position) => {
      if (seen.has(item.id)) return
      seen.add(item.id)
      track({ item_id: item.id, event_type: 'impression', position, surface: props.surface })
    })
  },
  { immediate: true },
)
```

Himpunan `seen` menjamin satu impresi per id item per instans grid; tanpanya, render ulang atau "Refresh" yang tumpang tindih akan menggelembungkan penyebut setiap CTR. *Watcher*-nya dikunci pada daftar id yang digabung karena Vue membuat array baru pada setiap pengambilan data. Dan `position` adalah indeks render, yang dibutuhkan Bab 12 untuk bias posisi. Catatan: sebuah item dihitung terlihat saat *dirender*, bukan saat digulir ke dalam pandangan; produksi memakai `IntersectionObserver`.

Dwell time dan bug urutan `pagehide`

Setiap event klik membawa `dwell_sec`, dan ia dibuat di halaman produk. Idenya sederhana: catat `performance.now()` saat produk dimuat, dan ketika pembeli pergi, kirim satu `click` dengan jumlah detik yang berlalu, posisi asal klik, dan surface asalnya. Dua hal terakhir itu berpindah lewat URL (`/products/:id?src=search&pos=3`), dan begitulah halaman produk tahu bagaimana ia dicapai.

frontend/src/views/Product.vue · dwell

```
let start = performance.now()
let viewItemId: number | null = null
let viewSurface = 'direct'
let viewPos = 0
let clickSent = true // guard: exactly one click per product view

function beginView() {
  start = performance.now()
  viewItemId = Number(route.params.id)
  viewSurface = String(route.query.src ?? 'direct')
  viewPos = Number(route.query.pos ?? 0)
  clickSent = false
}
function endView() {
  if (clickSent || viewItemId === null) return
  clickSent = true
  track({
    item_id: viewItemId,
    event_type: 'click',
    position: viewPos,
    surface: viewSurface,
    dwell_sec: Math.round(((performance.now() - start) / 1000) * 10) / 10,
  })
}
onBeforeRouteLeave(endView)
onBeforeRouteUpdate((to, from) => {
  if (to.params.id !== from.params.id) endView()
})
const stopPageHide = onPageHide(endView) // tab closed / full reload: queued before the beacon
flush
onUnmounted(() => {
  stopPageHide()
  endView()
})
```

`beginView()` dan `endView()` mengapit sebuah "tayangan" (*view*). Penjaga `clickSent` menjamin tepat satu klik per tayangan produk, meskipun `endView` dihubungkan ke empat jalan keluar yang berbeda:

- `onBeforeRouteLeave`: pembeli berpindah ke halaman lain di aplikasi.
- `onBeforeRouteUpdate`: pembeli berpindah dari satu produk ke produk lain, saat Vue Router *memakai ulang* instans komponen yang sama. Tanpa *hook* ini, dwell produk pertama tidak akan pernah tercatat.
- `onPageHide(endView)`: tab ditutup atau halaman dimuat ulang.
- `onUnmounted`: jaring pengaman terakhir.

Versi pertama halaman produk mendaftarkan *listener* `pagehide`-nya sendiri untuk mencatat klik. *Listener* milik tracker, yang menguras antrean ke `sendBeacon`, didaftarkan lebih dulu sehingga berjalan lebih dulu: ia mengosongkan antrean dan mengirim beacon, baru kemudian halaman produk mengantrekan kliknya. Klik itu terlewat dari flush terakhir dan hilang tanpa suara. Gejalanya halus: klik tercatat saat berpindah di dalam aplikasi, tetapi tidak pernah saat menutup tab.

Perbaikannya adalah registri *hook* di region tracker: sebuah view memanggil `onPageHide(fn)`, dan satu-satunya *listener* `pagehide` milik tracker menjalankan setiap hook (yang mengantrekan event-nya) *lalu* mengirim beacon. Pelajaran umum untuk desain *flush-on-exit* apa pun: beri flusher satu *listener* keluar dan biarkan produsen mendaftar kepadanya.

> **Tahukah kamu?**
>
> **`pagehide` bukan jaminan.** Dokumentasi MDN untuk `navigator.sendBeacon()` menjelaskan bahwa event keluar lawas (`unload`, `beforeunload`, dan `pagehide`) tidak andal, terutama di seluler: jika pengguna berpindah aplikasi lalu kemudian mematikan browser, tidak satu pun terpicu. MDN menyarankan mengirim analitik pada event `visibilitychange` sebagai gantinya, dan mencatat bahwa muatan beacon dibatasi sekitar 64 KiB. Tracker kita memakai `pagehide`, yang sederhana dan cukup baik untuk mendemonstrasikan desainnya; tracker produksi juga akan melakukan flush ketika `document.visibilityState` menjadi `hidden`.
>
> Sumber: Mozilla Contributors, MDN Web Docs, 2026

Penelusuran kode, bagian 2: server mencatatnya

Tabel events

Semua yang diketahui recommender tentang perilaku berasal dari satu tabel *append-only*, `events` di `backend/migrations/0001_init.sql`: `ts`, `user_id`, `session_id`, `item_id`, `event_type`, `position`, `dwell_sec`, `price_paid`, `surface`.

backend/migrations/0001_init.sql · events_table

```
-- One row per thing a user did to an item. Append-only: the recommender's
-- whole view of user behaviour comes from this table.
CREATE TABLE events (
    id         BIGSERIAL PRIMARY KEY,
    ts         TIMESTAMPTZ NOT NULL DEFAULT now(),
    user_id    BIGINT NOT NULL REFERENCES users,
    session_id BIGINT NOT NULL REFERENCES sessions,
    item_id    BIGINT NOT NULL REFERENCES items,
    event_type TEXT NOT NULL CHECK (event_type IN
('impression','click','add_to_cart','order','hide','return')),
    position   INT NULL,
    dwell_sec  DOUBLE PRECISION NULL,
    price_paid DOUBLE PRECISION NULL,
    surface    TEXT NULL
);
CREATE INDEX events_user_ts_idx ON events (user_id, ts DESC);
CREATE INDEX events_item_id_idx ON events (item_id);
```

Skemanya adalah kontrak antara "apa yang dapat diamati aplikasi web" dan "apa yang dapat dipelajari model".

Dibandingkan dengan skema simulator di Bab 1, data nyata menambahkan `surface` dan *foreign key* yang mencegah klien menulis event tentang item yang tidak ada. `CHECK` pada `event_type` adalah garis pertahanan terakhir, dan dua indeks melayani riwayat per pengguna dan agregat per item. Tidak ada `UPDATE` di mana pun: log bersifat *append-only*, dan sebuah pengembalian barang adalah event `return` yang baru.

Dari batch menjadi baris: `post_events`

backend/src/events.rs · post_events

```rust
async fn post_events(
    State(st): State<AppState>,
    MaybeUser(user): MaybeUser,
    Json(b): Json<EventBatch>,
) -> AppResult<StatusCode> {
    let Some(uid) = user else { return Ok(StatusCode::NO_CONTENT) };
    if b.events.len() > 100 {
        return Err(AppError::BadRequest("too many events (max 100)".into()));
    }
    if b.events.iter().any(|e| !matches!(e.event_type.as_str(), "impression" | "click" | "hide"))
{
        return Err(AppError::BadRequest("event_type not allowed from client".into()));
    }
    let Some(first) = b.events.first() else { return Ok(StatusCode::NO_CONTENT) };
    let mut tx = st.pool.begin().await?;
    let sid = touch_session(&mut tx, uid, first.surface.as_deref().unwrap_or("direct")).await?;
    for e in &b.events {
        let r = insert_event(
            &mut tx,
            uid,
            sid,
            e.item_id,
            &e.event_type,
            e.position,
            e.dwell_sec,
            None,
            e.surface.as_deref(),
        )
        .await;
        match r {
            Ok(()) => {}
            Err(sqlx::Error::Database(d)) if d.is_foreign_key_violation() => {
                return Err(AppError::BadRequest("unknown item".into()));
            }
            Err(e) => return Err(e.into()),
        }
    }
    tx.commit().await?;
    Ok(StatusCode::NO_CONTENT)
}
```

`MaybeUser` tidak pernah gagal, sehingga permintaan anonim mendapat `204`, yang memang diharapkan sebuah beacon. Pemeriksaan `matches!` menegakkan apa yang boleh diklaim browser: impresi, klik, dan *hide* adalah pengamatan atas layar pembeli sendiri. `add_to_cart` dan `order` adalah akibat dari aksi sisi server yang terautentikasi, sehingga API membuatnya sendiri; jika pembeli bisa memposting event `order`, ia bisa melatih recommender untuk menyukai item apa pun.

Satu panggilan `touch_session` menentukan sesi untuk seluruh batch; pelanggaran *foreign key* menjadi `400 unknown item`; dan `tx.commit()` membuat batch terlihat secara atomik, sehingga recommender tidak pernah melihat separuh batch.

Sesi: `touch_session`

Sesi adalah rangkaian aktivitas yang dipisahkan dari rangkaian berikutnya oleh jeda diam. Tabel `sessions` milik simulator memiliki struktur itu sejak awal; aplikasi nyata harus *menurunkannya*.

backend/src/sessions.rs · touch_session

```rust
/// Returns the id of the user's current session, creating one if the latest
/// session has been idle for more than 30 minutes.
pub async fn touch_session(conn: &mut PgConnection, user_id: i64, surface: &str) -> Result<i64,
sqlx::Error> {
    let existing: Option<(i64,)> = sqlx::query_as(
        "UPDATE sessions SET last_seen_at = now() WHERE id = (
             SELECT id FROM sessions
             WHERE user_id = $1 AND last_seen_at > now() - interval '30 minutes'
             ORDER BY last_seen_at DESC LIMIT 1)
         RETURNING id",
    )
    .bind(user_id)
    .fetch_optional(&mut *conn)
    .await?;
    if let Some((id,)) = existing {
        return Ok(id);
    }
    let (id,): (i64,) = sqlx::query_as(
        "INSERT INTO sessions (user_id, surface, device_os) VALUES ($1, $2, 'web') RETURNING id",
    )
    .bind(user_id)
    .bind(surface)
    .fetch_one(&mut *conn)
    .await?;
    Ok(id)
}
```

Satu `UPDATE ... WHERE id = (SELECT ... LIMIT 1) RETURNING id` menemukan sesi terbaru yang terlihat dalam 30 menit, memperbarui `last_seen_at`, dan mengembalikan id-nya; kalau tidak, sesi baru dimasukkan. Dua permintaan konkuren secara teori bisa sama-sama memasukkan satu sesi; kita menerimanya, karena itu hanya sedikit mengencerkan co-visitation. Setiap event sisi server (batch klien, `add_to_cart`, `order`) melewati fungsi ini, karena matriks co-visitation bergantung pada `session_id` yang valid.

Checkout: sinyal terkuat

Ini contoh terbaik mengapa satu penulis itu menguntungkan. Dalam satu transaksi, handler memuat keranjang, mengunci baris item dengan `SELECT ... FOR UPDATE ORDER BY id` (urutan deterministik mencegah *deadlock*), menolak baris yang tidak aktif atau stoknya kurang dengan `409`, mengelompokkan baris per toko menjadi satu pesanan per penjual, menyimpan `unit_price` pada harga akhir setelah diskon, mengurangi stok dan menyetel `out_of_stock_at` saat mencapai nol, menulis satu event `order` per baris lewat `touch_session`, lalu mengosongkan keranjang. Dua konsekuensi yang relevan bagi recommender: `Recommend` berikutnya berhenti mengusulkan produk yang sudah habis karena `History.at` membaca `out_of_stock_at`, dan setiap event `order` bersesuaian dengan baris pesanan yang nyata dan sudah dibayar.

backend/src/cart.rs · checkout

```
async fn checkout(
    State(st): State<AppState>,
    u: AuthUser,
    Json(b): Json<CheckoutBody>,
) -> AppResult<(StatusCode, Json<serde_json::Value>)> {
    let address = b.shipping_address.trim();
    if address.is_empty() {
        return Err(AppError::BadRequest("shipping address is required".into()));
    }
    let mut tx = st.pool.begin().await?;
    let cart: Vec<(i64, i32)> = sqlx::query_as("SELECT item_id, quantity FROM cart_items WHERE
user_id = $1")
        .bind(u.id)
        .fetch_all(&mut *tx)
        .await?;
    if cart.is_empty() {
        return Err(AppError::BadRequest("cart is empty".into()));
    }
    let ids: Vec<i64> = cart.iter().map(|(i, _)| *i).collect();
    // Row locks serialise concurrent checkouts of the same items.
    let locked: HashMap<i64, LockedItem> = sqlx::query_as::<_, LockedItem>(
        "SELECT id, shop_id, title, ROUND(price * (1 - discount_pct))::bigint AS final_price,
                free_shipping, stock, is_active
         FROM items WHERE id = ANY($1) ORDER BY id FOR UPDATE",
    )
    .bind(&ids)
    .fetch_all(&mut *tx)
    .await?
    .into_iter()
    .map(|i| (i.id, i))
    .collect();

    let mut by_shop: BTreeMap<i64, Vec<(&LockedItem, i32)>> = BTreeMap::new();
    for (item_id, qty) in &cart {
        let it = locked.get(item_id).ok_or(AppError::NotFound)?;
        if !it.is_active || *qty > it.stock {
            return Err(AppError::Conflict(format!("insufficient stock for {}", it.title)));
        }
        by_shop.entry(it.shop_id).or_default().push((it, *qty));
    }

    let sid = touch_session(&mut tx, u.id, "checkout").await?;
    let mut placed = Vec::new();
    for (shop_id, lines) in &by_shop {
        let subtotal: i64 = lines.iter().map(|(it, q)| it.final_price * *q as i64).sum();
        let shipping_fee = shipping_total(lines.iter().map(|(it, _)| (it.shop_id,
it.free_shipping)));
        let total = subtotal + shipping_fee;
        let (order_id,): (i64,) = sqlx::query_as(
            "INSERT INTO orders (buyer_id, shop_id, status, shipping_address, subtotal,
shipping_fee, total)
             VALUES ($1, $2, 'paid', $3, $4, $5, $6) RETURNING id",
        )
        .bind(u.id)
        .bind(shop_id)
        .bind(address)
        .bind(subtotal)
        .bind(shipping_fee)
        .bind(total)
        .fetch_one(&mut *tx)
        .await?;
        for (it, qty) in lines {
            sqlx::query("INSERT INTO order_items (order_id, item_id, quantity, unit_price) VALUES
($1, $2, $3, $4)")
                .bind(order_id)
                .bind(it.id)
                .bind(qty)
                .bind(it.final_price)
                .execute(&mut *tx)
```

```
                .await?;
            sqlx::query(
                "UPDATE items SET stock = stock - $2,
                        out_of_stock_at = CASE WHEN stock - $2 = 0 THEN now() ELSE out_of_stock_at
END
                 WHERE id = $1",
            )
            .bind(it.id)
            .bind(qty)
            .execute(&mut *tx)
            .await?;
            insert_event(
                &mut tx,
                u.id,
                sid,
                it.id,
                "order",
                None,
                None,
                Some((it.final_price * *qty as i64) as f64),
                Some("checkout"),
            )
            .await?;
        }
        placed.push(PlacedOrder { id: order_id, shop_id: *shop_id, total });
    }
    sqlx::query("DELETE FROM cart_items WHERE user_id = $1").bind(u.id).execute(&mut *tx).await?;
    tx.commit().await?;
    Ok((StatusCode::CREATED, Json(serde_json::json!({ "orders": placed }))))
}
```

Penelusuran kode, bagian 3: recommender membacanya kembali

Menjaga salinan di memori tetap segar: `Store`

Recommender membutuhkan seluruh riwayat event sebagai DataFrame karena pipeline dari bab-bab sebelumnya (`History.at`, `candidates.generate`, `features.build`) ditulis untuk tabel pandas. Membaca ulang 277,000 baris dari Postgres pada setiap permintaan akan lambat. Maka `Store` menyimpan tabel di memori dan menyegarkannya secara inkremental.

recommender/recsys/db.py · Store.refresh

```
def refresh(self) -> None:
    with self._lock:
        if self._tables and time.monotonic() - self._last_refresh < MIN_REFRESH_SEC:
            return
        try:
            self._refresh()
        except psycopg.OperationalError:
            self._conn = None  # reconnect once (e.g. Postgres restarted)
            self._refresh()
        self._last_refresh = time.monotonic()
```

`refresh()` melakukan tiga hal. Ia mengambil kunci (*lock*), sehingga *thread* gRPC yang konkuren tidak mengeluarkan refresh yang tumpang tindih. Ia menerapkan *throttle*: jika refresh terakhir kurang dari `MIN_REFRESH_SEC` (satu detik) yang lalu, ia langsung kembali, sehingga beban database dibatasi sekitar satu refresh per detik berapa pun jumlah permintaan yang datang. Dan ia mencoba ulang satu kali saat `OperationalError` setelah melepas koneksi, yang memang Anda inginkan setelah Postgres dimulai ulang.

Empat tabel kecil (categories, shops, items, users) dimuat ulang seluruhnya karena berubah di tempat. Events dan searches diambil dengan `WHERE id > last_id - 1000`. *Overlap* itu penting karena id `BIGSERIAL` ditetapkan saat *insert* tetapi baris baru terlihat saat *commit*: jika transaksi Y meng-*commit* event 501 sebelum X meng-*commit* event 500, query naif `id > 501` tidak akan pernah mengambil 500. Membaca ulang 1000 id terakhir dan menghapus duplikat di `_merge` (yang juga mengurutkan berdasarkan `ts`, seperti yang diasumsikan `History.at`) menoleransi *commit* yang tidak berurutan. Semua pembacaan memakai koneksi read-only.

RPC `Recommend`

recommender/recsys/service.py · recommend_rpc

```
def Recommend(self, request, context):
    t = time.perf_counter()
    self.store.refresh()  # pick up events logged since the last request
    k = min(max(request.k, 1), 50)
    model, version = self.model, self.meta["version"]
    recs, n_candidates = pipeline.recommend(self.store.snapshot(), _now(), model, request.user_id,
k)
    sources = pipeline.source_of(recs) if len(recs) else []
    contribs = ranker.contributions(model, recs) if len(recs) else []
    items = [
        pb.RecommendedItem(
            item_id=int(item_id), score=float(score), source=source, rank=rank,
            contributions=[pb.Contribution(feature=f, value=v, contribution=c) for f, v, c in
contrib],
        )  # fmt: skip
        for rank, (item_id, score, source, contrib) in enumerate(
            zip(recs["item_id"], recs["score"], sources, contribs), start=1
        )
    ]
    latency_ms = (time.perf_counter() - t) * 1000
    log.info("recommend user=%d k=%d candidates=%d latency_ms=%.0f", request.user_id, k,
n_candidates, latency_ms)
    return pb.RecommendResponse(
        items=items, model_version=version, candidates_considered=n_candidates,
latency_ms=latency_ms
    )
```

Metode ini menyegarkan store, membatasi `k` ke 1..50, dan mengambil referensi ke model dan versi saat ini *sebelum* pekerjaan apa pun, yang membuat *hot-swap* aman. Lalu ia memanggil `pipeline.recommend`, fungsi yang sama dengan yang dipakai CLI offline: evaluasi offline dan *serving* online berbagi satu jalur kode, pertahanan terbaik terhadap *training/serving skew*. `source_of` melabeli setiap baris dengan retriever terkuatnya (atau `trending`), `ranker.contributions` menambahkan lima nilai SHAP teratas, dan satu baris log INFO mencatat pengguna, `k`, kandidat, dan latensi. Pengguna anonim (`user_id = 0`) tidak butuh cabang khusus: tanpa riwayat berarti kandidat *trending* dan fitur pengguna NaN, yang bisa ditangani LightGBM.

Pelatihan ulang tanpa *downtime*

Input ranker bergeser seiring item bermunculan dan selera berubah, sehingga layanan melatih ulang dengan dua pemicu: *thread* latar belakang setiap `RECSYS_RETRAIN_INTERVAL_SEC` (enam jam secara bawaan) dan panggilan gRPC `Retrain` khusus admin di balik tombol pada halaman "How it works".

recommender/recsys/service.py · retrain

```
def retrain(store: Store, model_dir: Path = MODEL_DIR) -> tuple[lgb.Booster, dict, dict]:
    """Train + evaluate on the current DB snapshot, save ranker.txt + model_meta.json.

    Returns (model, meta, report) where meta is what's written to model_meta.json.
    """
    trained_at = datetime.now(timezone.utc).replace(microsecond=0)
    model, report = pipeline.train_and_evaluate(store.snapshot(), end=_now())
    metrics = {
        pipeline.metric_key(method, metric): float(v)
        for method, row in report["results"].iterrows()
        for metric, v in row.items()
    }
    metrics["candidate_recall"] = float(report["candidate_recall"])
    meta = {
        "version": trained_at.strftime("%Y%m%d-%H%M%S"),
        "trained_at": trained_at.isoformat().replace("+00:00", "Z"),
        "metrics": metrics,
        "importance": {k: float(v) for k, v in report["importance"].items()},
        "train_rows": int(report["train_rows"]),
    }
    model_dir.mkdir(parents=True, exist_ok=True)
    ranker.save(model, model_dir / "ranker.txt")
    (model_dir / "model_meta.json").write_text(json.dumps(meta, indent=2))
    return model, meta, report
```

`retrain` tingkat modul dipakai bersama oleh layanan dan CLI. Ia melatih pada *snapshot* store dengan `end=now`, sehingga jendela label bergeser mengikuti jam, mengubah tabel evaluasi menjadi kunci datar seperti `lightgbm_ranker/ndcg@20`, dan menulis `ranker.txt` serta `model_meta.json`.

Metodenya menambahkan dua pengaman. `retrain_lock.acquire(blocking=False)` membuat permintaan konkuren kedua gagal seketika (`ABORTED`, yang oleh Rust dipetakan ke `409`). Dan pertukaran `self.model, self.meta = model, meta` adalah dua penugasan referensi: panggilan `Recommend` yang sedang berjalan tetap memakai *booster* lama dan selesai secara konsisten, panggilan baru melihat yang baru, tanpa *lock* pada jalur baca.

Penelusuran kode, bagian 4: sisi Rust dari jembatannya

Rute rekomendasi

backend/src/recommender.rs · recommend_route

```
async fn recommendations(
    State(st): State<AppState>,
    MaybeUser(user): MaybeUser,
    Query(q): Query<RecsQuery>,
) -> AppResult<Json<Value>> {
    let k = q.k.unwrap_or(20).clamp(1, 50);
    let req = pb::RecommendRequest {
        user_id: user.unwrap_or(0),
        k: k as i32,
        surface: q.surface.unwrap_or_else(|| "home_feed".into()),
    };
    let mut rec = st.rec.clone();
    let res = tokio::time::timeout(Duration::from_secs(5), rec.recommend(req)).await;
    match res {
        Ok(Ok(resp)) => {
            let r = resp.into_inner();
            let items = hydrate(&st.pool, r.items).await?;
            Ok(Json(json!({
                "items": items,
                "model_version": r.model_version,
                "latency_ms": r.latency_ms,
                "candidates_considered": r.candidates_considered,
                "fallback": false,
            })))
        }
        other => {
            match other {
                Ok(Err(e)) => tracing::warn!("recommender error, using fallback: {e}"),
                _ => tracing::warn!("recommender timed out, using fallback"),
            }
            let items = fallback(&st.pool, k, None, None).await?;
            Ok(Json(json!({
                "items": items,
                "model_version": "fallback",
                "latency_ms": 0,
                "candidates_considered": 0,
                "fallback": true,
            })))
        }
    }
}
```

Handler membaca pengguna opsional, membatasi `k`, dan membangun permintaan. Kanal `connect_lazy()` di `main.rs` membuat API dapat start meskipun Python mati. Panggilan dibungkus dalam `tokio::time::timeout` 5 detik; jika berhasil, hasilnya dihidrasi dan dikembalikan dengan `"fallback": false`, sedangkan error gRPC atau *timeout* apa pun dicatat di log dan dijawab dari query fallback dengan `"model_version": "fallback"`, yang diubah frontend menjadi banner kuning. Menurun lebih baik daripada mati.

Hidrasi

gRPC mengembalikan id dan skor; browser membutuhkan judul dan harga.

backend/src/recommender.rs · hydrate

```rust
/// Turn gRPC items (ids only) into full cards, in the recommender's order.
/// Ids that no longer exist or are inactive are skipped.
async fn hydrate(pool: &PgPool, items: Vec<pb::RecommendedItem>) -> Result<Vec<RecCard>,
sqlx::Error> {
    let ids: Vec<i64> = items.iter().map(|i| i.item_id).collect();
    let mut cards: HashMap<i64, ProductCard> = fetch_cards(pool, &ids).await?.into_iter().map(|c|
(c.id, c)).collect();
    Ok(items
        .into_iter()
        .filter_map(|it| {
            cards.remove(&it.item_id).map(|card| RecCard {
                card,
                score: it.score,
                source: it.source,
                rank: it.rank,
                contributions: it
                    .contributions
                    .into_iter()
                    .map(|c| ContributionJson { feature: c.feature, value: c.value, contribution:
c.contribution })
                    .collect(),
            })
        })
        .collect())
}
```

Satu query SQL dengan `id = ANY($1)` membangun peta id-ke-kartu, dan kode menelusuri daftar recommender *menurut urutan recommender*, melewati id yang sementara itu hilang. Kartu yang kurang dari `k` lebih baik daripada menggagalkan seluruh feed.

Fallback

backend/src/recommender.rs · fallback

```rust
/// Used when the recommender is down: recent best-sellers, padded by rating count.
async fn fallback(
    pool: &PgPool,
    k: i64,
    category_id: Option<i64>,
    exclude_id: Option<i64>,
) -> Result<Vec<RecCard>, sqlx::Error> {
    let mut qb = QueryBuilder::new(format!("SELECT {CARD_COLS} {CARD_FROM} WHERE i.is_active AND
i.stock > 0"));
    if let Some(c) = category_id {
        qb.push(" AND i.category_id = ").push_bind(c);
    }
    if let Some(x) = exclude_id {
        qb.push(" AND i.id <> ").push_bind(x);
    }
    qb.push(
        " ORDER BY (SELECT count(*) FROM events e WHERE e.item_id = i.id AND e.event_type =
'order'
                    AND e.ts > now() - interval '7 days') DESC, i.rating_count DESC, i.id LIMIT
",
    )
    .push_bind(k);
    let cards = qb.build_query_as::<ProductCard>().fetch_all(pool).await?;
    Ok(cards
        .into_iter()
        .enumerate()
        .map(|(i, card)| RecCard {
            card,
            score: 0.0,
            source: "trending".into(),
            rank: i as i32 + 1,
            contributions: vec![],
        })
        .collect())
}
```

Kartu cetak dari restoran kita: item aktif yang stoknya ada, diurutkan menurut event `order` dalam tujuh hari terakhir, lalu jumlah rating. Ia melabeli setiap baris `source: "trending"`, skor 0, tanpa contributions, sehingga frontend tidak membutuhkan jalur terpisah. Ia sengaja tidak dipersonalisasi: pengalaman menurun yang sama untuk semua orang tidak bisa salah dengan cara yang mengejutkan dan spesifik pengguna.

Kontrak antarbahasa: satu `.proto`, dua generator kode

API Rust dan layanan Python hanya berbagi sebuah skema, satu berkas pendek:

proto/recommender.proto

```
syntax = "proto3";
package recommender.v1;

service Recommender {
  // Personalised home feed. user_id = 0 means anonymous (gets trending-driven results).
  rpc Recommend(RecommendRequest) returns (RecommendResponse);
  // "Similar items" on a product page: co-visitation, then same-category popular.
  rpc SimilarItems(SimilarItemsRequest) returns (RecommendResponse);
  // Retrain the ranker on the current DB contents and hot-swap it.
  rpc Retrain(RetrainRequest) returns (RetrainResponse);
  // Model version, metrics, feature importance + feature descriptions (for the learning page).
  rpc ModelInfo(ModelInfoRequest) returns (ModelInfoResponse);
}

message RecommendRequest { int64 user_id = 1; int32 k = 2; string surface = 3; }
message SimilarItemsRequest { int64 item_id = 1; int32 k = 2; int64 user_id = 3; }

message Contribution {
  string feature = 1;       // schema feature name, e.g. "uc_click_share_30d"
  double value = 2;         // the feature's value for this (user, item)
  double contribution = 3;  // LightGBM SHAP contribution to the score (+ pushes up)
}
message RecommendedItem {
  int64 item_id = 1;
  double score = 2;
  string source = 3;  // "recent" | "covisit" | "category_popular" | "trending" | "same_category"
  int32 rank = 4;     // 1-based
  repeated Contribution contributions = 5;  // top 5 by |contribution|; empty for SimilarItems
}
message RecommendResponse {
  repeated RecommendedItem items = 1;
  string model_version = 2;
  int64 candidates_considered = 3;
  double latency_ms = 4;
}

message RetrainRequest {}
message RetrainResponse { string model_version = 1; map<string, double> metrics = 2; int64
train_rows = 3; }

message ModelInfoRequest {}
message FeatureInfo { string name = 1; string group = 2; string description = 3; double gain_share
= 4; }
message ModelInfoResponse {
  string model_version = 1;
  string trained_at = 2;               // ISO-8601 UTC
  map<string, double> metrics = 3;     // keys "<method>/<metric>", e.g.
"lightgbm_ranker/ndcg@20", plus "candidate_recall"
  repeated FeatureInfo features = 4;   // all schema.FEATURES, sorted by gain_share desc
  int64 events_loaded = 5;
  int64 train_rows = 6;
}
```

Bagian-bagiannya terpetakan langsung ke produk:

- `Recommend` dan `SimilarItems` mengembalikan `RecommendResponse` dengan `items`, `model_version`, `candidates_considered`, dan `latency_ms`, angka-angka pada keterangan abu-abu di bawah "Recommended for you".
- `RecommendedItem.contributions` membawa lima kontribusi SHAP teratas per item: bahan mentah "Why this?".
- `Retrain` dan `ModelInfo` menyuplai halaman "How it works".
- Yang melintasi batas adalah ID, bukan objek, dan respons membawa versi model sehingga perilaku apa pun dapat dikaitkan dengan model penyebabnya.

> **Tahukah kamu?**
>
> **Nomor field adalah format *wire*.** Dalam Protocol Buffers, setiap field diidentifikasi di *wire* oleh nomornya, bukan namanya. Panduan bahasanya menyatakan bahwa nomor tidak boleh diubah setelah tipe pesan dipakai, bahwa nomor field 1 sampai 15 membutuhkan satu byte untuk di-*encode* sedangkan 16 sampai 2047 membutuhkan dua, dan bahwa nomor field yang dihapus sebaiknya dicadangkan (*reserved*) agar tidak pernah dipakai ulang. Di `recommender.proto`, field yang paling sering terisi seperti `item_id` dan `score` karena itu berada di nomor rendah.
>
> Sumber: Google, Protocol Buffers Language Guide (proto 3), 2026

Menghasilkan kode di kedua sisi

Rust menghasilkan kliennya pada saat *build*:

backend/build.rs

```rust
fn main() -> Result<(), Box<dyn std::error::Error>> {
    println!("cargo:rerun-if-changed=../proto/recommender.proto");
    tonic_prost_build::configure()
        .build_server(false)
        .compile_protos(&["../proto/recommender.proto"], &["../proto"])?;
    Ok(())
}
```

`build_server(false)` hanya menghasilkan separuh klien, `rerun-if-changed` membangun ulang saat proto berubah, dan `tonic::include_proto!("recommender.v1")` menarik item bertipe seperti `pb::RecommendRequest`. Salah ketik nama field adalah error kompilasi, bukan kejutan saat runtime.

Python menghasilkan stub-nya dengan satu perintah, dibungkus sebagai subperintah CLI agar tidak ada yang perlu mengingat flag-nya:

```
cd recommender && uv run recsys gen-proto
```

`grpc_tools.protoc` dengan `-I recsys/pb=../proto` membuat impor hasil generate terbaca `from recsys.pb import recommender_pb2`. Stub-nya di-*commit*, jadi menjalankan layanan tidak membutuhkan langkah codegen; harganya adalah harus ingat meregenerasi setelah mengubah proto, kompromi yang wajar untuk repositori pengajaran.

Jalankan

Skrip `ch13_latency.py` memanggil layanan gRPC yang *sedang berjalan* secara langsung (melewati Rust dan browser) dengan `Recommend`, `k = 20`, untuk 200 pengguna acak dengan *seed* tetap; panggilan pertama adalah pemanasan dan tidak dihitung.

```
# terminal A
cd recommender && uv run recsys serve
# terminal B (dari root repo)
cd book && uv run python code/ch13_latency.py
```

KELUARAN TERMINAL

```
=== Recommend(k=20) for 200 random users at 127.0.0.1:50051 ===
model_version=20261102-060535 (first call is a warm-up and not counted)
                            p50      p90      p99     mean      max
server latency_ms         839.9    892.5    985.0    840.1   1010.6
client wall ms            840.9    893.5    986.1    841.1   1011.8

candidates scored per request: mean 102, min 30, max 129
items returned per request:   mean 20.0
```

"server latency_ms" adalah pengukuran milik layanan itu sendiri; "client wall ms" menambahkan serialisasi dan lompatan *loopback*. Selisihnya sekitar satu milidetik, sehingga pada dasarnya seluruh waktu berada di dalam recommender. Setiap permintaan menilai antara 30 dan 129 kandidat dan mengembalikan tepat 20 item.

Ke mana 840 ms itu pergi? Skrip kedua, `ch13_profile.py`, berjalan offline pada data simulasi melalui jalur `pipeline.recommend` yang persis sama, mengukur waktu setiap tahap untuk tiga pengguna dengan riwayat yang sangat berbeda (median dari 7 kali jalan). Ia tidak membutuhkan layanan yang berjalan:

```
cd book && uv run python tools/run_all.py --only ch13_profile
```

KELUARAN TERMINAL

```
=== Stage timings for one request (median of 7 runs, milliseconds) ===
catalogue: 2,500 items, 276,378 events, cutoff = END
user    inter.  cands  History.at  generate  features   score  rerank   shap   total
light        4    110           9       213       437       1       2      6     668
median      22    112           8       215       443       1       2      5     674
heavy      133     95           8       213       443       1       2      6     674

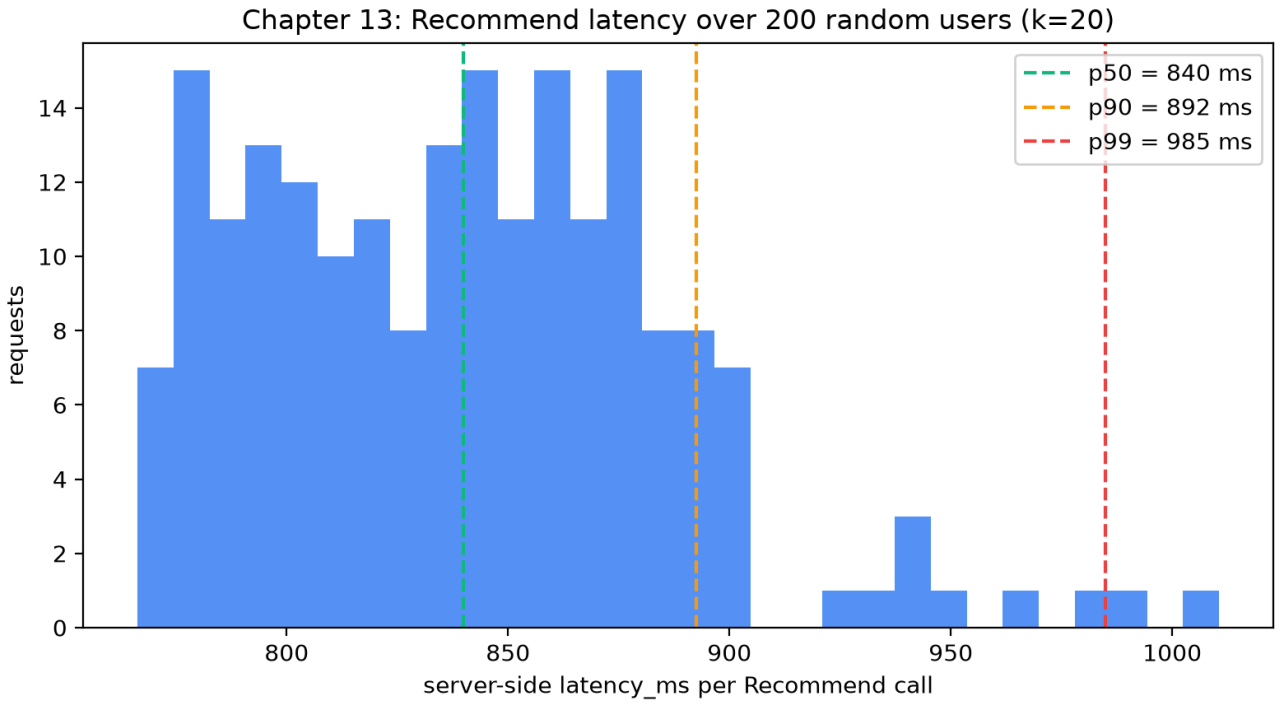
=== Inside features.build: every table is aggregated for ALL users/items, then joined ===
item_features              122 ms   item-level (same for every user)
user_features               57 ms   all users' events
user_item_features         152 ms   all users' events
user_category_features      57 ms   all users' events
user_shop_features          26 ms   all users' events
user_price_level            11 ms   all users' events
covisitation_matrix         23 ms   item-level (inside generate)

generate + features.build = 97% of the request on average.
Item-level work (item_features + covisitation_matrix): 144 ms;
user-side aggregations computed over every user, though one user asked: 303 ms.
Caching item-level frames saves the first number; filtering events to the requesting
user before the user-side aggregations is needed to remove the second.
```

Tabel per pengguna menunjukkan total 668 sampai 674 ms untuk pengguna dengan 4, 22, dan 133 interaksi. `History.at` memakan 8 sampai 9 ms, `generate` 213 sampai 215 ms, dan `features` 437 sampai 443 ms; `score`, `rerank`, dan `shap` bersama-sama hanya beberapa milidetik. Bagian kedua membedah `features.build`. Ada dua jenis pekerjaan di dalamnya. Pekerjaan tingkat item, yang sama untuk setiap pengguna, adalah `item_features` (122 ms) ditambah `covisitation_matrix` (23 ms, di dalam `generate`): total 144 ms. Bagian yang lebih besar adalah agregasi sisi pengguna: `user_features` (57 ms), `user_item_features` (152 ms), `user_category_features` (57 ms), `user_shop_features` (26 ms), dan `user_price_level` (11 ms) masing-masing mengagregasi event *semua pengguna*, padahal hanya satu pengguna yang meminta, dan baru sesudahnya barisnya digabungkan ke kandidat pengguna itu. Jumlahnya 303 ms. Modelnya adalah bagian yang murah.

Hasil

Latensi



Gambar 13.3: *Histogram latensi Recommend di sisi server untuk 200 pengguna acak, dengan p50, p90, dan p99 ditandai*

Histogram menunjukkan badan yang rapat antara sekitar 770 dan 900 ms, serta ekor pendek yang mencapai sekitar 1,010 ms. Mediannya 839.9 ms, persentil ke-90 892.5 ms, dan ke-99 985.0 ms. Ada dua pengamatan penting.

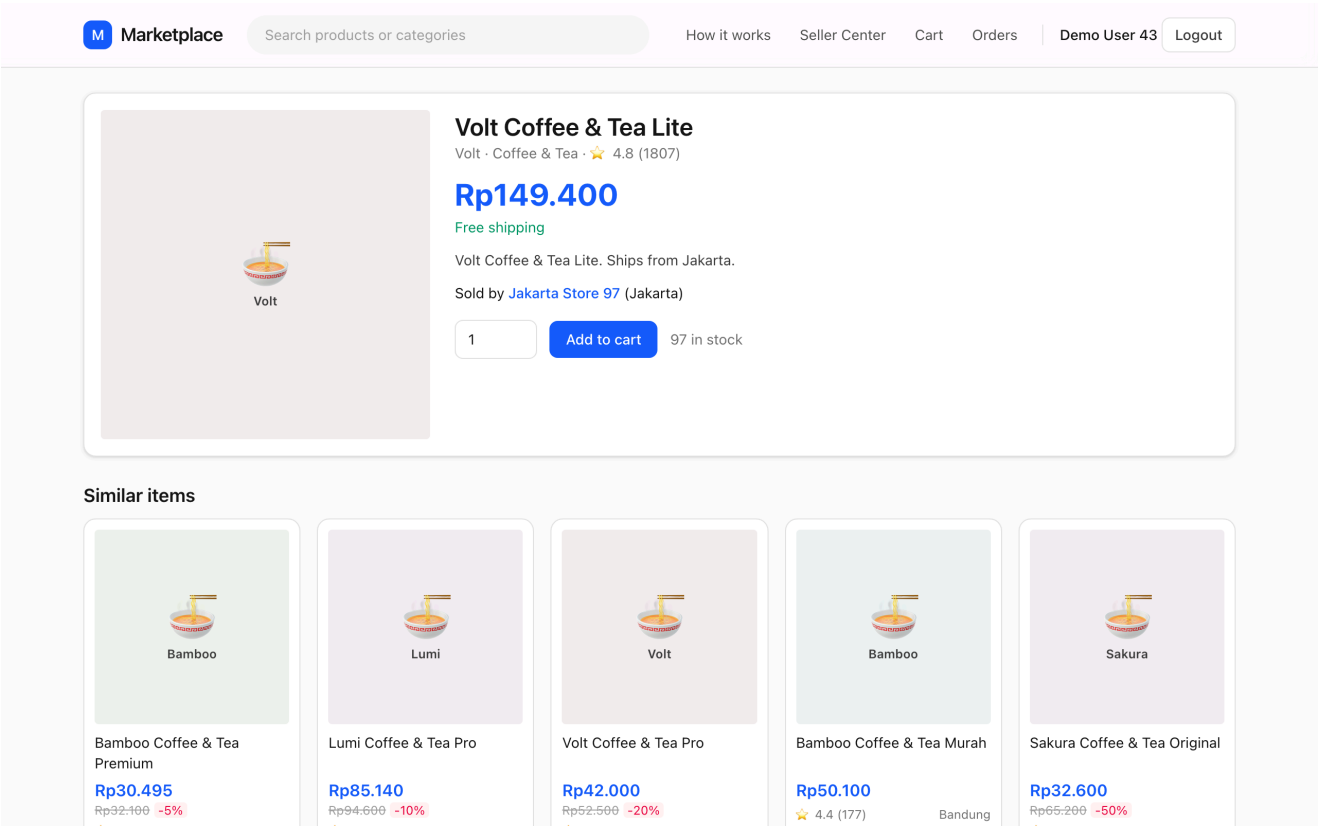
Pertama, **latensi nyaris tidak bergantung pada pengguna**. Profil menjelaskannya: `generate` plus `features.build` memakan rata-rata 97% dari permintaan, jumlah kandidat 95 sampai 112 apa pun riwayatnya, dan pekerjaan di dalamnya dihitung ulang pada setiap panggilan: *frame* tingkat item dan, yang lebih mahal, agregasi sisi pengguna atas event seluruh pengguna. Penilaian, *re-ranking*, dan SHAP bersama-sama memakan sekitar sepuluh milidetik.

Kedua, **0.84 dtk cukup untuk demo dan terlalu lambat untuk beranda sungguhan**. Bahkan panggilan teramati yang paling lambat (1,010.6 ms) hanya memakai sekitar seperlima dari anggaran 5 detik, tetapi pembeli akan merasakannya. Profil menunjukkan bahwa perbaikannya punya dua bagian. Meng-*cache frame* tingkat item (matriks co-visitation dan fitur item) selama sekitar 30 detik akan menghilangkan 144 ms pekerjaan tingkat item; statistik item lalu tertinggal hingga 30 dtk sementara klik *milik pembeli sendiri* tetap langsung. Namun itu saja menyisakan 303 ms yang lebih besar: agregasi sisi pengguna juga harus dibatasi pada event pengguna yang meminta *sebelum* diagregasi, bukan mengagregasi semua pengguna lalu menggabungkan. *Cache* saja tidak memperbaiki latensi.

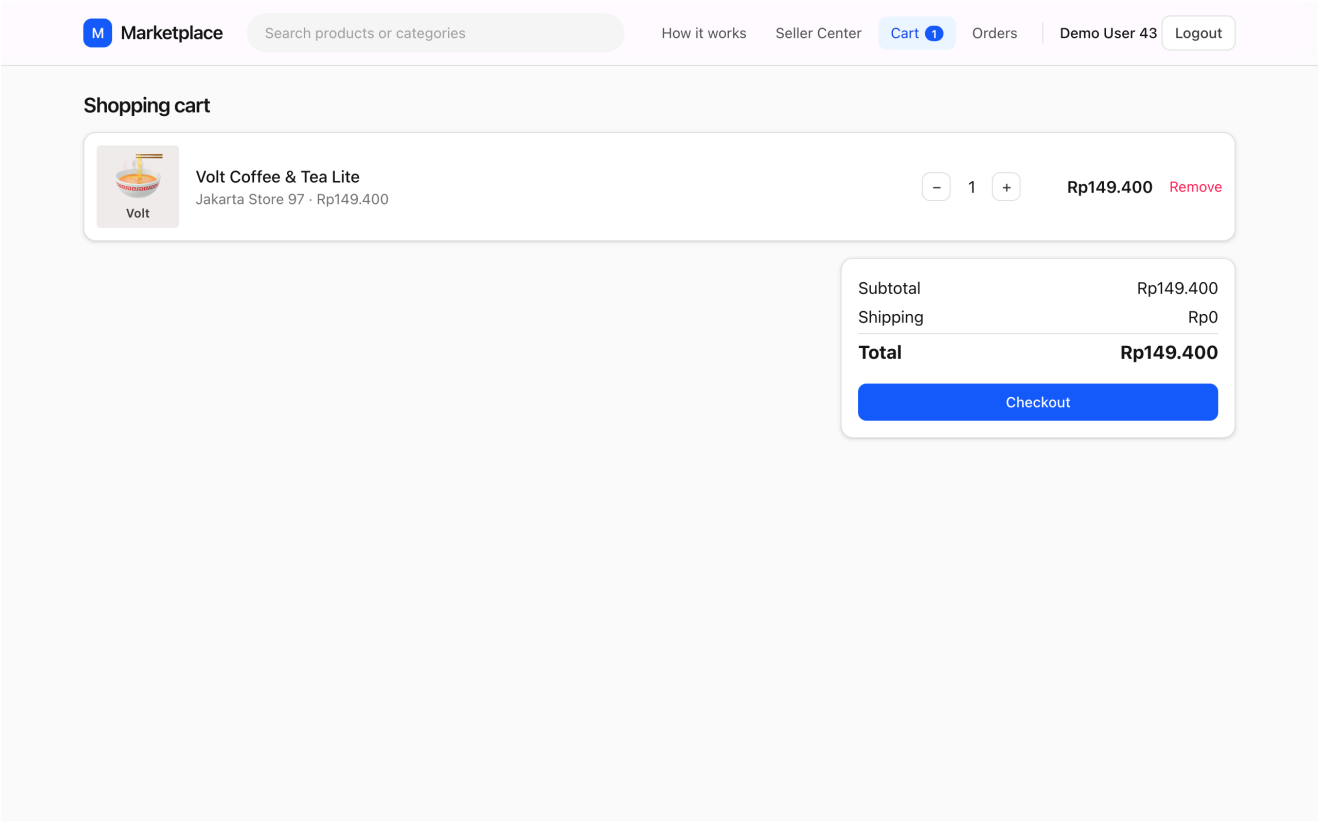
Tur screenshot

Screenshot berasal dari aplikasi yang berjalan, diambil dengan Playwright. Halaman lainnya mengikuti pola yang sama: login menyetel JWT dalam cookie `httpOnly`; pencarian adalah SQL `ILIKE` biasa yang

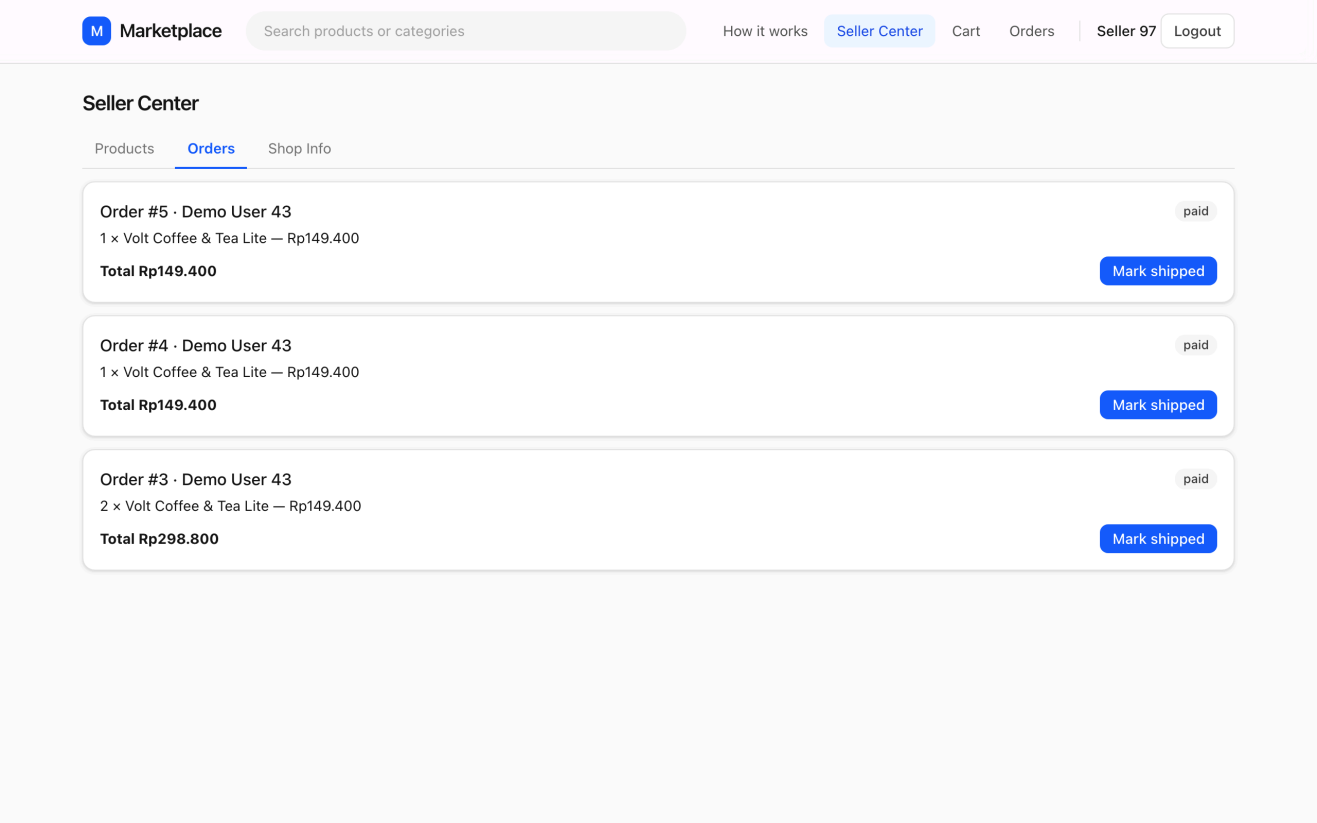
menulis baris `searches`; halaman produk memuat `/products/:id/similar` secara terpisah, sehingga recommender yang gagal tidak pernah memblokirnya; keranjang, checkout, dan pesanan menjalankan sisi penulis; dan produk yang baru ditayangkan di Seller Center adalah kasus *cold start* dari Bab 12.



Gambar 13.4: *Halaman produk dengan tombol Add to cart dan grid item serupa*

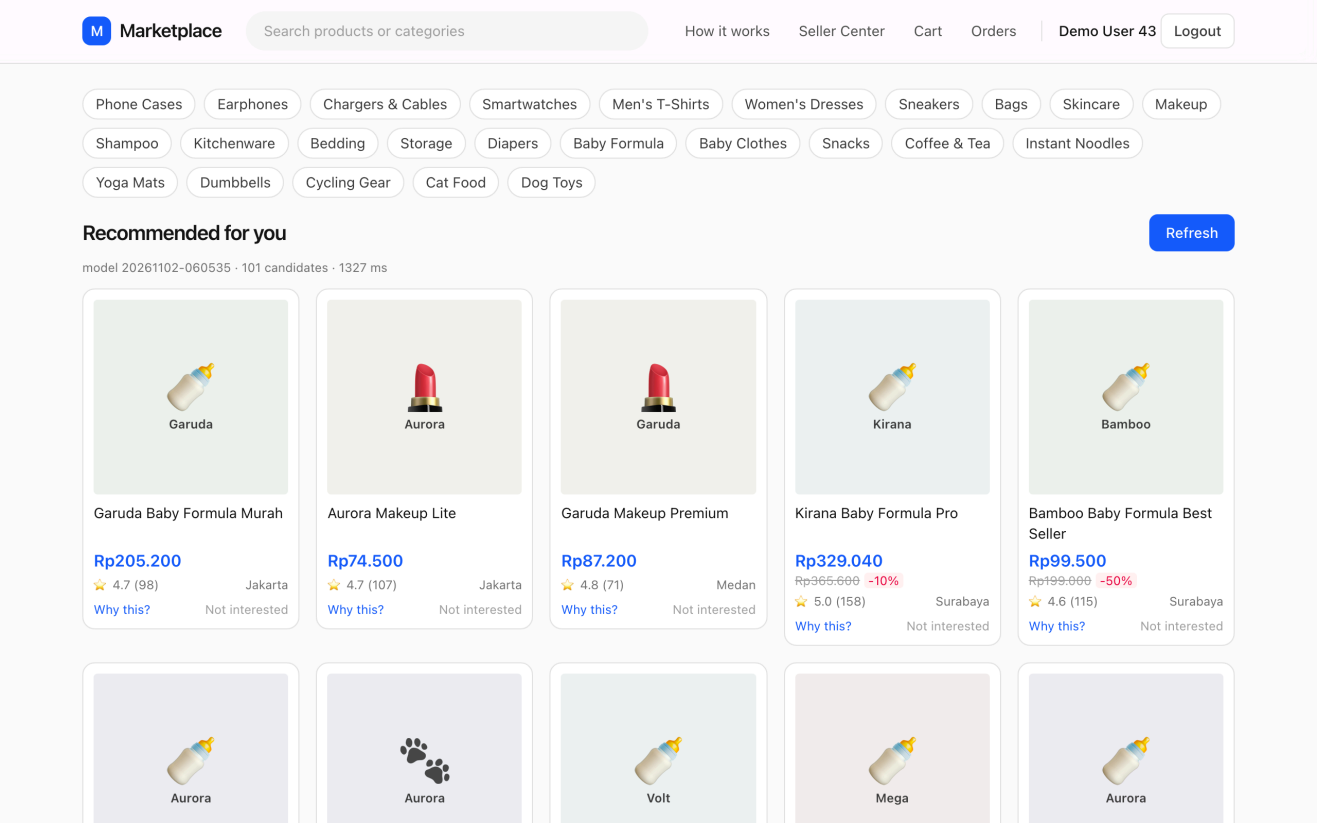


Gambar 13.5: *Keranjang belanja dengan satu baris, subtotal, ongkos kirim, dan tombol Checkout*



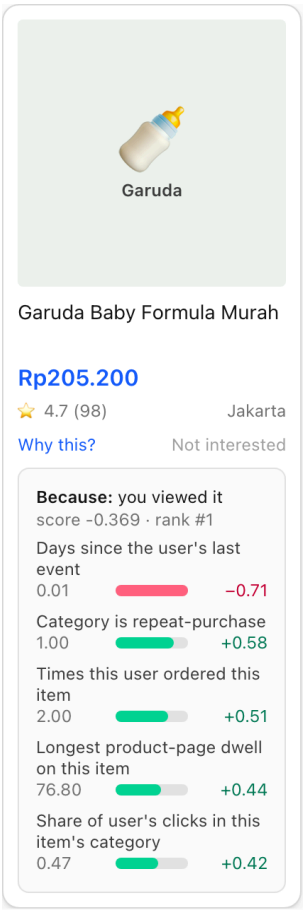
Gambar 13.6: *Seller Center, tab Orders: pesanan yang sudah dibayar dengan tombol Mark shipped*

Beranda yang dipersonalisasi. Setelah login sebagai `user43`, "Recommended for you" menampilkan keterangan berisi versi model, jumlah kandidat, dan latensi, dan setiap kartu memiliki "Why this?" dan "Not interested" (yang mencatat event `hide`). Pengunjung anonim mendapat "Popular now" dari `user_id = 0`, tanpa pelacakan.



Gambar 13.7: *Beranda untuk user43: feed yang dipersonalisasi dengan Why this? dan Not interested pada setiap kartu*

Why this? Panel menampilkan "Because: you viewed it", skor dan peringkat, serta lima fitur dengan kontribusi bertandanya. Namanya berasal dari `schema.FEATURES` melalui `ModelInfo`; batangnya adalah nilai SHAP yang dihitung di dalam `recommend_rpc`. Jika sebuah kartu muncul karena alasan yang tidak masuk akal, panel itu memberi tahu fitur mana yang perlu diselidiki.



Gambar 13.8: *Panel Why this? pada rekomendasi pertama*

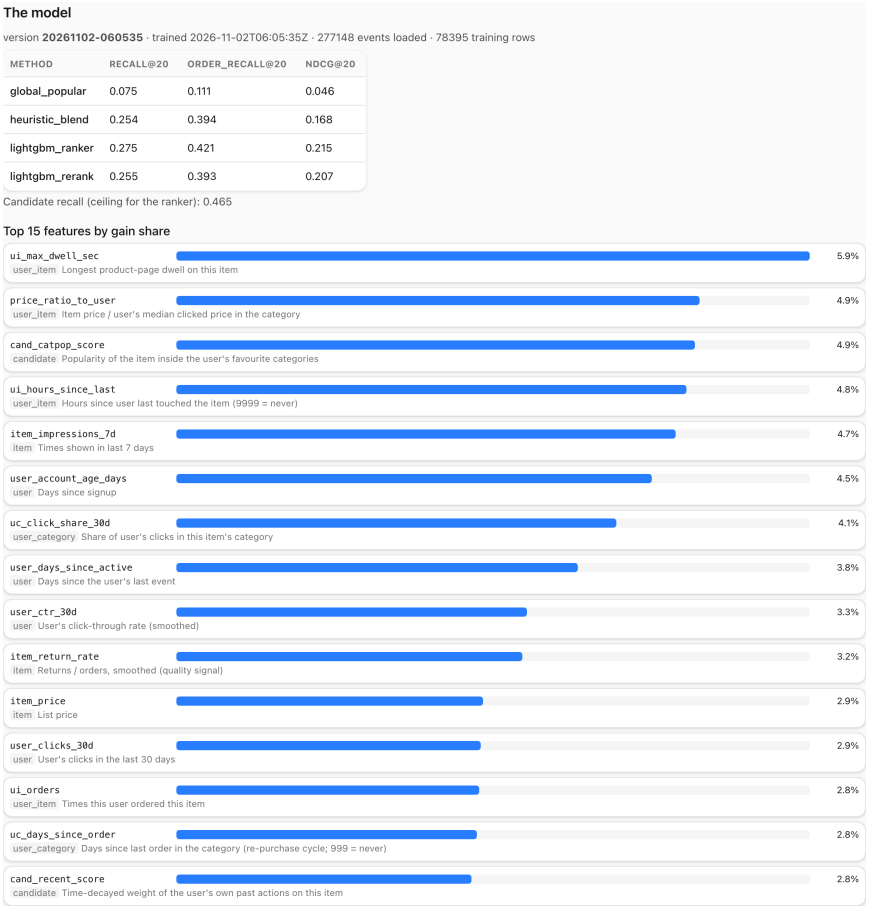
Your behaviour, live

TIME	EVENT	ITEM	SURFACE	POS	DWELL
1:19:21 PM	order	Volt Coffee & Tea Lite	checkout		
1:19:19 PM	click	Volt Coffee & Tea Lite	search	0	1.8
1:19:19 PM	impression	Titan Coffee & Tea Premium	similar	11	
1:19:19 PM	impression	Lumi Coffee & Tea Best Seller	similar	10	
1:19:19 PM	impression	Volt Coffee & Tea Murah	similar	9	
1:19:19 PM	impression	Garuda Coffee & Tea Best Seller	similar	8	
1:19:19 PM	impression	Lumi Coffee & Tea Original	similar	7	
1:19:19 PM	impression	Volt Coffee & Tea Lokal	similar	6	
1:19:19 PM	impression	Titan Coffee & Tea Best Seller	similar	5	
1:19:19 PM	impression	Sakura Coffee & Tea Original	similar	4	
1:19:19 PM	impression	Bamboo Coffee & Tea Murah	similar	3	
1:19:19 PM	impression	Volt Coffee & Tea Pro	similar	2	
1:19:19 PM	impression	Lumi Coffee & Tea Pro	similar	1	
1:19:19 PM	impression	Bamboo Coffee & Tea Premium	similar	0	
1:19:19 PM	impression	Volt Coffee & Tea Lokal	search	23	
1:19:19 PM	impression	Lumi Coffee & Tea Pro	search	22	
1:19:19 PM	impression	Titan Coffee & Tea Premium	search	21	
1:19:19 PM	impression	Lumi Coffee & Tea Lokal	search	20	
1:19:18 PM	add_to_cart	Volt Coffee & Tea Lite	product		
1:19:17 PM	impression	Sakura Coffee & Tea Murah	search	19	
1:19:17 PM	impression	Nusa Coffee & Tea Best Seller	search	18	
1:19:17 PM	impression	Bamboo Coffee & Tea Murah	search	17	
1:19:17 PM	impression	Aurora Coffee & Tea Premium	search	16	
1:19:17 PM	impression	Lumi Coffee & Tea Lokal	search	15	
1:19:17 PM	impression	Lumi Coffee & Tea Lokal	search	14	
1:19:17 PM	impression	Mega Coffee & Tea Lite	search	13	
1:19:17 PM	impression	Garuda Coffee & Tea Murah	search	12	
1:19:17 PM	impression	Nusa Coffee & Tea Premium	search	11	
1:19:17 PM	impression	Lumi Coffee & Tea Import	search	10	
1:19:17 PM	impression	Mega Coffee & Tea Premium	search	9	

Recent searches
coffee
coffee
coffee
coffee
coffee
cat food
cat food
dog toys
dog toys
makeup

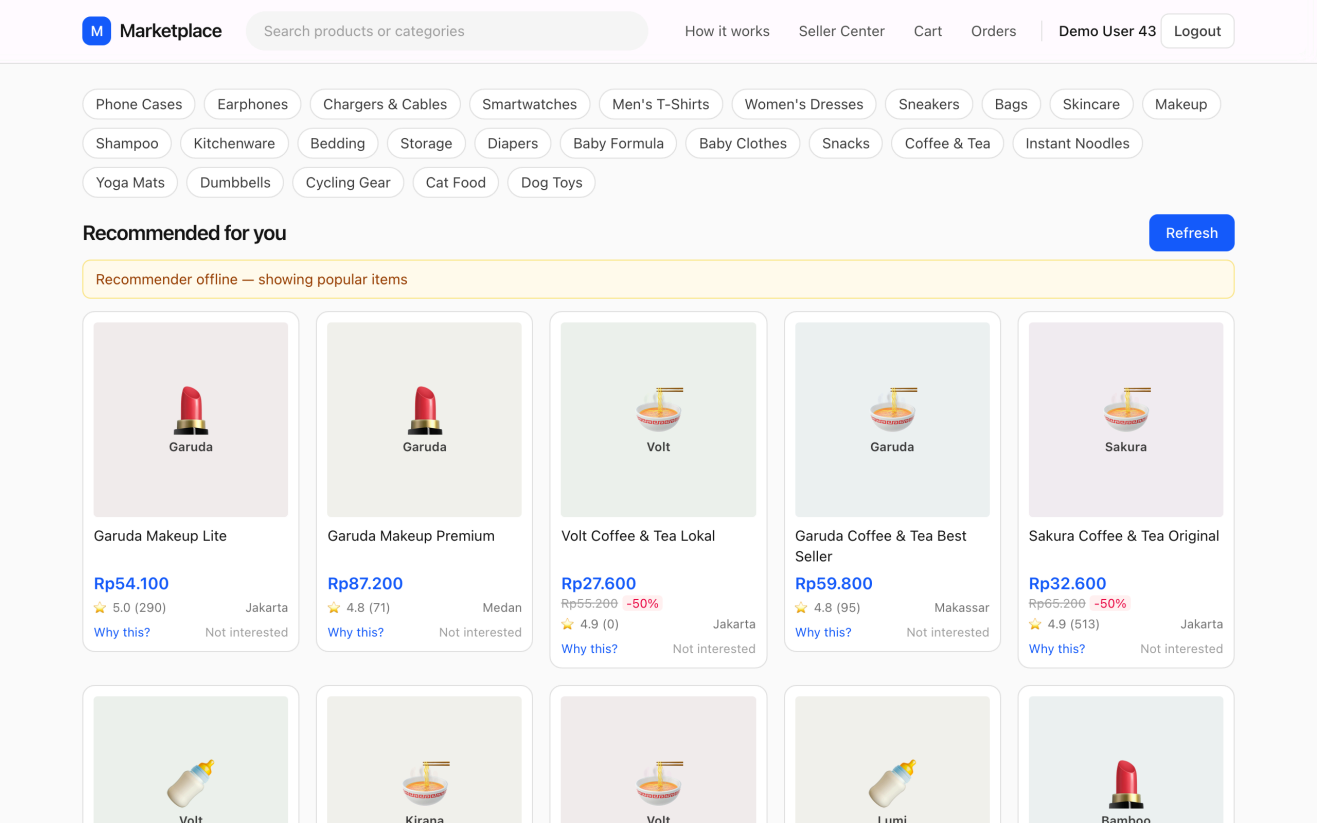
Gambar 13.9: *Tabel event langsung di halaman How it works: order, click dengan dwell, impresi menurut surface dan posisi*

Halaman model. "How it works" menampilkan tabel event langsung (baris yang sama dengan yang dibaca recommender), lalu bagian model: versi, events yang dimuat, tabel metrik untuk keempat metode, *candidate recall*, dan fitur teratas berdasarkan gain, dengan `ui_max_dwell_sec` di urutan pertama. Admin juga mendapat tombol "Retrain now"; setelah pelatihan ulang, versi, metrik, dan jumlah baris latih sedikit berbeda meskipun tidak ada kode yang berubah, karena setiap *cutoff* relatif terhadap `now` dan jendela label bergeser mengikuti jam.



Gambar 13.10: *Bagian model pada halaman How it works: versi, metrik per metode, candidate recall, dan fitur teratas*

Mode menurun. Dengan recommender dihentikan, banner kuning menggantikan keterangan model, kartu berasal dari query fallback, dan panel "Why this?" kosong karena item *trending* tidak punya contributions. Pengguna sedikit dirugikan; tidak ada yang terkunci di luar.



Gambar 13.11: *Beranda dengan banner kuning "Recommender offline"*

Jebakan dan praktik produksi

Waktu event versus waktu server. `events.ts` bernilai bawaan `now()` milik database: *timestamp* dari klien akan membiarkan jam ponsel yang melenceng menulis event dari masa depan. Biayanya, batch yang terlambat di-*flush* diberi cap waktu flush, tidak relevan untuk jendela 2 detik tetapi relevan untuk aplikasi *offline-first*.

Training/serving skew. Layanan memanggil `pipeline.recommend` dan job pelatihan memanggil `pipeline.train_and_evaluate`; keduanya memakai `History.at` dan `features.build`. Satu implementasi per fitur, bukan dua; sebuah *feature store* (Bab 14) mengindustrialisasi hal ini.

Anggaran latensi. Pada skala besar Anda tidak punya 840 ms; setiap tahap mendapat sepotong dari beberapa ratus milidetik. Ukur persentil, temukan biaya tetapnya, *cache* apa yang bisa di-*cache*, dan jaga pekerjaan yang dipersonalisasi tetap kecil.

Ringkasan

- Marketplace ini memiliki empat proses: frontend Vue, API Rust/Axum yang menjadi satu-satunya penulis, PostgreSQL sebagai buku besar bersama, dan recommender gRPC Python yang membaca Postgres lewat koneksi read-only.
- Sebuah klik menjadi peringkat yang berubah melalui impresi yang diantrekan dan di-flush oleh `useTracker`, `post_events` dan `touch_session` yang menulis baris dengan id sesi, `Store.refresh` yang membacanya secara inkremental, dan `Recommend` yang menjalankan

seluruh pipeline saat diminta. *Staleness* kasus terburuk adalah jeda flush ditambah *throttle* refresh ditambah waktu permintaan.
- Event saat keluar membutuhkan satu flusher yang berurutan: registri *hook* `onPageHide` memperbaiki bug ketika sebuah klik diantrekan setelah beacon sudah terkirim.
- Recommender tetap segar dengan refresh inkremental yang aman terhadap *overlap* (`id > last − 1000`, lalu hapus duplikat) karena id *auto-increment* ditetapkan saat insert tetapi terlihat saat commit.
- Satu berkas `.proto` adalah kontraknya; Rust menghasilkan klien di `build.rs`, Python menghasilkan stub dengan `recsys gen-proto`, dan respons membawa versi model, jumlah kandidat, dan latensi untuk observabilitas.
- Pelatihan ulang bersifat periodik dan sesuai permintaan, dijaga oleh *lock* non-blocking dan di-*hot-swap* dengan menugaskan dua referensi, sehingga *serving* tidak pernah berhenti.
- API Rust menghidrasi id menjadi kartu menurut urutan recommender dan beralih ke query *trending* nonpersonal saat error atau *timeout*, sehingga gangguan recommender menurunkan halaman alih-alih merusaknya.
- Diukur di mesin ini, `Recommend` memakan sekitar 840 ms pada median dan 985 ms pada persentil ke-99, hampir tidak bergantung pada pengguna. Profil tahap mengaitkan 97% darinya dengan pembuatan kandidat dan pembangunan fitur: 144 ms pekerjaan tingkat item yang bisa diserap *cache* ber-TTL pendek, dan 303 ms agregasi sisi pengguna atas semua pengguna yang hanya bisa dihilangkan dengan menyaring event ke pengguna yang meminta.

Kuis

Q13.1 Mengapa recommender Python membuka koneksi databasenya dengan `default_transaction_read_only=on`?

A. Ini membuat query `SELECT` lebih cepat dengan melewati lock.
B. Ini menjamin bahwa bug di recommender tidak dapat mengubah data aplikasi; setiap penulisan gagal dengan error.
C. Ini memungkinkan Postgres memakai ulang koneksi yang sama untuk API Rust.
D. Ini diwajibkan oleh protokol gRPC.

Q13.2 Himpunan tipe event mana yang diterima `POST /api/events` dari browser, dan mengapa `order` tidak termasuk di dalamnya?

A. `impression`, `click`, `hide`; `order` adalah akibat dari aksi sisi server (checkout) dan tidak boleh dapat dipalsukan dari klien.
B. `impression`, `click`, `order`; `hide` diturunkan di server.
C. Keenam tipe; constraint `CHECK` database adalah satu-satunya perlindungan.
D. Hanya `click`; impresi dihitung oleh server dari pemuatan halaman.

Q13.3 Pada bug `pagehide`, apa persisnya yang salah pada versi pertama halaman produk?

A. Panggilan `fetch` mengalami *timeout* setelah 2 detik.
B. *Listener* `pagehide` milik halaman produk berjalan setelah *listener* tracker sudah menguras antrean ke beacon, sehingga klik tidak pernah terkirim.

C. `sendBeacon` tidak dapat mengirim JSON.
D. Vue Router melepas (*unmount*) komponen sebelum `pagehide` terpicu.

Q13.4 Jelaskan dalam dua atau tiga kalimat mengapa `Store._refresh` membaca ulang 1000 id event terakhir alih-alih melakukan query `id > last_event_id`. Apa yang akan salah tanpa *overlap* itu?

Q13.5 `Recommend` diawali dengan `model, version = self.model, self.meta["version"]` dan `retrain` diakhiri dengan `self.model, self.meta = model, meta`. Jelaskan bagaimana kedua baris ini memungkinkan layanan menukar model saat ada lalu lintas tanpa lock pada jalur permintaan.

Q13.6 (Membaca kode.) Baca `touch_session` di `backend/src/sessions.rs`. Seorang pembeli memicu event pada pukul 10:00, 10:20, dan 11:05 pada hari yang sama, tanpa aktivitas lain. Berapa sesi yang dibuat, dan berapa `last_seen_at` sesi pertama di akhir hari? Jelaskan berdasarkan aturan 30 menit di SQL tersebut.

Q13.7 (Numerik.) Rute Rust menghentikan panggilan gRPC dengan *timeout* setelah 5 detik. Berdasarkan hasil di bab ini, berapa fraksi anggaran itu yang dipakai oleh panggilan sisi server teramati yang paling lambat (1,010.6 ms), dan berapa *staleness* kasus terburuk $T_{\text{stale}} = T_{\text{flush}} + T_{\text{refresh}} + L$ jika $T_{\text{flush}} = 2$ dtk, $T_{\text{refresh}} = 1$ dtk, dan L adalah latensi server p99 teramati sebesar 985.0 ms? Berikan kedua jawaban.

Q13.8 Apa yang dikembalikan fallback ketika recommender tidak tersedia, dan mengapa ia melabeli setiap item dengan `source: "trending"`, skor 0, dan contributions kosong?

Q13.9 Anda mengganti nama sebuah field di `recommender.proto` dengan mengubah nomornya dari `3` menjadi `7` dan membangun ulang hanya API Rust. Layanan Python masih berjalan dengan stub hasil generate yang lama. Apa hasil yang paling mungkin, dan mengapa panduan menyatakan nomor field tidak boleh diubah setelah dipakai?

A. Tidak terjadi apa-apa; format *wire* memakai nama field.
B. Kedua sisi berselisih tentang byte mana yang berarti field mana, sehingga data hilang atau salah dibaca tanpa suara.
C. Rust akan menolak untuk dikompilasi.
D. gRPC akan menegosiasikan nomor baru secara otomatis.

Q13.10 (Praktik langsung.) Buka `book/code/ch13_latency.py` dan ubah `k=20` pada pemanasan maupun loop pengukuran menjadi `k=5`, lalu lagi menjadi `k=50`, jalankan ulang skrip terhadap layanan yang aktif setiap kali. Catat `p50` dan baris "candidates scored per request". Apa yang Anda harapkan teramati tentang latensi dan jumlah kandidat, dan apa yang ditunjukkannya tentang ke mana waktu terpakai?

Bab 14 — Dari prototipe ke skala Shopee

Pukul 12:00 pada 9.9, dan flash sale baru saja dibuka. Dalam hitungan detik, jumlah orang di marketplace melonjak berkali-kali lipat dari puncak jam makan siang biasa. Setiap orang mendarat di beranda (*home feed*), dan setiap beranda harus berbeda. Pemeringkat (*ranker*) yang Anda latih di Bab 10 hanyalah beberapa ratus kilobyte pohon keputusan. Ia dibangun dari satu tabel Postgres, dan butuh sekitar 840 milidetik untuk menjawab satu permintaan di laptop. Tidak ada yang salah dengannya. Ukurannya pas untuk sebuah buku dan akhir pekan belajar, dan bentuknya hampir sama dengan sistem produksi. Tetapi ia tidak pernah dirancang untuk bertahan melewati siang itu.

Ada hal lain yang sama pentingnya. Flash sale tidak hanya membebani server. Ia mengubah datanya. Pembeli yang biasanya menjelajahi sepatu tiba-tiba memburu penanak nasi diskon. Model yang dilatih minggu lalu belum pernah melihat kerumunan flash sale, dan fitur-fiturnya masih mengatakan bahwa barang populer kemarin tetap yang terpopuler. Skala bukan hanya soal *throughput*. Skala juga soal kesegaran data, soal mengukur apakah sebuah perubahan benar-benar membantu, dan soal menjaga banyak tim, banyak tujuan, dan banyak model tetap jujur pada saat yang sama.

Bab penutup ini adalah sebuah peta. Kita tidak akan membangun klaster Kafka atau melatih transformer. Sebagai gantinya, kita menelusuri satu per satu komponen yang ditambahkan perusahaan seukuran Shopee di sekeliling inti yang sudah Anda pahami, dan menyebutkan dengan tepat masalah apa yang diselesaikan tiap komponen. Untuk setiap komponen, kita tunjuk bagian repositori ini yang akan digantikan atau diperluasnya. Di akhir bab, Anda seharusnya bisa membaca diagram arsitektur marketplace besar dan mengenali setiap kotaknya sebagai versi yang lebih besar dan lebih kokoh dari sesuatu yang sudah pernah Anda jalankan.

Yang akan Anda pelajari

- Bagaimana log *streaming* (Kafka) dan pemrosesan aliran (Flink) mengubah tabel `events` yang bersifat *batch* menjadi fitur waktu nyata, dan mengapa hal itu mengubah arti "segar".
- Untuk apa *feature store* digunakan, dan bagaimana ia mencegah *training/serving skew*.
- Mengapa pencarian tetangga terdekat hampiran (*approximate nearest neighbour*, ANN) tak terhindarkan pada miliaran item, dan cara memperkirakan biaya memorinya.
- Bagaimana pemeringkat Wide & Deep, DCN, DIN, dan MMoE multi-tugas berbeda dari pemeringkat *gradient-boosted* di repositori ini, dan kapan mereka sepadan dengan biayanya.
- Cara menggabungkan beberapa tujuan (GMV, iklan, keadilan bagi penjual) menjadi satu skor tanpa menyembunyikan trade-off-nya.
- Cara menjalankan uji A/B, termasuk uji-z dua proporsi lengkap dan perhitungan ukuran sampel yang dikerjakan manual.
- Apa yang perlu dipantau setelah sistem live, cara menganggarkan latensi, dan daftar periksa konkret tentang apa yang dibangun berikutnya di repositori ini.

Intuisi: pasar malam dan jaringan supermarket

Bayangkan sebuah lapak pasar malam yang menjual pisang goreng. Pemiliknya mengenal setiap pelanggan tetap. Ia melihat siapa yang datang, mengingat pesanan minggu lalu, menyesuaikan gula ketika cuaca berubah, dan memutuskan di tempat apa yang akan direkomendasikan. Itulah prototipe kita. Semuanya muat dalam satu kepala, yaitu satu database Postgres dan satu proses Python. Pemilik bisa mencoba ide baru saat makan siang dan tahu pada makan malam apakah ide itu berhasil.

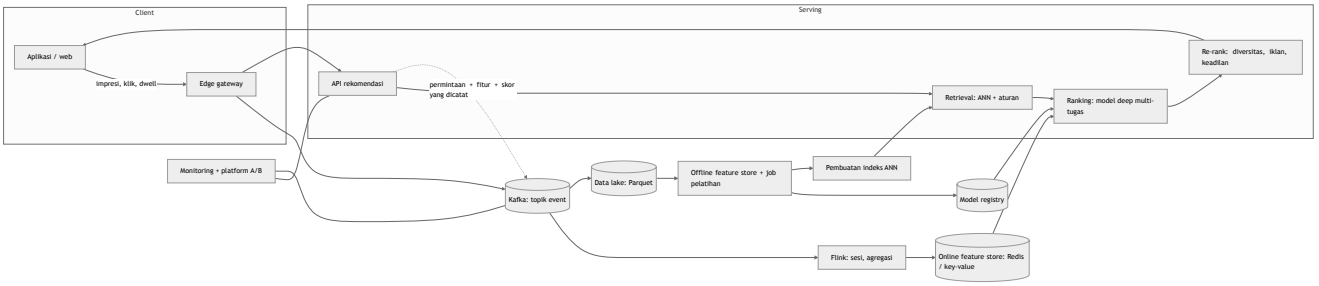
Sekarang bayangkan jaringan supermarket nasional: ratusan toko melaporkan penjualan lewat aliran pesan, tim data memelihara satu definisi bersama tentang "pelanggan setia", beberapa tim menjalankan promosi, dan bagian keuangan berkeras agar eksperimen diukur sebelum promosi diluncurkan secara nasional. Jaringan itu tidak membutuhkan pemilik yang lebih pintar. Ia membutuhkan *infrastruktur untuk menyepakati fakta, memindahkannya dengan cepat, dan mengukur perubahan*.

Itulah seluruh perbedaan antara repositori kita dan marketplace produksi. Gagasan-gagasannya (kandidat, fitur, pemeringkatan, pemeringkatan ulang) tetap utuh. Yang berlipat ganda adalah pipa-pipa di sekelilingnya. Sepanjang sisa bab ini kita akan terus mengajukan tiga pertanyaan yang sama pada setiap komponen baru:

1. Batasan prototipe mana yang dihilangkannya: kecepatan, kesegaran, skala, ketepatan, atau kemampuan bereksperimen?
2. Mode kegagalan baru apa yang dibawanya?
3. File mana di repositori ini yang akan digantikannya?

Algoritma: pipeline yang sama, diperluas skalanya

Sebelum masuk ke rincian, berikut gambaran utuhnya. Pipeline multi-tahap dari Bab 8 (ambil, peringkatkan, peringkatkan ulang) tidak berubah. Yang berubah adalah setiap panah dalam diagram menjadi layanan jaringan dengan latensi, kapasitas, dan pemilik masing-masing.



Gambar 14.1: *Arsitektur ala Shopee. Garis putus-putus adalah yang terpenting: serving mencatat apa yang ditampilkannya, beserta fitur dan skor yang digunakannya.*

Baca diagram dari kiri ke kanan. Perilaku mengalir dari aplikasi ke antrean pesan. Dari sana ada dua jalur. Jalur cepat (Flink ke *online feature store*) menjaga fitur tetap segar hingga hitungan detik. Jalur lambat (*data lake* ke job pelatihan) menghasilkan model dan indeks ANN setiap beberapa jam atau hari. *Serving* membaca dari keduanya: model dan indeks terbaru dari jalur lambat, fitur pengguna terbaru dari jalur cepat. Dan yang krusial, *serving* menulis balik *exposure log*, yaitu apa yang ditampilkannya, sehingga pelatihan berikutnya tahu apa yang benar-benar dilihat pengguna.

Di prototipe, lingkaran yang sama ada tetapi diciutkan menjadi satu mesin. `useTracker.ts` adalah klien, rute Rust `POST /api/events` adalah *edge gateway*, tabel Postgres `events` adalah Kafka sekaligus *data lake*, `db.Store.refresh` adalah konsumen aliran, `pipeline.train_and_evaluate` adalah job pelatihan, dan RPC `Recommend` adalah lapisan *serving*. Ketika sebuah pertanyaan dimulai dengan "bagaimana kita menskalakan ini?", petakan dulu pertanyaan itu ke daftar ini.

Log streaming dan fitur waktu nyata

Di prototipe, Rust menyisipkan satu baris per *event* ke Postgres, dan layanan Python melakukan *polling* untuk baris yang lebih baru dari id terakhir yang dilihatnya. Ini sederhana dan benar, dan berfungsi untuk beberapa ratus ribu baris. Pada skala besar, ia rusak dengan tiga cara:

- **Throughput penulisan.** Satu primary Postgres tidak akan sanggup menyerap jutaan *insert* per detik, dan penulisan *event* bersaing dengan pesanan dan keranjang.
- **Fan-out.** *Event* yang sama dibutuhkan banyak konsumen: pipeline fitur, sistem iklan, deteksi fraud, *data warehouse* analitik. Melakukan *polling* pada tabel menjadikan setiap konsumen pembaca database produksi.
- **Replay.** Ketika ditemukan bug pada definisi fitur, Anda perlu memproses ulang *event* bulan lalu. Tabel yang sekaligus menjadi sistem pencatatan utama (*system of record*) adalah tempat yang buruk untuk itu.

Jawaban standarnya adalah *message broker* berbasis log seperti Apache Kafka. Produser menambahkan *event* ke *topic*, yang dibagi menjadi *partition*. Setiap *consumer group* melacak *offset*-nya sendiri. Mempartisi berdasarkan `user_id` menjamin bahwa semua *event* seorang pengguna tiba berurutan di konsumen yang sama, persis yang dibutuhkan logika sesi. Karena log disimpan selama beberapa hari, konsumen baru dapat mulai dari awal, dan bug yang sudah diperbaiki dapat dijalankan ulang atas *event* lama.

Pemroses aliran seperti Apache Flink kemudian menghitung fitur secara inkremental. Ambil fitur `user_clicks_30d` dari Bab 9. Dalam SQL ia adalah `COUNT` atas sebuah jendela. Dalam sistem *streaming* ia menjadi operator berstatus: simpan *state* kecil per pengguna, tambah satu untuk setiap klik, kurangi klik yang baru saja keluar dari jendela 30 hari, lalu tulis nilai barunya ke *online store*. Bentuk umumnya adalah *event masuk, state diperbarui, fitur keluar*:

$$s_u^{(t+1)} = f\big(s_u^{(t)},\, e_{t+1}\big), \qquad x_u^{(t+1)} = g\big(s_u^{(t+1)}\big)$$

Di sini s_u adalah *state* pengguna u, e adalah *event* baru, f memperbarui *state*, dan g mengubah *state* menjadi nilai fitur x_u. Skor peluruhan seperti peluruhan eksponensial pada Bab 3 sangat cocok dalam bentuk ini, karena dapat diperbarui dalam waktu konstan: $s \leftarrow s \cdot 2^{-\Delta t / h} + w$, dengan h adalah waktu paruh dan w bobot *event*.

Streaming menambah kerumitan: *event* yang terlambat dan tidak berurutan membutuhkan jendela berbasis *event-time* dan *watermark*, *restart* tidak boleh menghitung ganda, dan menghitung fitur sekali di Flink dan sekali lagi secara *batch* mengundang *skew* yang dibahas berikutnya.

Feature store dan training/serving skew

Ingat demo kebocoran data (*leakage*) di Bab 9. Fitur yang mengintip masa depan membuat ndcg@20 *offline* tampak 0.5035 dan bukan 0.2116, dan model itu tidak berguna saat live. *Training/serving skew* adalah saudara yang lebih halus dari *leakage*: fiturnya sah di kedua tempat, tetapi dihitung secara berbeda. Contoh umum: pelatihan membaca `user_ctr_30d` dari job SQL yang mengecualikan trafik bot, sementara *serving* menghitungnya di kode aplikasi yang tidak mengecualikan. Model mempelajari hubungan dari satu distribusi lalu ditanyai dengan distribusi lain. Metrik *offline* tetap sangat bagus. Metrik *online* diam-diam melorot.

Feature store menyerang masalah ini dengan menjadikan definisi fitur sebagai satu artefak ber-versi dengan dua materialisasi:

- **offline store** (file kolumnar atau *warehouse*) yang dapat menjawab "berapa nilai fitur ini untuk pengguna u *pada waktu* t?" untuk jutaan baris pelatihan; dan
- **online store** (penyimpanan *key-value* berlatensi rendah) yang menjawab "berapa nilainya sekarang?" dalam satu milidetik.

Definisi yang sama memberi makan keduanya, sehingga angkanya selaras dari rancangannya. Separuh *offline* adalah tempat ketepatan titik-waktu (*point-in-time correctness*) berada. Ia melakukan "*as-of join*" yang dilakukan `data.History.at` di repositori ini: untuk setiap baris pelatihan dengan stempel waktu t, ambil nilai fitur terbaru dengan stempel waktu $\leq t$, tidak pernah yang lebih baru. Dalam arti ini, gagasannya persis sama dengan Bab 9. *Feature store* hanya meng-industrialisasikannya.

Pertahanan kedua terhadap *skew* adalah *mencatat fitur yang disajikan*. Alih-alih menghitung ulang fitur historis untuk pelatihan, lapisan *serving* menulis vektor fitur yang benar-benar digunakannya, bersama posisi item dan skor model, ke *exposure log* (garis putus-putus di diagram). Pelatihan kemudian belajar dari persis apa yang disajikan. Ini memakan penyimpanan, tetapi menghapus satu kelas bug sepenuhnya, dan memberi Anda *propensity* yang dibutuhkan untuk koreksi bias posisi pada Bab 12.

ANN pada miliaran item

Bab 7 menunjukkan indeks IVF yang memeriksa klaster vektor item. Pada 2,311 item yang tersedia, *brute force* membutuhkan 0.140 ms per kueri, dan indeks lebih lama dibangun daripada manfaatnya. Pada seratus juta atau satu miliar item, gambarannya berbalik. Ambil perkiraan memori kasar. Satu miliar item, masing-masing vektor 64 dimensi berisi *float* 4-byte:

$$10^9 \times 64 \times 4 \text{ bytes} = 2.56 \times 10^{11} \text{ bytes} = 256 \text{ GB}.$$

Itu tidak muat dengan nyaman di satu mesin, dan memindainya per permintaan mustahil. Karena itu sistem produksi melakukan tiga hal sekaligus.

Pertama, **shard** indeksnya. Setiap *shard* menyimpan sebagian katalog, dan kueri disebar ke semua *shard*, yang masing-masing mengembalikan hasil teratas lokalnya untuk digabungkan. Kedua, **kompres** vektornya. *Product quantisation* memecah vektor menjadi sub-vektor dan mengganti masing-masing dengan id *codeword* terdekatnya. Vektor 256-byte bisa menjadi 16 byte, menyusutkan indeks yang sama menjadi $10^9 \times 16 = 16$ GB. Ketiga, gunakan **indeks graf atau berklaster** (HNSW, IVF-PQ, atau ScaNN dari Google) agar setiap kueri hanya menyentuh sebagian kecil vektor. Trade-off-nya persis seperti pada tabel Bab 7, di mana memeriksa 4 dari 32 klaster mempertahankan recall@20 pada

0.975 dari *brute force* sambil memindai 12.5% item. Pada skala besar Anda mengatur kenop yang sama, hanya dengan target latensi yang lebih ketat dan katalog yang jauh lebih besar.

Indeks dibangun ulang, bukan disunting, dan *item tower* harus dijalankan ulang untuk seluruh katalog setiap kali model berubah, sehingga versi model dan indeks dirilis sebagai satu pasangan.

> **Tahukah kamu?**
>
> **Graph embedding skala web.** PinSage milik Pinterest menggabungkan *random walk* dengan konvolusi graf untuk mempelajari *embedding* pin. Ia dilatih pada graf dengan 3 miliar simpul (pin dan *board*) dan 18 miliar sisi, yang oleh para penulisnya disebut sebagai penerapan *graph embedding* mendalam terbesar pada saat publikasi.
>
> Sumber: Ying et al., KDD, 2018

Pemeringkat deep

Pohon *gradient-boosted* adalah pemeringkat yang sangat baik untuk beberapa lusin hingga beberapa ratus fitur padat, dan papan peringkat di akhir bab ini menunjukkan sejauh mana mereka bisa melangkah pada data simulasi kita. Di marketplace besar, masukan pemeringkat terlihat berbeda: miliaran id jarang (*sparse*) (pengguna, item, toko, kueri), urutan perilaku yang panjang, dan beberapa hasil yang diprediksi sekaligus. Pohon kurang cocok untuk ini, karena pohon tidak bisa mempelajari *embedding* untuk id yang hanya muncul sebulan sekali. Pemeringkat neural bisa. Berikut empat keluarga yang paling sering Anda dengar.

Wide & Deep. Tim Google Play mengamati dua kebutuhan yang saling melengkapi. *Memorisasi*: mempelajari bahwa pasangan tertentu, seperti "memasang aplikasi A dan ditampilkan aplikasi B", sangat prediktif. *Generalisasi*: menangani pasangan yang belum pernah muncul bersamaan, melalui *embedding*. Wide & Deep melatih keduanya bersama-sama. Bagian "wide" adalah model linear atas fitur silang (*cross-product*) yang jarang. Bagian "deep" adalah MLP atas *embedding*. Keluarannya dijumlahkan menjadi satu logit:

$$P(y = 1 \mid \mathbf{x}) = \sigma\big(\mathbf{w}_{\text{wide}}^{\top}[\mathbf{x}, \phi(\mathbf{x})] + \mathbf{w}_{\text{deep}}^{\top} a^{(L)} + b\big)$$

dengan $\phi(\mathbf{x})$ adalah transformasi *cross-product* dan $a^{(L)}$ adalah lapisan tersembunyi terakhir dari MLP. Di repositori kita, fitur silang buatan tangan dari Bab 9 (`uc_click_share_30d`, `price_ratio_to_user`) berperan sebagai bagian wide, dibuat oleh seorang *engineer* dan diberikan ke pohon. Pada **DCN** (Deep & Cross Network), persilangannya dipelajari: lapisan silang eksplisit mengalikan fitur masukan satu sama lain, lapis demi lapis, sehingga Anda tidak perlu lagi menulis tangan setiap interaksi.

DIN. Deep Interest Network milik Alibaba berangkat dari pengamatan tentang e-commerce. Riwayat pengguna memuat banyak minat, dan hanya sebagian yang relevan untuk item yang sedang dinilai. Memampatkan seluruh riwayat menjadi satu vektor tetap membuang kapasitas. DIN menambahkan *local activation unit*: item-item dalam riwayat pengguna diberi bobot menurut relevansinya terhadap item *kandidat*, sehingga pengguna yang sama mendapat vektor minat yang berbeda ketika kandidatnya casing ponsel dibandingkan penggorengan. Dalam pseudo-rumus:

$$\mathbf{v}_u(a) = \sum_{j=1}^{H} w_j(a)\,\mathbf{e}_j, \qquad w_j(a) = \mathrm{MLP}\big(\mathbf{e}_j,\ \mathbf{e}_a,\ \mathbf{e}_j \odot \mathbf{e}_a\big)$$

Di sini $\mathbf{e}_j$ adalah *embedding* dari H item lampau pengguna dan $\mathbf{e}_a$ adalah *embedding* kandidat. Bandingkan dengan *user tower* di Bab 7, yang merata-ratakan 20 item terakhir dengan bobot sama. DIN adalah gagasan yang sama, dengan bobot yang dipelajari dan bergantung pada kandidat. Model urutan berbasis Transformer, yang kini lazim, menggeneralisasikannya lebih jauh dengan *self-attention*.

MMoE multi-tugas. Marketplace peduli pada setidaknya tiga hasil: klik, tambah-ke-keranjang, dan pesanan. Di repositori ini kita memerasnya menjadi satu label (klik bernilai 1, keranjang 2, pesanan 3) dan melatih satu model LambdaMART. Itu murah, tetapi mengunci trade-off-nya dan hanya memberi model satu target yang berisik. Jaringan multi-tugas memprediksi setiap hasil dengan *head*-nya sendiri dan berbagi lapisan bawah. Versi naifnya, satu *trunk* bersama, menderita ketika tugas-tugas bertentangan: apa yang membuat pengguna mengklik (gambar yang menarik) berbeda dari apa yang membuatnya membeli. Rancangan *Multi-gate Mixture-of-Experts* mempertahankan beberapa sub-jaringan "ahli" (*expert*) bersama dan memberi setiap tugas gerbang (*gate*) sendiri yang memutuskan cara mencampurnya:

$$y_k = h_k\Big(\sum_{i=1}^{n} g_k(\mathbf{x})_i\, f_i(\mathbf{x})\Big), \qquad g_k(\mathbf{x}) = \mathrm{softmax}(W_{g_k}\,\mathbf{x})$$

untuk tugas k, jaringan ahli f_i, *head* tugas h_k, dan gerbang per-tugas g_k. Jika klik dan pesanan berkaitan erat, gerbang mereka akan memilih ahli yang mirip. Jika bertentangan, gerbang akan mengkhususkan diri. Karena pesanan jarang dibandingkan klik (corong pada Bab 1), *head* `order` yang jarang juga diuntungkan oleh representasi yang dipelajari *head* `click` yang padat. Skor produksi kemudian menggabungkan semua *head*, yang membawa kita ke topik berikutnya.

Tahukah kamu?

Atensi pada riwayat pengguna. Deep Interest Network (DIN) milik Alibaba mengganti vektor pengguna berpanjang tetap dari resep Embedding-and-MLP standar dengan *local activation unit* yang memberi bobot pada perilaku lampau pengguna menurut relevansinya terhadap iklan kandidat. Ia dilatih pada dataset produksi Alibaba berisi lebih dari 2 miliar sampel dan diterapkan di sistem iklan *display* online Alibaba, melayani trafik utama.

Sumber: Zhou et al., KDD, 2018

Tahukah kamu?

Wide & Deep di toko aplikasi. Makalah Wide & Deep dari Google melatih bersama model linear (memorisasi) dan jaringan deep (generalisasi), dan mengevaluasinya di Google Play, toko aplikasi komersial dengan lebih dari satu miliar pengguna aktif dan lebih dari satu juta aplikasi. Eksperimen online menunjukkan bahwa ia meningkatkan akuisisi aplikasi secara signifikan dibandingkan model wide-saja dan deep-saja.

Sumber: Cheng et al., DLRS Workshop, 2016

Pemeringkatan multi-tujuan: GMV, iklan, dan penjual

Setelah pemeringkat mengeluarkan $\hat{p}_{\text{click}}$, $\hat{p}_{\text{cart}}$, dan $\hat{p}_{\text{order}}$ secara terpisah, seseorang harus memutuskan cara memadukannya. Bentuk paling sederhana adalah jumlah berbobot, sering dengan suku bisnis untuk margin atau penempatan bersponsor:

$$\text{score}(u, i) = \Big(w_1\,\hat{p}_{\text{click}} + w_2\,\hat{p}_{\text{cart}} + w_3\,\hat{p}_{\text{order}}\Big) \cdot \big(1 + \beta_{\text{boost}}(i)\big) + \lambda_{\text{ads}}\,\text{bid}(i)\,\hat{p}_{\text{click}}$$

Bobot-bobot ini tidak dipelajari dari klik. Mereka adalah kebijakan. Menaikkan w_3 mengutamakan GMV (*gross merchandise value*), dan menaikkan λ_{ads} mengutamakan pendapatan iklan. *Boost* $\beta_{\text{boost}}(i)$ menyandikan aturan kampanye, seperti subsidi tingkat toko untuk merek mall selama sale. Ini adalah struktur yang sama dengan aturan *re-ranking* di Bab 11, yang dinaikkan dari "batasan dalam fungsi Python" menjadi bagian formal dari tujuan, dan sistem produksi yang baik bersikap eksplisit tentangnya. Semua orang dapat membaca bobotnya, memperdebatkannya, dan melihatnya di konfigurasi eksperimen. Bahayanya adalah keterkaitan tersembunyi: pemeringkat yang dilatih untuk memprediksi pesanan lalu dibobot ulang berdasarkan margin bisa mengejar barang mahal yang jarang terkonversi.

Keadilan bagi penjual masuk dengan dua cara. Sebagai **batasan**, ia seperti batas 4 item per toko di `rerank.rerank` pada repositori. Sebagai **tujuan**, Anda memberi penjual baru atau kecil sejumlah eksposur yang terkendali, karena marketplace yang hanya menampilkan toko-toko lama akan membuat sisi pasokannya sendiri kelaparan. Ini adalah gagasan eksplorasi dari Bab 11 dan Bab 12, diterapkan pada penjual alih-alih item. Apa pun pilihan Anda, *ukur secara terpisah*. Lacak metrik sisi penjual (porsi eksposur ke toko berusia di bawah 30 hari, konsentrasi antar toko) di samping metrik sisi pembeli dalam setiap eksperimen, dan perlakukan penurunan di salah satunya sebagai biaya.

> **Tahukah kamu?**
>
> **Multi-task learning untuk rekomendasi.** Makalah MMoE mengadaptasi struktur Mixture-of-Experts ke pembelajaran multi-tugas dengan membagi sub-model ahli di semua tugas sambil memberi setiap tugas jaringan gerbangnya sendiri. Para penulis menunjukkan bahwa ia berkinerja lebih baik daripada baseline ketika tugas-tugasnya kurang berkaitan, dan memperlihatkan peningkatan pada sistem rekomendasi konten skala besar di Google.
>
> Sumber: Ma et al., KDD, 2018

Uji A/B, lengkap dengan aritmetikanya

Setiap angka di papan peringkat di bawah adalah *offline*. Angka itu dihitung dari log tentang apa yang dilakukan pengguna di bawah sistem sebelumnya. Metrik *offline* memeringkatkan model kandidat dengan murah, tetapi tidak dapat memastikan apakah sebuah model membantu pengguna sungguhan, karena alasan dari Bab 2 dan 12: data log mencerminkan pilihan sistem lama, bias posisi sudah tertanam, dan metrik kita mengabaikan diversitas dan pendapatan. Penentu akhirnya adalah **uji A/B**: tetapkan pengguna secara acak ke kontrol (sistem saat ini) atau perlakuan (sistem baru), tunggu, lalu bandingkan metrik bisnis.

Mekanismenya mengikuti resep tetap:

1. Pilih *metrik utama* (misalnya konversi pesanan per pengguna) dan *metrik pengaman* (*guardrail*: latensi, eksposur sisi penjual, tingkat retur) sebelum peluncuran.
2. Hitung ukuran sampel yang dibutuhkan dari efek terkecil yang Anda pedulikan.
3. Acak **per pengguna**, dengan *hash* yang stabil, agar satu pengguna selalu melihat satu varian.
4. Periksa penugasannya sendiri. Jika Anda berniat membagi 50/50 dan mengamati 50.6/49.4 pada jutaan pengguna, ada yang rusak (*sample ratio mismatch*), dan hasilnya tidak bisa dipercaya.
5. Jalankan selama siklus mingguan penuh, lalu uji selisihnya.

Berikut contoh lengkap yang dikerjakan. **Semua angka di bawah ini hipotetis**, dibuat untuk menunjukkan cara kerjanya. Angka-angka itu bukan keluaran skrip repositori ini.

Misalkan setiap varian menerima $n = 100{,}000$ pengguna. Pada kontrol, 8,000 pengguna membuat pesanan, sehingga $\hat{p}_C = 0.0800$. Pada perlakuan, 8,240 pengguna, sehingga $\hat{p}_T = 0.0824$. Peningkatan absolut yang teramati adalah $0.0824 - 0.0800 = 0.0024$, peningkatan relatif 3%. Apakah ini nyata atau hanya derau?

Uji-z dua proporsi dimulai dari hipotesis nol bahwa kedua varian memiliki laju dasar yang sama. Di bawah hipotesis itu, estimasi terbaik untuk laju bersama adalah proporsi gabungan (*pooled*):

$$\hat{p} = \frac{x_C + x_T}{n_C + n_T} = \frac{8000 + 8240}{200000} = 0.0812$$

Galat baku selisih, di bawah hipotesis nol, adalah

$$\mathrm{SE} = \sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_C} + \frac{1}{n_T}\right)} = \sqrt{0.0812 \times 0.9188 \times \frac{2}{100000}} \approx 0.001222$$

Statistik uji menghitung berapa galat baku jarak selisih yang teramati dari nol:

$$z = \frac{\hat{p}_T - \hat{p}_C}{\mathrm{SE}} = \frac{0.0024}{0.001222} \approx 1.965$$

Untuk uji dua sisi, nilai-p adalah $2\,(1 - \Phi(1.965)) \approx 0.0494$, dengan Φ adalah CDF normal baku. Pada $\alpha = 0.05$ yang lazim, ini baru saja melewati ambang. Selang kepercayaan 95% untuk selisihnya (memakai galat baku tanpa *pooling*, juga sekitar 0.001222) kira-kira $[0.000006,\ 0.004794]$: ia nyaris tidak mengecualikan nol, sehingga peningkatan sebenarnya bisa berada di mana saja dari nyaris tidak ada hingga 6% relatif. Itulah pembacaan yang jujur. "Signifikan" di sini berarti "kita tidak bisa dengan nyaman menyingkirkan nol", bukan "peningkatannya 3%". Jika Anda mengintip dasbor setiap hari dan berhenti pada kali pertama $p < 0.05$, laju positif palsu Anda yang sebenarnya akan jauh lebih tinggi dari 5%. Tingkat signifikansi mengasumsikan satu kali pengamatan yang direncanakan sebelumnya.

Berapa pengguna yang kita butuhkan? Untuk efek absolut target δ pada signifikansi α dan daya (*power*) $1 - \beta$, ukuran sampel per varian kira-kira

$$n \approx \frac{2\,(z_{\alpha/2} + z_{\beta})^2\, p(1-p)}{\delta^2}$$

Dengan $p = 0.08$, $\delta = 0.0024$, $z_{\alpha/2} = 1.96$, dan $z_{\beta} = 0.8416$ (daya 80%):

$$n \approx \frac{2 \times (2.8016)^2 \times 0.08 \times 0.92}{0.0024^2} = \frac{2 \times 7.849 \times 0.0736}{0.00000576} \approx 200{,}600$$

Jadi sekitar 200,600 pengguna per varian, kira-kira dua kali lipat dari yang kita jalankan. Uji hipotetis kita *kurang bertenaga* (*underpowered*) untuk peningkatan relatif 3%, itulah sebabnya hasilnya mendarat di batas. Karena n yang dibutuhkan tumbuh sebesar $1/\delta^2$, menjadikan efek yang ingin dideteksi setengahnya berarti melipatempatkan trafik yang dibutuhkan. Inilah sebabnya perusahaan besar terobsesi pada teknik pengurangan varians, dan mengapa menguji banyak penyesuaian kecil sekaligus membutuhkan kehati-hatian.

Monitoring dan drift

Rekomender yang sudah dideploy bisa gagal secara diam-diam. Layanan tetap mengembalikan 200 OK dengan dua puluh item yang diformat sempurna, sementara kualitasnya merosot. Penyebab yang umum:

- **Data drift:** masukan berubah, misalnya versi aplikasi baru berhenti mengirim `dwell_sec`.
- **Concept drift:** hubungan fitur-hasil berubah setelah hari libur.
- **Feedback loop:** model hanya belajar tentang item yang pernah ditampilkannya (Bab 12).
- **Pipeline rusak:** aliran yang terlambat, fitur yang basi, indeks yang tidak dibangun ulang.

Masing-masing punya monitor. Untuk *data drift*, bandingkan distribusi tiap fitur pada satu jam terakhir dengan distribusi pelatihan menggunakan statistik sederhana seperti *population stability index* atau uji Kolmogorov–Smirnov, dan beri peringatan ketika porsi nilai yang hilang berubah. Untuk model, perhatikan distribusi skor prediksi, karena pergeseran mendadak adalah proksi murah untuk masukan yang rusak. Untuk hasil, lacak CTR dan konversi online menurut posisi dan menurut kelompok aktivitas pengguna, seperti yang dilakukan Bab 12 secara *offline*, dan bandingkan dengan **holdout** kecil: beberapa persen pengguna yang selalu menerima baseline sederhana. *Holdout* adalah penggaris jujur Anda, karena ia memberi tahu apakah seluruh rekomender masih mengalahkan popularitas, yang menurut papan peringkat kita selisihnya setidaknya 4x (pop_7d meraih 0.048, pemeringkat 0.215). Terakhir, pantau *kesegaran* itu sendiri: usia *event* terbaru yang terlihat oleh pemeringkat, dan usia indeks. Di repositori ini, padanan peringatan kesegaran adalah membandingkan `max(ts)` di tabel `events` dengan *event* terbaru yang dilihat `Store.refresh`.

Anggaran latensi

Sebuah permintaan adalah rantai panggilan, sehingga anggaran halaman (katakanlah 300 ms pada p99) dibagi ke seluruh tahap: gateway dan autentikasi, pengambilan fitur pengguna, *retrieval*, perakitan fitur, *ranking*, *re-ranking*, hidrasi, dan jaringan. Tiga prinsip membuat anggaran berjalan. Pertama, **ekor mendominasi**: halaman yang menyebar ke sepuluh layanan selambat layanan yang paling lambat, sehingga p99 lebih penting daripada median. Hasil kita sendiri menunjukkannya pada skala kecil: p50 server 839.9 ms, p99 985.0 ms, maksimum 1010.6 ms. Kedua, **menurun secara anggun alih-alih gagal**: setiap tahap membutuhkan tenggat dan *fallback* yang lebih murah, itulah sebabnya `GET /api/recommendations` mengembalikan daftar popularitas dengan `fallback: true` setelah *timeout* 5 detik. Ketiga, **simpan di *cache* apa yang dibagi**: statistik tingkat item berubah lambat dan dapat dihitung sekali untuk semua pengguna.

Panduan kode: pseudo-code untuk bagian yang belum ada

Tidak ada kode repositori yang disertakan di sini, karena tidak satu pun komponen ini sudah ada di repositori. Sketsa di bawah adalah pseudo-code: perpaduan bisnis dari bagian multi-tujuan. Bandingkan alur *serving*-nya dengan `RecommenderService.Recommend`: bentuknya sama, ditambah *retriever* berbasis *user tower*, pencarian *feature store* secara *batch*, beberapa *head* tugas, dan *exposure log* yang ditulis di akhir.

```
# Pseudo-code: blending task heads into one score.
def final_score(p, cands, w=(1.0, 2.0, 3.0), ads_weight=0.0):
    base  = w[0]*p["click"] + w[1]*p["cart"] + w[2]*p["order"]
    boost = np.array([c.campaign_boost for c in cands])   # e.g. 0.1 in a sale
    ads   = ads_weight * np.array([c.bid for c in cands]) * p["click"]
    return base * (1.0 + boost) + ads
```

Bobot `(1.0, 2.0, 3.0)` menggemakan skema label buku ini (klik 1, keranjang 2, pesanan 3) tetapi kini *eksplisit dan dapat disetel*.

Jalankan: papan peringkat buku

Gambar bab ini mengumpulkan setiap metode yang dievaluasi dalam buku ini pada pengguna simulasi yang sama, label yang sama, dan jendela evaluasi 7 hari yang sama. Hasilkan ulang gambar itu, beserta perbandingan utama yang diringkasnya, dengan:

```
cd book
uv run python tools/make_figures.py ch14
```

Gambar itu membaca `results/*.json` dari setiap skrip pendamping. Tiga baris teratasnya berasal dari perbandingan di Bab 10, yang terlihat seperti ini:

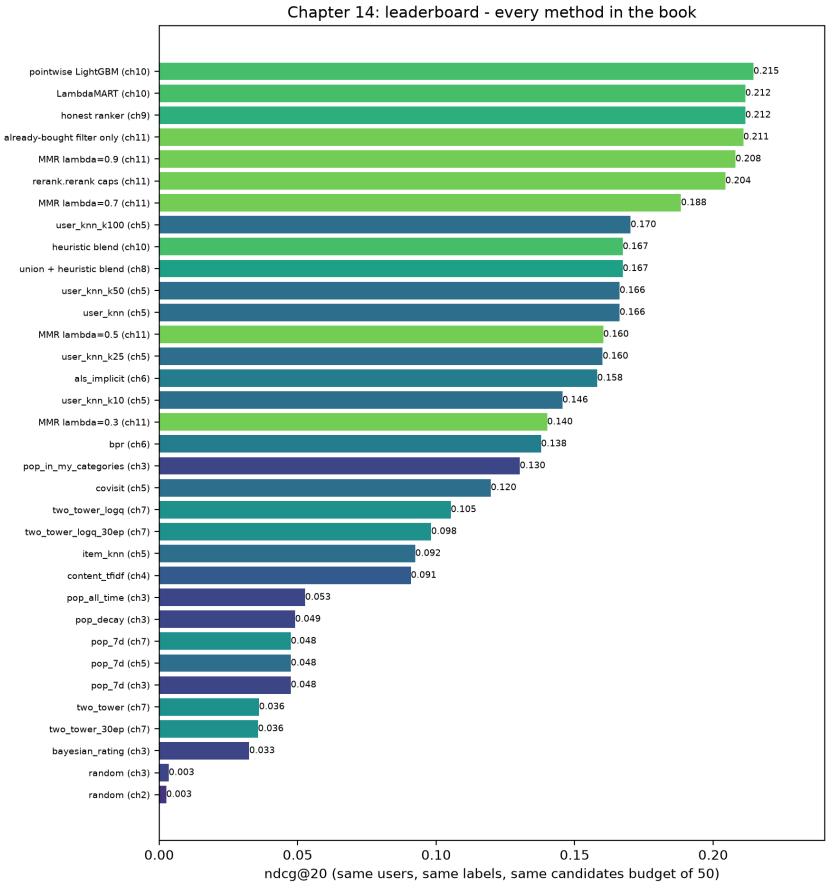
KELUARAN TERMINAL

```
=== Three ways to order the same candidates ===
                    recall@5  recall@10  recall@20  recall@50  order_recall@20  ndcg@20
coverage@20
method
heuristic blend       0.1153     0.1769     0.2526     0.3753           0.3978   0.1674
0.5682
pointwise LightGBM    0.1577     0.2176     0.2836     0.3915           0.4250   0.2146
0.4777
LambdaMART            0.1503     0.2015     0.2725     0.3824           0.4210   0.2116
0.4245

candidate recall ceiling: 0.4645
```

Heuristic blend, model *pointwise*, dan LambdaMART semuanya mengurutkan kandidat yang *sama*, yang batas atas recall-nya adalah 0.4645. Batas atas itulah sebabnya bahkan pemeringkat terbaik hanya mencapai recall@20 yang sederhana.

Hasil: membaca papan peringkat



Gambar 14.2: *Setiap metode dalam buku pada satu sumbu: ndcg@20 pada pengguna, label, dan anggaran kandidat yang sama.*

Grafik ini menceritakan kisah seluruh buku dalam satu gambar, dan sebaiknya dibaca dari bawah ke atas.

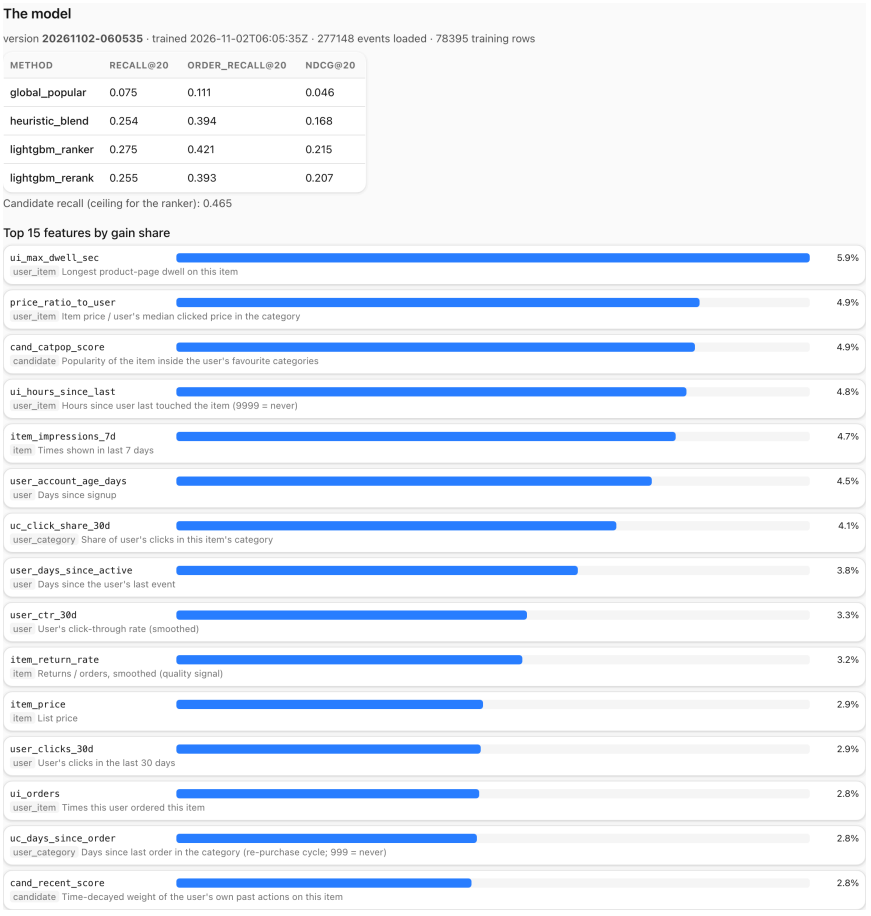
Baseline acak dan rating-saja (0.003 dan 0.033) adalah lantainya. Rating rata-rata Bayesian saja, tanpa gagasan tentang apa yang sebenarnya dibeli orang, hanya sedikit di atas peluang acak. Keluarga popularitas mendarat di sekitar 0.048 hingga 0.053. Popularitas jendela 7 hari, popularitas sepanjang waktu, dan versi dengan peluruhan nyaris tak bisa dibedakan, yang merupakan hasil jujur seperti yang dilaporkan Bab 3.

Model konten dan kolaboratif naik ke rentang 0.09 hingga 0.17. Pemfilteran konten TF-IDF mencapai 0.091, item-kNN 0.092, ALS 0.158, dan user-kNN dengan 100 tetangga 0.170. Keluarga *retriever* personal ini kira-kira melipattigakan baseline popularitas. Perhatikan bahwa *retriever* ko-visitasi sederhana (0.120) sudah lebih kuat daripada model two-tower terbaik (0.105), dan bahwa model two-tower baru sampai di sana setelah koreksi logQ dari Bab 7: lari tanpa koreksi menghasilkan 0.036. Model neural tidak otomatis lebih baik. Ia membutuhkan fungsi *loss* yang tepat, dan pada 2,495 item ia hanya punya sedikit ruang untuk bersinar.

Sistem multi-tahap menempati bagian atas grafik. *Heuristic blend* atas gabungan sumber kandidat mendapat 0.167. Pemeringkat terlatih atas kandidat yang sama mencapai 0.212 (LambdaMART) dan 0.215 (LightGBM pointwise). Pemeringkat jujur bebas-*leakage* dari Bab 9 (0.212) berada di tingkat yang sama, sementara kembarannya yang bocor dari bab yang sama mendapat 0.5035, itulah

sebabnya ia sengaja tidak ada di peringkat: ia adalah bug, bukan metode. Urutan batang hijau mengingatkan dua pelajaran sebelumnya: selisih antara dua tujuan pemeringkat itu sekitar satu simpangan baku antar-*seed* (Bab 10), dan baris-baris MMR dengan λ di bawah 1 *kehilangan* ndcg sebagai ganti diversitas (Bab 11).

Dua peringatan berlaku untuk setiap kesimpulan dari grafik ini. Pertama, datanya simulasi, dan model preferensi tersembunyinya lebih sederhana daripada dunia nyata. *Urutan* metode lebih bisa dipindahkan daripada nilai absolutnya. Kedua, ini adalah angka *offline* pada satu pembagian mingguan. Angka-angka ini membenarkan untuk mencoba sebuah metode dalam uji A/B sungguhan. Mereka tidak pernah menggantikan uji tersebut.



Gambar 14.3: *Halaman model pada aplikasi yang berjalan: versi, metrik evaluasi dari pemeringkat yang paling baru dilatih, dan fitur yang paling diandalkannya.*

Tangkapan layar di atas adalah panel "The model" pada `/how-it-works`: versi model, jumlah *event* yang dimuat, dan fitur teratas menurut porsi *gain*. Pada skala besar ia menjadi *model registry* dengan riwayat antar versi.

Jebakan dan cara produksi menanganinya

Menambah skala sebelum perlu. Menyalin arsitektur perusahaan besar untuk masalah kecil menambah beban operasional dan gangguan baru. Jika *event* Anda muat dalam satu database dan model dilatih ulang dalam hitungan menit, rancangan prototipe sudah benar; migrasilah ketika ada hambatan konkret.

Model deep tanpa baseline yang kuat. Pertahankan pemeringkat pohon *boosted* sebagai tolok ukur dan *fallback*; rilis model deep hanya jika ia menang secara online dengan selisih cukup untuk membenarkan biayanya.

Apa yang dibangun berikutnya di repositori ini

Jika Anda ingin melanjutkan proyek ini, berikut daftar periksa berprioritas. Setiap butir cukup kecil untuk dikerjakan dalam satu akhir pekan dan mengajarkan satu gagasan dari bab ini.

1. **Cache frame tingkat item dan saring event per pengguna.** Profil di Bab 13 membagi permintaan menjadi dua biaya: *frame* tingkat item (`item_features`, `covisitation_matrix`) yang dihitung ulang pada setiap panggilan, dan agregasi sisi pengguna atas event semua pengguna padahal hanya satu pengguna yang meminta, yang porsinya lebih besar. Simpan *frame* tingkat item di *cache* dengan TTL singkat, dikunci pada waktu *cache*, dan saring event ke pengguna yang meminta sebelum agregasi sisi pengguna. Ukur setelah setiap bagian; tidak ada yang sendirian menghilangkan biayanya.
2. **Catat eksposur.** Buat `Recommend` menulis satu baris per item yang ditampilkan (id permintaan, versi model, peringkat, skor, fitur). Latih pada eksposur alih-alih merekonstruksinya, dan gunakan untuk pembobotan *propensity*.
3. **Tambahkan holdout dan kerangka A/B mini.** *Hash* `user_id` menjadi *bucket*; kirim 5% ke *fallback* popularitas; catat *bucket* pada setiap *event*; bangun halaman yang menerapkan uji-z dari bab ini pada konversi pesanan.
4. **Pecah label menjadi beberapa head.** Latih model klik, keranjang, dan pesanan terpisah (tiga model LightGBM sudah cukup), ekspos bobot campuran di konfigurasi, dan bandingkan dengan label bertingkat tunggal.
5. **Ganti poller dengan antrean.** Minta Rust menerbitkan *event* ke *broker*, dan minta `Store` mengonsumsi darinya. Pertahankan Postgres sebagai *system of record*. Ini langkah jujur terkecil menuju arsitektur *streaming*.
6. **Tambahkan monitor drift.** Implementasikan pemeriksaan *drift* (misalnya *population stability index*) untuk 15 fitur teratas menurut kepentingan dan tampilkan di halaman `/how-it-works`.
7. **Tambahkan retriever two-tower sebagai sumber kelima.** Gunakan indeks item dari Bab 7 dengan koreksi logQ, dan periksa dengan skrip Bab 8 bahwa ia menaikkan recall gabungan sebelum Anda membayarnya dalam latensi.
8. **Lindungi penjual baru.** Tambahkan batas bawah eksposur untuk toko yang terdaftar dalam 7 hari terakhir dan ukur biayanya dalam ndcg, seperti yang dilakukan Bab 11 untuk diversitas.

> **Tips**
>
> Kerjakan daftar periksa secara berurutan. Butir 1 sampai 3 memberi Anda perangkat *pengukuran* (kecepatan, log, eksperimen). Tanpa itu Anda tidak bisa tahu apakah butir 4 sampai 8 membantu.

Tahukah kamu?

Pelatihan waktu nyata di produksi. Sistem Monolith milik ByteDance berargumen bahwa kerangka kerja serbaguna memisahkan pelatihan *batch* dari *serving* sehingga tidak dapat belajar dari umpan balik pelanggan secara waktu nyata. Ia memperkenalkan tabel *embedding* tanpa tabrakan (dengan *embedding* yang dapat kedaluwarsa dan penyaringan frekuensi untuk membatasi memori) dan arsitektur pelatihan online, dan telah diterapkan di BytePlus Recommend.

Sumber: Liu et al., arXiv, 2022

Ringkasan

- Menskalakan rekomender mempertahankan pipeline (ambil, peringkatkan, peringkatkan ulang) dan mengganti setiap panah dengan layanan: *event* melewati log seperti Kafka, dan job *streaming* menjaga fitur tetap segar.
- *Feature store* mendefinisikan setiap fitur sekali, dengan materialisasi *as-of* offline untuk pelatihan dan materialisasi online untuk *serving*, yang mencegah *training/serving skew*. Mencatat fitur yang disajikan adalah pertahanan kedua.
- Pada satu miliar item Anda melakukan *shard*, kompresi (*product quantisation*), dan memakai indeks ANN; indeks float32 64 dimensi berukuran 256 GB sebelum kompresi, sehingga aritmetika memori menentukan rancangan.
- Pemeringkat deep menambahkan apa yang tidak bisa dilakukan pohon: *embedding* untuk id jarang (Wide & Deep, DCN), atensi sadar-kandidat atas riwayat (DIN), dan beberapa *head* untuk klik, keranjang, dan pesanan (MMoE).
- Memadukan *head* menjadi satu skor adalah keputusan kebijakan: buat bobot, suku iklan, dan aturan keadilan penjual eksplisit dan ukur secara terpisah.
- Uji A/B membutuhkan metrik yang dideklarasikan sebelumnya, ukuran sampel yang dihitung di muka, pemeriksaan rasio sampel, dan satu kali pengamatan terencana. Contoh yang dikerjakan menunjukkan bagaimana $z \approx 1.965$ hipotetis hanya signifikan secara marginal, dan mengapa sekitar 200,600 pengguna per varian dibutuhkan.
- Pantau *drift* fitur, distribusi skor, hasil menurut posisi dan aktivitas, serta kesegaran, terhadap sebuah *holdout*. Anggarkan latensi berdasarkan p99, beri setiap tahap tenggat dan *fallback*.
- Papan peringkat menyatakan temuan buku ini secara *offline*: *retrieval* personal mengalahkan popularitas, dan pemeringkat terlatih atas kandidat multi-sumber mengalahkan semua yang lain. Eksperimen langsung menentukan apakah peningkatannya nyata.

Kuis

Q14.1 *Training/serving skew* paling langsung muncul ketika:

A. Set pelatihan lebih besar daripada set *serving*.
B. Sebuah fitur dihitung dengan kode atau filter yang berbeda pada saat pelatihan dan pada saat *serving*.
C. Model dilatih ulang terlalu sering.
D. Indeks ANN mengembalikan hasil hampiran.

Q14.2 Apa yang dilakukan *local activation unit* pada DIN?

A. Ia memampatkan seluruh riwayat pengguna menjadi satu vektor berpanjang tetap tanpa memandang kandidat.
B. Ia memberi bobot pada perilaku lampau pengguna menurut relevansinya terhadap item kandidat spesifik yang sedang dinilai.
C. Ia memilih sumber *retrieval* mana yang dipakai untuk tiap pengguna.
D. Ia menghilangkan bias posisi dari label pelatihan.

Q14.3 Pada model *multi-gate mixture-of-experts* (MMoE), apa tujuan adanya gerbang terpisah per tugas?

A. Agar setiap tugas (klik, keranjang, pesanan) dapat memilih kombinasi berbobot sendiri dari para ahli bersama.
B. Untuk melatih setiap tugas pada dataset yang berbeda.
C. Untuk menjamin semua tugas memiliki *loss* yang sama.
D. Untuk men-*cache* prediksi antar tugas.

Q14.4 Pada hasil latensi di Bab 13, p50 server adalah 839.9 ms dan p99 adalah 985.0 ms. Mengapa tim produksi menganggarkan berdasarkan p99 dan bukan p50, dan apa yang dilakukan repositori ini ketika sebuah permintaan terlalu lambat?

Q14.5 Jelaskan mengapa posisi yang lebih baik pada papan peringkat *offline* saja tidak membenarkan perilisan sebuah model. Gunakan pemeringkat bocor (ndcg@20 0.5035 dibandingkan 0.2116 untuk yang jujur) sebagai salah satu contoh, dan sebutkan satu alasan lain.

Q14.6 Sebuah uji A/B hipotetis mengirim 50,000 pengguna ke setiap varian. Kontrol memiliki 5,000 pesanan dan perlakuan memiliki 5,150. Hitung proporsi gabungan, galat baku, statistik z, dan putuskan pada $\alpha = 0.05$ (dua sisi) apakah peningkatannya signifikan. (Nilai-p dua sisi untuk $z = 1.57$ sekitar 0.116.)

Q14.7 Sebuah katalog memiliki 2 miliar item. Setiap vektor item memiliki 128 dimensi yang disimpan sebagai *float* presisi-setengah 2-byte. Berapa GB yang dibutuhkan indeks mentahnya? Jika *product quantisation* memampatkan tiap vektor menjadi 32 byte, berapa GB indeks terkompresnya?

Q14.8 Baca sketsa `final_score` dalam bab ini, dengan `w=(1.0, 2.0, 3.0)`, `ads_weight=0.0`. Sebuah item memiliki `p_click = 0.2`, `p_cart = 0.05`, `p_order = 0.01`, dan `campaign_boost = 0.1`. Berapa skor akhirnya? Berapa skornya jika *campaign boost* bernilai 0?

Q14.9 Anda merencanakan uji A/B 50/50 dan, setelah sehari, mengamati 50.6% pengguna di kontrol dan 49.4% di perlakuan, dari beberapa juta pengguna. Apa yang ditunjukkan hal ini, dan apakah Anda harus membaca hasil konversinya?

A. Itu derau yang wajar; baca hasilnya.
B. Itu *sample ratio mismatch*, menandakan bug pada pengacakan atau pencatatan; selidiki sebelum memercayai hasil apa pun.
C. Itu membuktikan bahwa perlakuan mengurangi trafik.
D. Itu berarti ukuran sampel terlalu kecil.

Q14.10 *Praktik langsung.* Buka `book/code/ch11_rerank.py` dan tambahkan `0.8` ke daftar `LAMBDAS` (di antara 0.9 dan 0.7). Jalankan dari `book/` dengan `uv run python code/ch11_rerank.py`. Nyatakan apa yang Anda harapkan untuk ndcg@20 dan jumlah kategori berbeda pada baris baru relatif terhadap baris 0.9 dan 0.7, lalu periksa, dan katakan apakah hal ini mendukung klaim dalam bab ini bahwa diversitas harus dibayar dengan relevansi.

Lampiran A — Kunci jawaban kuis

Jawaban untuk setiap kuis di akhir bab, lengkap dengan penjelasan singkat. Latihan praktik menjelaskan apa yang seharusnya Anda amati; angka persis Anda bergantung pada perubahan yang Anda buat.

Bab 1 — Apa yang Dilakukan Sistem Rekomendasi

Q1.1 — B. Retrieval dengan murah menyaring jutaan item menjadi beberapa ratus kandidat dan dinilai berdasarkan recall. Opsi A menggambarkan ranking, C menggambarkan re-ranking, dan D tidak berkaitan dengan serving. Apa pun yang terlewat oleh retrieval tidak akan pernah bisa dipulihkan oleh tahap-tahap setelahnya.

Q1.2 — C. Dwell time dan add-to-cart adalah produk sampingan dari perilaku normal, bukan pernyataan pendapat yang disengaja. Rating bintang, ulasan tertulis, dan ketukan jempol ke atas semuanya adalah umpan balik eksplisit karena pengguna memang memilih untuk menyatakan preferensi.

Q1.3 — B. Platform sungguhan hanya melihat log, sehingga simulator hanya menulis apa yang akan dicatat oleh sebuah aplikasi dan merahasiakan variabel laten. Karena itu, metode harus memulihkan sinyal dari jenis data yang sama dengan yang akan mereka lihat di produksi. Karena kebenaran dasarnya ada, kita juga tahu bahwa datanya memang dapat dipelajari, yang membantu saat mendiagnosis metode yang gagal.

Q1.4 — CTR adalah klik dibagi impresi, dan jumlah impresi adalah penyebut yang menunjukkan seberapa sering sebuah item benar-benar ditampilkan. Jika hanya klik yang dicatat, item yang tampak populer mungkin hanya sangat sering ditampilkan, dan Anda tidak bisa membedakan "ditampilkan lalu diabaikan" dari "tidak pernah ditampilkan". Anda juga tidak akan punya negatif untuk pelatihan dan tidak bisa mempelajari position bias.

Q1.5 — Sparsity adalah fraksi sel pengguna-item yang tidak memiliki interaksi tercatat (di sini 99.388%). Setiap pengguna hanya menyentuh sangat sedikit item, sehingga sebuah algoritma harus menyimpulkan selera dari segelintir sel yang terisi. Akibatnya, metode berbagi kekuatan statistik antar pengguna dan item: metode neighbourhood (bab 5), faktorisasi low-rank (bab 6), dan fitur konten. Jawaban apa pun yang menyebut salah satu konsekuensi tersebut benar.

Q1.6 — Hari sale pada hari Sabtu memiliki bobot mentah $1.2 \times 3.0 = 3.6$. Hari gajian pada hari Minggu yang bukan hari sale memiliki $1.2 \times 1.4 = 1.68$. Flag dikembalikan karena loop simulasi memakainya kembali: pada hari sale, diskon efektif dinaikkan 0.2, dan probabilitas order memiliki bonus gajian dan sale, sehingga waktu mengubah perilaku, bukan hanya volume trafik.

Q1.7 — 251 order. Kalikan ketiga rasio: $0.1059 \times 0.4295 \times 0.5517 \approx 0.02509$ (2.51% impresi yang berakhir menjadi order), dan $10{,}000 \times 0.02509 \approx 250.9$, yang dibulatkan menjadi 251.

Q1.8 — Kepadatan = 50,000 / (1,000 × 10,000) = 50,000 / 10,000,000 = 0.5%, sehingga sparsity = 99.5%. Bahkan contoh yang "tampak padat" ini membiarkan 199 dari setiap 200 sel kosong.

Q1.9 — Setelah perubahan, skrip mencetak bahwa 50% teratas (1,250 item) menerima 92.8% interaksi, sehingga separuh bawah katalog hanya berbagi sekitar 7% darinya. Top 5% (125 item) mendapat 35.1%, yang terletak di antara 12.5% untuk top 1% dan 50.3% untuk top 10%. Ini menunjukkan betapa curam interaksi terkonsentrasi di bagian kepala dan betapa tipisnya bagian ekor, itulah sebabnya baseline popularitas kuat.

Q1.10 — Dua alasan yang baik: (1) pengguna mungkin tidak pernah menyadari item tersebut karena berada di bagian bawah daftar atau di luar layar (exposure dan position bias), sehingga tidak ada klik tidak berarti tidak ada minat; (2) pengguna mungkin tertarik tetapi memilih item lain lebih dulu atau berniat kembali nanti. Memperlakukan ini sebagai negatif keras mengajari model untuk mengulang apa yang ditampilkan sebelumnya dan menghukum item karena penempatannya yang buruk.

Bab 2 — Mengukur keberhasilan

Q2.1 — B. Dengan pengacakan acak, sebagian event test lebih tua daripada event pelatihan, sehingga model belajar dari masa depan (lonjakan popularitas, selera pengguna di kemudian hari, event lain dari sesi yang sama). A bukan masalahnya, C tidak masuk akal, dan D tidak berkaitan dengan cara data dipisahkan. Pemisahan berbasis waktu menghilangkan kebocoran ini.

Q2.2 — C. Kedua daftar berisi himpunan item yang sama, sehingga recall@10, precision@10, dan coverage identik. NDCG mendiskon berdasarkan posisi, sehingga daftar yang menempatkan item yang dipesan (gain 7) di urutan pertama mendapat skor lebih tinggi daripada yang menempatkannya di urutan terakhir.

Q2.3 — B. Candidate recall adalah porsi positif masa depan yang muncul di himpunan kandidat, sehingga tidak ada ranker yang hanya mengurutkan ulang kandidat tersebut yang bisa melampauinya. Ia tidak "selalu lebih rendah" daripada recall akhir (A salah: ia adalah batas atas), ia bukan ukuran keluasan (C adalah coverage), dan meningkatkannya membutuhkan retriever yang lebih baik, bukan ranker yang lebih baik (D).

Q2.4 — Karena AP juga harus menghukum positif yang tidak pernah muncul. Positif yang terlewat tidak menambah apa pun ke pembilang tetapi tetap dihitung di penyebut R. Membagi dengan jumlah hit akan memberi hadiah yang sama pada daftar yang menemukan satu item di peringkat 1 dan mengabaikan empat lainnya seperti pada daftar yang menemukan kelimanya.

Q2.5 — Dengan inner join (atau merge yang dimulai dari `recs`), positif masa depan yang tidak pernah direkomendasikan akan hilang dari tabel. Jumlah baris per pengguna akan menjadi jumlah hit, bukan R, sehingga recall menjadi hits/hits = 1 untuk setiap orang yang punya hit, dan pengguna tanpa hit akan lenyap alih-alih mendapat skor 0. Left merge dari label menjaga setiap positif tetap ada di penyebut.

Q2.6 — `dropna(subset=["pos"])` membuang positif masa depan yang tidak direkomendasikan di top k (`pos`-nya NaN setelah left merge), sehingga hanya hit yang berkontribusi pada DCG. Konstanta `+ 2` dipakai karena `pos` berbasis nol sedangkan rumus memakai peringkat i berbasis satu: $\log_2(i+1) = \log_2(\text{pos}+2)$. Memakai `+ 1` akan membagi dengan $\log_2 1 = 0$ pada slot pertama.

Q2.7 — DCG@3 = 4.131, IDCG@3 = 7.631, NDCG@3 = 0.541. DCG = $0/\log_2 2 + 1/\log_2 3 + 7/\log_2 4 = 0.631 + 3.5 = 4.131$. Urutan ideal menempatkan order di urutan pertama: $7/\log_2 2 + 1/\log_2 3 = 7 + 0.631 = 7.631$. Rasionya $4.131/7.631 \approx 0.541$.

Q2.8 — precision@5 = 0.4 dan recall@5 = 0.5. Dua hit dalam lima slot menghasilkan $2/5 = 0.4$; dua dari empat positif masa depan ditemukan, menghasilkan $2/4 = 0.5$.

Q2.9 — NDCG adalah rata-rata kualitas ranking atas semua pengguna dan tidak melihat keluasan atau batas atas pipeline. Coverage menyingkap model yang menampilkan segelintir bestseller yang sama kepada semua orang (NDCG baik pada pekan yang didominasi popularitas, tetapi hanya sebagian kecil katalog yang pernah ditampilkan). Candidate recall menyingkap bahwa tahap pertama tidak pernah mengambil banyak item yang kemudian dibeli pengguna, sehingga ranker tidak punya apa-apa untuk dipilih dan peningkatan NDCG dari penyetelan ulang akan terbatas; NDCG saja tidak memberi tahu Anda apakah hambatannya ada di retrieval atau ranking.

Q2.10 — Coverage@20 seharusnya turun menjadi sekitar $20/2311 \approx 0.0087$, karena hanya 20 item berbeda yang pernah ditampilkan ke semua pengguna (jumlah item yang tersedia adalah 2311 pada output yang tercetak). Angka akurasi akan berbeda dari run acak, tetapi tidak akan dipersonalisasi. Latihan ini menunjukkan bahwa coverage dan akurasi adalah sumbu yang terpisah: acak punya coverage 1.0 dengan akurasi hampir nol, dan daftar konstan punya coverage hampir nol.

Bab 3 — Baseline popularitas

Q3.1 — B. Dalam simulator, setiap item memiliki kualitas tersembunyi yang tetap, sehingga popularitas tidak bergeser. Ketika popularitas yang mendasari stabil, hitungan 53 hari adalah estimasi yang lebih sedikit noise-nya

daripada hitungan 7 hari. Di marketplace nyata dengan flash sale dan musiman, jendela pendek sering menang, jadi setel pada data held-out.

Q3.2 — C. Dengan $v = 0$, bobot pada rating item itu sendiri, $v/(v+m)$, adalah nol, sehingga skor sama dengan rata-rata global C (4.324 pada cetakan). Nilai 4.4 atau 4.5 yang disebutkan hanyalah placeholder yang ditulis untuk item baru. Smoothing memperlakukannya sebagai "tidak ada informasi".

Q3.3 — B. Coverage@20 adalah jumlah item berbeda yang direkomendasikan dibagi katalog yang tersedia. Peringkat global memberikan 20 item yang sama kepada semua orang, sehingga hanya 20 item yang pernah ditampilkan. Inilah akar popularity bias dan feedback loop.

Q3.4 — Berikan `num` = total bintang (R · v), `den` = jumlah rating v, `prior` = rata-rata global C, dan `strength` = m. Fungsi kemudian mengembalikan (R·v + C·m)/(v + m). Memecah pecahan tersebut menghasilkan v/(v+m)·R + m/(v+m)·C, yaitu rata-rata Bayesian. Panggilan yang sama berlaku untuk CTR (klik, impresi, CTR global, pseudo-impresi).

Q3.5 — Item yang ditampilkan di atas mendapat lebih banyak impresi dan klik, yang menaikkan skor popularitasnya, yang menjaganya tetap di atas, sementara item yang tidak pernah ditampilkan tidak bisa mengumpulkan bukti. Hal-hal berikut melemahkannya: personalisasi, eksplorasi (ε-greedy, Thompson sampling), rasio yang dinormalisasi terhadap exposure seperti CTR yang di-smooth, koreksi position bias, atau re-ranking untuk diversitas.

Q3.6 — Tidak. Filternya adalah `age_days < window_days`, ketat, sehingga event dengan usia tepat 7.0 dikecualikan. Setiap interaksi yang lolos mempertahankan bobot penuh `w` (1, 3, atau 5) tanpa peluruhan, karena ini adalah jendela keras. Mengganti `<` menjadi `<=` akan menyertakan event berusia tepat 7 hari; untuk menyertakan seluruh hari ke-8 Anda akan memanggilnya dengan `window_days=8` (atau memakai `<= 8`).

Q3.7 — Sekitar 4.98. Order: 5 · 2^(−7/7) = 5 · 0.5 = 2.5. Klik: 1 · 2^(−14/7) = 0.25. Add-to-cart: 3 · 2^(−3/7) ≈ 3 · 0.743 = 2.229. Jumlah = 2.5 + 0.25 + 2.229 ≈ 4.98.

Q3.8 — Dengan v = 20: 20/120·4.8 + 100/120·4.0 = 0.8 + 3.333 = 4.133. Dengan v = 400: 400/500·4.8 + 100/500·4.0 = 3.84 + 0.80 = 4.64. Semakin banyak rating menggeser estimasi dari prior (4.0) menuju rata-rata item itu sendiri (4.8).

Q3.9 — Half-life = 10 hari / 2 = 5 hari, karena 25% = 2^(−2), yaitu dua half-life telah berlalu pada usia 10. Half-life yang lebih kecil memusatkan bobot pada event terbaru, sehingga skor bereaksi cepat terhadap perilaku baru tetapi bertumpu pada lebih sedikit event efektif, yang membuatnya lebih berisik.

Q3.10 — Dengan half-life yang sangat pendek (2 hari), pop_decay berperilaku seperti jendela pendek: perilakunya seharusnya bergerak menuju baris pop_7d yang berumur pendek, karena bertumpu pada sedikit event efektif. Dengan half-life yang sangat panjang (30 hari), bobot mendekati 1 untuk seluruh riwayat 53 hari, sehingga pop_decay mendekati pop_all_time. Ini sesuai dengan penjelasan bahwa, dengan popularitas simulasi yang stabil, lebih banyak riwayat efektif berarti lebih sedikit noise. Nilai persisnya bergantung pada run Anda; urutannya yang perlu diperiksa.

Bab 4 — Content-based filtering

Q4.1 — B. IDF besar untuk term yang langka dan kecil untuk term yang muncul di banyak dokumen, sehingga kata seperti "sneakers" bernilai lebih daripada "home". Preferensi panjang (A) bukan tujuan; normalisasi justru menghilangkannya, dan TF-IDF tidak membuat vektor menjadi ortogonal atau menghapus stop-word dengan sendirinya.

Q4.2 — B. Untuk vektor satuan, penyebut cosine adalah 1, sehingga cosine sama dengan dot product, itulah sebabnya satu perkalian sparse `X @ X.T` menghasilkan semua kemiripan berpasangan. Ia bukan jarak Euclidean (meskipun keduanya monoton terkait untuk vektor satuan), ia non-negatif untuk vektor TF-IDF, dan normalisasi menghilangkan ketergantungan pada panjang teks.

Q4.3 — C. Scorer content-based hanya membutuhkan atribut item itu sendiri, yang ada sejak saat item didaftarkan. Popularitas, co-visitation, dan metode berbasis order semuanya membutuhkan interaksi yang belum dimiliki item baru.

Q4.4 — Kedua item memiliki teks yang identik setelah tokenisasi (kata judul, kategori, dan merek yang sama), sehingga vektor TF-IDF mereka identik dan sudut di antaranya nol. Rekomender yang murni berbasis konten tidak bisa membedakan keduanya dan mungkin mengisi slot teratas dengan salinan produk yang sama; memecah seri membutuhkan sinyal perilaku seperti harga, rating, popularitas, atau kualitas penjual, atau langkah diversitas.

Q4.5 — Over-specialisation berarti sistem hanya merekomendasikan item yang mirip dengan riwayat pengguna, sehingga tidak pernah mengejutkan mereka atau menjangkau kebutuhan baru. Solusinya: padukan dengan sinyal kolaboratif seperti co-visitation, lakukan re-rank untuk diversitas (MMR atau batas per kategori), atau sisihkan slot untuk eksplorasi. Dua di antaranya sudah cukup.

Q4.6 — Tanpa guard, pengguna tanpa event positif memiliki `wsum == 0` dan `1 / wsum` menjadi tak hingga, sehingga mengalikan baris yang seluruhnya nol dengannya menghasilkan NaN. Dengan guard, pembaginya 1, baris `W @ X` tetap nol semua, dan `normalize` meninggalkan vektor nol. Profil menjadi nol semua dan setiap item mendapat skor 0, itulah sebabnya skrip menambahkan tie-break popularitas kecil untuk pengguna-pengguna ini.

Q4.7 — 0.571. Item A memiliki bobot $1 \times 2\hat{}0 = 1$. Item B memiliki bobot $3 \times 2\hat{}(-14/7) = 3 \times 0.25 = 0.75$. Porsi A adalah $1 / (1 + 0.75) = 0.5714$, yaitu 0.571. Event cart yang lebih lama, meskipun kekuatannya lebih tinggi, berkontribusi lebih sedikit daripada klik yang baru.

Q4.8 — 0.48. Vektornya berpanjang satu, sehingga cosine adalah dot product: $0.6 \times 0.8 + 0.8 \times 0 + 0 \times 0.6 = 0.48$. Keduanya hanya berbagi dimensi pertama, itulah sebabnya kemiripannya sedang, bukan tinggi.

Q4.9 — Pengamatan yang diharapkan: half-life yang sangat pendek seperti 1 hari membuang sebagian besar riwayat, sehingga profil bertumpu pada sedikit event dan kualitas biasanya turun; yang sangat panjang seperti 30 hari mendekati rata-rata tanpa bobot dan mungkin lambat bereaksi, sehingga half-life moderat di dekat baseline 7 hari biasanya terbaik. Angka aktual Anda mungkin berbeda, karena minat dalam simulator cukup stabil. Apa pun yang Anda amati, bandingkan ndcg@20 dan recall@20 dengan 0.0909 dan 0.1378 dan perlakukan perbedaan dalam beberapa perseribu dengan hati-hati.

Bab 5 — Collaborative filtering

Q5.1 — B. CF berbasis pengguna mengasumsikan bahwa orang yang sepakat di masa lalu kemungkinan akan sepakat lagi, sehingga item yang disukai pengguna dengan riwayat yang beririsan adalah rekomendasi yang baik. Ia tidak melihat judul (A), ia tidak hanya soal popularitas (C), dan ia tidak mengasumsikan selera statis (D) — itulah sebabnya peluruhan waktu dipakai di bagian lain buku ini.

Q5.2 — C. CF item-ke-item menghitung tabel item-serupa item–item yang mahal secara offline. Saat request, ia hanya mencari tetangga dari item di riwayat pengguna, sehingga biayanya tidak bergantung pada ukuran katalog dan jumlah pelanggan. User-kNN harus membandingkan target dengan semua pengguna pada setiap request.

Q5.3 — B. Jaccard hanya bertanya "ada atau tidak", yang cocok untuk sinyal biner seperti keanggotaan sesi yang sama dan mencegah pengguna berat mendominasi lewat hitungan. Ketika bobot membawa informasi (order = 5 klik), cosine pada matriks berbobot mempertahankan informasi itu. Kedua ukuran bersifat simetris.

Q5.4 — Setiap baris dinormalisasi sehingga bobot keluar dari setiap item sumber berjumlah 1. Tanpa itu, pasangan yang melibatkan item yang sangat populer akan memiliki hitungan mentah yang besar dan mendominasi setiap daftar kandidat terlepas dari seberapa kuat item-item itu sebenarnya berhubungan. Normalisasi membuat bobot sebanding antar item sumber dan mengurutkan tetangga setiap item berdasarkan kekuatan relatif; ia juga membuat matriks menjadi asimetris.

Q5.5 — Kedua item memiliki sedikit interaksi dan beberapa pengguna bersama, sehingga cosine menggelembung akibat dukungan yang sangat kecil (kemiripan maksimum tabel adalah 1.000, biasanya satu pengguna bersama). Perbaikan: wajibkan jumlah minimum pengguna bersama (support threshold), kecilkan kemiripan dengan n/(n+λ), atau batasi/sampel datanya. Salah satunya sudah cukup.

Q5.6 — Ia menggabungkan semua event dari item yang sama dalam satu sesi menjadi satu bobot terkuat, sehingga me-refresh halaman sepuluh kali dihitung sekali. Dengan `.sum()`, item yang diklik, dimasukkan keranjang, dan dipesan dalam satu sesi akan dihitung 1+3+5 = 9, bukan 5, dan loop klik berulang atau bot akan menggelembungkan bobot pasangan, mengacaukan skala kekuatan yang dimaksudkan.

Q5.7 — Cosine 0.632, Jaccard 0.333. Dot product-nya 2·1 + 0·1 + 1·0 = 2; norm-nya √5 = 2.236 dan √2 = 1.414, sehingga cosine = 2/3.162 = 0.632. Himpunan penggunanya X = {u1, u3} dan Y = {u1, u2}: irisan 1, gabungan 3, sehingga Jaccard = 1/3 = 0.333.

Q5.8 — 0.55. Skor(C) = Σ recency(seed) · w(seed→C) = 0.5·0.3 + 1.0·0.4 = 0.15 + 0.40 = 0.55. Ini adalah penjumlahan dalam `covisit (cand_recent_score * w`, dikelompokkan dan dijumlahkan per pengguna dan kandidat). Dua seed yang menunjuk ke kandidat yang sama saling memperkuat.

Q5.9 — B. item_knn memiliki coverage@20 0.8399 (tertinggi), dan ndcg@20 0.0925, di bawah user_knn (0.1662) dan covisit (0.1197). user_knn tidak memiliki coverage tertinggi (A salah); recall@5 covisit (0.0795) di bawah milik user_knn (0.1230) (C salah); ndcg pop_7d (0.0475) di bawah milik item_knn (D salah).

Q5.10 — Dalam run percobaan, ndcg@20 berubah 0.1662 (k=50), 0.1702 (k=100), 0.1706 (k=200), 0.1700 (k=400) dan recall@20 0.2273, 0.2419, 0.2409, 0.2405: peningkatan berhenti setelah sekitar k=100 dan kurvanya datar atau sedikit turun. Setelah titik tertentu, tetangga tambahan hanyalah pengguna yang mirip secara lemah yang suaranya menambah noise alih-alih sinyal, dan bobot kemiripan membuat kontribusi mereka kecil.

Bab 6 — Faktorisasi matriks (*matrix factorisation*)

Q6.1 — B. Faktor laten adalah satu dimensi yang dipelajari dari vektor pengguna dan item. Tidak ada yang melabelinya sebelumnya; optimizer menemukannya karena memberi setiap pengguna dan item sebuah nilai pada dimensi itu membantu merekonstruksi interaksi. Ia bukan kolom matriks mentah, hyperparameter, atau atribut yang dimasukkan penjual.

Q6.2 — B. p_{ui} adalah target biner "apakah pengguna ini menunjukkan minat" ($\mathbb{1}[R_{ui} > 0]$), dan $c_{ui} = 1 + \alpha R_{ui}$ membobot seberapa penting galat kuadrat pada sel itu. Sel yang tidak teramati mempertahankan $c = 1$, sehingga mereka adalah nol yang lemah, sedangkan order dan cart adalah positif yang kuat.

Q6.3 — C. $Y^\top Y$ tidak bergantung pada pengguna, sehingga dihitung sekali per sweep. Koreksi $Y^\top (C^u - I) Y$ hanya memiliki baris non-nol untuk n_u item yang disentuh pengguna, sehingga biaya per pengguna berskala dengan riwayat mereka, bukan katalog. Loss-nya tetap tidak konveks secara bersama-sama, dan trik ini tidak mengatakan apa-apa tentang regularisasi atau rating eksplisit.

Q6.4 — Sel kosong sebagian besar berarti "belum dilihat", bukan "tidak disukai". Mencocokkannya sebagai nol sungguhan akan mengklaim bahwa hampir seluruh katalog dibenci oleh setiap pengguna dan akan mendorong model memprediksi hampir nol di mana-mana. ALS implisit mempertahankannya sebagai nol tetapi hanya memberinya confidence dasar $c = 1$, sementara sel yang teramati mendapat $c = 1 + \alpha R$, sehingga loss terutama peduli untuk menjelaskan interaksi yang kita yakini.

Q6.5 — Dengan vektor mendekati nol, setiap selisih $x_u \cdot (y_i - y_j)$ sekitar 0. Maka $\sigma(0) = 0.5$ dan loss $-\ln 0.5 = \ln 2 \approx 0.693$ untuk setiap triple. Melihat nilai ini pada epoch 1 adalah pemeriksaan kewarasan bahwa model dimulai sebagai lemparan koin tanpa informasi dan loss tersambung dengan benar.

Q6.6 — $x_u = 23/34 \approx 0.676$. Confidence-nya $c = 1 + 4R = (5, 13)$. Pembilang: $\sum c\,p\,y = 5 \cdot 1 \cdot 2 + 13 \cdot 1 \cdot 1 = 23$. Penyebut: $\sum c\,y^2 + \lambda = 5 \cdot 4 + 13 \cdot 1 + 1 = 34$. Pembagiannya menghasilkan 0.676.

Q6.7 — $\sigma(-\ln 3) = 1/(1+3) = 0.25$; loss $= -\ln 0.25 = \ln 4 \approx 1.386$; $s = \sigma(\ln 3) = 0.75$. Selisihnya negatif, sehingga pasangan tersebut berurutan salah (negatif mengalahkan positif). Pengali $s = 0.75$ besar, sehingga pembaruannya besar, yang persis merupakan perilaku yang diinginkan BPR untuk pasangan yang urutannya salah.

Q6.8 — `sig` sama dengan $\sigma(-\text{gap}) = 1 - \sigma(\text{gap})$, yaitu $-\partial\,\text{loss}/\partial\,\text{gap}$. Dengan aturan rantai, setiap gradien adalah `sig` dikali turunan gap terhadap parameter tersebut: $y_i - y_j$ untuk pengguna, x_u untuk item positif, dan $-x_u$ untuk item negatif. `gj` negatif karena menaikkan gap mengharuskan menurunkan skor item negatif, sehingga y_j bergerak menjauh dari vektor pengguna. Pasangan yang sudah berurutan benar memiliki `sig` mendekati 0 dan hampir tidak mengubah apa pun.

Q6.9 — Recall@20 hanya bertanya apakah item masa depan pengguna muncul di mana saja dalam top 20; NDCG juga memberi hadiah untuk menempatkannya di dekat puncak. ALS menemukan lebih banyak item yang benar, tetapi mengurutkannya kurang tajam daripada user-kNN. Profil itu cocok untuk generator kandidat (dinilai dengan recall) yang memasok tahap ranking berikutnya (dinilai dengan NDCG), seperti ranker LightGBM di repo ini.

Q6.10 — Angka persisnya bergantung pada run Anda (skrip di-seed dan deterministik), jadi bandingkan dengan ndcg@20 0.1581 untuk `als_implicit` di `results/ch06_mf.txt`. Harapkan `alpha=1.0` mengubah ndcg@20 secara nyata, karena dengan alpha kecil sel teramati dan tak teramati membawa confidence yang hampir sama dan model berperilaku berbeda; arahnya tidak dijamin, jadi periksalah. Harapkan `factors=8` turun jauh di bawah run 32-faktor: 8 dimensi tidak bisa memisahkan selera yang tersebar di banyak kategori dan ribuan item. Loss ALS akhir tidak sebanding antar nilai `alpha` karena loss itu sendiri berubah bersama bobotnya. Pelajarannya adalah bahwa menyapu hyperparameter terhadap label evaluasi berarti menyetel pada test set; gunakan jendela validasi yang lebih awal dan laporkan pekan test sekali saja.

Bab 7 — Embedding dan retrieval two-tower

Q7.1 — B. Membagi cosine dengan τ kecil meregangkan selisih antar skor, sehingga softmax menjadi lebih tajam dan loss terkonsentrasi di puncak ranking, menghukum negatif yang sulit. Softmax yang lebih datar (A) adalah yang dihasilkan temperatur tinggi. Temperatur tidak mengubah dimensi embedding atau isi batch.

Q7.2 — C. Sebuah batch adalah sampel acak pasangan positif, sehingga setiap item muncul sebanding dengan jumlah interaksinya. Item populer karenanya muncul sebagai negatif di banyak baris dan menerima dorongan ke bawah yang lebih besar, meskipun serving menilai semua item secara seragam. Normalisasi L2 menetapkan norm menjadi 1, sehingga bukan penyebabnya.

Q7.3 — A. Vektor item hanya bergantung pada atribut item, sehingga dapat dihitung sekali dan disimpan dalam indeks. Skornya adalah inner product, yang memang untuk itulah struktur ANN dibangun. Tower yang menerima riwayat pengguna sebagai input item (D) akan merusak ini.

Q7.4 — Ia mengurangi $\log q_j$, log dari probabilitas item j terambil ke dalam batch, dari logit kolom j. Item populer (q_j besar) mendapat dorongan kecil dan item langka dorongan besar, yang membatalkan dorongan ke bawah ekstra yang diterima item populer karena di-oversample sebagai negatif. Koreksi ini hanya dipakai saat pelatihan; saat serving dipakai dot product biasa.

Q7.5 — Ia mengukur berapa banyak dari top-20 brute-force yang tepat masih dikembalikan oleh pencarian aproksimasi. Indeks ANN hanya perlu mengaproksimasi pencarian eksak, sehingga kesetiaan terhadap brute force memisahkan galat indeks dari kualitas model. NDCG akan mencampur keduanya dan tidak bisa memberi tahu mana yang kehilangan akurasi.

Q7.6 — Ia mengganti kemunculan item target dalam riwayat pengguna itu sendiri dengan indeks padding, sehingga user tower tidak bisa melihat jawabannya. Jika dihapus, model bisa menyalin target dari riwayat, sehingga training loss akan turun lebih cepat dan tampak lebih baik, tetapi evaluasi pada pekan held-out, di mana jalan pintas seperti itu tidak ada, akan lebih buruk. Ini adalah contoh kecil target leakage.

Q7.7 — Sekitar 43,159 dot product, atau sekitar 0.43% dari brute force. Penilaian centroid berbiaya 4,096; setiap cluster memiliki $10{,}000{,}000/4{,}096 \approx 2{,}441$ item, sehingga 16 probe memindai $16 \times 2{,}441 = 39{,}062.5$. Totalnya $4{,}096 + 39{,}062.5 = 43{,}158.5$, dan $43{,}158.5/10{,}000{,}000 \approx 0.43\%$, kira-kira 230 kali lebih sedikit pekerjaan.

Q7.8 — Sebesar $\ln(0.04/0.0004) = \ln 100 \approx 4.605$ satuan logit. Koreksi menambahkan $-\ln q$: $-\ln 0.04 = 3.219$ untuk item populer dan $-\ln 0.0004 = 7.824$ untuk item langka, dan selisihnya 4.605. Dalam bentuk probabilitas, skor ter-eksponensial item yang lebih langka dikalikan 100 relatif terhadap item populer.

Q7.9 — Pengamatan yang diharapkan (verifikasi pada run Anda): spesifikasi `two_tower` tetap jauh di bawah `two_tower_logq` pada ndcg@20 di setiap temperatur, karena popularity bias berasal dari skema sampling, bukan temperatur. Temperatur mengubah ketajaman loss, sehingga ndcg dan terutama coverage@20 akan bergeser antar run; nilai yang sangat rendah seperti 0.02 cenderung lebih sulit dioptimalkan. Inti latihan ini adalah bahwa selisih logQ bertahan dan temperatur hanyalah tombol penyetelan, bukan obatnya. Pulihkan hasil dengan `git checkout book/results`.

Bab 8 — Pembangkitan kandidat multi-sumber

Q8.1 — B. Item yang tidak ada di himpunan kandidat tidak akan pernah mencapai ranker, sehingga tahap pertama dinilai dari berapa banyak positif masa depan yang dikandungnya. False positive murah karena tahap kedua, dengan fitur yang lebih kaya, menyingkirkannya. Ranker tidak membutuhkan input yang bersih (D salah), dan precision dapat dihitung pada umpan balik implisit (A salah).

Q8.2 — C, `catpop`. Unique pair recall-nya 0.0968, dibandingkan 0.0524 untuk `covisit`, 0.0298 untuk `recent`, dan 0.0253 untuk `global_trending`. Ia juga memiliki recall mandiri tertinggi (0.3274), karena menjangkau pengguna lewat klik dan pencarian.

Q8.3 — B. Positive rate naik dari 0.0090 menjadi 0.1817 seiring semakin banyak sumber yang sepakat, sehingga jumlah sumber dan skor setiap sumber adalah fitur yang informatif. Membuang kandidat bersumber tunggal (A, C) akan membuang sebagian besar recall, karena 76,612 kandidat hanya memiliki satu sumber. Menghitung sumber tidak bisa menggantikan ranker (D), yang juga memakai fitur pengguna, item, dan cross.

Q8.4 — C. Hanya `global_trending`. `recent` kosong tanpa riwayat, `covisit` di-seed dari `recent` sehingga juga kosong, dan `catpop` membutuhkan category affinity dari klik atau pencarian. Outer join hanya mengisi kolom skor yang hilang; ia tidak mengarang kandidat (A, D).

Q8.5 — Gabungan berisi rata-rata 103.94 kandidat per pengguna, sehingga memotong daftar berperingkat pada K=100 membuang kandidat di ujung daftar yang lebih panjang, dan sebagian di antaranya adalah positif. Batas atas 0.4645 adalah recall dari seluruh himpunan kandidat tanpa pemotongan. Recall@K hanya bisa mencapainya ketika K setidaknya sebesar ukuran pool untuk setiap pengguna.

Q8.6 — Menyembunyikan adalah umpan balik negatif eksplisit, dan fitur yang dipelajari hanya mengubah probabilitas: model masih bisa sesekali menampilkan item yang disembunyikan. Filter keras pada pool menjamin item itu tidak pernah muncul lagi, lebih murah (lebih sedikit baris untuk dinilai) dan mudah diaudit. Pengguna menoleransi rekomendasi yang tidak relevan tetapi tidak yang telah mereka tolak secara eksplisit.

Q8.7 — Nilainya akan menjadi 3, bukan 1. Setelah penukaran, ketiga kolom skor sudah terisi 0.0, sehingga `notna().sum(axis=1)` menghitung ketiganya untuk setiap baris, dan suku trending tidak menambah apa-apa karena item tidak ada dalam daftar itu. Fitur tersebut tidak lagi memisahkan kandidat bersumber tunggal dari multi-sumber dan akan kehilangan sinyal yang menaikkan positive rate dari 0.0090 menjadi 0.1817.

Q8.8 — Item yang kehabisan stok di top 30 akan dihapus setelah `nlargest`, menyisakan kurang dari 30 item trending, dan daftar akan menyusut setiap kali item populer habis terjual. Memfilter dulu lalu mengambil top 30 selalu mengembalikan 30 item yang tersedia (jika jumlahnya cukup).

Q8.9 — 12.1880 + 35.8321 + 54.9102 + 30.0000 = 132.9303, dikurangi ukuran gabungan 103.9366 menghasilkan 28.9937, yaitu sekitar 29.0 proposal (pengguna, item) duplikat per pengguna, yang merupakan 28.9937 / 132.9303 = 21.8% dari jumlah. Itulah tumpang tindih antar sumber, yang dihitung oleh `cand_n_sources`.

Q8.10 — Harapkan `cands_per_user` `catpop` naik (lebih banyak kategori digabungkan, hingga 5 × 20 item per pengguna dibanding 3 × 20) dan recall-nya naik sedikit, karena positif masa depan pengguna kadang berada di kategori ke-4 dan ke-5. Kandidat per pengguna dan recall gabungan naik sedikit, dengan hasil yang semakin berkurang karena kategori tambahan memiliki porsi affinity yang rendah. Ranker kemudian harus menilai lebih banyak baris per request, yang menaikkan latensi, dan sebagian besar kandidat tambahan adalah negatif, sehingga tugas ranker memisahkan positif dari negatif menjadi lebih sulit. Hasilnya ditimpa oleh run tersebut, karena itu `git checkout` untuk memulihkannya.

Bab 9 — Rekayasa fitur untuk pemeringkatan

Q9.1 — B. Point-in-time correctness berarti setiap fitur untuk sebuah contoh pelatihan dihitung hanya dari informasi yang ada sebelum waktu cutoff contoh tersebut. Normalisasi, jumlah event yang sama, dan frekuensi refresh adalah pilihan rekayasa yang tidak berkaitan. Melanggar aturan ini disebut leakage dan menggelembungkan metrik offline.

Q9.2 — B. Event yang ber-timestamp tepat pada cutoff termasuk jendela label `[cutoff, cutoff + days)`, yang dipilih `data.labels` dengan `ts >= cutoff`. Operator `<` yang ketat dalam `History.at` karenanya menempatkan setiap event tepat di satu sisi batas, sehingga tidak ada event yang sekaligus menjadi input fitur dan label.

Q9.3 — B. Dengan smoothing, satu impresi dan satu klik dicampur dengan `s` impresi imajiner pada rate global, sehingga estimasi mendarat dekat `p0`. Ia bergerak menjauh hanya ketika impresi nyata terkumpul dan mengalahkan prior. Rate tetap bergantung pada klik item itu sendiri; ia hanya mempercayainya sebanding dengan buktinya.

Q9.4 — Nol akan berarti "baru saja disentuh", yang merupakan sinyal recency terkuat, sehingga akan mengacaukan "tidak pernah" dengan "tepat menit ini". Sentinel besar seperti 9999 terletak jauh di luar rentang nyata, sehingga sebuah tree dapat memisahkan "tidak pernah disentuh" dari setiap nilai nyata dengan satu threshold. (Model linear dan neural membutuhkan flag missing yang eksplisit.)

Q9.5 — Fitur item mendeskripsikan item untuk setiap pengguna (mis. `item_ctr_30d`). Fitur pengguna mendeskripsikan pengguna terlepas dari item (mis. `user_clicks_30d`). Fitur cross adalah agregat atas sepasang kunci (mis. `ui_clicks`, `uc_click_share_30d`, `us_orders`). Fitur cross membawa sebagian besar personalisasi, karena hanya merekalah yang berbeda antara dua pengguna untuk item yang sama. Fitur item sebagian besar membawa sinyal kualitas.

Q9.6 — Ya, tes akan gagal. Tes menulis ulang setiap event pasca-cutoff menjadi order item 0 dalam tabel yang dimanipulasi. Fitur yang membaca seluruh tabel tanpa filter karenanya akan berubah nilainya (item 0 akan mendapat banyak order, dan item lain akan kehilangan miliknya), sehingga frame yang dibangun ulang akan berbeda dari aslinya. `pd.testing.assert_frame_equal(before, _build(tampered, cutoff))` membandingkan kedua frame sel demi sel dan melempar galat begitu satu sel berbeda.

Q9.7 — CTR yang di-smooth = (3 + 20·0.10) / (10 + 20) = 5/30 ≈ 0.167. CTR mentahnya 3/10 = 0.30. Smoothing menarik estimasi lebih dari setengah jalan kembali ke 0.10 global karena item hanya memiliki 10 impresi dibandingkan kekuatan prior 20.

Q9.8 — item_trend = (6/3 + 0.1) / (14/14 + 0.1) = 2.1 / 1.1 ≈ 1.91. Nilainya di atas 1, sehingga item tersebut sedang memanas: tiga hari terakhir rata-rata 2 klik per hari dibanding 1 klik per hari selama dua pekan terakhir.

Q9.9 — Yang diharapkan: dengan jendela leak 1 hari, fitur yang bocor hanya mencakup sepertujuh jendela label, sehingga sebagian besar positif mendapat nilai nol dan kebocoran melemah; NDCG@20 dari ranker yang bocor

seharusnya turun di bawah 0.5035 untuk leak 7 hari, dan gain share `leaky_future_clicks` juga seharusnya turun. Kemungkinan besar ia tetap jelas di atas 0.2116 dari model yang jujur, dan fitur tersebut tetap berperingkat dekat puncak. Bahkan sepotong kecil masa depan sudah cukup untuk membuat metrik offline tidak dapat dipercaya, itulah sebabnya satu-satunya aturan yang aman adalah cutoff keras tanpa pengecualian.

Bab 10 — Learning to rank

Q10.1 — B. Loss pointwise menilai setiap baris secara independen, loss pairwise membandingkan dua item dari pengguna yang sama, dan loss listwise melihat seluruh daftar berperingkat. Keluarga model (tree, jaringan saraf) ortogonal terhadap pembedaan ini, dan ketiganya dapat memakai label bergradasi maupun biner.

Q10.2 — C. Setiap tree baru dicocokkan ke residual, yang untuk squared error sama dengan gradien negatif loss pada prediksi saat ini. Prediksi kemudian diperbarui sebesar η kali nilai leaf tree tersebut.

Q10.3 — D. Perubahan NDCG dari menukar sepasang item besar di dekat puncak dan untuk selisih gain yang besar, sehingga pasangan tersebut mendapat lambda besar. Perkalian ini tidak membuat NDCG dapat diturunkan; ia menggantikan gradien dengan gradien yang didefinisikan secara manual.

Q10.4 — Argumen `group=` LightGBM adalah daftar ukuran grup yang berurutan, bukan id pengguna. Jika baris seorang pengguna tidak berdampingan, batas-batasnya memotong lintas pengguna, sehingga "query" mencampur baris pengguna yang berbeda dan pasangan serta NDCG dihitung atas daftar yang tidak bermakna. Biasanya tidak ada galat, hanya model yang lebih buruk, yang membuat bug ini sulit disadari.

Q10.5 — Sedikit positif per daftar (sekitar seratus kandidat, sedikit aksi) memberi loss yang sadar-daftar sedikit sinyal tambahan dibanding label biner sederhana; label bergradasi (order) jarang dan berisik; dan batas atas candidate recall menekan perbedaan antara ranker yang baik. Selisihnya sekitar satu standar deviasi seed (0.2147 vs 0.2125 rata-rata ndcg@20, std 0.0014 dan 0.0021). Saya hanya akan berpindah jika tes offline dengan data lebih besar dan banyak seed, atau A/B test online, menunjukkan keunggulan yang konsisten.

Q10.6 — `pred_contrib=True` mengembalikan satu kolom per fitur ditambah satu kolom bias terakhir (skor baseline). Mengiris `[:, :-1]` membuang bias sehingga hanya fitur yang tersisa. Tanpanya, kolom bias, konstanta besar seperti -2.9, sering menjadi nilai absolut terbesar dan akan dilaporkan sebagai "fitur" pada indeks di luar nama fitur, menyebabkan galat indeks atau penjelasan teratas yang palsu.

Q10.7 — 0.393. Gain-nya 1, 0, 7; IDCG = 7·1 + 1·0.6309 + 0 = 7.631. Menukar posisi 1 dan 3: selisih gain $7 - 1 = 6$, selisih diskon $1 - 0.5 = 0.5$, sehingga perubahan DCG adalah 3.0 dan $3.0/7.631 = 0.393$.

Q10.8 — Sekitar +1.47. Kontribusi ditambah bias sama dengan skor, sehingga total kontribusi adalah $1.277 - (-2.917) = 4.194$. Mengurangi delapan yang ditampilkan (2.726) menyisakan 1.468, yang dibulatkan menjadi +1.47 (±0.003 akibat pembulatan).

Q10.9 — C. Penurunannya sekitar 1.6 standar deviasi noise, yang bersifat sugestif, bukan definitif. "Efek persis" yang tetap (A) mengabaikan noise seed, ambang 0.01 (B) bersifat sembarang, dan tanda negatif (D) persis seperti yang dihasilkan oleh grup yang berguna ketika dihapus.

Q10.10 — Yang diharapkan (hipotesis, perlu diperiksa dengan menjalankannya): membuat order bernilai sama dengan klik menghilangkan hadiah ekstra untuk aksi terkuat, sehingga model mungkin meranking item yang dimasukkan keranjang dan dipesan kurang tajam dan `order_recall@20` bisa turun. Apakah ndcg@20 berubah lebih dari standar deviasi seed 0.0021 harus Anda nilai dari run Anda; perubahan yang lebih kecil adalah noise. Latihan ini mengajarkan bahwa `label_gain` menyandikan nilai bisnis dan bahwa perubahannya harus dinilai terhadap noise seed.

Bab 11 — Re-ranking, diversitas, dan aturan bisnis

Q11.1 — B. Dengan $\lambda = 1$, suku $(1 - \lambda) \max \text{sim}$ nol, sehingga setiap langkah memilih relevansi tertinggi yang tersisa. Hasilnya adalah urutan asli; inilah sebabnya baris `MMR lambda=1.0` dalam hasil sama dengan

baseline LambdaMART yang tidak dimodifikasi.

Q11.2 — C. Membeli kembali kopi atau popok adalah perilaku normal yang seharusnya didorong oleh rekomender, sedangkan pengguna yang sudah memiliki kulkas tidak membutuhkan kulkas yang persis sama lagi. Aturan tersebut karenanya membuang item hanya ketika `ui_orders > 0` dan `item_is_consumable == 0`.

Q11.3 — C. Thompson sampling menarik sampel dari posterior setiap item, sehingga item dengan sedikit data memiliki distribusi yang lebar dan kadang memenangkan slot, sedangkan item buruk yang terukur dengan baik hampir tidak pernah. Eksplorasi karenanya mengecil otomatis seiring bukti terkumpul, tanpa ε tetap.

Q11.4 — Aturan harus eksplisit, berbatas waktu, dan dapat diaudit, yang tidak bisa Anda jamin di dalam model yang dilatih. Jika impresi yang di-boost atau disponsori masuk ke data pelatihan sebagai bukti biasa, pelatihan ulang berikutnya akan membakar kampanye itu ke dalam model secara permanen dan mengacaukan estimasi relevansi. Dalam re-ranker, boost adalah suku yang terlihat dengan tanggal mulai dan akhir yang dapat dimatikan dan diuji tanpa pelatihan ulang.

Q11.5 — Cap menghapus item dan tidak melakukan backfill, sehingga untuk pengguna yang pool kandidatnya terkonsentrasi di beberapa kategori atau toko, daftar yang tersisa bisa lebih pendek dari 50, dan deep recall turun. `recall@5` hampir tidak berubah karena beberapa item teratas jarang mengenai cap. (Ini penjelasan yang paling mungkin; skrip tidak mencatat panjang daftar.)

Q11.6 — 8 item, dan cap per kategori yang bertanggung jawab. Ketiga puluh kandidat berbagi satu kategori, sehingga hanya yang posisinya dalam kategori di bawah `MAX_PER_CATEGORY = 8` yang bertahan; cap toko (paling banyak 3 item per toko di sini, di bawah 4) tidak menghapus apa pun, dan `head(20)` memiliki kurang dari 20 baris untuk dipertahankan.

Q11.7 — Urutan: item 1, item 3, item 2. Langkah 1: skornya $0.5\times$ rel $= 0.45, 0.40, 0.25$, sehingga item 1 (0.9) dipilih. Langkah 2: item 2 mendapat skor $0.5 \cdot 0.8 - 0.5 \cdot 1 = -0.10$ (kategori sama dengan item 1), item 3 mendapat skor $0.5 \cdot 0.5 - 0 = 0.25$, sehingga item 3 dipilih dan item 2 berada terakhir.

Q11.8 — Beta(6, 16), rata-rata $6/22 \approx 0.273$, standar deviasi ≈ 0.093. Variansnya $ab/((a+b)^2(a+b+1)) = 96/(484 \cdot 23) \approx 0.00862$. Item baru memiliki Beta(1, 1) dengan standar deviasi ≈ 0.289, sekitar tiga kali lebih besar, sehingga sampelnya jauh lebih bervariasi dan ia akan sering menarik cukup tinggi untuk memenangkan slot sampai data mempersempitnya.

Q11.9 — A. Hasil menunjukkan -0.0035 ndcg dan $+1.45$ kategori pada $\lambda = 0.9$, dan -0.0715 ndcg pada $\lambda = 0.3$. B salah (coverage@20 turun dari 0.4245 menjadi 0.3129), C salah (filter saja mengorbankan 0.0007), dan D salah (cap menaikkan kategori dari 5.33 menjadi 6.58).

Q11.10 — Yang diharapkan (belum dijalankan di sini): dengan kategori saudara tidak lagi dihukum, item dari grup induk yang sama tidak lagi didorong turun, sehingga ndcg@20 pada $\lambda = 0.5$ seharusnya lebih tinggi daripada 0.1603 dalam tabel, sementara rata-rata kategori berbeda bisa tetap tinggi, karena kategori berbeda tetap dihitung per kategori. Halaman bisa saja terkonsentrasi pada lebih sedikit grup induk (misalnya hanya Pets). Bandingkan angka yang Anda ukur dengan prediksi ini dan dengan baris baseline.

Bab 12 — Cold start, konteks, dan bias

Q12.1 — B. `global_trending` tidak menerima argumen pengguna dan `generate` melakukan cross-join hasilnya ke setiap id pengguna, sehingga bahkan pengguna tanpa riwayat, atau `user_id = 0` anonim, menerima item yang trending secara global. Ia tidak berbasis sesi, ia berjalan pada setiap request bukan hanya saat gagal, dan ia meranking berdasarkan interaksi terbaru berbobot, bukan diskon.

Q12.2 — C. Dengan $x = 0$ dan $n = 0$, rumus tereduksi menjadi $p_0 s / s = p_0$, CTR rata-rata katalog. Ini memperlakukan item baru sebagai rata-rata sampai ada bukti yang menyatakan sebaliknya, alih-alih sebagai nol atau tidak terdefinisi. Smoothing karenanya menghilangkan penalti karena tidak punya data.

Q12.3 — C. Ranker memprediksi apa yang akan dilakukan pengguna dalam 7 hari ke depan, dengan label yang diagregasi per pekan. Jam request identik untuk setiap kandidat dalam grup seorang pengguna dan tidak memiliki label per-request untuk dipelajari. Konteks termasuk dalam model kedua yang real-time yang dilatih pada log impresi.

Q12.4 — Ranker dilatih pada klik, sehingga item dengan sedikit lebih banyak klik mendapat skor lebih tinggi dan ditampilkan lebih sering. Exposure yang lebih banyak menghasilkan klik yang lebih banyak, yang melebarkan selisih, sementara item yang tidak pernah ditampilkan tidak pernah mendapat kesempatan membuktikan diri. Teknik yang mengurangi loop ini mencakup eksplorasi (epsilon-greedy atau Thompson sampling, menyisihkan slot untuk item yang belum terbukti), re-ranking diversitas, dan mencatat impresi beserta posisinya sehingga propensity weighting dapat mengoreksi exposure. Dua di antaranya sudah cukup.

Q12.5 — Rekomender yang lambat merugikan pengguna sama seperti yang mati, sehingga timeout membatasi waktu tunggu terburuk hingga lima detik lalu menyajikan sesuatu yang berguna. JSON membawa `"fallback": true` dan `"model_version": "fallback"`. Frontend membaca flag tersebut dan menampilkan banner "Recommender offline".

Q12.6 — Q berperingkat lebih tinggi. Query mengurutkan pertama berdasarkan jumlah event `order` dalam tujuh hari terakhir secara menurun (Q punya 1, P punya 0) dan hanya memakai `rating_count` untuk memecah seri. `i.id` terakhir membuat urutan deterministik ketika kedua kunci sebelumnya seri, sehingga request berulang mengembalikan daftar yang sama.

Q12.7 — CTR mentah: A adalah $40/200 = 0.20$ dan B adalah $20/200 = 0.10$. Relevansi yang dikoreksi bias: A adalah $0.20/0.8 = 0.25$ dan B adalah $0.10/0.2 = 0.50$ (setara dengan setiap klik pada A berbobot 1.25 dan setiap klik pada B berbobot 5). B dua kali lebih relevan daripada A meskipun CTR mentahnya separuh. Angka mentah itu menyesatkan karena B ditampilkan di posisi yang jarang diperiksa pengguna.

Q12.8 — Praktik. Harapkan bucket baru "0 clicks" berisi jauh lebih sedikit pengguna daripada bucket lain, sehingga metriknya berisik dan rasio `LambdaMART / pop_7d` di sana kurang bisa dipercaya dibanding rasio untuk bucket besar. Tanpa riwayat, ranker hanya bisa memakai kandidat trending dan fitur tingkat item, jadi perhatikan apakah keunggulannya atas `pop_7d` menyusut dibanding 5.22x pada bucket pengguna berat. Jika bucket-nya kosong, semua pengguna evaluasi skrip memiliki setidaknya satu klik sebelumnya dan barisnya tidak akan muncul. Intinya adalah kohort dingin itu kecil dan berisik.

Q12.9 — B. Klik pada posisi dengan propensity yang sangat kecil mendapat bobot $1/\theta_k$, yang bisa berupa puluhan atau ratusan, sehingga satu klik yang beruntung bisa mendominasi pelatihan. Clipping membatasi bobot, menukar bias kecil dengan pengurangan variansi yang besar. Propensity adalah probabilitas dan tidak bisa negatif, bobot tidak perlu berjumlah satu, dan LightGBM menerima bobot besar.

Bab 13 — Membangun sistemnya

Q13.1 — B. Transaksi default read-only berarti setiap `INSERT`, `UPDATE`, atau `DELETE` dari rekomender gagal dengan `ReadOnlySqlTransaction`. Ini melindungi sistem of record dari bug dalam kode fitur atau model dan menjaga API Rust sebagai satu-satunya penulis. Ia tidak mempercepat pembacaan dan tidak ada hubungannya dengan gRPC.

Q13.2 — A. Browser hanya boleh melaporkan `impression`, `click`, dan `hide`, yang merupakan pengamatan atas layar pembeli sendiri. `order` (dan `add_to_cart`) adalah konsekuensi dari aksi server-side yang terautentikasi, sehingga API menulisnya sendiri; menerimanya dari klien akan membiarkan siapa saja memalsukan sinyal positif kuat untuk item apa pun.

Q13.3 — B. Listener `pagehide` milik tracker didaftarkan lebih dulu, sehingga ia mengosongkan antrean dan mengirim beacon sebelum listener halaman produk sendiri memasukkan klik ke antrean. Klik ditambahkan setelah kesempatan flush terakhir dan hilang. Perbaikannya adalah registri hook `onPageHide`: hook dijalankan dulu, lalu antrean dikosongkan dan di-beacon.

Q13.4 — Id auto-increment ditetapkan saat insert tetapi baris menjadi terlihat saat commit. Jika transaksi yang memegang id 500 commit setelah yang memegang id 501, refresh yang sudah melihat 501 tidak akan pernah mengambil 500 dengan `id > last`. Membaca ulang 1000 id terakhir dan melakukan de-duplikasi berdasarkan `event_id` menangkap commit yang terlambat itu dengan biaya beberapa baris yang berlebih.

Q13.5 — Setiap panggilan `Recommend` menyalin referensi ke booster dan versi saat ini ke variabel lokal di awal, sehingga ia mempertahankan pasangan model/versi yang konsisten selama seluruh durasinya. `retrain` membangun booster baru secara terpisah lalu menukar kedua atribut dengan penugasan referensi biasa. Request yang sedang berjalan selesai dengan model lama, request baru memakai yang baru, dan tidak diperlukan lock pada jalur baca.

Q13.6 — Dua sesi. Event pukul 10:00 membuat sesi A; pada 10:20 sesi terakhir terlihat 20 menit lalu (di dalam jendela 30 menit), sehingga A dipakai ulang dan `last_seen_at`-nya menjadi 10:20. Pada 11:05 selisih sejak 10:20 adalah 45 menit, yang melebihi 30, sehingga sesi B baru disisipkan. Sesi pertama berakhir hari itu dengan `last_seen_at` = 10:20.

Q13.7 — Sekitar 20.2% dari anggaran dan 3.985 detik. Pecahannya 1010.6 / 5000 = 0.2021. Staleness terburuknya 2 + 1 + 0.985 = 3.985 detik (delay flush + throttle refresh + latensi request).

Q13.8 — Ia mengembalikan item aktif yang tersedia stoknya, diurutkan berdasarkan jumlah event `order` dalam 7 hari terakhir, lalu berdasarkan rating count. Item diberi label `source: "trending"` dengan skor 0 dan tanpa kontribusi karena berasal dari query popularitas non-personal dan tidak ada model yang menilainya; ini memungkinkan frontend memakai ulang rendering kartu yang sama. Respons juga membawa `fallback: true` dan `model_version: "fallback"` agar UI dapat menampilkan banner offline.

Q13.9 — B. Protobuf mengidentifikasi field di wire berdasarkan nomor, bukan nama, sehingga dua sisi dengan penomoran berbeda untuk field yang sama tidak sepakat tentang byte mana yang berarti apa dan data terbuang atau salah dibaca tanpa galat kompilasi. Itulah sebabnya nomor field tidak boleh diubah atau dipakai ulang setelah pesan dipakai; hapus field dengan mencadangkan nomornya (reserve).

Q13.10 — Harapkan latensi `p50` berubah sangat sedikit (semua run tetap di pita yang kira-kira sama dengan run k=20), dan baris kandidat-per-request identik di ketiga run, karena `candidates.generate` dan `features.build` tidak bergantung pada `k`; hanya pemotongan re-rank akhir dan jumlah baris SHAP yang berubah. Ini menunjukkan bahwa waktu didominasi oleh pekerjaan tetap per request (membangun riwayat dan fitur), bukan oleh penilaian atau ukuran respons.

Bab 14 — Dari prototipe ke skala Shopee

Q14.1 — B. Skew berarti fitur tersebut sah di kedua tempat tetapi dihitung secara berbeda (kode berbeda, filter berbeda seperti trafik bot), sehingga model belajar pada satu distribusi dan di-query dengan distribusi lain. Ukuran dataset, frekuensi retrain, dan aproksimasi ANN adalah isu yang terpisah. Feature store dengan satu definisi dan pencatatan fitur yang disajikan adalah pertahanan standar.

Q14.2 — B. Local activation unit DIN membobot setiap perilaku masa lalu berdasarkan relevansinya terhadap kandidat yang sedang dinilai, sehingga vektor minat pengguna berubah menurut kandidat. Opsi A menggambarkan resep Embedding-and-MLP yang hendak diperbaiki oleh DIN. Opsi lainnya menggambarkan komponen yang tidak berkaitan.

Q14.3 — A. Setiap task memiliki softmax gate sendiri atas pool expert bersama, sehingga task yang berkaitan dapat memilih expert yang mirip dan task yang bertentangan dapat mengkhususkan diri. Gate tidak mengubah dataset, menyamakan loss, atau menyimpan cache apa pun.

Q14.4 — Pengguna merasakan bagian paling lambat dari sebuah request, dan halaman yang menyebar ke banyak layanan selambat layanan yang paling lambat, sehingga tail latency (p99) menentukan pengalaman; median menyembunyikan request yang lambat (di sini 985.0 ms pada p99 dibanding 839.9 ms pada p50). API Rust memberi panggilan gRPC timeout 5 detik; saat galat atau timeout ia mengembalikan daftar fallback berbasis popularitas dengan `fallback: true` alih-alih menggagalkan halaman.

Q14.5 — Angka offline berasal dari log sistem sebelumnya, sehingga dapat digelembungkan oleh leakage (ranker yang bocor mendapat skor 0.5035 dibanding 0.2116 untuk yang jujur, dan akan tidak berguna secara live). Mereka juga mewarisi position bias dan mengabaikan pendapatan, diversitas, dan efek pada penjual. Alasan sah lainnya: selection bias dan feedback loop (model hanya dinilai pada item yang ditampilkan sistem lama). Hanya A/B test yang diacak yang mengukur efek sebenarnya.

Q14.6 — Proporsi gabungan $\hat{p} = (5000 + 5150)/100000 = 0.1015$. SE $= \sqrt{0.1015 \times 0.8985 \times 2/50000} \approx 0.001910$. Selisihnya $0.1030 - 0.1000 = 0.0030$, sehingga $z = 0.0030/0.001910 \approx 1.57$. Nilai p dua sisi sekitar 0.116, yang di atas 0.05, sehingga peningkatan tersebut tidak signifikan secara statistik: data tidak dapat menyingkirkan efek nol, dan tes kemungkinan kurang bertenaga (underpowered).

Q14.7 — Mentah: $2 \times 10^9 \times 128 \times 2 = 5.12 \times 10^{11}$ byte $= 512$ GB. Terkompresi: $2 \times 10^9 \times 32 = 6.4 \times 10^{10}$ byte $= 64$ GB. Product quantisation karenanya memberi pengurangan 8x di sini, dengan harga sebagian recall, yang disetel dengan jumlah probe dan re-scoring.

Q14.8 — Dasar $= 1.0 \times 0.2 + 2.0 \times 0.05 + 3.0 \times 0.01 = 0.2 + 0.10 + 0.03 = 0.33$. Dengan boost, $0.33 \times (1 + 0.1) = 0.363$ (tanpa suku iklan). Dengan boost 0 skornya 0.33. Boost mengalikan seluruh campuran, sehingga sebuah kampanye mengangkat item secara proporsional terhadap seberapa baik model sudah menilainya.

Q14.9 — B. Dengan jutaan pengguna, deviasi 0.6 poin dari 50/50 jauh di luar kebetulan dan menandakan sample ratio mismatch: bug pada pengacakan, pencatatan, penyaringan bot, atau redirect yang kehilangan salah satu arm. Hasil dari tes seperti itu tidak dapat dipercaya sampai penyebabnya ditemukan.

Q14.10 — Harapkan baris $\lambda = 0.8$ berada di antara baris 0.9 dan 0.7 pada kedua ukuran: ndcg@20 antara sekitar 0.2081 dan 0.1884, dan kategori berbeda yang lebih banyak daripada pada 0.9 tetapi lebih sedikit daripada pada 0.7 (antara sekitar 6.78 dan 11.03). Trade-off yang monoton mendukung klaim tersebut: setiap langkah menuju diversitas yang lebih besar mengorbankan relevansi, sehingga bobot diversitas adalah keputusan produk yang harus diuji secara online.

Lampiran B — Glosarium

Istilah diurutkan menurut abjad berdasarkan istilah bahasa Inggrisnya. Bab tempat istilah diperkenalkan ada dalam tanda kurung siku.

Istilah	Arti
A/B test (*uji A/B*)	Eksperimen daring yang membagi pengguna secara acak antara kelompok kontrol dan perlakuan lalu membandingkan sebuah metrik; uji signifikansi menentukan apakah perbedaannya nyata. [14]
Ablation (*ablasi*)	Melatih ulang model dengan satu bagian dihilangkan (di sini: satu kelompok fitur) untuk mengukur seberapa besar kontribusi bagian itu. [10]
Add-to-cart (*tambah ke keranjang*)	Event yang dicatat saat pengguna memasukkan item ke keranjang; sinyal minat yang lebih kuat daripada klik. [1]
ALS (Alternating Least Squares) (*kuadrat terkecil bergantian*)	Algoritma faktorisasi matriks yang menetapkan vektor item lalu menyelesaikan vektor pengguna secara eksak, kemudian sebaliknya, berulang-ulang. [6]
ANN (Approximate Nearest Neighbour) (*tetangga terdekat hampiran*)	Pencarian yang menemukan vektor yang *hampir* paling dekat jauh lebih cepat daripada memeriksa setiap vektor. [7]
AP (Average Precision) (*presisi rata-rata*)	Rata-rata precision@k yang diambil pada setiap peringkat k di mana item relevan muncul. [2]
Bayesian average (*rata-rata Bayesian*)	Rating atau rasio yang ditarik ke arah prior global sampai data cukup terkumpul, sehingga 1 klik dari 1 tayangan tidak dianggap rasio 100%. [3]
BPR (Bayesian Personalised Ranking) (*pemeringkatan personal Bayesian*)	Fungsi tujuan berpasangan (*pairwise*): untuk setiap pengguna, beri skor item yang pernah ia interaksikan lebih tinggi daripada item yang belum. [6]
Candidate (*kandidat*)	Item yang diusulkan oleh sebuah retriever untuk seorang pengguna; hanya kandidat yang sampai ke ranker. [8]
Candidate recall (*recall kandidat*)	Porsi item positif pengguna di masa depan yang ada dalam himpunan kandidat; batas atas bagi ranker mana pun. [8]
Co-visitation (*ko-kunjungan*)	Menghitung seberapa sering dua item diinteraksikan dalam sesi yang sama; dasar dari "dilihat bersamaan". [5]
Cold start (*awal dingin*)	Pengguna atau item baru dengan sedikit atau tanpa riwayat, sehingga metode personal tidak punya bahan untuk dipelajari. [12]
Collaborative filtering (*penyaringan kolaboratif*)	Merekomendasikan berdasarkan perilaku pengguna yang mirip atau konsumsi bersama antar item, tanpa melihat konten item. [5]
Content-based filtering (*penyaringan berbasis konten*)	Merekomendasikan item yang atributnya (teks, kategori, merek) mirip dengan item yang disukai pengguna. [4]
Cosine similarity (*kemiripan kosinus*)	Kosinus sudut antara dua vektor: 1 untuk arah yang sama, 0 untuk yang tidak berkaitan. [4]
Coverage (*cakupan*)	Porsi katalog yang muncul di top-K siapa pun; ukuran keberagaman sederhana. [2]
CTR (Click-through rate) (*rasio klik*)	Jumlah klik dibagi jumlah impresi. [1]
Cutoff (*titik potong waktu*)	Saat di mana fitur dihitung; hanya event sebelumnya yang boleh dipakai. [2]
DCG / NDCG	Discounted cumulative gain: relevansi dijumlahkan dengan diskon $1/\log_2(\text{rank}+1)$; NDCG membaginya dengan DCG terbaik yang mungkin, sehingga bernilai 0–1. [2]

Istilah	Arti
Decay (half-life) (*peluruhan / waktu paruh*)	Membobot event lama secara eksponensial lebih kecil; dengan waktu paruh 7 hari, aksi seminggu lalu dihitung setengah. [3]
Dwell time (*durasi tinggal*)	Detik yang dihabiskan pengguna di halaman produk; lebih lama biasanya berarti lebih berminat. [1]
Embedding (*representasi vektor*)	Vektor yang dipelajari untuk merepresentasikan pengguna atau item sehingga hal-hal yang mirip saling berdekatan. [7]
Event (*kejadian*)	Satu aksi pengguna yang dicatat pada sebuah item: impresi, klik, add-to-cart, order, hide, atau return. [1]
Explicit feedback (*umpan balik eksplisit*)	Rating atau ulasan yang sengaja diberikan pengguna. [1]
Exploration (*eksplorasi*)	Sengaja menampilkan item yang belum pasti untuk mempelajarinya, dengan biaya jangka pendek yang kecil (ε-greedy, Thompson sampling). [11]
Fallback (*cadangan*)	Apa yang disajikan sistem saat recommender tidak tersedia atau pengguna tidak dikenal; di sini, produk terlaris terbaru. [12]
Feature (*fitur*)	Angka yang menggambarkan pasangan (pengguna, item) pada suatu saat, dipakai sebagai masukan model. [9]
Feature store (*penyimpanan fitur*)	Sistem yang menghitung, menyimpan, dan menyajikan fitur secara konsisten untuk pelatihan dan penyajian. [14]
Feedback loop (*putaran umpan balik*)	Rekomendasi model membentuk data yang kelak dipakai melatihnya, sehingga dapat memperkuat biasnya sendiri. [12]
Gradient boosting	Membangun model sebagai jumlah pohon keputusan kecil, masing-masing dilatih untuk memperbaiki kesalahan pohon sebelumnya. [10]
Hard filter (*filter keras*)	Aturan yang langsung membuang kandidat (stok habis, disembunyikan pengguna). [8]
Impression (*impresi*)	Sebuah item ditampilkan di layar. [1]
Implicit feedback (*umpan balik implisit*)	Perilaku yang menandakan minat tanpa berupa rating: melihat, klik, membeli. [1]
In-batch negatives (*negatif dalam batch*)	Memakai positif lain dalam satu batch pelatihan sebagai negatif bagi setiap contoh. [7]
IPW (Inverse propensity weighting) (*pembobotan propensitas terbalik*)	Membobot ulang umpan balik yang tercatat dengan kebalikan probabilitas sebuah item dilihat, untuk mengoreksi bias eksposur dan posisi. [12]
Item-to-item (*item ke item*)	Collaborative filtering yang merekomendasikan item mirip dengan item yang diinteraksikan pengguna, memakai kemiripan item–item. [5]
IVF (Inverted file index) (*indeks berkas terbalik*)	Metode ANN yang mengelompokkan vektor dan hanya mencari di klaster terdekat dengan kueri. [7]
Label	Apa yang sebenarnya terjadi setelah cutoff (0 tidak ada, 1 klik, 2 keranjang, 3 order); target pelatihan model. [9]
LambdaMART	Ranker pohon gradient-boosted yang gradiennya dibobot oleh seberapa besar pertukaran dua item akan mengubah NDCG. [10]
Latency budget (*anggaran latensi*)	Waktu yang boleh dipakai tiap tahap sebuah permintaan agar seluruh respons memenuhi tenggatnya. [14]

Istilah	Arti
Leakage (*kebocoran data*)	Memakai informasi dari setelah cutoff saat membangun fitur; menggelembungkan metrik offline. [9]
Learning to rank (*belajar memeringkat*)	Melatih model untuk mengurutkan item bagi suatu kueri atau pengguna, bukan memprediksi nilai tiap item secara terpisah. [10]
logQ correction (*koreksi logQ*)	Mengurangi logit sebuah item dengan log probabilitas pengambilan sampelnya saat berlatih dengan in-batch negatives, agar item populer tidak dihukum berlebihan. [7]
Long tail (*ekor panjang*)	Banyak item yang masing-masing mendapat sedikit perhatian tetapi bersama-sama membentuk porsi besar katalog. [1]
Matrix factorisation (*faktorisasi matriks*)	Menghampiri matriks pengguna–item sebagai hasil kali matriks faktor pengguna dan item berdimensi rendah. [6]
MMR (Maximal Marginal Relevance) (*relevansi marginal maksimal*)	Aturan re-ranking yang menimbang relevansi terhadap kemiripan dengan item yang sudah dipilih. [11]
Multi-task learning (*pembelajaran multi-tugas*)	Satu model memprediksi beberapa hasil (klik, keranjang, order) dengan lapisan bersama. [14]
Point-in-time correctness (*kebenaran pada titik waktu*)	Menghitung setiap fitur hanya dari data yang tersedia pada cutoff-nya. [9]
Popularity bias (*bias popularitas*)	Recommender cenderung memihak item yang sudah populer, yang lalu mendapat eksposur lebih banyak dan makin populer. [3]
Position bias (*bias posisi*)	Item yang tampil lebih tinggi mendapat lebih banyak klik terlepas dari relevansinya. [12]
Precision@K / Recall@K	Porsi top K yang relevan / porsi item relevan yang berada di top K. [2]
Re-ranking (*pemeringkatan ulang*)	Tahap akhir yang menerapkan aturan tingkat halaman (batas keberagaman, filter sudah-dibeli) pada skor model. [11]
Retrieval (*pengambilan*)	Tahap pertama: memilih secara murah beberapa ratus kandidat dari seluruh katalog. [8]
Session (*sesi*)	Rentetan aktivitas pengguna; di sini jeda 30 menit memulai sesi baru. [13]
SHAP	Metode yang memecah satu prediksi menjadi kontribusi per fitur yang jumlahnya sama dengan keluaran model. [10]
Smoothing (*penghalusan*)	Lihat *Bayesian average*. [3]
Sparsity (*kejarangan*)	Porsi matriks pengguna–item yang kosong; biasanya di atas 99%. [1]
TF-IDF	Term frequency × inverse document frequency: bobot kata yang tinggi untuk kata yang sering muncul dalam satu dokumen tetapi jarang secara keseluruhan. [4]
Training/serving skew (*ketidaksesuaian pelatihan/penyajian*)	Perbedaan cara fitur dihitung saat pelatihan dan saat penyajian, yang diam-diam menurunkan kualitas model. [14]
Two-tower model (*model dua menara*)	Model retrieval dengan jaringan saraf terpisah untuk pengguna dan item yang vektor keluarannya dibandingkan dengan dot product. [7]

Lampiran C — Mereproduksi setiap hasil

Setiap angka, grafik, dan tangkapan layar dalam buku ini dihasilkan oleh kode di repositori ini. Lampiran ini mencantumkan perintah persisnya, secara berurutan. Semua path relatif terhadap root repositori.

1. Perangkat

Perangkat	Kegunaan
PostgreSQL di `localhost:5432` (user `postgres`, password `postgres`)	basis data marketplace (bab 13, tangkapan layar, latensi)
Rust toolchain (`cargo`) dan `protoc`	server API di `backend/`
Node.js dan `npm`	aplikasi web di `frontend/`, serta mermaid/KaTeX untuk membangun PDF
`uv`	semua lingkungan Python: `recommender/` dan `book/`

2. Skrip pendamping offline (bab 1–12)

Skrip-skrip ini hanya memerlukan Python. Semuanya memakai marketplace simulasi yang sama (seed 7), yang di-*cache* di `book/build/sim.pkl` pada pemakaian pertama.

```
cd book
uv sync                                  # installs recsys from ../recommender in editable mode
uv run python tools/run_all.py --skip-live
```

`run_all.py` menjalankan `code/ch01_explore.py` … `code/ch12_coldstart_bias.py` secara berurutan. Setiap skrip mencetak teks yang dikutip di babnya, yang disimpan ke `results/<script>.txt`, dan menulis angkanya ke `results/<script>.json`. Eksekusi berhenti pada skrip pertama yang gagal. Untuk menjalankan ulang skrip satu bab saja:

```
uv run python tools/run_all.py --only ch06_mf
```

Skrip-skrip ini deterministik: mesin yang sama menghasilkan berkas yang identik. Antar mesin, waktu (milidetik per kueri di bab 7) berbeda, dan LightGBM serta PyTorch bisa berbeda pada angka desimal terakhir.

3. Sistem langsung (bab 13)

Penyiapan satu kali:

```
PGPASSWORD=postgres createdb -h localhost -U postgres marketplace
cp backend/.env.example backend/.env
(cd backend && cargo run -- migrate)
(cd recommender && uv sync && uv run recsys seed-db --yes)
(cd frontend && npm i)
```

Lalu jalankan ketiga layanan, masing-masing di terminalnya sendiri:

```
cd recommender && uv run recsys serve
cd backend && cargo run
cd frontend && npm run dev
```

Dengan recommender berjalan, skrip latensi memanggilnya langsung lewat gRPC:

```
cd book && uv run python tools/run_all.py --only ch13_latency
```

Karena `seed-db` menggeser riwayat simulasi agar berakhir tepat pada saat Anda menjalankannya, metrik dan latensi model langsung akan sedikit berbeda dari yang dicetak dalam buku ini.

4. Grafik

```
cd book && uv run python tools/make_figures.py
```

Perintah ini membaca `results/*.json` dan menulis setiap PNG di `assets/charts/`. Grafik tiap bab digambar oleh `tools/figures/chNN.py`.

5. Tangkapan layar

Dengan ketiga layanan berjalan dan aplikasi di `http://localhost:5173`:

```
cd book && uv run python tools/screenshots.py
```

Skrip ini login lewat UI sungguhan sebagai `user43@demo.test`, membuat pesanan, lalu login sebagai penjual toko tersebut dan sebagai `admin@demo.test`, dan menulis PNG ke `assets/screenshots/`. Banner *fallback* mengharuskan recommender dalam keadaan **dihentikan**:

```
# stop `recsys serve`, then:
cd book && uv run python tools/screenshots.py --only fallback
```

6. PDF

```
cd book
uv run playwright install chromium
npm i                                    # mermaid + KaTeX, rendered inside the page
uv run python tools/check.py --lang en
uv run python tools/check.py --lang id
uv run python tools/build.py --lang all
```

`check.py` memverifikasi bahwa setiap *include*, berkas keluaran, dan gambar ada, serta bahwa setiap bab memiliki kotak fakta, diagram, gambar, kuis, dan panjang minimum. `build.py` merender Markdown menjadi HTML, membukanya di Chromium *headless*, menunggu diagram dan rumus selesai dirender, lalu mencetak `dist/recsys-book-en.pdf` dan `dist/recsys-book-id.pdf` beserta jumlah halamannya.

Daftar Pustaka

Karya yang dikutip dalam teks dan dalam kotak "Tahukah kamu?", diurutkan menurut abjad berdasarkan penulis pertama.

- Abdollahpouri, H., Mansoury, M., Burke, R., & Mobasher, B. (2019). The Unfairness of Popularity Bias in Recommendation. Workshop on Recommendation in Multi-stakeholder Environments (RMSE), RecSys 2019.
- Amatriain, X., & Basilico, J. (2012). Netflix Recommendations: Beyond the 5 stars (Part 1). Netflix Technology Blog.
- Anderson, C. (2004). The Long Tail. Wired.
- Burges, C. J. C. (2010). From RankNet to LambdaRank to LambdaMART: An Overview. Microsoft Research Technical Report MSR-TR-2010-82.
- Burges, C. J. C., Svore, K. M., Bennett, P. N., Pastusiak, A., & Wu, Q. (2011). Learning to Rank Using an Ensemble of Lambda-Gradient Models. Proceedings of the Learning to Rank Challenge (PMLR 14), 25–35.
- Carbonell, J. and Goldstein, J. (1998). The use of MMR, diversity-based reranking for reordering documents and producing summaries. Proceedings of the 21st Annual International ACM SIGIR Conference (SIGIR).
- Chaney, A. J. B., Stewart, B. M., & Engelhardt, B. E. (2018). How algorithmic confounding in recommendation systems increases homogeneity and decreases utility. Proceedings of RecSys 2018.
- Chapelle, O. and Li, L. (2011). An empirical evaluation of Thompson sampling. Advances in Neural Information Processing Systems (NeurIPS/NIPS).
- Cheng, H.-T., Koc, L., Harmsen, J., Shaked, T., Chandra, T., Aradhye, H., et al. (2016). Wide & Deep Learning for Recommender Systems. Proceedings of the 1st Workshop on Deep Learning for Recommender Systems (DLRS).
- Covington, P., Adams, J., & Sargin, E. (2016). Deep Neural Networks for YouTube Recommendations. Proceedings of the 10th ACM Conference on Recommender Systems (RecSys '16).
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 29(5), 1189–1232.
- Funk, S. (2006). Netflix Update: Try This at Home. sifter.org blog, 11 December 2006.
- Gomez-Uribe, C. A., & Hunt, N. (2015). The Netflix recommender system: Algorithms, business value, and innovation. ACM Transactions on Management Information Systems.
- Google. (2026). Language Guide (proto 3). Protocol Buffers Documentation.
- Guo, R., Sun, P., Lindgren, E., Geng, Q., Simcha, D., Chern, F., & Kumar, S. (2020). Accelerating large-scale inference with anisotropic vector quantization. ICML (PMLR 119).
- He, X., Pan, J., Jin, O., Xu, T., Liu, B., Xu, T., Shi, Y., Atallah, A., Herbrich, R., Bowers, S., & Candela, J. Q. (2014). Practical Lessons from Predicting Clicks on Ads at Facebook. Proceedings of the Eighth International Workshop on Data Mining for Online Advertising (ADKDD, KDD workshop).
- Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative Filtering for Implicit Feedback Datasets. Proceedings of the IEEE International Conference on Data Mining (ICDM).
- IMDb. (2026). Ratings FAQ: How do you calculate the rank of movies and TV shows on the Top 250 charts? IMDb Help Center.
- Joachims, T., Granka, L., Pan, B., Hembrooke, H., & Gay, G. (2005). Accurately interpreting clickthrough data as implicit feedback. Proceedings of SIGIR 2005.
- Joachims, T., Swaminathan, A., & Schnabel, T. (2017). Unbiased learning-to-rank with biased feedback. Proceedings of WSDM 2017.
- Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. IEEE Transactions on Big Data.
- Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. ACM Transactions on Information Systems, 20(4).

- Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in Data Mining: Formulation, Detection, and Avoidance. ACM Transactions on Knowledge Discovery from Data, 6(4).
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. Advances in Neural Information Processing Systems 30 (NeurIPS).
- Krichene, W., & Rendle, S. (2020). On Sampled Metrics for Item Recommendation. Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD).
- Linden, G., Smith, B., & York, J. (2003). Amazon.com Recommendations: Item-to-Item Collaborative Filtering. IEEE Internet Computing, 7(1), 76–80.
- Liu, Z., Zou, L., Zou, X., Wang, C., Zhang, B., Tang, D., Zhu, B., Zhu, Y., Wu, P., Wang, K., & Cheng, Y. (2022). Monolith: Real Time Recommendation System With Collisionless Embedding Table. 5th Workshop on Online Recommender Systems and User Modeling (ORSUM), RecSys 2022.
- Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. Advances in Neural Information Processing Systems 30 (NeurIPS).
- Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S.-I. (2020). From local explanations to global understanding with explainable AI for trees. Nature Machine Intelligence, 2(1), 56–67.
- Ma, J., Zhao, Z., Yi, X., Chen, J., Hong, L., & Chi, E. H. (2018). Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts. KDD 2018.
- Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE TPAMI.
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv:1301.3781.
- Mozilla Contributors. (2026). Navigator: sendBeacon() method. MDN Web Docs.
- Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). BPR: Bayesian Personalized Ranking from Implicit Feedback. Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI).
- Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., & Riedl, J. (1994). GroupLens: An open architecture for collaborative filtering of netnews. Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW).
- Sarno, D. (2009). Netflix awards $1-million prize to recommendation wizards, announces 2nd contest. Los Angeles Times.
- Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-Based Collaborative Filtering Recommendation Algorithms. Proceedings of the 10th International World Wide Web Conference (WWW).
- Schein, A. I., Popescul, A., Ungar, L. H., & Pennock, D. M. (2002). Methods and metrics for cold-start recommendations. Proceedings of SIGIR 2002.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden Technical Debt in Machine Learning Systems. Advances in Neural Information Processing Systems (NeurIPS) 28.
- Singh, A. and Joachims, T. (2018). Fairness of exposure in rankings. Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD).
- Spärck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. Journal of Documentation, 28(1), 11–21.
- Thompson, W. R. (1933). On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. Biometrika, 25(3/4).
- Westergren, T. (2011). Tim Westergren, Founder of Pandora (radio interview). NPR.
- Yi, X., Yang, J., Hong, L., Cheng, D. Z., Heldt, L., Kumthekar, A., Zhao, Z., Wei, L., & Chi, E. (2019). Sampling-bias-corrected neural modeling for large corpus item recommendations. RecSys.
- Ying, R., He, R., Chen, K., Eksombatchai, P., Hamilton, W. L., & Leskovec, J. (2018). Graph Convolutional Neural Networks for Web-Scale Recommender Systems. KDD 2018.
- Zhou, G., Song, C., Zhu, X., Fan, Y., Zhu, H., Ma, X., Yan, Y., Jin, J., Li, H., & Gai, K. (2018). Deep Interest Network for Click-Through Rate Prediction. KDD 2018.

- Ziegler, C.-N., McNee, S. M., Konstan, J. A. and Lausen, G. (2005). Improving recommendation lists through topic diversification. Proceedings of the 14th International Conference on World Wide Web (WWW).